DEVOPS ENGINEERING MASTERY

DEVSECOPS, SRE & PRODUCTION RELIABILITY ENGINEERING

Secure Software Delivery, Application Security, Infrastructure Security, Site Reliability Engineering, Incident Management, Disaster Recovery, FinOps & Enterprise Production Operations

VOLUME 15 • FIRST EDITION

```
nahid@technova:~$ trivy image technova/api:1.4.2
✓ 0 CRITICAL, 0 HIGH — image clean
nahid@technova:~$ cosign verify technova/api:1.4.2
✓ Signature verified — Sigstore keyless
nahid@technova:~$ kubectl get slo technova-api -n prod
```

DevSecOps • SRE • OWASP • Kubernetes Security • SLI/SLO

First Edition • Published 9 November 2026 • Dhaka, Bangladesh

24 Chapters • 24 Guided Labs • 3 Major Projects • 120 MCQs

DevSecOps, SRE & Production Reliability Engineering

DEVOPS ENGINEERING MASTERY — Volume 15

Secure Software Delivery, Application Security, Infrastructure Security, Site Reliability Engineering, Incident Management, Disaster Recovery, FinOps ও Enterprise Production Operations

First Edition — Published 9 November 2026.

Hands-on environment: Ubuntu Server 24.04 LTS, a MERN (MongoDB, Express, React, Node.js) example application called TechNova, Docker, Kubernetes (kind/Minikube), Terraform, Prometheus and Grafana. All domains, IP addresses and credentials in examples are fictional (technova.example, 203.0.113.x, CHANGE-ME-...). Sample terminal outputs are illustrative. Security exercises are taught only for authorized, isolated lab environments — deliberately vulnerable training applications such as OWASP Juice Shop, local virtual machines and disposable clusters. Never scan or test a system you do not own or have explicit authorization to test. Official documentation (OWASP, NIST, CNCF, Kubernetes, HashiCorp, Sigstore, SLSA, cloud providers, FinOps Foundation) is the authority; no pricing or service limits are stated in this book.

How to use this book. Read the parts in order — each part builds on the previous one. Keep a lab machine beside you and do every hands-on exercise and every lab in the companion Lab Workbook before moving on. Do not copy commands blindly; read the explanation under each configuration field first. Destructive operations are always flagged with a WARNING box and a safe alternative. Never practise on production infrastructure: use disposable environments, never paste a real secret into an example, never commit a secret to Git, and never treat a scanner's clean report as proof of security — scanners are not a substitute for threat modeling, code review, secure architecture and human judgment.

A note on honesty. Reading a book alone does not make anyone an expert. This book teaches the foundations and the practical judgment to progress toward professional engineering — but the judgment itself comes from doing the labs, breaking a test environment, fixing it, and repeating. There is no shortcut for that practice.

Conventions. Bangla is the explanatory language; technical terms, command names, APIs and configuration keys stay in English. `monospace` marks commands, file paths and code. **CONCEPT** explains an idea, **REAL WORLD** shows an industry scenario, **WARNING** marks risk, **TROUBLESHOOTING** helps you fix a failure, and **LAB** marks hands-on work.

© 2026 DevOps Engineering Mastery series. All rights reserved. This book is for personal learning. Security and reliability exercises should first be practised in a disposable, authorized environment, never directly on production systems or on systems you do not own. The author and publisher accept no liability for data loss, service disruption, security incidents or cloud charges caused by running commands without a backup, a plan review or an approval gate.

Who this book is for

A learner who knows JavaScript, React, Next.js, Node.js, Express and MongoDB, understands Git and Linux, and has studied Linux server administration, networking, CI/CD, Docker, Kubernetes, Terraform, Ansible, observability and the major cloud platforms — and now wants to build secure, reliable production systems: DevSecOps pipelines, SRE practices, incident response, disaster recovery and cost governance.

Learning outcomes

By the end of this book you will be able to: explain DevSecOps principles and run a secure SDLC with threat modeling; harden Linux hosts, identities and secrets; write secure MERN code and assess it against the OWASP Top 10; run SAST, DAST and dependency scanning and triage the results; secure containers and Kubernetes workloads; build SBOMs and sign artifacts; enforce Policy as Code; manage vulnerabilities with CVE/CVSS reasoning; design SLIs, SLOs and error budgets; run monitoring, alerting and on-call well; manage incidents with blameless postmortems; plan capacity and disaster recovery; run controlled chaos experiments safely; and operate an enterprise DevSecOps and SRE platform with FinOps governance.

সূচিপত্র — Contents

PART I — Security Engineering Foundations

Chapter 01 — DevSecOps Fundamentals 7

Chapter 02 — Secure SDLC and Threat Modeling 14

Chapter 03 — Infrastructure Security 21

Chapter 04 — Identity and Access Security 27

Chapter 05 — Secrets, Encryption and Key Management 34

PART II — Application & Software Supply Chain Security

Chapter 06 — Secure Coding and Application Security 42

Chapter 07 — OWASP Top 10 and Web Security 50

Chapter 08 — SAST, DAST and Dependency Security 57

Chapter 09 — Container and Kubernetes Security 64

Chapter 10 — Software Supply Chain Security 72

PART III — DevSecOps Automation & Governance

Chapter 11 — Security in CI/CD 80

Chapter 12 — Policy as Code 90

Chapter 13 — Vulnerability Management 99

Chapter 14 — Cloud Security Posture 107

Chapter 15 — DevSecOps Governance and Compliance 115

PART IV — Site Reliability Engineering

Chapter 16 — SRE Fundamentals 124

Chapter 17 — SLIs, SLOs, SLAs and Error Budgets 132

Chapter 18 — Monitoring, Alerting and On-call 139

Chapter 19 — Incident Management and RCA 146

Chapter 20 — Performance and Capacity Engineering 153

PART V — Enterprise Reliability & Operations

Chapter 21 — Disaster Recovery and Business Continuity 161

Chapter 22 — Chaos Engineering 167

Chapter 23 — FinOps and Operational Governance 173

Chapter 24 — Enterprise DevSecOps and SRE Architecture 179

Companion Volumes

Lab Workbook — 24 Guided Labs • 15 Assignments • 3 Major Projects • Capstone

Assessment and Answer Guide — 120 MCQs • Short Answers • Scenarios • Case Studies • Final Exam

Quick Reference Handbook — Glossary (150+ Terms) • OWASP & Security References • Checklists • Decision Trees

Guided labs (24), major projects (3) and the capstone are in the separate *DevSecOps/SRE Lab Workbook*. MCQs, assignments, examinations and the answer key are in the separate *Assessment and Answer Guide*. Glossary, command and workflow references, checklists and decision trees are in the separate *Production Reliability Quick Reference Handbook*.

PART I • DEVOPS ENGINEERING MASTERY

Security Engineering Foundations

DevSecOps-এর নীতি, Secure SDLC ও Threat Modeling, Infrastructure Security, Identity ও Access Security, Secrets, Encryption ও Key Management।

অধ্যায় ১ • PART I — Security Engineering Foundations

Chapter 1 — DevSecOps Fundamentals

১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি DevSecOps-এর মূল ধারণা নিজের ভাষায় ব্যাখ্যা করতে পারবেন, DevOps ও DevSecOps-এর পার্থক্য স্পষ্টভাবে বলতে পারবেন এবং Shift-Left ও Shift-Right নিরাপত্তার অর্থ ও সীমা বুঝতে পারবেন। আপনি নিরাপত্তাকে শেষ মুহূর্তের পরীক্ষা হিসেবে নয়, বরং নকশা, কোড, বিল্ড, ডিপ্লয় ও পরিচালনার প্রতিটি ধাপে বস্তুতি একটি প্রকৌশল দায়িত্ব হিসেবে দেখতে শিখবেন। TechNova-র মতো একটি MERN অ্যাপ্লিকেশনে কোথায় কোন নিরাপত্তা নিয়ন্ত্রণ বসে, কে তার মালিক, এবং সেই নিয়ন্ত্রণ ব্যর্থ হলে কীভাবে তা ধরা পড়ে — এই পুরো চিত্রটি আপনার কাছে পরিষ্কার হবে। সবশেষে, আপনি একটি ছোট দলের জন্য বাস্তবসম্মত, অতিরিক্ত জটিল নয় এমন একটি DevSecOps কর্মপ্রবাহের খসড়া তৈরি করতে পারবেন।

- DevSecOps, Shift-Left, Shift-Right ও Security Automation-এর সংজ্ঞা ও উদ্দেশ্য ব্যাখ্যা করা।
- Security Scanning ও Security Engineering-এর পার্থক্য চিহ্নিত করা।
- DevOps Security Lifecycle-এর প্রতিটি ধাপে নিরাপত্তা নিয়ন্ত্রণ স্থাপন করা।
- নিরাপত্তার মালিকানা, দায়িত্ব ভাগাভাগি ও প্রোডাকশন নিরাপত্তা সংস্কৃতির নীতি প্রয়োগ করা।
- একটি ব্যর্থতা থেকে শিক্ষা নিয়ে প্রতিরোধমূলক ব্যবস্থা নকশা করা।

১.২ পূর্বশর্ত ঝালাই

এই অধ্যায়ে প্রবেশের আগে DevOps-এর মৌলিক ধারণাগুলো ঝালিয়ে নেওয়া দরকার। DevOps মানে উন্নয়ন (Development) ও পরিচালনা (Operations) দলের মধ্যে দেয়াল ভেঙে একই লক্ষ্যে কাজ করা: দ্রুত, নির্ভরযোগ্যভাবে ও বারবার সফটওয়্যার পৌঁছে দেওয়া। Continuous Integration (CI) মানে কোড বারবার একত্র করে স্বয়ংক্রিয়ভাবে পরীক্ষা করা, আর Continuous Delivery/Deployment (CD) মানে সেই পরীক্ষিত কোড নিরাপদে প্রোডাকশনে পৌঁছে দেওয়া। Git-এ সংস্করণ নিয়ন্ত্রণ, Docker-এ কনটেইনার, এবং Linux সার্ভারে অ্যাপ্লিকেশন চালানোর প্রাথমিক ধারণা থাকলে এই অধ্যায় সহজ হবে। নিরাপত্তার দিক থেকে Confidentiality (গোপনীয়তা), Integrity (অখণ্ডতা) ও Availability (উপলভ্যতা) — এই তিনটি নীতি, যাকে CIA Triad বলা হয়, মনে রাখুন। কোনো তথ্য শুধু অনুমোদিত ব্যক্তি দেখতে পারবে, তা পরিবর্তন হয়নি বলে নিশ্চিত হওয়া যাবে, এবং প্রয়োজনের সময় তা পাওয়া যাবে — এই তিনটি একসাথে না থাকলে নিরাপত্তা অসম্পূর্ণ থাকে।

আরেকটি গুরুত্বপূর্ণ ধারণা হলো ঝুঁকি (Risk)। ঝুঁকি মানে কোনো দুর্বলতাকে কাজে লাগিয়ে ক্ষতি হওয়ার সম্ভাবনা ও তার প্রভাবের গুণফল। সব ঝুঁকি সমান নয়; কিছু ঝুঁকি গ্রহণ করা যায়, কিছু কমাতে হয়, কিছু এড়াতে হয়, আবার কিছু অন্যের কাছে হস্তান্তর করা যায়। DevSecOps শেখার সময় এই বিচারবোধই সবচেয়ে মূল্যবান — কোনো টুলের নাম মুখস্থ করা নয়, বরং কোন ঝুঁকি আগে কমাবেন, তা ঠিক করতে পারা।

১.৩ প্রথম নীতি থেকে DevSecOps: নিরাপত্তা কেন শেষে যোগ করা যায় না

DevSecOps-এর প্রথম নীতি হলো: নিরাপত্তা কোনো বৈশিষ্ট্য (feature) নয়, এটি সফটওয়্যারের একটি গুণ (quality attribute)। একটি বাড়ির উদাহরণ নিন। বাড়ি তৈরি শেষ হওয়ার পর দরজায় তালা লাগানো যায়, কিন্তু ভিত্তি দুর্বল হলে পরে তালা লাগিয়ে লাভ হয় না। সফটওয়্যারেও একই কথা সত্য। নকশার সময় যদি ধরে নেওয়া হয় যে সব ব্যবহারকারী সৎ, সব

ইনপুট নিরাপদ, এবং নেটওয়ার্কের ভেতরে সবাই বিশ্বস্ত — তাহলে পরে যত স্ক্যানারই চালান, সেই ভুল ধারণার ওপর দাঁড়িয়ে থাকা ব্যবস্থা নিরাপদ হবে না। এই কারণে DevSecOps বলে: নিরাপত্তা শুরু থেকেই, প্রতিটি সিদ্ধান্তের ভেতরে থাকতে হবে।

সমস্যাটি কেন ঘটে, তা বোঝা জরুরি। ঐতিহ্যগতভাবে নিরাপত্তা দল ছিল আলাদা, তারা আসত শেষে — রিলিজের ঠিক আগে পরীক্ষা করতে। তখন কোনো বড় দুর্বলতা ধরা পড়লে দুটি খারাপ বিকল্প থাকত: রিলিজ পিছিয়ে দেওয়া, অথবা ঝুঁকি নিয়ে রিলিজ দেওয়া। ব্যবসার চাপে দ্বিতীয়টিই বেশি হতো। আবার ডেভেলপাররা নিরাপত্তাকে নিজের কাজ মনে করতেন না; তাঁরা ভাবতেন এটি নিরাপত্তা দলের দায়িত্ব। ফলে দুর্বলতা বারবার একই জায়গায় তৈরি হতো। DevSecOps এই বিভাজন ভাঙে। এর মূল কথা — Everyone is responsible for security, but responsibility without capability is unfair — অর্থাৎ সবার দায়িত্ব, কিন্তু দায়িত্বের সাথে সাথে দক্ষতা, টুল ও সময়ও দিতে হবে।

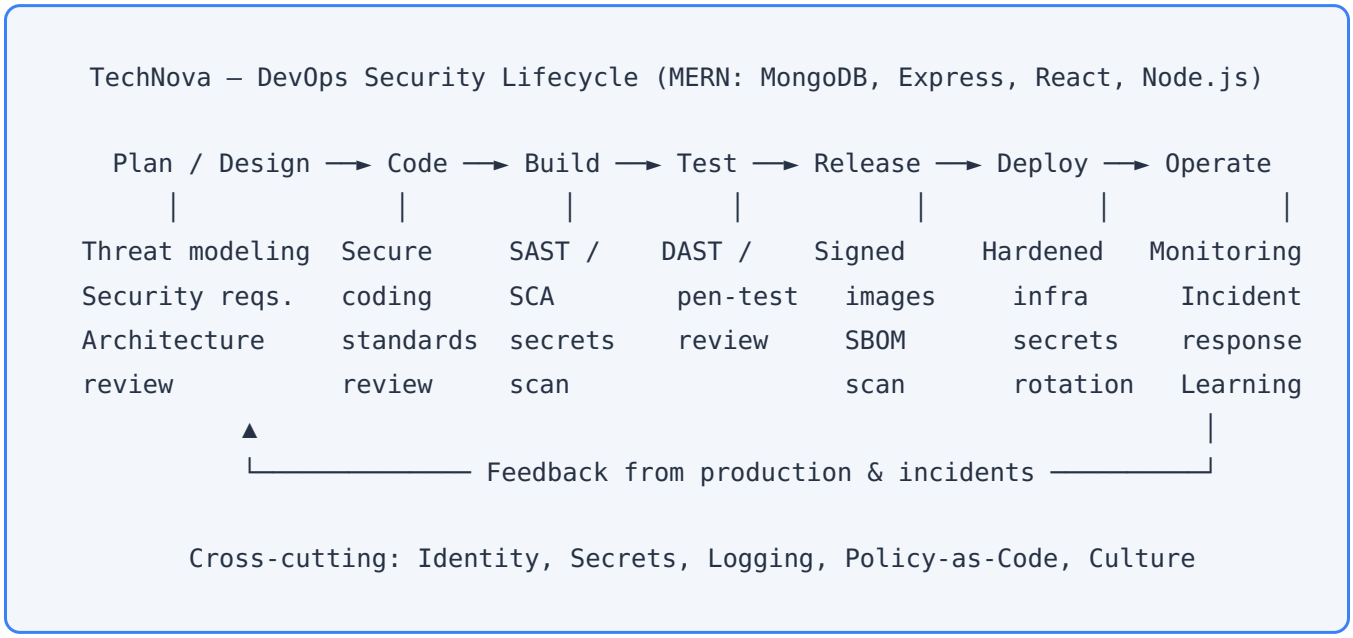
> **CONCEPT:** DevSecOps মানে DevOps-এর সাথে নিরাপত্তা যোগ করা নয়; DevSecOps মানে এমন DevOps যেখানে নিরাপত্তা শুরু থেকেই নকশা, কোড, পরীক্ষা ও পরিচালনার অবিচ্ছেদ্য অংশ। স্ক্যানার চালানো DevSecOps নয় — স্ক্যানারের ফলাফল বুঝে নকশা বদলানোই DevSecOps।

Shift-Left Security মানে নিরাপত্তা পরীক্ষাকে জীবনচক্রের বাম দিকে, অর্থাৎ আগের ধাপে সরিয়ে আনা — নকশা, কোড লেখা ও কমিটের সময়ই ধরা। এর সুবিধা হলো, তখন ভুল ঠিক করা সস্তা ও সহজ। একটি উদাহরণ: কোড লেখার সময় যদি এডিটরই বলে দেয় যে ব্যবহারকারীর ইনপুট যাচাই ছাড়া ডাটাবেস কোয়েরিতে ব্যবহার করা হচ্ছে, তাহলে ডেভেলপার তখনই ঠিক করতে পারেন। প্রোডাকশনে গিয়ে ধরা পড়লে একই ভুল ঠিক করতে সময়, অর্থ ও সুনাম — তিনটিই বেশি খরচ হয়। Shift-Right Security এর বিপরীত নয়, এর পরিপূরক। Shift-Right মানে প্রোডাকশনে চলমান ব্যবস্থাকে পর্যবেক্ষণ করা, আক্রমণের চেষ্টা ধরা, এবং বাস্তব ব্যবহার থেকে শেখা। Shift-Left বলে ভুল আগেই ধরো, Shift-Right বলে প্রোডাকশনে যা ঘটছে তা থেকে শিখে আগের ধাপকে আরও শক্ত করো। দুটোর একটিও ছাড়া চলে না।

Security Automation মানে মানুষের বারবার করা নিরাপত্তা কাজগুলোকে স্বয়ংক্রিয় করা — তবে অন্ধভাবে নয়। ভালো অটোমেশন হলো সেইটি যা সঠিক জায়গায়, সঠিক সিদ্ধান্তে থামে। যেমন, নির্ভরতা (dependency) তালিকায় পরিচিত দুর্বলতা থাকলে CI পাইপলাইন স্বয়ংক্রিয়ভাবে বিল্ড আটকে দেবে — এটি ভালো অটোমেশন। কিন্তু প্রতিটি ছোট সতর্কতায় বিল্ড আটকে দিলে ডেভেলপাররা সতর্কতাকে উপেক্ষা করতে শিখবেন, যাকে বলে Alert Fatigue। তাই অটোমেশনের নীতি হলো: গুরুতর ও নিশ্চিত ঝুঁকিতে কঠোর হও, অনিশ্চিত বিষয়ে মানুষের বিচারের জন্য পথ খোলা রাখো।

১.৪ আর্কিটেকচার: DevOps Security Lifecycle

নিচের চিত্রে TechNova-র MERN অ্যাপ্লিকেশনের জীবনচক্রে নিরাপত্তা কোথায় বসে, তা দেখানো হয়েছে। লক্ষ করুন, নিরাপত্তা কোনো একক ধাপ নয়; এটি একটি বলয়, যা প্রতিটি ধাপকে ঘিরে আছে এবং প্রোডাকশন থেকে পাওয়া শিক্ষা আবার নকশায় ফিরে আসে।



ধাপ	নিরাপত্তা নিয়ন্ত্রণ	মালিক	ব্যর্থ হলে কী হয়
Plan / Design	Threat modeling, security requirements	Architect + Product	ভুল অনুমানের ওপর ব্যবস্থা দাঁড়ায়
Code	Secure coding, code review, secrets scan	Developer	Injection, ভাঙা auth কোডে ঢোকে
Build	SAST, SCA, image scan, SBOM	Platform / DevOps	পরিচিত দুর্বলতা প্রোডাকশনে যায়
Test / Release	DAST, penetration test, signing	QA + Security	চলমান অবস্থার দুর্বলতা ধরা পড়ে না
Deploy / Operate	Hardening, monitoring, incident response	SRE / Operations	আক্রমণ ধরা পড়ে দেরিতে

এই টেবিলের সবচেয়ে গুরুত্বপূর্ণ শিক্ষা হলো মালিকানার স্পষ্টতা। প্রতিটি নিয়ন্ত্রণের একজন নামযুক্ত মালিক থাকতে হবে। মালিকহীন নিয়ন্ত্রণ মানে কার্যত কোনো নিয়ন্ত্রণ নেই। ছোট দলে একজন ব্যক্তি একাধিক ধাপের মালিক হতে পারেন, কিন্তু দায়িত্বটি লিখিত ও দৃশ্যমান হতে হবে।

> **WARNING:** Security Scanning ও Security Engineering এক নয়। স্ক্যানার শুধু পরিচিত প্যাটার্ন খুঁজে পায়; এটি আপনার ব্যবসায়িক যুক্তির ভুল, ভুল ট্রাস্ট অনুমান বা নকশার দুর্বলতা ধরতে পারে না। স্ক্যান সবুজ মানেই নিরাপদ নয় — এর মানে শুধু স্ক্যানার যা খুঁজছিল, তা পায়নি।

১.৫ অভ্যন্তরীণ কার্যপ্রণালী: নিয়ন্ত্রণ কীভাবে কাজ করে

একটি নিরাপত্তা নিয়ন্ত্রণ ভেতরে ভেতরে তিনটি কাজ করে: প্রতিরোধ (Prevent), শনাক্তকরণ (Detect) ও প্রতিক্রিয়া (Respond)। প্রতিরোধ মানে আক্রমণ যেন সফল না হয় — যেমন ইনপুট যাচাই, অনুমতি পরীক্ষা, এনক্রিপশন। শনাক্তকরণ মানে কিছু ঘটলে যেন তা দেখা যায় — যেমন লগ, অ্যালার্ট, অস্বাভাবিক আচরণ ধরা। প্রতিক্রিয়া মানে ধরা পড়ার পর কী করবেন — অ্যাকাউন্ট বন্ধ করা, কী ঘোরানো, সার্ভিস আলাদা করা। একটি মাত্র স্তরের ওপর নির্ভর করা বিপজ্জনক; ভালো

নকশায় একাধিক স্তর একসাথে কাজ করে, যাকে Defense in Depth বলা হয়। একটি স্তর ভেঙে গেলেও পরের স্তর আক্রমণকারীকে থামায়।

নিয়ন্ত্রণ কীভাবে ব্যর্থ হয়, তাও বোঝা দরকার। সবচেয়ে সাধারণ ব্যর্থতা হলো নিয়ন্ত্রণটি থাকলেও চালু না থাকা — যেমন ফায়ারওয়াল নিয়ম লেখা আছে কিন্তু প্রয়োগ হয়নি, অথবা লগ সংগ্রহ হচ্ছে কিন্তু কেউ দেখছে না। দ্বিতীয় ব্যর্থতা হলো ভুল কনফিগারেশন: নিয়মটি এমনভাবে লেখা যে বৈধ কাজ আটকে যায়, তখন মানুষ নিয়মটি বন্ধ করে দেয়। তৃতীয় ব্যর্থতা হলো নিয়ন্ত্রণটি পুরোনো হয়ে যাওয়া: নতুন আক্রমণ পদ্ধতি, নতুন নির্ভরতা বা নতুন ব্যবসায়িক বৈশিষ্ট্যের সাথে তা আর মেলে না। তাই প্রতিটি নিয়ন্ত্রণের জন্য একটি পরীক্ষা থাকতে হবে — নিয়ন্ত্রণটি আসলেই কাজ করছে কি না, তা নিয়মিত যাচাই করা। একে বলে Control Testing: শুধু নিয়ন্ত্রণ আছে কি না নয়, এটি কাজ করে কি না তা প্রমাণ করা।

DevSecOps-এ Policy as Code একটি শক্তিশালী ধারণা। নীতি মুখে বলা বা নথিতে লেখা থাকলে তা ভুলে যাওয়া যায়; কোড হিসেবে লিখলে তা স্বয়ংক্রিয়ভাবে পরীক্ষা করা যায়। যেমন, নীতি হতে পারে: প্রোডাকশন ডাটাবেসের পোর্ট ইন্টারনেটে খোলা যাবে না; সব কনটেইনার ইমেজে পরিচিত গুরুতর দুর্বলতা থাকলে ডিপ্লয় হবে না; সব সিক্রেট অবশ্যই Vault থেকে আসবে, কোডে থাকবে না। এই নীতিগুলো CI/CD পাইপলাইনে পরীক্ষা হিসেবে চললে মানুষের ভুলে যাওয়ার সুযোগ কমে। তবে সতর্কতা: নীতি যত বেশি হবে, তত বেশি ব্যতিক্রম (exception) তৈরি হবে। প্রতিটি ব্যতিক্রমের মেয়াদ, মালিক ও ঝুঁকি গ্রহণের অনুমোদন লিখিত থাকতে হবে, নইলে ব্যতিক্রমই নিয়ম হয়ে যায়।

১.৬ বাস্তব কমান্ড ও কনফিগারেশন: TechNova ল্যাবে শুরু

এই বিভাগের সব কাজ শুধু অনুমোদিত, বিচ্ছিন্ন ল্যাব পরিবেশে করবেন — technova.example ডোমেইন ও 203.0.113.x আইপি কাল্পনিক, বাস্তব কোনো ব্যবস্থায় পরীক্ষা করবেন না। প্রথমে TechNova রিপোজিটরিতে একটি প্রাথমিক নিরাপত্তা পরীক্ষা যোগ করার ধারণা দেখুন। নিচের GitHub Actions-এর খসড়াটি ধারণা বোঝার জন্য; বাস্তব ব্যবহারে আপনার দলের অনুমোদিত টুল ও সংস্করণ ব্যবহার করুন।

```
# .github/workflows/security.yml — ধারণাগত খসড়া (ল্যাব পরিবেশে)
name: security-checks
on: [push, pull_request]
jobs:
  scan:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - name: Install dependencies
        run: npm ci
      - name: Audit dependencies
        run: npm audit --audit-level=high
      - name: Secret scan (example)
        run: |
          echo "Run approved secret scanner here"
          git log --oneline -5
```

ল্যাব সার্ভারে (Ubuntu Server 24.04 LTS) মৌলিক অবস্থা যাচাই করার জন্য নিচের কমান্ডগুলো ব্যবহার করা হয়। এগুলো আক্রমণের টুল নয়; এগুলো আপনার নিজের সার্ভার সুস্থ আছে কি না তা দেখার উপায়।

```
# Ubuntu 24.04 LTS — TechNova lab server 203.0.113.10
sudo apt update && sudo apt upgrade -y
sudo ufw status verbose
sudo ss -tlnp
sudo systemctl status mongod node-app --no-pager
```

```
# কোন পোর্ট ইন্টারনেটে খোলা, তা নিজের সার্ভারে যাচাই করুন
# অপ্রয়োজনীয় সার্ভিস বন্ধ রাখাই প্রথম প্রতিরক্ষা।
```

Node.js অ্যাপ্লিকেশনে পরিবেশ ভেরিয়েবল থেকে সিক্রেট পড়ার সঠিক ধরনটি দেখুন। সিক্রেট কখনো কোডে, কখনো Git-এ যাবে না — এটি পরের অধ্যায়গুলোতে বিস্তারিত আসবে, কিন্তু অভ্যাসটি এখান থেকেই শুরু।

```
// server/config.js — সিক্রেট কোডে নয়, পরিবেশ থেকে
const config = {
  mongoUri: process.env.MONGO_URI || 'CHANGE-ME-mongo-uri',
  jwtSecret: process.env.JWT_SECRET || 'CHANGE-ME-jwt-secret',
  port: process.env.PORT || 3000,
};
if (config.jwtSecret.startsWith('CHANGE-ME')) {
  console.warn('WARNING: placeholder secret in use — do not deploy to production');
}
module.exports = config;
```

১.৭ প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — রিলিজের আগের রাতে দুর্বলতা ধরা পড়ল। TechNova-র React ফ্রন্টএন্ডের একটি নির্ভরতায় গুরুতর দুর্বলতা ধরা পড়েছে, আর সকালে বড় একটি ডেমো। পুরোনো পদ্ধতিতে হয় ডেমো পেছাত, নয় ঝুঁকি নিয়ে রিলিজ হতো। DevSecOps পদ্ধতিতে দলটি প্রথমে জিজ্ঞাসা করে: এই দুর্বলতা কি আমাদের অ্যাপ্লিকেশনে আদৌ কাজে লাগে? দুর্বল কোডপথটি কি আমরা ব্যবহার করি? যদি না হয়, ঝুঁকি কম এবং অস্থায়ী ব্যতিক্রম লিখে ডেমো করা যায়, সাথে পরের দিন ঠিক করার প্রতিশ্রুতি। যদি হ্যাঁ হয়, তাহলে নির্ভরতা আপডেট করা, অথবা আক্রান্ত বৈশিষ্ট্যটি সাময়িকভাবে বন্ধ রাখা — এই সিদ্ধান্ত নেওয়া যায় তথ্যের ভিত্তিতে, আতঙ্কের ভিত্তিতে নয়। শিক্ষা: দুর্বলতার তীব্রতা (severity) আর আপনার প্রেক্ষাপটে তার প্রভাব (exploitability) এক নয়।

পরিস্থিতি ২ — ডেভেলপার স্ক্যানারকে এড়িয়ে যাচ্ছেন। CI-তে SAST চালু করার পর ডেভেলপাররা অভিযোগ করছেন যে বিল্ড ধীর ও ভুল সতর্কতায় ভরা। এটি প্রযুক্তিগত সমস্যার চেয়ে সংস্কৃতির সমস্যা। সমাধান: প্রথমে সতর্কতাগুলোকে শুধু সতর্কতা হিসেবে দেখান, বিল্ড আটকাবেন না; সবচেয়ে বেশি ভুল সতর্কতা দেওয়া নিয়মগুলো ঠিক করুন; গুরুতর ও নিশ্চিত নিয়মগুলো ধীরে ধীরে কঠোর করুন; এবং ডেভেলপারদের দেখান কীভাবে ঠিক করতে হয় — শুধু ভুল ধরিয়ে দিয়ে ছেড়ে দেবেন না। নিরাপত্তা দল যদি শুধু বাধা দেয়, তাহলে মানুষ বাধা এড়াতে শিখবে; নিরাপত্তা দল যদি সাহায্য করে, তাহলে মানুষ সাহায্য চাইতে শিখবে।

পরিস্থিতি ৩ — প্রোডাকশনে অস্বাভাবিক লগইন। রাত ৩টায় একটি অ্যাডমিন অ্যাকাউন্ট থেকে অস্বাভাবিক দেশ থেকে লগইন চেষ্টা ধরা পড়েছে। Shift-Right এখানে কাজ করে: মনিটরিং অ্যালার্ট দেয়, দল অ্যাকাউন্টটি সাময়িকভাবে বন্ধ করে, পাসওয়ার্ড ও সেশন ঘোরায়, লগ পরীক্ষা করে দেখে আর কোনো অ্যাকাউন্টে একই চেষ্টা হয়েছে কি না, এবং শেষে শিক্ষা নিয়ে MFA বাধ্যতামূলক করে। এই ঘটনাটি শুধু একটি ঘটনা হিসেবে শেষ হয় না; এটি একটি Post-Incident Review-এ যায়, যেখান থেকে নকশা ও প্রক্রিয়ায় পরিবর্তন আসে।

> **REAL WORLD:** বড় প্রতিষ্ঠানগুলোতে নিরাপত্তা ঘটনা ঘটলে প্রথম প্রশ্ন হয় না “কে দোষী?”, প্রথম প্রশ্ন হয় “ব্যবস্থাটি কেন এই ভুলটি করতে দিল?” দোষ খোঁজা সংস্কৃতি মানুষকে ভুল লুকাতে শেখায়; ব্যবস্থা ঠিক করা সংস্কৃতি মানুষকে ভুল জানাতে শেখায়। Blameless Postmortem এই কারণেই DevSecOps সংস্কৃতির মূল অংশ।

১.৮ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তার খরচ শুধু টুল কেনার খরচ নয়। সবচেয়ে বড় খরচ হলো সময়: ডেভেলপারদের শেখার সময়, পাইপলাইন ধীর হওয়ার সময়, এবং ভুল সতর্কতা যাচাইয়ের সময়। আবার নিরাপত্তা না করার খরচও আছে: তথ্য ফাঁস, সেবা বন্ধ, আইনি জরিমানা ও সুনাম হারানো। ভালো সিদ্ধান্ত হলো ঝুঁকির ভিত্তিতে খরচ করা — সব জায়গায় সমান নয়। TechNova-র মতো ছোট দলের জন্য সবচেয়ে সাশ্রয়ী বিনিয়োগগুলো হলো: নির্ভরতা আপডেট রাখা, সিক্রেট Git-এ না দেওয়া, MFA চালু করা, ব্যাকআপ পরীক্ষা করা, এবং লগ দেখার অভ্যাস। এগুলো বিনামূল্যে বা কম খরচে করা যায়, কিন্তু প্রভাব অনেক বড়। ব্যয়বহুল টুল কেনার আগে জিজ্ঞাসা করুন: এই টুলটি কোন নির্দিষ্ট ঝুঁকি কমাবে, এবং সেই ঝুঁকি কি আমরা এখনই মুখোমুখি হচ্ছি? টুল কিনে নিরাপত্তা আসে না; টুল ব্যবহার করে, তার ফলাফলে কাজ করে নিরাপত্তা আসে।

Availability বনাম Reliability-এর পার্থক্যও এখানে মনে রাখুন। Availability মানে সেবাটি চালু আছে কি না; Reliability মানে সেবাটি সঠিকভাবে, প্রত্যাশিতভাবে কাজ করছে কি না। একটি সেবা চালু থাকলেও ভুল তথ্য দিতে পারে — তখন সেটি available কিন্তু reliable নয়। নিরাপত্তা ঘটনায় দুটোই ক্ষতিগ্রস্ত হতে পারে: আক্রমণে সেবা বন্ধ হতে পারে, আবার তথ্য বদলে গিয়ে ভুল সিদ্ধান্ত হতে পারে। তাই নিরাপত্তা শুধু গোপনীয়তার বিষয় নয়, এটি নির্ভরযোগ্যতারও বিষয়।

১.৯ সাধারণ ভুল

- **শেষে নিরাপত্তা যোগ করা:** রিলিজের আগে স্ক্যান চালিয়ে মনে করা যে কাজ শেষ। স্ক্যান একটি পরীক্ষা, নকশা নয়।
- **টুলকে দায়িত্ব দেওয়া:** “আমরা স্ক্যানার কিনেছি, তাই আমরা নিরাপদ” — এই ভাবনা। টুল ফলাফল দেয়, সিদ্ধান্ত মানুষকে নিতে হয়।
- **সব সতর্কতাকে সমান মনে করা:** গুরুতর ও তুচ্ছ সতর্কতা একসাথে দেখালে গুরুতরটি হারিয়ে যায়। অগ্রাধিকার ঠিক করুন।
- **মালিকানা অস্পষ্ট রাখা:** “সবাই দায়ী” মানে কার্যত “কেউ দায়ী নয়”। প্রতিটি নিয়ন্ত্রণের নামযুক্ত মালিক লিখুন।
- **ব্যতিক্রমকে স্থায়ী করা:** অস্থায়ী ছাড় কোনো মেয়াদ ছাড়া দিলে তা স্থায়ী দুর্বলতা হয়ে যায়।
- **প্রোডাকশনকে অদৃশ্য মনে করা:** শুধু কোড স্ক্যান করে প্রোডাকশন মনিটরিং না করা। আক্রমণ প্রোডাকশনেই ঘটে।
- **মানুষকে শত্রু মনে করা:** ডেভেলপারদের বাধা দিলে তারা নিরাপত্তাকে এড়িয়ে যাবে। সাহায্য করুন, বাধা নয়।

১.১০ ট্রাবলশুটিং

সমস্যা: CI পাইপলাইন নিরাপত্তা পরীক্ষায় বারবার ব্যর্থ হচ্ছে। প্রথমে দেখুন ব্যর্থতাটি কি নিশ্চিত দুর্বলতা, নাকি ভুল সতর্কতা। লগে নিয়মের নাম, ফাইলের পথ ও কারণ দেখুন। যদি একই নিয়ম বারবার ভুল সতর্কতা দেয়, নিয়মটি সাময়িকভাবে সতর্কতা মোডে নিয়ে টুলের সরবরাহকারী বা দলের সাথে আলোচনা করুন। যদি নিশ্চিত দুর্বলতা হয়, ঠিক করার মালিক নির্ধারণ করুন ও সময়সীমা দিন। পাইপলাইন বন্ধ করে দেওয়া শেষ উপায়, প্রথম উপায় নয়।

সমস্যা: স্ক্যানার অনেক ফলাফল দিচ্ছে, দল কিছুই করছে না। এটি অগ্রাধিকারের সমস্যা। ফলাফলগুলোকে প্রভাব ও কাজে লাগানোর সম্ভাবনা অনুযায়ী সাজান। প্রথমে ইন্টারনেটে উন্মুক্ত, প্রমাণিতভাবে কাজে লাগানো যায় এমন দুর্বলতাগুলো ঠিক করুন। বাকিগুলোকে ব্যাকলগে রাখুন ও নিয়মিত পর্যালোচনা করুন। শূন্য দুর্বলতা লক্ষ্য নয়; পরিচালিত ঝুঁকি লক্ষ্য।

সমস্যা: নিরাপত্তা দল ও ডেভেলপার দলের মধ্যে দ্বন্দ্ব। যৌথ লক্ষ্য ঠিক করুন: দ্রুত ও নিরাপদ রিলিজ। নিরাপত্তা দল ডেভেলপারদের কাজে অংশ নিন — কোড রিভিউয়ে সাহায্য করুন, নিরাপদ উদাহরণ দিন, প্রশিক্ষণ দিন। সিদ্ধান্তের জন্য একটি লিখিত প্রক্রিয়া রাখুন: কে ঝুঁকি গ্রহণ করতে পারেন, কতদিনের জন্য, কী শর্তে।

TROUBLESHOOTING: কোনো নিরাপত্তা নিয়ন্ত্রণ কাজ করছে কি না বুঝতে হলে শুধু "চালু আছে" দেখবেন না — পরীক্ষা করে দেখুন। একটি পরীক্ষামূলক দুর্বল নির্ভরতা ল্যাবে যোগ করে দেখুন CI তা ধরে কি না; একটি ভুয়া সিক্রেট কমিট করে দেখুন স্ক্যানার তা ধরে কি না। যে নিয়ন্ত্রণ পরীক্ষা করা যায় না, তার ওপর ভরসা করবেন না।

১.১১ সারসংক্ষেপ

DevSecOps মানে নিরাপত্তাকে জীবনচক্রের প্রতিটি ধাপে, শুরু থেকে, মালিকানা ও স্বয়ংক্রিয়তার সাথে যুক্ত করা। Shift-Left ভুল আগেই ধরে, Shift-Right প্রোডাকশন থেকে শেখে; দুটো একসাথে না থাকলে ব্যবস্থা অসম্পূর্ণ থাকে। স্ক্যানিং একটি উপকরণ, প্রকৌশল নয় — নকশা, হুমকি বিশ্লেষণ, কোড পর্যালোচনা ও মানুষের বিচার ছাড়া নিরাপত্তা আসে না। সবচেয়ে সাশ্রয়ী নিরাপত্তা হলো মৌলিক বিষয়গুলো ধারাবাহিকভাবে করা: নির্ভরতা আপডেট, সিক্রেট সুরক্ষা, MFA, ব্যাকআপ পরীক্ষা ও লগ পর্যবেক্ষণ। আর সবচেয়ে শক্তিশালী নিরাপত্তা সংস্কৃতি হলো সেইটি যেখানে ভুল জানানো নিরাপদ, সাহায্য চাওয়া সহজ, এবং প্রতিটি ঘটনা থেকে ব্যবস্থা উন্নত হয়।

১.১২ পর্যালোচনা প্রশ্ন

1. DevSecOps-এর সংজ্ঞা নিজের ভাষায় লিখুন এবং DevOps থেকে এটি কীভাবে আলাদা, তা ব্যাখ্যা করুন।
2. Shift-Left ও Shift-Right নিরাপত্তার মধ্যে পার্থক্য কী? একটি ছাড়া অন্যটি কেন অসম্পূর্ণ?
3. Security Scanning ও Security Engineering-এর পার্থক্য একটি উদাহরণসহ ব্যাখ্যা করুন।
4. Defense in Depth মানে কী? TechNova-র MERN অ্যাপে তিনটি স্তরের উদাহরণ দিন।
5. একটি নিরাপত্তা নিয়ন্ত্রণ কীভাবে ব্যর্থ হতে পারে? তিনটি ব্যর্থতার ধরন লিখুন।
6. Policy as Code-এর সুবিধা ও ঝুঁকি কী কী?
7. Alert Fatigue কী এবং এটি কীভাবে নিরাপত্তা কমিয়ে দেয়?
8. Availability ও Reliability-এর পার্থক্য ব্যাখ্যা করুন। একটি সেবা available কিন্তু reliable নয় — এমন উদাহরণ দিন।
9. Blameless Postmortem কেন নিরাপত্তা সংস্কৃতির জন্য গুরুত্বপূর্ণ?
10. ছোট দলের জন্য সবচেয়ে সাশ্রয়ী পাঁচটি নিরাপত্তা বিনিয়োগ কী কী এবং কেন?

১.১৩ ব্যবহারিক অনুশীলন

1. **জীবনচক্র মানচিত্র:** TechNova-র জন্য একটি এক পৃষ্ঠার DevOps Security Lifecycle মানচিত্র আঁকুন। প্রতিটি ধাপে একটি নিয়ন্ত্রণ, তার মালিক ও ব্যর্থতা ধরা পড়ার উপায় লিখুন। ল্যাব পরিবেশে সীমাবদ্ধ রাখুন।
2. **নিয়ন্ত্রণ পরীক্ষা:** আপনার ল্যাব রিপোজিটরিতে একটি ভুয়া সিক্রেট (যেমন CHANGE-ME-test-secret) কমিট করার চেষ্টা করে দেখুন আপনার স্ক্যানার তা ধরে কি না। ফলাফল লিখুন ও নিয়ন্ত্রণটি উন্নত করার একটি প্রস্তাব দিন। আসল সিক্রেট কখনো ব্যবহার করবেন না।
3. **ঘটনা পর্যালোচনা:** "রিলিজের আগের রাতে দুর্বলতা" পরিস্থিতিটি নিয়ে একটি সংক্ষিপ্ত Blameless Postmortem লিখুন: কী ঘটেছিল, ব্যবস্থা কেন তা হতে দিল, এবং ভবিষ্যতে কী বদলাবেন — ব্যক্তিকে দোষ না দিয়ে।

অধ্যায় ২ • PART I — Security Engineering Foundations

Chapter 2 — Secure SDLC and Threat Modeling

২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Secure Software Development Lifecycle (Secure SDLC)-এর প্রতিটি ধাপ ব্যাখ্যা করতে পারবেন, Attack Surface-এর ধারণা ব্যবহার করে TechNova-র MERN অ্যাপ্লিকেশনের উন্মুক্ত অংশগুলো চিহ্নিত করতে পারবেন, এবং STRIDE পদ্ধতিতে একটি ছোট বৈশিষ্ট্যের Threat Modeling করতে পারবেন। আপনি Trust Boundary কী, Risk Assessment কীভাবে করা হয়, Security Requirements কীভাবে লেখা হয়, এবং Architecture Security Review-তে কী প্রশ্ন করতে হয় — তা শিখবেন। সবশেষে, আপনি বুঝতে পারবেন কেন স্ক্যানার Threat Modeling-এর বিকল্প নয়, এবং কীভাবে হুমকি বিশ্লেষণ থেকে বাস্তব নিয়ন্ত্রণে পৌঁছানো যায়।

- Secure SDLC-এর ধাপগুলো ও প্রতিটি ধাপের নিরাপত্তা কাজ বর্ণনা করা।
- Attack Surface চিহ্নিত করা ও তা কমানোর উপায় ব্যাখ্যা করা।
- STRIDE-এর ছয়টি শ্রেণি দিয়ে হুমকি শ্রেণিবদ্ধ করা।
- Trust Boundary আঁকা ও সীমানা অতিক্রমের সময় যাচাই নির্ধারণ করা।
- ঝুঁকি মূল্যায়ন করে অগ্রাধিকার ঠিক করা ও Security Requirements লেখা।

২.২ পূর্বশর্ত ঝালাই

আগের অধ্যায় থেকে DevSecOps Lifecycle ও Defense in Depth মনে রাখুন। এই অধ্যায়ে আরও দুটি ধারণা দরকার। প্রথমটি হলো Asset (সম্পদ): যা রক্ষা করতে চান — ব্যবহারকারীর তথ্য, পাসওয়ার্ডের হ্যাশ, পেমেন্ট তথ্য, সোর্স কোড, সিক্রেট, ব্যবসায়িক যুক্তি। দ্বিতীয়টি হলো Threat (হুমকি): যে কেউ বা যা কিছু সম্পদের ক্ষতি করতে পারে — বাইরের আক্রমণকারী, ভেতরের অসন্তুষ্ট কর্মী, ভুল কনফিগারেশন, এমনকি দুর্ঘটনা। Vulnerability (দুর্বলতা) হলো সেই ফাঁক, যা দিয়ে হুমকি সম্পদে পৌঁছায়। হুমকি থাকলেই ক্ষতি হয় না; হুমকি, দুর্বলতা ও সম্পদ — তিনটি একসাথে মিললেই ঝুঁকি তৈরি হয়। এই তিনটি আলাদা করে দেখতে পারলে Threat Modeling সহজ হয়ে যায়। আরেকটি মনে রাখার বিষয়: আক্রমণকারী আপনার ব্যবস্থাকে আপনার মতো করে দেখে না; সে দেখে কোথায় ঢোকা যায়, কোথায় বিশ্বাস করা হয়, কোথায় যাচাই কম। আপনার কাজ হলো আক্রমণকারীর চোখে ব্যবস্থাটি দেখতে শেখা — কিন্তু শুধু অনুমোদিত ল্যাবে, শুধু নিজের ব্যবস্থায়, শেখার উদ্দেশ্যে।

২.৩ প্রথম নীতি থেকে Secure SDLC: নিরাপত্তা কেন একটি ধাপ নয়, একটি ধারা

Secure SDLC মানে সফটওয়্যার তৈরির প্রতিটি ধাপে নিরাপত্তা সিদ্ধান্ত নেওয়া। ঐতিহ্যগত SDLC-তে ধাপগুলো ছিল: প্রয়োজন নির্ধারণ, নকশা, বাস্তবায়ন, পরীক্ষা, ডিপ্লয়, রক্ষণাবেক্ষণ। Secure SDLC সেই ধাপগুলো বদলায় না; প্রতিটি ধাপে একটি প্রশ্ন যোগ করে: “এখানে কী ভুল হতে পারে, এবং আমরা এখন কী করব?” প্রয়োজন নির্ধারণের সময় প্রশ্ন হলো — এই বৈশিষ্ট্যটি কার তথ্য ছুঁবে, কী সংবেদনশীলতা আছে? নকশার সময় প্রশ্ন — কে কী করতে পারবে, কোথায় বিশ্বাসের সীমা? কোডের সময় প্রশ্ন — ইনপুট কীভাবে যাচাই হবে, ভুল হলে কী দেখানো হবে? পরীক্ষার সময় প্রশ্ন — আক্রমণকারীর মতো করে ভাবলে কী ভাঙে? ডিপ্লয়ের সময় প্রশ্ন — কনফিগারেশন কি নিরাপদ, সিক্রেট কি সুরক্ষিত? পরিচালনার সময় প্রশ্ন — অস্বাভাবিক কিছু ঘটছে কি?

এই ধারাটি কেন দরকার, তা খরচের দিক থেকে বোঝা যায়। নকশায় একটি ভুল ধরা পড়লে তা ঠিক করতে একটি বৈঠক ও একটি নথি বদলাতে হয়। কোডে ধরা পড়লে কয়েক লাইন বদলাতে হয়। পরীক্ষায় ধরা পড়লে আবার কোডে ফিরতে হয়।

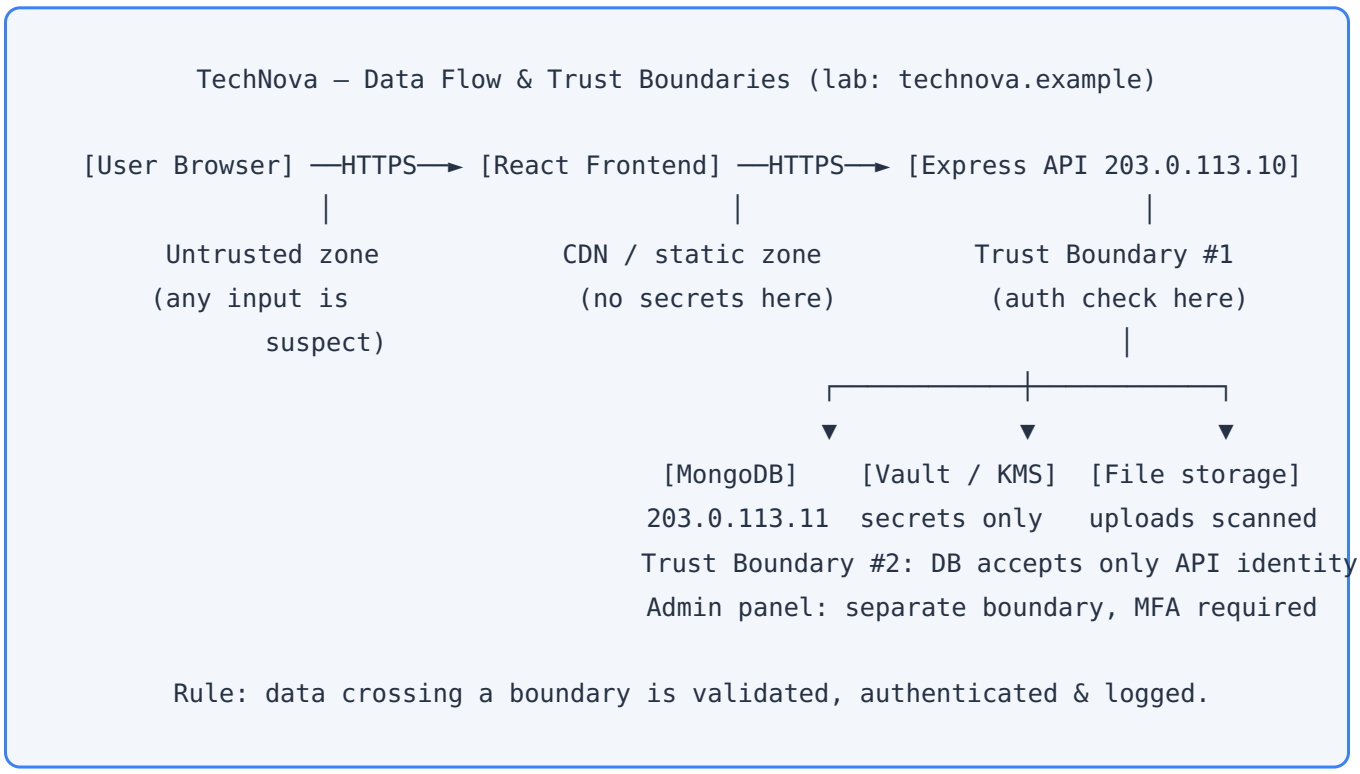
প্রোডাকশনে ধরা পড়লে গ্রাহককে জানাতে হয়, তথ্য ফাঁস সামলাতে হয়, আইনি প্রক্রিয়া আসতে পারে। তাই Secure SDLC-এর অর্থনীতি সহজ: যত আগে ধরবেন, তত সস্তায় ঠিক করবেন। কিন্তু এর অর্থ এই নয় যে পরের ধাপগুলো অপ্রয়োজনীয়; প্রতিটি ধাপ আগের ধাপের ভুল ধরার আরেকটি সুযোগ। কোনো একক ধাপ নিখুঁত নয় বলেই একাধিক ধাপ দরকার।

> **CONCEPT:** Attack Surface মানে আপনার ব্যবস্থার সেই সব জায়গা, যেখানে বাইরের কেউ তথ্য দিতে পারে, তথ্য নিতে পারে, বা আচরণ প্রভাবিত করতে পারে — API এন্ডপয়েন্ট, ফর্ম, ফাইল আপলোড, লগইন পৃষ্ঠা, অ্যাডমিন প্যানেল, এমনকি ভুল বার্তা (error message)। Attack Surface যত ছোট, আক্রমণকারীর সুযোগ তত কম। নিরাপত্তার প্রথম কাজ হলো এই পৃষ্ঠতলটি দেখা ও ছোট করা।

TechNova-র উদাহরণ নিন। MERN অ্যাপে React ফ্রন্টএন্ড, Express API, MongoDB ডাটাবেস। Attack Surface-এর মধ্যে আছে: পাবলিক API রুট (/api/auth/login, /api/products, /api/orders), ফাইল আপলোড (প্রোফাইল ছবি), অনুসন্ধান (search) প্যারামিটার, অ্যাডমিন রুট, পাসওয়ার্ড রিসেট প্রবাহ, তৃতীয় পক্ষের নির্ভরতা, এবং ব্যবস্থাপনা পোর্ট (MongoDB, SSH)। প্রতিটি জায়গায় প্রশ্ন একই: কে এখানে পৌঁছাতে পারে? কী দিতে পারে? কী পেতে পারে? যদি কোনো রুটের প্রয়োজন না থাকে, তা বন্ধ করুন। যদি কোনো প্যারামিটারের প্রয়োজন না থাকে, তা গ্রহণ করবেন না। যা গ্রহণ করবেন, তা যাচাই করুন। এই সরল নীতিই Attack Surface কমানোর মূল কথা।

২.৪ আর্কিটেকচার: Trust Boundary ও ডেটা প্রবাহ

Threat Modeling শুরু হয় ছবি আঁকা দিয়ে — কোড লেখার আগে, ব্যবস্থাটি কীভাবে কাজ করে তার একটি সরল চিত্র। নিচের চিত্রে TechNova-র ডেটা প্রবাহ ও Trust Boundary দেখানো হয়েছে। Trust Boundary হলো সেই রেখা, যার এক পাশের জিনিসকে আপনি বিশ্বাস করেন, অন্য পাশের জিনিসকে যাচাই ছাড়া বিশ্বাস করেন না।



Trust Boundary আঁকার নিয়ম: প্রথমে উপাদানগুলো (ব্যবহারকারী, ফ্রন্টএন্ড, API, ডাটাবেস, বাইরের সেবা) বাক্স হিসেবে আঁকুন; তারপর তীর দিয়ে ডেটা প্রবাহ দেখান — কে কাকে কী পাঠায়। এরপর জিজ্ঞাসা করুন: কোন বাক্সটি আক্রমণকারীর নিয়ন্ত্রণে যেতে পারে? ব্রাউজার যেতে পারে, তাই ব্রাউজার থেকে আসা সবকিছু সন্দেহজনক। ফ্রন্টএন্ডের কোড ব্যবহারকারীর

কাছে দৃশ্যমান, তাই সেখানে সিক্রেট রাখা যাবে না। API সার্ভার আপনার নিয়ন্ত্রণে, কিন্তু তার কাছে আসা প্রতিটি অনুরোধ যাচাই করতে হবে। ডাটাবেস শুধু API-র পরিচয় গ্রহণ করবে, সরাসরি ইন্টারনেট থেকে নয়। এই সীমানাগুলো স্পষ্ট হলে প্রতিটি সীমানায় কী যাচাই দরকার, তা ঠিক করা সহজ হয়।

সীমানা	যা অতিক্রম করে	যাচাই	ভুল হলে
Browser → API	লগইন, ফর্ম, API কল	Auth, ইনপুট যাচাই, Rate limit	Injection, ভাঙা auth
API → MongoDB	কোয়েরি, লেখা	Parameterized query, least privilege	তথ্য ফাঁস, তথ্য বদল
User → Admin	অ্যাডমিন কাজ	MFA, RBAC, audit log	অনুমোদনহীন অ্যাডমিন কাজ
App → বাইরের সেবা	পেমেন্ট, ইমেইল	TLS, API key, response যাচাই	ভুয়া সেবা, তথ্য ফাঁস

২.৫ অভ্যন্তরীণ কার্যপ্রণালী: STRIDE দিয়ে হুমকি ভাবা

STRIDE হলো Microsoft-এর তৈরি একটি মনে রাখার কাঠামো, যা ছয় ধরনের হুমকি মনে করিয়ে দেয়। এটি কোনো যাদুর তালিকা নয়; এটি একটি চেকলিস্ট, যাতে কোনো ধরনের হুমকি ভুলে না যান। প্রতিটি অক্ষরের অর্থ বাংলায় বুঝুন।

STRIDE	অর্থ	TechNova উদাহরণ	নিয়ন্ত্রণ
S — Spoofing	অন্যের পরিচয় নেওয়া	অন্যের টোকেন দিয়ে লগইন	শক্তিশালী auth, MFA, সেশন সুরক্ষা
T — Tampering	তথ্য বদলে দেওয়া	দাম বদলে অর্ডার করা	Integrity check, সার্ভার-সাইড যাচাই
R — Repudiation	কাজ অস্বীকার করা	“আমি অর্ডার করিনি” দাবি	Audit log, স্বাক্ষরিত লেনদেন
I — Information Disclosure	গোপন তথ্য ফাঁস	ভুল বার্তায় তথ্য দেখা	Least privilege, ভুল বার্তা নিয়ন্ত্রণ
D — Denial of Service	সেবা বন্ধ করা	অসংখ্য অনুরোধে API বন্ধ	Rate limit, ক্যাশ, স্কেল
E — Elevation of Privilege	বেশি ক্ষমতা পাওয়া	সাধারণ ব্যবহারকারী অ্যাডমিন হওয়া	RBAC, সার্ভার-সাইড অনুমতি পরীক্ষা

STRIDE ব্যবহারের পদ্ধতি সহজ: একটি বৈশিষ্ট্য নিন — যেমন পাসওয়ার্ড রিসেট। প্রতিটি অক্ষরের জন্য জিজ্ঞাসা করুন: এখানে কি Spoofing সম্ভব? কেউ কি অন্যের ইমেইলে রিসেট লিংক পাঠাতে পারে? Tampering সম্ভব? লিংকের টোকেন কি অনুমান করা যায়? Repudiation? কেউ কি রিসেট অস্বীকার করতে পারে — এবং তা কি গুরুত্বপূর্ণ? Information Disclosure? ভুল বার্তা কি বলে দেয় কোন ইমেইল নিবন্ধিত? Denial of Service? কেউ কি বারবার রিসেট চেয়ে ইনবক্স ভরিয়ে দিতে পারে? Elevation? রিসেট দিয়ে কি অ্যাডমিন অ্যাকাউন্ট দখল করা যায়? প্রতিটি “হ্যাঁ”-এর জন্য একটি নিয়ন্ত্রণ লিখুন, ঝুঁকি মূল্যায়ন করুন, এবং সিদ্ধান্ত নিন — এখনই ঠিক করবেন, নাকি গ্রহণ করবেন।

Risk Assessment মানে প্রতিটি হুমকির সম্ভাবনা ও প্রভাব অনুমান করা। একটি সহজ পদ্ধতি: সম্ভাবনা ১–৫, প্রভাব ১–৫ — গুণফল যত বেশি, অগ্রাধিকার তত বেশি। কিন্তু সংখ্যা মুখস্থ করবেন না; সংখ্যার পেছনের যুক্তি লিখুন। কেন সম্ভাবনা ৪? কারণ এই এন্ডপয়েন্ট ইন্টারনেটে খোলা ও স্বয়ংক্রিয়ভাবে পরীক্ষা করা যায়। কেন প্রভাব ৫? কারণ এতে পেমেন্ট তথ্য জড়িত। এই যুক্তি না থাকলে সংখ্যা শুধু অনুমান। আরেকটি গুরুত্বপূর্ণ কথা: ঝুঁকি শূন্য করা যায় না। লক্ষ্য হলো ঝুঁকিকে সহনীয় পর্যায়ে নামানো ও সেই সিদ্ধান্ত লিখে রাখা — কে, কখন, কেন এই ঝুঁকি গ্রহণ করলেন।

> **WARNING:** স্ক্যানার Threat Modeling-এর বিকল্প নয়। স্ক্যানার পরিচিত দুর্বলতা খুঁজে পায়; Threat Modeling আপনার ব্যবস্থার বিশেষ যুক্তি, বিশেষ বিশ্বাস ও বিশেষ তথ্য প্রবাহের দুর্বলতা খুঁজে পায় — যেমন দাম ক্লায়েন্ট থেকে আসে, অ্যাডমিন যাচাই শুধু ফ্রন্টএন্ডে হয়, বা রিসেট টোকেনের মেয়াদ নেই। এই ভুলগুলো কোনো স্ক্যানার ধরতে পারে না, শুধু মানুষের চিন্তা ধরতে পারে।

Security Requirements লেখার সময় তা পরীক্ষাযোগ্য করে লিখুন। “অ্যাপ্লিকেশন নিরাপদ হবে” — এটি প্রয়োজন নয়, এটি ইচ্ছা। “সব পাসওয়ার্ড কমপক্ষে ১২ অক্ষরের হবে, bcrypt দিয়ে হ্যাশ হবে, এবং ভুল লগইন পাঁচবারের পর অ্যাকাউন্ট সাময়িকভাবে বন্ধ হবে” — এটি পরীক্ষাযোগ্য প্রয়োজন। প্রতিটি প্রয়োজনের পাশে লিখুন: কীভাবে পরীক্ষা করবেন, কে পরীক্ষা করবেন, কখন। Architecture Security Review মানে নকশা চূড়ান্ত হওয়ার আগে এই প্রশ্নগুলো করা: তথ্য কোথায় থাকে, কে তা দেখে, সীমানা কোথায়, ব্যর্থ হলে কী হয়, লগ কোথায় যায়, সিক্রেট কোথায় থাকে, এবং আক্রমণকারী প্রথমে কোথায় আঘাত করবে বলে মনে হয়। এই পর্যালোচনা একটি আনুষ্ঠানিক সভা হতে হবে এমন নয়; একটি সংক্ষিপ্ত, লিখিত প্রশ্নোত্তরও যথেষ্ট — যদি তা সত্যিই করা হয়।

২.৬ বাস্তব কমান্ড ও কনফিগারেশন

Threat Modeling-এর ফলাফল কোডে ও কনফিগারেশনে নামাতে হয়। নিচে Express-এ নিরাপত্তা হেডার ও ইনপুট যাচাইয়ের একটি ধারণাগত উদাহরণ দেওয়া হলো — ল্যাব পরিবেশে পরীক্ষার জন্য, প্রোডাকশনে ব্যবহারের আগে আপনার দলের পর্যালোচনা প্রয়োজন।

```
// server/app.js — নিরাপত্তা হেডার ও JSON সীমা (ল্যাব উদাহরণ)
const express = require('express');
const helmet = require('helmet');
const app = express();
app.use(helmet());
app.use(express.json({ limit: '100kb' })); // বড় payload প্রত্যাখ্যান
// দাম কখনো ক্লায়েন্ট থেকে নেবেন না — সার্ভারে নির্ধারণ করুন
app.post('/api/orders', (req, res) => {
  const { productId, quantity } = req.body;
  if (!productId || !Number.isInteger(quantity) || quantity < 1) {
    return res.status(400).json({ error: 'Invalid request' });
  }
  // TODO: productId দিয়ে সার্ভার-সাইডে দাম দেখুন, ক্লায়েন্টের দাম বিশ্বাস করবেন না
  res.json({ ok: true });
});
module.exports = app;
```

Attack Surface যাচাইয়ের জন্য নিজের ল্যাব সার্ভারে কোন রুট উন্মুক্ত, তা তালিকা করুন। এটি অনুমোদিত নিজের ব্যবস্থায় করা পরীক্ষা; অন্যের ব্যবস্থায় নয়।

```
# ল্যাব: TechNova API রুট তালিকা (নিজের কোডবেসে)
grep -R "app\.\(get\|post\|put\|delete\)" server/ --include="*.js" | head -n 50
# ল্যাব সার্ভারে শোনা পোর্ট যাচাই
sudo ss -tlnp
# MongoDB কি শুধু localhost / private নেটওয়ার্কে শুনছে, তা যাচাই করুন
sudo grep -E "bindIp|port" /etc/mongod.conf
```

একটি সরল Threat Model নথির কাঠামো নিচে দেওয়া হলো। এটি একটি Markdown ফাইল হিসেবে রিপোজিটরিতে রাখুন, যাতে নকশা বদলালে নথিটিও বদলায়।

```
# Threat Model — Password Reset (TechNova, lab)
## বৈশিষ্ট্য: ব্যবহারকারী ইমেইল দিয়ে পাসওয়ার্ড রিসেট করেন
## সম্পদ: অ্যাকাউন্ট, ইমেইল, রিসেট টোকেন
## STRIDE:
## - Spoofing: অন্যের ইমেইলে রিসেট অনুরোধ — নিয়ন্ত্রণ: টোকেন শুধু সেই ইমেইলে যায়
## - Tampering: টোকেন অনুমান — নিয়ন্ত্রণ: যথেষ্ট দীর্ঘ, এলোমেলো টোকেন, একবার ব্যবহারযোগ্য
## - Information Disclosure: "এই ইমেইল নেই" বার্তা — নিয়ন্ত্রণ: সবসময় একই বার্তা
## - DoS: বারবার অনুরোধ — নিয়ন্ত্রণ: Rate limit, প্রতি ঘণ্টায় সীমা
## ঝুঁকি: টোকেন ফাঁস হলে অ্যাকাউন্ট দখল — মেয়াদ 15 মিনিট, একবার ব্যবহার, লগ রাখা
```

২.৭ প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — দাম ক্লায়েন্ট থেকে আসছিল। TechNova-র অর্ডার API ক্লায়েন্টের পাঠানো দাম বিশ্বাস করত। একজন ব্যবহারকারী ব্রাউজারের অনুরোধ বদলে ১০০ টাকার পণ্য ১ টাকায় কিনলেন। স্ক্যানার এই ভুল ধরেনি, কারণ কোডে কোনো পরিচিত প্যাটার্ন ছিল না — এটি ব্যবসায়িক যুক্তির ভুল। Threat Modeling-এ “Tampering: দাম কি সার্ভারে যাচাই হয়?” প্রশ্নটি থাকলে এই ভুল নকশাতেই ধরা পড়ত। শিক্ষা: সংবেদনশীল সিদ্ধান্ত সবসময় সার্ভারে নিন; ক্লায়েন্টের পাঠানো তথ্য শুধু অনুরোধ, প্রমাণ নয়।

পরিস্থিতি ২ — ভুল বার্তায় তথ্য ফাঁস। লগইন ব্যর্থ হলে API বলত “এই ইমেইল নিবন্ধিত নয়” অথবা “পাসওয়ার্ড ভুল”। এতে আক্রমণকারী বুঝতে পারত কোন ইমেইলগুলো আসল। সমাধান: সবসময় একই বার্তা — “ইমেইল বা পাসওয়ার্ড ভুল”। ছোট পরিবর্তন, বড় প্রভাব। Threat Modeling-এর Information Disclosure প্রশ্নটি এই ফাঁস ধরিয়ে দেয়।

পরিস্থিতি ৩ — অ্যাডমিন যাচাই শুধু ফ্রন্টএন্ডে। React-এ একটি রুট শুধু অ্যাডমিনদের দেখাত, কিন্তু API সেই যাচাই করত না। কেউ সরাসরি API কল করে অ্যাডমিন কাজ করে ফেললেন। শিক্ষা: ফ্রন্টএন্ডের যাচাই ব্যবহারকারীর সুবিধার জন্য, নিরাপত্তার জন্য নয়। নিরাপত্তা যাচাই সবসময় সার্ভারে, প্রতিটি অনুরোধে। Trust Boundary মনে রাখুন: ব্রাউজার অবিশ্বস্ত অঞ্চল।

> **REAL WORLD:** অনেক বড় তথ্য ফাঁসের পেছনে জটিল আক্রমণ নয়, একটি সরল নকশার ভুল থাকে — যেমন সার্ভার-সাইড যাচাই না থাকা, বা ভুল বার্তায় তথ্য দেওয়া। Threat Modeling-এর মূল্য এখানেই: এটি জটিল টুল নয়, এটি ধীরে ধীরে, প্রশ্ন করে ভাবার অভ্যাস।

২.৮ নিরাপত্তা ও খরচ বিবেচনা

Threat Modeling-এর খরচ মূলত সময়: একটি বৈশিষ্ট্যের জন্য ৩০–৬০ মিনিটের আলোচনা। এর বিনিময়ে আপনি পান আগেভাগে ধরা ভুল, স্পষ্ট অগ্রাধিকার, এবং একটি লিখিত সিদ্ধান্ত — যা পরে কেউ প্রশ্ন করলে দেখানো যায়। বিপরীতে, Threat Modeling না করার খরচ হলো প্রোডাকশনে ধরা ভুলের খরচ, যা বহুগুণ বেশি। ছোট দলের জন্য পরামর্শ: সব বৈশিষ্ট্যের জন্য পূর্ণ Threat Modeling করবেন না; সংবেদনশীল বৈশিষ্ট্য — লগইন, পেমেন্ট, অ্যাডমিন, ফাইল আপলোড, তথ্য রপ্তানি — সেগুলোর জন্য করুন। বাকিগুলোর জন্য একটি সংক্ষিপ্ত চেকলিস্ট যথেষ্ট। আরেকটি খরচের দিক: অতিরিক্ত নিয়ন্ত্রণ ব্যবহারকারীর অভিজ্ঞতা খারাপ করতে পারে। যেমন, প্রতিটি কাজে MFA চাইলে ব্যবহারকারী বিরক্ত হবেন। তাই নিয়ন্ত্রণ ঝুঁকির সাথে সামঞ্জস্যপূর্ণ হতে হবে — সংবেদনশীল কাজে কঠোর, সাধারণ কাজে হালকা।

২.৯ সাধারণ ভুল

- **Threat Modeling-কে একবারের কাজ মনে করা:** নকশা বদলালে হুমকিও বদলায়। Threat Model জীবন্ত নথি; বৈশিষ্ট্য বদলালে আপডেট করুন।

- **শুধু বাইরের আক্রমণকারী ভাবা:** ভেতরের ভুল, ভুল কনফিগারেশন ও দুর্ঘটনাও হুমকি।
- **ক্লায়েন্টকে বিশ্বাস করা:** ফ্রন্টএন্ডের যাচাইকে নিরাপত্তা মনে করা সবচেয়ে সাধারণ ভুল।
- **ভুল বার্তায় তথ্য দেওয়া:** বিস্তারিত ভুল বার্তা আক্রমণকারীকে সাহায্য করে। ব্যবহারকারীকে সহায়ক, আক্রমণকারীকে অসহায়ক বার্তা দিন।
- **ঝুঁকি লিখে না রাখা:** "আমরা জানতাম, কিন্তু ঠিক করিনি" — লিখিত না থাকলে পরে কেউ জানবে না কেন।
- **STRIDE-কে টিকচিহ্ন মনে করা:** ছয়টি অক্ষরে টিক দিলেই Threat Modeling হয় না; প্রতিটি হুমকির পেছনে আক্রমণের পথ ও নিয়ন্ত্রণ লিখতে হয়।

২.১০ ট্রাবলশুটিং

সমস্যা: Threat Modeling সভা দীর্ঘ ও অনির্দিষ্ট হয়ে যাচ্ছে। পরিধি ছোট করুন: একটি বৈশিষ্ট্য, একটি ডেটা প্রবাহ। সময়সীমা দিন (৪৫ মিনিট), প্রশ্নগুলো আগে পাঠান, এবং শেষে সিদ্ধান্ত লিখুন। নিখুঁত মডেল লক্ষ্য নয়; উপযোগী মডেল লক্ষ্য।

সমস্যা: দল বলছে "আমাদের কিছুই লুকানোর নেই, তাই Threat Modeling দরকার নেই।" তথ্য ছাড়াও সম্পদ আছে: সেবার উপলভ্যতা, সুনাম, ব্যবহারকারীর আস্থা। একটি উদাহরণ দিয়ে বোঝান: অর্ডারের দাম বদলে গেলে ব্যবসার ক্ষতি হয়, তথ্য ফাঁস না হলেও।

সমস্যা: ঝুঁকি মূল্যায়নে সবাই ভিন্ন সংখ্যা দিচ্ছে। সংখ্যা নিয়ে তর্ক করবেন না; যুক্তি নিয়ে আলোচনা করুন। "কেন সম্ভাবনা ৪?" — এই প্রশ্নের উত্তরই মূল্যবান। যুক্তি একমত হলে সংখ্যা নিজে থেকেই মিলবে।

> **TROUBLESHOOTING:** যদি কোনো বৈশিষ্ট্যের Threat Model লিখতে গিয়ে আটকে যান, তাহলে ডেটা প্রবাহ চিত্র থেকে শুরু করুন। চিত্র না থাকলে হুমকি অনুমান হয়ে যায়। প্রথমে বাক্স ও তীর আঁকুন, তারপর STRIDE প্রশ্ন করুন। চিত্রই চিন্তার মানচিত্র।

২.১১ সারসংক্ষেপ

Secure SDLC মানে প্রতিটি ধাপে "এখানে কী ভুল হতে পারে?" প্রশ্ন করা। Attack Surface দেখা ও ছোট করা, Trust Boundary আঁকা, STRIDE দিয়ে হুমকি ভাবা, ঝুঁকি মূল্যায়ন করে অগ্রাধিকার ঠিক করা, এবং পরীক্ষাযোগ্য Security Requirements লেখা — এই ধাপগুলো একসাথে Threat Modeling-এর মূল কাজ। স্ক্যানার এই কাজের বিকল্প নয়; স্ক্যানার পরিচিত প্যাটার্ন ধরে, Threat Modeling আপনার ব্যবস্থার বিশেষ যুক্তির ভুল ধরে। সবচেয়ে গুরুত্বপূর্ণ নিয়মটি মনে রাখুন: সংবেদনশীল সিদ্ধান্ত সার্ভারে নিন, ক্লায়েন্টকে বিশ্বাস করবেন না, এবং ঝুঁকির সিদ্ধান্ত লিখে রাখুন।

২.১২ পর্যালোচনা প্রশ্ন

1. Secure SDLC-এর ধাপগুলো লিখুন এবং প্রতিটি ধাপে একটি করে নিরাপত্তা কাজের উদাহরণ দিন।
2. Attack Surface কাকে বলে? TechNova-র পাঁচটি Attack Surface চিহ্নিত করুন।
3. Trust Boundary কী? একটি সীমানা অতিক্রমের সময় কী কী যাচাই দরকার, তা ব্যাখ্যা করুন।
4. STRIDE-এর প্রতিটি অক্ষরের অর্থ ও একটি করে উদাহরণ লিখুন।
5. Risk Assessment-এ সম্ভাবনা ও প্রভাব কীভাবে অনুমান করবেন? সংখ্যার চেয়ে যুক্তি কেন গুরুত্বপূর্ণ?
6. পরীক্ষাযোগ্য Security Requirement ও অস্পষ্ট ইচ্ছার মধ্যে পার্থক্য উদাহরণসহ দেখান।
7. Architecture Security Review-তে কোন পাঁচটি প্রশ্ন অবশ্যই করবেন?

8. “দাম ক্লায়েন্ট থেকে আসছিল” — এই ভুলটি কোন STRIDE শ্রেণিতে পড়ে এবং কীভাবে ঠিক করবেন?
9. স্ক্যানার কেন Threat Modeling-এর বিকল্প নয়? দুটি কারণ লিখুন।
10. ছোট দলে Threat Modeling-এর পরিধি কীভাবে ঠিক করবেন? কোন বৈশিষ্ট্যগুলোকে অগ্রাধিকার দেবেন?

২.১৩ ব্যবহারিক অনুশীলন

1. **ডেটা প্রবাহ আঁকুন:** TechNova-র লগইন প্রবাহের ডেটা প্রবাহ চিত্র আঁকুন, Trust Boundary চিহ্নিত করুন, এবং প্রতিটি সীমানায় যাচাই লিখুন।
2. **STRIDE অনুশীলন:** ফাইল আপলোড বৈশিষ্ট্যের জন্য STRIDE টেবিল তৈরি করুন: প্রতিটি শ্রেণিতে একটি হুমকি, তার আক্রমণ পথ ও একটি নিয়ন্ত্রণ লিখুন। ল্যাবে সীমাবদ্ধ রাখুন।
3. **Security Requirements লিখুন:** পাসওয়ার্ড রিসেটের জন্য পাঁচটি পরীক্ষাযোগ্য Security Requirement লিখুন, প্রতিটির পাশে পরীক্ষার পদ্ধতি উল্লেখ করুন।

অধ্যায় ৩ • PART I — Security Engineering Foundations

Chapter 3 — Infrastructure Security

৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Ubuntu Server 24.04 LTS-এ চলা TechNova-র অবকাঠামোর নিরাপত্তা নীতিগুলো ব্যাখ্যা করতে পারবেন এবং নিজের ল্যাব সার্ভারে তা প্রয়োগ করতে পারবেন। আপনি Linux নিরাপত্তা, SSH Hardening, ফাইলসিস্টেম অনুমতি, Process Isolation, Network Segmentation, ফায়ারওয়াল, Server Hardening ও Host Monitoring-এর প্রতিটি ধারণা প্রথম নীতি থেকে বুঝবেন — কী সমস্যা সমাধান করে, ভেতরে কীভাবে কাজ করে, কীভাবে ব্যর্থ হয়, এবং ব্যর্থতা কীভাবে ধরা পড়ে। সবশেষে, আপনি একটি সার্ভারের Attack Surface মূল্যায়ন করে একটি অগ্রাধিকারভিত্তিক Hardening পরিকল্পনা লিখতে পারবেন।

- Linux অনুমতি মডেল (user, group, other; read, write, execute) ব্যাখ্যা ও প্রয়োগ করা।
- SSH Hardening: কী-ভিত্তিক লগইন, root নিষেধ, পোর্ট ও কনফিগারেশন যাচাই করা।
- ফায়ারওয়াল ও Network Segmentation দিয়ে উন্মুক্ত পৃষ্ঠতল ছোট করা।
- Process Isolation ও Least Privilege দিয়ে একটি সার্ভিস আপস হলে ক্ষতি সীমিত করা।
- Host Monitoring ও লগ দিয়ে অস্বাভাবিক আচরণ শনাক্ত করা।

৩.২ পূর্বশর্ত ঝালাই

Linux-এর মৌলিক কমান্ড — ফাইল দেখা, অনুমতি দেখা, সার্ভিস চালু-বন্ধ করা — জানা থাকলে এই অধ্যায় সহজ হবে। মনে রাখুন: Linux-এ সবকিছু ফাইল; কনফিগারেশন, ডিভাইস, এমনকি প্রক্রিয়ার তথ্যও ফাইল হিসেবে দেখা যায়। প্রতিটি ফাইলের একজন মালিক (owner), একটি গ্রুপ, এবং তিন ধরনের অনুমতি থাকে — পড়া (r), লেখা (w), চালানো (x) — মালিক, গ্রুপ ও অন্যদের জন্য আলাদাভাবে। Root ব্যবহারকারী (UID 0) প্রায় সবকিছু করতে পারে; তাই root-এর ব্যবহার যত কম, তত নিরাপদ। Process মানে চলমান প্রোগ্রাম; প্রতিটি প্রক্রিয়া একটি ব্যবহারকারীর পরিচয়ে চলে, এবং সেই পরিচয়ের অনুমতির বাইরে যেতে পারে না — যদি না ভুল কনফিগারেশন তাকে বেশি ক্ষমতা দেয়। নেটওয়ার্কের দিক থেকে মনে রাখুন: একটি পোর্ট খোলা থাকা মানে সেই পোর্টে একটি সার্ভিস শুনছে; যে পোর্টের প্রয়োজন নেই, তা বন্ধ রাখাই নিরাপত্তা।

৩.৩ প্রথম নীতি থেকে অবকাঠামো নিরাপত্তা: সার্ভার কেন বিশ্বাসের সীমানা

অবকাঠামো নিরাপত্তার প্রথম নীতি হলো: আপনার অ্যাপ্লিকেশন যতই নিরাপদ হোক, তার নিচের সার্ভার আপস হলে সবকিছু আপস হয়। সার্ভার হলো ভিত্তি; ভিত্তি ভেঙে গেলে ওপরের কোনো নিরাপত্তাই টেকে না। সমস্যাটি কেন ঘটে? কারণ সার্ভার শুরুতে সুবিধার জন্য কনফিগার করা হয়, নিরাপত্তার জন্য নয়। ডিফল্ট পাসওয়ার্ড, খোলা পোর্ট, অপ্রয়োজনীয় সার্ভিস, পুরোনো প্যাকেজ — এগুলো সুবিধা দেয়, কিন্তু আক্রমণকারীকে সুযোগও দেয়। আক্রমণকারী আপনার অ্যাপ্লিকেশনের দুর্বলতা খুঁজে না পেলেও সার্ভারের দুর্বলতা খুঁজে পেতে পারে: একটি পুরোনো SSH সংস্করণ, একটি খোলা ডাটাবেস পোর্ট, একটি ভুল অনুমতির ফাইল। তাই Server Hardening মানে সার্ভারকে শুধু যা প্রয়োজন, ততটুকুই করতে দেওয়া — বাড়তি সবকিছু বন্ধ করা, কমানো, বা সীমিত করা।

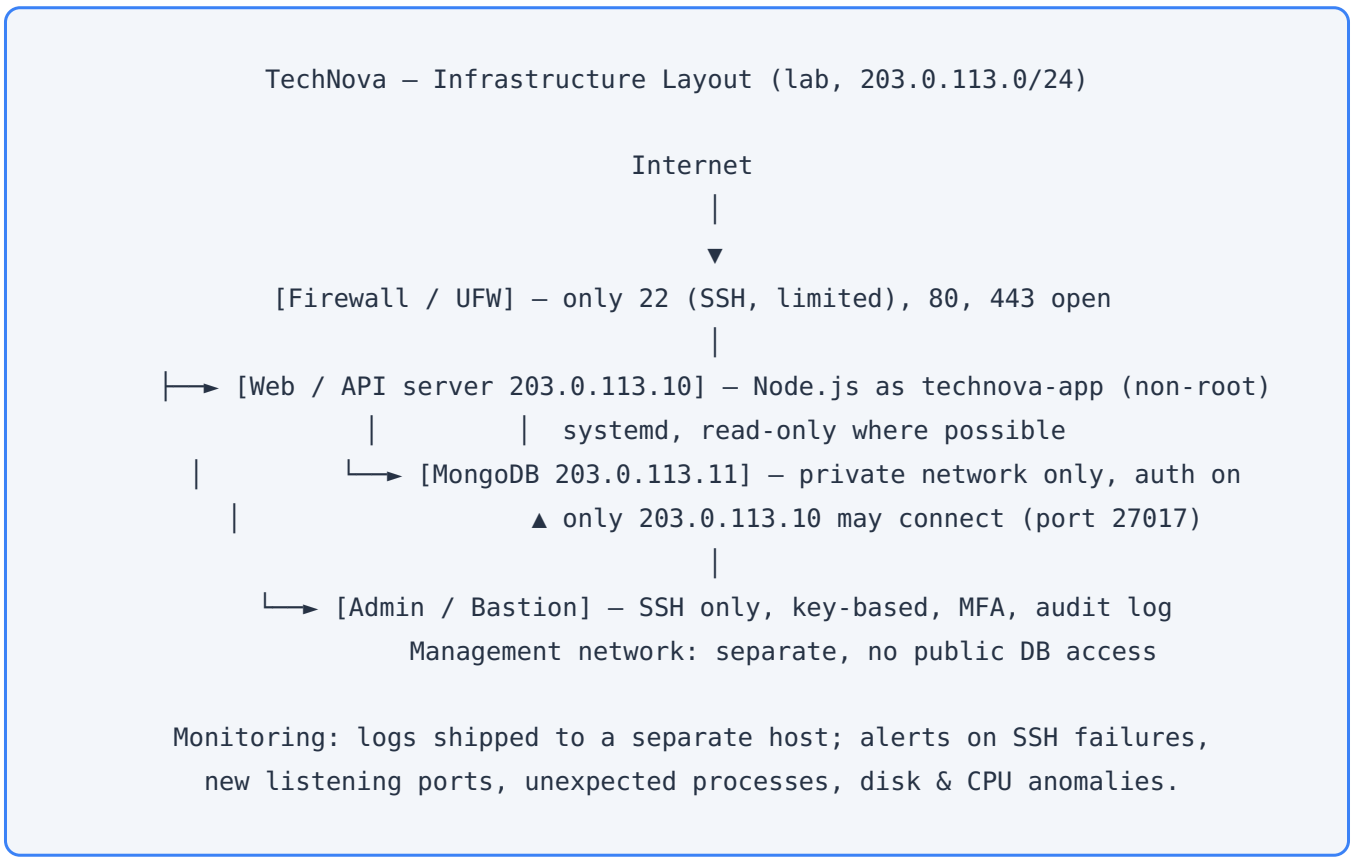
এই নীতির নাম Least Privilege — প্রতিটি ব্যবহারকারী, প্রক্রিয়া ও সার্ভিস শুধু তার কাজের জন্য যতটুকু প্রয়োজন, ততটুকু ক্ষমতা পাবে, তার বেশি নয়। একটি উদাহরণ: Node.js অ্যাপ্লিকেশনটি যদি শুধু নিজের ডাটাবেসে পড়া-লেখা করে, তাহলে তাকে root হিসেবে চালানোর কোনো কারণ নেই। একটি আলাদা, সীমিত ব্যবহারকারী (যেমন technova-app) হিসেবে

চালালে, অ্যাপ্লিকেশন আপস হলেও আক্রমণকারী পুরো সার্ভারের নিয়ন্ত্রণ পাবে না — শুধু সেই ব্যবহারকারীর ক্ষমতা পাবে। একইভাবে, MongoDB শুধু API সার্ভার থেকে সংযোগ নেবে, ইন্টারনেট থেকে নয়। এই সীমাবদ্ধতাগুলো একসাথে মিলে Blast Radius ছোট করে — অর্থাৎ একটি জায়গা ভাঙলে ক্ষতি কতদূর ছড়াবে, তা সীমিত রাখে।

> **CONCEPT:** Network Segmentation মানে নেটওয়ার্ককে ভাগ করে দেওয়া, যাতে একটি অংশ আপস হলে অন্য অংশে সহজে যাওয়া না যায়। যেমন, ওয়েব সার্ভার একটি অংশে, ডাটাবেস আরেকটি অংশে; ডাটাবেসের অংশে শুধু ওয়েব সার্ভার পৌঁছাতে পারে, ইন্টারনেট পারে না। দেয়াল যত বেশি, আক্রমণকারীর পথ তত কঠিন।

Process Isolation-এর ধারণাও একই নীতির সম্প্রসারণ। প্রতিটি সার্ভিস আলাদা প্রক্রিয়ায়, আলাদা ব্যবহারকারীতে, সম্ভব হলে আলাদা কনটেইনারে চলে — যাতে একটি প্রক্রিয়ার ভুল বা আপস অন্যটিকে স্পর্শ না করে। Docker-এ এটি আরও স্পষ্ট: প্রতিটি কনটেইনার নিজের ফাইলসিস্টেম, নেটওয়ার্ক ও প্রক্রিয়া জায়গা পায়। কিন্তু মনে রাখবেন, কনটেইনার কোনো যাদুর দেয়াল নয়; হোস্ট কার্নেল একটাই, এবং ভুল কনফিগারেশন — যেমন কনটেইনারকে root হিসেবে চালানো বা হোস্টের Docker সকেট দেওয়া — সেই দেয়াল ভেঙে দিতে পারে। Isolation একটি নকশার সিদ্ধান্ত, শুধু একটি টুলের বৈশিষ্ট্য নয়।

৩.৪ আর্কিটেকচার: TechNova-র বিভক্ত অবকাঠামো



স্তর	নিয়ন্ত্রণ	কেন
নেটওয়ার্ক	UFW, শুধু প্রয়োজনীয় পোর্ট	Attack Surface ছোট করা
হোস্ট	আপডেট, অপ্রয়োজনীয় সার্ভিস বন্ধ	পরিচিত দুর্বলতা কমানো
পরিচয়	SSH key, root নিষেধ, sudo লগ	অনুমোদনহীন প্রবেশ ঠেকানো

প্রক্রিয়া	Non-root সার্ভিস, systemd সীমা	Blast Radius সীমিত করা
তথ্য	ফাইল অনুমতি, এনক্রিপশন, ব্যাকআপ	তথ্য ফাঁস ও হারানো ঠেকানো
পর্যবেক্ষণ	লগ, অ্যালার্ট, অডিট	আক্রমণ দ্রুত ধরা

৩.৫ অভ্যন্তরীণ কার্যপ্রণালী: SSH, অনুমতি ও ফায়ারওয়াল কীভাবে কাজ করে

SSH (Secure Shell) কীভাবে কাজ করে, তা বুঝলে Hardening-এর কারণ বোঝা যায়। SSH-তে ক্লায়েন্ট ও সার্ভার প্রথমে এনক্রিপ্টেড চ্যানেল তৈরি করে, তারপর পরিচয় যাচাই করে। পাসওয়ার্ড-ভিত্তিক লগইনে আক্রমণকারী বারবার অনুমান করতে পারে (Brute Force); কী-ভিত্তিক লগইনে ব্যক্তিগত কী ছাড়া ঢোকা প্রায় অসম্ভব। তাই প্রথম Hardening হলো পাসওয়ার্ড লগইন বন্ধ করে শুধু কী অনুমোদন করা, root-এর সরাসরি লগইন নিষেধ করা, এবং ব্যর্থ চেষ্টার পর সাময়িক নিষেধাজ্ঞা দেওয়া। SSH কনফিগারেশন ফাইল `/etc/ssh/sshd_config`-এ এই সিদ্ধান্তগুলো লেখা থাকে; ফাইলটি বদলে সার্ভিস পুনরায় চালু করতে হয়, এবং বদলের আগে বর্তমান সেশন খোলা রাখতে হয় — যাতে ভুল হলে নিজেকে বাইরে আটকে না ফেলেন।

ফাইলসিস্টেম অনুমতি ভেতরে ভেতরে সংখ্যা দিয়ে কাজ করে: r=4, w=2, x=1। `chmod 640` মানে মালিক পড়া+লেখা (6), গ্রুপ শুধু পড়া (4), অন্যরা কিছুই নয় (0)। কনফিগারেশন ফাইল ও সিক্রেটের জন্য 600 বা 640 উপযুক্ত; সবার পড়া যায় এমন ফাইলের জন্য 644। ডিরেক্টরিতে x মানে ভেতরে ঢোকা যায়। সবচেয়ে বিপজ্জনক অনুমতিগুলো হলো 777 (সবাই সবকিছু করতে পারে) এবং SUID বিট (ফাইলটি মালিকের ক্ষমতায় চলে)। একটি SUID প্রোগ্রামে দুর্বলতা থাকলে সাধারণ ব্যবহারকারী root-এর ক্ষমতা পেতে পারেন — তাই SUID ফাইলগুলো নিয়মিত তালিকা করে পরীক্ষা করা উচিত। `find / -perm -4000` কমান্ডটি এই তালিকা দেয়; তালিকায় অপরিচিত কিছু থাকলে তা তদন্ত করুন।

ফায়ারওয়াল ভেতরে ভেতরে নিয়মের তালিকা মিলিয়ে কাজ করে: প্যাকেট এলে প্রথম মিলে যাওয়া নিয়ম অনুযায়ী অনুমোদন বা প্রত্যাখ্যান হয়। UFW (Uncomplicated Firewall) হলো Ubuntu-তে iptables-এর সহজ আবরণ। নীতি হওয়া উচিত Default Deny — অর্থাৎ যা স্পষ্টভাবে অনুমোদিত নয়, তা বন্ধ। শুধু 22, 80, 443 খুলুন; MongoDB-র 27017 কখনো ইন্টারনেটে খুলবেন না। আরেকটি গুরুত্বপূর্ণ কথা: ফায়ারওয়াল নিয়ম লিখলেই হবে না, তা পরীক্ষা করতে হবে — বাইরে থেকে পোর্ট স্ক্যান করে দেখুন আসলেই বন্ধ কি না (শুধু নিজের ল্যাব সার্ভারে, অনুমোদিতভাবে)।

> **WARNING:** SSH কনফিগারেশন বদলানোর আগে একটি সক্রিয় SSH সেশন খোলা রাখুন এবং নতুন সেশনে পরীক্ষা করে দেখুন লগইন হয় কি না। ভুল কনফিগারেশনে নিজেকে বাইরে আটকে ফেলা নতুনদের সবচেয়ে সাধারণ ও কষ্টকর ভুল। ল্যাবে হলেও এই অভ্যাসটি গড়ে তুলুন।

৩.৬ বাস্তব কমান্ড ও কনফিগারেশন

নিচের সব কাজ শুধু আপনার নিজের ল্যাব সার্ভারে (Ubuntu 24.04 LTS, 203.0.113.10) করবেন। প্রতিটি ধাপের পর যাচাই করুন।

```
# 1) সিস্টেম আপডেট ও অপ্রয়োজনীয় সার্ভিস দেখা
sudo apt update && sudo apt upgrade -y
sudo systemctl list-unit-files --type=service --state=enabled | head -n 40
sudo ss -tlnp

# 2) UFW — Default Deny, শুধু প্রয়োজনীয় পোর্ট
sudo ufw default deny incoming
sudo ufw default allow outgoing
```

```
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status verbose

# 3) SSH Hardening — /etc/ssh/sshd_config (ব্যাকআপ নিয়ে সম্পাদনা করুন)
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak
# ফাইলে নিশ্চিত করুন:
#   PermitRootLogin no
#   PasswordAuthentication no
#   PubkeyAuthentication yes
#   MaxAuthTries 3
#   AllowUsers technova-admin
sudo sshd -t && sudo systemctl reload sshd
# নতুন টার্মিনালে লগইন পরীক্ষা করুন, পুরোনো সেশন বন্ধ করবেন না।

# 4) ফাইল অনুমতি যাচাই
ls -l /etc/ssh/sshd_config
sudo chmod 600 /etc/ssh/sshd_config
ls -ld /home/technova-app
sudo find / -perm -4000 -type f 2>/dev/null | head -n 20

# 5) Non-root সার্ভিস ব্যবহারকারী ও systemd সীমা (ধারণা)
sudo useradd --system --no-create-home --shell /usr/sbin/nologin technova-app
# systemd unit-এ: User=technova-app, NoNewPrivileges=true,
# PrivateTmp=true — বিস্তারিত আপনার দলের অনুমোদিত unit ফাইলে দেখুন।
```

MongoDB-র জন্য ন্যূনতম নিরাপত্তা: `bindIp` শুধু private আইপি বা localhost, Authorization চালু, এবং শক্তিশালী পাসওয়ার্ড বা কী ফাইল। `/etc/mongod.conf`-এ `bindIp: 127.0.0.1,203.0.113.11` ও `security.authorization: enabled` যাচাই করুন। মনে রাখবেন, Authentication ছাড়া MongoDB ইন্টারনেটে খোলা থাকা বহু তথ্য ফাঁসের কারণ হয়েছে।

৩.৭ প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — MongoDB ইন্টারনেটে খোলা। TechNova-র এক ডেভেলপার পরীক্ষার জন্য MongoDB-র পোর্ট খুলে দিয়েছিলেন, ভুলে বন্ধ করেননি। কয়েকদিন পর লগে দেখা গেল অপরিচিত আইপি থেকে সংযোগের চেষ্টা। সমাধান: ফায়ারওয়ালে 27017 বন্ধ করা, `bindIp` ঠিক করা, Authorization চালু করা, পাসওয়ার্ড ঘোরানো, এবং লগ পরীক্ষা করে দেখা কোনো তথ্য পড়া বা বদলানো হয়েছে কি না। শিক্ষা: অস্থায়ী খোলা পোর্টেরও মালিক ও মেয়াদ থাকতে হবে; Hardening একটি একবারের কাজ নয়, নিয়মিত যাচাইয়ের কাজ।

পরিস্থিতি ২ — একটি সার্ভিস আপস হলো, কিন্তু ক্ষতি সীমিত। Node.js অ্যাপে একটি দুর্বলতা দিয়ে আক্রমণকারী শেল পেলেন, কিন্তু অ্যাপটি চলছিল non-root ব্যবহারকারী হিসেবে, systemd-র সীমা ছিল, এবং ডাটাবেসের পরিচয় শুধু পড়া-লেখার মধ্যে সীমিত ছিল। ফলে আক্রমণকারী পুরো সার্ভার দখল করতে পারলেন না; মনিটরিং অস্বাভাবিক প্রক্রিয়া ধরে ফেলল। এই ঘটনাই Least Privilege ও Isolation-এর মূল্য প্রমাণ করে — প্রতিরোধ ব্যর্থ হলেও ক্ষতি সীমিত থাকে।

পরিস্থিতি ৩ — SSH Brute Force। লগে দেখা গেল প্রতি মিনিটে শত শত ব্যর্থ SSH চেষ্টা। পাসওয়ার্ড লগইন বন্ধ থাকায় কোনো চেষ্টাই সফল হয়নি, কিন্তু লগ ভরে যাচ্ছিল ও সার্ভারের মনোযোগ নষ্ট হচ্ছিল। সমাধান: fail2ban-এর মতো ব্যবস্থায় বারবার ব্যর্থ আইপি সাময়িকভাবে বন্ধ করা, SSH পোর্টে উৎস সীমিত করা (যেখানে সম্ভব), এবং অ্যালার্টের সীমা ঠিক করা যাতে সত্যিকারের ঘটনা হারিয়ে না যায়।

> **REAL WORLD:** বাস্তব আক্রমণে আক্রমণকারী প্রথমে সহজ জিনিস খোঁজে — খোলা পোর্ট, ডিফল্ট পাসওয়ার্ড, পুরোনো সফটওয়্যার। জটিল শূন্য-দিন (zero-day) আক্রমণ বিরল; অবহেলিত মৌলিক বিষয়ই সবচেয়ে সাধারণ প্রবেশপথ। Hardening-এর ৮০% মূল্য আসে ২০% মৌলিক কাজ থেকে।

৩.৮ নিরাপত্তা ও খরচ বিবেচনা

Hardening-এর খরচ মূলত সময় ও মনোযোগ: আপডেট, কনফিগারেশন পর্যালোচনা, লগ দেখা। এর বিনিময়ে আপনি পান ছোট Attack Surface ও সীমিত Blast Radius। কিন্তু অতিরিক্ত Hardening-এরও খরচ আছে: খুব কঠোর নিয়ম বৈধ কাজ আটকে দিলে মানুষ নিয়ম এড়াতে শিখবে। যেমন, SSH-তে খুব কম সময়ে সেশন বন্ধ হলে ডেভেলপাররা বিরক্ত হয়ে অনিরাপদ উপায় খুঁজবেন। তাই ভারসাম্য রাখুন: সংবেদনশীল জায়গায় কঠোর, সাধারণ জায়গায় ব্যবহারযোগ্য। আরেকটি খরচের দিক: মনিটরিং-এর জন্য লগ সংরক্ষণে জায়গা ও খরচ লাগে; কী লগ রাখবেন, কতদিন রাখবেন, তা আগেই ঠিক করুন। সব লগ রাখা যেমন ব্যয়বহুল, তেমনি প্রয়োজনীয় লগ না রাখা ঘটনা তদন্তে অন্ধ করে দেয়।

Backup বনাম Disaster Recovery-এর পার্থক্য এখানে মনে রাখুন। Backup মানে তথ্যের অনুলিপি রাখা; Disaster Recovery মানে বিপর্যয়ের পর সেবা আবার চালু করার পুরো পরিকল্পনা — কে কী করবে, কতক্ষণে, কোন ক্রমে। ব্যাকআপ থাকলেই সেবা ফিরবে না, যদি তা পরীক্ষা না করা হয়। নিয়মিত Restore পরীক্ষা করুন: ব্যাকআপ থেকে ল্যাবে সেবা ফিরিয়ে আনুন, সময় মাপুন, এবং সমস্যা লিখুন। যে ব্যাকআপ পরীক্ষা করা হয়নি, তার ওপর ভরসা করবেন না।

৩.৯ সাধারণ ভুল

- **Root হিসেবে অ্যাপ চালানো:** সুবিধার জন্য root ব্যবহার করা সবচেয়ে বিপজ্জনক অভ্যাস।
- **ডাটাবেস পোর্ট ইন্টারনেটে খোলা:** MongoDB, Redis, PostgreSQL — কোনো ডাটাবেসই সরাসরি ইন্টারনেটে নয়।
- **ডিফল্ট পাসওয়ার্ড রাখা:** ডিফল্ট পরিচয়পত্র আক্রমণকারীর প্রথম চেষ্টা।
- **আপডেট না করা:** পরিচিত দুর্বলতা ঠিক করা সবচেয়ে সহজ নিরাপত্তা, তবু সবচেয়ে অবহেলিত।
- **ফায়ারওয়াল লিখে পরীক্ষা না করা:** নিয়ম আছে কিন্তু কাজ করছে কি না, তা না দেখা।
- **লগ সংগ্রহ করে না দেখা:** লগ শুধু জমা হলে তা প্রমাণ নয়, তা বোঝা।
- **777 অনুমতি দেওয়া:** “কাজ করছে না, তাই 777 দিলাম” — এটি সমস্যা লুকায়, ঠিক করে না।

৩.১০ ট্রাবলশুটিং

সমস্যা: SSH-এ লগইন হচ্ছে না। প্রথমে দেখুন সার্ভিস চলছে কি না (`systemctl status sshd`), কনফিগারেশন ঠিক কি না (`sshd -t`), এবং কী ফাইলের অনুমতি ঠিক কি না (ব্যক্তিগত কী 600, `.ssh` ডিরেক্টরি 700, `authorized_keys` 600)। লগ দেখুন: `sudo journalctl -u sshd --since "1 hour ago"`। ভুল অনুমতির কারণে কী প্রত্যাখ্যাত হওয়া খুব সাধারণ।

সমস্যা: অ্যাপ ডাটাবেসে সংযোগ পাচ্ছে না। ধাপে ধাপে যাচাই করুন: ডাটাবেস চলছে কি না, `bindIp` ঠিক কি না, ফায়ারওয়াল পোর্ট খোলা কি না, পরিচয়পত্র ঠিক কি না। নেটওয়ার্ক সমস্যা না পরিচয় সমস্যা, তা আলাদা করুন — লগের বার্তা পড়ুন, অনুমান করবেন না।

সমস্যা: ডিস্ক ভরে যাচ্ছে। লগ ঘূর্ণন (log rotation) যাচাই করুন, বড় ফাইল খুঁজুন (`du -sh /* | sort -h`), এবং অপ্রয়োজনীয় ফাইল মুছুন। ডিস্ক ভরে গেলে লগ লেখা বন্ধ হয় — আর তখনই আক্রমণ ধরা পড়ে না। ডিস্ক ব্যবহারে অ্যালার্ট সেট করুন।

TROUBLESHOOTING: কোনো পরিবর্তনের পর সেবা না চললে প্রথমে আগের অবস্থায় ফিরে যান, তারপর কারণ খুঁজুন। কনফিগারেশন বদলের আগে ব্যাকআপ নেওয়া এই কারণেই অপরিহার্য। "আগে কাজ করছিল" — এই তথ্যটিই সবচেয়ে মূল্যবান সূত্র।

৩.১১ সারসংক্ষেপ

অবকাঠামো নিরাপত্তা মানে সার্ভারকে শুধু প্রয়োজনীয় কাজটুকুই করতে দেওয়া, প্রতিটি স্তরে সীমা দেওয়া, এবং অস্বাভাবিক কিছু ঘটলে তা দেখতে পাওয়া। Least Privilege, Network Segmentation, Process Isolation ও Default Deny — এই চারটি নীতি একসাথে Attack Surface ছোট করে ও Blast Radius সীমিত করে। SSH-তে কী-ভিত্তিক লগইন, ফায়ারওয়ালে শুধু প্রয়োজনীয় পোর্ট, ফাইলে সঠিক অনুমতি, সার্ভিসে non-root ব্যবহারকারী, এবং নিয়মিত আপডেট ও লগ পর্যালোচনা — এই মৌলিক কাজগুলোই সবচেয়ে বড় সুরক্ষা দেয়। ব্যাকআপ থাকা মানেই Disaster Recovery নয়; Restore পরীক্ষা করেই তা প্রমাণ করতে হয়।

৩.১২ পর্যালোচনা প্রশ্ন

1. Least Privilege কাকে বলে? Node.js অ্যাপ root হিসেবে চালালে কী ঝুঁকি হয়, তা ব্যাখ্যা করুন।
2. Network Segmentation কীভাবে Blast Radius কমায়? TechNova-র উদাহরণ দিন।
3. SSH-তে পাসওয়ার্ডের চেয়ে কী-ভিত্তিক লগইন কেন নিরাপদ? তিনটি কারণ লিখুন।
4. chmod 640-এর অর্থ কী? একটি সিক্রেট ফাইলের জন্য কোন অনুমতি উপযুক্ত এবং কেন?
5. SUID কী এবং কেন তা নিয়মিত পরীক্ষা করা উচিত?
6. Default Deny নীতি কী? UFW-তে এটি কীভাবে প্রয়োগ করবেন?
7. MongoDB-র পোর্ট ইন্টারনেটে খোলা কেন বিপজ্জনক? সঠিক কনফিগারেশন কী হবে?
8. Process Isolation কাকে বলে? Docker কনটেইনার কি সম্পূর্ণ বিচ্ছিন্নতা দেয়? ব্যাখ্যা করুন।
9. Backup ও Disaster Recovery-এর পার্থক্য কী? Restore পরীক্ষা কেন জরুরি?
10. Host Monitoring-এ কোন পাঁচটি অস্বাভাবিকতা দেখলে আপনি তদন্ত শুরু করবেন?

৩.১৩ ব্যবহারিক অনুশীলন

1. **Hardening চেকলিস্ট:** আপনার ল্যাব Ubuntu সার্ভারে UFW অবস্থা, শোনা পোর্ট, SSH কনফিগারেশন ও SUID ফাইলের তালিকা তৈরি করুন। প্রতিটি খুঁজে পাওয়া বিষয়ে ঝুঁকি ও করণীয় লিখুন।
2. **Non-root সার্ভিস:** ল্যাবে একটি সাধারণ Node.js অ্যাপ non-root ব্যবহারকারী হিসেবে systemd দিয়ে চালান। root হিসেবে চালানোর চেয়ে কী কী সীমা তৈরি হলো, তা লিখুন।
3. **লগ তদন্ত:** ল্যাব সার্ভারে SSH লগে ব্যর্থ চেষ্টাগুলো দেখুন (`journalctl -u sshd`)। কয়টি ব্যর্থ চেষ্টা, কোন আইপি থেকে, কী প্যাটার্ন — একটি সংক্ষিপ্ত প্রতিবেদন লিখুন ও প্রতিরোধের প্রস্তাব দিন।

অধ্যায় ৪ • PART I — Security Engineering Foundations

Chapter 4 — Identity and Access Security

৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Identity and Access Management (IAM)-এর মূল ধারণাগুলো ব্যাখ্যা করতে পারবেন, Authentication ও Authorization-এর পার্থক্য স্পষ্ট করতে পারবেন, এবং RBAC ও ABAC-এর মধ্যে কোনটি কখন উপযুক্ত তা বিচার করতে পারবেন। আপনি Least Privilege, Multi-Factor Authentication (MFA), Zero Trust, Service Identity, Identity Federation, Workload Identity ও Access Review-এর প্রতিটি ধারণা TechNova-র প্রেক্ষাপটে প্রয়োগ করতে পারবেন। সবশেষে, আপনি একটি ছোট দলের জন্য একটি বাস্তবসম্মত অ্যাক্সেস নীতি লিখতে পারবেন — কে কী করতে পারে, কতদিনের জন্য, এবং কে তা পর্যালোচনা করবে।

- Authentication (পরিচয় যাচাই) ও Authorization (অনুমতি) আলাদা করে ব্যাখ্যা করা।
- RBAC ও ABAC-এর তুলনা করে উপযুক্ত মডেল বেছে নেওয়া।
- Least Privilege ও Zero Trust নীতি প্রয়োগ করা।
- MFA, Service Identity ও Workload Identity-এর ব্যবহার ব্যাখ্যা করা।
- Identity Federation ও নিয়মিত Access Review-এর প্রক্রিয়া নকশা করা।

৪.২ পূর্বশর্ত ঝালাই

আগের অধ্যায় থেকে Least Privilege ও Trust Boundary মনে রাখুন। এই অধ্যায়ে পরিচয়ের ভাষা ঝালিয়ে নিন। Identity (পরিচয়) মানে কে বা কী — একজন ব্যবহারকারী, একটি সার্ভিস, একটি ডিভাইস, একটি প্রক্রিয়া। Credential (পরিচয়পত্র) মানে সেই পরিচয়ের প্রমাণ — পাসওয়ার্ড, কী, টোকেন, সার্টিফিকেট। Principal মানে যার পক্ষে কাজ করা হচ্ছে — ব্যবহারকারী বা সার্ভিস। Session মানে লগইনের পর একটি নির্দিষ্ট সময়ের জন্য দেওয়া বিশ্বাসের অবস্থা। Token মানে সেই বিশ্বাসের বহনযোগ্য প্রমাণ — যেমন JWT (JSON Web Token)। মনে রাখবেন, পরিচয় যাচাই একবার হলেই সব কাজের অনুমতি মেলে না; প্রতিটি কাজের জন্য আলাদাভাবে অনুমতি পরীক্ষা করতে হয়। এই পার্থক্যটিই এই অধ্যায়ের কেন্দ্রবিন্দু।

৪.৩ প্রথম নীতি থেকে পরিচয়: কেন “কে তুমি” আর “তুমি কী করতে পারো” আলাদা

পরিচয় নিরাপত্তার প্রথম নীতি হলো: Authentication মানে “তুমি কে?”, Authorization মানে “তুমি কী করতে পারো?”। একটি বাড়ির উদাহরণ: দরজায় তালা খোলা মানে আপনি বাড়িতে ঢুকতে পারবেন (Authentication), কিন্তু তার মানে এই নয় যে আপনি সিন্দুক খুলতে পারবেন (Authorization)। সফটওয়্যারে এই দুটি গুলিয়ে ফেললে বিপদ হয়। যেমন, লগইন সফল হলেই যদি সব API রুট খুলে যায়, তাহলে সাধারণ ব্যবহারকারীও অ্যাডমিনের কাজ করতে পারবেন। সঠিক নকশায় প্রতিটি অনুরোধে দুটি প্রশ্ন করা হয়: প্রথমে, এই টোকেনটি কি আসল ও মেয়াদোত্তীর্ণ নয়? দ্বিতীয়ত, এই পরিচয়ের কি এই নির্দিষ্ট কাজটি করার অনুমতি আছে? দ্বিতীয় প্রশ্নের উত্তর সার্ভারে, প্রতিটি অনুরোধে, ব্যবসায়িক যুক্তির সাথে মিলিয়ে দিতে হয় — ফ্রন্টএন্ডে লুকিয়ে রাখা বোতাম অনুমতি নয়।

সমস্যাটি কেন ঘটে? কারণ পরিচয় ব্যবস্থা শুরুতে সরল থাকে — একটি অ্যাডমিন, কিছু ব্যবহারকারী — কিন্তু ব্যবসা বাড়লে ভূমিকা বাড়ে, ব্যতিক্রম বাড়ে, আর পুরোনো অনুমতি রয়ে যায়। কেউ দল বদলালে তার পুরোনো অ্যাক্সেস থেকে যায়; কেউ চাকরি ছাড়লে তার অ্যাকাউন্ট বন্ধ হতে দেরি হয়; একটি সার্ভিসের জন্য তৈরি কী বছরের পর বছর ঘোরানো হয় না। এই জমে থাকা অনুমতিকে বলে Privilege Creep — ধীরে ধীরে ক্ষমতা বাড়তে থাকা। এর প্রতিকার হলো নিয়মিত Access Review:

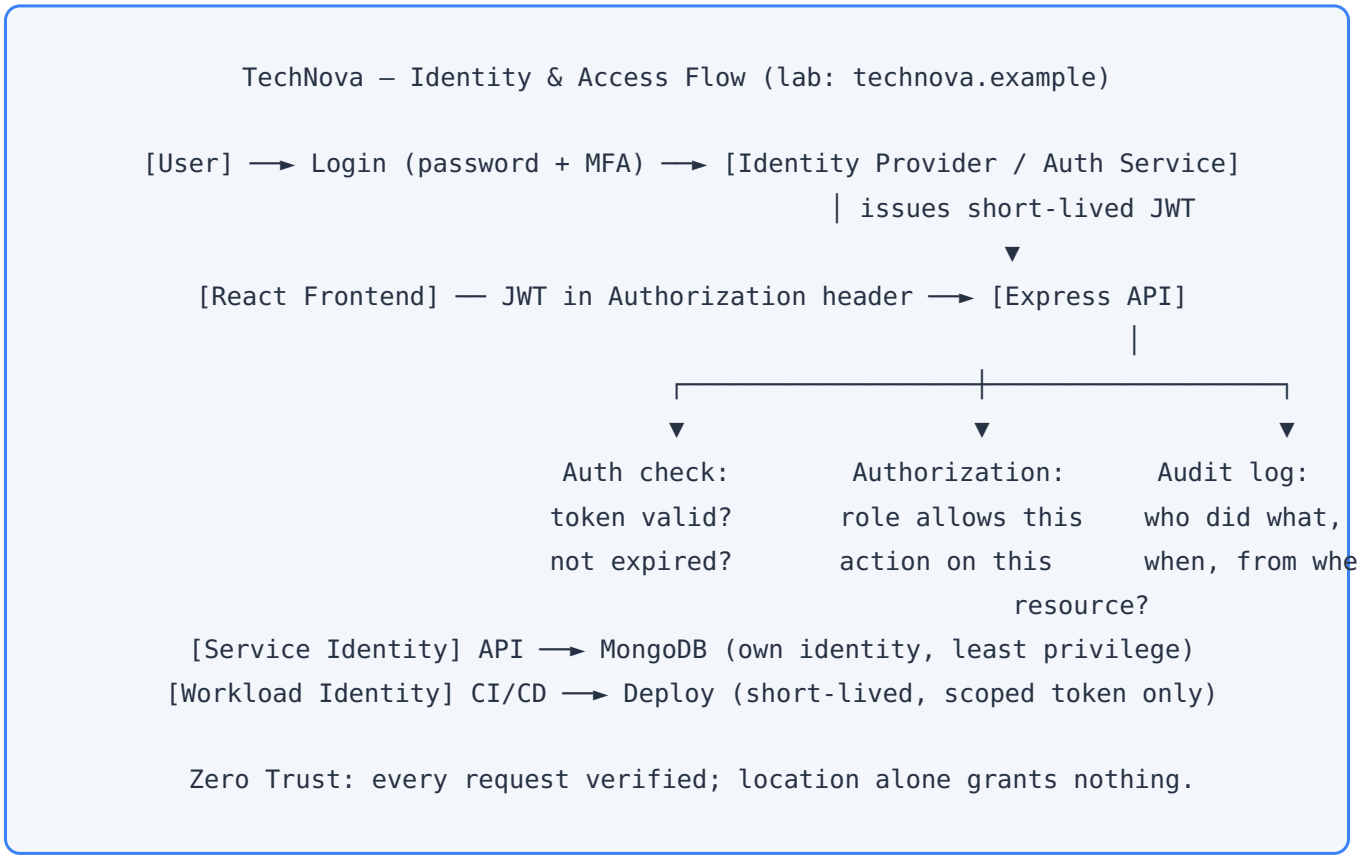
কে কী অ্যাক্সেস রাখে, তা কি এখনো প্রয়োজন, তা পর্যায়ক্রমে পরীক্ষা করা। Access Review কোনো আনুষ্ঠানিকতা নয়; এটি Privilege Creep-এর একমাত্র নিয়মিত প্রতিষেধক।

> **CONCEPT:** Zero Trust মানে “কখনো বিশ্বাস কোরো না, সবসময় যাচাই করো” (Never trust, always verify)। পুরোনো ধারণা ছিল, নেটওয়ার্কের ভেতরে সবাই বিশ্বস্ত; Zero Trust বলে, অবস্থান বিশ্বাসের প্রমাণ নয় — ভেতরে থাকলেও প্রতিটি অনুরোধে পরিচয়, ডিভাইস, প্রেক্ষাপট ও অনুমতি যাচাই করতে হবে। এটি একটি পণ্য নয়, একটি নকশার নীতি।

RBAC (Role-Based Access Control) মানে ভূমিকার ভিত্তিতে অনুমতি: ব্যবহারকারীকে একটি ভূমিকা দেওয়া হয় (যেমন admin, editor, viewer), আর ভূমিকাটির অনুমতি আগে থেকে ঠিক করা থাকে। এটি সরল, বোঝা সহজ, এবং ছোট থেকে মাঝারি দলের জন্য যথেষ্ট। ABAC (Attribute-Based Access Control) মানে বৈশিষ্ট্যের ভিত্তিতে অনুমতি: ব্যবহারকারীর বৈশিষ্ট্য (বিভাগ, অবস্থান), সম্পদের বৈশিষ্ট্য (সংবেদনশীলতা, মালিক), পরিবেশের বৈশিষ্ট্য (সময়, অবস্থান) — সব মিলিয়ে সিদ্ধান্ত। ABAC বেশি নমনীয়, কিন্তু বেশি জটিল ও ভুল করা সহজ। TechNova-র মতো ছোট দলের জন্য পরামর্শ: RBAC দিয়ে শুরু করুন, ব্যতিক্রমগুলো লিখুন; যখন RBAC-এ ভূমিকা অসংখ্য হয়ে যাবে ও ব্যতিক্রম নিয়ম হয়ে যাবে, তখন ABAC-এর কথা ভাবুন। জটিল মডেল আগে নেওয়া মানে বোঝা কঠিন এমন ব্যবস্থা তৈরি করা, যা ভুলভাবে কনফিগার হওয়ার সম্ভাবনা বেশি।

Service Identity ও Workload Identity হলো মানুষ নয়, এমন পরিচয়। আপনার Node.js অ্যাপ যখন MongoDB-তে সংযোগ করে, তখন সে কোন পরিচয়ে করে? যদি সবার জন্য একই পাসওয়ার্ড ব্যবহার হয়, তাহলে কোন সার্ভিস কী করেছে, তা বোঝা যায় না; আর একটি সার্ভিস আপস হলে সব সার্ভিসের পরিচয় ফাঁস হয়। সঠিক নকশায় প্রতিটি সার্ভিসের আলাদা পরিচয় থাকে, শুধু তার প্রয়োজনীয় অনুমতি থাকে, এবং সেই পরিচয়ের মেয়াদ সীমিত থাকে। Identity Federation মানে একটি বিশ্বস্ত পরিচয় প্রদানকারী (Identity Provider)-র ওপর ভরসা করা — যেমন, কোম্পানির কেন্দ্রীয় লগইন দিয়ে বিভিন্ন সেবায় ঢোকা, প্রতিটি সেবায় আলাদা পাসওয়ার্ড না রেখে। এর সুবিধা: এক জায়গায় MFA, এক জায়গায় অ্যাকাউন্ট বন্ধ করা। ঝুঁকি: সেই কেন্দ্রীয় জায়গাটি আপস হলে অনেক সেবা একসাথে ঝুঁকিতে পড়ে — তাই সেটিকে সবচেয়ে শক্তভাবে সুরক্ষিত করতে হয়।

৪.৪ আর্কিটেকচার: TechNova-র পরিচয় প্রবাহ



পরিচয়	প্রমাণ	অনুমতি	মেয়াদ
ব্যবহারকারী	পাসওয়ার্ড + MFA	ভূমিকা অনুযায়ী	সেশন সীমিত, টোকেন স্বল্পমেয়াদি
অ্যাডমিন	পাসওয়ার্ড + MFA (বাধ্যতামূলক)	অ্যাডমিন কাজ, অডিটসহ	স্বল্পমেয়াদি, পুনরায় যাচাই
API সার্ভিস	সার্ভিস পরিচয় / সিক্রেট	শুধু নিজের ডাটাবেস কাজ	ঘোরানো হয় নিয়মিত
CI/CD	Workload Identity, স্বল্পমেয়াদি টোকেন	শুধু ডিপ্লয়	প্রতি রানে নতুন

এই চিত্রের মূল শিক্ষা: টোকেন যত স্বল্পমেয়াদি, ফাঁস হলে ক্ষতি তত কম। দীর্ঘমেয়াদি টোকেন সুবিধাজনক, কিন্তু চুরি হলে দীর্ঘদিন ব্যবহার করা যায়। তাই Access Token স্বল্পমেয়াদি রাখুন, Refresh Token নিরাপদে সংরক্ষণ করুন ও প্রয়োজনে বাতিল (revoke) করার ব্যবস্থা রাখুন। লগআউট মানে শুধু ফ্রন্টএন্ড থেকে টোকেন মোছা নয়; সার্ভারে সেশন বাতিলের ব্যবস্থাও থাকতে হবে, নইলে চুরি যাওয়া টোকেন ব্যবহার করা যেতেই থাকবে।

৪.৫ অভ্যন্তরীণ কার্যপ্রণালী: টোকেন, MFA ও অনুমতি পরীক্ষা

JWT ভেতরে ভেতরে তিনটি অংশে গঠিত: Header, Payload ও Signature। Payload-এ পরিচয়, ভূমিকা ও মেয়াদের তথ্য থাকে; Signature দিয়ে যাচাই করা হয় যে তথ্য বদলানো হয়নি। গুরুত্বপূর্ণ কথা: JWT-র Payload এনক্রিপ্টেড নয়, শুধু এনকোড করা — যে কেউ তা পড়তে পারে। তাই JWT-তে সংবেদনশীল তথ্য (পাসওয়ার্ড, ব্যক্তিগত তথ্য) রাখবেন না। আরেকটি গুরুত্বপূর্ণ কথা: JWT যাচাই মানে স্বাক্ষর যাচাই ও মেয়াদ যাচাই; শুধু Payload পড়ে বিশ্বাস করা যাবে না। সার্ভারকে প্রতিটি অনুরোধে স্বাক্ষর যাচাই করতে হবে, এবং স্বাক্ষরের গোপন কী (signing secret) সুরক্ষিত রাখতে হবে — এটি ফাঁস হলে যে কেউ নিজের টোকেন বানাতে পারবে।

MFA (Multi-Factor Authentication) ভেতরে ভেতরে ভিন্ন ধরনের প্রমাণ যোগ করে: যা আপনি জানেন (পাসওয়ার্ড), যা আপনার কাছে আছে (ফোন, হার্ডওয়্যার কী), যা আপনি (আঙুলের ছাপ, মুখ)। একটি উপাদান ফাঁস হলেও অন্যটি ছাড়া ঢোকা যায় না। SMS-ভিত্তিক কোডের চেয়ে Authenticator অ্যাপ বা হার্ডওয়্যার কী বেশি নিরাপদ, কারণ SMS আটকানো বা SIM বদলে নেওয়া সম্ভব। অ্যাডমিন ও সংবেদনশীল কাজের জন্য MFA বাধ্যতামূলক করুন; সাধারণ ব্যবহারকারীর জন্য MFA উৎসাহিত করুন ও সহজ করুন। MFA চালু করা কঠিন হলে মানুষ তা এড়াবে — তাই নথি, সাহায্য ও পুনরুদ্ধারের পথ (recovery) স্পষ্ট রাখুন।

অনুমতি পরীক্ষার ভেতরের নিয়ম: পরীক্ষা হবে সার্ভারে, প্রতিটি অনুরোধে, সম্পদ ও কাজের ভিত্তিতে। একটি সাধারণ ভুল হলো তালিকা দেখানোর সময় অনুমতি পরীক্ষা করা, কিন্তু একক সম্পদ পড়ার সময় না করা — যেমন, ব্যবহারকারী শুধু নিজের অর্ডার দেখতে পারবে, কিন্তু URL-এ অন্যের অর্ডার আইডি দিলে সেটিও দেখা যায়। একে বলে Insecure Direct Object Reference (IDOR)। প্রতিকার: প্রতিটি অনুরোধে জিজ্ঞাসা করুন — এই পরিচয়ের কি এই নির্দিষ্ট সম্পদে এই নির্দিষ্ট কাজ করার অনুমতি আছে? এই প্রশ্নটি একটি সহায়ক ফাংশনে (middleware) কেন্দ্রীভূত করুন, প্রতিটি রুটে আলাদাভাবে নয় — নইলে কোনো রুটে ভুলে যাওয়া সহজ।

> **WARNING:** “লগইন করেছে মানেই অনুমতি আছে” — এই ধারণাই সবচেয়ে বিপজ্জনক। Authentication ছাড়া Authorization অর্থহীন নয়, কিন্তু Authentication-ই Authorization নয়। প্রতিটি সংবেদনশীল কাজে আলাদাভাবে অনুমতি পরীক্ষা করুন, বিশেষ করে অ্যাডমিন কাজ, অন্যের তথ্য পড়া, ও তথ্য মুছে ফেলায়।

৪.৬ বাস্তব কমান্ড ও কনফিগারেশন

নিচে Express-এ RBAC middleware-এর একটি ধারণাগত উদাহরণ দেওয়া হলো। এটি ল্যাবের জন্য; প্রোডাকশনে ব্যবহারের আগে আপনার দলের পর্যালোচনা ও পরীক্ষা প্রয়োজন।

```
// server/middleware/auth.js — ধারণাগত উদাহরণ (ল্যাব)
const jwt = require('jsonwebtoken');
function authenticate(req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.replace('Bearer ', '');
  if (!token) return res.status(401).json({ error: 'Unauthorized' });
  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET || 'CHANGE-ME-jwt-secret');
    req.user = payload; // { sub, role, exp } — সংবেদনশীল তথ্য নয়
    next();
  } catch (e) {
    return res.status(401).json({ error: 'Unauthorized' });
  }
}
function requireRole(...roles) {
  return (req, res, next) => {
    if (!req.user || !roles.includes(req.user.role)) {
      return res.status(403).json({ error: 'Forbidden' });
    }
    next();
  };
}
// ব্যবহার: app.get('/api/admin/users', authenticate, requireRole('admin'), handler);
// মনে রাখুন: ভূমিকা যাচাই সার্ভারে, প্রতিটি অনুরোধে।
```

ল্যাব সার্ভারে ব্যবহারকারী ও sudo অ্যাক্সেস পর্যালোচনা করুন। কে sudo করতে পারে, তা জানা পরিচয় নিরাপত্তার মৌলিক যাচাই।

```
# ল্যাব: কে sudo করতে পারে, শেষ লগইন কখন
getent group sudo
sudo grep -E "sudo|admin" /etc/group
last -n 20
sudo cat /var/log/auth.log | grep -i "failed password" | tail -n 20
# অপ্রয়োজনীয় অ্যাকাউন্ট আছে কি না, তা পর্যালোচনা করুন
getent passwd | cut -d: -f1,6,7
```

একটি সরল Access Review চেকলিস্ট: প্রতি মাসে — কে অ্যাডমিন, কে ছেড়ে গেছে, কোন সার্ভিস কী-র মেয়াদ শেষ, কোন টোকেন আর ব্যবহার হয় না। এই তালিকাটি একটি নথিতে রাখুন, মালিকের নাম ও তারিখসহ।

৪.৭ প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — চলে যাওয়া কর্মীর অ্যাকাউন্ট। একজন ডেভেলপার TechNova ছেড়ে গেলেন, কিন্তু তাঁর অ্যাকাউন্ট ও API কী বন্ধ হতে এক সপ্তাহ দেরি হলো। সেই সময়ে কেউ তাঁর পরিচয় ব্যবহার করে কোডবেসে ঢোকার চেষ্টা করলেন। শিক্ষা: অ্যাকাউন্ট বন্ধ করা চাকরি ছাড়ার প্রক্রিয়ার অংশ হতে হবে — একই দিনে, একটি চেকলিস্টে: কেন্দ্রীয় পরিচয়, Git, সার্ভার, ক্লাউড, সিক্রেট। Federation-এর সুবিধা এখানে: কেন্দ্রীয় জায়গায় বন্ধ করলে অনেক সেবা একসাথে বন্ধ হয়।

পরিস্থিতি ২ — IDOR: অন্যের অর্ডার দেখা। ব্যবহারকারী URL-এ অর্ডার আইডি বদলে অন্যের অর্ডার দেখতে পেলেন। ফ্রন্টএন্ড শুধু নিজের অর্ডার দেখাত, কিন্তু API মালিকানা যাচাই করত না। সমাধান: প্রতিটি অনুরোধে মালিকানা পরীক্ষা — এই অর্ডার কি এই ব্যবহারকারীর? এই ভুলটি RBAC দিয়েও ধরা পড়ে না, কারণ দুজনেই একই ভূমিকায়; এজন্য সম্পদ-ভিত্তিক যাচাই দরকার।

পরিস্থিতি ৩ — CI/CD টোকেন ফাঁস। একটি দীর্ঘমেয়াদি ডিপ্লয় টোকেন ভুলে পাবলিক রিপোজিটরিতে চলে গেল। টোকেনটি বাতিল করা হলো, কিন্তু ততক্ষণে কেউ তা ব্যবহার করে পরীক্ষামূলক ডিপ্লয় করেছিলেন। শিক্ষা: CI/CD-তে স্বল্পমেয়াদি Workload Identity ব্যবহার করুন, টোকেনের পরিধি শুধু ডিপ্লয়ে সীমিত রাখুন, এবং ফাঁস শনাক্ত হলে দ্রুত বাতিল ও ঘোরানোর প্রক্রিয়া আগে থেকে তৈরি রাখুন।

> **REAL WORLD:** বড় প্রতিষ্ঠানে সবচেয়ে বেশি ক্ষতি হয় চুরি যাওয়া পরিচয়পত্র দিয়ে, জটিল দুর্বলতা দিয়ে নয়। পাসওয়ার্ড পুনঃব্যবহার, ফাঁস হওয়া টোকেন, বন্ধ না হওয়া অ্যাকাউন্ট — এই মৌলিক বিষয়গুলোই আক্রমণকারীর প্রিয় প্রবেশপথ। MFA ও নিয়মিত Access Review এই পথটি অনেকটাই বন্ধ করে দেয়।

৪.৮ নিরাপত্তা ও খরচ বিবেচনা

পরিচয় নিরাপত্তার খরচের বড় অংশ হলো ব্যবহারকারীর অভিজ্ঞতা। প্রতিটি কাজে MFA চাইলে মানুষ বিরক্ত হবে; কখনো MFA চাইলে মানুষ তা এড়াতে শিখবে। ভালো ভারসাম্য: লগইনে MFA, সংবেদনশীল কাজে (পাসওয়ার্ড বদল, অ্যাডমিন কাজ, পেমেন্ট) আবার যাচাই, সাধারণ কাজে হালকা। আরেকটি খরচ: কেন্দ্রীয় Identity Provider-এর ওপর নির্ভরতা — এটি বন্ধ হলে সব সেবা বন্ধ হতে পারে। তাই পুনরুদ্ধারের পরিকল্পনা রাখুন: জরুরি অ্যাডমিন অ্যাক্সেস (Break Glass) কীভাবে কাজ করবে, তা লিখুন ও পরীক্ষা করুন — তবে সেই অ্যাকাউন্টটি সবচেয়ে শক্তভাবে সুরক্ষিত ও নিরীক্ষিত হতে হবে। খরচের দিক থেকে MFA, Access Review ও স্বল্পমেয়াদি টোকেন — এই তিনটি সবচেয়ে সাশ্রয়ী ও প্রভাবশালী বিনিয়োগ।

৪.৯ সাধারণ ভুল

- **Auth ও Authz গুলিয়ে ফেলা:** লগইন করলেই সব কাজের অনুমতি দেওয়া।
- **ফ্রন্টএন্ডে অনুমতি রাখা:** বোতাম লুকানোকে নিরাপত্তা মনে করা।
- **দীর্ঘমেয়াদি টোকেন:** মেয়াদহীন বা বছরের মেয়াদের টোকেন ব্যবহার করা।
- **JWT-তে সংবেদনশীল তথ্য:** Payload পড়া যায় — এতে গোপন তথ্য রাখা যাবে না।
- **Access Review না করা:** পুরোনো অনুমতি জমে Privilege Creep তৈরি হওয়া।
- **সবার জন্য একই সার্ভিস কী:** কোন সার্ভিস কী করল, তা বোঝা যায় না; একটি ফাঁস হলে সব ফাঁস হয়।
- **চলে যাওয়া কর্মীর অ্যাকাউন্ট খোলা রাখা:** অফবোর্ডিং চেকলিস্ট না থাকা।

৪.১০ ট্রাবলশুটিং

সমস্যা: বৈধ ব্যবহারকারী "Forbidden" পাচ্ছেন। প্রথমে টোকেনের ভূমিকা দেখুন: Payload-এ role কী আছে? তারপর রুটের প্রয়োজনীয় ভূমিকা দেখুন। ভূমিকা ঠিক থাকলে সম্পদ-ভিত্তিক যাচাই দেখুন — মালিকানা ঠিক আছে কি? লগে কোন ধাপে প্রত্যাখ্যান হলো, তা লিখুন — Authentication-এ নাকি Authorization-এ। দুটি আলাদা সমস্যা, আলাদা সমাধান।

সমস্যা: MFA-তে আটকে গেছেন। পুনরুদ্ধারের পথ ব্যবহার করুন: ব্যাকআপ কোড, বিকল্প পদ্ধতি, বা অনুমোদিত অ্যাডমিনের সাহায্য। পুনরুদ্ধারের পথটি নিজেই যেন দুর্বল না হয় — তা লিখিত, নিরীক্ষিত ও সীমিত হতে হবে। MFA চালুর আগে পুনরুদ্ধারের পথ পরীক্ষা করুন।

সমস্যা: টোকেন মেয়াদোত্তীর্ণ হচ্ছে বারবার। Access Token-এর মেয়াদ খুব ছোট কি না দেখুন, Refresh প্রবাহ ঠিক আছে কি না দেখুন, এবং ঘড়ির সময় (clock skew) মিলছে কি না যাচাই করুন। সার্ভার ও ক্লায়েন্টের ঘড়ি না মিললে বৈধ টোকেনও বাতিল মনে হতে পারে।

> **TROUBLESHOOTING:** অনুমতি সমস্যায় অনুমান করবেন না — টোকেনের Payload (ল্যাবে, নিজের টোকেন) দেখুন, রুটের middleware দেখুন, লগ দেখুন। "কার অনুমতি নেই" জানার আগে "কোন ধাপে আটকাল" জানা জরুরি।

৪.১১ সারসংক্ষেপ

পরিচয় নিরাপত্তা মানে প্রতিটি অনুরোধে দুটি প্রশ্ন করা: তুমি কে, এবং তুমি এই কাজটি করতে পারো কি? RBAC সরল ও ছোট দলের জন্য উপযুক্ত; ABAC নমনীয় কিন্তু জটিল। Zero Trust বলে অবস্থান বিশ্বাস নয়, প্রতিটি অনুরোধ যাচাই করতে হবে। MFA একটি উপাদান ফাঁস হলেও সুরক্ষা দেয়; স্বল্পমেয়াদি টোকেন ফাঁসের ক্ষতি কমায়; Service ও Workload Identity মানুষের পরিচয় থেকে সার্ভিসের পরিচয় আলাদা করে; Federation কেন্দ্রীয়ভাবে পরিচয় পরিচালনা সহজ করে; আর নিয়মিত Access Review Privilege Creep ঠেকায়। সবচেয়ে গুরুত্বপূর্ণ নিয়ম: অনুমতি পরীক্ষা সার্ভারে, প্রতিটি অনুরোধে, সম্পদ ও কাজের ভিত্তিতে।

৪.১২ পর্যালোচনা প্রশ্ন

1. Authentication ও Authorization-এর পার্থক্য উদাহরণসহ ব্যাখ্যা করুন।
2. RBAC ও ABAC-এর তুলনা করুন: কোনটি কখন উপযুক্ত এবং কেন?
3. Zero Trust-এর অর্থ কী? "ভেতরে থাকলেই বিশ্বস্ত" — এই ধারণা কেন ভুল?
4. Least Privilege ও Privilege Creep-এর সম্পর্ক ব্যাখ্যা করুন। Access Review কীভাবে সাহায্য করে?

5. MFA কীভাবে কাজ করে? SMS কোডের চেয়ে Authenticator অ্যাপ কেন বেশি নিরাপদ বলে মনে করা হয়?
6. JWT-র Payload-এ সংবেদনশীল তথ্য রাখা কেন বিপজ্জনক? স্বাক্ষর যাচাই কেন জরুরি?
7. IDOR কী? একটি উদাহরণ দিয়ে প্রতিকার ব্যাখ্যা করুন।
8. Service Identity ও Workload Identity কাকে বলে? সবার জন্য একই কী ব্যবহারের ঝুঁকি কী?
9. Identity Federation-এর সুবিধা ও ঝুঁকি কী কী?
10. Break Glass অ্যাকাউন্ট কী এবং কেন তা সবচেয়ে শক্তভাবে সুরক্ষিত হতে হবে?

৪.১৩ ব্যবহারিক অনুশীলন

1. **RBAC টেবিল:** TechNova-র জন্য একটি RBAC টেবিল তৈরি করুন: ভূমিকা (admin, editor, viewer, customer), প্রতিটি ভূমিকার অনুমতি, এবং একটি ব্যতিক্রমের উদাহরণ।
2. **IDOR পরীক্ষা:** ল্যাবে একটি API রুটে মালিকানা যাচাই যোগ করুন — ব্যবহারকারী শুধু নিজের সম্পদ পড়তে পারবে। যাচাই ছাড়া ও যাচাইসহ — দুটি আচরণ পরীক্ষা করে লিখুন।
3. **Access Review:** আপনার ল্যাব সার্ভারে sudo গ্রুপ, শেষ লগইন ও অপ্রয়োজনীয় অ্যাকাউন্টের একটি পর্যালোচনা প্রতিবেদন লিখুন। কোন অ্যাক্সেস রাখবেন, কোনটি সরাবেন — কারণসহ লিখুন।

অধ্যায় ৫ • PART I — Security Engineering Foundations

Chapter 5 — Secrets, Encryption and Key Management

৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Secrets Lifecycle-এর প্রতিটি ধাপ ব্যাখ্যা করতে পারবেন, API Key সুরক্ষার নীতি প্রয়োগ করতে পারবেন, এবং HashiCorp Vault ও Cloud Secrets Manager-এর ধারণাগত ব্যবহার বুঝতে পারবেন। আপনি KMS (Key Management Service), Key Rotation, TLS, Encryption at Rest ও Encryption in Transit, Certificate Management ও Secret Leakage Prevention-এর প্রতিটি ধারণা প্রথম নীতি থেকে ব্যাখ্যা করতে পারবেন। সবশেষে, আপনি TechNova-র জন্য একটি বাস্তবসম্মত সিক্রেট পরিচালনা পরিকল্পনা লিখতে পারবেন — কোথায় সিক্রেট থাকবে, কে তা পড়তে পারবে, কখন তা ঘোরানো হবে, এবং ফাঁস হলে কী করবেন।

- Secrets Lifecycle: তৈরি, সংরক্ষণ, ব্যবহার, ঘোরানো ও বাতিল — প্রতিটি ধাপ ব্যাখ্যা করা।
- Vault ও Cloud Secrets Manager-এর ধারণা ও ব্যবহারের সীমা বোঝা।
- Encryption at Rest ও in Transit-এর পার্থক্য ও TLS-এর ভূমিকা ব্যাখ্যা করা।
- KMS, Key Rotation ও Certificate Management-এর নীতি প্রয়োগ করা।
- Secret Leakage শনাক্ত করা ও ফাঁসের পর প্রতিক্রিয়া পরিকল্পনা করা।

৫.২ পূর্বশর্ত ঝালাই

আগের অধ্যায় থেকে পরিচয়, Least Privilege ও Trust Boundary মনে রাখুন। এই অধ্যায়ে তিনটি মৌলিক পার্থক্য ঝালিয়ে নিন। প্রথমত, Secret (গোপন তথ্য) মানে যা জানলে ক্ষমতা পাওয়া যায় — পাসওয়ার্ড, API Key, টোকেন, ব্যক্তিগত কী, ডাটাবেস পরিচয়পত্র। Key (চাবি) মানে এনক্রিপশনে ব্যবহৃত গোপন মান — যা দিয়ে তথ্য আটকানো ও খোলা হয়। Certificate (সার্টিফিকেট) মানে একটি পরিচয়ের সাথে একটি পাবলিক কী-র সংযোগের প্রমাণ, যা বিশ্বস্ত কর্তৃপক্ষ স্বাক্ষর করে। দ্বিতীয়ত, Encryption (এনক্রিপশন) মানে তথ্যকে চাবি ছাড়া পড়া যায় না এমন রূপে বদলানো; Hashing (হ্যাশিং) মানে তথ্য থেকে একমুখী সারাংশ তৈরি করা — হ্যাশ থেকে আসল তথ্য ফেরানো যায় না, তাই পাসওয়ার্ড হ্যাশ করে রাখা হয়, এনক্রিপ্ট করে নয়। তৃতীয়ত, Symmetric Encryption-এ একই চাবি আটকায় ও খোলে; Asymmetric Encryption-এ পাবলিক কী আটকায়, ব্যক্তিগত কী খোলে। TLS-এ দুটোই ব্যবহৃত হয় — বিস্তারিত নিচে আসছে।

৫.৩ প্রথম নীতি থেকে সিক্রেট: কেন সিক্রেট কোডে রাখা যায় না

সিক্রেট পরিচালনার প্রথম নীতি হলো: সিক্রেট যেখানে থাকে, সেখানে ক্ষমতা থাকে। কোডে সিক্রেট রাখলে যে কেউ কোড দেখলেই ক্ষমতা পেয়ে যায় — ডেভেলপার, ঠিকাদার, এমনকি ভুলে পাবলিক হয়ে যাওয়া রিপোজিটরির যে কেউ। Git-এর ইতিহাস থেকে সিক্রেট মোছা কঠিন, কারণ ইতিহাসে তা থেকে যায়; “মুছে ফেলেছি” বললেই তা মুছে যায় না। আবার পরিবেশ ভেরিয়েবলে (Environment Variable) সিক্রেট রাখাও নিখুঁত নয় — তা প্রক্রিয়ার তালিকায়, লগে, বা ভুল কনফিগারেশনে ফাঁস হতে পারে। তবু পরিবেশ ভেরিয়েবল কোডে রাখার চেয়ে ভালো, কারণ কোড ও সিক্রেট আলাদা থাকে। সবচেয়ে ভালো পদ্ধতি হলো একটি নির্দিষ্ট Secrets Manager ব্যবহার করা — যেখানে সিক্রেট এনক্রিপ্টেড থাকে, কে তা পড়েছে তার লগ থাকে, মেয়াদ ও ঘোরানোর নিয়ম থাকে, এবং প্রয়োজনে দ্রুত বাতিল করা যায়।

Secrets Lifecycle-এর ধাপগুলো বুঝুন। তৈরি (Creation): সিক্রেট যথেষ্ট দীর্ঘ, এলোমেলো ও অনন্য হতে হবে — “password123” বা সব সেবায় একই কী চলবে না। সংরক্ষণ (Storage): সিক্রেট এনক্রিপ্টেড অবস্থায়, শুধু অনুমোদিত

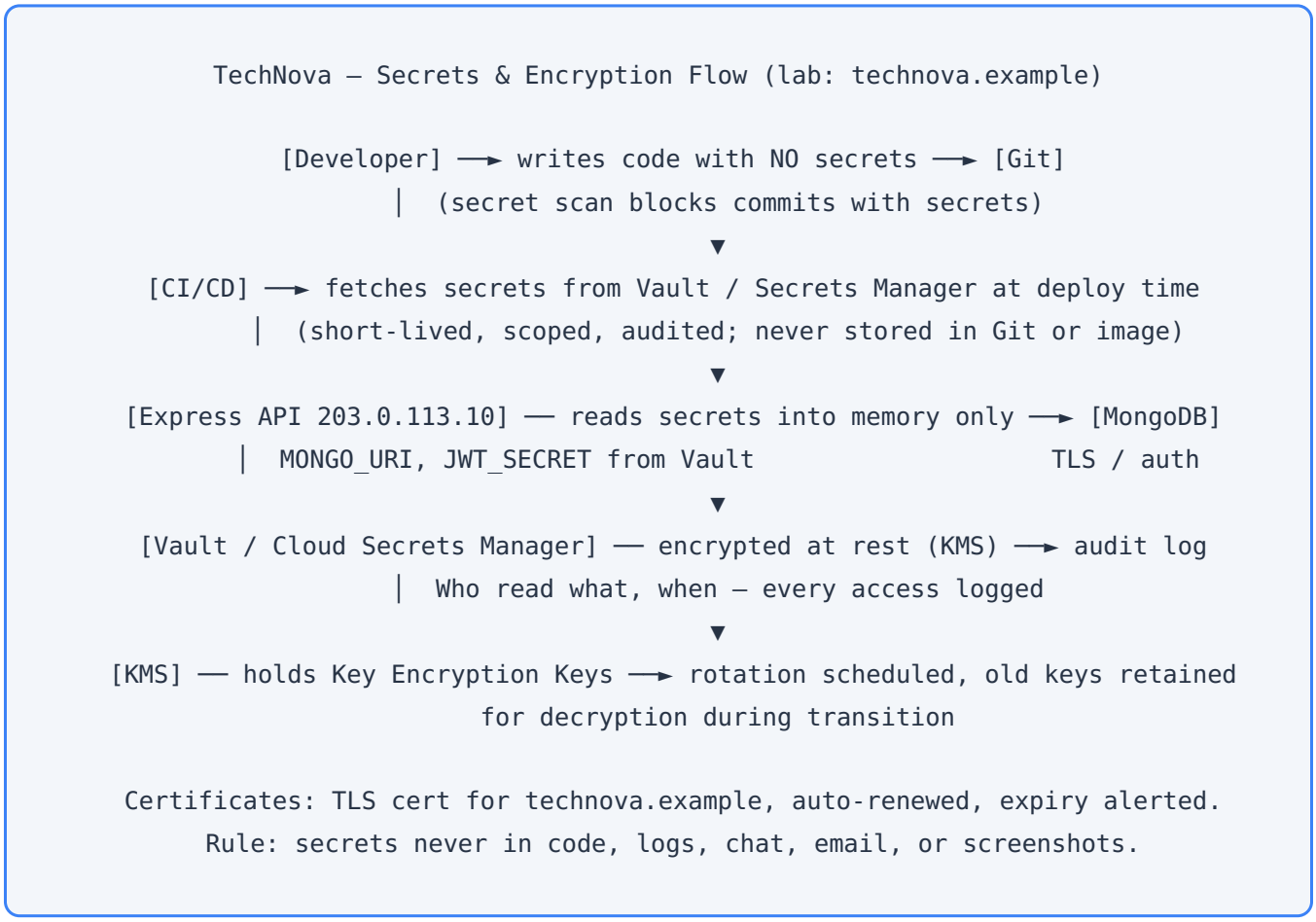
পরিচয়ের কাছে পৌঁছানো যায় এমন জায়গায় থাকবে। বিতরণ (Distribution): সিক্রেট নিরাপদ চ্যানেলে, প্রয়োজনের সময়, শুধু প্রয়োজনীয় সার্ভিসে যাবে — ইমেইল বা চ্যাটে নয়। ব্যবহার (Use): সিক্রেট শুধু স্মৃতিতে (memory) থাকবে যতক্ষণ প্রয়োজন, লগে লেখা হবে না, ভুল বার্তায় দেখানো হবে না। ঘোরানো (Rotation): নিয়মিত বিরতিতে বা সন্দেহ হলে সিক্রেট বদলানো হবে — পুরোনোটি বাতিল করে নতুনটি দেওয়া। বাতিল (Revocation): কর্মী চলে গেলে, সার্ভিস বন্ধ হলে, বা ফাঁস হলে সিক্রেট সাথে সাথে বাতিল করতে হবে। প্রতিটি ধাপে প্রশ্ন একটাই: এই সিক্রেটটি কে জানে, কোথায় আছে, কতদিন থাকবে, এবং ফাঁস হলে কী হবে?

> **CONCEPT:** Encryption at Rest মানে সংরক্ষিত তথ্যের এনক্রিপশন — ডিস্কে, ডাটাবেসে, ব্যাকআপে। Encryption in Transit মানে চলমান তথ্যের এনক্রিপশন — ব্রাউজার থেকে সার্ভারে, সার্ভার থেকে ডাটাবেসে। দুটোই দরকার; একটি ছাড়া অন্যটি অসম্পূর্ণ। ডিস্ক এনক্রিপ্টেড থাকলেও নেটওয়ার্কে খোলা তথ্য পাঠালে তা আটকানো যায়; আবার TLS থাকলেও ডিস্ক চুরি হলে খোলা তথ্য পড়া যায়।

TLS (Transport Layer Security) কীভাবে কাজ করে, তা সরলভাবে বুঝুন। প্রথমে ক্লায়েন্ট ও সার্ভার হাত মেলায় (Handshake): সার্ভার তার সার্টিফিকেট দেখায়, ক্লায়েন্ট যাচাই করে সার্টিফিকেটটি বিশ্বস্ত কর্তৃপক্ষের স্বাক্ষর করা ও ডোমেইনের সাথে মেলে কি না। তারপর তারা একটি অস্থায়ী, গোপন Session Key-তে সম্মত হয় — Asymmetric Encryption দিয়ে সেই সম্মতি নিরাপদে হয়, তারপর Symmetric Encryption দিয়ে দ্রুত তথ্য আদান-প্রদান হয়। এই কারণে TLS শুধু এনক্রিপশন দেয় না; এটি পরিচয় যাচাই (সার্ভারটি আসলেই technova.example কি না) ও অখণ্ডতা (তথ্য পথে বদলানো হয়নি) — দুটোই দেয়। TLS ছাড়া HTTP মানে পথে যে কেউ তথ্য পড়তে ও বদলাতে পারে। তাই প্রোডাকশনে সব সংবেদনশীল যোগাযোগে HTTPS বাধ্যতামূলক; ব্যতিক্রমের কোনো স্থান নেই।

KMS-এর ধারণা হলো চাবির চাবি। তথ্য এনক্রিপ্ট করার চাবি (Data Key) নিজেই আরেকটি চাবি (Key Encryption Key) দিয়ে এনক্রিপ্ট করা থাকে, আর সেই চাবিটি KMS-এ সুরক্ষিত থাকে। এর সুবিধা: Data Key ফাঁস হলেও KMS-এর চাবি ছাড়া তা ব্যবহার করা যায় না; KMS-এ কে চাবি ব্যবহার করল, তার লগ থাকে; চাবি ঘোরানো যায় তথ্য আবার এনক্রিপ্ট না করেই (নতুন চাবি দিয়ে পুরোনো Data Key আবার আটকে)। Key Rotation মানে নিয়মিত বিরতিতে চাবি বদলানো — যাতে একটি চাবি ফাঁস হলে তার ক্ষতি সীমিত থাকে, এবং দীর্ঘদিন একই চাবি ব্যবহারের ঝুঁকি কমে। ঘোরানোর সময় পুরোনো চাবি দিয়ে এনক্রিপ্ট করা তথ্য পড়ার ব্যবস্থা রাখতে হয়, নইলে তথ্য হারিয়ে যাবে — তাই ঘোরানোর পরিকল্পনায় পুরোনো ও নতুন চাবির সহাবস্থানের সময় রাখুন।

৫.৪ আর্কিটেকচার: TechNova-র সিক্রেট প্রবাহ



সিক্রেটের ধরন	কোথায় থাকবে	কোথায় থাকবে না	ঘোরানো
ডাটাবেস পরিচয়পত্র	Vault / Secrets Manager	কোড, Git, পরিবেশ ফাইল	৯০ দিন বা সন্দেহ হলে
JWT Signing Secret	Vault, শুধু API পড়ে	ফ্রন্টএন্ড, লগ	৯০ দিন, পুরোনো টোকেন মেয়াদ শেষে
API Key (বাইরের সেবা)	Vault, পরিধি সীমিত	ক্লায়েন্ট কোড, চ্যাট	সেবার নীতি অনুযায়ী
TLS ব্যক্তিগত কী	সার্ভারে সুরক্ষিত, 600 অনুমতি	Git, ব্যাকআপে খোলা অবস্থায়	সার্টিফিকেট নবায়নে
CI/CD টোকেন	Workload Identity, স্বল্পমেয়াদি	রিপোজিটরি, লগ	প্রতি রানে নতুন

৫.৫ অভ্যন্তরীণ কার্যপ্রণালী: Vault, KMS ও সার্টিফিকেট কীভাবে কাজ করে

HashiCorp Vault ধারণাগতভাবে একটি সুরক্ষিত ভান্ডার: সিক্রেট এনক্রিপ্টেড অবস্থায় থাকে, এবং কেউ তা পড়তে চাইলে প্রথমে নিজের পরিচয় প্রমাণ করতে হয়, তারপর নীতি (Policy) যাচাই হয় — এই পরিচয়ের কি এই সিক্রেট পড়ার অনুমতি আছে? অনুমতি থাকলে Vault সিক্রেটটি দেয়, এবং সেই পড়ার লগ রাখে। Vault-এর শক্তি হলো Dynamic Secrets: প্রতিটি প্রয়োজনে একটি অস্থায়ী, অনন্য পরিচয়পত্র তৈরি করা — যেমন, ডাটাবেসে সংযোগের জন্য ১ ঘণ্টার মেয়াদের ব্যবহারকারী।

মেয়াদ শেষ হলে তা নিজে থেকেই বাতিল হয়; ফাঁস হলেও ক্ষতি সীমিত। Vault চালানো নিজেই একটি দায়িত্ব: Vault-এর নিজের চাবি, ব্যাকআপ, উচ্চ উপলভ্যতা ও অডিট — এগুলো ঠিক রাখতে হয়। ছোট দলের জন্য Cloud Secrets Manager (যেমন AWS Secrets Manager, Google Secret Manager) সহজ হতে পারে, কারণ পরিচালনার ভার ক্লাউডের; কিন্তু নীতি একই — কোডে সিক্রেট নয়, লগে সিক্রেট নয়, ঘোরানো ও বাতিলের ব্যবস্থা থাকতে হবে।

Certificate Management-এর ভেতরের কাজ: সার্টিফিকেটের মেয়াদ থাকে (সাধারণত ৯০ দিন থেকে ১ বছর)। মেয়াদ শেষ হলে ব্রাউজার সতর্ক করে, ব্যবহারকারী আস্থা হারায়, সেবা বন্ধ মনে হয়। তাই মেয়াদ শেষের অন্তত ৩০ দিন আগে অ্যালার্ট, স্বয়ংক্রিয় নবায়ন, এবং নবায়নের পর পরীক্ষা — এই তিনটি থাকতে হবে। সার্টিফিকেটের ব্যক্তিগত কী সবচেয়ে সংবেদনশীল: তা ফাঁস হলে যে কেউ আপনার ডোমেইনের ছদ্মবেশে সার্ভার সাজাতে পারে। ব্যক্তিগত কী 600 অনুমতিতে, শুধু প্রয়োজনীয় সার্ভিস পড়তে পারে এমনভাবে রাখুন, এবং কখনো Git-এ দেবেন না। Let's Encrypt-এর মতো স্বয়ংক্রিয় কর্তৃপক্ষ ছোট দলের জন্য উপযুক্ত, কারণ নবায়ন স্বয়ংক্রিয় করা যায়; তবে নবায়ন ব্যর্থ হলে অ্যালার্ট যেন আসে, তা নিশ্চিত করুন।

Secret Leakage Prevention মানে ফাঁস ঠেকানো ও ফাঁস ধরা — দুটোই। ঠেকানো: কমিটের আগে স্ক্যানার, CI-তে স্ক্যানার, কোড রিভিউয়ে সিক্রেট পরীক্ষা, এবং ডেভেলপারদের শিক্ষা — “সিক্রেট কোডে নয়”। ধরা: Git ইতিহাস স্ক্যান, পাবলিক রিপোজিটরি পর্যবেক্ষণ, লগে সিক্রেট প্যাটার্ন খোঁজা, এবং ক্লাউডে অস্বাভাবিক ব্যবহারের অ্যালার্ট। ফাঁস ধরা পড়লে প্রতিক্রিয়া স্পষ্ট হতে হবে: প্রথমে সিক্রেটটি বাতিল করুন (শুধু মুছে ফেললে হবে না — ইতিহাসে তা থেকে যায়), তারপর নতুন সিক্রেট দিন, তারপর লগ পরীক্ষা করে দেখুন ফাঁস হওয়া সিক্রেটটি ব্যবহার হয়েছে কি না, এবং শেষে শিক্ষা নিয়ে প্রক্রিয়া বদলান। ফাঁস লুকানো সবচেয়ে খারাপ প্রতিক্রিয়া; দ্রুত বাতিল করাই সবচেয়ে ভালো প্রতিক্রিয়া।

> **WARNING:** ফাঁস হওয়া সিক্রেট শুধু মুছে ফেললে তা নিরাপদ হয় না। Git ইতিহাস, ব্যাকআপ, লগ, স্ক্রিনশট — যেকোনো জায়গায় তা থেকে যেতে পারে। ফাঁসের একমাত্র নিরাপদ প্রতিক্রিয়া হলো সিক্রেটটি বাতিল (revoke) করা ও ঘোরানো। “কেউ দেখেনি” — এই অনুমান করবেন না।

৫.৬ বাস্তব কমান্ড ও কনফিগারেশন

নিচের সব কাজ ল্যাব পরিবেশে, প্লেসহোল্ডার সিক্রেট দিয়ে করবেন। আসল সিক্রেট কখনো উদাহরণে লিখবেন না।

```
# 1) এলোমেলো সিক্রেট তৈরি (ল্যাব, প্লেসহোল্ডার)
openssl rand -hex 32
# আউটপুটটি শুধু ল্যাবে ব্যবহার করুন; আসল সিক্রেট Vault-এ রাখুন।

# 2) TLS সার্টিফিকেট যাচাই (নিজের ডোমেইনে, ল্যাব)
echo | openssl s_client -connect technova.example:443 -servername technova.example 2>/dev/null
| openssl x509 -noout -dates -subject
# মেয়াদ শেষের তারিখ দেখুন; 30 দিনের মধ্যে শেষ হলে অ্যালার্ট দিন।

# 3) ফাইলে সিক্রেট আছে কি না, নিজের রিপোজিটরিতে খোঁজা (ল্যাব)
grep -R -i "password\s*=\|api[_-]\?key\s*=\|CHANGE-ME" server/ --include="*.js" | head -n 20
# CHANGE-ME প্লেসহোল্ডার ছাড়া আসল মান থাকলে তা সরান ও ঘোরান।

# 4) পরিবেশ ফাইলের অনুমতি (ল্যাব)
ls -l .env
chmod 600 .env
# .env কখনো Git-এ যাবে না — .gitignore-এ তা আছে কি না যাচাই করুন:
grep -E "^\.env$" .gitignore || echo "WARNING: .env not ignored"

# 5) Vault ধারণা — CLI উদাহরণ (আপনার দলের Vault থাকলে, ল্যাবে)
# vault kv put secret/technova/mongo password="CHANGE-ME-mongo-password"
```

```
# vault kv get secret/technova/mongo
# প্রতিটি পড়ার লগ Vault-এ থাকে — কে, কখন, কী পড়ল।
```

Node.js-এ Vault থেকে সিক্রেট পড়ার ধারণা: অ্যাপ শুরুতে একবার সিক্রেট পড়ে স্মৃতিতে রাখে, লগে লেখে না, এবং ভুল বার্তায় দেখায় না। নিচের খসড়াটি ধারণা বোঝার জন্য।

```
// server/secrets.js — ধারণাগত খসড়া (ল্যাব)
async function loadSecrets() {
  // বাস্তবে: Vault / Cloud Secrets Manager থেকে পড়ুন, অনুমোদিত পরিচয়ে।
  // এখানে শুধু পরিবেশ থেকে পড়া দেখানো হয়েছে — প্রোডাকশনে Vault ব্যবহার করুন।
  const mongoUri = process.env.MONGO_URI;
  if (!mongoUri || mongoUri.includes('CHANGE-ME')) {
    throw new Error('MONGO_URI is missing or placeholder — refusing to start');
  }
  return { mongoUri };
}
module.exports = { loadSecrets };
```

মনে রাখবেন, “প্লেসহোল্ডার থাকলে অ্যাপ চালু হবে না” — এই কঠোরতা ইচ্ছাকৃত। প্রোডাকশনে ভুল সিক্রেট দিয়ে চালু হওয়ার চেয়ে চালু না হওয়া নিরাপদ।

৫.৭ প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — Git-এ API Key ফাঁস। একজন ডেভেলপার ভুলে একটি বাইরের সেবার API Key কমিট করে ফেললেন। কয়েক ঘণ্টা পর সেই Key দিয়ে অস্বাভাবিক ব্যবহার ধরা পড়ল — বিল বাড়ছিল। দলটি প্রথমে Key-টি বাতিল করল, নতুন Key দিল, লগ পরীক্ষা করে ক্ষতির পরিমাণ নির্ধারণ করল, এবং CI-তে Secret Scan বাধ্যতামূলক করল। শিক্ষা: ফাঁসের পর প্রথম কাজ বাতিল করা, তদন্ত পরে; আর প্রতিরোধের জন্য স্ক্যানারকে বাধ্যতামূলক করা, ঐচ্ছিক নয়।

পরিস্থিতি ২ — সার্টিফিকেটের মেয়াদ শেষ। ছুটির দিনে technova.example-এর TLS সার্টিফিকেটের মেয়াদ শেষ হলো; ব্যবহারকারীরা ব্রাউজারে সতর্কতা দেখতে পেলেন। স্বয়ংক্রিয় নবায়ন ব্যর্থ হয়েছিল, কিন্তু অ্যালার্ট কেউ দেখেননি। সমাধান: নবায়ন স্বয়ংক্রিয় করা, মেয়াদ শেষের ৩০, ৭ ও ১ দিন আগে অ্যালার্ট, এবং নবায়নের পর স্বয়ংক্রিয় পরীক্ষা। শিক্ষা: স্বয়ংক্রিয় ব্যবস্থারও পর্যবেক্ষণ দরকার; “স্বয়ংক্রিয়” মানে “নজরদারিহীন” নয়।

পরিস্থিতি ৩ — Key Rotation-এ সেবা বন্ধ। JWT Signing Secret ঘোরানোর সময় পুরোনো টোকেনগুলো হঠাৎ বাতিল হয়ে গেল; ব্যবহারকারীরা লগআউট হয়ে গেলেন। কারণ, ঘোরানোর পরিকল্পনায় পুরোনো ও নতুন চাবির সহাবস্থানের সময় রাখা হয়নি। সমাধান: কিছুদিন দুটি চাবিই গ্রহণ করা, নতুন টোকেন নতুন চাবিতে দেওয়া, পুরোনো টোকেন স্বাভাবিক মেয়াদে শেষ হতে দেওয়া, তারপর পুরোনো চাবি সরানো। শিক্ষা: Rotation একটি প্রক্রিয়া, একটি মুহূর্ত নয়; সহাবস্থান ও পরীক্ষা ছাড়া ঘোরাবেন না।

> **REAL WORLD:** বড় ক্লাউড বিলের অনেক ঘটনার পেছনে ফাঁস হওয়া API Key থাকে। আক্রমণকারী Key পেয়ে ব্যয়বহুল সম্পদ চালায়, বিল ভুক্তভোগীর কাছে আসে। তাই API Key-র পরিধি সীমিত রাখা, ব্যবহারের সীমা দেওয়া, অস্বাভাবিক ব্যবহারে অ্যালার্ট দেওয়া, এবং নিয়মিত ঘোরানো — এই চারটি একসাথে না থাকলে একটি Key-ই বড় ক্ষতির কারণ হতে পারে।

৫.৮ নিরাপত্তা ও খরচ বিবেচনা

Secrets Manager-এর খরচ মূলত পরিচালনা ও শেখার খরচ: Vault চালাতে হলে তার ব্যাকআপ, উচ্চ উপলভ্যতা ও অডিট ঠিক রাখতে হয়; Cloud Secrets Manager-এ প্রতি পড়ায় সামান্য খরচ হয়, কিন্তু পরিচালনার ভার কমে। ছোট দলের জন্য পরামর্শ: শুরুতে Cloud Secrets Manager বা পরিবেশ ভেরিয়েবল + কঠোর নিয়ম দিয়ে শুরু করুন; Vault-এর প্রয়োজন তখনই, যখন Dynamic Secrets, বিস্তারিত অডিট ও কেন্দ্রীয় নীতির প্রয়োজন হবে। অতিরিক্ত জটিল ব্যবস্থা নেওয়া মানে তা ঠিকভাবে পরিচালনা না করার ঝুঁকি নেওয়া। আরেকটি খরচের দিক: Rotation-এর খরচ — প্রতিটি ঘোরানোয় পরীক্ষা, সমন্বয় ও ঝুঁকি থাকে; তাই ঘোরানোর বিরতি ঝুঁকির সাথে সামঞ্জস্যপূর্ণ হতে হবে। সংবেদনশীল সিক্রেট ঘন ঘন, কম সংবেদনশীল সিক্রেট কম ঘন ঘন। সবশেষে, এনক্রিপশনের খরচ: আধুনিক হার্ডওয়্যারে TLS ও ডিস্ক এনক্রিপশনের কার্যক্ষমতা খরচ নগণ্য; “এনক্রিপশন ধীর করবে” — এই যুক্তিতে এনক্রিপশন না করা আর গ্রহণযোগ্য নয়।

৫.৯ সাধারণ ভুল

- **সিক্রেট কোডে রাখা:** সবচেয়ে সাধারণ ও বিপজ্জনক ভুল; Git ইতিহাসে তা থেকে যায়।
- **সব সেবায় একই সিক্রেট:** একটি ফাঁস হলে সব ফাঁস হয়।
- **পাসওয়ার্ড এনক্রিপ্ট করে রাখা:** পাসওয়ার্ড হ্যাশ করতে হয়, এনক্রিপ্ট নয় — হ্যাশ একমুখী।
- **JWT-তে সংবেদনশীল তথ্য:** Payload পড়া যায়; এতে গোপন তথ্য রাখা যাবে না।
- **ফাঁসের পর শুধু মোছা:** বাতিল না করলে ফাঁস চলতেই থাকে।
- **সার্টিফিকেটের মেয়াদ না দেখা:** স্বয়ংক্রিয় নবায়নেও অ্যালার্ট দরকার।
- **লগে সিক্রেট লেখা:** ভুল বার্তা, ডিবাগ লগ — সব জায়গায় সিক্রেট ফাঁস হতে পারে।
- **Rotation ছাড়া দীর্ঘমেয়াদি চাবি:** বছরের পর বছর একই চাবি ব্যবহার করা।

৫.১০ ট্রাবলশুটিং

সমস্যা: অ্যাপ সিক্রেট পড়তে পারছে না। প্রথমে দেখুন অ্যাপের পরিচয়ের কি Vault-এ পড়ার অনুমতি আছে? নীতি ঠিক আছে কি? সিক্রেটের পথ (path) ঠিক আছে কি? লগে অনুমতি প্রত্যাখ্যানের বার্তা আছে কি? অনুমতি সমস্যা না পথের সমস্যা, তা আলাদা করুন।

সমস্যা: TLS সার্টিফিকেট অবিশ্বস্ত দেখাচ্ছে। মেয়াদ, ডোমেইন নাম, ও চেইন যাচাই করুন: `openssl s_client` দিয়ে দেখুন কোন ধাপে ব্যর্থ হচ্ছে। ঘড়ির সময় ভুল থাকলেও সার্টিফিকেট অবিশ্বস্ত মনে হতে পারে — সার্ভারের ঘড়ি যাচাই করুন।

সমস্যা: Rotation-এর পর কিছু সার্ভিস কাজ করছে না। কোন সার্ভিস এখনো পুরোনো সিক্রেট ব্যবহার করছে, তা খুঁজুন। সব সার্ভিসে নতুন সিক্রেট পৌঁছেছে কি না, তা যাচাই করুন। পুরোনো সিক্রেটটি সহাবস্থানের সময় শেষ হওয়ার আগে বাতিল করবেন না।

> **TROUBLESHOOTING:** সিক্রেট সমস্যায় কখনো আসল সিক্রেট চ্যাটে, টিকিটে বা লগে পাঠাবেন না। সমস্যা বর্ণনা করুন সিক্রেটের নাম, পথ ও সময় দিয়ে — মান দিয়ে নয়। তদন্তের সময়ও সিক্রেট সুরক্ষিত রাখুন।

৫.১১ সারসংক্ষেপ

সিক্রেট যেখানে থাকে, সেখানে ক্ষমতা থাকে — তাই সিক্রেট কোডে, Git-এ, লগে বা চ্যাটে রাখা যাবে না। Secrets Lifecycle-এর প্রতিটি ধাপ — তৈরি, সংরক্ষণ, বিতরণ, ব্যবহার, ঘোরানো, বাতিল — লিখিত ও নিরীক্ষিত হতে হবে। Vault ও Cloud

Secrets Manager সিক্রেটকে কেন্দ্রীয়ভাবে, এনক্রিপ্টেড অবস্থায়, অডিটসহ রাখে; KMS চাবির চাবি সুরক্ষিত রাখে; TLS চলমান তথ্যকে পরিচয় যাচাইসহ সুরক্ষিত করে; Encryption at Rest সংরক্ষিত তথ্যকে সুরক্ষিত করে। ফাঁস হলে প্রথম কাজ বাতিল করা, শুধু মোছা নয়। আর সবচেয়ে গুরুত্বপূর্ণ কথা: পাসওয়ার্ড হ্যাশ করতে হয়, এনক্রিপ্ট নয়; এবং যে সিক্রেট পরীক্ষা করা যায় না, তার ওপর ভরসা করবেন না।

৫.১২ পর্যালোচনা প্রশ্ন

1. Secrets Lifecycle-এর ধাপগুলো লিখুন এবং প্রতিটি ধাপে একটি করে ঝুঁকি ও নিয়ন্ত্রণ দিন।
2. সিক্রেট কোডে রাখা কেন বিপজ্জনক, যদিও রিপোজিটরি ব্যক্তিগত (private)? তিনটি কারণ লিখুন।
3. Hashing ও Encryption-এর পার্থক্য কী? পাসওয়ার্ডের জন্য কোনটি উপযুক্ত এবং কেন?
4. Encryption at Rest ও Encryption in Transit-এর পার্থক্য উদাহরণসহ ব্যাখ্যা করুন।
5. TLS Handshake সরলভাবে ব্যাখ্যা করুন: পরিচয় যাচাই, Session Key ও এনক্রিপশন কীভাবে হয়?
6. Vault-এর Dynamic Secrets কী এবং কেন তা স্থির সিক্রেটের চেয়ে নিরাপদ?
7. KMS-এ “চাবির চাবি” ধারণাটি কীভাবে কাজ করে? Key Rotation-এর সময় সহাবস্থান কেন দরকার?
8. Certificate-এর মেয়াদ শেষ হলে কী হয়? স্বয়ংক্রিয় নবায়নেও অ্যালার্ট কেন দরকার?
9. Secret ফাঁস হলে সঠিক প্রতিক্রিয়ার ধাপগুলো ক্রমানুসারে লিখুন। শুধু মুছে ফেলা কেন যথেষ্ট নয়?
10. ছোট দলের জন্য Vault বনাম Cloud Secrets Manager — কোনটি কখন বেছে নেবেন? যুক্তিসহ লিখুন।

৫.১৩ ব্যবহারিক অনুশীলন

1. **সিক্রেট নিরীক্ষা:** আপনার ল্যাব রিপোজিটরিতে Secret Scan চালান (অনুমোদিত স্ক্যানার দিয়ে)। কোনো প্লেসহোল্ডার ছাড়া আসল মান পেলে তা বাতিল ও ঘোরানোর পরিকল্পনা লিখুন — আসল মান কোথাও লিখবেন না।
2. **Rotation পরিকল্পনা:** TechNova-র পাঁচটি সিক্রেটের জন্য একটি Rotation টেবিল তৈরি করুন: সিক্রেট, মালিক, বিরতি, সহাবস্থানের সময়, বাতিলের পদ্ধতি।
3. **TLS যাচাই:** ল্যাবে `openssl s_client` দিয়ে technova.example-এর (বা আপনার ল্যাব ডোমেইনের) সার্টিফিকেটের মেয়াদ, ডোমেইন ও চেইন যাচাই করুন। মেয়াদ শেষের অ্যালার্টের একটি চেকলিস্ট লিখুন।

PART II • DEVOPS ENGINEERING MASTERY

Application & Software Supply Chain Security

Secure Coding, OWASP Top 10, SAST/DAST ৩ Dependency Security, Container ৩ Kubernetes Security, Software Supply Chain Security — SBOM, Signing ৩ SLSA।

অধ্যায় ৬ • PART II — Application & Software Supply Chain Security

Chapter 6 — Secure Coding and Application Security

৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে শিক্ষার্থী নিরাপদ কোড লেখার নীতিগুলোকে শুধু মুখস্থ নিয়ম হিসেবে নয়, বরং আক্রমণকারী কীভাবে একটি অ্যাপ্লিকেশনের ভেতরে ঢোকে সেই যুক্তি থেকে বুঝতে পারবে। শিক্ষার্থী ব্যাখ্যা করতে পারবে কেন ইনপুটকে কখনোই বিশ্বাস করা যায় না, কেন আউটপুট এনকোডিং ইনপুট ভ্যালিডেশনের বিকল্প নয়, এবং কেন প্রমাণীকরণ, সেশন ও অনুমোদন — এই তিনটি আলাদা সমস্যা, যাদের একটিকে আরেকটি দিয়ে ঢাকা যায় না। শিক্ষার্থী একটি MERN অ্যাপ্লিকেশনে — MongoDB, Express, React ও Node.js দিয়ে তৈরি TechNova-র কাল্পনিক অ্যাপে — দুর্বল কোড চিনে তাকে নিরাপদ কোডে বদলাতে পারবে। সে পাসওয়ার্ড হ্যাশিং, নিরাপদ সেশন কুকি, ভূমিকা-ভিত্তিক অনুমোদন, নিরাপদ ত্রুটি বার্তা এবং লগে সংবেদনশীল তথ্য না রাখার নিয়ম প্রয়োগ করতে পারবে। সবশেষে শিক্ষার্থী বুঝবে নিরাপত্তা কোনো এককালীন বৈশিষ্ট্য নয়; এটি কোড লেখার প্রতিটি সিদ্ধান্তে জড়িয়ে থাকা একটি অভ্যাস, যেখানে প্রতিটি সীমানা — ব্যবহারকারী, ডাটাবেস, বাইরের API, ফাইল সিস্টেম — একটি করে বিশ্বাসের সীমানা হিসেবে বিবেচিত হয়।

৬.২ পূর্বশর্ত পুনরালোচনা

এই অধ্যায়ে ঢোকার আগে শিক্ষার্থীর JavaScript ও Node.js-এর মৌলিক জ্ঞান থাকা দরকার: Express-এ রাউট, মিডলওয়্যার ও `req – res` চক্র কীভাবে কাজ করে, MongoDB-তে Mongoose মডেল কীভাবে তৈরি হয়, এবং React-এ ব্যবহারকারীর ইনপুট কীভাবে রাষ্ট্রে জমা হয় ও সার্ভারে পাঠানো হয়। HTTP-র মৌলিক ধারণাও দরকার — অনুরোধের হেডার, বডি, কুকি ও স্ট্যাটাস কোড — কারণ নিরাপত্তার বহু সিদ্ধান্ত এই স্তরেই নেওয়া হয়। আগের খণ্ডগুলো থেকে CI/CD, Docker ও পরিবেশ ভেরিয়েবলের ধারণা থাকলে ভালো হয়, কারণ গোপন তথ্য কোডে না রেখে পরিবেশে রাখার অভ্যাস এখানে বারবার আসবে। যারা পাসওয়ার্ড হ্যাশিং সম্পর্কে আগে শোনেনি, তাদের জন্য সংক্ষেপে মনে রাখা যায়: হ্যাশ হলো একমুখী রূপান্তর — পাসওয়ার্ড থেকে হ্যাশ তৈরি করা যায়, কিন্তু হ্যাশ থেকে পাসওয়ার্ড ফিরিয়ে আনা যায় না। আর সেশন হলো সার্ভার ও ব্রাউজারের মধ্যে একটি চলমান পরিচয়ের প্রমাণ, সাধারণত কুকির মাধ্যমে বহন করা হয়। এই দুটি ধারণা না বুঝলে পরের অংশগুলো কঠিন লাগবে, তাই প্রয়োজনে এখানে থেমে সেগুলো ঝালিয়ে নেওয়া উচিত।

৬.৩ প্রথম নীতি থেকে ব্যাখ্যা: নিরাপদ কোডিং কেন দরকার

একটি অ্যাপ্লিকেশন মূলত একটি বিশ্বাসের যন্ত্র। ব্যবহারকারী তার নাম, পাসওয়ার্ড, ঠিকানা, টাকার তথ্য অ্যাপকে দেয় এই বিশ্বাসে যে অ্যাপ সেগুলো শুধু তার অনুমোদিত কাজেই ব্যবহার করবে। কিন্তু অ্যাপের প্রতিটি প্রবেশপথ — একটি ফর্মের ক্ষেত্র, একটি URL প্যারামিটার, একটি API-র JSON বডি, একটি আপলোড করা ফাইল, এমনকি একটি HTTP হেডার — আক্রমণকারীর কাছেও একই পথ। নিরাপদ কোডিংয়ের মূল সমস্যা তাই এই: প্রোগ্রাম যেভাবে ডেটাকে দেখে, আক্রমণকারী সেভাবে দেখে না। প্রোগ্রামার যখন একটি ক্ষেত্রকে ভাবে "ব্যবহারকারীর নাম", আক্রমণকারী সেটিকে দেখে "ডাটাবেসে পাঠানো যাবে এমন একটি নির্দেশের টুকরো" হিসেবে। যখন প্রোগ্রামার ভাবে "এই সংখ্যাটি পণ্যের দাম", আক্রমণকারী ভাবে "এই সংখ্যাটি বদলে দিলে কি বিনামূল্যে পণ্য পাওয়া যাবে?" এই দৃষ্টিভঙ্গির পার্থক্যই প্রায় সব অ্যাপ্লিকেশন দুর্বলতার জন্ম দেয়।

এই সমস্যা ঘটে কারণ কম্পিউটার ডেটা ও নির্দেশের মধ্যে নিজে থেকে পার্থক্য করে না। একটি স্ট্রিং কখনো শুধুই লেখা, আবার কখনো SQL কোয়েরি, কখনো HTML, কখনো শেল কমান্ড — পার্থক্যটা করে প্রসঙ্গ, অর্থাৎ সেই স্ট্রিংটিকে কোথায় ব্যবহার করা হচ্ছে। নিরাপদ কোডিং তাই মূলত প্রসঙ্গ-সচেতন কোডিং: প্রতিটি ডেটা কোন প্রসঙ্গে যাচ্ছে তা বুঝে, সেই প্রসঙ্গের নিয়ম অনুযায়ী তাকে প্রস্তুত করা। ডাটাবেসে যাওয়ার আগে প্যারামিটারাইজ করা, ব্রাউজারে দেখানোর আগে এনকোড করা,

শেলে পাঠানোর আগে এড়িয়ে যাওয়া বা কঠোরভাবে সীমাবদ্ধ করা — এগুলো আলাদা আলাদা কৌশল মনে হলেও এদের মূলে একই নীতি: ডেটাকে কখনো নির্দেশ হিসেবে ব্যাখ্যা হতে দেওয়া যাবে না।

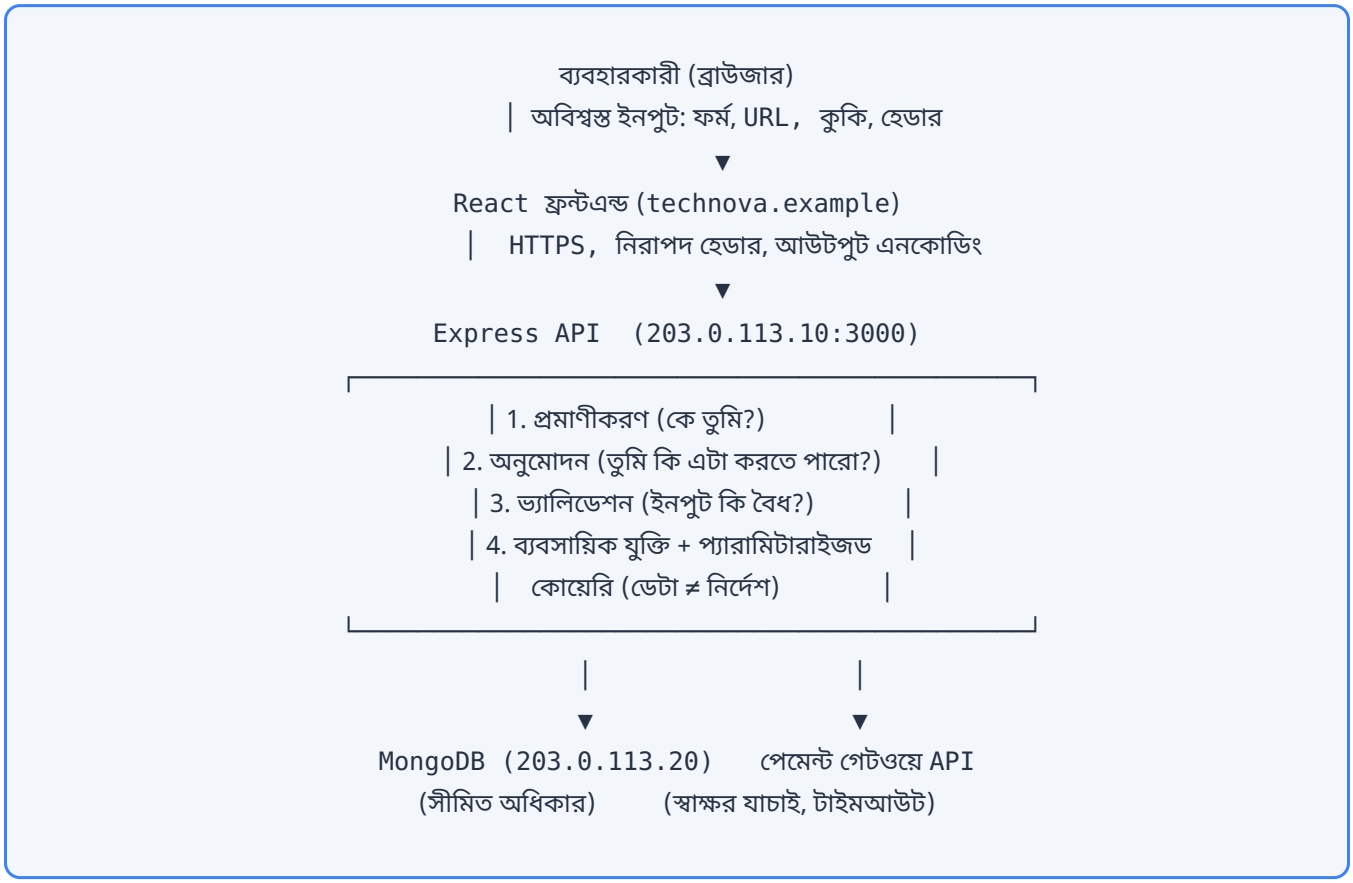
আরেকটি প্রথম নীতি হলো সর্বনিম্ন সুবিধার নীতি — least privilege। একটি ফাংশন, একটি ব্যবহারকারী, একটি সার্ভিস অ্যাকাউন্ট — প্রত্যেকের শুধু ততটুকু ক্ষমতা থাকা উচিত যতটুকু তার কাজের জন্য একান্ত দরকার। কারণ কোনো অংশ আপসিত হলে ক্ষতির পরিমাণ নির্ভর করে সেই অংশের ক্ষমতার ওপর, আক্রমণকারীর দক্ষতার ওপর নয়। একটি রিড-অনলি ডাটাবেস ব্যবহারকারী দিয়ে ডেটা মুছে ফেলা যায় না; একটি শুধু নিজের প্রোফাইল দেখতে পারা টোকেন দিয়ে অন্যের প্রোফাইল বদলানো যায় না — যদি অনুমোদন ঠিকমতো পরীক্ষা করা হয়। একইভাবে প্রতিরক্ষার গভীরতা — defence in depth — বলে একটিমাত্র নিয়ন্ত্রণের ওপর ভরসা না করে একাধিক স্তরে নিয়ন্ত্রণ বসাতে: ভ্যালিডেশন, প্যারামিটারাইজড কোয়েরি, অনুমোদন পরীক্ষা, লগিং ও পর্যবেক্ষণ একসঙ্গে কাজ করলে একটি স্তর ভেঙে গেলেও পরের স্তর আক্রমণ থামাতে পারে।

> **CONCEPT:** নিরাপত্তা কোনো বৈশিষ্ট্য নয়, এটি একটি বৈশিষ্ট্য-বিরোধী গুণ — অর্থাৎ এটি সেই সব কাজ, যা অ্যাপ্লিকেশনকে করতে দেওয়া হয় না। ভালো নিরাপদ কোড লেখা মানে ব্যবহারকারীর বৈধ কাজকে বাধা না দিয়ে অবৈধ কাজকে অসম্ভব বা অত্যন্ত কঠিন করে তোলা। এই ভারসাম্যই প্রকৌশলীর বিচারবুদ্ধির পরীক্ষা।

সবশেষে মনে রাখতে হবে, নিরাপত্তা স্ক্যানিং আর নিরাপত্তা প্রকৌশল এক নয়। স্ক্যানার একটি পরিচিত ভুলের ধরন খুঁজে পেতে পারে, কিন্তু সে বুঝতে পারে না ব্যবসায়িক যুক্তিতে কী ভুল আছে — যেমন একজন ব্যবহারকারী নিজের ছাড়ের কুপন অন্যের অ্যাকাউন্টে প্রয়োগ করছে, বা দাম বদলে অর্ডার করছে। এই ধরনের যুক্তিগত ত্রুটি ধরতে পারে শুধু মানুষ, যে ব্যবসার নিয়ম বোঝে। তাই স্ক্যানার সহায়ক, কিন্তু তার ওপর নির্ভর করে নিরাপত্তা সম্পূর্ণ মনে করা ভুল।

৬.৪ স্থাপত্য: TechNova MERN অ্যাপে বিশ্বাসের সীমানা

TechNova-র কাল্পনিক অ্যাপটি ধরা যাক: React ফ্রন্টএন্ড, Express API, MongoDB ডাটাবেস, এবং একটি পেমেন্ট গেটওয়ের সঙ্গে সংযোগ। ব্যবহারকারীর ব্রাউজার থেকে অনুরোধ আসে, Nginx বা লোড ব্যালান্সার পেরিয়ে Express-এ পৌঁছায়, সেখান থেকে Mongoose হয়ে MongoDB-তে যায়, আর পেমেন্টের সময় বাইরের গেটওয়ের API-তে যায়। প্রতিটি সংযোগস্থল একটি বিশ্বাসের সীমানা — trust boundary। ব্রাউজার থেকে আসা সবকিছু অবিশ্বস্ত; ডাটাবেস থেকে আসা ডেটাও সম্পূর্ণ বিশ্বস্ত নয়, কারণ সেখানে আগে জমা হওয়া ক্ষতিকর ডেটা থাকতে পারে — একে বলে stored আক্রমণ। বাইরের API-র উত্তরও যাচাই ছাড়া বিশ্বাস করা যায় না।



এই চিত্রের গুরুত্বপূর্ণ শিক্ষা হলো, নিরাপত্তা পরীক্ষা শুধু এক জায়গায় নয় — সীমানা পেরোনোর প্রতিটি ধাপে। ফ্রন্টএন্ডে ভ্যালিডেশন ব্যবহারকারীর অভিজ্ঞতা ভালো করে, কিন্তু নিরাপত্তা দেয় না, কারণ আক্রমণকারী ব্রাউজার বাইপাস করে সরাসরি API-তে অনুরোধ পাঠাতে পারে। তাই সার্ভার-সাইড ভ্যালিডেশনই আসল প্রতিরক্ষা; ক্লায়েন্ট-সাইড ভ্যালিডেশন শুধু সুবিধা। একইভাবে React স্বয়ংক্রিয়ভাবে কিছু XSS থেকে রক্ষা করে, কিন্তু `dangerouslySetInnerHTML` ব্যবহার করলেই সেই সুরক্ষা ভেঙে যায়।

৬.৫ অভ্যন্তরীণ কার্যপ্রণালী

ইনপুট ভ্যালিডেশন। ভ্যালিডেশন মানে ইনপুটকে একটি অনুমোদিত আকৃতির সঙ্গে মেলানো — যাকে বলে allowlist। একটি ইমেইল ক্ষেত্রে শুধু বৈধ ইমেইল আকৃতি চলবে, বয়সের ক্ষেত্রে শুধু ১৮ থেকে ১২০-এর মধ্যে পূর্ণসংখ্যা, ভূমিকার ক্ষেত্রে শুধু `user` বা `admin`-এর মতো নির্দিষ্ট মান। Denylist — অর্থাৎ খারাপ শব্দের তালিকা বানিয়ে সেগুলো আটকানো — প্রায় সবসময় ব্যর্থ হয়, কারণ আক্রমণকারী নতুন উপায়ে একই জিনিস লিখতে পারে। ভ্যালিডেশন তিনটি স্তরে ভাবা উচিত: আকৃতি (টাইপ, দৈর্ঘ্য, বিন্যাস), অর্থ (সীমার মধ্যে মান, ব্যবসায়িক নিয়ম) এবং প্রসঙ্গ (এই ব্যবহারকারী এই মানটি পাঠাতে পারে কি না)। Zod বা Joi-এর মতো স্কিমা ভ্যালিডেটর এই কাজকে ঘোষণামূলক করে — নিয়ম এক জায়গায় লেখা থাকে, প্রতিটি রাউটে ছড়িয়ে থাকে না।

আউটপুট এনকোডিং। ভ্যালিডেশন ইনপুট ঢোকার সময় কাজ করে; এনকোডিং কাজ করে আউটপুট বেরোনোর সময়। একই ডেটা HTML-এ দেখাতে হলে `<` কে `<` করতে হয়, JavaScript-এ বসাতে হলে অন্য নিয়ম, URL-এ বসাতে হলে আরেক নিয়ম। ভুল ধারণা হলো ইনপুট পরিষ্কার করে নিলেই আর এনকোডিং দরকার নেই — কিন্তু ডেটা ডাটাবেসে জমা হওয়ার পর, অন্য সিস্টেম থেকে আসার পর, বা পরে কোনো নতুন প্রসঙ্গে ব্যবহৃত হলে সেই "পরিষ্কার" ধারণা ভেঙে যায়। তাই নিয়ম হলো: সংরক্ষণ করো আসল ডেটা, এনকোড করো ব্যবহারের মুহূর্তে, সেই প্রসঙ্গের নিয়মে।

প্রমাণীকরণ ও পাসওয়ার্ড। পাসওয়ার্ড কখনোই সরল পাঠ্যে সংরক্ষণ করা যায় না। bcrypt বা Argon2-এর মতো ধীর, লবণযুক্ত হ্যাশ ব্যবহার করতে হয় — ধীর হওয়াটা এখানে গুণ, কারণ আক্রমণকারী কোটি কোটি অনুমান চালাতে গেলে সময়

লাগবে। লবণ প্রতিটি পাসওয়ার্ডকে আলাদা করে, যাতে একই পাসওয়ার্ডের হ্যাশও আলাদা হয়। পাসওয়ার্ড নীতি হিসেবে দৈর্ঘ্যকেই প্রাধান্য দিতে হয় — আট অক্ষরের জটিল পাসওয়ার্ডের চেয়ে বারো অক্ষরের সহজ বাক্যাংশ ভালো হতে পারে — এবং পরিচিত ফাঁস হওয়া পাসওয়ার্ডের তালিকার বিরুদ্ধে পরীক্ষা করা উচিত। ব্যর্থ লগইন প্রচেষ্টায় হার-সীমা (rate limiting) ও লকআউট থাকা দরকার, তবে এমনভাবে যাতে বৈধ ব্যবহারকারী হয়রানির শিকার না হয়।

সেশন ব্যবস্থাপনা। লগইনের পর সার্ভার একটি সেশন শনাক্তকারী দেয়, যা ব্রাউজার কুকিতে রাখে। নিরাপদ সেশন মানে: শনাক্তকারী দীর্ঘ, এলোমেলো ও অনুমান-অযোগ্য; কুকিতে `HttpOnly` (JavaScript পড়তে পারবে না), `Secure` (শুধু HTTPS-এ যাবে) ও `SameSite` (ক্রস-সাইট অনুরোধে যাবে না বা সীমিত যাবে) ফ্ল্যাগ; লগইনের পর সেশন ID বদলে দেওয়া, যাতে আগের চুরি যাওয়া ID কাজে না লাগে; নিষ্ক্রিয়তার পর মেয়াদ শেষ হওয়া; এবং লগআউটে সার্ভার-সাইডে সেশন মুছে ফেলা। JWT ব্যবহার করলে মনে রাখতে হবে JWT নিজে থেকে বাতিল করা যায় না — তাই সংক্ষিপ্ত মেয়াদ, রিফ্রেশ টোকেনের আবর্তন ও বাতিল তালিকার কৌশল দরকার।

অনুমোদন। প্রমাণীকরণ বলে "তুমি কে", অনুমোদন বলে "তুমি কী করতে পারো"। সবচেয়ে সাধারণ ভুল হলো প্রমাণীকরণ পরীক্ষা করেই অনুমোদন ধরে নেওয়া — যেমন লগইন করা ব্যবহারকারীকে অন্যের অর্ডার দেখতে দেওয়া, কারণ URL-এ শুধু ID বদলে দেওয়া যায়। এই দুর্বলতাকে বলে IDOR — Insecure Direct Object Reference। সঠিক নিয়ম: প্রতিটি সংবেদনশীল অপারেশনে সার্ভার-সাইডে পরীক্ষা করো, এই নির্দিষ্ট ব্যবহারকারী এই নির্দিষ্ট বস্তুটির ওপর এই নির্দিষ্ট কাজটি করতে পারে কি না। ফ্রন্টএন্ডে বোতাম লুকানো অনুমোদন নয় — সেটি শুধু প্রদর্শন।

ত্রুটি ব্যবস্থাপনা ও লগিং নিরাপত্তা। ত্রুটি বার্তা ব্যবহারকারীকে সাহায্য করবে, আক্রমণকারীকে নয়। "এই ইমেইল দিয়ে অ্যাকাউন্ট নেই" বলা আক্রমণকারীকে জানিয়ে দেয় কোন ইমেইল নিবন্ধিত; বরং "ইমেইল বা পাসওয়ার্ড ভুল" — একই বার্তা দুই ক্ষেত্রেই — ব্যবহার করা উচিত। স্ট্যাক ট্রেস, ডাটাবেস ত্রুটি, ফাইলের পথ — এগুলো কখনো ব্যবহারকারীকে দেখানো যাবে না; সেগুলো শুধু সার্ভার লগে যাবে, যেখানে সেগুলো দেখার অধিকার সীমিত। লগে পাসওয়ার্ড, টোকেন, ক্রেডিট কার্ড নম্বর, ব্যক্তিগত শনাক্তকারী তথ্য কখনো লেখা যাবে না। লগ লেখার আগে সংবেদনশীল ক্ষেত্রগুলোকে মাস্ক বা বাদ দিতে হয়, এবং লগ সংরক্ষণের মেয়াদ ও প্রবেশাধিকার নির্ধারণ করতে হয়।

> **WARNING:** `eval()`, `Function()` কনস্ট্রাক্টর, বা ব্যবহারকারীর ইনপুট দিয়ে তৈরি স্ট্রিংকে কোড হিসেবে চালানো — এগুলো প্রায় সবসময়ই বিপজ্জনক। একইভাবে `$where` বা অবজেক্ট ইনজেকশনের মতো MongoDB অপারেটরে ব্যবহারকারীর অবজেক্ট সরাসরি পাঠানো যাবে না। সন্দেহ হলে সেই প্যাটার্নটি এড়িয়ে যাওয়াই নিরাপদ।

৬.৬ ব্যবহারিক কমান্ড ও কনফিগারেশন: দুর্বল বনাম নিরাপদ কোড

নিচের উদাহরণগুলো TechNova-র MERN অ্যাপ থেকে নেওয়া। প্রতিটি জোড়ায় প্রথমে দুর্বল কোড, তারপর নিরাপদ কোড দেখানো হয়েছে। পার্থক্যটা মনোযোগ দিয়ে পড়ো — নিরাপদ সংস্করণে কী যোগ হয়েছে, আর কেন সেটি দরকার।

নোএসকিউএল ইনজেকশন — দুর্বল: ব্যবহারকারীর পাঠানো অবজেক্ট সরাসরি কোয়েরিতে গেলে আক্রমণকারী অপারেটর ঢুকিয়ে দিতে পারে।

```
// দুর্বল — req.body সরাসরি কোয়েরিতে
app.post('/api/login', async (req, res) => {
  const user = await User.findOne({ email: req.body.email, password: req.body.password });
  if (user) return res.json({ ok: true });
  res.status(401).json({ error: 'Login failed' });
});
// আক্রমণ: { "email": { "$gt": "" }, "password": { "$gt": "" } } পাঠালে
// কোয়েরি সব ব্যবহারকারীকে মেলাতে পারে।
```

```
// নিরাপদ — টাইপ যাচাই + bcrypt তুলনা + প্যারামিটারাইজড কোয়েরি
const bcrypt = require('bcrypt');
const { z } = require('zod');
const loginSchema = z.object({
  email: z.string().email().max(254),
  password: z.string().min(8).max(128)
});

app.post('/api/login', async (req, res) => {
  const parsed = loginSchema.safeParse(req.body);
  if (!parsed.success) return res.status(400).json({ error: 'Invalid input' });
  const { email, password } = parsed.data;
  const user = await User.findOne({ email }).select('+passwordHash');
  if (!user) return res.status(401).json({ error: 'Email or password is incorrect' });
  const ok = await bcrypt.compare(password, user.passwordHash);
  if (!ok) return res.status(401).json({ error: 'Email or password is incorrect' });
  // সফল হলে সেশন তৈরি করো, লগইনের পর সেশন ID বদলে দাও
  req.session.regenerate(() => {
    req.session.userId = user._id.toString();
    req.session.role = user.role;
    res.json({ ok: true });
  });
});
```

অনুমোদন — দুর্বল বনাম নিরাপদ:

```
// দুর্বল — শুধু লগইন পরীক্ষা, মালিকানা পরীক্ষা নেই
app.get('/api/orders/:id', requireLogin, async (req, res) => {
  const order = await Order.findById(req.params.id);
  res.json(order); // যে কেউ যেকোনো ID দিয়ে অন্যের অর্ডার দেখতে পারে
});

// নিরাপদ — মালিকানা + ভূমিকা পরীক্ষা
app.get('/api/orders/:id', requireLogin, async (req, res) => {
  const order = await Order.findById(req.params.id);
  if (!order) return res.status(404).json({ error: 'Not found' });
  const isOwner = order.userId.toString() === req.session.userId;
  const isAdmin = req.session.role === 'admin';
  if (!isOwner && !isAdmin) return res.status(403).json({ error: 'Forbidden' });
  res.json(order);
});
```

নিরাপদ কুকি ও হেডার:

```
const helmet = require('helmet');
const session = require('express-session');
app.use(helmet()); // নিরাপদ HTTP হেডার
app.use(session({
  secret: process.env.SESSION_SECRET, // CHANGE-ME-... নয়, পরিবেশে রাখো
  resave: false,
  saveUninitialized: false,
  cookie: {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production', // শুধু HTTPS
    sameSite: 'lax',
    maxAge: 1000 * 60 * 30 // ৩০ মিনিট নিষ্ক্রিয়তা
```

```
  }
}));
```

সিক্রেট ব্যবস্থাপনা ও নির্ভরতা পরীক্ষা:

```
# সিক্রেট কখনো কোডে নয় — .env ফাইলে, যা git-এ যাবে না
# .env (উদাহরণ, আসল মান নয়)
SESSION_SECRET=CHANGE-ME-long-random-secret
MONGODB_URI=mongodb://203.0.113.20:27017/technova
# .gitignore-এ অবশ্যই .env থাকতে হবে

# নির্ভরতার পরিচিত দুর্বলতা পরীক্ষা
npm audit
npm audit fix
# নিয়মিত আপডেটের জন্য Dependabot বা Renovate চালু রাখো
```

৬.৭ প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — দাম বদলের আক্রমণ। TechNova-র চেকআউটে ফ্রন্টএন্ড থেকে পণ্যের দাম পাঠানো হতো। একজন ব্যবহারকারী ব্রাউজারের ডেভেলপার টুল দিয়ে দাম শূন্য করে অর্ডার করল। শিক্ষা: দাম, ছাড়, কর — কোনো ব্যবসায়িক মানই ক্লায়েন্ট থেকে বিশ্বাস করা যাবে না; সার্ভারকে ডাটাবেস থেকে দাম নিয়ে নিজে হিসাব করতে হবে। ক্লায়েন্ট শুধু পণ্যের ID ও পরিমাণ পাঠাবে।

পরিস্থিতি ২ — লগে পাসওয়ার্ড। ডিবাগ করতে গিয়ে একজন ডেভেলপার পুরো `req.body` লগ করলেন। সেই লগ কেন্দ্রীয় লগ সার্ভারে গেল, যেখানে বহু মানুষের প্রবেশাধিকার ছিল। ফলে পাসওয়ার্ড ছড়িয়ে পড়ল লগের মাধ্যমে। শিক্ষা: লগ লেখার আগে সংবেদনশীল ক্ষেত্র বাদ দাও, আর লগের প্রবেশাধিকারও ডাটাবেসের মতোই সীমিত রাখো।

পরিস্থিতি ৩ — ভাঙা সেশন। একজন ব্যবহারকারী পাবলিক কম্পিউটারে লগইন করে ব্রাউজার বন্ধ করে চলে গেলেন, কিন্তু সার্ভারে সেশন রয়ে গেল। পরের ব্যবহারকারী একই ব্রাউজার খুলে অ্যাকাউন্টে ঢুকে গেল। শিক্ষা: নিষ্ক্রিয়তার মেয়াদ সংক্ষিপ্ত রাখো, লগআউটে সার্ভার-সাইড সেশন মুছে ফেলো, আর সংবেদনশীল কাজের আগে আবার পাসওয়ার্ড চাওয়ার ব্যবস্থা রাখো।

> **REAL WORLD:** বাস্তব ঘটনাগুলোতে দেখা যায়, বড় ক্ষতির পেছনে প্রায়ই কোনো জটিল আক্রমণ থাকে না — থাকে একটি সহজ ভুল: সার্ভার-সাইড পরীক্ষা না থাকা, ডিফল্ট পাসওয়ার্ড বদলানো না হওয়া, বা লগে সিক্রেট লেখা। তাই নিরাপত্তা প্রকৌশলের সবচেয়ে বড় মূল্য হলো সহজ ভুলগুলোকে পদ্ধতিগতভাবে অসম্ভব করে তোলা — কোড রিভিউ চেকলিস্ট, স্বয়ংক্রিয় লিন্টার, আর পরীক্ষায় আক্রমণ-কেস লেখা।

৬.৮ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তার খরচ শুধু টাকায় নয় — সময়, জটিলতা ও ব্যবহারকারীর সুবিধাতেও। অতিরিক্ত কঠোর পাসওয়ার্ড নীতি ব্যবহারকারীকে বিরক্ত করে, আর অতিরিক্ত লকআউট বৈধ ব্যবহারকারীকে আটকে দেয়। প্রতিটি নিয়ন্ত্রণের আগে জিজ্ঞেস করো: এই সম্পদ কতটা মূল্যবান, হুমকি কে, আর নিয়ন্ত্রণটি আক্রমণকারীর খরচ কতটা বাড়ায়? পাসওয়ার্ড হ্যাশে bcrypt-এর খরচ-গুণক বাড়ালে লগইন ধীর হয় — প্রতি লগইনে কয়েকশো মিলিসেকেন্ড — কিন্তু আক্রমণকারীর ব্রুট-ফোর্স খরচ হাজার গুণ বাড়ে। এই লেনদেনটি গ্রহণযোগ্য। আবার প্রতিটি অনুরোধে ভারী ক্রিপ্টো করা বা প্রতিটি পৃষ্ঠায় একাধিক বাইরের যাচাই চালানো কর্মক্ষমতা নষ্ট করতে পারে। তাই নিরাপত্তা ও কর্মক্ষমতার ভারসাম্য মাপতে হয় বাস্তব পরিমাপ দিয়ে, অনুমানে নয়। লগ সংরক্ষণের খরচও ভাবতে হয়: সবকিছু লগ করলে স্টোরেজ খরচ বাড়ে, আবার কিছুই না লগ করলে ঘটনা ঘটার

পর তদন্ত করা যায় না। মাঝামাঝি পথ হলো: নিরাপত্তা-প্রাসঙ্গিক ঘটনা — লগইন সাফল্য ও ব্যর্থতা, অনুমোদন ব্যর্থতা, সন্দেহজনক ইনপুট — বিস্তারিত লগ করা, আর সাধারণ ঘটনা সংক্ষেপে।

৬.৯ সাধারণ ভুল

- ক্লায়েন্ট-সাইড ভ্যালিডেশনকে নিরাপত্তা ভাবা — আক্রমণকারী ব্রাউজার বাইপাস করতে পারে।
- পাসওয়ার্ড সরল পাঠ্যে বা দুর্বল হ্যাশে (MD5, SHA-1) রাখা, অথবা লবণ ছাড়া হ্যাশ করা।
- ত্রুটি বার্তায় "অ্যাকাউন্ট নেই" বনাম "পাসওয়ার্ড ভুল" আলাদা করে জানানো — ব্যবহারকারী শনাক্তকরণ ফাঁস হয়।
- URL-এ ID বদলে অন্যের ডেটা দেখা যাওয়া — অনুমোদন পরীক্ষা না থাকা।
- লগে পাসওয়ার্ড, টোকেন বা পূর্ণ `req.body` লেখা।
- সিক্রেট কোডে বা git-এ কমিট করা; `.env` ভুলে প্রকাশ করা।
- `dangerouslySetInnerHTML`-এ ব্যবহারকারীর HTML সরাসরি বসানো।
- নিয়মিত নির্ভরতা আপডেট না করা — পুরোনো লাইব্রেরিতে পরিচিত দুর্বলতা থেকে যাওয়া।

৬.১০ সমস্যা সমাধান

সমস্যা: বৈধ ব্যবহারকারী লগইন করতে পারছে না। প্রথমে দেখো হ্যাশ তুলনা ঠিক আছে কি না, পাসওয়ার্ড ক্ষেত্রের দৈর্ঘ্য ডাটাবেসে কাটা যাচ্ছে কি না, আর হার-সীমা বৈধ ব্যবহারকারীকে আটকাচ্ছে কি না। লগে শুধু "লগইন ব্যর্থ" লেখা থাকলে কারণ বোঝা যাবে না — সার্ভার-সাইড লগে কারণের কোড রাখো, ব্যবহারকারীকে সাধারণ বার্তা দাও।

সমস্যা: সেশন হঠাৎ শেষ হয়ে যাচ্ছে। কুকির `Secure` ফ্ল্যাগ HTTP পরিবেশে থাকলে ব্রাউজার কুকি পাঠাবে না; লোড ব্যালান্সারের পেছনে সেশন স্টোর ভাগ করা না থাকলে এক সার্ভারের সেশন অন্য সার্ভার চিনবে না। তাই প্রোডাকশনে Redis-এর মতো ভাগ করা সেশন স্টোর ব্যবহার করো।

সমস্যা: ভ্যালিডেটর বৈধ ইনপুট আটকাচ্ছে। সম্ভবত স্কিমা খুব কঠোর — যেমন নামে ইউনিকোড অক্ষর বা হাইফেন আটকে যাচ্ছে। Allowlist বাস্তবসম্মত রাখো, আর ভ্যালিডেশন ব্যর্থতার বার্তায় কোন নিয়ম ভাঙল তা স্পষ্ট করো — তবে আক্রমণকারীকে ভেতরের নিয়মের বিস্তারিত জানিও না।

> **TROUBLESHOOTING:** নিরাপত্তা সমস্যা ডিবাগ করার সময় কখনো প্রোডাকশনে লগিং বাড়িয়ে সংবেদনশীল ডেটা লগ করো না। সমস্যাটি স্টেজিং পরিবেশে পুনরায় তৈরি করো, সেখানে অস্থায়ীভাবে বিস্তারিত লগ চালু করো, কাজ শেষে তা বন্ধ করো ও লগ মুছে ফেলো।

৬.১১ সারাংশ

নিরাপদ কোডিংয়ের মূল কথা: কোনো ইনপুটকে বিশ্বাস করো না, ডেটাকে নির্দেশ হতে দিও না, প্রমাণীকরণ ও অনুমোদন আলাদা করে পরীক্ষা করো, সেশনকে সুরক্ষিতভাবে বহন করো, ত্রুটি ও লগে সংবেদনশীল তথ্য ফাঁস করো না, আর সিক্রেট কোডের বাইরে রাখো। এই নীতিগুলো একবার অভ্যাসে পরিণত হলে প্রতিটি নতুন বৈশিষ্ট্য লেখার সময় নিরাপত্তা আলাদা করে ভাবতে হয় না — সেটি কোডের গঠনের ভেতরেই থাকে। মনে রাখো, স্ক্যানার সাহায্য করতে পারে, কিন্তু ব্যবসায়িক যুক্তির নিরাপত্তা মানুষকেই নিশ্চিত করতে হয়।

৬.১২ পর্যালোচনা প্রশ্ন

1. ইনপুট ভ্যালিডেশন ও আউটপুট এনকোডিংয়ের মধ্যে পার্থক্য কী? একটির অভাব অন্যটি দিয়ে পূরণ করা যায় না কেন?
2. Allowlist ও denylist ভ্যালিডেশনের মধ্যে কোনটি নিরাপদ এবং কেন? একটি উদাহরণ দাও।
3. NoSQL ইনজেকশন কীভাবে ঘটে? Mongoose-এ `{ "$gt": "" }` পাঠালে কী হয় তা ব্যাখ্যা করো।
4. প্রমাণীকরণ ও অনুমোদনের পার্থক্য ব্যাখ্যা করো। IDOR কী এবং তা কীভাবে ঠেকানো যায়?
5. bcrypt কেন MD5 বা SHA-256-এর চেয়ে পাসওয়ার্ডের জন্য ভালো? লবণের ভূমিকা কী?
6. নিরাপদ সেশন কুকিতে কোন কোন ফ্ল্যাগ থাকা উচিত এবং প্রতিটি কী রক্ষা করে?
7. লগইন ব্যর্থতার বার্তায় "অ্যাকাউন্ট নেই" বলা কেন বিপজ্জনক?
8. লগে কী কী লেখা যাবে না এবং কেন? লগের প্রবেশাধিকার কেন গুরুত্বপূর্ণ?
9. সর্বনিম্ন সুবিধার নীতি ও প্রতিরক্ষার গভীরতা বলতে কী বোঝায়? প্রতিটির একটি করে উদাহরণ দাও।
10. স্ক্যানিং বনাম নিরাপত্তা প্রকৌশলের পার্থক্য কী? স্ক্যানার কোন ধরনের ত্রুটি ধরতে পারে না?

৬.১৩ ব্যবহারিক অনুশীলন

1. **দুর্বল লগইন ঠিক করো:** নিজের বিচ্ছিন্ন ল্যাবে একটি Express লগইন রাউট লেখো যেখানে প্রথমে ইচ্ছাকৃতভাবে NoSQL ইনজেকশন সম্ভব রাখো, তারপর Zod ভ্যালিডেশন ও bcrypt দিয়ে তা ঠিক করো। আক্রমণ-প্রচেষ্টা ও ঠিক করা সংস্করণ — দুটোরই পরীক্ষা লেখো।
2. **IDOR পরীক্ষা:** দুটি ব্যবহারকারী ও তাদের অর্ডার তৈরি করো। প্রথম ব্যবহারকারী হিসেবে লগইন করে দ্বিতীয় ব্যবহারকারীর অর্ডার ID দিয়ে অনুরোধ পাঠাও। অনুমোদন পরীক্ষা যোগ করার আগে ও পরে ফলাফল লিপিবদ্ধ করো।
3. **লগ পরিষ্কার করো:** একটি রাউটে পুরো `req.body` লগ করা আছে — এমন কোড খুঁজে বের করে সংবেদনশীল ক্ষেত্র বাদ দিয়ে নিরাপদ লগার লেখো। পাসওয়ার্ড ও টোকেন মাস্ক হচ্ছে কি না, তা পরীক্ষা করো।
4. **সেশন কঠোর করো:** তোমার ল্যাব অ্যাপে `helmet`, নিরাপদ কুকি ফ্ল্যাগ ও ৩০ মিনিটের নিষ্ক্রিয় মেয়াদ চালু করো। ব্রাউজারের ডেভেলপার টুল দিয়ে কুকি পরীক্ষা করে দেখো ফ্ল্যাগগুলো ঠিক আছে কি না।

অধ্যায় ৭ • PART II — Application & Software Supply Chain Security

Chapter 7 — OWASP Top 10 and Web Security

৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে শিক্ষার্থী OWASP Top 10 (2021) তালিকার দশটি বিভাগের নাম, অর্থ ও পারস্পরিক পার্থক্য ব্যাখ্যা করতে পারবে এবং বুঝবে তালিকাটি কেন ঝুঁকির গুরুত্ব অনুযায়ী সাজানো হয়নি, বরং একটি সাধারণ ভাষা হিসেবে ব্যবহৃত হয়। সে Injection, Broken Access Control, Cryptographic Failures, Security Misconfiguration, SSRF, XSS ও CSRF-এর মতো আক্রমণ কীভাবে কাজ করে তা প্রথম নীতি থেকে বলতে পারবে এবং TechNova-র MERN অ্যাপে প্রতিটির প্রতিকার দেখাতে পারবে। শিক্ষার্থী নিরাপদ HTTP হেডার, API নিরাপত্তা ও নিরাপদ ল্যাব অনুশীলনের নিয়ম জানবে — বিশেষ করে শুধু ইচ্ছাকৃতভাবে দুর্বল প্রশিক্ষণ অ্যাপ, যেমন OWASP Juice Shop, বিচ্ছিন্ন পরিবেশে ব্যবহার করার নিয়ম। সবশেষে সে বুঝবে Top 10 একটি চেকলিস্ট নয়; এটি আলোচনার শুরু, শেষ নয় — এর বাইরে ব্যবসায়িক যুক্তির ত্রুটি, নকশার ত্রুটি ও প্রক্রিয়াগত দুর্বলতাও থাকে।

৭.২ পূর্বশর্ত পুনরালোচনা

অধ্যায় ৬ থেকে ভ্যালিডেশন, এনকোডিং, প্রমাণীকরণ ও অনুমোদনের ধারণা এখানে ভিত্তি হিসেবে কাজ করবে। HTTP-র ক্ষেত্রে অনুরোধের পদ্ধতি (GET, POST, PUT, DELETE), হেডার, কুকি ও স্ট্যাটাস কোড, এবং ব্রাউজারের same-origin নীতি — অর্থাৎ একটি ওয়েবসাইট অন্য ওয়েবসাইটের ডেটা সরাসরি পড়তে পারে না — এই ধারণাগুলো ঝালিয়ে নাও। SQL ও NoSQL ইনজেকশনের মৌলিক ধারণা, পাসওয়ার্ড হ্যাশিং এবং সেশন কুকির ফ্ল্যাগগুলোও মনে রাখা দরকার। OWASP সম্পর্কে সংক্ষেপে: OWASP হলো Open Web Application Security Project — একটি উন্মুক্ত সম্প্রদায় যারা ওয়েব নিরাপত্তার নির্দেশিকা, সরঞ্জাম ও প্রশিক্ষণ উপকরণ তৈরি করে। তাদের Top 10 তালিকাটি বাস্তব তথ্য ও সম্প্রদায়ের জরিপের ভিত্তিতে সবচেয়ে সাধারণ ও প্রভাবশালী দশটি ওয়েব অ্যাপ্লিকেশন ঝুঁকি তুলে ধরে। ২০২১ সংস্করণটিই এই অধ্যায়ের ভিত্তি, কারণ এটিই এই বইয়ের নির্দেশিত সংস্করণ এবং এখনো বহু চুক্তি, পরীক্ষা ও সরঞ্জামে উল্লেখ করা হয়।

৭.৩ প্রথম নীতি থেকে ব্যাখ্যা: OWASP Top 10 কী ও কেন

ওয়েব নিরাপত্তার সমস্যা হলো, হাজার হাজার সম্ভাব্য দুর্বলতা আছে, কিন্তু একটি দল সবকিছু একসঙ্গে মনে রাখতে পারে না। OWASP Top 10 সেই সমস্যার সমাধান হিসেবে দশটি পরিবার বানিয়েছে — প্রতিটি পরিবার একই মূল কারণের বহু দুর্বলতাকে একত্র করে। যেমন Injection পরিবারে SQL, NoSQL, কমান্ড ও LDAP ইনজেকশন সবাই আছে, কারণ এদের মূল কারণ এক: অবিশ্বস্ত ডেটাকে নির্দেশ হিসেবে ব্যাখ্যা করা। তালিকাটি বোঝার সবচেয়ে গুরুত্বপূর্ণ বিষয় হলো, ক্রমটি তীব্রতার ক্রম নয়। A01 মানে এই নয় যে Broken Access Control সবসময় A10 SSRF-এর চেয়ে ভয়াবহ; ক্রমটি প্রচলন ও প্রভাবের সমন্বয়ে তৈরি। একটি নির্দিষ্ট অ্যাপে A10-ই সবচেয়ে বিপজ্জনক হতে পারে, যদি সেই অ্যাপ অভ্যন্তরীণ সেবার সঙ্গে কথা বলে। তাই তালিকাটিকে অগ্রাধিকারের যন্ত্র হিসেবে নয়, বরং একটি অভিধান হিসেবে ব্যবহার করো: কোনো সমস্যার নাম দিলে দল একই জিনিস বোঝে, আর কোনো বিভাগ বাদ পড়ছে কি না তা পরীক্ষা করা যায়।

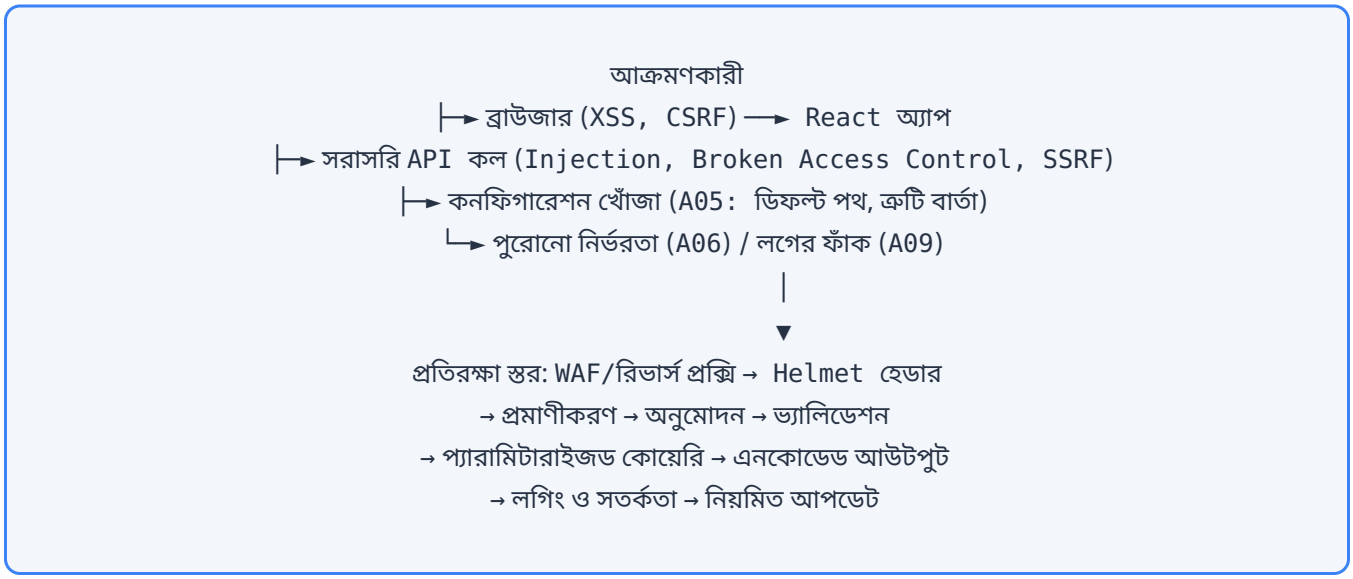
দ্বিতীয় নীতি হলো, তালিকার অর্ধেকের বেশি সমস্যার মূল কারণ কোডের একটি লাইনে নয়, সিদ্ধান্তের একটি স্তরে। Insecure Design বলে নকশাতেই যদি নিয়ন্ত্রণ না থাকে — যেমন পাসওয়ার্ড পুনরুদ্ধারে কোনো হার-সীমা না রাখা, বা ছাড়ের নিয়মে কোনো সীমা না রাখা — তবে যত ভালো কোডই লেখা হোক, আক্রমণ থামবে না। Security Misconfiguration বলে ডিফল্ট সেটিং, অপ্রয়োজনীয় বৈশিষ্ট্য ও অতিরিক্ত তথ্য ফাঁস থেকেই বহু আক্রমণ শুরু হয়। এই কারণে Top 10 শেখা মানে শুধু আক্রমণের নাম শেখা নয়; এটি শেখা যে নিরাপত্তা কোড, কনফিগারেশন, নকশা ও প্রক্রিয়া — এই চার স্তরেই বাস করে।

কোড	বিভাগ (OWASP Top 10:2021)	এক লাইনে অর্থ
A01	Broken Access Control	ব্যবহারকারী এমন কাজ করতে পারে যা তার করা উচিত নয়।
A02	Cryptographic Failures	সংবেদনশীল ডেটা পরিবহনে বা সংরক্ষণে ঠিকমতো সুরক্ষিত নয়।
A03	Injection	অবিশ্বস্ত ইনপুটকে নির্দেশ হিসেবে চালানো হয়।
A04	Insecure Design	নকশাতেই নিরাপত্তা নিয়ন্ত্রণের অভাব।
A05	Security Misconfiguration	ডিফল্ট, অপ্রয়োজনীয় বৈশিষ্ট্য ও ভুল কনফিগারেশন।
A06	Vulnerable and Outdated Components	পুরোনো বা দুর্বল লাইব্রেরি ও উপাদান ব্যবহার।
A07	Identification and Authentication Failures	দুর্বল পরিচয় যাচাই, পাসওয়ার্ড নীতি ও সেশন ব্যবস্থাপনা।
A08	Software and Data Integrity Failures	আপডেট, লাইব্রেরি ও পাইপলাইন যাচাই ছাড়া বিশ্বাস করা।
A09	Security Logging and Monitoring Failures	আক্রমণ ঘটে, কিন্তু সময়মতো কেউ টের পায় না।
A10	Server-Side Request Forgery (SSRF)	সার্ভারকে দিয়ে আক্রমণকারীর পছন্দের অনুরোধ পাঠানো।

এই তালিকা মুখস্থ করার আগে একটি বিষয় লক্ষ করো: ২০২১ সংস্করণে XSS আলাদা বিভাগ নয় — এটি এখন Injection (A03)-এর ভেতরে। একইভাবে XXE এখন Security Misconfiguration (A05)-এর ভেতরে। নতুন বিভাগ হিসেবে এসেছে Insecure Design (A04), Software and Data Integrity Failures (A08) ও SSRF (A10)। এই পরিবর্তনগুলোই বলে দেয়, OWASP শুধু নতুন আক্রমণ যোগ করে না — সে মূল কারণ অনুযায়ী তালিকাটিকে পুনর্গঠন করে।

> **CONCEPT:** Top 10-এর দশটি বিভাগকে দশটি আলাদা বাক্স ভেবো না; এগুলো দশটি দৃষ্টিকোণ। একটি আক্রমণ একাধিক বিভাগে পড়তে পারে — যেমন একটি দুর্বল পাসওয়ার্ড পুনরুদ্ধার প্রবাহ একইসঙ্গে A04 (নকশা) ও A07 (প্রমাণীকরণ) হতে পারে। বিভাগ নিয়ে তর্ক না করে মূল কারণটি ঠিক করাই প্রকৌশলীর কাজ।

৭.৪ স্থাপত্য: আক্রমণের পথ ও প্রতিরক্ষার স্তর



এই চিত্রের মূল বার্তা হলো, কোনো একক স্তর সব আক্রমণ থামাতে পারে না। XSS থামাতে দরকার আউটপুট এনকোডিং, Content Security Policy ও নিরাপদ কুকি — তিনটি একসঙ্গে। SSRF থামাতে দরকার URL যাচাই, allowlist ও নেটওয়ার্ক বিভাজন। Broken Access Control থামাতে দরকার সার্ভার-সাইড অনুমোদন পরীক্ষা, যা কোনো হেডার বা স্ক্যানার দিয়ে প্রতিস্থাপন করা যায় না।

৭.৫ অভ্যন্তরীণ কার্যপ্রণালী: প্রতিটি বিভাগ গভীরে

A01 Broken Access Control। এর মূলে আছে অনুমোদন পরীক্ষার অভাব বা ভুল পরীক্ষা। IDOR ছাড়াও এর মধ্যে পড়ে ভূমিকা পরিবর্তন করে অ্যাডমিন হওয়া, নিষ্ক্রিয় অ্যাকাউন্টের টোকেন দিয়ে প্রবেশ, ও CORS ভুল কনফিগারেশন। অভ্যন্তরীণভাবে এটি ঘটে কারণ ডেভেলপার ধরে নেন, ব্যবহারকারী শুধু সেই পথেই যাবে যা UI দেখায়। কিন্তু API সরাসরি ডাকা যায়। প্রতিকার: প্রতিটি রাউটে "এই ব্যবহারকারী, এই বস্তু, এই কাজ" — তিনটি পরীক্ষা; deny by default — অর্থাৎ অনুমতি না থাকলে প্রবেশ বন্ধ।

A02 Cryptographic Failures। আগে এর নাম ছিল Sensitive Data Exposure। নাম বদলের কারণ: সমস্যাটি ডেটা প্রকাশ পাওয়া নয়, ক্রিপ্টোগ্রাফি ঠিকমতো ব্যবহার না করা। দুর্বল অ্যালগরিদম, পুরোনো TLS, হার্ডকোড করা কী, পাসওয়ার্ড সরল পাঠ্যে রাখা, সংবেদনশীল ডেটা HTTP-তে পাঠানো — সবই এখানে। প্রতিকার: পরিবহনে TLS 1.2 বা তার চেয়ে নতুন, সংরক্ষণে শক্তিশালী অ্যালগরিদম, কী ব্যবস্থাপনা আলাদা সেবায়, আর সংবেদনশীল ক্ষেত্রে এনক্রিপশন।

A03 Injection ও XSS। Injection-এর ক্ষেত্রে আক্রমণকারী ডেটার ভেতরে নির্দেশ লুকিয়ে দেয়। XSS হলো এর একটি বিশেষ রূপ, যেখানে নির্দেশটি চালায় ভুক্তভোগীর ব্রাউজার। Stored XSS-এ ক্ষতিকর স্ক্রিপ্ট ডাটাবেসে জমা হয় ও পরে সবার ব্রাউজারে চলে; Reflected XSS-এ একটি লিংকের মাধ্যমে আসে; DOM-based XSS-এ ব্রাউজারের JavaScript-ই অনিরাপদভাবে DOM বদলায়। React ডিফল্টভাবে টেক্সট এস্কেপ করে, তাই সাধারণ JSX নিরাপদ, কিন্তু `dangerouslySetInnerHTML` বা `innerHTML` ব্যবহার করলেই সুরক্ষা ভেঙে যায়। প্রতিকার: প্যারামিটারাইজড কোয়েরি, প্রসঙ্গ-অনুযায়ী এনকোডিং, Content Security Policy।

CSRF। Cross-Site Request Forgery-তে আক্রমণকারী ভুক্তভোগীর ব্রাউজারকে দিয়ে ভুক্তভোগীর পরিচয়ে একটি অনুরোধ পাঠায় — যেমন একটি ক্ষতিকর পাতা থেকে TechNova-তে টাকা স্থানান্তরের ফর্ম জমা দেওয়া। ব্রাউজার কুকি স্বয়ংক্রিয়ভাবে পাঠায় বলেই এটি সম্ভব। প্রতিকার: `SameSite` কুকি, CSRF টোকেন (প্রতিটি ফর্মে একটি গোপন, একবারের মান যা সার্ভার যাচাই করে), এবং সংবেদনশীল কাজে GET নয়, POST ব্যবহার করা।

A04 Insecure Design। এটি কোডের ভুল নয়, নকশার ভুল। উদাহরণ: পাসওয়ার্ড পুনরুদ্ধারে সীমাহীন প্রচেষ্টা, কুপন বারবার ব্যবহার, বা অ্যাকাউন্ট মুছে ফেলার আগে নিশ্চিতকরণ না থাকা। প্রতিকার: হুমকি মডেলিং — অর্থাৎ নকশার সময়ই জিজ্ঞেস করা, "কে কীভাবে এটি ভাঙতে পারে?" — এবং নিরাপদ নকশার প্যাটার্ন ব্যবহার।

A05 Security Misconfiguration। ডিফল্ট পাসওয়ার্ড, অপ্রয়োজনীয় পোর্ট খোলা, বিস্তারিত ত্রুটি বার্তা, ডিরেক্টরি তালিকা চালু থাকা, নিরাপত্তা হেডার না থাকা — এগুলো সবই কনফিগারেশনের ভুল। প্রতিকার: কঠোর ডিফল্ট, অপ্রয়োজনীয় বৈশিষ্ট্য বন্ধ করা, পরিবেশ আলাদা করা, ও স্বয়ংক্রিয় কনফিগারেশন পরীক্ষা।

A06 Vulnerable and Outdated Components। পুরোনো লাইব্রেরি, ফ্রেমওয়ার্ক বা সার্ভার সফটওয়্যারে পরিচিত দুর্বলতা থাকলে আক্রমণকারী সহজে ঢুকতে পারে। প্রতিকার: নিয়মিত `npm audit`, নির্ভরতার তালিকা, ও আপডেটের নিয়মিত চক্র — এই বিষয়ে বিস্তারিত অধ্যায় ৮ ও ১০-এ আছে।

A07 Identification and Authentication Failures। দুর্বল পাসওয়ার্ড নীতি, ব্রুট-ফোর্স সুরক্ষা না থাকা, সেশন fixation, ও মাল্টি-ফ্যাক্টর প্রমাণীকরণের অভাব এখানে পড়ে। প্রতিকার: শক্তিশালী হ্যাশ, হার-সীমা, নিরাপদ সেশন ও প্রয়োজনে MFA।

A08 Software and Data Integrity Failures। আপডেট বা প্লাগইন যাচাই ছাড়া ইনস্টল করা, CI/CD পাইপলাইনে স্বাক্ষর যাচাই না করা — এগুলো সরবরাহ শৃঙ্খলের দুর্বলতা। বিস্তারিত অধ্যায় ১০-এ।

A09 Security Logging and Monitoring Failures। লগ না রাখা, লগে সংবেদনশীল তথ্য রাখা, বা লগ দেখে কেউ সতর্ক না হওয়া — এর ফলে আক্রমণ মাসের পর মাস চলতে পারে। প্রতিকার: নিরাপত্তা-প্রাসঙ্গিক ঘটনা লগ করা, লগ কেন্দ্রীভূত করা, ও সতর্কতা নির্ধারণ করা।

A10 SSRF। সার্ভার যখন ব্যবহারকারীর দেওয়া URL ধরে অনুরোধ পাঠায়, আক্রমণকারী তখন অভ্যন্তরীণ ঠিকানা — যেমন `http://169.254.169.254/` ক্লাউড মেটাডেটা, বা `http://203.0.113.20:27017/` অভ্যন্তরীণ সেবা — লক্ষ্য করতে পারে। প্রতিকার: URL-এর allowlist, DNS পুনঃপরীক্ষা, অভ্যন্তরীণ নেটওয়ার্কে প্রবেশ নিষেধ, ও টাইমআউট।

> **WARNING:** SSRF-র উদাহরণে উল্লেখিত মেটাডেটা ঠিকানা ও অভ্যন্তরীণ IP শুধু বোঝার জন্য দেওয়া হয়েছে। নিজের নয় এমন কোনো সিস্টেমে SSRF পরীক্ষা করা যাবে না। ল্যাবে শুধু নিজের তৈরি বিচ্ছিন্ন সেবার বিরুদ্ধে, অনুমোদিত পরিবেশে পরীক্ষা করো।

৭.৬ ব্যবহারিক কমান্ড ও কনফিগারেশন

নিরাপদ HTTP হেডার (Helmet):

```
const helmet = require('helmet');
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'"],
      imgSrc: ["'self'", 'data:'],
      objectSrc: ["'none'"]
    }
  },
  hsts: { maxAge: 31536000, includeSubDomains: true },
  frameguard: { action: 'deny' },
  noSniff: true
}));
// Content-Security-Policy: ব্রাউজারকে বলে কোন উৎস থেকে স্ক্রিপ্ট চলবে
// HSTS: ব্রাউজারকে সবসময় HTTPS ব্যবহার করতে বলে
// X-Frame-Options / frameguard: ক্লিকজ্যাকিং ঠেকায়
// X-Content-Type-Options: MIME sniffing ঠেকায়
```

XSS — দুর্বল বনাম নিরাপদ (React):

```
// দুর্বল — ব্যবহারকারীর HTML সরাসরি বসানো
function Comment({ text }) {
  return <div dangerouslySetInnerHTML={{ __html: text }} />;
}
// নিরাপদ — React স্বয়ংক্রিয়ভাবে এস্কেপ করে
function Comment({ text }) {
  return <div>{text}</div>;
}
// যদি সমৃদ্ধ লেখা (rich text) দরকার হয়, DOMPurify-এর মতো
// স্যানিটাইজার দিয়ে অনুমোদিত ট্যাগের allowlist ব্যবহার করো।
```

CSRF টোকেন ও SSRF প্রতিরোধ:

```
// CSRF: csurf-জাতীয় মিডলওয়্যার + SameSite কুকি
const csrf = require('csurf');
app.use(csrf({ cookie: true }));

// SSRF প্রতিরোধ — ধারণা: URL allowlist + অভ্যন্তরীণ ঠিকানা নিষেধ
const ALLOWED_HOSTS = ['api.technova.example'];
function isAllowedUrl(raw) {
  let u;
  try { u = new URL(raw); } catch { return false; }
  if (u.protocol !== 'https:') return false;
  if (!ALLOWED_HOSTS.includes(u.hostname)) return false;
  return true;
}
// বাস্তব প্রয়োগে DNS সমাধান করে IP পরীক্ষা করতে হয়,
// কারণ হোস্টনাম বদলে অভ্যন্তরীণ IP-তে যেতে পারে (DNS rebinding)।
// উৎপাদনে এই যাচাই একটি আলাদা প্রক্সি বা গেটওয়েতে রাখা ভালো।
```

নিরাপদ ল্যাব — OWASP Juice Shop (শুধু বিচ্ছিন্ন পরিবেশে):

```
# Juice Shop একটি ইচ্ছাকৃতভাবে দুর্বল প্রশিক্ষণ অ্যাপ।
# শুধু নিজের কম্পিউটারে, বিচ্ছিন্ন Docker নেটওয়ার্কে চালাও।
docker run --rm -p 3000:3000 bkimminich/juice-shop
# ব্রাউজারে http://localhost:3000 খুলে অনুশীলন করো।
# কখনো এটি ইন্টারনেটে প্রকাশ করো না, কখনো অন্যের সিস্টেমে চালিও না।
# কাজ শেষে কন্টেইনার বন্ধ করে দাও: docker stop <container-id>
```

৭.৭ প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — মন্তব্যে Stored XSS। TechNova-র পণ্য পর্যালোচনায় একজন ব্যবহারকারী স্ক্রিপ্ট লিখে জমা দিল। সেই পর্যালোচনা দেখতে আসা প্রতিটি ব্যবহারকারীর ব্রাউজারে স্ক্রিপ্ট চলল। ক্ষতি শুধু তখনই থামল যখন আউটপুট এনকোডিং ঠিক করা হলো ও CSP চালু করা হলো। শিক্ষা: জমা হওয়া ডেটা দেখানোর সময় এনকোডিং সবচেয়ে গুরুত্বপূর্ণ, কারণ সেখানে আক্রমণ বহু মানুষকে একসঙ্গে আঘাত করে।

পরিস্থিতি ২ — ভুল কনফিগারেশনে অ্যাডমিন প্যানেল খোলা। একটি স্টেজিং সার্ভারের ডিফল্ট পাসওয়ার্ড বদলানো হয়নি, আর সেটি ইন্টারনেটে প্রকাশ পেয়েছিল। শিক্ষা: স্টেজিং ও প্রোডাকশন পরিবেশ আলাদা রাখো, ডিফল্ট বদলাও, ও বাইরে থেকে প্রবেশ সীমিত করো।

পরিস্থিতি ৩ — SSRF দিয়ে অভ্যন্তরীণ সেবা দেখা। একটি "ছবির URL থেকে প্রিভিউ" বৈশিষ্ট্য ব্যবহারকারীর দেওয়া যেকোনো URL ধরত। আক্রমণকারী অভ্যন্তরীণ সেবার ঠিকানা দিল। শিক্ষা: ব্যবহারকারীর URL দিয়ে সার্ভার-সাইড অনুরোধ পাঠানোর আগে allowlist ও নেটওয়ার্ক নিয়ন্ত্রণ অপরিহার্য; এই বৈশিষ্ট্যটি আদৌ দরকার কি না, তা প্রথমে প্রশ্ন করো।

> **REAL WORLD:** বাস্তবে বহু বড় ঘটনা A05 ও A09-এর সংমিশ্রণে ঘটে: ভুল কনফিগারেশনে দরজা খোলা থাকে, আর লগ না দেখার কারণে কেউ মাসের পর মাস টের পায় না। তাই "সেট করে ভুলে যাওয়া" নয় — কনফিগারেশন ও লগের নিয়মিত পর্যালোচনাই প্রোডাকশন মান।

৭.৮ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা হেডার ও CSP চালু করার খরচ প্রায় শূন্য, কিন্তু ভুল CSP বৈধ স্ক্রিপ্ট আটকে দিতে পারে — তাই প্রথমে রিপোর্ট-অনলি মোডে চালিয়ে দেখো কোনটি ভাঙছে। WAF (Web Application Firewall) আক্রমণ থামাতে পারে, কিন্তু এটি আসল প্রতিকারের বিকল্প নয়; WAF একটি আস্তরণ, কোডের ত্রুটি ঢাকার উপায় নয়। লগিং ও পর্যবেক্ষণের খরচ হলো স্টোরেজ ও মানুষের সময় — প্রতিটি সতর্কতা কেউ একজনকে দেখতে হয়। তাই সতর্কতা এমনভাবে সাজাও যাতে শব্দ (noise) কম হয় ও আসল ঘটনা চোখে পড়ে। API নিরাপত্তায় হার-সীমা ও কোটা চালু করলে বৈধ ব্যবহারকারীর ওপর প্রভাব পড়তে পারে; সীমা নির্ধারণ করো বাস্তব ব্যবহারের পরিমাপ দেখে।

৭.৯ সাধারণ ভুল

- Top 10-কে চেকলিস্ট ভাবা — তালিকার বাইরের ত্রুটি উপেক্ষা করা।
- XSS ঠেকাতে শুধু `<script>` শব্দ আটকানো — denylist দিয়ে XSS থামে না।
- CSRF সুরক্ষা ছাড়া কুকি-ভিত্তিক প্রমাণীকরণ ব্যবহার করা।
- SSRF প্রতিরোধে শুধু হোস্টনাম পরীক্ষা করা — DNS rebinding ও রিডাইরেক্ট ভুলে যাওয়া।
- ত্রুটি বার্তায় স্ট্যাক ট্রেস প্রকাশ করা (A05)।
- Juice Shop-এর মতো প্রশিক্ষণ অ্যাপ ইন্টারনেটে প্রকাশ করা বা অন্যের সিস্টেমে পরীক্ষা চালানো।
- ক্রিপ্টোগ্রাফিতে নিজের তৈরি অ্যালগরিদম ব্যবহার করা — প্রমাণিত লাইব্রেরি ব্যবহার করো।
- লগ রাখা কিন্তু কখনো না দেখা — A09-এর সবচেয়ে সাধারণ রূপ।

৭.১০ সমস্যা সমাধান

সমস্যা: CSP চালু করার পর বৈধ স্ক্রিপ্ট ভাঙছে। ব্রাউজার কনসোলে CSP লঙ্ঘনের প্রতিবেদন দেখো। প্রথমে `Content-Security-Policy-Report-Only` হেডার দিয়ে পরীক্ষা করো, কোন উৎসগুলো দরকার তা শনাক্ত করো, তারপর ধীরে ধীরে নীতি কঠোর করো। ইনলাইন স্ক্রিপ্ট এড়িয়ে বাইরের ফাইলে সরানোই দীর্ঘমেয়াদি সমাধান।

সমস্যা: বৈধ অনুরোধ CSRF টোকেনের কারণে ব্যর্থ হচ্ছে। টোকেন ফর্মে ঠিকমতো যাচ্ছে কি না, সেশন ঠিক আছে কি না, আর একাধিক ট্যাবে টোকেন আবর্তন হচ্ছে কি না — দেখো। API-র ক্ষেত্রে কুকি-ভিত্তিক প্রমাণীকরণের বদলে Authorization হেডারে টোকেন ব্যবহার করলে CSRF-র ঝুঁকি কমে, কারণ ব্রাউজার সেই হেডার স্বয়ংক্রিয়ভাবে পাঠায় না।

সমস্যা: স্ক্যানার বহু সতর্কতা দিচ্ছে, কোনটি আগে ঠিক করবে বুঝতে পারছ না। তীব্রতা দিয়ে নয়, শোষণযোগ্যতা ও সম্পদের মূল্য দিয়ে অগ্রাধিকার দাও: ইন্টারনেটে প্রকাশিত, প্রমাণীকরণ ছাড়া শোষণযোগ্য, ও সংবেদনশীল ডেটার সঙ্গে যুক্ত দুর্বলতা আগে।

> **TROUBLESHOOTING:** কোনো সন্দেহজনক অনুরোধ দেখলে প্রথমে লগ সংরক্ষণ করো, তারপর তদন্ত করো। লগ মুছে ফেলা বা আক্রমণকারীকে জানিয়ে দেওয়া তদন্ত নষ্ট করে। বিচ্ছিন্ন পরিবেশে পুনরায় তৈরি করা ছাড়া প্রোডাকশনে আক্রমণ-পরীক্ষা চালিও না।

৭.১১ সারাংশ

OWASP Top 10 (2021) হলো ওয়েব নিরাপত্তার একটি সাধারণ ভাষা: A01 Broken Access Control, A02 Cryptographic Failures, A03 Injection, A04 Insecure Design, A05 Security Misconfiguration, A06 Vulnerable

and Outdated Components, A07 Identification and Authentication Failures, A08 Software and Data Integrity Failures, A09 Security Logging and Monitoring Failures ও A10 SSRF। প্রতিটি বিভাগের মূল কারণ বুঝলে প্রতিকারও বোঝা যায়: অনুমোদন পরীক্ষা, শক্তিশালী ক্রিপ্টোগ্রাফি, প্যারামিটারাইজড কোয়েরি ও এনকোডিং, নিরাপদ নকশা, কঠোর কনফিগারেশন, নিয়মিত আপডেট, নিরাপদ প্রমাণীকরণ, যাচাইকৃত সরবরাহ, লগিং ও পর্যবেক্ষণ, এবং SSRF-র বিরুদ্ধে allowlist। হেডার, CSP ও CSRF টোকেন হলো সহায়ক স্তর, মূল প্রতিকার নয়। আর সব অনুশীলন শুধু অনুমোদিত, বিচ্ছিন্ন ল্যাবে — Juice Shop-এর মতো প্রশিক্ষণ অ্যাপে।

৭.১২ পর্যালোচনা প্রশ্ন

1. OWASP Top 10 (2021)-এর দশটি বিভাগের নাম লেখো। ২০১৭ থেকে ২০২১-এ কী কী বিভাগ নতুন এসেছে?
2. Top 10-এর ক্রম তীব্রতার ক্রম নয় — এই কথার অর্থ কী? একটি উদাহরণ দাও।
3. Stored, Reflected ও DOM-based XSS-এর মধ্যে পার্থক্য ব্যাখ্যা করো।
4. CSRF কীভাবে কাজ করে? SameSite কুকি ও CSRF টোকেন কীভাবে এটি ঠেকায়?
5. SSRF কী? ক্লাউড মেটাডেটা ঠিকানা কেন SSRF-তে লক্ষ্যবস্তু হয়?
6. Content Security Policy কী এবং এটি XSS-কে কীভাবে সীমিত করে?
7. Insecure Design (A04) ও Security Misconfiguration (A05)-এর মধ্যে পার্থক্য কী?
8. Broken Access Control-এর তিনটি উদাহরণ দাও এবং প্রতিটির প্রতিকার লেখো।
9. API নিরাপত্তায় হার-সীমা, প্রমাণীকরণ ও অনুমোদনের ভূমিকা কী?
10. নিরাপদ ল্যাব অনুশীলনের নিয়মগুলো কী কী? Juice Shop কেন ইন্টারনেটে প্রকাশ করা যাবে না?

৭.১৩ ব্যবহারিক অনুশীলন

1. **Juice Shop ল্যাব:** বিচ্ছিন্ন Docker পরিবেশে OWASP Juice Shop চালাও। একটি Stored XSS ও একটি Broken Access Control চ্যালেঞ্জ সমাধান করো এবং প্রতিটির মূল কারণ ও প্রতিকার লিখে রাখো। কাজ শেষে কন্টেইনার বন্ধ করো।
2. **হেডার পরীক্ষা:** তোমার ল্যাব অ্যাপে Helmet চালু করার আগে ও পরে নিরাপত্তা হেডারগুলো পরীক্ষা করো। CSP রিপোর্ট-অনলি মোডে চালিয়ে কোন স্ক্রিপ্ট উৎস দরকার তা শনাক্ত করো।
3. **CSRF প্রদর্শন:** নিজের ল্যাব অ্যাপে CSRF টোকেন ছাড়া একটি ফর্ম তৈরি করো, তারপর একটি আলাদা স্থানীয় পাতা থেকে সেই ফর্মে অনুরোধ পাঠিয়ে দেখো কী ঘটে। তারপর টোকেন যোগ করে দেখো আক্রমণ থেমে যায় কি না।
4. **SSRF নকশা পর্যালোচনা:** একটি "URL থেকে প্রিভিউ" বৈশিষ্ট্যের নকশা লেখো যেখানে allowlist, প্রোটোকল সীমা, টাইমআউট ও নেটওয়ার্ক নিয়ন্ত্রণ আছে। ব্যাখ্যা করো কেন প্রতিটি নিয়ন্ত্রণ দরকার।

অধ্যায় ৮ • PART II — Application & Software Supply Chain Security

Chapter 8 — SAST, DAST and Dependency Security

৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে শিক্ষার্থী SAST (Static Application Security Testing), DAST (Dynamic Application Security Testing) ও SCA (Software Composition Analysis)-এর মধ্যে পার্থক্য ব্যাখ্যা করতে পারবে এবং জানবে কোনটি কখন কাজ করে, কোনটি কী ধরতে পারে আর কোনটি পারে না। সে Semgrep, SonarQube, OWASP ZAP ও নির্ভরতা স্ক্যানারের কাজের নীতি বুঝবে, CI/CD পাইপলাইনে সেগুলোকে সঠিক জায়গায় বসাতে পারবে, এবং false positive — অর্থাৎ ভুল সতর্কতা — চিনে তা সামলাতে পারবে। শিক্ষার্থী দুর্বলতা ত্রিয়াজ — অর্থাৎ কোন দুর্বলতা আগে ঠিক করবে তা নির্ধারণ — ও প্রতিকার কর্মপ্রবাহ (remediation workflow) তৈরি করতে পারবে, যেখানে প্রতিটি আবিষ্কারের মালিক, সময়সীমা ও যাচাইয়ের নিয়ম থাকে। সবশেষে সে বুঝবে স্ক্যানিং নিরাপত্তা প্রকৌশলের বিকল্প নয়; এটি প্রকৌশলকে সাহায্য করে, প্রতিস্থাপন করে না।

৮.২ পূর্বশর্ত পুনরালোচনা

অধ্যায় ৬ ও ৭ থেকে Injection, XSS, প্রমাণীকরণ ও অনুমোদনের ধারণা এখানে কাজে লাগবে, কারণ স্ক্যানার মূলত সেই দুর্বলতাগুলোই খোঁজে। CI/CD-র মৌলিক ধারণা — কমিট, পুল রিকোয়েস্ট, বিল্ড, পরীক্ষা ও ডিপ্লয় — জানা দরকার, কারণ স্ক্যানিংয়ের আসল মূল্য নির্ভর করে তাকে পাইপলাইনের কোন ধাপে বসানো হচ্ছে তার ওপর। npm ও `package.json`-এর ধারণাও দরকার: একটি অ্যাপের বেশিরভাগ কোড নিজের লেখা নয়, অন্যের লেখা লাইব্রেরি — আর সেই লাইব্রেরিগুলোতেই দুর্বলতা লুকিয়ে থাকতে পারে। CVE সম্পর্কে সংক্ষেপে: CVE হলো পরিচিত দুর্বলতার একটি সর্বজনীন শনাক্তকারী নম্বর, আর CVSS হলো তার তীব্রতা মাপার একটি স্কোর। এই দুটি শব্দ পরের অংশগুলোতে বারবার আসবে।

৮.৩ প্রথম নীতি থেকে ব্যাখ্যা: স্ক্যানিং কী সমস্যা সমাধান করে

মানুষের চোখে হাজার হাজার লাইন কোড পড়ে প্রতিটি দুর্বলতা খুঁজে পাওয়া অসম্ভব — বিশেষ করে যখন কোড প্রতিদিন বদলায়। স্ক্যানিং সেই সমস্যার সমাধান: যন্ত্রকে শেখানো হয় পরিচিত ভুলের ধরন চিনতে, যাতে মানুষ তার সময় ব্যয় করতে পারে সেই জায়গায় যেখানে যন্ত্র পারে না — ব্যবসায়িক যুক্তি, নকশা ও প্রসঙ্গ বোঝায়। কিন্তু এখানেই প্রথম ভুল বোঝাবুঝি শুরু হয়: স্ক্যানার যা খুঁজে পায়, তা শুধু পরিচিত ধরন। একটি সম্পূর্ণ নতুন ধরনের ত্রুটি, বা একটি নকশার ত্রুটি, স্ক্যানারের চোখে পড়ে না। তাই স্ক্যানারকে নিরাপত্তার প্রমাণ ভাবা ভুল; স্ক্যানার পরিষ্কার আসা মানে "পরিচিত ধরনের ভুল পাওয়া যায়নি", "কোনো ভুল নেই" নয়।

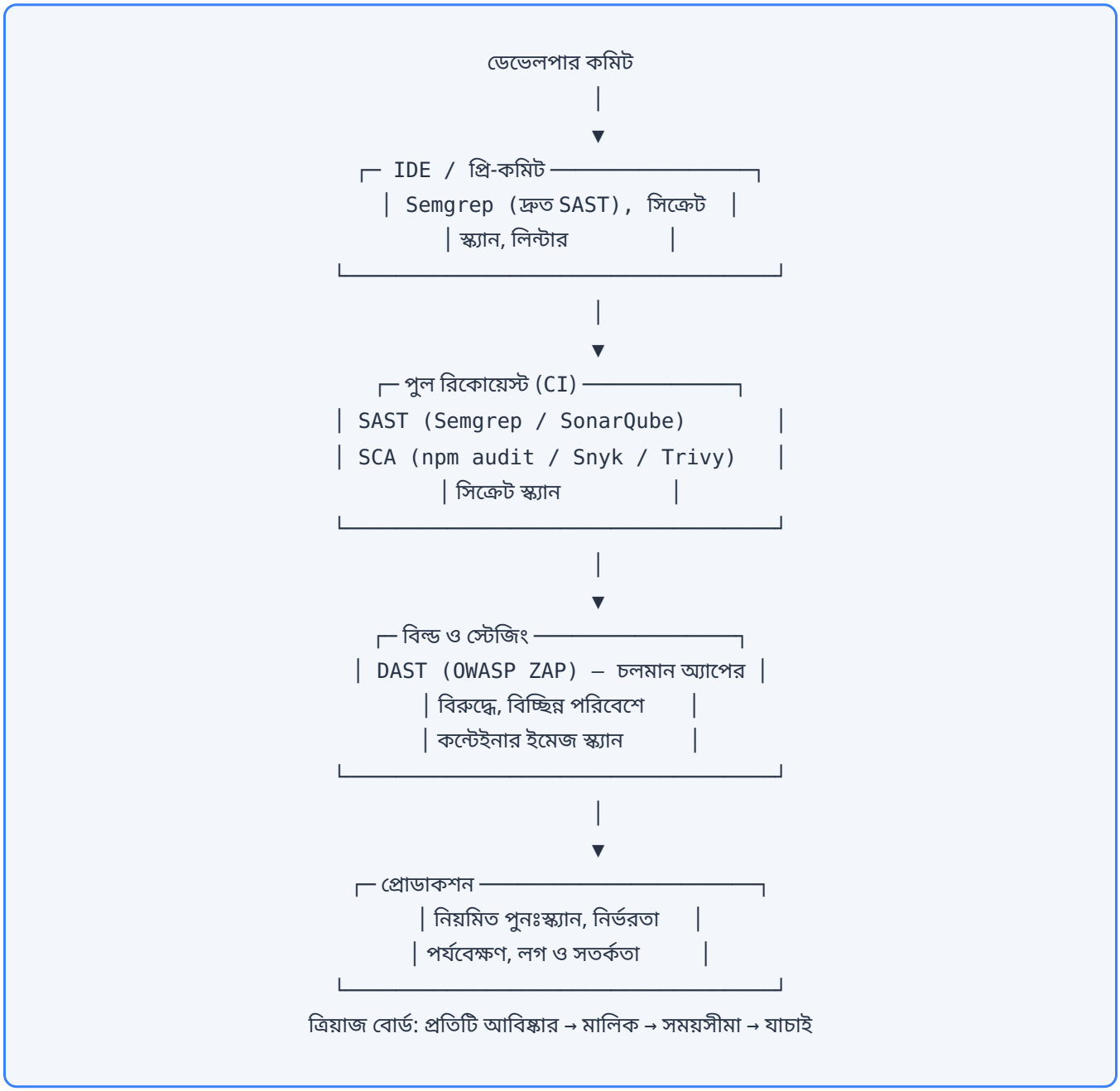
তিনটি পদ্ধতির পার্থক্য বোঝার জন্য একটি সহজ উপমা কাজ করে: SAST হলো বাড়ি বানানোর নকশা পড়ে দেখা — দেয়ালের ভেতরে তার কোথায় যাচ্ছে, তা নির্মাণের আগেই পরীক্ষা করা। DAST হলো বাড়ি বানানো শেষে দরজা-জানালা ধাক্কা দিয়ে দেখা — বাইরে থেকে আসল আক্রমণের মতো পরীক্ষা করা। SCA হলো বাড়িতে ব্যবহৃত প্রতিটি ইট, তার ও পাইপের উৎস পরীক্ষা করা — সেগুলোর কোনোটিতে পরিচিত ত্রুটি আছে কি না। তিনটি একসঙ্গে না থাকলে ছবি অসম্পূর্ণ থাকে: SAST চলমান অ্যাপের কনফিগারেশন দেখতে পারে না, DAST কোডের ভেতরের পথ দেখতে পারে না, আর SCA নিজের লেখা কোডের ত্রুটি দেখতে পারে না।

এই তিনটির ব্যর্থতার ধরনও আলাদা। SAST-এর ব্যর্থতা হলো false positive — কোডে একটি বিপজ্জনক দেখতে ধরন থাকলেও বাস্তবে সেটি নিরাপদ, কারণ ডেটা সেখানে পৌঁছায় না। DAST-এর ব্যর্থতা হলো false negative — অ্যাপের যে অংশে লগইন বা বিশেষ অবস্থা দরকার, স্ক্যানার সেখানে পৌঁছাতে পারে না, তাই ত্রুটি অদেখা থেকে যায়। SCA-র ব্যর্থতা হলো প্রসঙ্গহীনতা — একটি দুর্বল লাইব্রেরি ব্যবহৃত হলেও তার দুর্বল ফাংশনটি আদৌ ডাকা হয় কি না, তা SCA জানে না।

এই সীমাগুলো না বুঝলে দল হয় স্ক্যানারের প্রতিটি সতর্কতায় আতঙ্কিত হয়, নয়তো স্ক্যানারকে সম্পূর্ণ অগ্রাহ্য করে — দুটিই ভুল।

> **CONCEPT:** স্ক্যানিং বনাম নিরাপত্তা প্রকৌশল: স্ক্যানিং প্রশ্ন করে "এই পরিচিত ভুলটি কি আছে?" প্রকৌশল প্রশ্ন করে "এই সিস্টেমটি কীভাবে ভাঙা যেতে পারে, এবং ভাঙলে কী ক্ষতি হবে?" প্রথমটি যন্ত্রের কাজ, দ্বিতীয়টি মানুষের। ভালো দল দুটিকেই ব্যবহার করে — যন্ত্রকে পুনরাবৃত্তিমূলক খোঁজার কাজে, মানুষকে বিচারবুদ্ধির কাজে।

৮.৪ স্থাপত্য: পাইপলাইনে স্ক্যানিং কোথায় বসে



ত্রিয়াজ বোর্ড: প্রতিটি আবিষ্কার → মালিক → সময়সীমা → যাচাই

এই স্থাপত্যের মূল নীতি হলো "বাম দিকে সরানো" — shift left। যত আগে দুর্বলতা ধরা পড়ে, তত সস্তায় তা ঠিক করা যায়। IDE-তে ধরা পড়া একটি ত্রুটি ঠিক করতে কয়েক মিনিট লাগে; প্রোডাকশনে ধরা পড়লে লাগে দিন, সঙ্গে গ্রাহকের ক্ষতি ও জরুরি ডিপ্লয়ের ঝুঁকি। কিন্তু বাম দিকে সরানোর অর্থ এই নয় যে ডান দিকের পরীক্ষা বাদ দেওয়া যাবে — প্রোডাকশন

কনফিগারেশন ও চলমান আচরণ শুধু ডান দিকেই দেখা যায়। তাই প্রতিটি স্তরে উপযুক্ত সরঞ্জাম, আর প্রতিটি আবিষ্কারের জন্য একটি স্পষ্ট কর্মপ্রবাহ দরকার।

৮.৫ অভ্যন্তরীণ কার্যপ্রণালী

SAST কীভাবে কাজ করে। SAST প্রথমে কোডকে পার্স করে একটি বিমূর্ত গাছ (AST) বানায়, তারপর সেই গাছের ওপর নিয়ম চালায়। উন্নত SAST টেইন্ট বিশ্লেষণ (taint analysis) করে: অবিশ্বস্ত উৎস (যেমন `req.body`) থেকে ডেটা কোন পথে বিপজ্জনক সিংক (যেমন ডাটাবেস কোয়েরি বা HTML আউটপুট) পর্যন্ত যায়, তা অনুসরণ করে। মাঝখানে যদি স্যানিটাইজার বা ভ্যালিডেটর থাকে, তবে সতর্কতা কমে যায়। Semgrep এই নীতিতে কাজ করে, তবে তার নিয়মগুলো পড়া ও লেখা সহজ — অনেকটা কোডের ধরন মেলানোর মতো। SonarQube এর সঙ্গে কোডের গুণমান, জটিলতা ও পরীক্ষার আচ্ছাদনও মাপে, তাই এটি শুধু নিরাপত্তার সরঞ্জাম নয়, কোড স্বাস্থ্যের সরঞ্জামও। SAST-এর শক্তি: কোড চালানো ছাড়াই, প্রতিটি শাখা পরীক্ষা করা যায়। দুর্বলতা: চলমান অবস্থার তথ্য — যেমন কনফিগারেশন বা ডাটাবেসের অবস্থা — সে দেখতে পারে না।

DAST কীভাবে কাজ করে। OWASP ZAP-এর মতো DAST সরঞ্জাম চলমান অ্যাপের সঙ্গে আসল HTTP অনুরোধ পাঠিয়ে কথা বলে। প্রথমে এটি অ্যাপটি ঘুরে দেখে (crawl) — কোন পাতা, ফর্ম ও API আছে — তারপর প্রতিটি জায়গায় পরীক্ষামূলক ইনপুট পাঠিয়ে উত্তর দেখে। উত্তর যদি অস্বাভাবিক হয় — যেমন ত্রুটি বার্তায় ডাটাবেসের তথ্য, বা ইনপুট সরাসরি উত্তরে ফিরে আসে — তবে সতর্কতা তৈরি হয়। ZAP-কে প্রমাণীকরণ শেখাতে হয়, নইলে সে লগইনের পেছনের পাতায় পৌঁছাতে পারে না। DAST-এর শক্তি: এটি আসল আচরণ দেখে, তাই false positive কম। দুর্বলতা: এটি শুধু সেই পথেই যেতে পারে যা সে খুঁজে পায়; লুকানো API বা বিশেষ অবস্থার পথ অদেখা থাকে।

SCA ও নির্ভরতা স্ক্যানিং। SCA প্রথমে নির্ভরতার তালিকা তৈরি করে — `package-lock.json` থেকে প্রতিটি লাইব্রেরি ও তার সংস্করণ — তারপর সেই তালিকাকে পরিচিত দুর্বলতার ডাটাবেসের (যেমন GitHub Advisory Database, NVD) সঙ্গে মেলায়। `npm audit` এই কাজটি বিনামূল্যে করে, তবে তার প্রতিবেদনে প্রসঙ্গ থাকে না। উন্নত সরঞ্জাম বলে দেয় দুর্বল ফাংশনটি আদৌ ডাকা হচ্ছে কি না (reachability)। নির্ভরতা নিরাপত্তার আসল চ্যালেঞ্জ হলো পরিবর্তনের গতি: প্রতিদিন নতুন CVE প্রকাশ হয়, আর একটি লাইব্রেরি আপডেট করলে অন্য লাইব্রেরি ভেঙে যেতে পারে। তাই SCA-র সঙ্গে একটি আপডেট নীতি দরকার — কখন স্বয়ংক্রিয়ভাবে আপডেট করা হবে, কখন মানুষের অনুমোদন লাগবে।

False positive ও ত্রিয়াজ। False positive সামলানোর প্রথম নিয়ম: সতর্কতাটি পড়ে বুঝতে হবে, অন্ধভাবে বন্ধ করা যাবে না। SAST-এর একটি সতর্কতা যদি বলে "এই কোয়েরিতে ইনজেকশন সম্ভব", তবে দেখতে হবে ডেটা কোথা থেকে আসছে, মাঝখানে ভ্যালিডেশন আছে কি না, আর কোয়েরিটি আসলে কী করছে। যদি প্রমাণিত হয় এটি নিরাপদ, তবে সেই সিদ্ধান্ত লিখে রাখতে হয় — কোডে মন্তব্য, বা সরঞ্জামের suppression ফাইলে, কারণসহ — যাতে পরের বার আবার একই প্রশ্ন না ওঠে। ত্রিয়াজ মানে অগ্রাধিকার দেওয়া: শোষণযোগ্যতা (ইন্টারনেট থেকে কি প্রমাণীকরণ ছাড়া আক্রমণ করা যায়?), প্রভাব (সংবেদনশীল ডেটা কি জড়িত?), ও প্রসঙ্গ (দুর্বল কোডটি কি আসলেই ব্যবহৃত হয়?) — এই তিনটি দিয়ে সাজানো। CVSS স্কোর একটি শুরু, কিন্তু প্রসঙ্গ ছাড়া স্কোর একা সিদ্ধান্ত নিতে পারে না।

> **WARNING:** DAST কখনো প্রোডাকশন অ্যাপের বিরুদ্ধে অনুমোদন ছাড়া চালানো যাবে না। ZAP-এর আক্রমণাত্মক স্ক্যান ডেটা বদলে দিতে পারে, অ্যাকাউন্ট লক করতে পারে, বা সেবা বন্ধ করে দিতে পারে। DAST শুধু বিচ্ছিন্ন স্টেজিং পরিবেশে, বা যেখানে স্পষ্ট অনুমোদন আছে, সেখানে চালাও।

৮.৬ ব্যবহারিক কমান্ড ও কনফিগারেশন

```
# --- SAST: Semgrep ---
# ইনস্টল (Ubuntu 24.04, Python পরিবেশে)
pip install semgrep
# বর্তমান কোডে নিরাপত্তা নিয়ম চালানো
semgrep --config auto .
# শুধু JavaScript/TypeScript নিয়ম, JSON প্রতিবেদনসহ
semgrep --config auto --json --output semgrep-report.json .
# নিজের নিয়ম: উদাহরণ — eval() ব্যবহার নিষেধ
# .semgrep/rules.yml-এ নিয়ম লেখো, তারপর:
semgrep --config .semgrep/rules.yml .

# --- SAST: SonarQube (স্থানীয় Docker, ল্যাবের জন্য) ---
docker run -d --name sonarqube -p 9000:9000 sonarqube:lts-community
# ব্রাউজারে http://localhost:9000 — প্রাথমিক লগইন admin / admin
# (প্রথম লগইনে পাসওয়ার্ড বদলে ফেলো; এটি শুধু ল্যাবের উদাহরণ)
# স্ক্যানার চালানো:
# sonar-scanner -Dsonar.projectKey=technova -Dsonar.host.url=http://localhost:9000

# --- SCA: npm audit ---
npm audit
npm audit --audit-level=high
npm audit fix
# আপডেটের আগে পরীক্ষা চালাও — আপডেট ভাঙতে পারে:
npm update && npm test

# --- DAST: OWASP ZAP (শুধু নিজের স্টেজিং অ্যাপের বিরুদ্ধে) ---
docker run --rm -v $(pwd):/zap/wrk:rw \
  -t ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py -t http://203.0.113.30:3000 -r zap-report.html
# baseline স্ক্যান: নিরাপদ, প্যাসিভ পরীক্ষা
# পূর্ণ আক্রমণাত্মক স্ক্যান (zap-full-scan.py) শুধু বিচ্ছিন্ন
# স্টেজিংয়ে, অনুমোদন নিয়ে চালাবে — প্রোডাকশনে নয়।
```

CI-তে স্ক্যান বসানো (GitHub Actions ধারণা):

```
# .github/workflows/security.yml — ধারণা
# প্রতিটি পুল রিকোয়েস্টে:
#  1. npm audit --audit-level=high (ব্যর্থ হলে বিল্ড থামবে)
#  2. semgrep --config auto --error (নতুন আবিষ্কারে বিল্ড থামবে)
#  3. সিক্রেট স্ক্যান (gitleaks)
# গুরুতর আবিষ্কারে বিল্ড থামানো উচিত, কিন্তু প্রথম সপ্তাহে
# শুধু প্রতিবেদন তৈরি করে দলকে অভ্যস্ত করো — হঠাৎ করে সব
# বিল্ড থামিয়ে দিলে দল স্ক্যানার বন্ধ করে দেবে।
```

৮.৭ প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — শত শত সতর্কতা, কোনটি আগে? TechNova-র প্রথম পূর্ণ স্ক্যানে ৩০০টি সতর্কতা এলো। দল আতঙ্কিত হয়ে সবকিছু একসঙ্গে ঠিক করতে গিয়ে কিছুই শেষ করতে পারল না। সঠিক পদ্ধতি ছিল ত্রিয়াজ: প্রথমে ইন্টারনেটে প্রকাশিত ও প্রমাণীকরণ ছাড়া শোষণযোগ্য পাঁচটি, তারপর সংবেদনশীল ডেটার সঙ্গে যুক্তগুলো, বাকিগুলো সময়সূচি অনুযায়ী। শিক্ষা: স্ক্যানের সাফল্য মাপা হয় সতর্কতার সংখ্যায় নয়, ঠিক হওয়া গুরুতর দুর্বলতার সংখ্যায়।

পরিস্থিতি ২ — একটি লাইব্রেরি আপডেটে অ্যাপ ভাঙল। একটি গুরুতর CVE-র কারণে একটি লাইব্রেরি আপডেট করা হলো, কিন্তু নতুন সংস্করণে API বদলে যাওয়ায় চেকআউট ভেঙে গেল। শিক্ষা: আপডেটের আগে পরীক্ষা চালাও, ছোট ছোট আপডেট নিয়মিত করো — বড় জমে থাকা আপডেট একসঙ্গে করা ঝুঁকিপূর্ণ।

পরিস্থিতি ৩ — DAST প্রোডাকশন ডেটা বদলে দিল। একজন নতুন সদস্য ভুল করে ZAP-এর পূর্ণ স্ক্যান প্রোডাকশনের দিকে চালিয়ে দিলেন; স্ক্যানের পরীক্ষামূলক ফর্ম জমা দেওয়ায় নকল অর্ডার তৈরি হলো। শিক্ষা: DAST-এর লক্ষ্য URL কনফিগারেশনে হার্ডকোড করো না — পরিবেশ ভেরিয়েবল দিয়ে সীমাবদ্ধ করো, আর পূর্ণ স্ক্যানের জন্য অনুমোদনের ধাপ রাখো।

> **REAL WORLD:** বাস্তব দলগুলোতে স্ক্যানিং ব্যর্থ হয় প্রযুক্তির কারণে নয়, প্রক্রিয়ার কারণে: সতর্কতার কোনো মালিক নেই, সময়সীমা নেই, আর "পরে ঠিক করব" বলে জমে থাকে। প্রতিটি আবিষ্কারকে একটি টিকিট বানাও — মালিক, সময়সীমা, ও যাচাইয়ের নিয়মসহ — নইলে স্ক্যান শুধু একটি প্রতিবেদন হয়েই থাকে।

৮.৮ নিরাপত্তা ও খরচ বিবেচনা

স্ক্যানিং সরঞ্জামের খরচ তিন রকম: লাইসেন্স বা পরিচালনার খরচ, পাইপলাইনের সময় (প্রতিটি বিল্ডে কয়েক মিনিট বাড়ে), ও মানুষের সময় — false positive যাচাই করা। SonarQube-এর মতো ভারী সরঞ্জাম চালাতে সার্ভার দরকার; Semgrep হালকা ও দ্রুত। এই খরচের বিপরীতে লাভ হলো প্রোডাকশনে ধরা পড়া দুর্বলতার খরচ এড়ানো — যা জরুরি প্যাচ, গ্রাহকের ক্ষতি ও বিশ্বাস হারানোর খরচের চেয়ে অনেক কম। তবে একটি সতর্কতা: অতিরিক্ত কঠোর গেট — প্রতিটি ছোট সতর্কতায় বিল্ড থামানো — দলকে স্ক্যানার এড়াতে শেখায়। শুরুতে শুধু গুরুতর আবিষ্কারে বিল্ড থামাও, ধীরে ধীরে কঠোরতা বাড়াও।

৮.৯ সাধারণ ভুল

- শুধু SAST চালিয়ে DAST ও SCA বাদ দেওয়া — বা উল্টোটা।
- DAST প্রোডাকশনে চালানো, বা অনুমোদন ছাড়া অন্যের সাইট স্ক্যান করা।
- False positive-কে প্রমাণ ছাড়া বন্ধ করে দেওয়া — কারণ না লিখে suppression করা।
- CVSS স্কোরকেই একমাত্র অগ্রাধিকার ভাবা — প্রসঙ্গ উপেক্ষা করা।
- স্ক্যানের আবিষ্কারের মালিক ও সময়সীমা না থাকা।
- `npm audit fix` চালিয়ে পরীক্ষা না করে ডিপ্লয় করা।
- সিক্রেট স্ক্যান না করা — সিক্রেট কোডে থাকলে SAST তা ধরতে পারে না।
- স্ক্যান পরিষ্কার মানেই নিরাপদ ভাবা।

৮.১০ সমস্যা সমাধান

সমস্যা: Semgrep বহু false positive দিচ্ছে। নিয়মগুলো পর্যালোচনা করো — `--config auto` সব নিয়ম চালায়, যার কিছু তোমার কোডের জন্য প্রাসঙ্গিক নয়। শুধু প্রয়োজনীয় নিয়মের সেট বেছে নাও, আর যাচাই করা false positive-কে কারণসহ suppression করো।

সমস্যা: ZAP লগইনের পেছনে যেতে পারছে না। প্রমাণীকরণের কনটেক্সট সেট করো — লগইন URL, ফর্ম ক্ষেত্র, ও সেশন চিহ্ন। প্রয়োজনে একটি পরীক্ষামূলক অ্যাকাউন্ট বানাও যার শুধু স্টেজিংয়ে প্রবেশাধিকার আছে।

সমস্যা: npm audit-এর আবিষ্কার ঠিক করা যাচ্ছে না, কারণ সরাসরি নির্ভরতা নয়। এটি একটি transitive নির্ভরতা — অর্থাৎ তোমার লাইব্রেরির লাইব্রেরি। প্রথমে মূল লাইব্রেরি আপডেট করো; না হলে `overrides` দিয়ে সংস্করণ বদলাও, তবে পরীক্ষা করে।

> **TROUBLESHOOTING:** স্ক্যানার নিজে ভুল করতে পারে — সরঞ্জামের সংস্করণ, নিয়মের সংস্করণ ও কনফিগারেশন লিপিবদ্ধ রাখো, যাতে একই কোডে দুই বার ভিন্ন ফল এলে কারণ খুঁজে পাওয়া যায়। প্রতিবেদন সংরক্ষণ করো, শুধু স্ক্রিনে দেখে মুছে ফেলো না।

স্ক্যান ফলাফলকে সংস্কৃতিতে বদলানো। একটি দলে স্ক্যানিং তখনই কাজ করে যখন ডেভেলপার স্ক্যানারকে শত্রু নয়, সহকারী হিসেবে দেখে। এর জন্য স্ক্যানের ফলাফল আসতে হবে দ্রুত, প্রসঙ্গসহ, ও ঠিক করার উপায়সহ। একটি সতর্কতা যদি শুধু বলে "এখানে সমস্যা আছে" কিন্তু না বলে কোথায়, কেন, ও কীভাবে ঠিক করতে হয়, তবে ডেভেলপার সেটি উপেক্ষা করবে। তাই ভালো SAST নিয়মের সঙ্গে থাকে উদাহরণ, প্রভাবের ব্যাখ্যা ও নিরাপদ কোডের নমুনা। একইভাবে SCA-র প্রতিবেদনে থাকা উচিত কোন সংস্করণে দুর্বলতা ঠিক হয়েছে, আপডেট করলে কী ভাঙতে পারে, ও বিকল্প পথ কী। এই প্রসঙ্গ দেওয়ার কাজটি সরঞ্জামের নয়, দলের — নিয়ম বেছে নেওয়া, প্রতিবেদন সাজানো ও ডেভেলপারকে শেখানো। যখন একজন ডেভেলপার প্রথমবার একটি সতর্কতা ঠিক করে বুঝতে পারে কেন সেটি গুরুত্বপূর্ণ ছিল, তখন সে পরের বার নিজে থেকেই সেই ভুল এড়ায় — এটিই স্ক্যানিংয়ের আসল লাভ, সংখ্যায় মাপা যায় না এমন লাভ।

পরিমাপ ও ধারাবাহিক উন্নতি। স্ক্যানিং কর্মসূচির সাফল্য মাপতে হয় কয়েকটি সূচক দিয়ে: নতুন কোডে গুরুতর দুর্বলতা আসার হার কমছে কি না, আবিষ্কার থেকে প্রতিকার পর্যন্ত গড় সময় কত, false positive-এর হার কত, আর কত শতাংশ আবিষ্কার সময়সীমার মধ্যে বন্ধ হচ্ছে। এই সংখ্যাগুলো মাসে একবার পর্যালোচনা করো। যদি দেখো একই ধরনের দুর্বলতা বারবার আসছে — যেমন একই ধরনের Injection — তবে সমস্যাটি ব্যক্তিগত নয়, পদ্ধতিগত: হয়তো প্রশিক্ষণ দরকার, হয়তো একটি নিরাপদ লাইব্রেরি বা সাহায্যকারী ফাংশন দরকার যা সঠিক প্যাটার্নকে সহজ করে দেয়। যেমন একটি নিরাপদ কোয়েরি সাহায্যকারী ফাংশন থাকলে ডেভেলপারকে প্রতিবার প্যারামিটারাইজেশনের কথা মনে রাখতে হয় না — সঠিক পথটিই সহজ পথ হয়ে যায়। এই নীতিকে বলে paved road — নিরাপদ পথকে সবচেয়ে সহজ পথ বানানো।

DAST-এর গভীরতা ও সীমা আরও বিস্তারিতভাবে। ZAP-এর মতো DAST সরঞ্জামকে দুটি মোডে ভাবা যায়: প্যাসিভ ও সক্রিয়। প্যাসিভ মোডে সরঞ্জাম শুধু ট্রাফিক দেখে — হেডার, কুকি, ত্রুটি বার্তা — কোনো ক্ষতিকর ইনপুট পাঠায় না, তাই এটি নিরাপদ। সক্রিয় মোডে সে পরীক্ষামূলক ইনপুট পাঠায় — এখানেই ঝুঁকি, কারণ সেই ইনপুট ডেটা বদলে দিতে পারে। প্রোডাকশন-সদৃশ স্টেজিংয়ে সক্রিয় স্ক্যান চালানোর আগে ডেটা ব্যাকআপ ও পুনরুদ্ধারের পরিকল্পনা থাকা উচিত, আর স্ক্যানের সময় কেউ নজর রাখবে। এছাড়া DAST-এর একটি গুরুত্বপূর্ণ সীমা হলো ব্যবসায়িক যুক্তি: ZAP বলতে পারে একটি ফর্মে Injection আছে কি না, কিন্তু বলতে পারে না একজন ব্যবহারকারী অন্যের ছাড় ব্যবহার করছে কি না — সেটি ধরতে মানুষের পরীক্ষা দরকার। তাই DAST-কে হাতে-কলমে অনুপ্রবেশ পরীক্ষা বা যুক্তি পরীক্ষার বিকল্প ভাবা যাবে না।

৮.১১ সারাংশ

SAST কোড পড়ে, DAST চলমান অ্যাপ পরীক্ষা করে, আর SCA নির্ভরতার তালিকা যাচাই করে — তিনটি মিলে একটি সম্পূর্ণ ছবি দেয়, কোনো একটিই একা যথেষ্ট নয়। Semgrep দ্রুত ও লেখার সহজ নিয়ম দেয়, SonarQube কোড স্বাস্থ্যসহ গভীর বিশ্লেষণ দেয়, OWASP ZAP চলমান অ্যাপের আসল আচরণ দেখায়, আর `npm audit` নির্ভরতার পরিচিত দুর্বলতা ধরে। প্রতিটি আবিষ্কারকে ত্রিয়াজ করো — শোষণযোগ্যতা, প্রভাব ও প্রসঙ্গ দিয়ে — মালিক ও সময়সীমা দিয়ে ঠিক করো, আর যাচাই করে বন্ধ করো। স্ক্যানিং প্রকৌশলের বিকল্প নয়; এটি প্রকৌশলীকে সঠিক জায়গায় মনোযোগ দিতে সাহায্য করে।

৮.১২ পর্যালোচনা প্রশ্ন

1. SAST, DAST ও SCA-এর মধ্যে পার্থক্য ব্যাখ্যা করো। প্রতিটি কী ধরতে পারে আর কী পারে না?
2. Taint analysis কী? SAST কীভাবে উৎস থেকে সিংক পর্যন্ত ডেটার পথ অনুসরণ করে?
3. Semgrep ও SonarQube-এর মধ্যে পার্থক্য কী? কখন কোনটি ব্যবহার করবে?
4. OWASP ZAP কীভাবে কাজ করে? প্রমাণীকরণ ছাড়া ZAP-এর সীমা কী?
5. False positive ও false negative-এর মধ্যে পার্থক্য কী? প্রতিটির একটি করে উদাহরণ দাও।
6. SCA কীভাবে কাজ করে? Transitive নির্ভরতা বলতে কী বোঝায়?
7. ত্রিয়াজে CVSS স্কোর ছাড়াও কোন কোন বিষয় বিবেচনা করা উচিত?
8. Shift left বলতে কী বোঝায়? এটি খরচ কীভাবে কমায়?
9. DAST প্রোডাকশনে চালানো কেন বিপজ্জনক? নিরাপদ বিকল্প কী?
10. স্ক্যানিং বনাম নিরাপত্তা প্রকৌশলের পার্থক্য ব্যাখ্যা করো।

৮.১৩ ব্যবহারিক অনুশীলন

1. **Semgrep চালাও:** তোমার ল্যাব MERN অ্যাপে `semgrep --config auto` চালাও। পাঁচটি আবিষ্কার বেছে নিয়ে প্রতিটি যাচাই করো — সত্যিকারের দুর্বলতা, নাকি false positive? কারণসহ লিপিবদ্ধ করো।
2. **ZAP baseline:** বিচ্ছিন্ন স্টেজিংয়ে চলমান ল্যাব অ্যাপের বিরুদ্ধে ZAP baseline স্ক্যান চালাও। প্রতিবেদন থেকে দুটি আবিষ্কার ঠিক করো ও আবার স্ক্যান করে যাচাই করো।
3. **নির্ভরতা ত্রিয়াজ:** `npm audit` চালিয়ে আবিষ্কারগুলোকে ত্রিয়াজ করো — কোনটি আগে ঠিক করবে, কেন? একটি আপডেট করে পরীক্ষা চালিয়ে দেখো কিছু ভাঙে কি না।
4. **CI গেট তৈরি করো:** একটি পুল রিকোয়েস্ট পাইপলাইনে Semgrep ও npm audit যোগ করো, যেখানে শুধু উচ্চ তীব্রতার আবিষ্কারে বিল্ড থামবে। একটি ইচ্ছাকৃত দুর্বল কমিট দিয়ে পরীক্ষা করো গেটটি কাজ করে কি না।

অধ্যায় ৯ • PART II — Application & Software Supply Chain Security

Chapter 9 — Container and Kubernetes Security

৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে শিক্ষার্থী ব্যাখ্যা করতে পারবে কেন কন্টেইনার নিজে থেকে নিরাপদ নয় — এটি একটি বিচ্ছিন্নতা ব্যবস্থা, একটি নিরাপত্তা সীমানা নয়। সে Docker ইমেজ নিরাপত্তা, নন-রুট কন্টেইনার, ইমেজ স্ক্যানিং, Linux capabilities, seccomp, Kubernetes RBAC, Pod Security Standards, Network Policy, সিক্রেট নিরাপত্তা, রানটাইম নিরাপত্তা ও admission নীতি — প্রতিটি কী সমস্যা সমাধান করে তা বলতে পারবে। শিক্ষার্থী TechNova-র MERN অ্যাপের জন্য একটি নিরাপদ Dockerfile, একটি সীমিত RBAC ভূমিকা, একটি Pod Security সীমাবদ্ধতা ও একটি Network Policy লিখতে পারবে। সবশেষে সে বুঝবে কন্টেইনার নিরাপত্তা একটি স্তর নয়, বহু স্তরের সমন্বয়: ইমেজ, রানটাইম, অর্কেস্ট্রেটর, নেটওয়ার্ক ও সরবরাহ শৃঙ্খল একসঙ্গে।

৯.২ পূর্বশর্ত পুনরালোচনা

আগের খণ্ডগুলো থেকে Docker-এর মৌলিক ধারণা — ইমেজ, কন্টেইনার, Dockerfile, লেয়ার — এবং Kubernetes-এর মৌলিক ধারণা — Pod, Deployment, Service, Namespace — জানা থাকা দরকার। Linux-এর ক্ষেত্রে ব্যবহারকারী (UID), ফাইলের অনুমতি ও প্রক্রিয়া বিচ্ছিন্নতার ধারণা ঝালিয়ে নাও। RBAC সম্পর্কে সংক্ষেপে: RBAC হলো Role-Based Access Control — কে কোন সম্পদের ওপর কী কাজ করতে পারে, তা ভূমিকা দিয়ে নির্ধারণ করা। এই অধ্যায়ে Kubernetes-এর প্রসঙ্গে RBAC-ই সবচেয়ে গুরুত্বপূর্ণ ধারণা। Seccomp ও capabilities শব্দ দুটি নতুন হতে পারে — সেগুলোর ব্যাখ্যা ৯.৫ অংশে দেওয়া আছে, তাই এখন শুধু নামটি মনে রাখলেই চলবে।

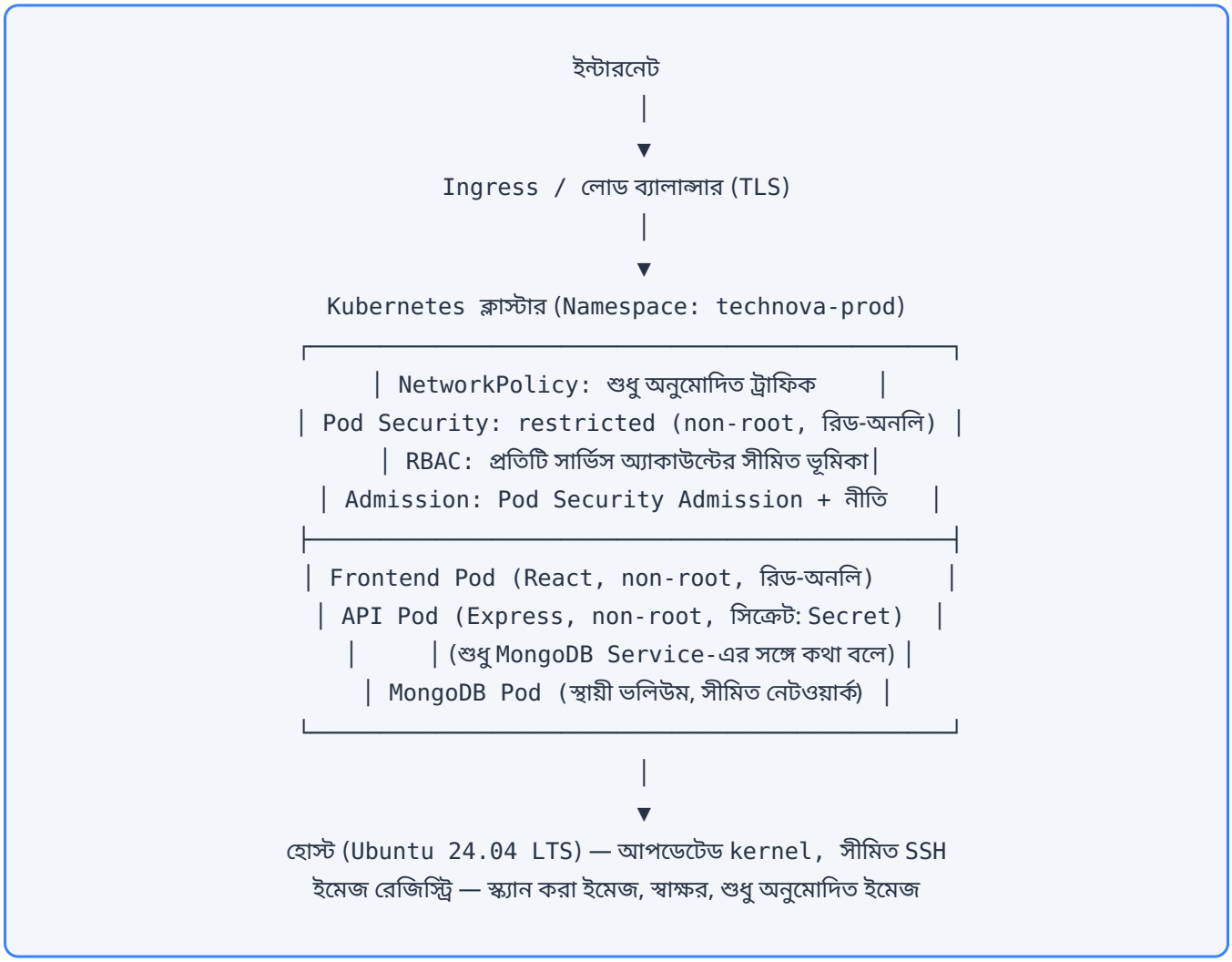
৯.৩ প্রথম নীতি থেকে ব্যাখ্যা: কন্টেইনার আসলে কী বিচ্ছিন্ন করে

একটি কন্টেইনার মূলত একটি প্রক্রিয়া, যাকে Linux kernel কয়েকটি কৌশলে আলাদা করে দেখায়: namespace দিয়ে তাকে নিজের একটি জগৎ দেখানো হয় — নিজের প্রক্রিয়া তালিকা, নেটওয়ার্ক, ফাইল সিস্টেম — আর cgroup দিয়ে তার সম্পদ সীমিত করা হয়। কিন্তু গুরুত্বপূর্ণ সত্য হলো, সব কন্টেইনার একই kernel ভাগ করে। হোস্টের kernel-এ একটি দুর্বলতা থাকলে, বা কন্টেইনারকে অতিরিক্ত ক্ষমতা দেওয়া হলে, সেই বিচ্ছিন্নতা ভেঙে যেতে পারে — একে বলে container escape। তাই কন্টেইনারকে ভার্চুয়াল মেশিনের মতো ভাবা ভুল: ভার্চুয়াল মেশিনের নিজের kernel থাকে, কন্টেইনারের থাকে না। এই একটি সত্য থেকেই কন্টেইনার নিরাপত্তার প্রায় সব নিয়ম আসে: কন্টেইনারকে যত কম ক্ষমতা দেওয়া যায়, তত ভালো; ইমেজে যত কম জিনিস থাকে, তত কম আক্রমণের পৃষ্ঠ; আর হোস্ট ও অর্কেস্ট্রেটরের নিরাপত্তা কন্টেইনারের নিরাপত্তার মতোই গুরুত্বপূর্ণ।

দ্বিতীয় নীতি হলো অপরিবর্তনীয়তা — immutability। একটি ইমেজ একবার তৈরি হলে সেটি বদলানো উচিত নয়; বদলাতে হলে নতুন ইমেজ তৈরি করো। কারণ চলমান কন্টেইনারে হাতে বদল করলে কেউ জানে না আসলে কী চলছে, আর সেই বদল পরের ডিপ্লয়ে হারিয়ে যায়। একইভাবে সিক্রেট কখনো ইমেজে থাকা উচিত নয় — ইমেজ একটি বিতরণযোগ্য বস্তু, যে কেউ সেটি ডাউনলোড করে তার ভেতরের সিক্রেট পড়তে পারে। এই নীতিগুলো না বুঝলে কন্টেইনার শুধু ডিপ্লয় সহজ করে, নিরাপত্তা দেয় না।

> **CONCEPT:** কন্টেইনার নিরাপত্তার প্রশ্নটি "কন্টেইনার কি নিরাপদ?" নয়, বরং "এই কন্টেইনার আপসিত হলে আক্রমণকারী আর কী করতে পারবে?" একটি নন-রুট, রিড-অনলি, নেটওয়ার্ক-সীমিত কন্টেইনার আপসিত হলেও ক্ষতি সীমিত থাকে; একটি privileged, রুট কন্টেইনার আপসিত হলে পুরো হোস্ট বিপদে পড়ে। নিরাপত্তা মাপা হয় ক্ষতির পরিসর দিয়ে।

৯.৪ স্থাপত্য: TechNova-র নিরাপদ কন্টেইনার স্তর



এই চিত্রে প্রতিটি স্তরের কাজ আলাদা: ইমেজ স্তরে স্ক্যান ও স্বাক্ষর, Pod স্তরে নিরাপত্তা প্রসঙ্গ, নেটওয়ার্ক স্তরে Network Policy, প্রবেশাধিকার স্তরে RBAC, আর ডিপ্লয়ের মুহূর্তে admission নীতি। কোনো একটি স্তর বাদ গেলে অন্য স্তরগুলো সেই ফাঁক পূরণ করতে পারে না — যেমন Network Policy ছাড়া একটি আপসিত API Pod পুরো ক্লাস্টারে ঘুরতে পারে।

৯.৫ অভ্যন্তরীণ কার্যপ্রণালী

ইমেজ নিরাপত্তা। একটি Docker ইমেজ অনেকগুলো লেয়ারের সমষ্টি — বেস ইমেজ, নির্ভরতা, অ্যাপ কোড। প্রতিটি লেয়ারে দুর্বলতা থাকতে পারে। নিরাপদ ইমেজ মানে: ছোট ও অফিসিয়াল বেস ইমেজ (যেমন `node:20-alpine` বা distroless), অপ্রয়োজনীয় প্যাকেজ না রাখা, মাল্টি-স্টেজ বিল্ডে শুধু চলার জন্য দরকারি ফাইল রাখা, আর নিয়মিত পুনর্নির্মাণ করা — কারণ বেস ইমেজের আপডেট না নিলে পুরোনো দুর্বলতা থেকে যায়। ইমেজ স্ক্যানিং (Trivy, Grype) ইমেজের প্রতিটি

লেয়ারের প্যাকেজকে পরিচিত দুর্বলতার ডাটাবেসের সঙ্গে মেলায়। কিন্তু স্ক্যান পরিষ্কার মানেই নিরাপদ নয় — স্ক্যান শুধু পরিচিত দুর্বলতা ধরে, আর ইমেজে থাকা সিক্রেট বা ভুল কনফিগারেশন আলাদাভাবে পরীক্ষা করতে হয়।

নন-রুট কন্টেইনার। ডিফল্টভাবে Docker কন্টেইনার রুট হিসেবে চলে — কন্টেইনারের ভেতরের রুট হোস্টের রুট নয়, কিন্তু তবুও তার ক্ষমতা অনেক। নন-রুট ব্যবহারকারী (যেমন UID 10001) দিয়ে চালালে আক্রমণকারী কন্টেইনারের ভেতরেও সীমিত ক্ষমতা পায়। Dockerfile-এ `USER node` লেখা, আর Kubernetes-এ `runAsNonRoot: true` ও `runAsUser` নির্ধারণ করা — দুটিই দরকার, কারণ একটি ভুলে গেলে অন্যটি রক্ষা করে না। মনে রাখো: ফাইলের মালিকানা ও অনুমতি নন-রুট ব্যবহারকারীর সঙ্গে সামঞ্জস্যপূর্ণ হতে হবে, নইলে অ্যাপ ফাইল পড়তে পারবে না।

Linux capabilities ও seccomp। Linux-এ রুটের ক্ষমতা আসলে অনেক ছোট ছোট ক্ষমতার সমষ্টি — যেমন নেটওয়ার্ক পোর্ট বাঁধা, সময় বদলানো, বা অন্য প্রক্রিয়া দেখা। Capabilities ব্যবস্থা সেই ক্ষমতাগুলোকে আলাদা করে দেয়, যাতে একটি প্রক্রিয়াকে শুধু দরকারি ক্ষমতাটুকু দেওয়া যায়। Kubernetes-এ ডিফল্টভাবে অনেক capability বাদ দেওয়া উচিত — `drop: ["ALL"]` — আর শুধু দরকার হলে নির্দিষ্টটি যোগ করা। Seccomp (secure computing mode) আরও এক ধাপ নিচে কাজ করে: এটি সিস্টেম কলের তালিকা সীমিত করে — প্রক্রিয়া কোন কোন kernel কল করতে পারবে, তা নির্ধারণ করে। একটি ওয়েব অ্যাপের সাধারণত কয়েক ডজন সিস্টেম কলই দরকার; বাকিগুলো বন্ধ করে দিলে আক্রমণকারীর পালানোর পথ কমে। বাস্তবে `RuntimeDefault` seccomp প্রোফাইল ব্যবহার করা একটি ভালো শুরু — এটি নিরাপদ ডিফল্ট দেয়, কাস্টম প্রোফাইল লেখার জটিলতা ছাড়াই।

Kubernetes RBAC। RBAC-এ চারটি মূল বস্তু: Role (একটি Namespace-এ কী করা যাবে), ClusterRole (পুরো ক্লাস্টারে), RoleBinding (কাকে Role দেওয়া হলো) ও ClusterRoleBinding। নীতি: প্রতিটি সার্ভিস অ্যাকাউন্টের শুধু তার কাজের জন্য দরকারি অনুমতি থাকবে। একটি API Pod-এর Secret পড়ার অনুমতি থাকতে পারে, কিন্তু Pod মুছে ফেলার অনুমতি থাকা উচিত নয়। সবচেয়ে বিপজ্জনক ভুল হলো `cluster-admin` দেওয়া — এটি পুরো ক্লাস্টারের চাবি। RBAC পরীক্ষা করার সময় `kubectl auth can-i` দিয়ে দেখো একটি অ্যাকাউন্ট আসলে কী করতে পারে — ধারণা দিয়ে নয়, পরীক্ষা দিয়ে।

Pod Security Standards। Kubernetes-এর Pod Security Standards তিনটি স্তর দেয়: Privileged (কোনো সীমা নেই, শুধু বিশ্বস্ত কাজে), Baseline (পরিচিত খারাপ অভ্যাস বন্ধ — যেমন privileged কন্টেইনার), ও Restricted (সবচেয়ে কঠোর — নন-রুট, রিড-অনলি ফাইল সিস্টেম, capabilities বাদ দেওয়া)। নতুন কাজে Restricted লক্ষ্য করা উচিত। Pod Security Admission এই নীতি Namespace-ভিত্তিক লেবেল দিয়ে প্রয়োগ করে — নীতি ভাঙা Pod তৈরি হতে বাধা দেয়।

Network Policy। ডিফল্টভাবে Kubernetes-এ সব Pod সব Pod-এর সঙ্গে কথা বলতে পারে। Network Policy সেই ডিফল্ট বদলে দেয়: শুধু নির্দিষ্ট Pod-এর মধ্যে, নির্দিষ্ট পোর্টে ট্রাফিক চলবে। যেমন API Pod শুধু MongoDB Pod-এর 27017 পোর্টে যেতে পারবে, আর কেউ API Pod-এ শুধু Ingress থেকে আসতে পারবে। এটি মাইক্রোসেগমেন্টেশন — একটি Pod আপসিত হলে আক্রমণকারী পাশের Pod-এ যেতে পারে না। মনে রাখো: Network Policy কাজ করতে CNI প্লাগইনকে তা সমর্থন করতে হয়; নীতি লেখার পর আসল ট্রাফিক দিয়ে পরীক্ষা করো।

সিক্রেট নিরাপত্তা। Kubernetes Secret ডিফল্টভাবে base64 এনকোড করা থাকে — এনক্রিপ্ট করা নয়। তাই etcd-তে Secret এনক্রিপশন চালু করা, Secret-এর প্রবেশাধিকার RBAC দিয়ে সীমিত করা, আর সম্ভব হলে বাইরের সিক্রেট ব্যবস্থাপক (যেমন Vault বা ক্লাউড সিক্রেট সেবা) ব্যবহার করা উচিত। সিক্রেট কখনো ইমেজে, কনফিগম্যাপে, বা লগে রাখা যাবে না। আর Secret ঘোরানো (rotation) — অর্থাৎ নিয়মিত বদলানো — একটি প্রক্রিয়া হিসেবে থাকা উচিত, জরুরি অবস্থায় নয়।

রানটাইম নিরাপত্তা ও admission নীতি। রানটাইম নিরাপত্তা মানে চলমান কন্টেইনারের আচরণ দেখা — অস্বাভাবিক প্রক্রিয়া, অপ্রত্যাশিত নেটওয়ার্ক সংযোগ, বা ফাইল বদল শনাক্ত করা। Falco-এর মতো সরঞ্জাম এই কাজ করে। Admission নীতি মানে Pod তৈরির মুহূর্তে পরীক্ষা — যেমন Pod Security Admission, বা Kyverno/OPA Gatekeeper-এর মতো নীতি

ইঞ্জিন, যা বলে দেয় "এই ক্লাস্টারে `:latest` ট্যাগের ইমেজ চলবে না" বা "প্রতিটি ইমেজ স্বাক্ষরিত হতে হবে"। Admission হলো শেষ দরজা — এখানে ভুল আটকানো গেলে ভুল প্রোডাকশনে পৌঁছায় না।

> **WARNING:** `--privileged` ফ্ল্যাগ বা `privileged: true` কখনো সাধারণ অ্যাপে ব্যবহার করো না। Privileged কন্টেইনার হোস্টের প্রায় সব ক্ষমতা পায় — এটি কন্টেইনার বিচ্ছিন্নতাকে প্রায় সম্পূর্ণ ভেঙে দেয়। শুধু বিশেষ সিস্টেম কাজে, আলাদা Namespace ও কঠোর পর্যালোচনার পর এটি ব্যবহার করা যায়।

৯.৬ ব্যবহারিক কমান্ড ও কনফিগারেশন

নিরাপদ Dockerfile (মাল্টি-স্টেজ, নন-রুট):

```
# বিল্ড ধাপ — এখানে dev নির্ভরতা থাকতে পারে
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# চলার ধাপ — শুধু দরকারি ফাইল
FROM node:20-alpine
WORKDIR /app
# নন-রুট ব্যবহারকারী (node ব্যবহারকারী আগে থেকেই আছে)
COPY --from=build --chown=node:node /app/dist ./dist
COPY --from=build --chown=node:node /app/node_modules ./node_modules
COPY --from=build --chown=node:node /app/package.json ./
USER node
EXPOSE 3000
# স্বাস্থ্য পরীক্ষা
HEALTHCHECK --interval=30s --timeout=3s \
  CMD node -e "fetch('http://localhost:3000/health').then(r => { if (!r.ok)
process.exit(1); })"
CMD ["node", "dist/index.js"]
```

ইমেজ স্ক্যানিং:

```
# Trivy দিয়ে ইমেজ স্ক্যান
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  aquasec/trivy image technova-api:1.0.0
# উচ্চ তীব্রতার আবিষ্কারে ব্যর্থ হওয়া (CI গেটের জন্য)
trivy image --severity HIGH,CRITICAL --exit-code 1 technova-api:1.0.0
# সিক্রেট স্ক্যানও চালাও — ইমেজে সিক্রেট আছে কি না
trivy image --scanners secret technova-api:1.0.0
```

RBAC — সীমিত ভূমিকা:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: technova-api
  namespace: technova-prod
---
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: technova-api-role
  namespace: technova-prod
rules:
  - apiGroups: [""]
    resources: ["secrets", "configmaps"]
    verbs: ["get", "list"]
  # লক্ষ করো: pods, deployments মুছে ফেলা বা বদলানোর অনুমতি নেই
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: technova-api-binding
  namespace: technova-prod
subjects:
  - kind: ServiceAccount
    name: technova-api
roleRef:
  kind: Role
  name: technova-api-role
  apiGroup: rbac.authorization.k8s.io
# পরীক্ষা:
# kubectl auth can-i delete pods --as=system:serviceaccount:technova-prod:technova-api -n
technova-prod
# উত্তর হওয়া উচিত: no
```

Pod Security ও Network Policy:

```
# Pod-এ নিরাপত্তা প্রসঙ্গ (Restricted স্তরের দিকে)
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  seccompProfile:
    type: RuntimeDefault
  fsGroup: 10001
containers:
  - name: api
    image: technova-api:1.0.0
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]
---
# Namespace-এ Pod Security Admission চালু করা
# kubectl label namespace technova-prod \
#   pod-security.kubernetes.io/enforce=restricted \
#   pod-security.kubernetes.io/audit=restricted
---
# Network Policy — API শুধু MongoDB-তে যেতে পারবে
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-to-mongo
  namespace: technova-prod
spec:
```

```
  podSelector:
    matchLabels:
      app: technova-api
  policyTypes: ["Egress"]
  egress:
    - to:
        - podSelector:
            matchLabels:
              app: mongodb
      ports:
        - port: 27017
          protocol: TCP
```

৯.৭ প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — রুট কন্টেইনার থেকে পলায়ন। একটি লগ সংগ্রহকারী কন্টেইনার privileged হিসেবে চলছিল, কারণ ডেভেলপার হোস্টের ফাইল পড়তে চেয়েছিলেন। সেই কন্টেইনার আপসিত হওয়ার পর আক্রমণকারী হোস্টে পৌঁছে গেল। শিক্ষা: হোস্টের তথ্য দরকার হলে DaemonSet-এর মতো নির্দিষ্ট ব্যবস্থা ব্যবহার করো, অ্যাপ কন্টেইনারকে privileged করো না।

পরিস্থিতি ২ — `:latest` ট্যাগের বিপদ। ডিপ্লয়ে `:latest` ব্যবহার করা হতো। একদিন একটি ভাঙা ইমেজ latest হয়ে গেল, আর সব Pod পুনরায় চালু হয়ে ভাঙা সংস্করণে চলে গেল — কেউ জানল না কোন সংস্করণ চলছে। শিক্ষা: অপরিবর্তনীয় ট্যাগ ব্যবহার করো — যেমন git কমিটের হ্যাশ — যাতে প্রতিটি ডিপ্লয় শনাক্তযোগ্য হয় ও রোলব্যাক সম্ভব হয়।

পরিস্থিতি ৩ — Network Policy না থাকায় পার্শ্বীয় চলাচল। একটি কম গুরুত্বপূর্ণ ফ্রন্টএন্ড Pod আপসিত হলো, আর সেখান থেকে আক্রমণকারী সরাসরি ডাটাবেস Pod-এ পৌঁছে গেল, কারণ কোনো Network Policy ছিল না। শিক্ষা: Network Policy শুধু নিরাপত্তা নয়, এটি ক্ষতির পরিসর সীমিত করার প্রধান উপায়।

> **REAL WORLD:** বাস্তবে কন্টেইনার নিরাপত্তার সবচেয়ে সাধারণ ব্যর্থতা হলো ডিফল্টের ওপর ভরসা করা: ডিফল্ট রুট ব্যবহারকারী, ডিফল্ট খোলা নেটওয়ার্ক, ডিফল্ট cluster-admin। নিরাপদ ক্লাস্টার মানে ইচ্ছাকৃতভাবে প্রতিটি ডিফল্টকে প্রশ্ন করা ও কঠোর করা — এবং সেই সিদ্ধান্তগুলোকে কোডে (GitOps-এ) লিপিবদ্ধ রাখা।

৯.৮ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তা নিয়ন্ত্রণের খরচ এখানে মূলত জটিলতা ও ডিবাগের সময়। Restricted Pod Security চালু করলে কিছু পুরোনো অ্যাপ ভাঙতে পারে — যেমন রুট হিসেবে ফাইল লেখা অ্যাপ। তাই প্রথমে audit মোডে চালিয়ে দেখো কোন Pod নীতি ভাঙছে, তারপর ধীরে ধীরে enforce করো। Network Policy-র খরচ হলো পরীক্ষার সময় — ভুল নীতি বৈধ ট্রাফিক আটকে দিতে পারে। Seccomp ও capabilities-এর খরচ কম, কারণ RuntimeDefault প্রোফাইল বেশিরভাগ অ্যাপে কাজ করে। রানটাইম নিরাপত্তা সরঞ্জামের খরচ হলো সার্ভার সম্পদ ও সতর্কতার শব্দ — শুরুতে শুধু গুরুতর আচরণে সতর্কতা চালু করো।

৯.৯ সাধারণ ভুল

- কন্টেইনারকে ভার্চুয়াল মেশিন ভাবা — একই kernel ভাগ করা ভুলে যাওয়া।
- রুট হিসেবে কন্টেইনার চালানো, বা `privileged: true` ব্যবহার করা।
- ইমেজে সিক্রেট রাখা, বা `:latest` ট্যাগ ব্যবহার করা।

- বড় বেস ইমেজে অপ্রয়োজনীয় প্যাকেজ রাখা — আক্রমণের পৃষ্ঠ বাড়ানো।
- RBAC-এ `cluster-admin` দেওয়া, বা Secret-এর প্রবেশাধিকার সীমিত না করা।
- Network Policy না লেখা — সব Pod সব Pod-এর সঙ্গে কথা বলতে দেওয়া।
- ইমেজ স্ক্যান না করা, বা স্ক্যানের আবিষ্কার উপেক্ষা করা।
- Admission নীতি ছাড়া ডিপ্লয় — ভুল Pod প্রোডাকশনে পৌঁছে যাওয়া।

৯.১০ সমস্যা সমাধান

সমস্যা: নন-রুট Pod ফাইল লিখতে পারছে না। ভলিউম বা ফাইলের মালিকানা রুটের। `fsGroup` নির্ধারণ করো, বা initContainer দিয়ে মালিকানা ঠিক করো। রিড-অনলি ফাইল সিস্টেমে লেখার দরকার হলে নির্দিষ্ট পথে `emptyDir` ভলিউম মাউন্ট করো।

সমস্যা: Network Policy চালু করার পর অ্যাপ ডাটাবেসে যেতে পারছে না। নীতিতে egress নিয়ম ও DNS (পোর্ট ৫৩) ভুলে যাওয়া হয়েছে কি না দেখো — DNS ছাড়া নাম সমাধান হয় না। প্রথমে নীতি ছাড়া সংযোগ পরীক্ষা করো, তারপর নীতি যোগ করে আবার পরীক্ষা করো।

সমস্যা: Pod Security Admission Pod আটকে দিচ্ছে। আটকে যাওয়ার কারণের বার্তা পড়ো — সাধারণত `runAsNonRoot` বা capabilities-এর অভাব। Namespace-এ audit মোডে চালিয়ে কোন Podগুলো ভাঙছে তা শনাক্ত করো, তারপর একে একে ঠিক করো।

> **TROUBLESHOOTING:** কন্টেইনার সমস্যা ডিবাগ করার সময় `kubectl exec` দিয়ে ভেতরে ঢুকে দেখা সহায়ক, কিন্তু প্রোডাকশনে exec-এর প্রবেশাধিকার সীমিত রাখো — এটি নিজেই একটি নিরাপত্তা ঝুঁকি। ডিবাগের জন্য একটি আলাদা, অস্থায়ী Pod ব্যবহার করো।

ইমেজ বিল্ডের গভীরতা। নিরাপদ ইমেজ তৈরির একটি গুরুত্বপূর্ণ অভ্যাস হলো মাল্টি-স্টেজ বিল্ডকে আরও এগিয়ে নেওয়া: বিল্ড ধাপে কম্পাইলার, পরীক্ষার সরঞ্জাম ও সোর্স কোড থাকলেও চলার ধাপে সেগুলোর কোনোটিই থাকা উচিত নয়। কারণ চলার ধাপে থাকা প্রতিটি সরঞ্জাম — যেমন একটি শেল, একটি প্যাকেজ ম্যানেজার, বা একটি কম্পাইলার — আক্রমণকারীর কাজে লাগতে পারে। Distroless ইমেজ এই নীতিকে চরমে নিয়ে যায়: সেখানে শেলই থাকে না, শুধু অ্যাপ ও তার রানটাইম থাকে। তবে distroless-এর খরচ হলো ডিবাগের অসুবিধা — ভেতরে ঢুকে দেখার উপায় কম। তাই শুরুতে ছোট Alpine ইমেজ দিয়ে শুরু করা, আর পরিপক্ক হলে distroless বিবেচনা করা একটি বাস্তবসম্মত পথ। আরেকটি অভ্যাস হলো ইমেজে শুধু নির্দিষ্ট সংস্করণের প্যাকেজ ব্যবহার করা — `apt-get install curl` নয়, `curl=8.5.0-*`-এর মতো নির্দিষ্ট সংস্করণ — যাতে প্রতিটি বিল্ড একই জিনিস তৈরি করে ও SBOM-এর সঙ্গে মেলে।

ক্লাস্টার-স্তরের নিরাপত্তা। Pod-এর নিরাপত্তা যতই ভালো হোক, ক্লাস্টারের নিয়ন্ত্রণ সমতল (control plane) দুর্বল থাকলে সবই ব্যর্থ হয়। API সার্ভারের প্রবেশাধিকার সীমিত রাখো — শুধু অনুমোদিত নেটওয়ার্ক থেকে, প্রমাণীকরণসহ। etcd-তে সংরক্ষিত ডেটা — যেখানে Secret থাকে — এনক্রিপ্ট করা উচিত, আর etcd-র ব্যাকআপও এনক্রিপ্ট করে সংরক্ষণ করা উচিত। নোডের নিরাপত্তাও ভুলে যেও না: হোস্ট অপারেটিং সিস্টেম নিয়মিত আপডেট করা, SSH-তে কী-ভিত্তিক প্রমাণীকরণ ও সীমিত প্রবেশ, আর অপ্রয়োজনীয় সেবা বন্ধ রাখা। ক্লাস্টারের অডিট লগ চালু রাখো — কে কী করল, কখন করল — কারণ কোনো ঘটনা ঘটলে এই লগই তদন্তের ভিত্তি। ছোট দলের জন্য পরিচালিত Kubernetes সেবা ব্যবহার করা প্রায়ই ভালো, কারণ নিয়ন্ত্রণ সমতলের নিরাপত্তা তখন সরবরাহকারীর দায়িত্বে থাকে, তবে তাতে নিজের Pod, RBAC ও Network Policy-র দায়িত্ব কমে না।

রানটাইমে কী দেখবে। রানটাইম নিরাপত্তা শুধু সরঞ্জাম বসানো নয়, কী দেখতে হবে তা জানা। সন্দেহজনক লক্ষণগুলো হলো: একটি ওয়েব Pod থেকে শেল চালু হওয়া, একটি Pod থেকে অপ্রত্যাশিত বাইরের ঠিকানায় সংযোগ, ফাইল সিস্টেমে অপ্রত্যাশিত লেখা, বা একটি Pod-এর হঠাৎ করে বেশি CPU ব্যবহার। এই লক্ষণগুলো দেখলে প্রথমে Pod-টিকে বিচ্ছিন্ন করো — নেটওয়ার্ক নীতি দিয়ে বা Pod মুছে দিয়ে — তারপর লগ ও ইমেজ পরীক্ষা করো। রানটাইম সতর্কতায় সবচেয়ে বড় ভুল হলো প্রতিটি ছোট ঘটনায় আতঙ্কিত হওয়া; শুরুতে শুধু উচ্চ-আস্থার নিয়ম চালু করো, যেমন শেল চালু হওয়া বা সংবেদনশীল ফাইল পড়া, আর ধীরে ধীরে নিয়ম বাড়াও।

৯.১১ সারাংশ

কন্টেইনার নিরাপত্তা মানে একটিমাত্র সরঞ্জাম নয়, বরং স্তরে স্তরে সীমা বসানো: ছোট, স্ক্যান করা, স্বাক্ষরিত ইমেজ; নন-রুট, রিড-অনলি, capabilities-বাদ দেওয়া Pod; RuntimeDefault seccomp; সীমিত RBAC; Restricted Pod Security; Network Policy দিয়ে মাইক্রোসেগমেন্টেশন; সিক্রেটের এনক্রিপশন ও সীমিত প্রবেশাধিকার; রানটাইম পর্যবেক্ষণ; এবং admission নীতিতে শেষ পরীক্ষা। প্রতিটি স্তরের প্রশ্ন একই: আক্রমণকারী এখানে পৌঁছালে আর কী করতে পারবে? উত্তর যত ছোট, নিরাপত্তা তত ভালো।

৯.১২ পর্যালোচনা প্রশ্ন

1. কন্টেইনার ও ভার্চুয়াল মেশিনের মধ্যে নিরাপত্তার দিক থেকে মূল পার্থক্য কী?
2. নন-রুট কন্টেইনার কেন নিরাপদ? Dockerfile ও Pod-এ কী কী নির্ধারণ করতে হয়?
3. Linux capabilities কী? `drop: ["ALL"]` লেখার অর্থ কী?
4. Seccomp কীভাবে কাজ করে? RuntimeDefault প্রোফাইল কেন ভালো শুরু?
5. RBAC-এ Role, ClusterRole, RoleBinding ও ClusterRoleBinding-এর পার্থক্য ব্যাখ্যা করো।
6. Pod Security Standards-এর তিনটি স্তর কী কী? নতুন কাজে কোনটি লক্ষ্য করা উচিত?
7. Network Policy কীভাবে পার্শ্বীয় চলাচল থামায়? DNS-র জন্য কী মনে রাখতে হয়?
8. Kubernetes Secret ডিফল্টভাবে কেন নিরাপদ নয়? কী কী ব্যবস্থা নেওয়া উচিত?
9. ইমেজ স্ক্যানিং কী ধরে আর কী ধরে না? `:latest` ট্যাগ কেন বিপজ্জনক?
10. Admission নীতি কী? Pod Security Admission ও নীতি ইঞ্জিনের ভূমিকা ব্যাখ্যা করো।

৯.১৩ ব্যবহারিক অনুশীলন

1. **নিরাপদ Dockerfile:** একটি Node.js অ্যাপের জন্য মাল্টি-স্টেজ, নন-রুট Dockerfile লেখো। Trivy দিয়ে স্ক্যান করো এবং উচ্চ তীব্রতার আবিষ্কার থাকলে বেস ইমেজ বদলে ঠিক করো।
2. **RBAC পরীক্ষা:** একটি সীমিত Role ও ServiceAccount তৈরি করো। `kubectl auth can-i` দিয়ে পরীক্ষা করো সে কী করতে পারে আর কী পারে না। তারপর একটি Pod-এ সেই ServiceAccount ব্যবহার করো।
3. **Network Policy:** একটি ল্যাব Namespace-এ দুটি Pod তৈরি করো — একটি API, একটি ডাটাবেস। প্রথমে কোনো নীতি ছাড়া সংযোগ পরীক্ষা করো, তারপর শুধু API থেকে ডাটাবেসে যাওয়ার অনুমতি দিয়ে নীতি লেখো ও আবার পরীক্ষা করো।
4. **Pod Security:** একটি Namespace-এ Pod Security Admission audit মোডে চালাও। একটি privileged Pod তৈরি করার চেষ্টা করে দেখো কী বার্তা আসে, তারপর Pod-টিকে Restricted স্তরে ঠিক করো।

অধ্যায় ১০ • PART II — Application & Software Supply Chain Security

Chapter 10 — Software Supply Chain Security

১০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে শিক্ষার্থী ব্যাখ্যা করতে পারবে সফটওয়্যার সরবরাহ শৃঙ্খল (software supply chain) বলতে কী বোঝায় — কোড লেখা থেকে শুরু করে লাইব্রেরি, বিল্ড, পরীক্ষা, স্বাক্ষর, বিতরণ ও ডিপ্লয় পর্যন্ত প্রতিটি ধাপ — এবং প্রতিটি ধাপে আক্রমণকারী কীভাবে ঢুকতে পারে। সে SBOM, CycloneDX, SPDX, artifact provenance, Cosign ও Sigstore, SLSA কাঠামো, নির্ভরতার অখণ্ডতা, বিল্ড নিরাপত্তা ও রিলিজ যাচাই — প্রতিটি ধারণার অর্থ, কাজ ও সীমা বলতে পারবে। শিক্ষার্থী TechNova-র MERN অ্যাপের জন্য একটি SBOM তৈরি করতে, একটি artifact-এ স্বাক্ষর করতে ও তা যাচাই করতে পারবে, এবং একটি রিলিজের আগে যাচাই-তালিকা প্রয়োগ করতে পারবে। সবশেষে সে বুঝবে সরবরাহ শৃঙ্খল নিরাপত্তা কোনো একক সরঞ্জাম নয়; এটি প্রমাণের একটি শৃঙ্খল — প্রতিটি ধাপে "এটি কোথা থেকে এলো, কে এটি বানিয়েছে, আর এটি বদলানো হয়নি" — এই প্রশ্নের উত্তর দেওয়া।

১০.২ পূর্বশর্ত পুনরালোচনা

অধ্যায় ৮ থেকে SCA ও নির্ভরতা স্ক্যানিং, অধ্যায় ৯ থেকে ইমেজ স্ক্যানিং ও স্বাক্ষরের ধারণা এখানে ভিত্তি হিসেবে কাজ করবে। CI/CD পাইপলাইনের ধারণা — সোর্স কোড, বিল্ড, artifact, রেজিস্ট্রি ও ডিপ্লয় — ঝালিয়ে নাও, কারণ সরবরাহ শৃঙ্খল মূলত এই পাইপলাইনেরই নিরাপত্তা। হ্যাশ সম্পর্কে সংক্ষেপে: হ্যাশ একটি ফাইলের আঙুলের ছাপ — ফাইল সামান্য বদলালেও হ্যাশ সম্পূর্ণ বদলে যায়, তাই হ্যাশ দিয়ে অখণ্ডতা যাচাই করা যায়। ডিজিটাল স্বাক্ষর সম্পর্কে: স্বাক্ষর হলো হ্যাশের ওপর ব্যক্তিগত কী দিয়ে করা একটি সিলমোহর, যা সরকারি কী দিয়ে যাচাই করা যায় — এটি প্রমাণ করে কে স্বাক্ষর করেছে ও ফাইলটি বদলানো হয়নি। এই দুটি ধারণা না বুঝলে SBOM ও স্বাক্ষরের অংশ কঠিন লাগবে।

১০.৩ প্রথম নীতি থেকে ব্যাখ্যা: সরবরাহ শৃঙ্খল কেন আক্রমণের লক্ষ্য

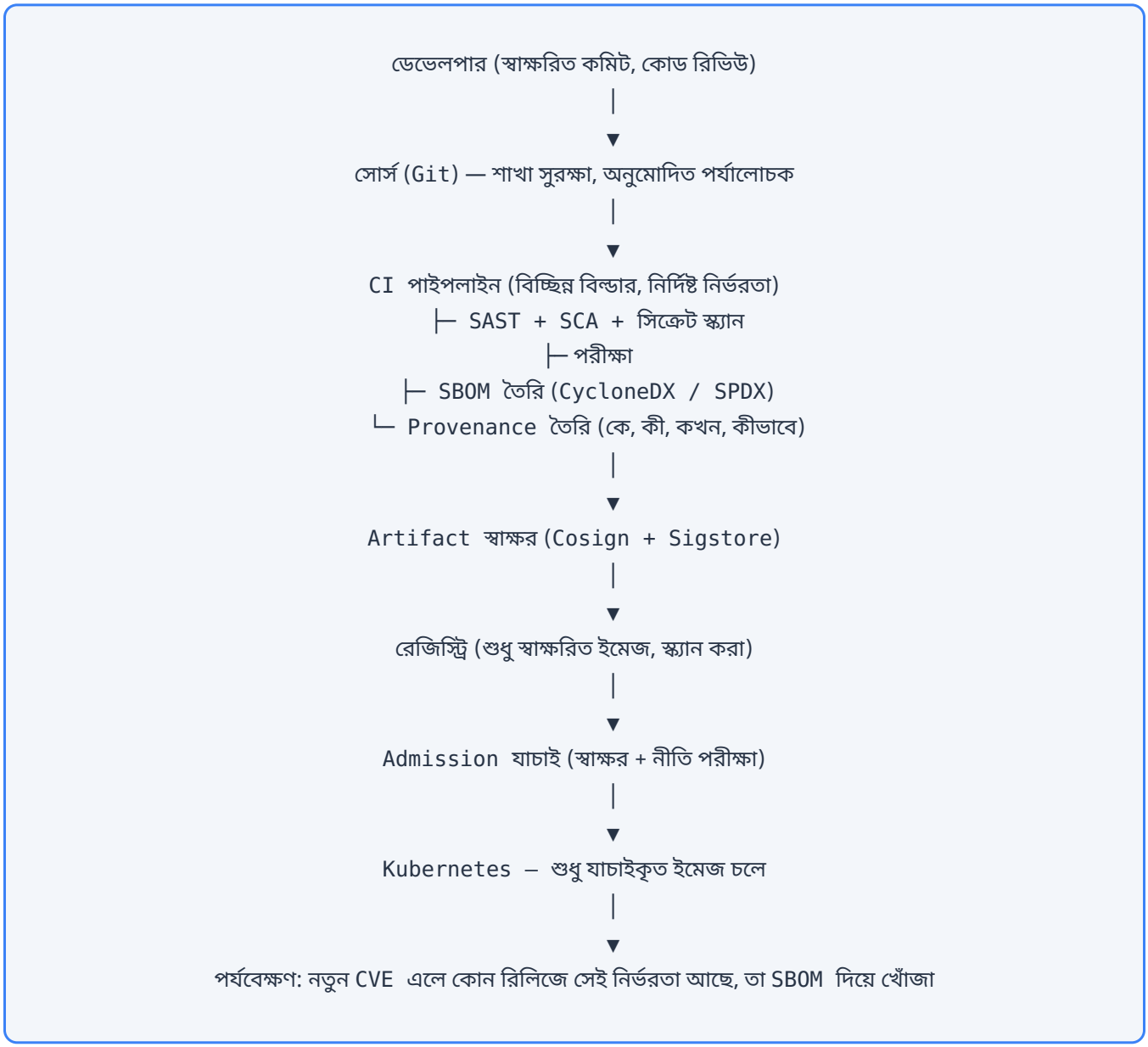
একটি আধুনিক অ্যাপের বেশিরভাগ কোড নিজের লেখা নয়। TechNova-র MERN অ্যাপে Express, React, Mongoose, শত শত ছোট লাইব্রেরি, বেস ইমেজ, বিল্ড সরঞ্জাম ও ডিপ্লয় স্ক্রিপ্ট — সব মিলিয়ে হাজার হাজার উপাদান। আক্রমণকারীর যুক্তি সহজ: সরাসরি TechNova-র সার্ভার ভাঙা কঠিন হতে পারে, কিন্তু তার ব্যবহৃত একটি জনপ্রিয় লাইব্রেরিতে ক্ষতিকর কোড ঢুকিয়ে দিলে সেই লাইব্রেরি ব্যবহার করা হাজার হাজার অ্যাপ একসঙ্গে আপসিত হয়। একে বলে সরবরাহ শৃঙ্খল আক্রমণ — আক্রমণ সরাসরি লক্ষ্যে নয়, তার সরবরাহকারীর ওপর।

এই সমস্যা ঘটে কারণ সফটওয়্যার তৈরি একটি বিশ্বাসের শৃঙ্খল: ডেভেলপার লাইব্রেরিকে বিশ্বাস করে, বিল্ড সিস্টেম সোর্স কোডকে বিশ্বাস করে, ডিপ্লয় সিস্টেম বিল্ড করা artifact-কে বিশ্বাস করে। শৃঙ্খলের কোনো একটি জায়গায় বিশ্বাস ভাঙলে পুরো শৃঙ্খল ভেঙে যায় — কিন্তু ভাঙনটি চোখে দেখা যায় না, কারণ artifact দেখতে একই রকম লাগে। তাই সরবরাহ শৃঙ্খল নিরাপত্তার মূল প্রশ্ন হলো: প্রতিটি ধাপে কীভাবে প্রমাণ রাখা যায় যে যা আসার কথা ছিল, তাই এসেছে, আর মাঝখানে কেউ বদলায়নি? এই প্রশ্নের উত্তরের তিনটি অংশ: তালিকা (কী কী আছে — SBOM), উৎস প্রমাণ (কে, কোথায়, কীভাবে বানিয়েছে — provenance), ও স্বাক্ষর (এটি যাচাইযোগ্য — Cosign/Sigstore)।

আরেকটি প্রথম নীতি হলো, বিশ্বাস কমানো — অর্থাৎ প্রতিটি ধাপে যত কম অনুমান করা যায়, তত ভালো। একটি পাইপলাইন যদি ইন্টারনেট থেকে যেকোনো স্ক্রিপ্ট ডাউনলোড করে চালায়, তবে সেই স্ক্রিপ্টের মালিকের ওপর পুরো বিশ্বাস রাখা হয়। নির্ভরতার সংস্করণ যদি নির্দিষ্ট না থাকে, তবে প্রতিটি বিল্ডে ভিন্ন কোড আসতে পারে। বিল্ড যদি পুনরুৎপাদনযোগ্য না হয় — অর্থাৎ একই সোর্স থেকে দুই বার ভিন্ন artifact তৈরি হয় — তবে কেউ যাচাই করতে পারে না artifact-টি আসল সোর্স থেকেই এসেছে কি না। এই কারণে SLSA কাঠামো বিল্ডের পুনরুৎপাদনযোগ্যতা ও বিচ্ছিন্নতাকে এত গুরুত্ব দেয়।

CONCEPT: সরবরাহ শৃঙ্খল নিরাপত্তাকে ভাবো একটি খাদ্য সরবরাহ শৃঙ্খলের মতো: প্রতিটি উপাদানের উৎস, প্রতিটি ধাপের প্রক্রিয়া, ও প্রতিটি হস্তান্তরের সিলমোহর লিপিবদ্ধ থাকে। একটি উপাদানের উৎস জানা না থাকলে, বা সিলমোহর ভাঙা থাকলে, পুরো পণ্যটি সন্দেহজনক। সফটওয়্যারে SBOM হলো উপাদানের তালিকা, provenance হলো প্রক্রিয়ার বিবরণ, আর স্বাক্ষর হলো সিলমোহর।

১০.৪ স্থাপত্য: TechNova-র সরবরাহ শৃঙ্খল



এই চিত্রের মূল বার্তা হলো, নিরাপত্তা শৃঙ্খলের প্রতিটি হস্তান্তরে যাচাই হয়। সোর্স থেকে বিল্ডে যাওয়ার সময় স্বাক্ষরিত কমিট যাচাই হয়, বিল্ড থেকে রেজিস্ট্রিতে যাওয়ার সময় SBOM ও provenance তৈরি হয়, আর রেজিস্ট্রি থেকে ক্লাস্টারে যাওয়ার সময় স্বাক্ষর যাচাই হয়। কোনো একটি হস্তান্তরে যাচাই না থাকলে আক্রমণকারী সেই ফাঁক দিয়ে ঢুকতে পারে।

১০.৫ অভ্যন্তরীণ কার্যপ্রণালী

সরবরাহ শৃঙ্খল হুমকি। প্রধান হুমকিগুলো হলো: নির্ভরতা বিভ্রান্তি (dependency confusion) — ব্যক্তিগত লাইব্রেরির নামে একটি সর্বজনীন লাইব্রেরি প্রকাশ করে পাইপলাইনকে ভুল সংস্করণ ডাউনলোড করানো; টাইপোস্কোয়াটিং — জনপ্রিয় লাইব্রেরির নামের কাছাকাছি নামে ক্ষতিকর লাইব্রেরি প্রকাশ; আপসিত রক্ষণাবেক্ষণকারী — একটি লাইব্রেরির রক্ষণাবেক্ষণকারীর অ্যাকাউন্ট দখল করে ক্ষতিকর সংস্করণ প্রকাশ; বিল্ড সিস্টেমে ঢোকা — CI-তে ক্ষতিকর স্ক্রিপ্ট চালিয়ে artifact বদলে দেওয়া; ও আপডেটে ঢোকা — আপডেট সার্ভার বা প্রক্রিয়া দখল করে ক্ষতিকর আপডেট পাঠানো। প্রতিটি হুমকির প্রতিকার আলাদা, কিন্তু সাধারণ নীতি এক: উৎস যাচাই, সংস্করণ নির্দিষ্টকরণ, ও স্বাক্ষর যাচাই।

SBOM, CycloneDX ও SPDX। SBOM হলো Software Bill of Materials — একটি সফটওয়্যারে কী কী উপাদান আছে তার তালিকা: প্রতিটি লাইব্রেরির নাম, সংস্করণ, লাইসেন্স, হ্যাশ ও উৎস। CycloneDX ও SPDX হলো SBOM লেখার দুটি মান — দুটিই JSON বা XML-এ লেখা যায়, দুটিই সরঞ্জাম দিয়ে তৈরি ও পড়া যায়। পার্থক্য: CycloneDX নিরাপত্তা ও সরবরাহ শৃঙ্খলের ওপর বেশি জোর দেয়, SPDX লাইসেন্সের ওপর বেশি জোর দেয়। TechNova-র জন্য যেকোনো একটি বেছে নিয়ে ধারাবাহিকভাবে ব্যবহার করাই গুরুত্বপূর্ণ। SBOM-এর আসল মূল্য তখনই বোঝা যায় যখন একটি নতুন CVE প্রকাশ হয়: SBOM দিয়ে সঙ্গে সঙ্গে খুঁজে পাওয়া যায় কোন রিলিজে সেই দুর্বল লাইব্রেরি আছে, আর কোনটিতে নেই — অনুমানে নয়, তালিকা দেখে।

Provenance ও SLSA। Provenance হলো একটি artifact-এর জন্ম-বিবরণ: কোন সোর্স কমিট থেকে, কোন বিল্ডারে, কোন সময়ে, কোন ধাপে এটি তৈরি হয়েছে। SLSA হলো Supply-chain Levels for Software Artifacts — একটি কাঠামো যা বিল্ডের নিরাপত্তাকে স্তরে ভাগ করে: সোর্সের সুরক্ষা, বিল্ডের বিচ্ছিন্নতা, provenance-এর উপস্থিতি, ও পুনরুৎপাদনযোগ্যতা। উচ্চতর স্তরে বিল্ডারকে বিচ্ছিন্ন ও যাচাইযোগ্য হতে হয়, আর provenance-কে স্বাক্ষরিত হতে হয়। SLSA কোনো সরঞ্জাম নয় — এটি একটি পরিমাপক, যা দিয়ে একটি দল বলতে পারে "আমাদের বিল্ড এই স্তরে আছে, আর পরের স্তরে যেতে কী দরকার"। পুনরুৎপাদনযোগ্য বিল্ড মানে একই সোর্স থেকে একই artifact বারবার তৈরি করা যায় — এটি প্রমাণ করে বিল্ডের সময় কিছু বদলানো হয়নি।

Cosign ও Sigstore। স্বাক্ষরের পুরোনো সমস্যা ছিল কী ব্যবস্থাপনা: ব্যক্তিগত কী কোথায় রাখা হবে, সরকারি কী কীভাবে বিতরণ করা হবে? Sigstore সেই সমস্যার সমাধান করে: এটি একটি সর্বজনীন স্বচ্ছ লগ (Rekor) রাখে — প্রতিটি স্বাক্ষর সেখানে লিপিবদ্ধ হয় — আর অস্থায়ী শংসাপত্র দেয়, যাতে দীর্ঘমেয়াদি কী সংরক্ষণের দরকার হয় না। Cosign হলো Sigstore ব্যবহার করে কন্টেইনার ইমেজ ও অন্যান্য artifact-এ স্বাক্ষর করার সরঞ্জাম। স্বাক্ষর যাচাই মানে: Rekor-এ স্বাক্ষরটি আছে কি না দেখা, শংসাপত্রের পরিচয় যাচাই করা, ও artifact-এর হ্যাশ মেলানো। কীলেস স্বাক্ষর (keyless signing) মানে ডেভেলপারকে নিজে কী সংরক্ষণ করতে হয় না — তার পরিচয় (যেমন GitHub অ্যাকাউন্ট) দিয়েই অস্থায়ী শংসাপত্র তৈরি হয়।

নির্ভরতার অখণ্ডতা ও বিল্ড নিরাপত্তা। নির্ভরতার অখণ্ডতা মানে: ডাউনলোড করা প্রতিটি লাইব্রেরি ঠিক সেই সংস্করণ, যার হ্যাশ লকফাইলে আছে। `package-lock.json` কমিট করা, লকফাইল ছাড়া ইনস্টল না করা (`npm ci` ব্যবহার করা, `npm install` নয়), ও নির্ভরতার হ্যাশ যাচাই করা — এগুলোই অখণ্ডতার ভিত্তি। বিল্ড নিরাপত্তা মানে: বিল্ডার বিচ্ছিন্ন থাকবে, বিল্ড স্ক্রিপ্ট পর্যালোচিত হবে, সিক্রেট বিল্ড লগে ফাঁস হবে না, আর বিল্ডের প্রতিটি ধাপ লিপিবদ্ধ থাকবে। একটি ভালো অভ্যাস হলো বিল্ডে ইন্টারনেটের প্রবেশ সীমিত করা — শুধু অনুমোদিত রেজিস্ট্রি থেকে নির্ভরতা আনা।

রিলিজ যাচাই। রিলিজের আগে যাচাই মানে: SBOM আছে কি না, স্বাক্ষর যাচাই হয় কি না, স্ক্যান পরিষ্কার কি না, পরীক্ষা পাস করেছে কি না, ও পরিবর্তনের তালিকা (changelog) আছে কি না — এই সবকিছু পরীক্ষা করা। ডিপ্লয়ের সময় admission নীতি আবার স্বাক্ষর যাচাই করে — যাতে রেজিস্ট্রিতে কেউ হাতে একটি অস্বাক্ষরিত ইমেজ ঢুকিয়ে দিলেও সেটি ক্লাস্টারে না চলে।

> **WARNING:** SBOM বা স্বাক্ষর থাকলেই একটি উপাদান নিরাপদ হয়ে যায় না — সেগুলো শুধু প্রমাণ দেয় উপাদানটি কী ও কোথা থেকে এসেছে। একটি পরিচিত দুর্বল লাইব্রেরির SBOM থাকলেও সেটি দুর্বলই থাকে; SBOM শুধু সেই দুর্বলতা দ্রুত খুঁজে পেতে সাহায্য করে। স্বাক্ষরও একই — স্বাক্ষরিত ক্ষতিকর কোডও ক্ষতিকর, যদি স্বাক্ষরকারী নিজেই আপসিত হয়। তাই যাচাইয়ের সঙ্গে স্ক্যানিং ও পর্যালোচনাও দরকার।

১০.৬ ব্যবহারিক কমান্ড ও কনফিগারেশন

```
# --- নির্ভরতার অখণ্ডতা ---
# লকফাইল অনুযায়ী, ঠিক সংস্করণ ইনস্টল করো (CI-তে এটিই ব্যবহার করো)
npm ci
# লকফাইল ছাড়া ইনস্টল করো না — npm install সংস্করণ বদলে দিতে পারে
# লকফাইল সবসময় git-এ কমিট করো

# --- SBOM তৈরি: Syft (CycloneDX) ---
# ইমেজ থেকে SBOM
syft technova-api:1.0.0 -o cyclonedx-json -o sbom.cyclonedx.json
# সোর্স ডিরেক্টরি থেকে SBOM
syft . -o spdx-json -o sbom.spdx.json
# SBOM-এ কী আছে দেখো:
# cat sbom.cyclonedx.json | head -n 60

# --- SBOM দিয়ে দুর্বলতা খোঁজা: Grype ---
grype sbom:sbom.cyclonedx.json
# একটি নির্দিষ্ট CVE আছে কি না খোঁজা:
# grype sbom:sbom.cyclonedx.json | grep CVE-2026-XXXX

# --- Cosign দিয়ে স্বাক্ষর (Sigstore, কীলেস) ---
# ইনস্টল: https://docs.sigstore.dev/cosign/installation থেকে
# স্বাক্ষর (পরিচয় যাচাইয়ের জন্য ব্রাউজার খুলতে পারে — ল্যাবে করো)
cosign sign --yes technova.example/technova-api:1.0.0
# যাচাই — শুধু নির্দিষ্ট পরিচয়ের স্বাক্ষর গ্রহণ করো
cosign verify \
  --certificate-identity-regexp '.*@technova.example' \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com \
  technova.example/technova-api:1.0.0

# --- SLSA provenance যাচাইয়ের ধারণা ---
# GitHub Actions-এ SLSA জেনারেটর ব্যবহার করলে provenance
# স্বয়ংক্রিয়ভাবে তৈরি হয়। যাচাইয়ের সময় দেখো:
#   - সোর্স কমিটটি প্রত্যাশিত শাখার কি না
#   - বিল্ডারটি প্রত্যাশিত কর্মপ্রবাহের কি না
#   - artifact-এর হ্যাশ provenance-এর হ্যাশের সঙ্গে মেলে কি না
```

রিলিজ যাচাই-তালিকা (CI গেটের ধারণা):

```
# রিলিজের আগে প্রতিটি ধাপ পাস করতে হবে:
#  1. সব পরীক্ষা পাস
#  2. SAST + SCA স্ক্যান — গুরুতর আবিষ্কার নেই
#  3. SBOM তৈরি হয়েছে ও সংরক্ষিত হয়েছে
#  4. ইমেজ স্বাক্ষরিত হয়েছে (Cosign)
#  5. স্বাক্ষর যাচাই সফল হয়েছে
```

```
#  6. Changelog ও সংস্করণ ট্যাগ আছে
# কোনো ধাপ ব্যর্থ হলে রিলিজ থেমে যাবে — মানুষের হস্তক্ষেপ ছাড়া নয়।
```

১০.৭ প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — একটি লাইব্রেরিতে জরুরি CVE। একটি জনপ্রিয় লগিং লাইব্রেরিতে গুরুতর দুর্বলতা প্রকাশ হলো। SBOM ছাড়া দল জানত না তাদের কোন সেবায় সেই লাইব্রেরি আছে — খুঁজতে দিন লেগে যেত। SBOM থাকায় কয়েক মিনিটে তালিকা পাওয়া গেল, আর শুধু আক্রান্ত সেবাগুলো প্যাচ করা হলো। শিক্ষা: SBOM-এর মূল্য জরুরি অবস্থায় বোঝা যায়, শান্ত সময়ে নয়।

পরিস্থিতি ২ — অস্বাক্ষরিত ইমেজ ডিপ্লয়। একজন ডেভেলপার স্থানীয়ভাবে তৈরি একটি ইমেজ সরাসরি রেজিস্ট্রিতে ঠেলে দিলেন। Admission নীতি স্বাক্ষর যাচাই করে সেটি আটকে দিল। শিক্ষা: স্বাক্ষর যাচাই শুধু নীতি হিসেবে লিখলে হয় না — সেটি admission-এ প্রয়োগ করতে হয়, নইলে কেউ ভুলে বা ইচ্ছাকৃতভাবে তা বাইপাস করতে পারে।

পরিস্থিতি ৩ — নির্ভরতা বিভ্রান্তি। TechNova-র একটি ব্যক্তিগত লাইব্রেরির নামে সর্বজনীন রেজিস্ট্রিতে একটি লাইব্রেরি পাওয়া গেল। পাইপলাইন ভুল করে সর্বজনীনটি ডাউনলোড করতে যাচ্ছিল। শিক্ষা: ব্যক্তিগত রেজিস্ট্রিকে অগ্রাধিকার দাও, সর্বজনীন রেজিস্ট্রির ওপর নির্ভরতা সীমিত করো, আর লাইব্রেরির নাম ও সংস্করণের ওপর নজর রাখো।

> **REAL WORLD:** বাস্তব সরবরাহ শৃঙ্খল আক্রমণগুলোতে দেখা যায়, আক্রমণকারী সবচেয়ে দুর্বল জায়গাটি বেছে নেয় — প্রায়ই সেটি একটি ছোট, কম নজরদার লাইব্রেরি বা একটি বিল্ড স্ক্রিপ্ট। তাই নিরাপত্তা শুধু নিজের কোডে নয়, নিজের নির্ভরতার রক্ষণাবেক্ষণকারী, বিল্ড সেবা ও রেজিস্ট্রির ওপরও নির্ভর করে। এই কারণেই SBOM, স্বাক্ষর ও provenance — প্রমাণের শৃঙ্খল — এত গুরুত্বপূর্ণ।

১০.৮ নিরাপত্তা ও খরচ বিবেচনা

সরবরাহ শৃঙ্খল নিরাপত্তার খরচ মূলত প্রক্রিয়া ও সংরক্ষণের খরচ: প্রতিটি রিলিজের SBOM ও provenance সংরক্ষণ করতে হয়, স্বাক্ষর যাচাইয়ে সময় লাগে, আর বিল্ডের বিচ্ছিন্নতা বজায় রাখতে পরিচালনার সময় লাগে। তবে এই খরচের বিপরীতে লাভ হলো জরুরি অবস্থায় দ্রুত প্রতিক্রিয়া ও গ্রাহকের আস্থা — বিশেষ করে যেসব গ্রাহক SBOM চায়, তাদের কাছে এটি একটি প্রয়োজনীয়তা হয়ে উঠছে। একটি ভারসাম্য: শুরুতে সবচেয়ে গুরুত্বপূর্ণ সেবাগুলোতে SBOM ও স্বাক্ষর চালু করো, তারপর ধীরে ধীরে সব সেবায় ছড়িয়ে দাও। SLSA-র উচ্চ স্তরে যাওয়ার খরচ বেশি — পুনরুৎপাদনযোগ্য বিল্ড তৈরি করা কঠিন — তাই স্তর বেছে নাও ঝুঁকি অনুযায়ী, সব সেবায় একই স্তর চাপিয়ে দিও না।

১০.৯ সাধারণ ভুল

- SBOM তৈরি করা কিন্তু কখনো না দেখা — জরুরি অবস্থায় খুঁজে না পাওয়া।
- স্বাক্ষর করা কিন্তু যাচাই না করা — admission-এ প্রয়োগ না থাকা।
- লকফাইল কমিট না করা, বা CI-তে `npm install` ব্যবহার করা।
- স্বাক্ষরের ব্যক্তিগত কী git-এ বা ইমেজে রাখা।
- SBOM-কে নিরাপত্তার প্রমাণ ভাবা — SBOM শুধু তালিকা, প্রতিকার নয়।
- বিল্ড স্ক্রিপ্টে ইন্টারনেট থেকে যাচাই ছাড়া স্ক্রিপ্ট চালানো।
- পুরোনো রিলিজের SBOM মুছে ফেলা — পুরোনো রিলিজে কোন দুর্বলতা ছিল, তা আর জানা যায় না।
- সব নির্ভরতাকে সমান বিশ্বাস করা — জনপ্রিয়তা নিরাপত্তার প্রমাণ নয়।

১০.১০ সমস্যা সমাধান

সমস্যা: Cosign যাচাই ব্যর্থ হচ্ছে। প্রথমে দেখো স্বাক্ষরটি কোন পরিচয়ে করা হয়েছে, আর যাচাইয়ে কোন পরিচয় প্রত্যাশা করা হচ্ছে — দুটি না মিললে যাচাই ব্যর্থ হবে। শংসাপত্রের মেয়াদ ও Rekor লগের উপস্থিতিও পরীক্ষা করো।

সমস্যা: SBOM-এ একটি নির্ভরতা অনুপস্থিত। সম্ভবত সেটি বিল্ডের সময় ডাউনলোড হয়েছে, কিন্তু লকফাইলে নেই — অথবা সেটি একটি সিস্টেম প্যাকেজ, যা Syft-এর স্ক্যানে ধরা পড়েনি। SBOM তৈরির উৎস (সোর্স, ইমেজ, নাকি চলমান কন্টেইনার) পরীক্ষা করো — প্রতিটি উৎস ভিন্ন তালিকা দেয়।

সমস্যা: বিল্ড পুনরুৎপাদনযোগ্য নয় — একই সোর্স থেকে ভিন্ন হ্যাশ। সাধারণ কারণ: টাইমস্ট্যাম্প, এলোমেলো ক্রম, বা নির্দিষ্ট নয় এমন নির্ভরতার সংস্করণ। বিল্ডে নির্দিষ্ট সময় (SOURCE_DATE_EPOCH) ব্যবহার করো, নির্ভরতা নির্দিষ্ট করো, আর বিল্ড লগে পার্থক্য খুঁজে বের করো।

> **TROUBLESHOOTING:** সরবরাহ শৃঙ্খলের সমস্যা তদন্ত করার সময় SBOM, provenance ও বিল্ড লগ — তিনটি একসঙ্গে সংরক্ষণ করো। শুধু একটি দেখলে পুরো ছবি পাওয়া যায় না। পুরোনো রিলিজের প্রমাণ মুছে ফেলো না — তদন্তে সেগুলোই দরকার হয়।

সরবরাহ শৃঙ্খলকে সংস্কৃতি হিসেবে। SBOM, স্বাক্ষর ও SLSA — এগুলো সরঞ্জাম দিয়ে শুরু করা যায়, কিন্তু টিকিয়ে রাখতে হয় প্রক্রিয়া দিয়ে। একটি দলে সরবরাহ শৃঙ্খল নিরাপত্তা তখনই কাজ করে যখন প্রতিটি নতুন নির্ভরতা যোগ করার আগে কেউ প্রশ্ন করে: এই লাইব্রেরির রক্ষণাবেক্ষণকারী কে, এটি কতদিন ধরে রক্ষণাবেক্ষণ হচ্ছে, এর বিকল্প কী, আর এটি না থাকলে কী হয়? এই প্রশ্নগুলো একটি ছোট পর্যালোচনা-তালিকায় লিখে রাখা যায়, যা পুল রিকোয়েস্টে নতুন নির্ভরতা যোগ করার সময় পরীক্ষা করা হয়। একইভাবে প্রতিটি নতুন বিল্ড স্ক্রিপ্ট বা CI ধাপ যোগ করার আগে প্রশ্ন করো: এই ধাপটি কী কী করতে পারে, এর কী কী সিক্রেট দরকার, আর এটি আপসিত হলে কী ক্ষতি হবে? এই অভ্যাসগুলো সরঞ্জামের চেয়ে ধীরে গড়ে ওঠে, কিন্তু এগুলোই দীর্ঘমেয়াদে সবচেয়ে বেশি রক্ষা করে।

গ্রাহক ও নিয়ন্ত্রকের দৃষ্টিকোণ। SBOM এখন শুধু নিরাপত্তা দলের দাবি নয় — অনেক গ্রাহক, বিশেষ করে সরকারি ও বড় প্রতিষ্ঠানের গ্রাহক, রিলিজের সঙ্গে SBOM চায়। তাদের প্রশ্ন সহজ: "তোমাদের পণ্যে কী কী আছে, আর সেগুলোতে পরিচিত দুর্বলতা আছে কি না?" SBOM থাকলে এই প্রশ্নের উত্তর দেওয়া যায় প্রমাণসহ; না থাকলে উত্তর দিতে হয় অনুমানে, যা আস্থা নষ্ট করে। একইভাবে স্বাক্ষর ও provenance গ্রাহককে প্রমাণ দেয় যে পণ্যটি দাবি করা উৎস থেকেই এসেছে। এই কারণে সরবরাহ শৃঙ্খল নিরাপত্তাকে শুধু খরচ হিসেবে না দেখে, আস্থার বিনিয়োগ হিসেবে দেখা উচিত — এটি বিক্রয় ও চুক্তির ক্ষেত্রেও কাজে লাগে। তবে একটি সতর্কতা: SBOM-এ সংবেদনশীল তথ্য — যেমন অভ্যন্তরীণ পথ বা ব্যক্তিগত নির্ভরতার নাম — থাকতে পারে, তাই SBOM কাকে দেওয়া হচ্ছে তা নিয়ন্ত্রণ করো।

ভবিষ্যৎ ও ধারাবাহিক উন্নতি। সরবরাহ শৃঙ্খল নিরাপত্তা একটি চলমান ক্ষেত্র: নতুন হুমকি, নতুন মান ও নতুন সরঞ্জাম আসছে। একটি দলের জন্য বাস্তবসম্মত পথ হলো ধাপে ধাপে এগোনো: প্রথমে লকফাইল ও `npm ci` নিশ্চিত করা, তারপর SBOM তৈরি ও সংরক্ষণ করা, তারপর স্বাক্ষর ও যাচাই চালু করা, আর তারপর SLSA স্তর মাপা ও উন্নত করা। প্রতিটি ধাপে মাপো কী কাজ করছে আর কী ভাঙছে। মনে রাখো, লক্ষ্য নিখুঁত শৃঙ্খল নয় — লক্ষ্য এমন একটি শৃঙ্খল, যেখানে প্রতিটি ধাপে প্রমাণ আছে, প্রতিটি প্রমাণ যাচাই করা যায়, আর কোনো ধাপ ভাঙলে তা দ্রুত ধরা পড়ে।

১০.১১ সারাংশ

সফটওয়্যার সরবরাহ শৃঙ্খল নিরাপত্তা মানে প্রমাণের শৃঙ্খল তৈরি করা: SBOM দিয়ে জানা কী আছে, provenance দিয়ে জানা কোথা থেকে এলো ও কীভাবে বানানো হলো, Cosign ও Sigstore দিয়ে স্বাক্ষর করে যাচাইযোগ্য করা, আর SLSA দিয়ে বিল্ডের নিরাপত্তার স্তর মাপা। নির্ভরতার অখণ্ডতা — লকফাইল, `npm ci`, হ্যাশ যাচাই — এই শৃঙ্খলের ভিত্তি।

রিলিজের আগে যাচাই-তালিকা ও ডিপ্লয়ের সময় admission যাচাই সেই শৃঙ্খলকে প্রয়োগ করে। মনে রাখো: SBOM ও স্বাক্ষর নিরাপত্তা নয়, এগুলো নিরাপত্তার প্রমাণ — আসল নিরাপত্তা আসে স্ক্যানিং, পর্যালোচনা ও সীমিত বিশ্বাস থেকে। এই অধ্যায় দিয়ে Part II শেষ হলো: নিরাপদ কোড থেকে শুরু করে OWASP Top 10, স্ক্যানিং, কন্টেইনার নিরাপত্তা ও সরবরাহ শৃঙ্খল — এই পাঁচটি অধ্যায় একসঙ্গে একটি সম্পূর্ণ অ্যাপ্লিকেশন নিরাপত্তা দৃষ্টিভঙ্গি দেয়।

১০.১২ পর্যালোচনা প্রশ্ন

1. সফটওয়্যার সরবরাহ শৃঙ্খল বলতে কী বোঝায়? এর প্রতিটি ধাপে একটি করে হুমকির উদাহরণ দাও।
2. SBOM কী? CycloneDX ও SPDX-এর মধ্যে পার্থক্য কী?
3. Provenance কী এবং SLSA কাঠামোতে এর ভূমিকা কী?
4. Cosign ও Sigstore কীভাবে কাজ করে? কীলেস স্বাক্ষর বলতে কী বোঝায়?
5. Dependency confusion ও typosquatting ব্যাখ্যা করো। প্রতিটির প্রতিকার কী?
6. `npm ci` ও `npm install`-এর মধ্যে পার্থক্য কী? CI-তে কোনটি ব্যবহার করা উচিত এবং কেন?
7. পুনরুৎপাদনযোগ্য বিল্ড বলতে কী বোঝায়? এটি কেন গুরুত্বপূর্ণ?
8. স্বাক্ষর যাচাই admission-এ প্রয়োগ করা কেন দরকার? শুধু নীতি লিখলে কী সমস্যা?
9. SBOM জরুরি CVE প্রতিক্রিয়ায় কীভাবে সাহায্য করে?
10. SLSA-র স্তরগুলো কী মাপে? সব সেবায় একই স্তর চাপিয়ে দেওয়া কেন ঠিক নয়?

১০.১৩ ব্যবহারিক অনুশীলন

1. **SBOM তৈরি করো:** তোমার ল্যাব অ্যাপের ইমেজ থেকে Syft দিয়ে CycloneDX SBOM তৈরি করো। Grype দিয়ে সেই SBOM স্ক্যান করে দুটি আবিষ্কার খুঁজে বের করো ও তাদের ত্রিয়াজ করো।
2. **স্বাক্ষর ও যাচাই:** একটি ল্যাব ইমেজে Cosign দিয়ে কীলেস স্বাক্ষর করো, তারপর যাচাই করো। ইমেজটি সামান্য বদলে আবার যাচাই করে দেখো যাচাই ব্যর্থ হয় কি না।
3. **রিলিজ যাচাই-তালিকা:** একটি রিলিজের জন্য যাচাই-তালিকা লেখো — পরীক্ষা, স্ক্যান, SBOM, স্বাক্ষর ও changelog। একটি ইচ্ছাকৃতভাবে অস্বাক্ষরিত ইমেজ দিয়ে পরীক্ষা করো তালিকাটি তা আটকায় কি না।
4. **সরবরাহ শৃঙ্খল মানচিত্র:** তোমার ল্যাব অ্যাপের সরবরাহ শৃঙ্খলের একটি চিত্র আঁকো — সোর্স থেকে ডিপ্লয় পর্যন্ত প্রতিটি ধাপ, প্রতিটি ধাপের হুমকি, ও প্রতিটি হুমকির প্রতিকার। কোন ধাপে প্রমাণ সবচেয়ে দুর্বল, তা চিহ্নিত করো।

PART III • DEVOPS ENGINEERING MASTERY

DevSecOps Automation & Governance

CI/CD-তে Security, Policy as Code, Vulnerability Management, Cloud Security Posture, Governance ও Compliance।

অধ্যায় ১১ • PART III — DevSecOps Automation & Governance

Chapter 11 — Security in CI/CD

১১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — নিরাপদ পাইপলাইনের স্থাপত্য আঁকতে এবং ব্যাখ্যা করতে; পাইপলাইনের অনুমতি (pipeline permissions) সর্বনিম্ন অধিকারের নীতিতে সাজাতে; গোপন তথ্য (secrets) নিরাপদে পরিচালনা করতে এবং দীর্ঘমেয়াদি চাবির বদলে OIDC ও স্বল্পমেয়াদি শংসাপত্র (short-lived credentials) ব্যবহার করতে; SAST, SCA, DAST ও কনটেইনার স্ক্যানিংয়ের মধ্যে পার্থক্য বলতে এবং প্রতিটিকে পাইপলাইনের সঠিক ধাপে বসাতে; আর্টিফ্যাক্টে স্বাক্ষর (artifact signing) ও SBOM যুক্ত করে সরবরাহ শৃঙ্খলের বিশ্বাসযোগ্যতা তৈরি করতে; ডেপ্লয়মেন্ট অনুমোদন (deployment approvals) ও পরিবেশ সুরক্ষা (environment protection) কনফিগার করতে; এবং নিরাপদ রিলিজ প্রকৌশলের (secure release engineering) একটি সম্পূর্ণ GitHub Actions ওয়ার্কফ্লো লিখতে, চালাতে ও ব্যর্থ হলে নির্ণয় করতে।

আরও গুরুত্বপূর্ণভাবে, আপনি বিচারক্ষমতা অর্জন করবেন: কোন স্ক্যানকে ব্লকিং গেট করা উচিত আর কোনটিকে প্রথমে সতর্কবার্তা হিসেবে রাখা উচিত, কখন পাইপলাইন থামানো নিরাপত্তা আর কখন তা শুধুই দলের কাজ বন্ধ করে দেওয়া, এবং কীভাবে নিরাপত্তাকে ডেভেলপারের গতির শত্রু না বানিয়ে তার অংশ বানানো যায়।

১১.২ পূর্বশর্ত পুনরালোচনা

এই অধ্যায়ে ঢোকার আগে আপনার জানা থাকা উচিত — CI/CD পাইপলাইন কীভাবে কাজ করে: কোড কমিট হলে বিল্ড, টেস্ট, আর্টিফ্যাক্ট তৈরি ও ডেপ্লয় ধাপগুলো কীভাবে চলে। GitHub Actions-এর মৌলিক ধারণা — ওয়ার্কফ্লো, জব, স্টেপ, রানার ও সিক্রেট — আপনার চেনা থাকা দরকার। কনটেইনার ধারণা, অর্থাৎ Dockerfile থেকে ইমেজ তৈরি এবং রেজিস্ট্রিতে পুশ, এবং Node.js অ্যাপ্লিকেশনে npm নির্ভরতা (dependency) কীভাবে কাজ করে, তাও প্রয়োজন।

মনে রাখবেন, TechNova-র MERN অ্যাপ্লিকেশনটি technova.example ডোমেইনে চলে। এর ফ্রন্টএন্ড React, ব্যাকএন্ড Express ও Node.js, ডেটাবেস MongoDB। পাইপলাইনটি কোডকে রিপোজিটরি থেকে নিয়ে প্রোডাকশন সার্ভার 203.0.113.10 পর্যন্ত পৌঁছে দেয়। এই পুরো পথটাই আক্রমণের পৃষ্ঠতল (attack surface)। পাইপলাইনে ঢোকা প্রতিটি উপাদান — সোর্স কোড, নির্ভরতা, বিল্ড টুল, সিক্রেট, রানার, রেজিস্ট্রি — কোনো না কোনোভাবে আপস করা যেতে পারে। তাই নিরাপত্তা পাইপলাইনের শেষে যোগ করা কোনো প্লাগইন নয়; এটি পাইপলাইনের নকশারই অংশ।

একটি ভুল ধারণা শুরুতেই ভেঙে নেওয়া দরকার: স্ক্যানিং মানেই নিরাপত্তা প্রকৌশল নয়। স্ক্যানার একটি যন্ত্র, যা পরিচিত সমস্যা খুঁজে পায়। নিরাপত্তা প্রকৌশল হলো সেই সমস্যাগুলো যাতে তৈরি না হয়, তৈরি হলেও প্রোডাকশনে না পৌঁছায়, আর পৌঁছে গেলেও যাতে ক্ষতি সীমিত থাকে — সেই পুরো ব্যবস্থার নকশা। এই অধ্যায় সেই নকশাই শেখাবে।

১১.৩ প্রথম নীতি থেকে বোঝা: পাইপলাইন কেন আক্রমণের লক্ষ্য

প্রথম নীতি থেকে শুরু করা যাক। সফটওয়্যার সরবরাহ শৃঙ্খল (software supply chain) হলো সেই শৃঙ্খল, যার মাধ্যমে লেখকের কোড ব্যবহারকারীর মেশিনে পৌঁছায়। এই শৃঙ্খলের প্রতিটি সংযোগস্থল একটি বিশ্বাসের সিদ্ধান্ত: আমরা বিশ্বাস করি কমিটটি সত্যিকারের ডেভেলপার করেছেন, নির্ভরতাটি আসল প্যাকেজ, বিল্ড সার্ভারটি আপস হয়নি, আর্টিফ্যাক্টটি বদলানো হয়নি, ডেপ্লয় ধাপটি অনুমোদিত। আক্রমণকারী এই বিশ্বাসের যেকোনো একটি ভেঙে দিতে পারলেই প্রোডাকশনে ম্যালিসিয়াস কোড ঢোকাতে পারে — আর ব্যবহারকারী তাকে TechNova-র অফিসিয়াল রিলিজ ভেবে চালাবে।

সমস্যাটি কেন ঘটে? তিনটি গভীর কারণ আছে। প্রথমত, পাইপলাইন অত্যন্ত ক্ষমতাশালী: এটি প্রোডাকশন সার্ভারে লেখার অধিকার, ক্লাউড অ্যাকাউন্টে ডেপ্লয় করার অধিকার এবং গোপন তথ্য পড়ার অধিকার রাখে। যে সিস্টেম সবচেয়ে বেশি

ক্ষমতা রাখে, সেটিই সবচেয়ে লোভনীয় লক্ষ্য। দ্বিতীয়ত, পাইপলাইন স্বয়ংক্রিয়: মানুষের চোখ ছাড়াই হাজার হাজার বার চলে, তাই একটি ছোট আপস দীর্ঘদিন ধরা না পড়ে বারবার ছড়াতে পারে। তৃতীয়ত, পাইপলাইন অনেক উপাদান একত্র করে: ওপেন সোর্স প্যাকেজ, তৃতীয় পক্ষের অ্যাকশন, বেস ইমেজ, বিল্ড ক্যাশ — প্রতিটি একটি করে প্রবেশপথ।

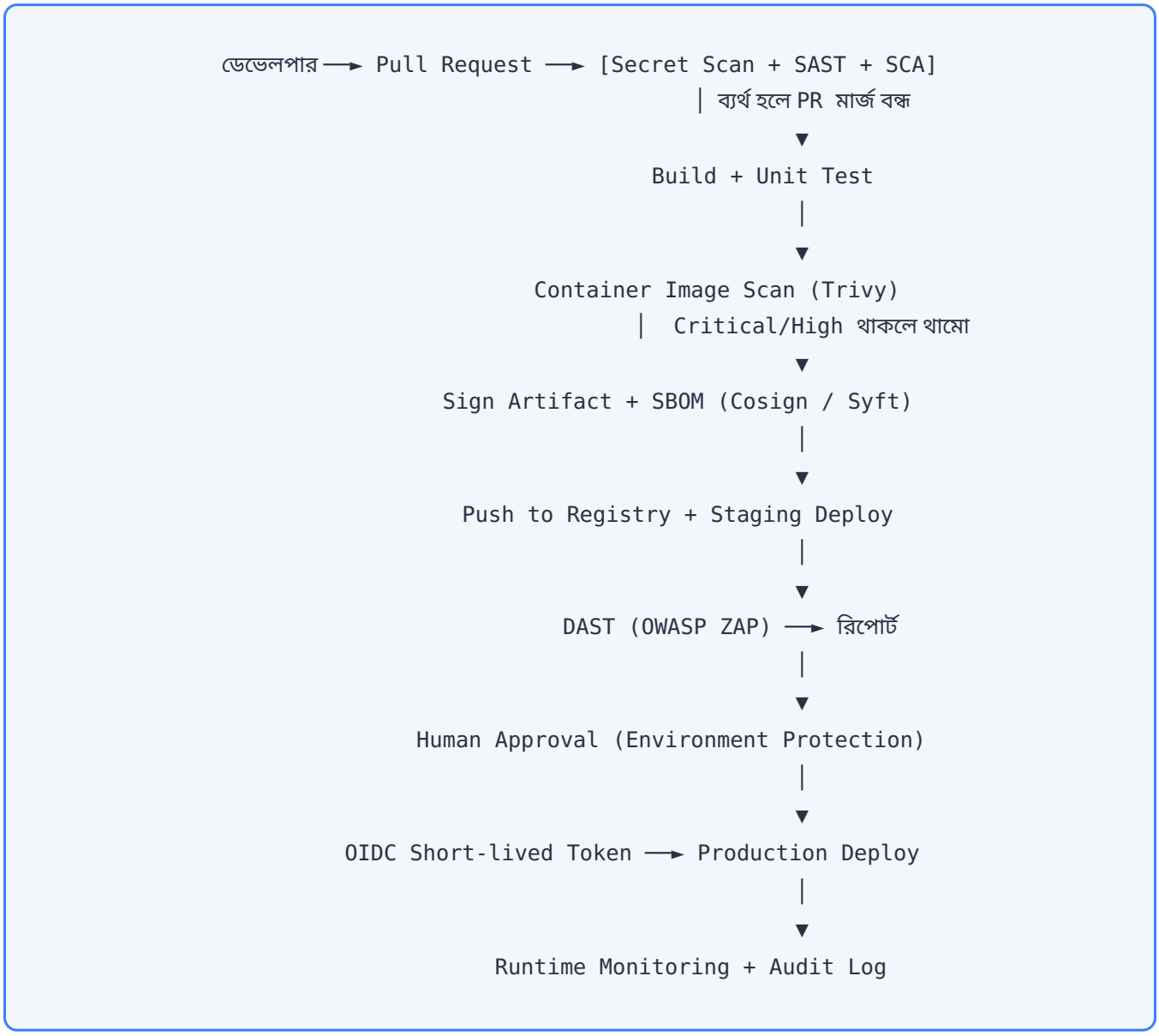
এই সমস্যার সমাধানের মূলনীতি চারটি। এক, সর্বনিম্ন অধিকার (least privilege): পাইপলাইনের প্রতিটি জব শুধু সেইটুকু অধিকার পাবে, যেটুকু তার কাজের জন্য অপরিহার্য, আর এক মুহূর্তের জন্যও বেশি নয়। দুই, যাচাইযোগ্যতা (verifiability): প্রতিটি আর্টিফ্যাক্টের উৎস, উপাদান ও স্বাক্ষর যাচাই করা যাবে — কে বানিয়েছে, কী দিয়ে বানিয়েছে, মাঝপথে বদলেছে কি না। তিন, প্রতিরক্ষার গভীরতা (defense in depth): একটি নিয়ন্ত্রণ ব্যর্থ হলেও পরেরটি ধরবে — স্ক্যান মিস করলে স্বাক্ষর যাচাই, স্বাক্ষর মিস করলে রানটাইম নীতি। চার, ব্যর্থতা নিরাপদ দিকে (fail closed): সন্দেহ হলে পাইপলাইন থামবে, চালিয়ে যাবে না।

> **CONCEPT:** DevOps বনাম DevSecOps — DevOps গতি ও স্বয়ংক্রিয়তার সংস্কৃতি; DevSecOps সেই একই গতিকে নিরাপত্তা-সহ নেওয়ার শৃঙ্খলা। DevSecOps মানে পাইপলাইনের শেষে একজন নিরাপত্তা বিশেষজ্ঞ বসিয়ে দেওয়া নয়; মানে প্রতিটি ধাপে নিরাপত্তা সিদ্ধান্তকে স্বয়ংক্রিয়, পরিমাপযোগ্য ও ডেভেলপারের কর্মপ্রবাহের ভেতরে নিয়ে আসা। স্ক্যানিং একটি কাজ; নিরাপত্তা প্রকৌশল হলো সেই কাজকে এমনভাবে সাজানো যাতে ভুল করাও কঠিন হয়।

সিক্রেটের সমস্যাটি বিশেষভাবে বোঝা দরকার। দীর্ঘমেয়াদি চাবি — যেমন একটি AWS অ্যাক্সেস কী বা একটি স্থায়ী ডেপ্লয় টোকেন — একবার ফাঁস হলে আক্রমণকারী সেটি দিয়ে যতদিন খুশি, যেখান থেকে খুশি ঢুকতে পারে, যতক্ষণ না কেউ খেয়াল করে চাবিটি ঘোরায় (rotate)। আর চাবি ঘোরানো প্রায়ই ভুলে যাওয়া হয়। OIDC (OpenID Connect) এই সমস্যার মূলে আঘাত করে: পাইপলাইন কোনো স্থায়ী চাবি সংরক্ষণ করে না; প্রতিটি রানে পরিচয় প্রদানকারী (identity provider) সাময়িকভাবে যাচাই করে যে এই ওয়ার্কফ্লোটি সত্যিই এই রিপোজিটরি থেকে চলছে, তারপর মাত্র কয়েক মিনিটের জন্য বৈধ একটি স্বল্পমেয়াদি টোকেন দেয়। চাবি চুরি করার মতো কিছুই সংরক্ষিত থাকে না, আর টোকেনের মেয়াদ ফুরিয়ে গেলে সেটি নিজেই মূল্যহীন হয়ে যায়।

১১.৪ নিরাপদ পাইপলাইনের স্থাপত্য

TechNova-র নিরাপদ পাইপলাইনটি দেখতে এরকম: ডেভেলপার কোড পুশ করেন; প্রথমে চলে গোপন তথ্য স্ক্যান (secret scanning) ও SAST; তারপর নির্ভরতা স্ক্যান (SCA); তারপর বিল্ড ও কনটেইনার স্ক্যান; তারপর আর্টিফ্যাক্টে স্বাক্ষর ও SBOM সংযুক্তি; তারপর স্টেজিংয়ে DAST; তারপর মানুষের অনুমোদন; তারপর OIDC দিয়ে প্রোডাকশনে ডেপ্লয়। প্রতিটি ধাপের ফলাফল একটি গেট: গুরুতর (critical) সমস্যা পেলে পাইপলাইন থেমে যায়।



লক্ষ করুন, স্ক্যানগুলো এক জায়গায় জড়ো করা হয়নি; সেগুলো ছড়িয়ে দেওয়া হয়েছে সেই ধাপে, যেখানে সমস্যাটি সবচেয়ে সস্তায় ধরা যায়। সোর্স কোডের সমস্যা PR পর্যায়ে ধরা সবচেয়ে সস্তা — ডেভেলপার তখনো প্রসঙ্গ মনে রাখেন, ঠিক করতে পাঁচ মিনিট লাগে। একই সমস্যা প্রোডাকশনে ধরা পড়লে তার খরচ হয় হটফিক্স, রোলব্যাক, গ্রাহক-বিজ্ঞপ্তি ও সুনাম — অর্থাৎ বহুগুণ। একে বলে শিফট-লেফট (shift-left): নিরাপত্তাকে যত বামে, অর্থাৎ উন্নয়নের যত আগে নেওয়া যায়, তত ভালো। কিন্তু শিফট-লেফটের সীমাও আছে: DAST-এর মতো কিছু পরীক্ষা চলমান অ্যাপ ছাড়া করা যায় না, তাই সেগুলো ডানদিকে, স্টেজিংয়ে থাকে। ভালো পাইপলাইন দুই দিকেই গেট বসায়।

পাইপলাইন অনুমতি নকশার ক্ষেত্রে মূল নিয়ম: ডিফল্টভাবে সবকিছু বন্ধ, তারপর প্রয়োজনে খোলা। GitHub Actions-এ GITHUB_TOKEN-এর ডিফল্ট অনুমতি সীমিত রাখা, প্রতিটি জবের permissions ব্লকে শুধু প্রয়োজনীয় স্কোপ দেওয়া, এবং pull_request ইভেন্ট থেকে আসা ওয়ার্কফ্লোকে কখনোই লেখার (write) অধিকার বা সিক্রেট না দেওয়া — এগুলো অপরিহার্য। কারণ fork থেকে আসা PR-এ আক্রমণকারী নিজের কোড চালাতে পারে; সেই কোড যদি সিক্রেট পড়তে পারে বা প্রোডাকশনে পুশ করতে পারে, তাহলে গেম শেষ। তাই PR ওয়ার্কফ্লো শুধু পড়ে ও পরীক্ষা করে; ডেপ্লয় করে শুধু মূল শাখার (main) সুরক্ষিত ওয়ার্কফ্লো।

> **WARNING:** কখনোই পাইপলাইনে `pull_request_target` ইভেন্টের সঙ্গে PR-এর কোড চেকআউট করে চালাবেন না, যদি না আপনি ঠিক জানেন কী করছেন — এই সংমিশ্রণে আক্রমণকারীর কোড আপনার সিক্রেট ও লেখার অধিকারসহ চলতে পারে। একইভাবে, স্ক্রিপ্ট ইনজেকশন থেকে সাবধান: `${{ github.event.pull_request.title }}`-এর মতো মান সরাসরি shell স্ক্রিপ্টে বসালে আক্রমণকারী শিরোনামে কমান্ড লুকিয়ে চালাতে পারে; এগুলো সবসময় environment variable হিসেবে পাস করুন।

১১.৫ অভ্যন্তরীণ কার্যপ্রণালী: স্ক্যানগুলো আসলে কী করে

SAST (Static Application Security Testing) সোর্স কোড না চালিয়ে বিশ্লেষণ করে। এটি কোডের গঠন (abstract syntax tree) পড়ে পরিচিত বিপজ্জনক প্যাটার্ন খোঁজে: যেমন ব্যবহারকারীর ইনপুট সরাসরি ডেটাবেস কোয়েরিতে যাওয়া (SQL injection-এর পথ), অথবা `eval()`-এ অবিশ্বস্ত স্ট্রিং ঢোকা, অথবা কঠিনভাবে লেখা (hardcoded) পাসওয়ার্ড। এর শক্তি হলো গতি ও প্রাথমিক সনাক্তকরণ; দুর্বলতা হলো মিথ্যা ইতিবাচক (false positive) — কোডটি আসলে নিরাপদ, কিন্তু প্যাটার্ন দেখে সন্দেহ করা — এবং প্রসঙ্গ না বোঝা। তাই SAST-এর ফলাফলকে অন্ধভাবে ব্লক না করে তীব্রতা অনুযায়ী সাজাতে হয়, আর নিয়মিত টিউন করতে হয়।

SCA (Software Composition Analysis) আপনার কোড নয়, আপনার নির্ভরতাগুলো পরীক্ষা করে: package-lock.json-এ তালিকাভুক্ত প্রতিটি npm প্যাকেজের সংস্করণকে পরিচিত দুর্বলতার ডেটাবেসের (যেমন GitHub Advisory Database) সঙ্গে মেলায়। MERN অ্যাপে আপনার নিজের কোড হয়তো মাত্র কয়েক হাজার লাইন, কিন্তু নির্ভরতাগুলো মিলিয়ে লাখ লাখ লাইন — আক্রমণ প্রায়ই এখান দিয়েই আসে। SCA আরও দেখে লাইসেন্স ঝুঁকি ও পুরোনো সংস্করণ। তবে মনে রাখবেন: একটি প্যাকেজে দুর্বলতা থাকা মানেই আপনার অ্যাপ আক্রান্ত নয় — দুর্বল ফাংশনটি আপনার কোড আদৌ ব্যবহার করে কি না, সেটাই নির্ধারণ করে প্রকৃত ঝুঁকি (reachability)। আধুনিক SCA টুল এই পার্থক্য ধরার চেষ্টা করে।

DAST (Dynamic Application Security Testing) চলমান অ্যাপ্লিকেশনকে বাইরে থেকে আক্রমণকারীর মতো পরীক্ষা করে: OWASP ZAP স্টেজিং অ্যাপে HTTP অনুরোধ পাঠিয়ে দেখে সার্ভার কীভাবে সাড়া দেয় — যেমন সিকিউরিটি হেডার অনুপস্থিত কি না, ত্রুটি বার্তায় ভেতরের তথ্য ফাঁস হচ্ছে কি না। DAST-এর শক্তি হলো বাস্তবতা: এটি সত্যিকারের চলমান আচরণ দেখে। দুর্বলতা হলো গতি (ধীর) ও কভারেজ (লগইন-সুরক্ষিত পথগুলোতে পৌঁছাতে প্রমাণীকরণ সাজাতে হয়)। কনটেইনার স্ক্যানিং (যেমন Trivy) ইমেজের স্তরগুলো খুলে OS প্যাকেজ ও অ্যাপ নির্ভরতার দুর্বলতা খোঁজে — কারণ বেস ইমেজের পুরোনো OpenSSL-ও আপনার অ্যাপের ঝুঁকি।

আর্টিফ্যাক্ট স্বাক্ষর (artifact signing) এই প্রশ্নের উত্তর দেয়: এই ইমেজটি কি সত্যিই আমাদের পাইপলাইন বানিয়েছে, মাঝপথে কেউ বদলায়নি তো? Cosign-এর মতো টুল একটি ক্রিপ্টোগ্রাফিক স্বাক্ষর তৈরি করে, যা রেজিস্ট্রিতে ইমেজের সঙ্গে সংরক্ষিত হয়। ডেপ্লয়ের সময় স্বাক্ষর যাচাই করা হয়; স্বাক্ষর মেলেনি মানে ডেপ্লয় বন্ধ। SBOM (Software Bill of Materials) হলো আর্টিফ্যাক্টের উপাদান-তালিকা: কোন প্যাকেজের কোন সংস্করণ ভেতরে আছে। নতুন দুর্বলতা প্রকাশিত হলে SBOM দেখেই বলা যায় আমাদের কোন ইমেজগুলো আক্রান্ত, কোড খুঁজে বেড়াতে হয় না। স্বাক্ষর ছাড়া SBOM আর SBOM ছাড়া স্বাক্ষর — দুটোই অসম্পূর্ণ।

পরীক্ষা	কী দেখে	কখন চলে	শক্তি	সীমা
Secret scanning	হার্ডকোড করা চাবি, টোকেন	কমিট / PR	তাৎক্ষণিক, সস্তা	শুধু পরিচিত প্যাটার্ন
SAST	সোর্স কোডের দুর্বল প্যাটার্ন	PR / বিল্ড	আগে ধরে, সস্তা	False positive বেশি
SCA	নির্ভরতার পরিচিত CVE	PR / বিল্ড	সরবরাহ শৃঙ্খল ধরে	Reachability বোঝে না সবসময়
	ইমেজের OS ও প্যাকেজ দুর্বলতা	বিল্ডের পর	বেস ইমেজ ধরে	রানটাইম আচরণ দেখে না

Container scan				
DAST	চলমান অ্যাপের আচরণ	স্টেজিং	বাস্তব আচরণ	ধীর, কভারেজ সীমিত

১১.৬ ব্যবহারিক কমান্ড ও কনফিগারেশন

নিচের সম্পূর্ণ GitHub Actions ওয়ার্কফ্লোটি TechNova-র MERN অ্যাপের জন্য একটি প্রোডাকশন-উপযোগী নমুনা। এটি অনুশীলনের ল্যাবে ব্যবহারের জন্য; বাস্তব রিপোজিটরিতে বসানোর আগে নিজের পরিবেশ অনুযায়ী নাম, রেজিস্ট্রি ও অনুমতি মিলিয়ে নিন। লক্ষ করুন: প্রতিটি জবের permissions আলাদা করে সীমিত করা হয়েছে, সিক্রেট সরাসরি লগে ছাপা হয় না, এবং গুরুতর সমস্যায় বিল্ড ব্যর্থ হয়।

```
# .github/workflows/secure-release.yml
name: Secure Release Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  contents: read          # ডিফল্ট: শুধু পড়া

jobs:
  secret-and-sast:
    runs-on: ubuntu-24.04
    permissions:
      contents: read
      security-events: write
    steps:
      - uses: actions/checkout@v4
        with:
          persist-credentials: false
      - name: Secret scan (Gitleaks)
        uses: gitleaks/gitleaks-action@v2
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
      - name: SAST (CodeQL)
        uses: github/codeql-action/init@v3
        with:
          languages: javascript-typescript
      - uses: github/codeql-action/analyze@v3

  sca:
    runs-on: ubuntu-24.04
    permissions:
      contents: read
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: npm
      - run: npm ci
      - name: SCA audit (fail on high/critical)
        run: npm audit --audit-level=high
```

```
      - name: Dependency review (PR only)
        if: github.event_name == 'pull_request'
        uses: actions/dependency-review-action@v4

  build-and-scan:
    needs: [secret-and-sast, sca]
    runs-on: ubuntu-24.04
    permissions:
      contents: read
      packages: write
      id-token: write       # OIDC-র জন্য; কোনো স্থায়ী চাবি নয়
    outputs:
      image: ${{ steps.meta.outputs.tags }}
    steps:
      - uses: actions/checkout@v4
      - name: Build image
        run: docker build -t ghcr.io/technova/web:${{ github.sha }} .
      - name: Container scan (Trivy, fail on CRITICAL)
        uses: aquasecurity/trivy-action@0.20.0
        with:
          image-ref: ghcr.io/technova/web:${{ github.sha }}
          format: sarif
          output: trivy-results.sarif
          severity: CRITICAL,HIGH
          exit-code: '1'
      - name: Generate SBOM
        run: |
          curl -sSfL https://raw.githubusercontent.com/anchore/syft/main/install.sh \
            | sh -s -- -b /usr/local/bin
          syft ghcr.io/technova/web:${{ github.sha }} \
            -o spdx-json > sbom.spdx.json
      - name: Sign image (Cosign, keyless via OIDC)
        env:
          COSIGN_YES: 'true'
        run: |
          curl -sSfL https://github.com/sigstore/cosign/releases/download/v2.4.0/cosign-linux-
amd64 \
            -o /usr/local/bin/cosign && chmod +x /usr/local/bin/cosign
          cosign sign --yes ghcr.io/technova/web:${{ github.sha }}
      - name: Push image
        run: docker push ghcr.io/technova/web:${{ github.sha }}

  dast-staging:
    needs: [build-and-scan]
    if: github.event_name == 'push'
    runs-on: ubuntu-24.04
    environment: staging
    permissions:
      contents: read
    steps:
      - name: Deploy to staging 203.0.113.20
        run: echo "Deploying image to staging (lab only)"
      - name: DAST baseline (OWASP ZAP)
        uses: zaproxy/action-baseline@v0.12.0
        with:
          target: https://staging.technova.example

  deploy-production:
    needs: [dast-staging]
```

```
    if: github.event_name == 'push'
    runs-on: ubuntu-24.04
    environment: production   # GitHub-এ এই environment-এ required reviewer বসান
    permissions:
      contents: read
      id-token: write
    steps:
      - name: Verify image signature before deploy
        run: cosign verify ghcr.io/technova/web:${{ github.sha }}
      - name: Deploy with short-lived cloud credentials (OIDC)
        run: echo "Assuming cloud role via OIDC; no stored key used"
```

লোকাল ল্যাবে স্ক্যানগুলো হাতে চালিয়ে দেখা যায়। Trivy দিয়ে ইমেজ স্ক্যান, npm audit দিয়ে নির্ভরতা পরীক্ষা, আর Gitleaks দিয়ে ইতিহাসে ফাঁস হওয়া সিক্রেট খোঁজা — তিনটি কমান্ডই শেখার জন্য মূল্যবান।

```
# Ubuntu 24.04 ল্যাব — TechNova MERN অ্যাপ (~/technova-web)
cd ~/technova-web

# ১. নির্ভরতার দুর্বলতা দেখুন (শুধু রিপোর্ট, ঠিক করবেন পরে)
npm audit
npm audit --audit-level=high

# ২. কনটেইনার ইমেজ স্ক্যান (Trivy ইনস্টল থাকলে)
trivy image --severity HIGH,CRITICAL technova-web:local
trivy image --exit-code 1 --severity CRITICAL technova-web:local

# ৩. গিট ইতিহাসে ফাঁস হওয়া সিক্রেট খুঁজুন
gitleaks detect --source . --verbose

# ৪. SBOM তৈরি করে দেখুন ভেতরে কী কী প্যাকেজ আছে
syft packages technova-web:local -o table | head -n 40

# ৫. Dockerfile-এর নিরাপত্তা ত্রুটি পরীক্ষা (hadolint)
hadolint Dockerfile
```

সিক্রেট পরিচালনার ব্যবহারিক নিয়মগুলো মুখস্থ রাখুন: সিক্রেট কখনো কোডে, কখনো .env ফাইলে কমিট করে নয় — GitHub Secrets বা Vault-এর মতো সিক্রেট ম্যানেজারে রাখুন; প্রতিটি পরিবেশের (staging, production) সিক্রেট আলাদা করুন; সিক্রেটকে লগে ছাপা বন্ধ করুন — GitHub স্বয়ংক্রিয়ভাবে পরিচিত সিক্রেট মাস্ক করে, কিন্তু `echo` দিয়ে ঘুরিয়ে ছাপালে সেই সুরক্ষা কাজ করে না; আর নিয়মিত ঘোরানোর (rotation) সময়সূচি রাখুন। স্থানীয় উন্নয়নে .env.example-এ শুধু চাবির নাম রাখুন, মান নয়।

```
# .env.example — মান নয়, শুধু নাম (কমিট করা নিরাপদ)
MONGO_URI=CHANGE-ME-mongo-uri
JWT_SECRET=CHANGE-ME-jwt-secret
STRIPE_KEY=CHANGE-ME-stripe-key

# ভুল: .env ফাইলটি কখনো কমিট করবেন না
# .gitignore-এ অবশ্যই থাকতে হবে:
# .env
# *.pem
# node_modules/
```

> **REAL WORLD:** TechNova-তে একবার একজন ডেভেলপার ভুলে AWS চাবিসহ .env ফাইল কমিট করে ফেলেছিলেন। গিট ইতিহাস থেকে ফাইল মুছলেও চাবিটি ইতিহাসে থেকে যায় — তাই শুধু ফাইল মুছে কমিট করা যথেষ্ট নয়; চাবিটি সঙ্গে সঙ্গে বাতিল (revoke) করে নতুন চাবি নিতে হয়েছিল, আর Gitleaks-কে PR গেট হিসেবে বসানো হয়েছিল যাতে ভবিষ্যতে এমন কমিট মার্জই না হয়। শিক্ষা: ফাঁস হওয়া সিক্রেটকে কখনো শুধু মুছে ফেলা যায় না — সেটিকে মৃত ঘোষণা করতে হয়।

১১.৭ প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — শুক্রবার সন্ধ্যায় জরুরি হটফিক্স। প্রোডাকশনে একটি গুরুতর বাগ; ডেভেলপার চাইছেন স্ক্যান বাদ দিয়ে সরাসরি ডেপ্লয় করতে। সঠিক সিদ্ধান্ত: স্ক্যান বাদ দেওয়া নয়, বরং জরুরি পথ (emergency path) ব্যবহার করা — হটফিক্স শাখা থেকেও ন্যূনতম গেট (secret scan + SCA + container scan) চলবে, DAST পরে স্টেজিংয়ে চলবে, আর ডেপ্লয়ের জন্য দুজন অনুমোদনকারীর একজন জরুরি অনুমোদন দেবেন, যার রেকর্ড অডিট লগে থাকবে। গতি আর নিরাপত্তার মধ্যে বেছে নেওয়ার বদলে, জরুরি অবস্থার জন্য আগেই একটি দ্রুত কিন্তু নিরাপদ পথ তৈরি রাখাই প্রোডাকশন মান।

পরিস্থিতি ২ — নতুন নির্ভরতায় Critical CVE। npm audit-এ দেখা গেল একটি জনপ্রিয় প্যাকেজে গুরুতর দুর্বলতা, কিন্তু প্যাচ করা সংস্করণে আপগ্রেড করলে অ্যাপের একটি ফিচার ভেঙে যায়। এখানে বিচার দরকার: প্রথমে দেখুন দুর্বল ফাংশনটি আপনার কোড আদৌ ব্যবহার করে কি না (reachability)। যদি ব্যবহার না করে, তাহলে ঝুঁকি কম — সাময়িক ব্যতিক্রম (exception) নথিভুক্ত করে আপগ্রেডের পরিকল্পনা করুন। যদি ব্যবহার করে, তাহলে ফিচারটি সাময়িকভাবে বন্ধ করা বা বিকল্প প্যাকেজে যাওয়াও বিবেচনা করুন। কোনো অবস্থাতেই গেটটি নীরবে বন্ধ করে ডেপ্লয় করা যাবে না — ব্যতিক্রমের মালিক, মেয়াদ ও পর্যালোচনার তারিখ লিখিত থাকতে হবে।

পরিস্থিতি ৩ — তৃতীয় পক্ষের অ্যাকশন আপস। পাইপলাইনে ব্যবহৃত একটি জনপ্রিয় GitHub Action-এর নতুন সংস্করণে ম্যালিসিয়াস কোড ঢুকে গেছে — এমন ঘটনা বাস্তবে ঘটেছে। প্রতিরোধ: অ্যাকশনগুলোকে ট্যাগের বদলে কমিট SHA দিয়ে পিন করুন (যেমন `actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683`), যাতে নতুন সংস্করণ নিজে থেকে ঢুকতে না পারে; অপরিচিত অ্যাকশন ব্যবহারের আগে তার সোর্স ও রক্ষণাবেক্ষণ দেখুন; আর নিজের সংস্থার অনুমোদিত অ্যাকশনের তালিকা রাখুন।

পরিস্থিতি ৪ — প্রোডাকশন ডেপ্লয়ে ভুল ইমেজ। একজন অপারেটর ভুল করে পুরোনো, স্বাক্ষরবিহীন ইমেজ ডেপ্লয় করতে গেলেন। স্বাক্ষর যাচাই ধাপ থাকলে ডেপ্লয় সেখানেই থেমে যাবে — এই ঘটনাই দেখায় কেন স্বাক্ষর যাচাই ডেপ্লয় ধাপের ভেতরে, পাইপলাইনের শেষ প্রান্তে থাকা দরকার, শুধু বিল্ড ধাপে নয়।

১১.৮ নিরাপত্তা ও খরচ বিবেচনা

নিরাপত্তার খরচ তিন রকম: সরাসরি খরচ (স্ক্যানার, সিক্রেট ম্যানেজার, স্বাক্ষর পরিকাঠামো), সময়ের খরচ (পাইপলাইন ধীর হয়, ডেভেলপার অপেক্ষা করেন), আর জটিলতার খরচ (আরও একটি সিস্টেম রক্ষণাবেক্ষণ করতে হয়)। কিন্তু অনিরাপত্তার খরচ অনেক বড়: ডেটা লঙ্ঘন, সেবা বন্ধ, আইনি জরিমানা, সুনামহানি। বুদ্ধিমানের কাজ হলো খরচকে সেই জায়গায় দেওয়া, যেখানে তা সবচেয়ে বেশি ঝুঁকি কমায় — যেমন PR পর্যায়ের দ্রুত স্ক্যানে, দীর্ঘমেয়াদি চাবি তুলে OIDC-তে, আর স্বাক্ষর যাচাইয়ে।

খরচ কমানোর ব্যবহারিক উপায়: দ্রুত স্ক্যান আগে, ধীর স্ক্যান পরে — PR-এ শুধু পরিবর্তিত কোডের SAST, সম্পূর্ণ স্ক্যান রাতের নির্ধারিত রানে; ক্যাশিং ব্যবহার করুন, কিন্তু ক্যাশের বিষাক্তকরণ (cache poisoning) থেকে সাবধান — ক্যাশের চাবিতে (key) লকফাইল হ্যাশ অন্তর্ভুক্ত করুন; আর স্ক্যানের ফলাফলকে তীব্রতা অনুযায়ী সাজিয়ে শুধু কার্যকর (actionable) সমস্যায় ডেভেলপারের মনোযোগ চান। শত শত নিম্ন-তীব্রতার সতর্কবার্তা দিয়ে ডেভেলপারকে ডুবিয়ে দিলে

তারা সব সতর্কবার্তাই উপেক্ষা করতে শেখে — একে বলে সতর্কবার্তা-ক্লান্তি (alert fatigue), যা নিরাপত্তার সবচেয়ে নীরব শত্রু।

নিরাপত্তা দিক থেকে আরও কিছু বিষয়: রানারদের (runner) আলাদা রাখুন — প্রোডাকশন ডেপ্লয় করা রানার যেন PR-এর অবিশ্বস্ত কোড না চালায়; বিল্ড আর্টিফ্যাক্টে সিক্রেট যাতে না ঢোকে, সেজন্য মাল্টি-স্টেজ Dockerfile-এ সিক্রেট শুধু বিল্ড ধাপে ব্যবহার করে চূড়ান্ত ইমেজে না রাখুন; আর সমস্ত পাইপলাইন লগ অপরিবর্তনীয়ভাবে (immutably) সংরক্ষণ করুন, যাতে ঘটনার পর তদন্ত করা যায়।

১১.৯ সাধারণ ভুল

- **সব স্ক্যানকে একদিনে ব্লকিং করা:** প্রথম দিনেই সমস্ত গেট কঠোর করলে দল কাজ করতে পারবে না; শুরুতে সতর্কবার্তা হিসেবে চালান, ত্রুটির হার কমান, তারপর ধাপে ধাপে কঠোর করুন।
- **সিক্রেট কোডে বা লগে রাখা:** হার্ডকোড করা চাবি, লগে টোকেন ছাপা, স্ক্রিনশটে সিক্রেট শেয়ার করা — সবই একই ভুলের রূপ।
- **দীর্ঘমেয়াদি চাবি পাইপলাইনে সংরক্ষণ:** ঘোরানো হয় না এমন চাবি সময়ের বোমা; OIDC-তে যান।
- **অ্যাকশন ট্যাগে পিন না করা:** `@v4` মানে যেকোনো নতুন v4; কমিট SHA-তে পিন করুন।
- **PR ওয়ার্কফ্লোকে লেখার অধিকার দেওয়া:** fork থেকে আসা কোডকে সিক্রেট ও লেখার অধিকার দেওয়া মারাত্মক।
- **শুধু স্ক্যান করে নিশ্চিন্ত হওয়া:** স্ক্যান না ধরা সমস্যা, ভুল কনফিগারেশন ও রানটাইম আক্রমণ — এগুলোর জন্য স্বাক্ষর, নীতি ও পর্যবেক্ষণও দরকার।
- **SBOM ছাড়া স্বাক্ষর, বা স্বাক্ষর ছাড়া SBOM:** একটি ছাড়া অন্যটি অসম্পূর্ণ; দুটোই একসঙ্গে করুন।
- **ব্যতিক্রমের কোনো মেয়াদ না রাখা:** সাময়িক ছাড় চিরস্থায়ী হয়ে যায়; প্রতিটি ছাড়ের মালিক ও মেয়াদ লিখুন।

১১.১০ সমস্যা নির্ণয়

সমস্যা: Trivy স্ক্যানে Critical পেয়ে পাইপলাইন থেমে গেছে, কিন্তু অ্যাপ ঠিক আছে মনে হচ্ছে। নির্ণয়: প্রথমে দেখুন দুর্বলতাটি কোন প্যাকেজে — বেস ইমেজের OS প্যাকেজে হলে বেস ইমেজ আপডেট করুন (`node:20-bookworm` থেকে নতুন প্যাচ সংস্করণে যান); অ্যাপ নির্ভরতায় হলে `npm audit fix` বা নির্দিষ্ট প্যাকেজ আপগ্রেড করুন। যদি দুর্বল ফাংশনটি ব্যবহৃত না হয়, তাহলে নথিভুক্ত ব্যতিক্রম নিন, গেট বন্ধ করবেন না।

সমস্যা: OIDC দিয়ে ক্লাউড ভূমিকা নেওয়া যাচ্ছে না — "not authorized" ত্রুটি। নির্ণয়: ক্লাউডের ট্রাস্ট পলিসিতে রিপোজিটরি, শাখা ও পরিবেশের শর্ত মিলছে কি না দেখুন; টোকেনের audience ও subject দাবিগুলো যাচাই করুন; আর দেখুন টোকেনের মেয়াদ পেরিয়ে গেছে কি না — দীর্ঘ চলমান জবে টোকেন মেয়াদোত্তীর্ণ হতে পারে, তখন জব ভেঙে ছোট করুন।

সমস্যা: Cosign যাচাই ব্যর্থ — স্বাক্ষর মিলছে না। নির্ণয়: ইমেজ ট্যাগ ও ডাইজেস্ট মিলছে কি না দেখুন — ট্যাগ বদলে গেলে স্বাক্ষর মিলবে না; স্বাক্ষরকারী পরিচয় (issuer) ও যাচাই নীতি মিলছে কি না দেখুন; আর নিশ্চিত হোন ইমেজটি পুনর্নির্মাণের পর আবার স্বাক্ষর করা হয়েছে কি না।

সমস্যা: DAST-এ শত শত সতর্কবার্তা, বেশিরভাগই অপ্রাসঙ্গিক। নির্ণয়: ZAP-এর প্রসঙ্গ (context) ও প্রমাণীকরণ সঠিকভাবে সাজানো হয়নি — লগইন করা সেশন ছাড়া স্ক্যান শুধু পাবলিক পাতা দেখে; নিয়মগুলো টিউন করুন, অপ্রাসঙ্গিক নিয়ম নিষ্ক্রিয় করুন, আর ফলাফলকে তীব্রতা অনুযায়ী সাজান।

TROUBLESHOOTING: পাইপলাইন ব্যর্থ হলে প্রথমে লগের প্রথম ব্যর্থ ধাপটি দেখুন, শেষেরটি নয় — পরের ধাপগুলোর ব্যর্থতা প্রায়ই প্রথম ব্যর্থতারই প্রতিধ্বনি। তারপর জিজ্ঞাসা করুন: এটি কি নতুন কোডের সমস্যা, নতুন নির্ভরতার সমস্যা, নাকি পরিবেশের সমস্যা (যেমন রেজিস্ট্রি বা পরিচয় প্রদানকারী ডাউন)? তিনটির প্রতিকার আলাদা।

১১.১১ সারসংক্ষেপ

নিরাপদ CI/CD মানে পাইপলাইনের প্রতিটি ধাপে বিশ্বাসকে যাচাইয়ের আওতায় আনা: সর্বনিম্ন অধিকার, স্বল্পমেয়াদি OIDC শংসাপত্র, সঠিক জায়গায় সঠিক স্ক্যান (secret, SAST, SCA, কনটেইনার, DAST), আর্টিফ্যাক্ট স্বাক্ষর ও SBOM, এবং মানুষের অনুমোদনসহ ডেপ্লয় গেট। স্ক্যানিং নিরাপত্তা প্রকৌশলের একটি অংশ মাত্র; আসল কাজ হলো এমন একটি ব্যবস্থা তৈরি করা, যেখানে ভুল করাও কঠিন, ভুল ধরা পড়ে দ্রুত, আর ক্ষতি সীমিত থাকে। TechNova-র পাইপলাইন এই নীতিগুলোরই একটি জীবন্ত উদাহরণ — আর আপনার কাজ হলো এগুলোকে নিজের ল্যাবে চালিয়ে, ভেঙে ও ঠিক করে আয়ত্ত করা।

১১.১২ পর্যালোচনা প্রশ্ন

1. সফটওয়্যার সরবরাহ শৃঙ্খল বলতে কী বোঝায়, এবং পাইপলাইন কেন আক্রমণকারীর কাছে লোভনীয় লক্ষ্য?
2. সর্বনিম্ন অধিকার, যাচাইযোগ্যতা, প্রতিরক্ষার গভীরতা ও fail-closed — এই চার নীতির প্রতিটি একটি উদাহরণসহ ব্যাখ্যা করুন।
3. SAST, SCA, DAST ও কনটেইনার স্ক্যানিংয়ের মধ্যে পার্থক্য কী, এবং প্রতিটি পাইপলাইনের কোন ধাপে বসে কেন?
4. দীর্ঘমেয়াদি চাবির বদলে OIDC ও স্বল্পমেয়াদি শংসাপত্র কেন নিরাপদ? চাবি ফাঁস হলে কী করতে হয়?
5. আর্টিফ্যাক্ট স্বাক্ষর ও SBOM একে অপরের পরিপূরক কীভাবে? একটি ছাড়া অন্যটি কেন অসম্পূর্ণ?
6. Pull request ওয়ার্কফ্লোকে লেখার অধিকার বা সিক্রেট দেওয়া কেন বিপজ্জনক? `pull_request_target`-এর ঝুঁকি কী?
7. GitHub Actions-এ অ্যাকশনকে কমিট SHA দিয়ে পিন করা কেন গুরুত্বপূর্ণ?
8. সতর্কবার্তা-ক্লান্তি (alert fatigue) কী, এবং এটি কীভাবে নিরাপত্তাকে দুর্বল করে?
9. জরুরি হটফিক্সের সময় স্ক্যান বাদ না দিয়ে কী করা উচিত? জরুরি পথের নকশা কেমন হবে?
10. শিফট-লেফট বলতে কী বোঝায়, এবং এর সীমা কোথায় — কোন পরীক্ষা ডানদিকে থাকতেই হয়?

১১.১৩ ব্যবহারিক অনুশীলন

1. **ল্যাব পাইপলাইন তৈরি:** নিজের ল্যাবে একটি নমুনা MERN রিপোজিটরি বানিয়ে উপরের ওয়ার্কফ্লোটি বসান। ইচ্ছাকৃতভাবে একটি হার্ডকোড করা সিক্রেট ও একটি পুরোনো দুর্বল নির্ভরতা যোগ করে দেখুন পাইপলাইন কোথায় থামে, তারপর সেগুলো ঠিক করে পাইপলাইন সবুজ করুন।
2. **OIDC অনুকরণ:** একটি স্থায়ী চাবি দিয়ে ডেপ্লয় ধাপ লিখুন, তারপর সেটিকে OIDC-ভিত্তিক স্বল্পমেয়াদি শংসাপত্রে বদলান। দুটি পদ্ধতির ট্রাস্ট পলিসি ও মেয়াদের পার্থক্য লিখে রাখুন।
3. **স্বাক্ষর ও SBOM যাচাই:** একটি ইমেজ বানিয়ে Syft দিয়ে SBOM তৈরি করুন, Cosign দিয়ে স্বাক্ষর করুন, তারপর ইমেজটি সামান্য বদলে (ট্যাগ বদলে) যাচাই চালিয়ে দেখুন কীভাবে ব্যর্থ হয়।
4. **ব্যতিক্রম প্রক্রিয়া:** একটি Critical দুর্বলতার জন্য এক পৃষ্ঠার ব্যতিক্রম নথি লিখুন — ঝুঁকি, reachability বিশ্লেষণ, ক্ষতিপূরণমূলক নিয়ন্ত্রণ, মালিক ও মেয়াদসহ।

অধ্যায় ১২ • PART III — DevSecOps Automation & Governance

Chapter 12 — Policy as Code

১২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — Policy as Code বলতে কী বোঝায় এবং নথিভুক্ত নীতির বদলে কোড হিসেবে নীতি লেখা কেন শ্রেষ্ঠ, তা ব্যাখ্যা করতে; Open Policy Agent (OPA) ও Rego ভাষার মৌলিক ধারণা — নিয়ম, প্রশ্ন (query), ইনপুট, ডেটা — ব্যবহার করে নীতি লিখতে ও পরীক্ষা করতে; Kyverno দিয়ে Kubernetes-এ নীতি প্রয়োগ করতে, যার মধ্যে validate, mutate ও generate নীতির পার্থক্য বলা যায়; Terraform পরিকল্পনা (plan) যাচাই করে বিপজ্জনক ক্লাউড কনফিগারেশন ডেপ্লয়ের আগেই আটকাতে; Kubernetes admission নীতির কাজ বোঝাতে; নীতির পরীক্ষা (policy testing) লিখে ভুল নীতি প্রোডাকশনে যাওয়া রোধ করতে; এবং সম্মতি (compliance) স্বয়ংক্রিয়করণের একটি বাস্তবসম্মত কাঠামো তৈরি করতে।

১২.২ পূর্বশর্ত পুনরালোচনা

এই অধ্যায়ে Kubernetes-এর মৌলিক ধারণা — Pod, Deployment, Namespace, ServiceAccount — আপনার চেনা থাকা দরকার, কারণ Kyverno-র নীতিগুলো এই সম্পদগুলোর ওপরেই প্রয়োগ হয়। Terraform-এর মৌলিক ধারণা — resource, plan, apply — এবং JSON/YAML কাঠামো পড়তে পারা প্রয়োজন, কারণ Rego নীতির ইনপুট সাধারণত JSON হয়। আগের অধ্যায়ের CI/CD পাইপলাইন ও OPA নয় এমন নীতির ধারণা — যেমন কোড রিভিউয়ে মানুষের চেকলিস্ট — মনে রাখলে Policy as Code-এর পার্থক্যটি স্পষ্ট হবে।

TechNova-র প্রসঙ্গে ভাবুন: দলটির নিয়ম হলো — কোনো Pod root ব্যবহারকারী হিসেবে চলবে না, কোনো ইমেজ `latest` ট্যাগ ব্যবহার করবে না, কোনো S3 বাকেট (বা সমতুল্য স্টোরেজ) পাবলিক হবে না, আর প্রতিটি সম্পদে `team` ও `environment` লেবেল থাকতে হবে। একসময় এই নিয়মগুলো ছিল একটি Wiki পাতায়; কেউ পড়ত, কেউ পড়ত না; ভুল ধরা পড়ত প্রোডাকশনে ঘটনার পর। Policy as Code এই নিয়মগুলোকে পরীক্ষাযোগ্য, সংস্করণ-নিয়ন্ত্রিত কোডে রূপান্তর করে, যা প্রতিটি পরিবর্তনে স্বয়ংক্রিয়ভাবে প্রয়োগ হয়।

১২.৩ প্রথম নীতি থেকে বোঝা: নীতি কেন কোড হওয়া উচিত

প্রথম নীতি থেকে শুরু করা যাক। নীতি (policy) হলো একটি সিদ্ধান্তের নিয়ম: এই পরিস্থিতিতে এই কাজটি অনুমোদিত, ওই কাজটি নয়। সংস্থায় নীতি সবসময় ছিল — কিন্তু যতদিন নীতি মানুষের মাথায় বা PDF নথিতে থাকে, ততদিন তার প্রয়োগ নির্ভর করে মানুষের স্মৃতি, মনোযোগ ও সদিচ্ছার ওপর। মানুষ ভুল করে, ব্যস্ত থাকে, চাপে থাকে — আর নীতি ভাঙা হয় নীরবে। সমস্যাটি তাই প্রয়োগের (enforcement) সমস্যা, নীতির অভাবের নয়।

Policy as Code এই সমস্যার মূলে আঘাত করে: নীতিকে এমন একটি ভাষায় লেখা হয়, যা মানুষ পড়তে পারে আবার মেশিন চালাতে পারে। তখন নীতি পরীক্ষা করা যায় (এই ইনপুট দিলে নীতি কি এই সিদ্ধান্ত দেয়?), সংস্করণ নিয়ন্ত্রণে রাখা যায় (কে কখন নীতি বদলাল?), পর্যালোচনা করা যায় (PR-এ নীতির পরিবর্তনও রিভিউ হয়), এবং স্বয়ংক্রিয়ভাবে প্রয়োগ করা যায় (নীতি ভাঙা পরিবর্তন ডেপ্লয়ের আগেই আটকে যায়)। নীতি তখন আর পরামর্শ থাকে না; তা হয়ে ওঠে সিস্টেমেরই অংশ।

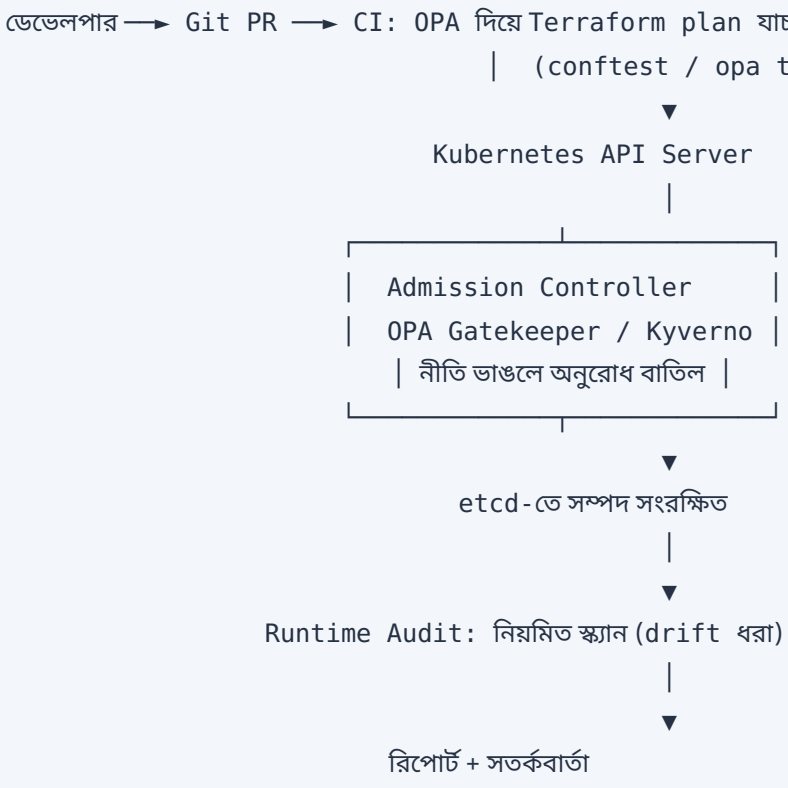
এখানে একটি গুরুত্বপূর্ণ পার্থক্য বুঝতে হবে: নীতি (policy) বনাম নিয়ন্ত্রণ (control) বনাম প্রক্রিয়া (process)। নীতি বলে কী হওয়া উচিত ("কোনো Pod root হিসেবে চলবে না")। নিয়ন্ত্রণ হলো সেই নীতি প্রয়োগের যান্ত্রিক ব্যবস্থা (admission controller যা root Pod আটকায়)। প্রক্রিয়া হলো মানুষের কর্মপদ্ধতি (নীতি ভাঙলে ব্যতিক্রম চাইতে হয়)। Policy as Code নীতি ও নিয়ন্ত্রণকে একই কোডে বাঁধে, আর প্রক্রিয়াকে তার চারপাশে সাজায়।

CONCEPT: OPA (Open Policy Agent) হলো একটি সাধারণ উদ্দেশ্যের নীতি ইঞ্জিন — এটি যেকোনো JSON ইনপুট নিয়ে নীতির বিরুদ্ধে সিদ্ধান্ত দিতে পারে: Kubernetes অনুরোধ, Terraform পরিকল্পনা, API অনুরোধ, CI/CD অনুমোদন। Rego হলো OPA-র ঘোষণামূলক (declarative) ভাষা: আপনি বলেন কী সত্য হতে হবে, কীভাবে গণনা করতে হবে তা নয়। Kyverno হলো Kubernetes-এর জন্য বিশেষায়িত নীতি ইঞ্জিন — এটি Kubernetes সম্পদেই নীতি লেখে (YAML-এ), তাই Kubernetes দলের কাছে সহজ, কিন্তু OPA-র মতো সাধারণ উদ্দেশ্যে ব্যবহার করা যায় না। দুটির মধ্যে বেছে নেওয়া আসলে প্রশ্ন: আপনার নীতি কি শুধু Kubernetes-এর জন্য, নাকি সব জায়গায় একই নীতি চালাতে চান?

নীতি কেন ব্যর্থ হয়, সেটাও প্রথম নীতি থেকে বোঝা দরকার। নীতি ব্যর্থ হয় যখন তা অস্পষ্ট হয় ("নিরাপদ কনফিগারেশন ব্যবহার করুন" — নিরাপদ মানে কী?), যখন তা পরীক্ষা করা যায় না, যখন তার ব্যতিক্রমের পথ নেই (তখন মানুষ নীতি এড়িয়ে যায়), অথবা যখন তা এত কঠোর হয় যে কাজ বন্ধ হয়ে যায় (তখন দল নীতিকে শত্রু ভাবে)। ভালো নীতি সুনির্দিষ্ট, পরীক্ষাযোগ্য, ব্যতিক্রম-সহনশীল ও পর্যায়ক্রমে কঠোর হয়: প্রথমে audit মোডে (শুধু সতর্কবার্তা), তারপর enforce মোডে (আটকে দেওয়া)।

১২.৪ স্থাপত্য: নীতি কোথায় বসে

নীতি তিনটি জায়গায় বসতে পারে: ডেপ্লয়ের আগে (CI/CD-তে Terraform plan যাচাই, Dockerfile যাচাই), ভর্তির সময় (admission time — Kubernetes API সার্ভারে অনুরোধ আসার মুহূর্তে যাচাই), এবং চলমান অবস্থায় (runtime — ইতিমধ্যে চলমান সম্পদ নিয়মিত নিরীক্ষা)। তিনটির কাজ আলাদা: প্রথমটি ডেভেলপারকে দ্রুত প্রতিক্রিয়া দেয়; দ্বিতীয়টি শেষ প্রতিরক্ষা রেখা হিসেবে ভুল সম্পদ ক্লাস্টারে ঢুকতে দেয় না; তৃতীয়টি পুরোনো ভুল ও পরিবর্তন (drift) ধরে। ভালো ব্যবস্থায় তিনটিই থাকে, একই নীতি তিন জায়গায় পুনরায় ব্যবহার করে।



OPA-র স্থাপত্য বোঝা সহজ: OPA একটি সিদ্ধান্ত নেয় — ইনপুট (যেমন একটি Pod-এর বিবরণ) ও ডেটা (যেমন অনুমোদিত ইমেজের তালিকা) নিয়ে নীতির বিরুদ্ধে মূল্যায়ন করে উত্তর দেয়: অনুমোদিত নাকি অস্বীকৃত, এবং কেন। OPA নিজে কিছু আটকায় না; আটকায় সেই সিস্টেম, যে OPA-কে জিজ্ঞাসা করে — Kubernetes-এ সেটি Gatekeeper, Terraform-এ সেটি CI ধাপ। এই পৃথকীকরণ (decoupling) শক্তিশালী: একই নীতি ইঞ্জিন বহু সিস্টেমে ব্যবহার করা যায়। Kyverno ভিন্নভাবে কাজ করে: এটি Kubernetes-এর ভেতরে একটি admission webhook হিসেবে বসে, নীতিকে Kubernetes-এর নিজস্ব সম্পদ (Custom Resource) হিসেবে সংরক্ষণ করে, এবং তিন ধরনের কাজ করে — validate (যাচাই ও আটকানো), mutate (অনুরোধ বদলে মানানসই করা, যেমন লেবেল যোগ করা), ও generate (অনুরোধের ভিত্তিতে নতুন সম্পদ তৈরি করা, যেমন প্রতিটি Namespace-এ NetworkPolicy তৈরি করা)।

১২.৫ অভ্যন্তরীণ কার্যপ্রণালী: Rego কীভাবে ভাবে

Rego শেখার সবচেয়ে বড় বাধা হলো এটি প্রচলিত প্রোগ্রামিং ভাষার মতো নয়। Rego ঘোষণামূলক: আপনি নিয়ম লেখেন, যা সত্য বা মিথ্যা হয়। একটি নিয়মের মাথা (head) বলে কী উৎপন্ন হবে, আর দেহ (body) বলে কখন তা সত্য। যেমন `deny[msg] { ... }` মানে: দেহের শর্তগুলো সত্য হলে `deny` সেটে `msg` বার্তাটি যোগ হবে। OPA ইনপুটের বিরুদ্ধে সব নিয়ম মূল্যায়ন করে; `deny` সেট খালি থাকলে অনুমোদন, না থাকলে অস্বীকৃতি।

Rego-তে কিছু মৌলিক ধারণা: `input` হলো মূল্যায়ন করা হচ্ছে এমন বস্তু (যেমন Pod-এর JSON); `data` হলো বাইরের তথ্য (যেমন অনুমোদিত রেজিস্ট্রির তালিকা); `some` দিয়ে চলকের অস্তিত্ব ঘোষণা করা হয়; আর `contains` বা `if` দিয়ে সেট ও শর্ত লেখা হয়। Rego-তে লুপ নেই; বদলে সেট ও বোধগম্যতা (comprehension) দিয়ে সংগ্রহের ওপর নিয়ম প্রয়োগ করা হয়। প্রথমে এটি অপরিচিত লাগে, কিন্তু অভ্যাস হলে দেখা যায়: নীতি আসলে সেট-ভিত্তিক যুক্তিই — "এই বৈশিষ্ট্যযুক্ত সব Pod-ই নিষিদ্ধ"।

নীতির মূল্যায়ন কীভাবে হয়, ধাপে ধাপে: প্রথমে ইনপুট JSON হিসেবে আসে; তারপর OPA নীতির প্রতিটি নিয়মের দেহ পরীক্ষা করে, চলকগুলোকে ইনপুটের মানের সঙ্গে বাঁধে; যে নিয়মগুলোর দেহ সত্য, তাদের মাথা ফলাফল সেটে যোগ হয়; শেষে জিজ্ঞাসাকারী (querier) ফলাফল দেখে সিদ্ধান্ত নেয়। গুরুত্বপূর্ণ: Rego-তে কোনো নিয়মের দেহে অনির্ধারিত (undefined) কিছু থাকলে সেটি মিথ্যা হিসেবে ধরা হয় — অর্থাৎ নীতি লেখার সময় অনুপস্থিত ক্ষেত্র (field) নিয়ে সাবধান হতে হয়, নইলে নীতি নীরবে কিছু ধরবে না।

ধারণা	OPA / Rego	Kyverno
নীতির ভাষা	Rego (সাধারণ উদ্দেশ্য)	Kubernetes YAML (JMESPath মিল)
প্রয়োগের জায়গা	Kubernetes, Terraform, API, CI/CD	শুধু Kubernetes (admission)
কাজের ধরন	Validate (সিদ্ধান্ত দেয়)	Validate, Mutate, Generate
শেখার বক্ররেখা	খাড়া (নতুন ভাষা)	মৃদু (পরিচিত YAML)
পরীক্ষা	opa test (ইউনিট টেস্ট)	Kyverno CLI test

১২.৬ ব্যবহারিক কমান্ড ও কনফিগারেশন

প্রথমে একটি কাজের Rego নীতি দেখুন: কোনো Pod root ব্যবহারকারী হিসেবে চলতে পারবে না, কোনো কনটেইনার `latest` ট্যাগের ইমেজ ব্যবহার করতে পারবে না, আর প্রতিটি Pod-এ `team` লেবেল থাকতে হবে। এই তিনটি নিয়মই TechNova-র বাস্তব নীতি।

```
# policies/kubernetes.rego — OPA / Rego নীতি (TechNova)
package kubernetes.admission

import rego.v1

# নিয়ম ১: root ব্যবহারকারী নিষিদ্ধ
deny contains msg if {
        some container in input.request.object.spec.containers
        container.securityContext.runAsUser == 0
        msg := sprintf("Container %v must not run as root (runAsUser 0)", [container.name])
}

# নিয়ম ২: latest ট্যাগ নিষিদ্ধ (ইমেজের সংস্করণ নির্দিষ্ট হতে হবে)
deny contains msg if {
        some container in input.request.object.spec.containers
        endswith(container.image, ":latest")
        msg := sprintf("Container %v must not use :latest tag", [container.name])
}

# নিয়ম ৩: team লেবেল বাধ্যতামূলক
deny contains msg if {
        not input.request.object.metadata.labels.team
        msg := "Every Pod must have a 'team' label"
}

# নিয়ম ৪: অনুমোদিত রেজিস্ট্রি ছাড়া ইমেজ নয়
deny contains msg if {
        some container in input.request.object.spec.containers
        not startswith(container.image, "ghcr.io/technova/")
        msg := sprintf("Image %v is not from the approved registry", [container.image])
}
```

এই নীতি পরীক্ষা করার পদ্ধতিটিই Policy as Code-কে শক্তিশালী করে: নীতির সঙ্গে তার পরীক্ষাও লেখা হয়, যাতে ভুল নীতি প্রোডাকশনে না যায়।

```
# policies/kubernetes_test.rego — Rego ইউনিট টেস্ট
package kubernetes.admission

import rego.v1

test_root_denied if {
        deny["Container web must not run as root (runAsUser 0)"] with input as {
                "request": {"object": {
                        "metadata": {"labels": {"team": "platform"}},
                        "spec": {"containers": [
                                {"name": "web", "image": "ghcr.io/technova/web:1.4.0",
                                 "securityContext": {"runAsUser": 0}}
                        ]}
                }}
        }
}

test_good_pod_allowed if {
        count(deny) == 0 with input as {
                "request": {"object": {
                        "metadata": {"labels": {"team": "platform"}},
                        "spec": {"containers": [
```

```
                                {"name": "web", "image": "ghcr.io/technova/web:1.4.0",
                                 "securityContext": {"runAsUser": 10001}}
                        ]}
                }}
        }
}

# ল্যাবে চালান:
# opa test policies/ -v
# opa eval --data policies/kubernetes.rego --input pod.json "data.kubernetes.admission.deny"
```

এবার একই নিয়ম Kyverno-তে দেখুন — লক্ষ করুন, এটি সাধারণ Kubernetes YAML, তাই Kubernetes জানা কেউ Rego না শিখেও নীতি লিখতে পারেন।

```
# kyverno/require-non-root.yaml — Kyverno ClusterPolicy
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-non-root-and-labels
  annotations:
    policies.kyverno.io/title: Require non-root and team label
spec:
  validationFailureAction: Enforce   # প্রথমে Audit দিয়ে শুরু করুন
  background: true                   # চলমান সম্পদও নিরীক্ষা করবে
  rules:
    - name: check-run-as-user
      match:
        any:
          - resources:
              kinds: [Pod]
      validate:
        message: "Containers must not run as root."
        pattern:
          spec:
            containers:
              - securityContext:
                  runAsUser: ">0"
    - name: require-team-label
      match:
        any:
          - resources:
              kinds: [Pod]
      validate:
        message: "Every Pod must have a team label."
        pattern:
          metadata:
            labels:
              team: "?*"
    - name: deny-latest-tag
      match:
        any:
          - resources:
              kinds: [Pod]
      validate:
        message: "Images must not use the :latest tag."
        deny:
          conditions:
            any:
```

```
              - key: "{{ request.object.spec.containers[].image }}"
                operator: AnyIn
                value: ["*:latest"]
```

```
# Kyverno mutate উদাহরণ: লেবেল না থাকলে স্বয়ংক্রিয়ভাবে যোগ করা
# kyverno/add-default-label.yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: add-default-labels
spec:
  rules:
    - name: add-team-label
      match:
        any:
          - resources:
              kinds: [Deployment]
      mutate:
        patchStrategicMerge:
          metadata:
            labels:
              team: platform
              environment: staging

# ল্যাবে প্রয়োগ ও পরীক্ষা:
# kubectl apply -f kyverno/require-non-root.yaml
# kubectl get clusterpolicy
# kyverno test kyverno/ --test-case test.yaml
# kubectl run test-pod --image=ghcr.io/technova/web:latest --dry-run=server -o yaml
```

Terraform নীতি যাচাইয়ের উদাহরণ: ডেপ্লয়ের আগে `terraform plan`-এর JSON আউটপুট OPA দিয়ে পরীক্ষা করা হয় — যেমন, কোনো স্টোরেজ বাকেট পাবলিক ACL পাবে না।

```
# policies/terraform.rego — Terraform plan যাচাই
package terraform

import rego.v1

deny contains msg if {
        some resource in input.planned_values.root_module.resources
        resource.type == "aws_s3_bucket"
        resource.values.acl == "public-read"
        msg := sprintf("S3 bucket %v must not be public", [resource.address])
}

# CI ধাপে:
# terraform plan -out=tfplan.binary
# terraform show -json tfplan.binary > tfplan.json
# conftest test tfplan.json --policy policies/terraform.rego
# অথবা: opa eval --data policies/terraform.rego --input tfplan.json "data.terraform.deny"
```

> **WARNING:** নীতি প্রথম দিনেই `Enforce` মোডে বসাবেন না। প্রথমে `Audit` মোডে চালান — কী ভাঙত, তা দেখুন; বিদ্যমান কাজের ওপর প্রভাব মাপুন; ব্যতিক্রমের তালিকা তৈরি করুন; তারপর ধাপে ধাপে Enforce করুন। হঠাৎ করে কঠোর নীতি বসিয়ে প্রোডাকশন ডেপ্লয় বন্ধ করে দেওয়া Policy as Code-এর সবচেয়ে সাধারণ ও সবচেয়ে ক্ষতিকর ভুল।

সম্মতি স্বয়ংক্রিয়করণ (compliance automation) এই কাঠামোর স্বাভাবিক পরিণতি। যখন নীতি কোডে লেখা থাকে, তখন সম্মতি যাচাই আর বার্ষিক হাতে-করা পরীক্ষা থাকে না — প্রতিটি পরিবর্তনে স্বয়ংক্রিয়ভাবে যাচাই হয়। উদাহরণ: TechNova-র একটি সম্মতি নিয়ম বলে প্রতিটি প্রোডাকশন সম্পদে এনক্রিপশন চালু থাকতে হবে। এই নিয়মটি একবার Rego বা Kyverno নীতি হিসেবে লেখা হলে, CI-তে Terraform যাচাই, admission-এ Kubernetes যাচাই, আর runtime-এ CSPM নিরীক্ষা — তিন জায়গাতেই একই নিয়ম চলে। অডিটের সময় আর আলাদা করে প্রমাণ জোগাড় করতে হয় না; পাইপলাইনের লগ, নীতির সিদ্ধান্ত লগ ও CSPM রিপোর্ট মিলেই প্রমাণ হয়। তবে এখানে একটি সতর্কতা আছে: স্বয়ংক্রিয় সম্মতি শুধু সেই নিয়মগুলোই যাচাই করতে পারে, যেগুলো কোডে প্রকাশ করা যায়। মানুষের বিচার দরকার এমন বিষয় — যেমন একটি ঝুঁকি গ্রহণ করা যুক্তিসঙ্গত কি না — সেগুলো স্বয়ংক্রিয় করা যায় না; সেগুলোর জন্য মানুষের পর্যালোচনা ও নথিভুক্ত সিদ্ধান্ত দরকার। ভালো ব্যবস্থা তাই দুটির সংমিশ্রণ: যা কোডে প্রকাশ করা যায়, তা স্বয়ংক্রিয়; যা বিচার দরকার, তা মানুষের — কিন্তু সেই বিচারের রেকর্ডও সিস্টেমে থাকে। নীতির সংস্করণ নিয়ন্ত্রণ এখানে আবার গুরুত্বপূর্ণ হয়ে ওঠে: সম্মতি নিয়ম বদলালে, সেই পরিবর্তনও PR-এ পর্যালোচিত হয়, পরীক্ষা চলে, আর পুরোনো সংস্করণে ফিরে যাওয়া যায়। এইভাবে Policy as Code শুধু নিরাপত্তা দেয় না, সংস্থাকে অডিটের জন্য সবসময় প্রস্তুত রাখে।

১২.৭ প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — জরুরি ডেপ্লয় নীতিতে আটকে গেছে। একটি গুরুতর বাগ ঠিক করতে ডেপ্লয় করতে গিয়ে দেখা গেল Pod-এ `team` লেবেল নেই বলে Kyverno আটকে দিয়েছে। সঠিক প্রতিক্রিয়া: নীতি বন্ধ করা নয়; লেবেল যোগ করে আবার ডেপ্লয় করা, আর পরে CI-তে একটি চেক যোগ করা যাতে লেবেলহীন ম্যানিফেস্ট PR পর্যায়েই ধরা পড়ে। নীতিকে বাইপাস করার জন্য জরুরি পথ থাকতে পারে, কিন্তু সেই পথে দুজনের অনুমোদন, সময়সীমা ও অডিট রেকর্ড থাকা চাই।

পরিস্থিতি ২ — নীতির নিজের বাগ। একটি Rego নীতিতে `not` শর্তের ভুল ব্যবহারের কারণে বৈধ Pod-ও আটকে যাচ্ছে। এখানেই নীতির পরীক্ষা (opa test) কাজে আসে: পরীক্ষা ছাড়া নীতি বদলানো মানে প্রোডাকশন ক্লাস্টারে অপরীক্ষিত কোড ডেপ্লয় করা। প্রতিটি নীতি পরিবর্তন PR-এ যাবে, পরীক্ষা চলবে, আর ক্যানারি হিসেবে প্রথমে একটি Namespace-এ Audit মোডে চলবে।

পরিস্থিতি ৩ — দলগুলোর ভিন্ন প্রয়োজন। প্ল্যাটফর্ম দল কঠোর নীতি চায়, অ্যাপ দল নমনীয়তা চায়। সমাধান: সংস্থা-ব্যাপী বেসলাইন নীতি (যেমন root নিষিদ্ধ) সবাই মানে; দল-নির্দিষ্ট নীতি Namespace-ভিত্তিক ব্যতিক্রম হিসেবে নথিভুক্ত হয়, মালিক ও মেয়াদসহ। নীতি একরকম নয় — নীতি একটি স্তরবিন্যাস (hierarchy)।

পরিস্থিতি ৪ — Terraform drift। কেউ কনসোল থেকে হাতে বাকেট পাবলিক করে দিয়েছে; Terraform plan যাচাই তা ধরতে পারে না, কারণ পরিবর্তনটি Terraform-এর বাইরে হয়েছে। তাই runtime audit দরকার — নিয়মিতভাবে ক্লাউডের বাস্তব অবস্থা নীতির বিরুদ্ধে পরীক্ষা করা, যেমনটা পরের অধ্যায়গুলোতে CSPM প্রসঙ্গে আসবে। নীতি শুধু প্রবেশদ্বারে নয়, সার্বক্ষণিক পাহারায় থাকতে হয়।

১২.৮ নিরাপত্তা ও খরচ বিবেচনা

Policy as Code-এর নিরাপত্তা ঝুঁকি নিজেই ভাবনার বিষয়: নীতি ইঞ্জিন যদি আপস হয়, তাহলে আক্রমণকারী নীতি বদলে নিজের পথ খুলে নিতে পারে। তাই নীতি সংরক্ষণাগার (policy repository) সুরক্ষিত রাখুন — শাখা সুরক্ষা (branch

protection), CODEOWNERS-এর মাধ্যমে নীতি পরিবর্তনে নিরাপত্তা দলের অনুমোদন, আর নীতি ডেপ্লয়ের অডিট লগ। OPA-র সিদ্ধান্ত লগ (decision log) চালু রাখুন, যাতে কোনো অনুরোধ কেন অনুমোদিত বা অস্বীকৃত হয়েছে, তা পরে তদন্ত করা যায়।

খরচের দিক: OPA ও Kyverno ওপেন সোর্স, তাই লাইসেন্স খরচ নেই; খরচ হলো শেখার সময়, নীতি রক্ষণাবেক্ষণ, আর ভুল নীতির কারণে থেমে যাওয়া ডেপ্লয়ের সুযোগ-খরচ। এই খরচ কমানোর উপায়: ছোট থেকে শুরু করা (তিন-চারটি উচ্চ-মূল্যের নীতি), কমিউনিটির প্রমাণিত নীতি লাইব্রেরি (যেমন Kyverno-র নীতি সংগ্রহ) থেকে নিয়ে নিজের পরিবেশে মানিয়ে নেওয়া, আর নীতির কভারেজ ও লঙ্ঘনের মেট্রিক নিয়মিত পর্যালোচনা করা। মনে রাখবেন, একটি নীতি যা কেউ বোঝে না, সেটি রক্ষণাবেক্ষণ করা যায় না — নীতির সঙ্গে তার যুক্তি (rationale) বাংলায় বা ইংরেজিতে মন্তব্য হিসেবে লিখুন।

১২.৯ সাধারণ ভুল

- **অস্পষ্ট নীতি লেখা:** "নিরাপদ ইমেজ ব্যবহার করুন" — পরীক্ষা করা যায় না; বদলে বলুন "শুধু ghcr.io/technova/ রেজিস্ট্রির ইমেজ, latest ছাড়া"।
- **পরীক্ষা ছাড়া নীতি ডেপ্লয়:** Rego বা Kyverno নীতির ইউনিট টেস্ট না লেখা — নীতির বাগ প্রোডাকশন বন্ধ করে দেয়।
- **প্রথম দিনেই Enforce:** Audit ছাড়া সরাসরি Enforce করে দলকে থামিয়ে দেওয়া।
- **নীতিকে সবকিছুর সমাধান ভাবা:** নীতি আটকাতে পারে, কিন্তু ঠিক করতে পারে না — ডেভেলপারকে সঠিক পথটাও দেখাতে হয় (mutate নীতি, টেমপ্লেট, নথি)।
- **ব্যতিক্রমের কোনো পথ না রাখা:** ব্যতিক্রম ছাড়া নীতি ভেঙে ফেলা হয় নীরবে; ব্যতিক্রমসহ নীতি টিকে থাকে।
- **নীতির সংস্করণ নিয়ন্ত্রণ না করা:** ক্লাস্টারে হাতে নীতি বদলানো, Git-এ না রাখা — তখন কে কী বদলাল, জানা যায় না।
- **একই নীতি তিন জায়গায় তিন রকম লেখা:** CI, admission ও runtime-এ নীতি আলাদা হলে তারা পরস্পরবিরোধী সিদ্ধান্ত দেয়; এক উৎস থেকে পুনরায় ব্যবহার করুন।

১২.১০ সমস্যা নির্ণয়

সমস্যা: Kyverno নীতি Pod আটকাচ্ছে, কিন্তু কেন বোঝা যাচ্ছে না। নির্ণয়: `kubectl describe pod` ও Kyverno-র ইভেন্ট দেখুন — নীতির নাম ও বার্তা সেখানে থাকে; `kubectl get events --field-selector reason=PolicyViolation` দিয়ে লঙ্ঘনগুলো দেখুন; আর নীতির `validationFailureAction` Audit না Enforce, তা যাচাই করুন।

সমস্যা: opa test পাস করছে, কিন্তু আসল ইনপুটে নীতি কাজ করছে না। নির্ণয়: পরীক্ষার ইনপুট আর আসল ইনপুটের কাঠামো মিলছে কি না দেখুন — যেমন আসল Pod-এ `securityContext` অনুপস্থিত থাকলে `runAsUser == 0` শর্তটি undefined হয়ে মিথ্যা হয়; অনুপস্থিত ক্ষেত্রের জন্য `default` বা `object.get()` ব্যবহার করুন।

সমস্যা: Terraform plan যাচাই কিছু ধরছে না। নির্ণয়: `terraform show -json`-এর আউটপুটে `planned_values` কাঠামো দেখুন — মডিউল ব্যবহার করলে সম্পদ `child_modules`-এর ভেতরে থাকে, `root_module.resources`-এ নয়; নীতিকে পুনরাবৃত্তভাবে (recursively) সব মডিউল ঘুরতে হবে।

সমস্যা: নীতি মূল্যায়ন ধীর হয়ে গেছে। নির্ণয়: নীতির সংখ্যা ও জটিলতা দেখুন — প্রতিটি অনুরোধে সব নীতি মূল্যায়ন হয়; ভারী নীতি (যেমন বড় তালিকার বিরুদ্ধে মিল) হালকা করুন, অপ্রয়োজনীয় নীতি সরান, আর OPA-র সিদ্ধান্ত লগে মূল্যায়নের সময় পরিমাপ করুন।

TROUBLESHOOTING: নীতি নিয়ে সমস্যা হলে প্রথমে প্রশ্ন করুন: নীতিটি কি ভুল সিদ্ধান্ত দিচ্ছে, নাকি সঠিক সিদ্ধান্ত দিচ্ছে কিন্তু ভুল জায়গায় প্রয়োগ হচ্ছে? প্রথমটি নীতির যুক্তির ভুল; দ্বিতীয়টি নীতির সুযোগ (scope) বা মোডের ভুল — দুটির প্রতিকার আলাদা, আর লগ না দেখে আন্দাজে নীতি বদলাবেন না।

১২.১১ সারসংক্ষেপ

Policy as Code মানে নীতিকে মানুষের স্মৃতি থেকে মেশিনের পরীক্ষায় নিয়ে আসা: OPA ও Rego সাধারণ উদ্দেশ্যের নীতি ইঞ্জিন, Kyverno Kubernetes-এর জন্য সহজ পথ, আর Terraform plan যাচাই ডেপ্লয়ের আগের প্রতিরক্ষা। ভালো নীতি সুনির্দিষ্ট, পরীক্ষাযোগ্য, পর্যায়ক্রমে কঠোর ও ব্যতিক্রম-সহনশীল হয়। নীতি নিজেও কোড — তাই তারও পরীক্ষা, পর্যালোচনা, সংস্করণ নিয়ন্ত্রণ ও পর্যবেক্ষণ দরকার। TechNova-র root-নিষেধ, latest-নিষেধ ও লেবেল-বাধ্যতামূলক নীতিগুলো এই নীতিরই জীবন্ত উদাহরণ।

১২.১২ পর্যালোচনা প্রশ্ন

1. Policy as Code বলতে কী বোঝায়, এবং নথিভুক্ত নীতির তুলনায় এর সুবিধা কী কী?
2. নীতি, নিয়ন্ত্রণ ও প্রক্রিয়ার মধ্যে পার্থক্য কী? একটি উদাহরণ দিয়ে বোঝান।
3. OPA ও Kyverno-র মধ্যে পার্থক্য কী, এবং কখন কোনটি বেছে নেবেন?
4. Rego-তে `deny` সেট কীভাবে কাজ করে? একটি নিয়মের মাথা ও দেহ বলতে কী বোঝায়?
5. Rego-তে অনুপস্থিত ক্ষেত্র (field) কেন বিপজ্জনক, এবং কীভাবে তা সামলাতে হয়?
6. Validate, Mutate ও Generate নীতির পার্থক্য কী? প্রতিটির একটি উদাহরণ দিন।
7. Terraform plan যাচাই কীভাবে কাজ করে, এবং মডিউল ব্যবহার করলে কী সতর্কতা দরকার?
8. Admission নীতি বলতে কী বোঝায়, এবং এটি CI যাচাইয়ের থেকে কীভাবে আলাদা?
9. নীতি প্রথমে Audit মোডে চালানো কেন গুরুত্বপূর্ণ?
10. নীতি ইঞ্জিন নিজে আপস হলে কী হতে পারে, এবং কীভাবে তা রক্ষা করবেন?

১২.১৩ ব্যবহারিক অনুশীলন

1. **Rego নীতি লেখা:** উপরের চারটি নিয়মের Rego নীতি নিজে লিখুন, তারপর একটি বৈধ Pod ও তিনটি অবৈধ Pod-এর ইনপুট দিয়ে `opa test` চালিয়ে যাচাই করুন।
2. **Kyverno ল্যাব:** ল্যাব ক্লাস্টারে (kind বা minikube) Kyverno ইনস্টল করে Audit মোডে নীতি বসান, একটি root Pod ডেপ্লয় করার চেষ্টা করে লঙ্ঘন দেখুন, তারপর Enforce মোডে বদলে আবার চেষ্টা করুন।
3. **Terraform গেট:** একটি Terraform কনফিগারেশনে ইচ্ছাকৃতভাবে একটি পাবলিক বাকেট যোগ করে plan-এর JSON OPA দিয়ে যাচাই করুন; দেখুন নীতি কীভাবে তা ধরে।
4. **নীতি পর্যালোচনা:** নিজের লেখা একটি নীতির জন্য পাঁচটি পরীক্ষা-কেস লিখুন — বৈধ, অবৈধ, সীমানা-কেস (যেমন ক্ষেত্র অনুপস্থিত) ও ব্যতিক্রমসহ।

অধ্যায় ১৩ • PART III — DevSecOps Automation & Governance

Chapter 13 — Vulnerability Management

১৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — দুর্বলতা ব্যবস্থাপনার সম্পূর্ণ জীবনচক্র (lifecycle) ব্যাখ্যা করতে: আবিষ্কার থেকে প্রতিকার পর্যন্ত প্রতিটি ধাপের কাজ, মালিক ও সময়সীমা বলতে; CVE ও CVSS-এর অর্থ বোঝাতে এবং CVSS-এর ভিত্তি মেট্রিক (base metrics) ও তীব্রতা স্তর (severity bands) সঠিকভাবে ব্যাখ্যা করতে; ঝুঁকি-ভিত্তিক অগ্রাধিকার (risk prioritization) নির্ধারণ করতে — শুধু স্কোর নয়, প্রসঙ্গ (exposure, reachability, exploitability) দেখে; দুর্বলতা স্ক্যানিং, প্যাচ ব্যবস্থাপনা, ব্যতিক্রম পরিচালনা ও প্রতিকার SLA-র নকশা করতে; দুর্বলতা প্রতিবেদন (reporting) লিখতে যা প্রযুক্তিগত দল ও ব্যবস্থাপনা — দুই পক্ষেরই কাজে আসে; এবং ক্রমাগত মূল্যায়নের (continuous assessment) একটি বাস্তবসম্মত চক্র চালাতে।

১৩.২ পূর্বশর্ত পুনরালোচনা

আগের দুই অধ্যায় থেকে আপনার জানা থাকা উচিত: SCA ও কনটেইনার স্ক্যানিং কীভাবে পরিচিত দুর্বলতা খুঁজে পায়, এবং SBOM কীভাবে উপাদানের তালিকা দেয়। এই অধ্যায় সেই স্ক্যানের ফলাফল নিয়ে কাজ করে: স্ক্যানার যখন শত শত দুর্বলতা দেখায়, তখন কোনটি আগে ঠিক করবেন, কোনটি পরে, কোনটি গ্রহণ করবেন — সেই সিদ্ধান্তের শৃঙ্খলাই দুর্বলতা ব্যবস্থাপনা। CVE (Common Vulnerabilities and Exposures) হলো একটি দুর্বলতার সর্বজনীন পরিচয় — যেমন CVE-2021-44228 — যা দিয়ে সবাই একই দুর্বলতা নিয়ে কথা বলতে পারে। CVSS (Common Vulnerability Scoring System) হলো সেই দুর্বলতার তীব্রতা মাপার একটি মানসম্মত কাঠামো। দুটি এক নয়: CVE বলে কোন দুর্বলতা, CVSS বলে তা কতটা গুরুতর হতে পারে।

TechNova-র প্রসঙ্গে ভাবুন: সোমবার সকালের স্ক্যানে ১৪৭টি দুর্বলতা দেখা গেল — ৩টি Critical, ২১টি High, বাকিগুলো Medium ও Low। দলের হাতে এই সপ্তাহে মাত্র দুজন ডেভেলপার-দিন। সব ঠিক করা সম্ভব নয়। কোন তিনটি আগে? শুধু স্কোর দেখে সিদ্ধান্ত নিলে ভুল হতে পারে — হয়তো একটি Critical দুর্বলতা এমন একটি ফাংশনে আছে, যা TechNova-র কোড কখনো ডাকে না, আর একটি High দুর্বলতা আছে ইন্টারনেট-মুখী লগইন পাতায়। এই অধ্যায় সেই বিচার শেখাবে।

১৩.৩ প্রথম নীতি থেকে বোঝা: দুর্বলতা কেন অনিবার্য, আর ব্যবস্থাপনা কেন অপরিহার্য

প্রথম নীতি থেকে শুরু করা যাক। কোনো জটিল সফটওয়্যারই সম্পূর্ণ ত্রুটিমুক্ত হয় না — কারণ সফটওয়্যার লেখে মানুষ, নির্ভর করে হাজার হাজার অন্য মানুষের লেখা কোডের ওপর, আর চলে এমন পরিবেশে যা প্রতিনিয়ত বদলায়। নতুন দুর্বলতা প্রতিদিন প্রকাশিত হয়; গতকাল নিরাপদ বলে জানা সংস্করণ আজ দুর্বল বলে প্রমাণিত হতে পারে। তাই লক্ষ্য "শূন্য দুর্বলতা" নয় — লক্ষ্য হলো দুর্বলতা জানা, মাপা, অগ্রাধিকার দেওয়া ও সময়মতো ঠিক করা, যাতে আক্রমণকারীর আগে আপনি পদক্ষেপ নিতে পারেন।

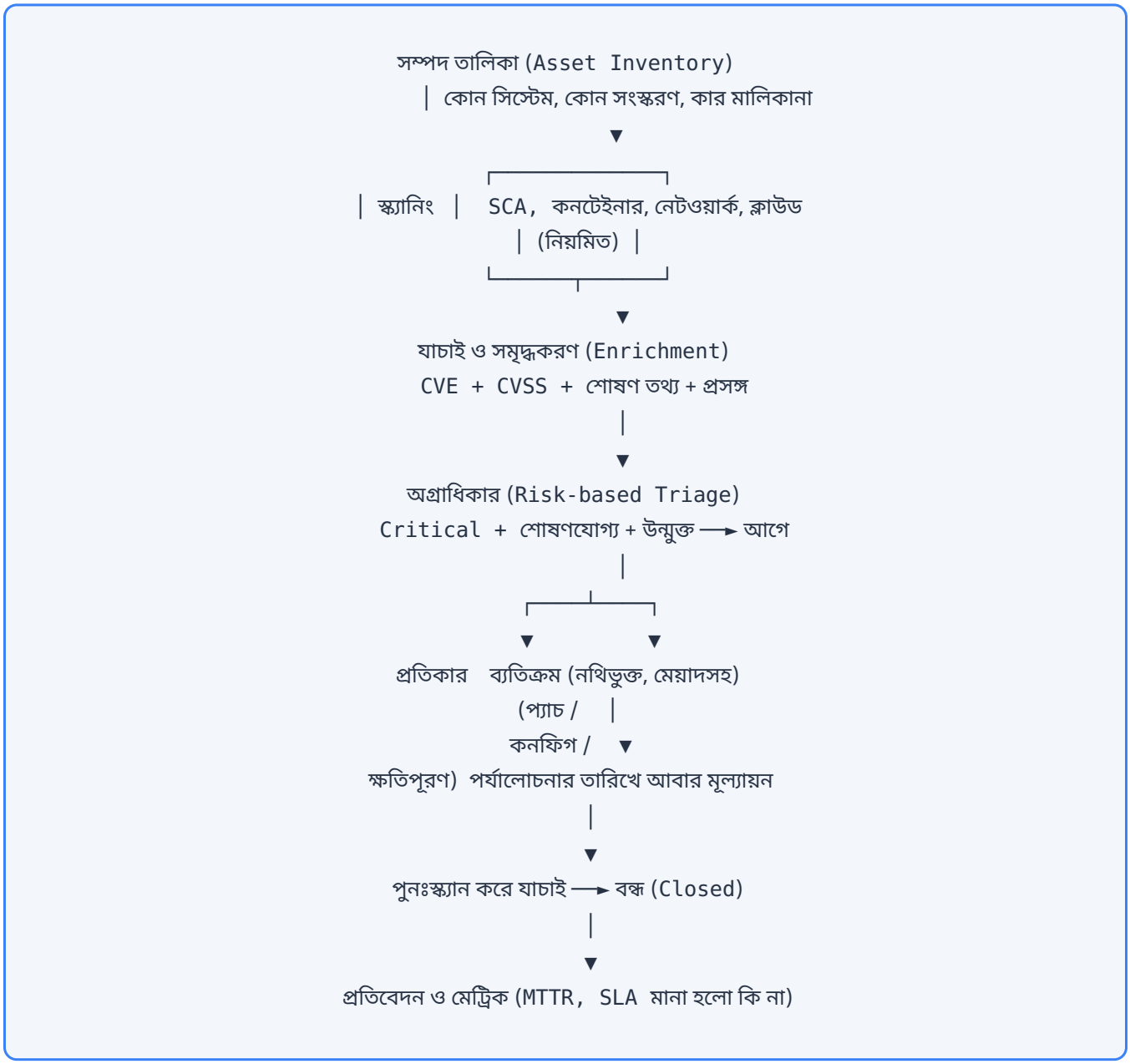
দুর্বলতা জীবনচক্রের প্রতিটি ধাপের নিজস্ব সমস্যা আছে। আবিষ্কার (discovery): সব সম্পদ কি স্ক্যানের আওতায় আছে? ছায়া IT (shadow IT) — দলের অজান্তে চলা সার্ভার বা SaaS — স্ক্যান হয় না। যাচাই (validation): স্ক্যানার যা দেখায়, তা কি সত্যিই দুর্বল? মিথ্যা ইতিবাচক সময় নষ্ট করে, মিথ্যা নেতিবাচক (missed) বিপদ ডেকে আনে। অগ্রাধিকার (prioritization): কোনটি আগে? প্রতিকার (remediation): প্যাচ করবেন, কনফিগারেশন বদলাবেন, নাকি ক্ষতিপূরণমূলক নিয়ন্ত্রণ (compensating control) দেবেন? যাচাই (verification): প্যাচটি সত্যিই কাজ করেছে কি না, আবার স্ক্যান করে দেখা। প্রতিবেদন (reporting): কে কী জানবে, কতটা বিস্তারিত? প্রতিটি ধাপে ব্যর্থতা সম্ভব, আর ব্যর্থতা নীরব — কেউ খেয়াল না করলে দুর্বলতা মাসের পর মাস থেকে যায়।

CVSS বোঝা এই কারণে গুরুত্বপূর্ণ। CVSS একটি দুর্বলতার অন্তর্নিহিত বৈশিষ্ট্য থেকে একটি সংখ্যা (০.০ থেকে ১০.০) বের করে। এর ভিত্তি মেট্রিক (base metrics) আটটি: আক্রমণ ভেক্টর (Attack Vector — নেটওয়ার্ক থেকে আক্রমণ করা যায়, নাকি স্থানীয় প্রবেশাধিকার দরকার?), আক্রমণের জটিলতা (Attack Complexity — বিশেষ শর্ত দরকার কি না?), প্রয়োজনীয় অধিকার (Privileges Required — আক্রমণকারীর কি আগে থেকে কোনো অ্যাকাউন্ট দরকার?), ব্যবহারকারীর মিথস্ক্রিয়া (User Interaction — ভুক্তভোগীর ক্লিক দরকার কি না?), প্রভাবের পরিধি (Scope — দুর্বল উপাদান ছাড়িয়ে অন্য উপাদান আক্রান্ত হয় কি না?), এবং গোপনীয়তা, অখণ্ডতা ও প্রাপ্যতার ওপর প্রভাব (Confidentiality, Integrity, Availability Impact — প্রতিটি None, Low বা High)। এই মেট্রিকগুলো একসঙ্গে একটি সূত্রে বসে ভিত্তি স্কোর তৈরি করে। তীব্রতা স্তরগুলো মানসম্মত: ০.০ হলো None, ০.১–৩.৯ Low, ৪.০–৬.৯ Medium, ৭.০–৮.৯ High, এবং ৯.০–১০.০ Critical।

> **CONCEPT:** CVSS স্কোর একটি দুর্বলতার অন্তর্নিহিত তীব্রতা মাপে — আপনার পরিবেশের ঝুঁকি নয়। একই CVSS ৯.৮ দুর্বলতা ইন্টারনেট-মুখী সার্ভারে ভয়ংকর, আর বিচ্ছিন্ন অভ্যন্তরীণ ল্যাবে সীমিত। CVSS-এর আরও দুটি স্তর আছে — Temporal (সময়ের সঙ্গে বদলায়, যেমন শোষণের কোড প্রকাশিত হয়েছে কি না) ও Environmental (আপনার পরিবেশের প্রসঙ্গ) — কিন্তু বেশিরভাগ প্রতিবেদনে শুধু ভিত্তি স্কোরই দেওয়া থাকে। তাই স্কোরকে শুরুর বিন্দু ভাবুন, শেষ সিদ্ধান্ত নয়। কোনো নির্দিষ্ট CVE-র স্কোর মুখস্থ করে বলা এই বইয়ের কাজ নয়; স্কোর যাচাই করতে হলে NVD বা বিক্রেতার পরামর্শ (advisory) দেখুন — সেখানকার সংখ্যাই প্রামাণিক।

ঝুঁকির সূত্রটি মনে রাখুন: ঝুঁকি = হুমকি × দুর্বলতা × প্রভাব। CVSS মূলত দুর্বলতার তীব্রতা মাপে; হুমকি (আক্রমণকারী কি এই দুর্বলতা সক্রিয়ভাবে শোষণ করছে?) ও প্রভাব (এই সিস্টেম আক্রান্ত হলে কী ক্ষতি?) — এই দুটি আপনার পরিবেশ থেকে আসে। তাই অগ্রাধিকারের সঠিক প্রশ্নটি হলো: এই দুর্বলতা, এই সিস্টেমে, এই মুহূর্তে, কতটা শোষণযোগ্য ও কতটা ক্ষতিকর? সেই প্রশ্নের উত্তরেই Critical-এর চেয়ে High আগে ঠিক হতে পারে।

১৩.৪ স্থাপত্য: দুর্বলতা ব্যবস্থাপনার চক্র



এই চক্রের কেন্দ্রে আছে সম্পদ তালিকা (asset inventory)। আপনি যা জানেন না, তা স্ক্যান করতে পারবেন না; যা স্ক্যান করতে পারেন না, তা ঠিক করতে পারবেন না। TechNova-র তালিকায় আছে: প্রোডাকশন সার্ভার 203.0.113.10, স্টেজিং 203.0.113.20, MongoDB 203.0.113.30, প্রতিটি কনটেইনার ইমেজ ও তার SBOM, আর প্রতিটি নির্ভরতার সংস্করণ। তালিকা ছাড়া স্ক্যানিং অন্ধ। স্ক্যানিংয়ের ধরনগুলো: নির্ভরতা ও কনটেইনার স্ক্যান (আগের অধ্যায়ে দেখা), নেটওয়ার্ক স্ক্যান (খোলা পোর্ট ও সেবার দুর্বলতা), ক্লাউড কনফিগারেশন স্ক্যান (পরের অধ্যায়ের CSPM), আর প্রয়োজনে অনুমোদিত পরিবেশে পেনিট্রেশন পরীক্ষা — যা শুধুমাত্র অনুমোদিত, বিচ্ছিন্ন ল্যাবে, লিখিত অনুমতি নিয়ে করা যায়।

অগ্রাধিকারের কাঠামোটি একটি সারণিতে সাজানো যায় — স্কোর, শোষণযোগ্যতা ও উন্মুক্ততা — এই তিন অক্ষে।

অগ্রাধিকার	শর্ত	সাধারণ SLA
P1 — জরুরি	Critical + সক্রিয়ভাবে শোষিত হচ্ছে + ইন্টারনেট-মুখী	২৪–৭২ ঘণ্টা
P2 — উচ্চ	High/Critical + শোষণের কোড প্রকাশিত + উন্মুক্ত	৭–১৪ দিন

P3 — মাঝারি	Medium/High, অভ্যন্তরীণ, শোষণ কঠিন	৩০ দিন
P4 — নিম্ন	Low, বা reachability নেই	৯০ দিন বা পরবর্তী রিলিজ

এই SLA-গুলো উদাহরণ মাত্র — প্রতিটি সংস্থা নিজের ঝুঁকি-সহনশীলতা অনুযায়ী এগুলো নির্ধারণ করে। গুরুত্বপূর্ণ বিষয় হলো SLA থাকা, তার মালিক থাকা, আর SLA ভাঙলে তা দৃশ্যমান হওয়া। SLA ছাড়া দুর্বলতা ব্যবস্থাপনা শুধু তালিকা বানানো।

১৩.৫ অভ্যন্তরীণ কার্যপ্রণালী: স্ক্যানার থেকে প্রতিকার পর্যন্ত

স্ক্যানার কীভাবে কাজ করে, তা বোঝা যাচাইয়ের জন্য অপরিহার্য। SCA স্ক্যানার লকফাইল পড়ে প্রতিটি প্যাকেজ-সংস্করণকে দুর্বলতা ডেটাবেসের সঙ্গে মেলায় — মিল পেলে CVE নম্বর, প্রভাবিত সংস্করণের পরিসর ও ঠিক করা সংস্করণ জানায়। কনটেইনার স্ক্যানার ইমেজের স্তর খুলে OS প্যাকেজ তালিকা বের করে একই মিল চালায়। নেটওয়ার্ক স্ক্যানার পোর্টে সংযোগ করে সেবার ব্যানার ও আচরণ থেকে সংস্করণ অনুমান করে। তিনটি পদ্ধতিই "পরিচিত" দুর্বলতা খোঁজে — অজানা (zero-day) দুর্বলতা স্ক্যানার ধরতে পারে না। এই সীমাটি বোঝা গুরুত্বপূর্ণ: স্ক্যান পরিষ্কার মানেই নিরাপদ নয়।

প্যাচ ব্যবস্থাপনা (patch management) হলো প্রতিকারের মূল কাজ। প্যাচের ধাপগুলো: প্রথমে পরীক্ষার পরিবেশে প্যাচ প্রয়োগ করে দেখুন অ্যাপ ভাঙে কি না; তারপর স্টেজিংয়ে; তারপর ক্যানারি হিসেবে প্রোডাকশনের একটি ছোট অংশে; তারপর পুরো প্রোডাকশনে। জরুরি প্যাচে এই ধাপগুলো সংক্ষিপ্ত হয়, কিন্তু বাদ দেওয়া হয় না — বরং পরীক্ষার সময় কমিয়ে ও পর্যবেক্ষণ বাড়িয়ে ঝুঁকি সামলানো হয়। প্যাচের পর পুনঃস্ক্যান করে যাচাই করুন: দুর্বলতাটি সত্যিই গেছে কি না। প্যাচ ছাড়াও প্রতিকারের পথ আছে: কনফিগারেশন পরিবর্তন (যেমন দুর্বল প্রোটোকল বন্ধ করা), ক্ষতিপূরণমূলক নিয়ন্ত্রণ (যেমন WAF নিয়ম দিয়ে আক্রমণের পথ বন্ধ করা), অথবা ঝুঁকিপূর্ণ ফিচারটি সাময়িকভাবে বন্ধ করা। কখনো কখনো সঠিক প্রতিকার হলো প্যাচ নয় — সঠিক প্রতিকার হলো আক্রান্ত উপাদানটি সরিয়ে ফেলা।

ব্যতিক্রম পরিচালনা (exception handling) হলো সেই শৃঙ্খলা, যা দুর্বলতা ব্যবস্থাপনাকে বাস্তবসম্মত রাখে। সব দুর্বলতা এখনই ঠিক করা যায় না — হয়তো প্যাচ নেই, হয়তো আপগ্রেডে অ্যাপ ভাঙে, হয়তো ঝুঁকি নগণ্য। তখন ব্যতিক্রম নেওয়া হয়: লিখিত নথি, যাতে আছে দুর্বলতার বিবরণ, ঝুঁকি বিশ্লেষণ, ক্ষতিপূরণমূলক নিয়ন্ত্রণ, মালিকের নাম, মেয়াদ (সাধারণত ৩০–৯০ দিন) ও পর্যালোচনার তারিখ। মেয়াদ ফুরালে ব্যতিক্রম নিজে থেকেই মেয়াদোত্তীর্ণ হয় — আবার মূল্যায়ন করতে হয়। ব্যতিক্রম ছাড়া দুর্বলতা উপেক্ষা করা হয় নীরবে; ব্যতিক্রমসহ উপেক্ষা করা হয় সচেতন সিদ্ধান্তে। পার্থক্যটি অডিটের সময় স্পষ্ট হয়।

> **WARNING:** কখনোই কোনো নির্দিষ্ট CVE-র স্কোর মুখস্থ বা অনুমান করে বলবেন না — স্কোর সময়ের সঙ্গে বদলাতে পারে (নতুন বিশ্লেষণে পুনর্মূল্যায়ন হয়), আর ভুল স্কোর ভুল অগ্রাধিকার তৈরি করে। প্রতিটি CVE-র জন্য NVD (National Vulnerability Database) বা বিক্রেতার পরামর্শ দেখুন, এবং প্রতিবেদনে উৎস ও তারিখ উল্লেখ করুন। একইভাবে, "এই দুর্বলতা শোষিত হচ্ছে" — এমন দাবি শুধুমাত্র নির্ভরযোগ্য উৎস (যেমন CISA KEV তালিকা) থেকে যাচাই করে বলুন।

১৩.৬ ব্যবহারিক কমান্ড ও কনফিগারেশন

```
# Ubuntu 24.04 ল্যাব — TechNova দুর্বলতা চক্র
cd ~/technova-web

# ১. নির্ভরতার দুর্বলতার তালিকা JSON আকারে (প্রতিবেদনের জন্য)
npm audit --json > /tmp/audit.json
npm audit --audit-level=high
```

```
# ২. নিরাপদে ঠিক করা যায় এমন প্যাচগুলো দেখুন ও প্রয়োগ করুন
npm audit fix --dry-run
npm audit fix

# ৩. একটি নির্দিষ্ট প্যাকেজের তথ্য দেখুন (সংস্করণ, পরামর্শ)
npm view express versions --json | tail -n 20
npm audit --package lodash

# ৪. কনটেইনার ইমেজের দুর্বলতা সারণি আকারে
trivy image --format table --severity HIGH,CRITICAL technova-web:local
trivy image --format json -o /tmp/trivy.json technova-web:local

# ৫. সিস্টেম প্যাকেজ আপডেট (ল্যাব সার্ভার 203.0.113.10)
sudo apt update && sudo apt list --upgradable
sudo unattended-upgrades --dry-run --debug

# ৬. খোলা পোর্ট দেখুন — অপ্রয়োজনীয় পোর্টই দুর্বলতার পথ
sudo ss -tuln
sudo nmap -sV 203.0.113.10   # শুধু নিজের ল্যাব সার্ভারে, অনুমতি নিয়ে
```

দুর্বলতা প্রতিবেদনের একটি নমুনা কাঠামো — প্রযুক্তিগত দলের জন্য বিস্তারিত, ব্যবস্থাপনার জন্য সারসংক্ষেপ:

```
# দুর্বলতা প্রতিবেদন — TechNova — সপ্তাহ: 2026-11-09 (নমুনা)
মোট দুর্বলতা: 147 (Critical: 3, High: 21, Medium: 68, Low: 55)
নতুন এই সপ্তাহে: 12 | বন্ধ হয়েছে: 18 | SLA পেরিয়েছে: 2

P1 — Critical, ইন্টারনেট-মুখী, শোষণের কোড প্রকাশিত
  - উপাদান: web ইমেজের বেস প্যাকেজ (বিস্তারিত: Trivy রিপোর্ট /tmp/trivy.json)
  - মালিক: প্ল্যাটফর্ম দল | সময়সীমা: 72 ঘণ্টা | অবস্থা: প্যাচ পরীক্ষাধীন

P2 — High, reachability নিশ্চিত
  - উপাদান: npm নির্ভরতা (npm audit রিপোর্ট দেখুন)
  - মালিক: ব্যাকএন্ড দল | সময়সীমা: 14 দিন

ব্যতিক্রম (মেয়াদসহ):
  - 1টি Medium: reachability নেই, ক্ষতিপূরণমূলক নিয়ন্ত্রণ চালু,
  মালিক: রাহাত, মেয়াদ: 2026-12-09, পর্যালোচনা: 2026-12-02

মেট্রিক: গড় প্রতিকার সময় (MTTR) Critical: 2.1 দিন | High: 9 দিন
```

> **REAL WORLD:** TechNova-তে একবার একটি High দুর্বলতার প্যাচ স্টেজিংয়ে ঠিকঠাক চললেও প্রোডাকশনে অ্যাপের লগইন ভেঙে গিয়েছিল — কারণ প্যাচ করা লাইব্রেরির আচরণে সামান্য পরিবর্তন ছিল, যা ইউনিট টেস্ট ধরেনি। তারপর থেকে নিয়ম হলো: প্রতিটি নিরাপত্তা প্যাচের সঙ্গে ধোঁয়া পরীক্ষা (smoke test) — লগইন, প্রধান পাতা, পেমেন্ট প্রবাহ — স্টেজিং ও ক্যানারি দুই জায়গাতেই চালাতে হয়। প্যাচ মানেই শুধু সংস্করণ বদলানো নয়; প্যাচ মানেই আচরণ যাচাই।

ক্রমাগত মূল্যায়ন (continuous assessment) মানে এই চক্রকে একবারের প্রকল্প থেকে দৈনন্দিন অভ্যাসে বদলানো। TechNova-তে স্ক্যান চলে প্রতিটি PR-এ, প্রতিটি বিল্ডে, প্রতি রাতে পূর্ণাঙ্গ স্ক্যান, আর প্রতি সপ্তাহে অগ্রাধিকার পর্যালোচনা। নতুন CVE প্রকাশিত হলে SBOM-এর সঙ্গে মিলিয়ে দেখা হয় — আমাদের কোন ইমেজ আক্রান্ত? — কোড খুঁজে বেড়াতে হয় না। এই চক্রের সাফল্য নির্ভর করে তিনটি অভ্যাসের ওপর। প্রথম, সম্পদ তালিকা সবসময় হালনাগাদ রাখা: নতুন সার্ভার, নতুন ইমেজ, নতুন SaaS — সবই তালিকায় ও স্ক্যানের আওতায় আসতে হবে, নইলে অন্ধ জায়গা তৈরি হয়। দ্বিতীয়, স্ক্যানের

ফলাফলকে টিকিট ব্যবস্থার সঙ্গে যুক্ত করা: প্রতিটি P1/P2 দুর্বলতা একটি টিকিট, যার মালিক ও সময়সীমা আছে — ইমেইলে হারিয়ে যাওয়া স্প্রেডশিট নয়। তৃতীয়, নিয়মিত পর্যালোচনা: প্রতি সপ্তাহে দল দেখে — কতটি বন্ধ হয়েছে, কতটি SLA পেরিয়েছে, কোন ব্যতিক্রমের মেয়াদ ফুরোচ্ছে? এই পর্যালোচনা ছাড়া চক্রটি ধীরে ধীরে থেমে যায়, আর দুর্বলতা আবার জমতে থাকে। মনে রাখবেন, আক্রমণকারীও ক্রমাগত মূল্যায়ন করে — নতুন দুর্বলতা প্রকাশিত হওয়ার ঘণ্টার মধ্যে তারা স্ক্যান শুরু করে। আপনার চক্রকে তাদের চেয়ে দ্রুত হতে হবে, অথবা অন্তত তাদের চেয়ে পিছিয়ে থাকা চলবে না।

১৩.৭ প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — সক্রিয়ভাবে শোষিত Critical দুর্বলতা। সকালে খবর এল: TechNova-র ব্যবহৃত একটি লাইব্রেরিতে Critical দুর্বলতা সক্রিয়ভাবে শোষিত হচ্ছে। প্রতিক্রিয়া: প্রথমে SBOM দেখে কোন ইমেজ ও সেবা আক্রান্ত তা চিহ্নিত করুন; তারপর ক্ষতিপূরণমূলক নিয়ন্ত্রণ চালু করুন (যেমন WAF নিয়ম, আক্রান্ত এন্ডপয়েন্ট সাময়িক বন্ধ); তারপর প্যাচ পরীক্ষা করে জরুরি ডেপ্লয় করুন; আর সমান্তরালে লগ দেখুন আক্রমণের চিহ্ন আছে কি না। এই পরিস্থিতিতে যোগাযোগও প্রতিকারের অংশ — কে কী করছে, তা সবাইকে জানান।

পরিস্থিতি ২ — প্যাচ নেই, শুধু পরামর্শ আছে। একটি দুর্বলতার প্যাচ এখনো প্রকাশিত হয়নি, কিন্তু শোষণের কোড আছে। তখন বিকল্প: আক্রান্ত ফিচার বন্ধ করা, নেটওয়ার্ক স্তরে প্রবেশাধিকার সীমিত করা, পর্যবেক্ষণ বাড়ানো, আর বিক্রেতার প্যাচের জন্য অপেক্ষা করা — এই সময়ে ব্যতিক্রম নথিভুক্ত করে রাখা। "প্যাচ নেই" মানে "কিছু করার নেই" নয়।

পরিস্থিতি ৩ — স্ক্যানে শত শত Low দুর্বলতা, দল হতাশ। সব Low একসঙ্গে ঠিক করার চেষ্টা করবেন না। Low-গুলোকে একত্র করে পরবর্তী নিয়মিত রিলিজের সঙ্গে ঠিক করুন; বেস ইমেজ আপডেটে অনেক Low একসঙ্গে যায়। মনোযোগ দিন P1 ও P2-তে — সেখানেই ঝুঁকি।

পরিস্থিতি ৪ — পুরোনো সিস্টেম, প্যাচ করা যায় না। একটি পুরোনো অভ্যন্তরীণ টুলের নির্ভরতা এত পুরোনো যে আপগ্রেড করলে টুলটিই ভেঙে যায়। তখন সিদ্ধান্ত: টুলটিকে নেটওয়ার্কে বিচ্ছিন্ন করা (segmentation), শুধু VPN দিয়ে প্রবেশাধিকার, নিয়মিত পর্যবেক্ষণ, আর প্রতিস্থাপনের পরিকল্পনা। পুরোনো সিস্টেমকে খোলা নেটওয়ার্কে রেখে "পরে দেখা যাবে" বলা কোনো পরিকল্পনা নয়।

১৩.৮ নিরাপত্তা ও খরচ বিবেচনা

দুর্বলতা ব্যবস্থাপনার খরচ: স্ক্যানার ও ডেটাবেস, প্যাচ পরীক্ষার সময়, ডাউনটাইমের ঝুঁকি, আর দলের মনোযোগ। খরচ কমানোর উপায়: স্ক্যান স্বয়ংক্রিয় করা (প্রতিটি বিল্ডে, প্রতি রাতে), প্যাচ পরীক্ষা স্বয়ংক্রিয় করা (smoke test), আর বেস ইমেজ নিয়মিত আপডেট করে অনেক দুর্বলতা একসঙ্গে কমানো। সবচেয়ে বড় খরচ বাঁচে অগ্রাধিকারে: ভুল দুর্বলতায় সময় দিলে আসল ঝুঁকি থেকে যায়।

নিরাপত্তা দিক: স্ক্যানের ফলাফল নিজেই সংবেদনশীল — এতে লেখা থাকে আপনার সিস্টেম কোথায় দুর্বল। স্ক্যান রিপোর্ট শুধু প্রয়োজনীয় লোকের কাছে রাখুন, সংরক্ষণাগারে প্রবেশাধিকার সীমিত করুন। প্যাচ ডেপ্লয়ের সময়ও সাবধান: প্যাচের উৎস যাচাই করুন — অপরিচিত উৎস থেকে প্যাচ নেওয়া নতুন দুর্বলতা ঢোকাতে পারে। আর মনে রাখবেন, দুর্বলতা ব্যবস্থাপনা একটি চলমান চক্র — একবার স্ক্যান করে থেমে গেলে পরের সপ্তাহের নতুন দুর্বলতা ধরা পড়বে না।

১৩.৯ সাধারণ ভুল

- **শুধু CVSS স্কোর দেখে অগ্রাধিকার:** প্রসঙ্গ ছাড়া স্কোর অন্ধ; exposure ও reachability দেখুন।
- **মিথ্যা ইতিবাচককে অবহেলা:** যাচাই না করে সব স্ক্যান ফলাফল সত্যি ধরা, অথবা সবই মিথ্যা বলে উড়িয়ে দেওয়া — দুটোই ভুল।
- **প্যাচের পর যাচাই না করা:** প্যাচ ডেপ্লয় করেই বন্ধ ধরে নেওয়া; পুনঃস্ক্যান ছাড়া বন্ধ নয়।

- **ব্যতিক্রমের মেয়াদ না রাখা:** সাময়িক ছাড় চিরস্থায়ী হয়ে যাওয়া।
- **সম্পদ তালিকা ছাড়া স্ক্যান:** ছায়া সিস্টেম স্ক্যানের বাইরে থেকে যাওয়া।
- **স্ক্যান রিপোর্ট সবার কাছে খোলা রাখা:** আক্রমণকারীর জন্য মানচিত্র বানিয়ে দেওয়া।
- **শূন্য দুর্বলতার লক্ষ্য:** অসম্ভব লক্ষ্য দলকে হতাশ করে; লক্ষ্য হোক সময়মতো প্রতিকার।
- **নির্দিষ্ট CVE-র স্কোর অনুমান করে বলা:** সবসময় প্রামাণিক উৎস (NVD/বিক্রেতা) যাচাই করুন।

১৩.১০ সমস্যা নির্ণয়

সমস্যা: একই দুর্বলতা বারবার ফিরে আসছে। নির্ণয়: প্যাচটি কি সত্যিই ডেপ্লয় হয়েছে, নাকি পুরোনো ইমেজ আবার ডেপ্লয় হয়েছে? SBOM ও ইমেজ ডাইজেস্ট মিলিয়ে দেখুন; বেস ইমেজের সংস্করণ পিন করা আছে কি না দেখুন; আর দেখুন প্যাচটি কোনো শাখায় আটকে আছে কি না।

সমস্যা: স্ক্যানার একটি দুর্বলতা দেখাচ্ছে, কিন্তু প্যাচ করা সংস্করণে আপগ্রেড করেও যাচ্ছে না। নির্ণয়: লকফাইলে পুরোনো সংস্করণ আটকে আছে কি না দেখুন (`npm ls <package>`); transitive নির্ভরতা (অন্য প্যাকেজের ভেতরে থাকা প্যাকেজ) আপডেট হয়েছে কি না দেখুন; প্রয়োজনে overrides/resolutions ব্যবহার করুন।

সমস্যা: SLA বারবার ভাঙছে। নির্ণয়: SLA কি বাস্তবসম্মত? মালিক কি স্পষ্ট? প্যাচ পরীক্ষার পরিবেশ কি প্রস্তুত? প্রায়ই সমস্যা SLA-তে নয়, প্রতিকার প্রক্রিয়ার বাধায় — যেমন স্টেজিং পরিবেশের অভাব বা অনুমোদনের ধীরতা।

সমস্যা: স্ক্যানে কিছুই পাওয়া যাচ্ছে না — সন্দেহজনকভাবে পরিষ্কার। নির্ণয়: স্ক্যানার কি সঠিক লক্ষ্য স্ক্যান করছে? লকফাইল ও ইমেজের পথ ঠিক আছে কি না, ডেটাবেস আপডেট হয়েছে কি না, আর স্ক্যানের সুযোগে (scope) কিছু বাদ পড়েছে কি না — দেখুন। পরিষ্কার ফলাফল যাচাই করুন, উদযাপন নয়।

> **TROUBLESHOOTING:** দুর্বলতা নিয়ে বিভ্রান্তি হলে তিনটি প্রশ্নে ফিরে যান: (১) দুর্বলতাটি কোন উপাদানে, কোন সংস্করণে? (২) আমাদের পরিবেশে সেটি কি আদৌ শোষণযোগ্য? (৩) ঠিক করার সবচেয়ে নিরাপদ দ্রুত পথ কোনটি? এই তিন প্রশ্নের উত্তর না পাওয়া পর্যন্ত প্যাচ বা ব্যতিক্রম — কোনো সিদ্ধান্তই নেবেন না।

১৩.১১ সারসংক্ষেপ

দুর্বলতা ব্যবস্থাপনা হলো আবিষ্কার, যাচাই, ঝুঁকি-ভিত্তিক অগ্রাধিকার, প্রতিকার, পুনঃযাচাই ও প্রতিবেদনের একটি নিরন্তর চক্র। CVE পরিচয় দেয়, CVSS অন্তর্নিহিত তীব্রতা মাপে — ভিত্তি মেট্রিক ও তীব্রতা স্তর (None, Low, Medium, High, Critical) দিয়ে — কিন্তু প্রকৃত ঝুঁকি নির্ধারণ করে আপনার পরিবেশের প্রসঙ্গ: উন্মুক্ততা, শোষণযোগ্যতা ও প্রভাব। ভালো ব্যবস্থাপনা মানে শূন্য দুর্বলতা নয়; মানে প্রতিটি দুর্বলতার মালিক, সময়সীমা ও যাচাই থাকা।

১৩.১২ পর্যালোচনা প্রশ্ন

1. দুর্বলতা জীবনচক্রের ধাপগুলো কী কী, এবং প্রতিটি ধাপে ব্যর্থতা কীভাবে ঘটতে পারে?
2. CVE ও CVSS-এর মধ্যে পার্থক্য কী? CVSS-এর ভিত্তি মেট্রিকগুলো কী কী?
3. CVSS তীব্রতা স্তরগুলো (None, Low, Medium, High, Critical) কোন স্কোর পরিসরে পড়ে?
4. কেন শুধু CVSS স্কোর দেখে অগ্রাধিকার দেওয়া ভুল? Reachability ও exposure কীভাবে সিদ্ধান্ত বদলায়?
5. ঝুঁকি = হুমকি × দুর্বলতা × প্রভাব — এই সূত্রটি অগ্রাধিকারে কীভাবে প্রয়োগ করবেন?
6. প্যাচ ছাড়া প্রতিকারের পথগুলো কী কী? ক্ষতিপূরণমূলক নিয়ন্ত্রণ বলতে কী বোঝায়?

7. ব্যতিক্রম নথিতে কী কী থাকা উচিত, এবং মেয়াদ কেন অপরিহার্য?
8. প্রতিকার SLA কী, এবং SLA বারবার ভাঙলে কীভাবে নির্ণয় করবেন?
9. স্ক্যান রিপোর্ট কেন সংবেদনশীল তথ্য, এবং কীভাবে তা সুরক্ষিত রাখবেন?
10. একই দুর্বলতা বারবার ফিরে এলে কী কী কারণ থাকতে পারে?

১৩.১৩ ব্যবহারিক অনুশীলন

1. **অগ্রাধিকার অনুশীলন:** নিজের ল্যাব অ্যাপে `npm audit` চালিয়ে ১০টি দুর্বলতার তালিকা বানান; প্রতিটির CVSS স্তর, reachability ও exposure বিচার করে P1–P4-এ সাজান, এবং আপনার যুক্তি লিখুন।
2. **প্যাচ চক্র:** একটি High দুর্বলতা বেছে নিয়ে পরীক্ষার পরিবেশে প্যাচ করুন, smoke test চালান, পুনঃস্ক্যান করে যাচাই করুন, এবং পুরো চক্রটি নথিভুক্ত করুন।
3. **ব্যতিক্রম নথি:** একটি দুর্বলতার জন্য সম্পূর্ণ ব্যতিক্রম নথি লিখুন — ঝুঁকি বিশ্লেষণ, ক্ষতিপূরণমূলক নিয়ন্ত্রণ, মালিক ও মেয়াদসহ।
4. **প্রতিবেদন লেখা:** এক সপ্তাহের স্ক্যান ফলাফলের ভিত্তিতে এক পৃষ্ঠার প্রতিবেদন লিখুন — ব্যবস্থাপনার জন্য সারসংক্ষেপ ও প্রযুক্তিগত দলের জন্য বিস্তারিত, দুটোই।

অধ্যায় ১৪ • PART III — DevSecOps Automation & Governance

Chapter 14 — Cloud Security Posture

১৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — ক্লাউড নিরাপত্তা ভঙ্গি (cloud security posture) বলতে কী বোঝায়, তা ব্যাখ্যা করতে; ক্লাউড IAM নিরাপত্তা, নেটওয়ার্ক উন্মুক্ততা, স্টোরেজ নিরাপত্তা ও Kubernetes নিরাপত্তা ভঙ্গির মূল ঝুঁকিগুলো চিহ্নিত করতে; ক্লাউড কনফিগারেশন নিরীক্ষা (auditing) ও CSPM (Cloud Security Posture Management)-এর ধারণা ও কাজের পদ্ধতি বোঝাতে; নিরাপত্তা অনুসন্ধান (security findings) পড়ে অগ্রাধিকার দিতে; ক্রমাগত সম্মতি (continuous compliance) বজায় রাখার চক্র তৈরি করতে; এবং AWS, Azure, GCP ও IBM Cloud-এর নিরাপত্তা মডেলের তুলনা করতে — সেবার ক্ষমতা বা মূল্য উদ্ভাবন না করে, শুধু সাধারণভাবে নথিভুক্ত ধারণার ভিত্তিতে।

১৪.২ পূর্বশর্ত পুনরালোচনা

এই অধ্যায়ে ক্লাউডের মৌলিক ধারণা — ভার্চুয়াল মেশিন, স্টোরেজ বাকেট, ভার্চুয়াল নেটওয়ার্ক, IAM ভূমিকা ও নীতি — আপনার চেনা থাকা দরকার। আগের অধ্যায়গুলো থেকে মনে রাখুন: Policy as Code-এর নীতি, OIDC-র স্বল্পমেয়াদি শংসাপত্র, আর দুর্বলতা ব্যবস্থাপনার চক্র। ক্লাউডে সেই একই ধারণাগুলো বৃহত্তর পরিসরে প্রয়োগ হয়: শত শত সম্পদ, বহু দল, আর প্রতিনিয়ত বদলানো কনফিগারেশন।

সবচেয়ে গুরুত্বপূর্ণ ভিত্তি ধারণাটি হলো যৌথ দায়িত্ব মডেল (shared responsibility model): ক্লাউড প্রদানকারী (provider) ক্লাউডের নিরাপত্তার দায়িত্ব নেয় — ডেটা সেন্টার, হার্ডওয়্যার, হাইপারভাইজার; আর গ্রাহক ক্লাউডে যা রাখে, তার নিরাপত্তার দায়িত্ব নেয় — নিজের ডেটা, নিজের কনফিগারেশন, নিজের পরিচয় ও প্রবেশাধিকার ব্যবস্থাপনা। এই সীমারেখা সেবা অনুযায়ী বদলায়: IaaS-এ গ্রাহকের দায়িত্ব বেশি, SaaS-এ কম। কিন্তু একটি সত্য সব সেবায় এক: ভুল কনফিগারেশনের দায় গ্রাহকের। ক্লাউডে বেশিরভাগ ঘটনা হ্যাকিংয়ের চেয়ে ভুল কনফিগারেশন থেকে ঘটে — পাবলিক বাকেট, খোলা পোর্ট, অতিরিক্ত অধিকার।

১৪.৩ প্রথম নীতি থেকে বোঝা: ক্লাউডে ভুল কনফিগারেশন কেন প্রধান ঝুঁকি

প্রথম নীতি থেকে শুরু করা যাক। ঐতিহ্যবাহী ডেটা সেন্টারে নিরাপত্তা ছিল শারীরিক ও নেটওয়ার্ক-কেন্দ্রিক: ফায়ারওয়াল, সার্ভার রুমের তালা। ক্লাউডে নিরাপত্তা মূলত API-কেন্দ্রিক: প্রতিটি সম্পদ একটি API কলের মাধ্যমে তৈরি হয়, আর প্রতিটি কনফিগারেশন একটি ঘোষণা। এর অর্থ, একটি ভুল API কল বা একটি ভুল নীতির লাইন মুহূর্তে একটি স্টোরেজ বাকেটকে বিশ্বের কাছে খুলে দিতে পারে — কোনো শারীরিক বাধা নেই। আর ক্লাউডের গতি এই ঝুঁকিকে বহুগুণ করে: Infrastructure as Code দিয়ে শত শত সম্পদ মিনিটে তৈরি হয়; ভুল কনফিগারেশনও একই গতিতে ছড়ায়।

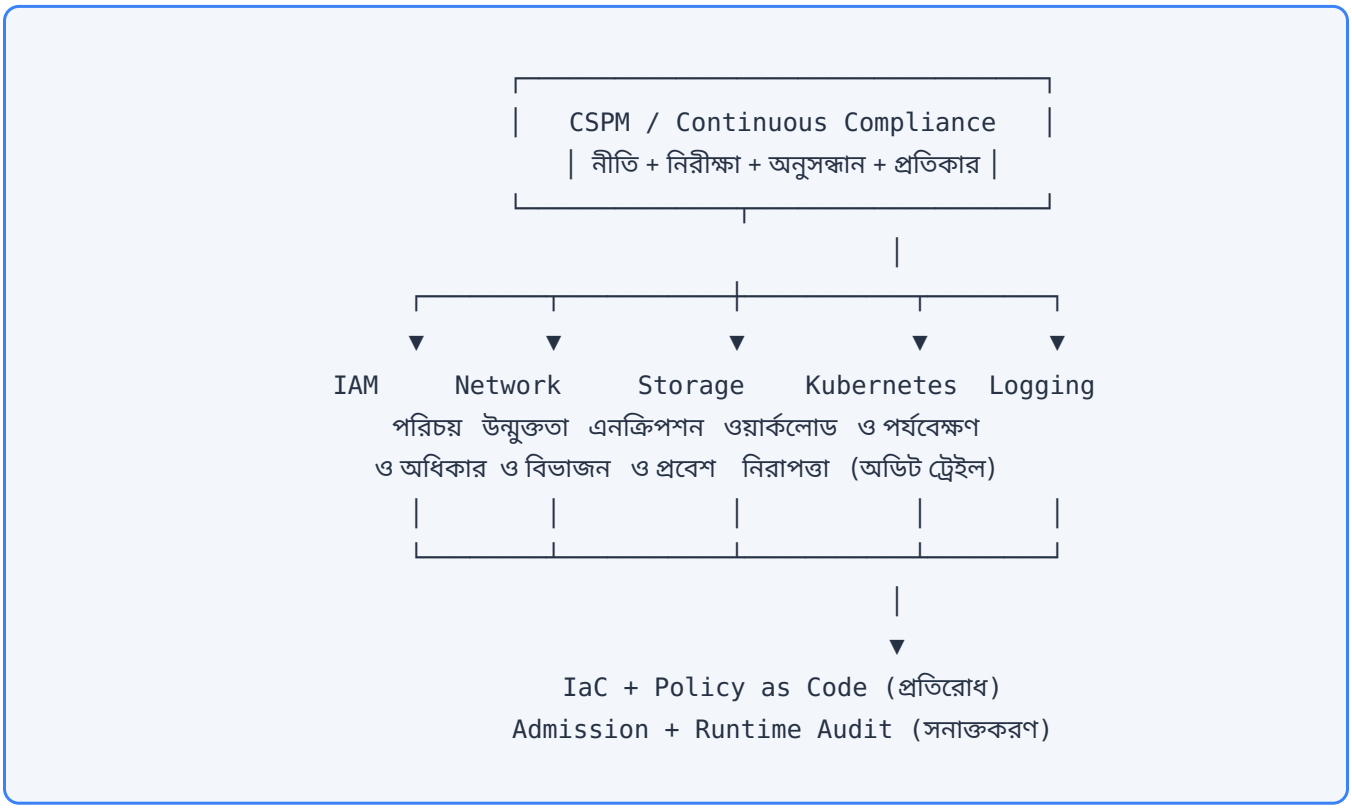
সমস্যাটি কেন ঘটে? চারটি কারণ। প্রথমত, ক্লাউডের ডিফল্ট কখনো কখনো সুবিধার পক্ষে থাকে, নিরাপত্তার পক্ষে নয় — যেমন নতুন বাকেটে পাবলিক প্রবেশাধিকার বন্ধ থাকলেও, একটি ভুল নীতি তা খুলে দিতে পারে। দ্বিতীয়ত, কনফিগারেশন ছড়িয়ে থাকে: কনসোল, Terraform, CLI, অ্যাপ কোড — এক জায়গায় ভুল করলে অন্য জায়গা তা জানে না। তৃতীয়ত, অধিকার জমে যায়: দলগুলো প্রয়োজনে অধিকার নেয়, প্রয়োজন ফুরালে ফেরত দেয় না — ফলে প্রতিটি পরিচয়ের হাতে প্রয়োজনের চেয়ে বেশি ক্ষমতা জমে। চতুর্থত, পরিবর্তন (drift): Terraform-এ যা লেখা, বাস্তবে তা নাও থাকতে পারে — কেউ কনসোল থেকে হাতে বদলে দিয়েছে।

এই সমস্যার সমাধানের মূলনীতিও চারটি। এক, ডিফল্ট-নিরাপদ (secure by default): নতুন সম্পদ তৈরির সময়ই নিরাপদ কনফিগারেশন বাধ্যতামূলক — যেমন এনক্রিপশন চালু, পাবলিক প্রবেশাধিকার বন্ধ। দুই, সর্বনিম্ন অধিকার ও তার নিয়মিত পর্যালোচনা: অধিকার শুধু প্রয়োজনে, আর ব্যবহার না হলে ফেরত। তিন, ক্রমাগত নিরীক্ষা: বাস্তব অবস্থাকে নীতির বিরুদ্ধে

নিয়মিত মেলানো — এটাই CSPM-এর কাজ। চার, প্রতিকার স্বয়ংক্রিয়করণ: ভুল ধরা পড়লে হয় সতর্কবার্তা, নয় স্বয়ংক্রিয় সংশোধন — তবে স্বয়ংক্রিয় সংশোধন সাবধানে, কারণ ভুল সংশোধন সেবা বন্ধ করে দিতে পারে।

> **CONCEPT:** CSPM (Cloud Security Posture Management) হলো ক্লাউডের বাস্তব কনফিগারেশনকে নীতি ও ভালো অনুশীলনের (best practice) বিরুদ্ধে ক্রমাগত পরীক্ষা করার শৃঙ্খলা ও টুলশ্রেণি। CSPM API-র মাধ্যমে ক্লাউড অ্যাকাউন্ট পড়ে — যেমন কোন বাকেট পাবলিক, কোন নিরাপত্তা দল (security group) 0.0.0.0/0-তে খোলা, কোন IAM ব্যবহারকারীর MFA নেই — এবং সেগুলোকে অনুসন্ধান (finding) হিসেবে সাজায়। CSPM প্রতিকার করে না নিজে; এটি দেখায়, অগ্রাধিকার দেয়, আর কিছু ক্ষেত্রে স্বয়ংক্রিয় প্রতিকারের পথ খুলে দেয়। CSPM-কে স্ক্যানারের সঙ্গে গুলিয়ে ফেলবেন না: স্ক্যানার দুর্বলতা খোঁজে কোড ও ইমেজে; CSPM কনফিগারেশন খোঁজে ক্লাউড সম্পদে।

১৪.৪ স্থাপত্য: ক্লাউড নিরাপত্তা ভঙ্গির চার স্তম্ভ



IAM নিরাপত্তা: পরিচয় (identity) হলো ক্লাউডের নতুন সীমান্ত (perimeter)। মূল প্রশ্নগুলো: কার কী অধিকার আছে? অধিকারটি কি প্রয়োজনের চেয়ে বেশি? MFA কি চালু আছে? দীর্ঘমেয়াদি চাবি কি ঘুরছে? ভূমিকা (role) কি শুধু নির্দিষ্ট কাজের জন্য সীমিত, নাকি `*` দিয়ে সবকিছু খোলা? TechNova-র নিয়ম: মানুষের জন্য MFA বাধ্যতামূলক, মেশিনের জন্য OIDC-ভিত্তিক স্বল্পমেয়াদি ভূমিকা, আর কোনো স্থায়ী চাবি নয়। IAM-এর সবচেয়ে বিপজ্জনক ভুল হলো অতিরিক্ত অধিকারের ভূমিকা (over-privileged role) — একটি আপস হওয়া ভূমিকা দিয়ে আক্রমণকারী পুরো অ্যাকাউন্ট দখল করতে পারে।

নেটওয়ার্ক উন্মুক্ততা: কোন পোর্ট ইন্টারনেটের দিকে খোলা? ডেটাবেস কি পাবলিক IP-তে শুনছে, নাকি শুধু ব্যক্তিগত নেটওয়ার্কে? নিরাপত্তা দল (security group) কি 0.0.0.0/0-তে SSH (পোর্ট ২২) খুলে রেখেছে? TechNova-র MERN অ্যাপে শুধু HTTPS (৪৪৩) পাবলিক; MongoDB 203.0.113.30 শুধু অ্যাপ সার্ভারের ব্যক্তিগত নেটওয়ার্ক থেকে পৌঁছানো যায়। নেটওয়ার্ক বিভাজন (segmentation) — পাবলিক, অ্যাপ ও ডেটা স্তর আলাদা সাবনেটে — এই নীতিরই প্রয়োগ।

স্টোরেজ নিরাপত্তা: বাকেট পাবলিক কি না, এনক্রিপশন চালু কি না, সংস্করণ (versioning) ও লগিং চালু কি না, আর কে কী পড়তে পারে? স্টোরেজের সবচেয়ে সাধারণ বিপর্যয় হলো পাবলিক বাকেটে সংবেদনশীল ডেটা। প্রতিরোধ: সংস্থা-ব্যাপী পাবলিক প্রবেশাধিকার বন্ধের নীতি, বাকেট নীতির পর্যালোচনা, আর নিয়মিত CSPM নিরীক্ষা।

Kubernetes নিরাপত্তা ভঙ্গি: ক্লাস্টারে কে কী করতে পারে (RBAC), Pod-গুলো কি নীতিমালা মানে (আগের অধ্যায়ের Kyverno), নোডের OS ও kubelet কি আপডেট, সিক্রেট কীভাবে সংরক্ষিত, আর নেটওয়ার্ক নীতি (NetworkPolicy) দিয়ে Pod-এর মধ্যে যোগাযোগ সীমিত কি না? Kubernetes-এর ড্যাশবোর্ড বা API সার্ভার ইন্টারনেটে খোলা থাকা মারাত্মক ভুল।

স্তম্ভ	সাধারণ ভুল	সঠিক অনুশীলন
IAM	অতিরিক্ত অধিকার, MFA নেই, স্থায়ী চাবি	সর্বনিম্ন অধিকার, MFA, OIDC স্বল্পমেয়াদি ভূমিকা
Network	0.0.0.0/0-তে SSH/ডেটাবেস খোলা	শুধু ৪৪৩ পাবলিক, বিভাজন, ব্যক্তিগত সাবনেট
Storage	পাবলিক বাকেট, এনক্রিপশন বন্ধ	পাবলিক বন্ধ, এনক্রিপশন চালু, লগিং চালু
Kubernetes	root Pod, খোলা RBAC, NetworkPolicy নেই	নীতি, RBAC পর্যালোচনা, NetworkPolicy
Logging	অডিট লগ বন্ধ বা কেউ দেখে না	কেন্দ্রীয় লগ, সতর্কবার্তা, অপরিবর্তনীয় সংরক্ষণ

১৪.৫ অভ্যন্তরীণ কার্যপ্রণালী: CSPM কীভাবে কাজ করে ও অনুসন্ধান কীভাবে পড়তে হয়

CSPM-এর কাজের ধাপগুলো: প্রথমে সংযোগ — CSPM ক্লাউড অ্যাকাউন্টে শুধু-পড়া (read-only) ভূমিকা নিয়ে API-র মাধ্যমে সম্পদের তালিকা ও কনফিগারেশন সংগ্রহ করে। তারপর মূল্যায়ন — প্রতিটি সম্পদকে নীতি ও ভালো অনুশীলনের সংগ্রহের বিরুদ্ধে মেলানো হয় (যেমন CIS Benchmarks — ক্লাউড নিরাপত্তার জন্য সর্বজনীনভাবে পরিচিত পরীক্ষার সংগ্রহ)। তারপর অনুসন্ধান তৈরি — প্রতিটি লঙ্ঘন একটি finding, যার তীব্রতা, প্রভাবিত সম্পদ, প্রমাণ ও প্রতিকারের নির্দেশনা থাকে। তারপর অগ্রাধিকার — তীব্রতা, উন্মুক্ততা ও ডেটার সংবেদনশীলতা দেখে সাজানো। শেষে প্রতিকার — হয় মানুষ ঠিক করে, নয় স্বয়ংক্রিয় প্রতিকার (auto-remediation) চালু থাকে।

অনুসন্ধান (finding) পড়ার সময় তিনটি প্রশ্ন করুন: এটি কি সত্যিই ভুল, নাকি প্রসঙ্গের কারণে গ্রহণযোগ্য? (যেমন একটি পাবলিক বাকেট, যাতে শুধু পাবলিক ছবি আছে — সেটি নকশাগত সিদ্ধান্ত হতে পারে, কিন্তু নথিভুক্ত থাকতে হবে।) এটি কতটা উন্মুক্ত — ইন্টারনেট থেকে পৌঁছানো যায়, নাকি শুধু অভ্যন্তরীণ? আর এর প্রভাব কী — এতে কী ডেটা আছে, আক্রান্ত হলে কী ক্ষতি? একটি High finding, যা অভ্যন্তরীণ ও খালি বাকেটে, একটি Medium finding-এর চেয়ে কম জরুরি হতে পারে, যা পাবলিক ও গ্রাহকের ডেটাসহ।

ক্রমাগত সম্মতি (continuous compliance) মানে সম্মতিকে একটি বার্ষিক পরীক্ষা থেকে দৈনন্দিন চক্রে বদলানো: নীতি কোডে লেখা, প্রতিটি পরিবর্তনে যাচাই, drift নিয়মিত ধরা, আর প্রমাণ (evidence) স্বয়ংক্রিয়ভাবে সংগ্রহ করা। অডিটের সময় তখন আর হাতে স্ক্রিনশট জোগাড় করতে হয় না — CSPM ও পাইপলাইনের লগই প্রমাণ। এই ধারণাটি পরের অধ্যায়ে governance প্রসঙ্গে আরও বিস্তারিত আসবে।

ক্লাউড প্রদানকারীদের তুলনা করার সময় সতর্ক থাকুন: AWS, Azure, GCP ও IBM Cloud — প্রত্যেকেরই IAM, নেটওয়ার্ক, স্টোরেজ ও লগিংয়ের নিজস্ব সেবা ও নাম আছে, আর সেগুলো প্রতিনিয়ত বদলায়। এই বই কোনো সেবার নির্দিষ্ট ক্ষমতা বা মূল্য দাবি করে না — সেগুলো যাচাই করতে হলে সংশ্লিষ্ট প্রদানকারীর সরকারি নথি দেখুন। তুলনার সঠিক স্তর হলো ধারণাগত: চারটি প্রদানকারীতেই যৌথ দায়িত্ব মডেল আছে; চারটিতেই পরিচয়-ভিত্তিক প্রবেশাধিকার, নেটওয়ার্ক বিভাজন, এনক্রিপশন ও অডিট লগের ধারণা আছে; পার্থক্য হলো নাম, ডিফল্ট আচরণ ও পরিচালনার পদ্ধতিতে। মাল্টি-ক্লাউডে কাজ

করলে এই ধারণাগত মিলটিই আপনার ভরসা — প্রতিটি ক্লাউডে একই নীতি (যেমন "কোনো স্টোরেজ পাবলিক নয়") ভিন্ন ভাষায় লিখতে হয়, কিন্তু নীতির যুক্তি একই থাকে।

> **WARNING:** CSPM-এর স্বয়ংক্রিয় প্রতিকার (auto-remediation) সাবধানে চালু করুন। একটি ভুল স্বয়ংক্রিয় নিয়ম প্রোডাকশন বাকেটের প্রবেশাধিকার বন্ধ করে সেবা থামিয়ে দিতে পারে। প্রথমে শুধু সতর্কবার্তা মোডে চালান, প্রতিকারের প্রভাব মাপুন, তারপর শুধু নিরাপদ ও বিপরীতমুখী (reversible) প্রতিকার স্বয়ংক্রিয় করুন — যেমন পাবলিক প্রবেশাধিকার বন্ধ করা, যেখানে ভুল হলে আবার খোলা যায়।

১৪.৬ ব্যবহারিক কমান্ড ও কনফিগারেশন

নিচের কমান্ডগুলো ল্যাবের জন্য — TechNova-র নিজস্ব সার্ভার ও ল্যাব ক্লাউড অ্যাকাউন্টে অনুমোদিত পরীক্ষা হিসেবে চালান। অন্য কারো ক্লাউড অ্যাকাউন্ট স্ক্যান করা অনুমোদন ছাড়া কখনোই করবেন না।

```
# Ubuntu 24.04 ল্যাব — ক্লাউড ভঙ্গি পরীক্ষা (TechNova ল্যাব অ্যাকাউন্ট)

# ১. উন্মুক্ত পোর্ট ও সেবা দেখুন (নিজের সার্ভার 203.0.113.10)
sudo ss -tuln
sudo ufw status verbose

# ২. ফায়ারওয়াল: শুধু প্রয়োজনীয় পোর্ট খোলা রাখুন
sudo ufw allow 22/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status numbered

# ৩. Kubernetes RBAC পর্যালোচনা (ল্যাব ক্লাস্টার)
kubectl get clusterroles --no-headers | head -n 20
kubectl get clusterrolebindings -o wide
kubectl auth can-i --list --as=system:serviceaccount:default:web-sa

# ৪. Pod নিরাপত্তা প্রসঙ্গ দেখুন
kubectl get pods -A -o jsonpath='{range .items[*]}{.metadata.name}{" "}
{.spec.containers[*].securityContext.runAsUser}{"\n"}{end}'

# ৫. NetworkPolicy আছে কি না দেখুন
kubectl get networkpolicy -A

# ৬. স্টোরেজ/বাকেট নীতি পর্যালোচনা (ল্যাব — প্রদানকারীর CLI দিয়ে)
# AWS ল্যাব উদাহরণ (শুধু নিজের অ্যাকাউন্টে):
# aws s3api get-public-access-block --bucket technova-lab-assets
# aws s3api get-bucket-encryption --bucket technova-lab-assets
# aws iam get-credential-report --query Content --output text | base64 -d

# ৭. Terraform-এ নিরাপদ ডিফল্টের উদাহরণ (নমুনা)
# resource-এ public_access_block ও encryption একসঙ্গে লিখুন —
# বিস্তারিত নমুনা ১২ অধ্যায়ের Terraform নীতি উদাহরণে দেখুন।
```

```
# নিরাপদ Terraform নমুনা — স্টোরেজ বাকেট (TechNova ল্যাব, নমুনা)
# terraform {
#   required_version = ">= 1.5"
# }
# resource "aws_s3_bucket" "assets" {
```

```
#   bucket = "technova-lab-assets"
# }
# resource "aws_s3_bucket_public_access_block" "assets" {
#   bucket                  = aws_s3_bucket.assets.id
#   block_public_acls       = true
#   block_public_policy     = true
#   ignore_public_acls      = true
#   restrict_public_buckets = true
# }
# resource "aws_s3_bucket_server_side_encryption_configuration" "assets" {
#   bucket = aws_s3_bucket.assets.id
#   rule {
#     apply_server_side_encryption_by_default {
#       sse_algorithm = "AES256"
#     }
#   }
# }
# দ্রষ্টব্য: এটি একটি নমুনা; বাস্তব অ্যাকাউন্টে বসানোর আগে
# প্রদানকারীর বর্তমান নথি ও নিজের নীতি মিলিয়ে নিন।
```

CSPM অনুসন্ধান পড়ার একটি নমুনা চেকলিস্ট: অনুসন্ধানটি কোন সম্পদে? তীব্রতা কী? প্রমাণ কী (কোন কনফিগারেশন লাইন)? উন্মুক্ততা কতটা? ডেটার সংবেদনশীলতা কী? প্রতিকারের নির্দেশনা কী, এবং তা কি বিপরীতমুখী? এই ছয় প্রশ্নের উত্তর ছাড়া কোনো অনুসন্ধানে হাত দেবেন না — নইলে ভুল সম্পদে ভুল প্রতিকার হবে।

> **REAL WORLD:** TechNova-তে CSPM-এ প্রথম সপ্তাহে ২০০-র বেশি অনুসন্ধান এসেছিল — বেশিরভাগই ল্যাবের পুরোনো বাকেট ও পরীক্ষামূলক ভূমিকা। দলটি প্রথমে ভয় পেয়েছিল, তারপর শিখেছিল: অনুসন্ধানকে সম্পদের মালিক ও পরিবেশ অনুযায়ী ভাগ করা, ল্যাবের অনুসন্ধানকে আলাদা সারিতে রাখা, আর প্রোডাকশনের পাবলিক-উন্মুক্ত অনুসন্ধানগুলোকে প্রথমে ঠিক করা। এক মাসে প্রোডাকশনের গুরুতর অনুসন্ধান শূন্যে নেমেছিল — ল্যাবেরগুলো তখনো ছিল, কিন্তু সেগুলো জানা ও গ্রহণ করা ঝুঁকি হিসেবে নথিভুক্ত ছিল।

ক্লাউড কনফিগারেশন নিরীক্ষা (auditing) শুধু CSPM টুল চালানো নয় — এটি একটি নিয়মিত শৃঙ্খলা। TechNova-তে নিরীক্ষার চক্রটি এরকম: প্রতিদিন স্বয়ংক্রিয় CSPM স্ক্যান, প্রতি সপ্তাহে নতুন অনুসন্ধানের পর্যালোচনা, প্রতি মাসে IAM অধিকারের পর্যালোচনা, আর প্রতি ত্রৈমাসিকে পূর্ণাঙ্গ ভঙ্গি পর্যালোচনা — যেখানে প্রতিটি স্তম্ভের (IAM, নেটওয়ার্ক, স্টোরেজ, Kubernetes, লগিং) অবস্থা, প্রবণতা ও ব্যতিক্রম দেখা হয়। নিরীক্ষার সময় লগের ভূমিকাও অপরিহার্য: ক্লাউডের অডিট ট্রেইল (যেমন API কলের লগ) দেখায় কে কখন কী বদলেছে — drift-এর মূল কারণ খুঁজতে এই লগই সবচেয়ে নির্ভরযোগ্য উৎস। লগ ছাড়া নিরীক্ষা শুধু বর্তমান অবস্থা দেখায়; লগসহ নিরীক্ষা দেখায় অবস্থা কীভাবে বদলেছে, আর কে বদলেছে। আরেকটি গুরুত্বপূর্ণ অভ্যাস: নিরীক্ষার ফলাফলকে শাস্তি হিসেবে নয়, শেখার উপাদান হিসেবে দেখা। একটি দল যদি ভুল কনফিগারেশনের জন্য শাস্তি পায়, তাহলে পরের বার তারা ভুল লুকাবে; যদি ভুল থেকে শেখার সংস্কৃতি থাকে, তাহলে তারা ভুল রিপোর্ট করবে ও ঠিক করবে। এই সংস্কৃতি ছাড়া সেরা CSPM টুলও ব্যর্থ হয়, কারণ মানুষই তখন নিরীক্ষার বাইরে কাজ করতে শুরু করে।

১৪.৭ প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — পাবলিক বাকেট আবিষ্কার। CSPM সতর্ক করল: একটি বাকেট পাবলিক, আর তাতে গ্রাহকের আপলোড আছে। প্রতিক্রিয়া: প্রথমে প্রবেশাধিকার বন্ধ করুন (স্বয়ংক্রিয় প্রতিকার থাকলে সেটিই করবে), তারপর লগ দেখুন কেউ ডেটা পড়েছে কি না, তারপর বাকেটটি কেন পাবলিক হয়েছিল তা খুঁজুন — Terraform-এর ভুল, নাকি হাতে বদলানো? — আর মূল কারণ ঠিক করুন। শুধু বন্ধ করলেই হবে না; কারণ না ঠিক করলে আবার খুলবে।

পরিস্থিতি ২ — অতিরিক্ত অধিকারের ভূমিকা। একটি CI ভূমিকার হাতে পুরো অ্যাকাউন্টের অধিকার — কারণ একসময় ডেপ্লয়ের জন্য সবকিছু দরকার হয়েছিল। প্রতিক্রিয়া: গত ৯০ দিনের ব্যবহারের লগ (access advisor) দেখে কোন অধিকার আসলে ব্যবহৃত হয়েছে, তা বের করুন; ভূমিকাকে শুধু সেই অধিকারগুলোতে সীমিত করুন; আর OIDC-র মাধ্যমে শাখা ও পরিবেশ অনুযায়ী শর্ত যোগ করুন।

পরিস্থিতি ৩ — মাল্টি-ক্লাউড drift। Terraform বলে বাকেট ব্যক্তিগত, কিন্তু বাস্তবে পাবলিক — কেউ কনসোল থেকে বদলেছে। প্রতিক্রিয়া: drift সনাক্তকরণ চালু করুন, কনসোলে হাতে পরিবর্তন নিষিদ্ধ করুন (বা সীমিত করুন), আর নিয়মিত `terraform plan`-এ drift দেখুন। মানুষের হাতে পরিবর্তন বন্ধ না করলে IaC শুধু কাগজে থাকে।

পরিস্থিতি ৪ — Kubernetes API সার্ভার উন্মুক্ত। স্ক্যানে দেখা গেল ক্লাস্টারের API এন্ডপয়েন্ট ইন্টারনেট থেকে পৌঁছানো যায়। এটি জরুরি: API-কে ব্যক্তিগত নেটওয়ার্কে সীমিত করুন, প্রমাণীকরণ ও অনুমোদন পর্যালোচনা করুন, আর লগে অননুমোদিত চেষ্টা আছে কি না দেখুন।

১৪.৮ নিরাপত্তা ও খরচ বিবেচনা

CSPM-এর খরচ: টুলের খরচ (ওপেন সোর্স বিকল্পও আছে, যেমন Prowler, CloudSploit-ধরনের টুল), API কলের খরচ, আর অনুসন্ধান ঠিক করার সময়। খরচ কমানোর উপায়: শুধু প্রোডাকশন অ্যাকাউন্টে কঠোর নীতি, ল্যাবে হালকা; অনুসন্ধানকে পরিবেশ ও মালিক অনুযায়ী ভাগ করে মনোযোগ কেন্দ্রীভূত করা; আর IaC-তে নিরাপদ ডিফল্ট বসিয়ে অনুসন্ধান তৈরি হওয়াই কমানো — প্রতিকার করার চেয়ে প্রতিরোধ সস্তা।

নিরাপত্তা দিক: CSPM-এর নিজের প্রবেশাধিকার সীমিত রাখুন — শুধু-পড়া ভূমিকা, আর সেই ভূমিকার শংসাপত্র সুরক্ষিত রাখুন। CSPM-এর রিপোর্টও সংবেদনশীল — এতে আপনার ক্লাউডের দুর্বলতার মানচিত্র থাকে। আর মনে রাখবেন, CSPM শুধু কনফিগারেশন দেখে; অ্যাপের দুর্বলতা, নেটওয়ার্ক আক্রমণ বা পরিচয়ের আপস — এগুলো ধরতে পর্যবেক্ষণ ও সনাক্তকরণের (detection) অন্য স্তর দরকার। প্রতিরক্ষার গভীরতা এখানেও প্রযোজ্য।

প্রদানকারীদের তুলনা প্রসঙ্গে একটি সতর্কতা: কোনো প্রদানকারীর সেবার নাম, ক্ষমতা বা মূল্য এই বইয়ে দাবি করা হয়নি, কারণ সেগুলো বদলায়। বাস্তব সিদ্ধান্ত নেওয়ার সময় প্রতিটি প্রদানকারীর সরকারি নথি, মূল্য ক্যালকুলেটর ও পরীক্ষামূলক (trial) পরিবেশ দেখুন। ধারণাগত তুলনাই টেকসই: যৌথ দায়িত্ব, সর্বনিম্ন অধিকার, এনক্রিপশন, লগিং — এই নীতিগুলো চারটি প্রদানকারীতেই একইভাবে প্রযোজ্য।

১৪.৯ সাধারণ ভুল

- **ডিফল্টকে নিরাপদ ধরে নেওয়া:** নতুন সম্পদ মানেই নিরাপদ নয়; প্রতিটি কনফিগারেশন যাচাই করুন।
- **0.0.0.0/0-তে সব খুলে দেওয়া:** পরীক্ষার সুবিধার জন্য খোলা পোর্ট প্রোডাকশনে থেকে যাওয়া।
- **অতিরিক্ত অধিকারের ভূমিকা:** `*` অধিকার দিয়ে শুরু করা, আর কখনো কমানো না।
- **MFA ছাড়া মানব-পরিচয়:** শুধু পাসওয়ার্ডে ক্লাউড অ্যাকাউন্ট — ফিশিংয়ের এক ক্লিকেই দখল।
- **Drift উপেক্ষা করা:** Terraform-এ লেখা আর বাস্তবের মধ্যে পার্থক্য না দেখা।
- **CSPM অনুসন্ধানকে শুধু সংখ্যা ভাবা:** ২০০ অনুসন্ধান মানেই ২০০ সমান ঝুঁকি নয়; প্রসঙ্গ দেখুন।
- **স্বয়ংক্রিয় প্রতিকার অন্ধভাবে চালু করা:** ভুল প্রতিকার সেবা বন্ধ করে দিতে পারে।
- **প্রদানকারীর সেবার ক্ষমতা বা মূল্য অনুমান করা:** সবসময় সরকারি নথি যাচাই করুন।

১৪.১০ সমস্যা নির্ণয়

সমস্যা: CSPM একটি বাকেটকে পাবলিক বলছে, কিন্তু Terraform বলছে ব্যক্তিগত। নির্ণয়: বাকেট নীতি, ACL ও Public Access Block — তিনটি স্তরই দেখুন; কোনো একটি স্তরে পাবলিক অনুমতি থাকলেই বাকেট উন্মুক্ত হতে পারে; আর drift আছে কি না দেখতে `terraform plan` চালান।

সমস্যা: একটি ভূমিকা কাজ করছে না — "AccessDenied"। নির্ণয়: ভূমিকার নীতি, ট্রাস্ট পলিসি ও শর্ত (condition) — তিনটি দেখুন; ক্লাউডের Access Advisor বা Policy Simulator দিয়ে কোন অধিকার অনুপস্থিত, তা বের করুন; আর দেখুন অনুরোধটি কোন অঞ্চল (region) থেকে আসছে — অঞ্চল-ভিত্তিক শর্তও থাকতে পারে।

সমস্যা: Kubernetes Pod নেটওয়ার্কে পৌঁছাতে পারছে না। নির্ণয়: NetworkPolicy আছে কি না দেখুন — একটি ভুল NetworkPolicy সব যোগাযোগ বন্ধ করে দিতে পারে; `kubectl describe networkpolicy` ও Pod-এর লেবেল মিলিয়ে দেখুন; আর DNS ও Service-এর কনফিগারেশন যাচাই করুন।

সমস্যা: CSPM অনুসন্ধান হঠাৎ বেড়ে গেছে। নির্ণয়: নতুন নীতি যোগ হয়েছে কি না, নাকি নতুন সম্পদ তৈরি হয়েছে? নীতির পরিবর্তন হলে অনুসন্ধান বাড়া স্বাভাবিক; সম্পদের পরিবর্তন হলে সেই সম্পদের মালিককে জানান। সংখ্যা দেখে আতঙ্কিত হবেন না — পরিবর্তনের কারণ খুঁজুন।

> **TROUBLESHOOTING:** ক্লাউড সমস্যা নির্ণয়ের সময় সবসময় বাস্তব অবস্থা দেখুন, কাগজের অবস্থা নয়: Terraform-এ যা লেখা, কনসোলে যা দেখা যায়, আর API যা ফেরত দেয় — তিনটি মিলিয়ে দেখুন। তিনটির মধ্যে অমিল থাকলে সেটিই drift, আর drift-ই বেশিরভাগ ক্লাউড ঘটনার মূল।

১৪.১১ সারসংক্ষেপ

ক্লাউড নিরাপত্তা ভঙ্গি মানে কনফিগারেশনের নিরাপত্তা: IAM-এ সর্বনিম্ন অধিকার, নেটওয়ার্কে বিভাজন, স্টোরেজে পাবলিক-বন্ধ ও এনক্রিপশন, Kubernetes-এ নীতি ও RBAC, আর সবকিছুর ওপরে CSPM দিয়ে ক্রমাগত নিরীক্ষা। যৌথ দায়িত্ব মডেলে ভুল কনফিগারেশনের দায় গ্রাহকের — তাই প্রতিরোধ (IaC-তে নিরাপদ ডিফল্ট), সনাক্তকরণ (CSPM) ও প্রতিকার (অগ্রাধিকারসহ) — এই তিনটি একসঙ্গে চালাতে হয়। AWS, Azure, GCP ও IBM Cloud-এর ধারণাগত মিলটিই মাল্টি-ক্লাউডে আপনার ভরসা; নির্দিষ্ট সেবার ক্ষমতা বা মূল্য সবসময় সরকারি নথি থেকে যাচাই করুন।

১৪.১২ পর্যালোচনা প্রশ্ন

1. যৌথ দায়িত্ব মডেল বলতে কী বোঝায়, এবং IaaS বনাম SaaS-এ গ্রাহকের দায়িত্ব কীভাবে বদলায়?
2. ক্লাউডে ভুল কনফিগারেশন কেন প্রধান ঝুঁকি, এবং এর চারটি কারণ কী?
3. CSPM কী, এবং এটি স্ক্যানার থেকে কীভাবে আলাদা?
4. IAM-এ অতিরিক্ত অধিকারের ভূমিকা কেন বিপজ্জনক? Access Advisor কীভাবে সাহায্য করে?
5. নেটওয়ার্ক বিভাজন (segmentation) বলতে কী বোঝায়? TechNova-র তিন স্তরের উদাহরণ দিন।
6. একটি বাকেট পাবলিক কি না, তা নির্ধারণে কোন তিনটি স্তর দেখতে হয়?
7. Drift বলতে কী বোঝায়, এবং এটি কীভাবে সনাক্ত ও প্রতিরোধ করবেন?
8. CSPM অনুসন্ধান পড়ার সময় কোন ছয়টি প্রশ্ন করতে হয়?
9. স্বয়ংক্রিয় প্রতিকার সাবধানে চালু করা কেন দরকার?
10. AWS, Azure, GCP ও IBM Cloud-এর তুলনায় ধারণাগত মিল কী, এবং নির্দিষ্ট সেবার তথ্য কোথায় যাচাই করবেন?

১৪.১৩ ব্যবহারিক অনুশীলন

1. **ল্যাব নিরীক্ষা:** নিজের ল্যাব সার্ভারে `ss -tuln` ও `ufw status` চালিয়ে খোলা পোর্টের তালিকা বানান; প্রতিটি পোর্টের প্রয়োজনীয়তা লিখুন, অপ্রয়োজনীয় পোর্ট বন্ধ করুন।
2. **RBAC পর্যালোচনা:** ল্যাব ক্লাস্টারে ClusterRole ও ClusterRoleBinding-এর তালিকা দেখে একটি অতিরিক্ত অধিকারের ভূমিকা চিহ্নিত করুন এবং তাকে সর্বনিম্ন অধিকারে সীমিত করার প্রস্তাব লিখুন।
3. **CSPM অনুকরণ:** একটি Terraform কনফিগারেশনে ইচ্ছাকৃতভাবে একটি পাবলিক বাকেট ও একটি খোলা নিরাপত্তা দল যোগ করে OPA নীতি দিয়ে তা ধরুন; তারপর নিরাপদ কনফিগারেশনে বদলে আবার যাচাই করুন।
4. **তুলনা নথি:** AWS, Azure, GCP ও IBM Cloud-এর নিরাপত্তা ধারণা (IAM, নেটওয়ার্ক, স্টোরেজ, লগিং) নিয়ে এক পৃষ্ঠার ধারণাগত তুলনা লিখুন — কোনো সেবার ক্ষমতা বা মূল্য দাবি না করে, শুধু সরকারি নথির রেফারেন্সসহ।

অধ্যায় ১৫ • PART III — DevSecOps Automation & Governance

Chapter 15 — DevSecOps Governance and Compliance

১৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — নিরাপত্তা শাসন (security governance) বলতে কী বোঝায় এবং এটি দৈনন্দিন নিরাপত্তা কাজ থেকে কীভাবে আলাদা, তা ব্যাখ্যা করতে; ঝুঁকি ব্যবস্থাপনার চক্র — ঝুঁকি চিহ্নিত করা, মূল্যায়ন করা, প্রতিকার করা ও গ্রহণ করা — প্রয়োগ করতে; অডিট প্রমাণ (audit evidence) কী, কীভাবে সংগ্রহ করতে হয়, এবং কেন স্বয়ংক্রিয় প্রমাণ হাতে-তৈরি প্রমাণের চেয়ে শক্তিশালী, তা বলতে; নীতি প্রয়োগ (policy enforcement) ও পরিবর্তন ব্যবস্থাপনার (change management) নকশা করতে; সম্মতি (compliance) ধারণা, ডেটা সুরক্ষা, প্রবেশাধিকার পর্যালোচনা (access review) ও নিরাপত্তা মেট্রিকের কাঠামো তৈরি করতে; এবং সাংগঠনিক নিরাপত্তা পরিচালনা (organizational security operations) — ভূমিকা, দায়িত্ব, সংস্কৃতি ও ক্রমাগত উন্নতি — ব্যাখ্যা করতে।

১৫.২ পূর্বশর্ত পুনরালোচনা

এই অধ্যায় আগের চার অধ্যায়ের ওপর দাঁড়িয়ে আছে। অধ্যায় ১১ থেকে: পাইপলাইনের গেট, স্বাক্ষর ও অনুমোদন। অধ্যায় ১২ থেকে: Policy as Code ও নীতি প্রয়োগ। অধ্যায় ১৩ থেকে: দুর্বলতা ব্যবস্থাপনা, SLA ও ব্যতিক্রম। অধ্যায় ১৪ থেকে: CSPM, ক্রমাগত সম্মতি ও drift। এই অধ্যায় সেই সব প্রযুক্তিগত নিয়ন্ত্রণকে একটি শাসন কাঠামোয় বাঁধে: কে সিদ্ধান্ত নেয়, কীভাবে সিদ্ধান্ত নথিভুক্ত হয়, কীভাবে প্রমাণ করা যায় যে নিয়ন্ত্রণগুলো সত্যিই কাজ করছে, আর কীভাবে সংস্থা সময়ের সঙ্গে উন্নত হয়।

TechNova-র প্রসঙ্গে ভাবুন: সংস্থাটি বড় হচ্ছে, গ্রাহকরা জিজ্ঞাসা করছে — "তোমাদের নিরাপত্তা ব্যবস্থা কী? আমাদের ডেটা কীভাবে সুরক্ষিত?" — আর একজন অডিটর আসছেন তিন মাস পর। দলের হাতে প্রযুক্তি আছে — পাইপলাইন, নীতি, স্ক্যানার — কিন্তু সেগুলো যে কাজ করছে, তার প্রমাণ ছড়িয়ে আছে লগে, চ্যাটে, মানুষের মাথায়। Governance হলো সেই প্রমাণকে সংগ্রহ, সাজানো ও দেখানোর শৃঙ্খলা — যাতে নিরাপত্তা শুধু করা না হয়, প্রমাণও করা যায়।

১৫.৩ প্রথম নীতি থেকে বোঝা: শাসন কেন প্রযুক্তি নয়, সিদ্ধান্তের কাঠামো

প্রথম নীতি থেকে শুরু করা যাক। Governance বলতে বোঝায়: কে কী সিদ্ধান্ত নেওয়ার অধিকার রাখে, সিদ্ধান্ত কীভাবে নেওয়া হয়, আর সিদ্ধান্তের জন্য কে দায়ী থাকে। এটি প্রযুক্তি নয়; এটি ক্ষমতা ও দায়িত্বের কাঠামো। একটি সংস্থায় নিরাপত্তা ব্যর্থ হয় শুধু প্রযুক্তির অভাবে নয় — ব্যর্থ হয় যখন কেউ জানে না কে ঝুঁকি গ্রহণের সিদ্ধান্ত নেবে, যখন ব্যতিক্রমের মালিক নেই, যখন অডিটের সময় প্রমাণ নেই, আর যখন একটি দল নিরাপত্তাকে অন্য দলের সমস্যা ভাবে।

ঝুঁকি ব্যবস্থাপনা এই কাঠামোর হৃদয়। ঝুঁকি হলো অনিশ্চয়তার মধ্যে ক্ষতির সম্ভাবনা। ঝুঁকি ব্যবস্থাপনার চক্র চার ধাপের: চিহ্নিত করা (কী ভুল হতে পারে?), মূল্যায়ন করা (কতটা সম্ভাব্য, কতটা ক্ষতিকর?), প্রতিকার করা (ঝুঁকি কমানো, স্থানান্তর করা, এড়ানো, নাকি গ্রহণ করা?), আর পর্যবেক্ষণ করা (ঝুঁকি কি বদলেছে?)। চারটি প্রতিকার-পথের পার্থক্য বোঝা গুরুত্বপূর্ণ: কমানো (mitigate — নিয়ন্ত্রণ বসিয়ে ঝুঁকি কমানো), স্থানান্তর (transfer — যেমন বীমা), এড়ানো (avoid — ঝুঁকিপূর্ণ কাজটি না করা), আর গ্রহণ (accept — সচেতনভাবে ঝুঁকি নেওয়া, নথিভুক্ত করে)। গ্রহণ করা মানে উপেক্ষা করা নয় — গ্রহণ করা মানে সিদ্ধান্ত নেওয়া, যার মালিক ও মেয়াদ আছে।

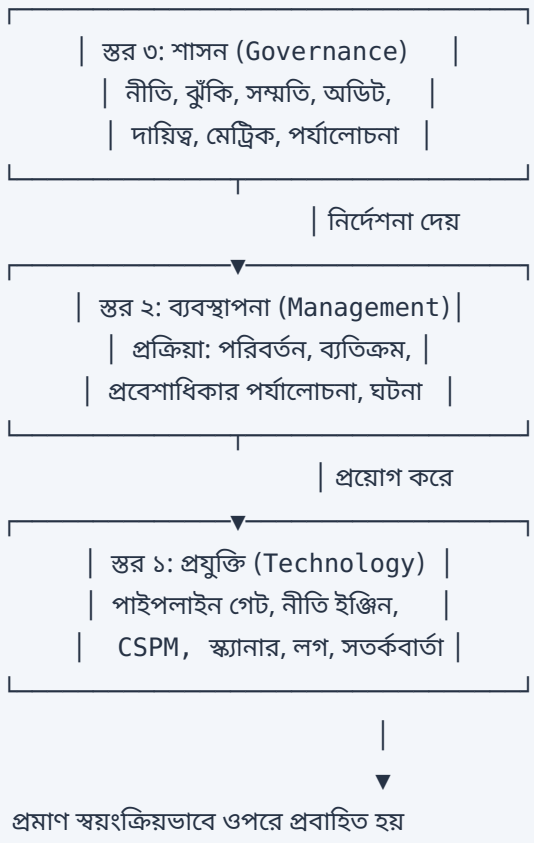
সম্মতি (compliance) বলতে বোঝায় প্রযোজ্য নিয়ম, মান ও চুক্তি মেনে চলা। সম্মতি নিজে নিরাপত্তা নয় — একটি সংস্থা সম্মতিপূর্ণ হয়েও অনিরাপদ হতে পারে, যদি নিয়মগুলো ন্যূনতম স্তরের হয়। কিন্তু সম্মতি একটি দরকারি কাঠামো দেয়: কী

করতে হবে, কী প্রমাণ রাখতে হবে, কে যাচাই করবে। এই বই কোনো নির্দিষ্ট মান বা আইনের বিস্তারিত ব্যাখ্যা দেয় না — সেগুলো প্রযোজ্য ক্ষেত্রে সরকারি উৎস ও আইনি পরামর্শ থেকে যাচাই করতে হয়। এখানে শেখানো হচ্ছে সম্মতির কাজের পদ্ধতি: নিয়ন্ত্রণকে নীতিতে, নীতিকে কোডে, আর কোডের ফলাফলকে প্রমাণে বদলানো।

> **CONCEPT:** অডিট প্রমাণ (audit evidence) হলো সেই রেকর্ড, যা দেখায় একটি নিয়ন্ত্রণ সত্যিই কাজ করেছে — যেমন পাইপলাইনের লগে দেখা যাচ্ছে স্ক্যান চলেছে ও পাস করেছে, অনুমোদনের রেকর্ডে দেখা যাচ্ছে কে কখন ডেপ্লয় অনুমোদন করেছেন, আর CSPM রিপোর্টে দেখা যাচ্ছে কোনো পাবলিক বাকেট নেই। ভালো প্রমাণের তিনটি গুণ: সম্পূর্ণতা (সব ঘটনা আছে কি না), অখণ্ডতা (বদলানো যায়নি তো?), আর খুঁজে পাওয়ার যোগ্যতা (প্রয়োজনে দ্রুত বের করা যায় কি না?)। হাতে বানানো স্ক্রিনশট দুর্বল প্রমাণ; স্বয়ংক্রিয়ভাবে সংগৃহীত, সময়-চিহ্নিত (timestamped) ও অপরিবর্তনীয় লগ শক্তিশালী প্রমাণ।

পরিবর্তন ব্যবস্থাপনা (change management) হলো শাসনের দৈনন্দিন প্রয়োগ: প্রোডাকশনে প্রতিটি পরিবর্তন — কোড, কনফিগারেশন, অধিকার, নীতি — অনুমোদিত, পরীক্ষিত, নথিভুক্ত ও বিপরীতমুখী (reversible) হতে হবে। DevOps-এ এর অর্থ এই নয় যে প্রতিটি পরিবর্তনে সপ্তাহব্যাপী কমিটির বৈঠক বসবে; অর্থ হলো পরিবর্তনের পথটি স্বয়ংক্রিয় ও নিরীক্ষাযোগ্য হবে: PR, পর্যালোচনা, স্বয়ংক্রিয় পরীক্ষা, অনুমোদন, ডেপ্লয়, পর্যবেক্ষণ, রোলব্যাক পরিকল্পনা — সবই লগে থাকে। জরুরি পরিবর্তনের পথও থাকবে, কিন্তু সেখানেও অনুমোদন ও পরবর্তী পর্যালোচনা থাকবে।

১৫.৪ স্থাপত্য: শাসনের তিন স্তর



এই তিন স্তরের সম্পর্ক বোঝা গুরুত্বপূর্ণ। প্রযুক্তি ছাড়া শাসন শুধু কাগজ; শাসন ছাড়া প্রযুক্তি দিকহীন — কেউ জানে না কোন নিয়ন্ত্রণ কেন আছে, কে তার মালিক, আর তা কাজ করছে কি না। মাঝের ব্যবস্থাপনা স্তরটি দুটিকে জোড়ে: প্রক্রিয়াগুলো — পরিবর্তন অনুমোদন, ব্যতিক্রম পরিচালনা, প্রবেশাধিকার পর্যালোচনা, ঘটনা প্রতিক্রিয়া — প্রযুক্তির ফলাফলকে শাসনের সিদ্ধান্তে রূপান্তর করে, আর শাসনের সিদ্ধান্তকে প্রযুক্তির কনফিগারেশনে।

ডেটা সুরক্ষা এই কাঠামোর একটি বিশেষ অংশ। প্রশ্নগুলো: কোন ডেটা সংবেদনশীল (গ্রাহকের ব্যক্তিগত তথ্য, পেমেন্ট তথ্য, গোপন তথ্য)? তা কোথায় থাকে, কে পড়তে পারে, কতদিন রাখা হয়, আর কীভাবে মুছে ফেলা হয়? নীতিগুলো: ডেটা শ্রেণিবিন্যাস (classification — পাবলিক, অভ্যন্তরীণ, গোপনীয়), এনক্রিপশন (স্থির অবস্থায় ও চলাচলে), প্রবেশাধিকার সর্বনিম্নকরণ, আর সংরক্ষণ ও মুছে ফেলার নিয়ম (retention & deletion)। TechNova-র MERN অ্যাপে গ্রাহকের পাসওয়ার্ড কখনো সরল পাঠ্যে সংরক্ষণ করা হয় না — শুধু হ্যাশ করা হয়; পেমেন্ট তথ্য সরাসরি সংরক্ষণ করা হয় না — পেমেন্ট প্রদানকারীর (যেমন Stripe/SSLCommerz) মাধ্যমে পরিচালিত হয়; আর লগে ব্যক্তিগত তথ্য ছাপা হয় না।

প্রবেশাধিকার পর্যালোচনা (access review) হলো নিয়মিতভাবে জিজ্ঞাসা করা: এই ব্যক্তির কি এখনো এই অধিকার দরকার? দল বদলেছে, প্রকল্প শেষ হয়েছে, কর্মী চলে গেছে — কিন্তু অধিকার থেকে গেছে? প্রতি ত্রৈমাসিকে ম্যানেজাররা নিজের দলের অধিকারের তালিকা পর্যালোচনা করেন; অপ্রয়োজনীয় অধিকার ফেরত নেওয়া হয়; আর পর্যালোচনার রেকর্ড অডিট প্রমাণ হিসেবে থাকে। এই পর্যালোচনা ছাড়া সর্বনিম্ন অধিকার শুধু একটি শুরুর অবস্থা — সময়ের সঙ্গে তা আবার ফুলে ওঠে।

শাসন উপাদান	প্রশ্ন	প্রমাণ
ঝুঁকি ব্যবস্থাপনা	কোন ঝুঁকি, কতটা, কে গ্রহণ করল?	ঝুঁকি নথি (register), ব্যতিক্রম নথি
পরিবর্তন ব্যবস্থাপনা	পরিবর্তনটি কে অনুমোদন করল?	PR, অনুমোদন লগ, ডেপ্লয় লগ
প্রবেশাধিকার পর্যালোচনা	অধিকারটি কি এখনো দরকার?	পর্যালোচনার রেকর্ড, প্রত্যাহারের লগ
নীতি প্রয়োগ	নীতি কি সব জায়গায় চলছে?	OPA/Kyverno লগ, CSPM রিপোর্ট
ডেটা সুরক্ষা	সংবেদনশীল ডেটা কোথায়, কে পড়ে?	শ্রেণিবিন্যাস, এনক্রিপশন, লগ
নিরাপত্তা মেট্রিক	আমরা কি উন্নত হচ্ছি?	ড্যাশবোর্ড, প্রবণতা (trend)

১৫.৫ অভ্যন্তরীণ কার্যপ্রণালী: মেট্রিক, অডিট ও সংস্কৃতি কীভাবে কাজ করে

নিরাপত্তা মেট্রিক (security metrics) হলো শাসনের ভাষা — সংখ্যা ছাড়া উন্নতি প্রমাণ করা যায় না। ভালো মেট্রিকের গুণ: এটি সিদ্ধান্ত বদলায়। উদাহরণ: Critical দুর্বলতার গড় প্রতিকার সময় (MTTR), SLA মানার হার, পাইপলাইন গেটে আটকে যাওয়া ডেপ্লয়ের সংখ্যা, নীতি লঙ্ঘনের সংখ্যা ও প্রবণতা, প্রবেশাধিকার পর্যালোচনায় ফেরত নেওয়া অধিকারের সংখ্যা, আর ব্যতিক্রমের সংখ্যা ও মেয়াদোত্তীর্ণের হার। খারাপ মেট্রিকের উদাহরণ: শুধু স্ক্যানের সংখ্যা — বেশি স্ক্যান মানেই বেশি নিরাপত্তা নয়। মেট্রিককে প্রবণতা হিসেবে দেখুন: এই মাসে কি গত মাসের চেয়ে ভালো? কোন দিকে যাচ্ছি?

অডিটের কাজের পদ্ধতি: অডিটর একটি নিয়ন্ত্রণের অস্তিত্ব নয়, তার কার্যকারিতা যাচাই করেন। তিনি জিজ্ঞাসা করবেন: নীতিটি কোথায় লেখা? এটি কি প্রয়োগ হয়? গত তিন মাসে এর প্রমাণ কী? একটি ব্যতিক্রম দেখান — তার মালিক ও মেয়াদ কোথায়? একটি ব্যর্থতা দেখান — কীভাবে ধরা পড়েছিল, কীভাবে ঠিক হয়েছিল? এই প্রশ্নগুলোর উত্তর যদি লগ, PR ও রিপোর্টে স্বয়ংক্রিয়ভাবে থাকে, তাহলে অডিট একটি যাচাই প্রক্রিয়া হয়; যদি উত্তর খুঁজতে হাতে চ্যাট ঘাঁটতে হয়, তাহলে অডিট একটি সংকট হয়।

সাংগঠনিক নিরাপত্তা পরিচালনা (organizational security operations) মানে ভূমিকা ও সংস্কৃতি। ভূমিকাগুলো: প্রতিটি দলে নিরাপত্তা চ্যাম্পিয়ন (security champion) — যিনি নিরাপত্তা দলের সঙ্গে দলের সেতু; প্ল্যাটফর্ম দল — যারা নিরাপদ ডিফল্ট ও টুল দেয়; নিরাপত্তা দল — যারা নীতি, ঝুঁকি ও ঘটনা দেখে; আর ব্যবস্থাপনা — যারা ঝুঁকি গ্রহণের সিদ্ধান্ত নেয়। সংস্কৃতির

নীতি: নিরাপত্তা ঘটনা রিপোর্ট করাকে শাস্তি নয়, পুরস্কার হিসেবে দেখা — কারণ লুকানো ঘটনা সবচেয়ে বিপজ্জনক; দোষ নয়, মূল কারণ খোঁজা (blameless postmortem); আর নিরাপত্তাকে ডেভেলপারের কাজের অংশ বানানো — নিরাপদ পথটিকে সহজ পথ বানানো।

> **REAL WORLD:** TechNova-তে একবার একজন জুনিয়র ডেভেলপার ভুলে একটি পরীক্ষামূলক সার্ভারের শংসাপত্র একটি পাবলিক চ্যানেলে শেয়ার করে ফেলেছিলেন। তিনি ভয়ে লুকাতে পারতেন; বদলে তিনি সঙ্গে সঙ্গে জানিয়েছিলেন। শংসাপত্রটি মিনিটের মধ্যে বাতিল করা হয়েছিল, আর ঘটনাটি থেকে শেখা হয়েছিল: শংসাপত্র শেয়ার করার নিরাপদ পদ্ধতি (সিক্রেট ম্যানেজারের অস্থায়ী লিংক) সবাইকে শেখানো হয়েছিল। দলটি শিখেছিল: দ্রুত রিপোর্ট করা ঘটনার ক্ষতি কমায়; লুকানো ঘটনা ক্ষতি বাড়ায়।

১৫.৬ ব্যবহারিক কমান্ড ও কনফিগারেশন

শাসন মূলত নথি ও প্রক্রিয়া, কিন্তু তার প্রমাণ আসে কমান্ড ও লগ থেকে। নিচে TechNova-র ল্যাবে প্রমাণ সংগ্রহের ব্যবহারিক পদ্ধতি দেওয়া হলো।

```
# Ubuntu 24.04 ল্যাব — অডিট প্রমাণ সংগ্রহ (TechNova)
# প্রমাণ: পাইপলাইন, নীতি, প্রবেশাধিকার ও পরিবর্তনের রেকর্ড

# ১. Git ইতিহাস: কে কখন কী বদলাল (পরিবর্তন ব্যবস্থাপনার প্রমাণ)
git log --since="2026-10-01" --pretty=format:"%h %ad %an %s" --date=short
git log --grep="security" --oneline | head -n 20

# ২. শাখা সুরক্ষা যাচাই (GitHub CLI — নিজের রিপোজিটরিতে)
gh api repos/technova/web/branches/main/protection --jq '.required_pull_request_reviews'

# ৩. Kubernetes RBAC ও নীতি অবস্থা (ল্যাব ক্লাস্টার)
kubectl get clusterpolicy -o wide
kubectl get events --field-selector reason=PolicyViolation --sort-by=.lastTimestamp | head -n
20

# ৪. লগের অখণ্ডতা: গুরুত্বপূর্ণ লগের হ্যাশ সংরক্ষণ (নমুনা)
sha256sum /var/log/technova/deploy.log >> /var/log/technova/deploy.log.sha256
cat /var/log/technova/deploy.log.sha256

# ৫. প্রবেশাধিকার তালিকা (ল্যাব সার্ভার 203.0.113.10)
sudo getent passwd | cut -d: -f1,3
sudo lastlog | head -n 20
sudo grep -i "sudo:" /var/log/auth.log | tail -n 20

# ৬. মেট্রিকের নমুনা: গত ৩০ দিনের ডেপ্লয় ও ব্যর্থতা গুনুন
git log --since="30 days ago" --oneline | wc -l
# CI ইতিহাস থেকে: সফল বনাম ব্যর্থ রানের অনুপাত (GitHub Actions UI বা API)
```

```
# ঝুঁকি নথি (Risk Register) — নমুনা কাঠামো (Markdown)
# | ঝুঁকি | সম্ভাবনা | প্রভাব | মালিক | প্রতিকার | অবস্থা | পর্যালোচনা |
# | পাবলিক বাকেটে গ্রাহক ডেটা | মাঝারি | উচ্চ | প্ল্যাটফর্ম দল |
#   CSPM + নীতি + পর্যালোচনা | চলমান | 2026-12-01 |
# | দীর্ঘমেয়াদি চাবি ফাঁস | মাঝারি | উচ্চ | নিরাপত্তা দল |
#   OIDC-তে স্থানান্তর | চলমান | 2026-11-30 |
# | একক ব্যক্তির ওপর নির্ভরতা | উচ্চ | মাঝারি | ব্যবস্থাপনা |
#   জ্ঞান ভাগাভাগি, নথিভুক্তকরণ | চলমান | 2026-12-15 |
```

```
# ব্যতিক্রম নথি — নমুনা
# ব্যতিক্রম: E-2026-011 | দুর্বলতা: পুরোনো লাইব্রেরি (Medium)
# ঝুঁকি: সীমিত (অভ্যন্তরীণ, reachability নেই)
# ক্ষতিপূরণ: NetworkPolicy দিয়ে বিচ্ছিন্ন, পর্যবেক্ষণ বাড়ানো
# মালিক: রাহাত | মেয়াদ: 2026-12-09 | পর্যালোচনা: 2026-12-02
# অনুমোদনকারী: নিরাপত্তা প্রধান
```

পরিবর্তন ব্যবস্থাপনার একটি নমুনা চেকলিস্ট — প্রতিটি প্রোডাকশন পরিবর্তনে: পরিবর্তনের বিবরণ ও কারণ লেখা আছে কি? পরীক্ষা হয়েছে কি (ইউনিট, ইন্টিগ্রেশন, smoke)? রোলব্যাক পরিকল্পনা আছে কি? অনুমোদন আছে কি (PR পর্যালোচনা, পরিবেশ অনুমোদন)? পর্যবেক্ষণ পরিকল্পনা আছে কি (কোন মেট্রিক দেখা হবে)? আর পরিবর্তনের পর যাচাই হয়েছে কি? এই চেকলিস্টটি PR টেমপ্লেটে বসিয়ে দিলে পরিবর্তন ব্যবস্থাপনা স্বয়ংক্রিয়ভাবে নথিভুক্ত হয় — আলাদা বৈঠকের দরকার হয় না।

> **WARNING:** সম্মতি (compliance) আর নিরাপত্তা (security) এক নয় — এই ভুলটি সবচেয়ে ব্যয়বহুল। একটি চেকলিস্টে টিক দিয়ে সম্মত হওয়া যায়, কিন্তু আক্রমণকারী চেকলিস্ট পড়ে না। সম্মতিকে মেঝে (floor) ভাবুন — ন্যূনতম স্তর — ছাদ (ceiling) নয়। আপনার লক্ষ্য সম্মতি পেরিয়ে প্রকৃত ঝুঁকি কমানো। আর কোনো নির্দিষ্ট মান বা আইনের বিস্তারিত দাবি করার আগে সরকারি উৎস ও বিশেষজ্ঞ পরামর্শ যাচাই করুন — এই বই সেই দাবিগুলো করে না।

নীতি প্রয়োগ (policy enforcement) শাসনের দৃষ্টিতে শুধু প্রযুক্তিগত বাধা নয় — এটি একটি সিদ্ধান্ত, যার মালিক ও ব্যতিক্রমের পথ থাকতে হয়। TechNova-তে প্রতিটি প্রয়োগকৃত নীতির পেছনে একটি লিখিত যুক্তি আছে: এই নীতি কেন আছে, কোন ঝুঁকি কমায়, কে তার মালিক, আর ব্যতিক্রম চাইতে হলে কী করতে হয়। এই নথি ছাড়া নীতি শুধু একটি বাধা — দল জানে না কেন এটি মানতে হবে, তাই এড়িয়ে যাওয়ার চেষ্টা করে। আরেকটি গুরুত্বপূর্ণ দিক হলো ক্রমাগত উন্নতি: প্রতি ত্রৈমাসিকে শাসন কাঠামো নিজেই পর্যালোচিত হয় — কোন নীতি অপ্রয়োজনীয় হয়ে গেছে, কোন মেট্রিক আর সিদ্ধান্ত বদলাচ্ছে না, কোন প্রক্রিয়া ধীর হয়ে গেছে? শাসন যদি নিজেকে পর্যালোচনা না করে, তাহলে তা ধীরে ধীরে আমলাতন্ত্রে পরিণত হয় — অনেক নিয়ম, কম নিরাপত্তা। ভালো শাসন তাই নিজের ওপরেও একই শৃঙ্খলা প্রয়োগ করে, যা সে সংস্থার ওপর প্রয়োগ করে: পরিমাপ, পর্যালোচনা, আর প্রয়োজনে বদলানো। এই আত্ম-পর্যালোচনাই DevSecOps Governance-কে একটি জীবন্ত ব্যবস্থা রাখে, একটি স্থির নথির সংগ্রহ নয়।

১৫.৭ প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — অডিটর আসছেন তিন মাস পর। প্রতিক্রিয়া: আতঙ্কে হাতে প্রমাণ বানানো নয়; বরং এখন থেকে স্বয়ংক্রিয় প্রমাণ সংগ্রহ চালু করা — পাইপলাইন লগ সংরক্ষণ, অনুমোদনের রেকর্ড, CSPM রিপোর্ট, ব্যতিক্রম নথি, প্রবেশাধিকার পর্যালোচনার রেকর্ড — সব এক জায়গায়। তিন মাসের স্বয়ংক্রিয় প্রমাণ একজন অডিটরকে হাতে বানানো এক সপ্তাহের প্রমাণের চেয়ে বেশি আশ্বস্ত করে।

পরিস্থিতি ২ — একটি ঝুঁকি গ্রহণের সিদ্ধান্ত দরকার। একটি পুরোনো সিস্টেমের দুর্বলতা ঠিক করা যাচ্ছে না; দল ঝুঁকি গ্রহণ করতে চায়। সঠিক পথ: ঝুঁকি নথিতে লেখা — সম্ভাবনা, প্রভাব, ক্ষতিপূরণমূলক নিয়ন্ত্রণ, মালিক, মেয়াদ — আর সিদ্ধান্ত নেবেন সেই ব্যক্তি, যার ঝুঁকি গ্রহণের অধিকার আছে (সাধারণত ব্যবস্থাপনা, ডেভেলপার নয়)। ঝুঁকি গ্রহণ একটি সিদ্ধান্ত, একটি অভ্যাস নয়।

পরিস্থিতি ৩ — নিরাপত্তা ঘটনা ঘটেছে। একটি শংসাপত্র ফাঁস হয়েছে। শাসনের দৃষ্টিতে প্রতিক্রিয়া: শংসাপত্র বাতিল, প্রভাব মূল্যায়ন, গ্রাহককে জানানো (প্রয়োজনে, নীতি অনুযায়ী), মূল কারণ বিশ্লেষণ, আর প্রতিরোধমূলক ব্যবস্থা — সবই নথিভুক্ত। ঘটনার পরের নথিই পরের অডিটের প্রমাণ। দোষারোপ নয়, শিক্ষা — এই সংস্কৃতিই ঘটনা রিপোর্ট করাকে উৎসাহিত করে।

পরিস্থিতি ৪ — দলগুলোর মধ্যে দ্বন্দ্ব: গতি বনাম নিরাপত্তা। অ্যাপ দল বলছে নিরাপত্তা গেট গতি কমাচ্ছে; নিরাপত্তা দল বলছে গেট না থাকলে ঝুঁকি বাড়বে। শাসনের সমাধান: মেট্রিক দিয়ে কথা বলা — গেটে কত সময় লাগছে, কতটি আসল সমস্যা ধরা পড়ছে, আর গেটের কারণে কতটি ঘটনা এড়ানো গেছে? সংখ্যা ছাড়া এই বিতর্ক আবেগের লড়াই; সংখ্যাসহ এটি একটি নকশা সিদ্ধান্ত।

১৫.৮ নিরাপত্তা ও খরচ বিবেচনা

শাসনের খরচ: সময় (পর্যালোচনা, নথিভুক্তকরণ, অডিট প্রস্তুতি), টুল (লগ সংরক্ষণ, CSPM, টিকিট ব্যবস্থা), আর প্রক্রিয়ার ধীরতা। খরচ কমানোর উপায়: স্বয়ংক্রিয়করণ — প্রমাণ স্বয়ংক্রিয়ভাবে সংগ্রহ করা, পর্যালোচনা টিকিট স্বয়ংক্রিয়ভাবে তৈরি করা, মেট্রিক ড্যাশবোর্ড স্বয়ংক্রিয়ভাবে আপডেট করা। হাতে করা শাসন ব্যয়বহুল ও ভুলপ্রবণ; স্বয়ংক্রিয় শাসন সস্তা ও নির্ভরযোগ্য।

নিরাপত্তা দিক: শাসনের রেকর্ড নিজেই সংবেদনশীল — ঝুঁকি নথিতে লেখা থাকে সংস্থা কোথায় দুর্বল; অডিট রিপোর্টে লেখা থাকে কী কী সমস্যা আছে। এই নথিগুলোর প্রবেশাধিকার সীমিত রাখুন, সংরক্ষণ এনক্রিপ্ট করুন, আর শেয়ার করার সময় প্রয়োজন-ভিত্তিতে শেয়ার করুন। আর মনে রাখবেন, শাসন একটি চলমান প্রক্রিয়া — একবার অডিট পাস করলেই শেষ নয়; পরের অডিটের আগেই ঝুঁকি বদলে যায়।

খরচ-ঝুঁকির ভারসাম্য: প্রতিটি নিয়ন্ত্রণের খরচ ও ঝুঁকি-কমানোর মূল্য মাপুন। একটি নিয়ন্ত্রণ, যার খরচ তার কমানো ঝুঁকির চেয়ে বেশি, সেটি পুনর্বিবেচনা করুন — হয় সস্তা বিকল্প খুঁজুন, নয় ঝুঁকি গ্রহণ করুন (নথিভুক্ত করে)। শাসন মানে সবকিছুতে নিয়ন্ত্রণ নয়; শাসন মানে সঠিক জায়গায় সঠিক নিয়ন্ত্রণ।

১৫.৯ সাধারণ ভুল

- **সম্মতিকে নিরাপত্তা ভাবা:** চেকলিস্টে টিক দিলেই নিরাপদ ভাবা — আক্রমণকারী চেকলিস্ট পড়ে না।
- **শাসনকে শুধু কাগজ বানানো:** নীতি লিখে তাকে প্রয়োগ না করা — কাগজে নীতি, বাস্তবে ভিন্ন আচরণ।
- **ঝুঁকি গ্রহণের মালিক না থাকা:** কেউ সিদ্ধান্ত নেয় না, ঝুঁকি নীরবে থেকে যায়।
- **হাতে বানানো প্রমাণ:** অডিটের আগে স্ক্রিনশট জোগাড় করা — দুর্বল, ধীর, অবিশ্বস্ত।
- **ব্যতিক্রমের মেয়াদ না থাকা:** সাময়িক ছাড় চিরস্থায়ী হয়ে যাওয়া।
- **প্রবেশাধিকার পর্যালোচনা না করা:** অধিকার জমে যাওয়া, চলে যাওয়া কর্মীর অধিকার থেকে যাওয়া।
- **ঘটনা লুকানো:** দোষারোপের সংস্কৃতি ঘটনা রিপোর্ট বন্ধ করে দেয়।
- **মেট্রিক ছাড়া শাসন:** উন্নতি হচ্ছে কি না, তা না জানা — সংখ্যা ছাড়া শাসন অন্ধ।

১৫.১০ সমস্যা নির্ণয়

সমস্যা: অডিটর একটি নিয়ন্ত্রণের প্রমাণ চাইলেন, কিন্তু প্রমাণ খুঁজে পাওয়া যাচ্ছে না। নির্ণয়: প্রমাণ কি আদৌ সংগ্রহ করা হয়েছিল, নাকি সংগ্রহ করা হলেও খুঁজে পাওয়া যাচ্ছে না? প্রথমটি সংগ্রহের ব্যর্থতা — স্বয়ংক্রিয় সংগ্রহ চালু করুন। দ্বিতীয়টি সংগঠনের ব্যর্থতা — প্রমাণের তালিকা (index) ও সংরক্ষণের নিয়ম বানান।

সমস্যা: ব্যতিক্রমের সংখ্যা বাড়ছে, মেয়াদোত্তীর্ণ ব্যতিক্রমও অনেক। নির্ণয়: ব্যতিক্রম কি আসল সমস্যার লক্ষণ? হয়তো নীতিটি বাস্তবসম্মত নয়, নয়তো প্রতিকারের সংস্থান নেই। ব্যতিক্রমের মূল কারণ দেখুন — নীতি বদলাতে হবে, নাকি সংস্থান বাড়াতে হবে?

সমস্যা: দলগুলো শাসনকে বাধা ভাবছে। নির্ণয়: শাসন কি কাজের সঙ্গে মিশে আছে, নাকি আলাদা বোঝা? PR টেমপ্লেট, স্বয়ংক্রিয় প্রমাণ, নিরাপদ ডিফল্ট — এগুলো শাসনকে কাজের ভেতরে নিয়ে আসে। শাসন যদি শুধু বৈঠক ও ফর্ম হয়, তাহলে দল তাকে এড়িয়ে যাবে।

সমস্যা: মেট্রিক দেখাচ্ছে সব ভালো, কিন্তু ঘটনা ঘটছে। নির্ণয়: মেট্রিক কি সঠিক জিনিস মাপছে? স্ক্যানের সংখ্যা বাড়া মানে নিরাপত্তা বাড়া নয়। মেট্রিককে ফলাফলের সঙ্গে মেলান: MTTR, SLA মানার হার, ঘটনার সংখ্যা ও তীব্রতা — এই মেট্রিকগুলোই আসল চিত্র দেয়।

> **TROUBLESHOOTING:** শাসন নিয়ে সমস্যা হলে জিজ্ঞাসা করুন: সিদ্ধান্তটি কে নিয়েছে, কোথায় নথিভুক্ত হয়েছে, আর প্রমাণটি কোথায়? এই তিন প্রশ্নের যেকোনো একটির উত্তর না থাকলে সমস্যাটি শাসনের — প্রযুক্তির নয়। প্রযুক্তি দিয়ে শাসনের সমস্যা ঠিক করা যায় না; সিদ্ধান্তের কাঠামো ঠিক করতে হয়।

১৫.১১ সারসংক্ষেপ

DevSecOps Governance মানে নিরাপত্তাকে সিদ্ধান্তের কাঠামোয় বাঁধা: ঝুঁকি ব্যবস্থাপনা (চিহ্নিত, মূল্যায়ন, প্রতিকার, পর্যবেক্ষণ), পরিবর্তন ব্যবস্থাপনা (অনুমোদিত, পরীক্ষিত, নথিভুক্ত, বিপরীতমুখী পরিবর্তন), নীতি প্রয়োগ (Policy as Code দিয়ে), অডিট প্রমাণ (স্বয়ংক্রিয়, অখণ্ড, খুঁজে পাওয়া যায় এমন), ডেটা সুরক্ষা (শ্রেণিবিন্যাস, এনক্রিপশন, সংরক্ষণ), প্রবেশাধিকার পর্যালোচনা (নিয়মিত, নথিভুক্ত), নিরাপত্তা মেট্রিক (সিদ্ধান্ত বদলায় এমন সংখ্যা), আর সাংগঠনিক নিরাপত্তা পরিচালনা (ভূমিকা, সংস্কৃতি, ক্রমাগত উন্নতি)। PART III-এর শিক্ষা এখানে এসে পূর্ণ হয়: অধ্যায় ১১-এর পাইপলাইন, ১২-এর নীতি, ১৩-এর দুর্বলতা ব্যবস্থাপনা ও ১৪-এর ক্লাউড ভঙ্গি — সবই এই শাসন কাঠামোর ভেতরে কাজ করে। প্রযুক্তি নিয়ন্ত্রণ দেয়; শাসন সেই নিয়ন্ত্রণকে বিশ্বাসযোগ্য, প্রমাণযোগ্য ও টেকসই করে।

১৫.১২ পর্যালোচনা প্রশ্ন

1. Governance বলতে কী বোঝায়, এবং এটি দৈনন্দিন নিরাপত্তা কাজ থেকে কীভাবে আলাদা?
2. ঝুঁকি ব্যবস্থাপনার চার ধাপ কী কী? ঝুঁকি কমানো, স্থানান্তর, এড়ানো ও গ্রহণের মধ্যে পার্থক্য কী?
3. অডিট প্রমাণের তিনটি গুণ কী কী? হাতে বানানো প্রমাণ কেন দুর্বল?
4. সম্মতি (compliance) ও নিরাপত্তা (security) এক নয় কেন? একটি উদাহরণ দিন।
5. পরিবর্তন ব্যবস্থাপনায় প্রতিটি প্রোডাকশন পরিবর্তনে কী কী থাকা উচিত?
6. ডেটা শ্রেণিবিন্যাস, এনক্রিপশন ও সংরক্ষণ নীতি — প্রতিটি কেন দরকার?
7. প্রবেশাধিকার পর্যালোচনা (access review) কী, এবং না করলে কী হয়?
8. ভালো নিরাপত্তা মেট্রিকের গুণ কী? একটি ভালো ও একটি খারাপ মেট্রিকের উদাহরণ দিন।
9. Blameless postmortem বলতে কী বোঝায়, এবং এটি ঘটনা রিপোর্ট করাকে কীভাবে উৎসাহিত করে?
10. শাসনের তিন স্তর (প্রযুক্তি, ব্যবস্থাপনা, শাসন) একে অপরের সঙ্গে কীভাবে সম্পর্কিত?

১৫.১৩ ব্যবহারিক অনুশীলন

1. **ঝুঁকি নথি তৈরি:** নিজের ল্যাবের জন্য পাঁচটি ঝুঁকির নথি (risk register) লিখুন — সম্ভাবনা, প্রভাব, মালিক, প্রতিকার ও পর্যালোচনার তারিখসহ।
2. **অডিট প্রমাণ সংগ্রহ:** একটি নমুনা পরিবর্তনের জন্য সম্পূর্ণ প্রমাণ প্যাকেজ বানান — PR লিংক, পরীক্ষার ফলাফল, অনুমোদন, ডেপ্লয় লগ ও পর্যবেক্ষণ — এবং একজন সহকর্মীকে দিয়ে যাচাই করান।

3. **প্রবেশাধিকার পর্যালোচনা:** ল্যাব সার্ভারের ব্যবহারকারী ও তাদের অধিকারের তালিকা বানিয়ে পর্যালোচনা করুন — কোন অধিকার অপ্রয়োজনীয়, তা চিহ্নিত করুন।
4. **মেট্রিক ড্যাশবোর্ড:** এক পৃষ্ঠার নিরাপত্তা মেট্রিক ড্যাশবোর্ডের নকশা আঁকুন — কোন মেট্রিক থাকবে, কোথা থেকে আসবে, কে দেখবে, আর কোন সিদ্ধান্ত বদলাবে?

PART IV • DEVOPS ENGINEERING MASTERY

Site Reliability Engineering

SRE-এর দর্শন, SLI/SLO/SLA ও Error Budget, Monitoring/Alerting/On-call, Incident Management ও RCA, Performance ও Capacity Engineering।

অধ্যায় ১৬ • PART IV — Site Reliability Engineering

Chapter 16 — SRE Fundamentals

১৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Site Reliability Engineering (SRE) বলতে কী বোঝায় তা প্রথম নীতি থেকে ব্যাখ্যা করতে পারবেন। আপনি বুঝবেন কেন শুধু কোড লিখলেই একটি সিস্টেম নির্ভরযোগ্য হয় না, কেন অপারেশনকে প্রকৌশল সমস্যা হিসেবে দেখতে হয়, এবং কীভাবে SRE দল DevOps দলের সঙ্গে পাশাপাশি কাজ করে। আপনি Toil চিনতে পারবেন, তাকে পরিমাপ করতে পারবেন, এবং তাকে স্বয়ংক্রিয় করার সিদ্ধান্ত নিতে পারবেন। আপনি Production Ownership বলতে কী বোঝায়, একটি সেবার স্থাপত্য (Service Architecture) নির্ভরযোগ্যতাকে কীভাবে প্রভাবিত করে, এবং একটি সংস্থায় Reliability Culture কীভাবে গড়ে ওঠে — তা ব্যাখ্যা করতে পারবেন। সবচেয়ে গুরুত্বপূর্ণভাবে, আপনি একটি বাস্তব TechNova সেবার ক্ষেত্রে SRE সিদ্ধান্তের যুক্তি দাঁড় করাতে পারবেন: কোন কাজটি স্বয়ংক্রিয় করবেন, কোনটি করবেন না, কখন মানুষের হস্তক্ষেপ দরকার, এবং কখন সিস্টেমকে নিজে থেকে সেরে উঠতে দেওয়া উচিত।

- SRE-এর দর্শন ও ইতিহাস ব্যাখ্যা করা এবং DevOps থেকে তার পার্থক্য ও মিল চিহ্নিত করা।
- Reliability-কে পরিমাপযোগ্য প্রকৌশল লক্ষ্য হিসেবে সংজ্ঞায়িত করা।
- Toil শনাক্ত করা, পরিমাপ করা এবং তা কমানোর পরিকল্পনা করা।
- Automation, Production Ownership এবং Service Architecture-এর মধ্যে সম্পর্ক বিশ্লেষণ করা।
- Reliability Culture গড়ে তোলার নীতিমালা প্রয়োগ করা।

১৬.২ প্রাক-প্রয়োজনীয় পুনরালোচনা

এই অধ্যায়ে প্রবেশের আগে আপনার DevOps-এর মূল ধারণাগুলো জানা থাকা দরকার। Continuous Integration (CI) এবং Continuous Delivery (CD) বলতে বোঝায় কোড পরিবর্তন স্বয়ংক্রিয়ভাবে পরীক্ষা করে উৎপাদনে পৌঁছে দেওয়া। Infrastructure as Code (IaC) বলতে বোঝায় সার্ভার, নেটওয়ার্ক ও ডেটাবেসের কনফিগারেশনকে কোড হিসেবে লেখা, যাতে তা পুনরুৎপাদনযোগ্য হয়। Container এবং Kubernetes বলতে বোঝায় অ্যাপ্লিকেশনকে বিচ্ছিন্ন পরিবেশে চালানো এবং তা স্বয়ংক্রিয়ভাবে স্কেল করা। Monitoring বলতে বোঝায় সিস্টেম কী করছে তা পর্যবেক্ষণ করা, আর Logging বলতে বোঝায় ঘটনাগুলোর লিখিত রেকর্ড রাখা। এই অধ্যায়ে আমরা ধরে নিচ্ছি আপনি TechNova-এর MERN অ্যাপ্লিকেশনের সঙ্গে পরিচিত: React ফ্রন্টএন্ড, Node.js ও Express ব্যাকএন্ড, MongoDB ডেটাবেস, এবং technova.example ডোমেইনে চলা সেবা। আপনি যদি এই ধারণাগুলো ভুলে গিয়ে থাকেন, তবে চিন্তা করবেন না — প্রতিটি ধারণা যেখানে দরকার সেখানে আবার সংক্ষেপে ব্যাখ্যা করা হয়েছে। SRE শেখার জন্য সবচেয়ে গুরুত্বপূর্ণ পূর্বজ্ঞানটি প্রযুক্তিগত নয়, মানসিক: একটি সিস্টেম কখনো নিখুঁত হয় না, ব্যর্থতা স্বাভাবিক, এবং আমাদের কাজ ব্যর্থতা দূর করা নয় — ব্যর্থতার প্রভাব নিয়ন্ত্রণ করা।

১৬.৩ প্রথম নীতি থেকে ব্যাখ্যা: SRE কেন দরকার?

একটি সফটওয়্যার সিস্টেমের মূল সমস্যাটি সহজ ভাষায় বলা যায়: ব্যবহারকারী চায় সেবাটি সবসময় কাজ করুক, আর ডেভেলপার চায় নতুন ফিচার দ্রুত পৌঁছে দিতে। এই দুটি চাওয়ার মধ্যে টানাপোড়েন আছে। দ্রুত পরিবর্তন মানেই বেশি ঝুঁকি; আর ঝুঁকি মানেই বেশি বিভ্রাট। পুরনো দিনে এই টানাপোড়েন সামলাত Operations দল — তারা পরিবর্তনকে ধীর করে দিত, কারণ তাদের সাফল্য মাপা হতো স্থিতিশীলতা দিয়ে। আর Development দলের সাফল্য মাপা হতো গতি দিয়ে। ফলে দুই দলের মধ্যে দ্বন্দ্ব তৈরি হতো। Google-এ এই সমস্যার উত্তর হিসেবে SRE জন্ম নেয়: অপারেশনের কাজটি সফটওয়্যার প্রকৌশলীদের হাতে দেওয়া, যাতে স্থিতিশীলতাও কোড দিয়ে অর্জন করা যায়। SRE-এর মূল নীতিটি হলো — অপারেশন একটি সফটওয়্যার সমস্যা। যদি কোনো কাজ বারবার হাতে করতে হয়, তবে সেটিকে স্বয়ংক্রিয় করার কোড লেখাই সঠিক

উত্তর। যদি কোনো সিস্টেম বারবার ভাঙে, তবে সেটিকে আরও শক্ত করে গড়ে তোলাই সঠিক উত্তর, শুধু ভাঙা অংশ মেরামত করা নয়।

SRE-এর দর্শনের কেন্দ্রে আছে একটি বাস্তব স্বীকারোক্তি: ১০০% নির্ভরযোগ্যতা অসম্ভব এবং অপ্রয়োজনীয়। কোনো সিস্টেমই কখনো পুরোপুরি ব্যর্থতামুক্ত হয় না — হার্ডওয়্যার নষ্ট হয়, নেটওয়ার্ক ছিঁড়ে যায়, মানুষ ভুল করে, সফটওয়্যারে বাগ থাকে। তাই লক্ষ্য হওয়া উচিত নিখুঁত হওয়া নয়, বরং যথেষ্ট নির্ভরযোগ্য হওয়া — ব্যবহারকারীর প্রত্যাশা অনুযায়ী নির্ভরযোগ্য হওয়া। এই ধারণা থেকেই পরের অধ্যায়ে আমরা SLI, SLO ও Error Budget শিখব। SRE প্রকৌশলী তাই একই সঙ্গে দুটি কাজ করেন: সিস্টেমকে নির্ভরযোগ্য রাখা এবং পরিবর্তনের গতি বজায় রাখা। তিনি কোড লেখেন, কিন্তু তার কোডের উদ্দেশ্য ফিচার নয় — নির্ভরযোগ্যতা। তিনি অটোমেশন লেখেন, ড্যাশবোর্ড বানান, ব্যর্থতার মোড বিশ্লেষণ করেন, এবং উৎপাদন ব্যবস্থার মালিকানা নেন।

> **CONCEPT:** SRE বলতে বোঝায় — অপারেশনকে সফটওয়্যার প্রকৌশল সমস্যা হিসেবে দেখা। একজন SRE প্রকৌশলী উৎপাদন সিস্টেমের নির্ভরযোগ্যতা, স্কেলযোগ্যতা ও কার্যকারিতার জন্য দায়ী, এবং সেই দায়িত্ব তিনি পালন করেন কোড, অটোমেশন ও নকশার মাধ্যমে, শুধু হাতে-কলমে মেরামত করে নয়।

TechNova-এর উদাহরণে ভাবুন। TechNova-এর MERN অ্যাপ্লিকেশনে ব্যবহারকারীরা অর্ডার দেন, পেমেন্ট করেন, প্রোফাইল দেখেন। যদি MongoDB ডেটাবেসটি প্রতি সপ্তাহে ডিস্ক পূর্ণ হয়ে থেমে যায়, তবে পুরনো অপারেশন পদ্ধতিতে কেউ একজন প্রতি সপ্তাহে হাতে ডিস্ক পরিষ্কার করতেন। SRE পদ্ধতিতে প্রথমে জিজ্ঞাসা করা হয় — কেন ডিস্ক পূর্ণ হয়? লগ রোটেশন নেই? পুরনো ডেটা মুছে ফেলা হয় না? এরপর একটি স্থায়ী সমাধান কোড করা হয়: স্বয়ংক্রিয় লগ রোটেশন, ডিস্ক ব্যবহারের সতর্কবার্তা, এবং প্রয়োজনে স্বয়ংক্রিয় স্কেলিং। এটাই SRE-এর পার্থক্য: সমস্যার মূল কারণে আঘাত করা, লক্ষণে নয়।

১৬.৪ SRE বনাম DevOps: মিল, পার্থক্য ও ভুল ধারণা

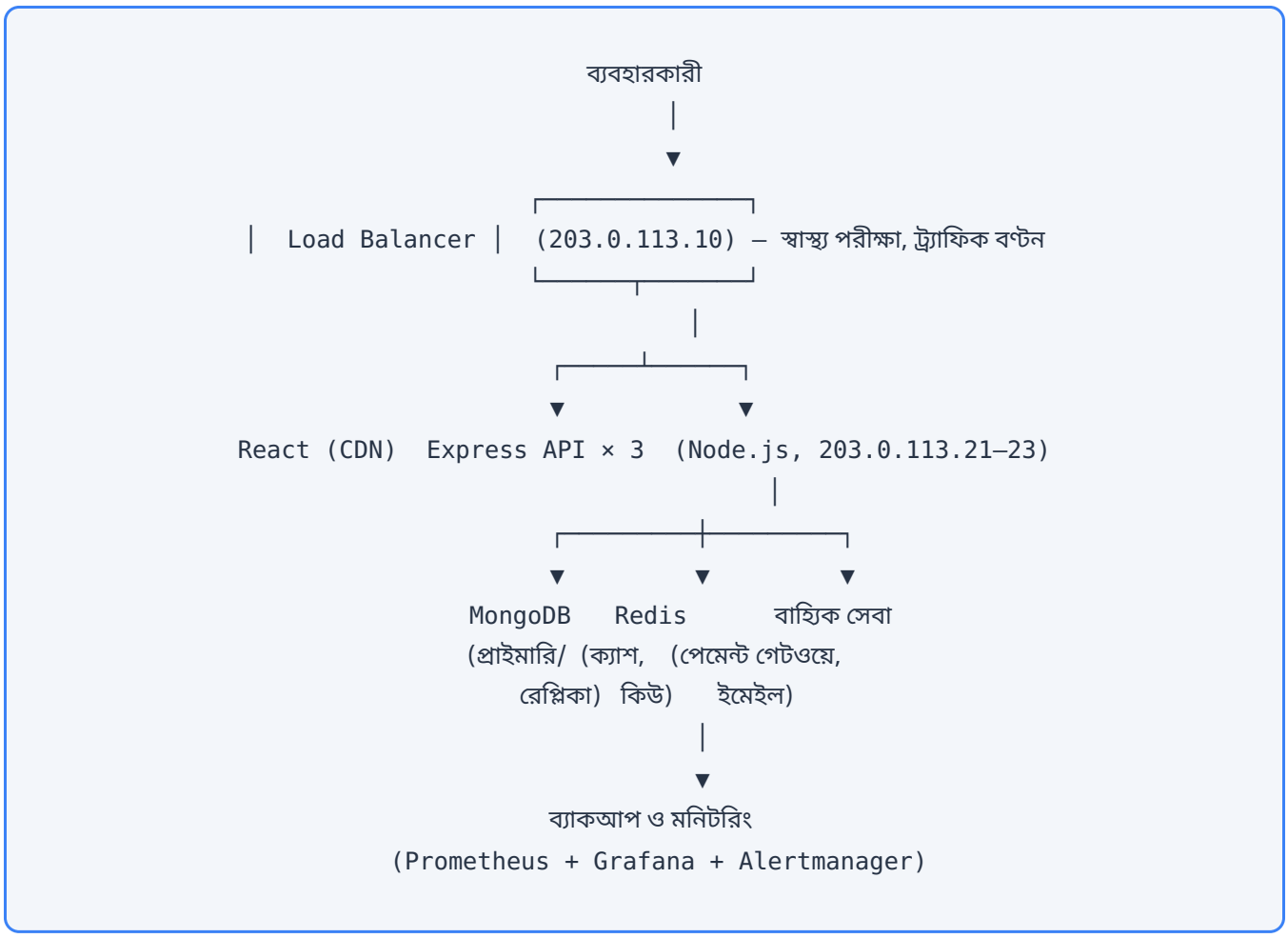
অনেকে SRE ও DevOps-কে একই জিনিস মনে করেন, আবার অনেকে তাদের সম্পূর্ণ আলাদা মনে করেন। সত্য মাঝখানে। DevOps একটি সংস্কৃতি ও অনুশীলনের সমষ্টি — ডেভেলপমেন্ট ও অপারেশনের মধ্যে দেয়াল ভেঙে ফেলা, স্বয়ংক্রিয় পাইপলাইন, দ্রুত ফিডব্যাক, ভাগ করা দায়িত্ব। SRE হলো DevOps-এর একটি নির্দিষ্ট বাস্তবায়ন — একটি ভূমিকা, একটি দল, এবং একটি পরিমাপযোগ্য পদ্ধতি যার কেন্দ্রে আছে নির্ভরযোগ্যতা। সহজভাবে বললে, DevOps বলে কী করতে হবে — সহযোগিতা করো, স্বয়ংক্রিয় করো, দ্রুত শেখো। SRE বলে কীভাবে করতে হবে — SLI দিয়ে পরিমাপ করো, SLO দিয়ে লক্ষ্য ঠিক করো, Error Budget দিয়ে পরিবর্তনের গতি নিয়ন্ত্রণ করো।

দিক	DevOps	SRE
প্রকৃতি	সংস্কৃতি ও অনুশীলন	ভূমিকা, দল ও প্রকৌশল পদ্ধতি
কেন্দ্রীয় লক্ষ্য	গতি, সহযোগিতা, ফিডব্যাক	নির্ভরযোগ্যতা, পরিমাপ, ভারসাম্য
সাফল্যের মাপকাঠি	ডেপ্লয় ফ্রিকোয়েন্সি, লিড টাইম	SLO পূরণ, Error Budget, Toil-এর হার
দায়িত্ব	সব দলের ভাগ করা	SRE দলের নির্দিষ্ট মালিকানা, কিন্তু ডেভেলপারদের সঙ্গে ভাগ করা
সরঞ্জাম	CI/CD, IaC, কন্টেইনার	একই সরঞ্জাম, সঙ্গে SLO, মনিটরিং, ইনসিডেন্ট ব্যবস্থাপনা

একটি সাধারণ ভুল ধারণা হলো SRE মানে শুধু On-call থাকা বা সার্ভার দেখাশোনা করা। বাস্তবে On-call SRE কাজের একটি ছোট অংশ। SRE-এর বেশিরভাগ সময় যায় অটোমেশন, স্থাপত্য উন্নয়ন, ক্ষমতা পরিকল্পনা এবং ব্যর্থতা বিশ্লেষণে।

আরেকটি ভুল ধারণা হলো SRE থাকলে ডেভেলপারদের আর উৎপাদন নিয়ে ভাবতে হয় না। উল্টোটি সত্য: SRE সফল হয় তখনই যখন ডেভেলপাররাও উৎপাদনের মালিকানা নেন — নিজেদের কোডের ব্যর্থতার জন্য নিজেরাই দায়ী থাকেন। TechNova-তে তাই নিয়মটি এমন হওয়া উচিত: যে দল ফিচার লেখে, সেই দলই তার নির্ভরযোগ্যতার জন্য দায়ী; SRE দল তাদের প্ল্যাটফর্ম, সরঞ্জাম ও নির্দেশনা দেয়।

১৬.৫ স্থাপত্য: একটি নির্ভরযোগ্য সেবা কীভাবে গঠিত হয়



উপরের চিত্রে TechNova-এর সরল স্থাপত্য দেখানো হয়েছে। লক্ষ্য করুন, প্রতিটি স্তরে নির্ভরযোগ্যতার নকশা আছে। Load Balancer নিশ্চিত করে একটি API সার্ভার ভেঙে গেলেও ট্র্যাফিক অন্যগুলোতে যায়। API সার্ভার তিনটি — একটিতে সমস্যা হলে সেবা থেমে যায় না। MongoDB-তে প্রাইমারি ও রেপ্লিকা আছে — প্রাইমারি ভেঙে গেলে রেপ্লিকা দায়িত্ব নেয়। Redis ক্যাশ ও কিউ হিসেবে কাজ করে, যাতে ডেটাবেসের উপর চাপ কমে। আর সবকিছুর উপর নজর রাখে মনিটরিং স্তর। এই নকশার মূল নীতিগুলো হলো: Redundancy (একাধিক কপি), Isolation (এক অংশের ব্যর্থতা অন্যকে ডোবাবে না), Graceful Degradation (আংশিক ব্যর্থতাতেও মূল কাজ চলবে), এবং Observability (ভিতরে কী হচ্ছে তা দেখা যাবে)। একটি সেবার স্থাপত্য যত ভালো, SRE-এর কাজ তত সহজ। আর স্থাপত্য খারাপ হলে কোনো পরিমাণ On-call বা মনিটরিং তা বাঁচাতে পারে না। তাই SRE প্রকৌশলী নকশার আলোচনায় শুরু থেকেই থাকেন — ডেপ্লয়মেন্টের পরে নয়।

১৬.৬ অভ্যন্তরীণ কর্মনীতি: Toil, Automation ও Production Ownership

Toil বলতে বোঝায় এমন হাতে করা, পুনরাবৃত্তিমূলক, যান্ত্রিক কাজ যা সিস্টেমের উন্নতি করে না — যেমন প্রতিদিন হাতে লগ মুছে ফেলা, প্রতিবার ডেপ্লয়ের সময় হাতে সার্ভার রিস্টার্ট করা, বা একই ধরনের টিকিট বারবার সমাধান করা। Toil-এর বৈশিষ্ট্য হলো এটি Manual, Repetitive, Automatable, Tactical (কৌশলগত নয়), এবং সময়ের সঙ্গে বাড়তে থাকে — সিস্টেম বড়

হলে Toil-ও বড় হয়। Google-এর SRE নীতি অনুযায়ী, একজন SRE-এর সময়ের সর্বোচ্চ ৫০% Toil-এ যাওয়া উচিত; বাকি ৫০% যাওয়া উচিত প্রকৌশল কাজে — অটোমেশন, স্থাপত্য, নির্ভরযোগ্যতা উন্নয়ন। যদি Toil ৫০% ছাড়িয়ে যায়, তবে দলটি ধীরে ধীরে শুধু আগুন নেভাতে ব্যস্ত হয়ে পড়ে, আর সিস্টেম কখনো ভালো হয় না।

Automation-এর সিদ্ধান্ত নেওয়ার সময় একটি সহজ প্রশ্ন করুন: এই কাজটি কি ভবিষ্যতে আবার হবে? যদি হ্যাঁ হয়, তবে তা স্বয়ংক্রিয় করার খরচ একবার দিতে হয়, কিন্তু হাতে করার খরচ প্রতিবার দিতে হয়। তবে সবকিছু স্বয়ংক্রিয় করা ঠিক নয়। যে কাজ বিরল, জটিল, বা বিচার-বিবেচনা দাবি করে — যেমন একটি অস্বাভাবিক ইনসিডেন্টের সময় সিদ্ধান্ত নেওয়া — সেখানে মানুষের উপস্থিতি দরকার। ভালো অটোমেশন মানুষকে সরিয়ে দেয় না; মানুষকে সঠিক জায়গায় রাখে। Production Ownership বলতে বোঝায় উৎপাদন সেবার জন্য দায়িত্ব নেওয়া — শুধু কোড লেখা নয়, কোডটি উৎপাদনে কেমন চলছে, কতটা নির্ভরযোগ্য, কত খরচ হচ্ছে, ব্যবহারকারীর অভিজ্ঞতা কেমন — সবকিছুর জন্য দায়ী থাকা। TechNova-তে এর অর্থ হলো: API দলের ডেভেলপার শুধু Endpoint লেখেন না; তিনি তার Endpoint-এর Latency, Error Rate ও SLO-ও দেখেন।

> **WARNING:** Toil-কে স্বাভাবিক মনে করবেন না। যে কাজ প্রতি সপ্তাহে হাতে করতে হয়, তা একটি বাগ — সিস্টেমের বাগ, আপনার সময়সূচির নয়। Toil বাড়তে দিলে দল পুড়ে যায় (Burnout), প্রতিভাবান প্রকৌশলীরা চলে যান, আর সিস্টেমের নির্ভরযোগ্যতা কমতে থাকে।

১৬.৭ বাস্তব কমান্ড ও কনফিগারেশন

TechNova-এর Ubuntu Server 24.04 LTS-এ একটি সাধারণ SRE পরীক্ষা দিয়ে শুরু করা যাক — সেবাটি চলছে কি না, কতটা সম্পদ ব্যবহার করছে, এবং কোথায় Toil লুকিয়ে আছে তা দেখা।

```
systemctl status technova-api --no-pager
ss -tulpn | grep -E '3000|27017|6379'
df -h /var/log /var/lib/mongodb
free -h && uptime
journalctl -u technova-api --since "1 hour ago" --no-pager | tail -n 50
```

স্বয়ংক্রিয় স্বাস্থ্য পরীক্ষার একটি ছোট স্ক্রিপ্ট — এটি Toil কমানোর প্রথম ধাপ:

```
#!/bin/bash
# /usr/local/bin/technova-health.sh — TechNova স্বাস্থ্য পরীক্ষা (lab / production)
set -euo pipefail
API="http://203.0.113.21:3000/health"
if curl -fsS --max-time 5 "$API" | grep -q '"status":"ok"'; then
  echo "OK: API healthy"
else
  echo "FAIL: API unhealthy — paging on-call" >> /var/log/technova-health.log
  exit 1
fi
df -h / | awk 'NR==2 {gsub("%","",$5); if ($5 > 80) print "WARN: disk " $5 "% used"}'
```

Toil পরিমাপের জন্য দল সাপ্তাহিক একটি সহজ হিসাব রাখতে পারে: প্রতিটি হাতে করা কাজের সময় লিখে রাখুন, সপ্তাহ শেষে যোগ করুন, এবং মোট কর্মঘণ্টার শতাংশ বের করুন। যদি তা ৫০%-এর বেশি হয়, তবে পরের স্প্রিন্টে অটোমেশনের কাজকে অগ্রাধিকার দিন। Kubernetes-এ TechNova API-এর একটি ন্যূনতম Deployment-এর অংশ নিচে দেখানো হলো — লক্ষ্য করুন Readiness ও Liveness Probe কীভাবে স্বয়ংক্রিয় পুনরুদ্ধারে সাহায্য করে:

```
apiVersion: apps/v1
kind: Deployment
```

```
metadata:
  name: technova-api
  namespace: technova
spec:
  replicas: 3
  selector:
    matchLabels: { app: technova-api }
  template:
    metadata:
      labels: { app: technova-api }
    spec:
      containers:
      - name: api
        image: registry.technova.example/api:1.4.0
        ports: [{ containerPort: 3000 }]
        livenessProbe:
          httpGet: { path: /health, port: 3000 }
          initialDelaySeconds: 10
          periodSeconds: 10
        readinessProbe:
          httpGet: { path: /ready, port: 3000 }
          initialDelaySeconds: 5
          periodSeconds: 5
        resources:
          requests: { cpu: "250m", memory: "512Mi" }
          limits: { cpu: "1000m", memory: "1Gi" }
```

১৬.৮ উৎপাদন পরিস্থিতি

পরিস্থিতি ১ — ডিস্ক পূর্ণ হয়ে API থেমে গেল। শুক্রবার রাত ২টায় Alert এলো: ডিস্ক ৯৫% পূর্ণ। পুরনো পদ্ধতিতে কেউ উঠে হাতে লগ মুছে ফেলতেন। SRE পদ্ধতিতে প্রথমে সেবা পুনরুদ্ধার করা হয় (লগ রোটেট করে জায়গা করা), তারপর মূল কারণ খোঁজা হয় (কোন লগ এত বড় হচ্ছে, কেন রোটেশন হয়নি), তারপর স্থায়ী সমাধান: logrotate কনফিগারেশন, ডিস্ক সতর্কবার্তা ৭০%-এ, এবং লগের মাত্রা (Log Level) পর্যালোচনা। **পরিস্থিতি ২ — ডেপ্লয়ের পর Error Rate বেড়ে গেল।** নতুন সংস্করণে একটি বাগ থাকায় প্রতি ১০০টি অনুরোধে ৮টি ব্যর্থ হচ্ছে। SRE দল Error Budget পরীক্ষা করে দেখে বাজেট এখনো আছে কি না, এবং প্রয়োজনে Rollback করে। তারপর প্রশ্ন করা হয় — কেন পরীক্ষায় বাগটি ধরা পড়েনি? Canary Deployment ছিল কি না? **পরিস্থিতি ৩ — একজন প্রকৌশলী Toil-এ ডুবে আছেন।** তিনি প্রতিদিন দুই ঘণ্টা হাতে ডেটা ঠিক করেন। দল সিদ্ধান্ত নেয় পরের দুই সপ্তাহে সেই কাজটি স্বয়ংক্রিয় করতে, যদিও তাতে নতুন ফিচার একটু দেরি হবে। এই সিদ্ধান্তের যুক্তি হলো: দুই সপ্তাহের বিনিয়োগে প্রতিদিন দুই ঘণ্টা চিরতরে বাঁচবে।

পরিস্থিতি ৬ — Toil-এর হিসাব ও বিনিয়োগের যুক্তি। TechNova-এর SRE দল প্রতি সপ্তাহে একটি সহজ স্প্রেডশিট রাখে: কাজের নাম, কতবার হয়েছে, প্রতিবার কত মিনিট লেগেছে, আর তা স্বয়ংক্রিয় করতে কত ঘণ্টা লাগবে। একটি কাজ সপ্তাহে ৫ বার হয়, প্রতিবার ২০ মিনিট লাগে — মাসে প্রায় ৭ ঘণ্টা। স্বয়ংক্রিয় করতে লাগবে ১০ ঘণ্টা — অর্থাৎ দেড় মাসে বিনিয়োগ উঠে আসবে, তারপর প্রতিমাসে ৭ ঘণ্টা বাঁচবে। এই হিসাব ব্যবস্থাপনাকে দেখানো হলে অটোমেশনের কাজ অনুমোদন পায় — কারণ তা আবেগের কথা নয়, সংখ্যার কথা। আরেকটি নিয়ম: Toil-এর কাজ স্বয়ংক্রিয় করার সময় সেই কাজের জ্ঞানও নথিভুক্ত করুন — স্ক্রিপ্টে মন্তব্য লিখুন, Runbook হালনাগাদ করুন — যাতে স্ক্রিপ্টের লেখক চলে গেলেও অন্যরা বুঝতে পারেন।

পরিস্থিতি ৭ — নির্ভরযোগ্যতা সংস্কৃতির আচার। TechNova-তে প্রতি শুক্রবার ৩০ মিনিটের একটি Reliability Review হয়: গত সপ্তাহের Alert, ইনসিডেন্ট, Toil-এর হিসাব, আর Error Budget-এর অবস্থা — সবাই একসঙ্গে দেখে। কোনো দোষারোপ নেই, শুধু প্রশ্ন: কী শিখলাম, কী বদলাব? এই নিয়মিত আচারের ফলে নির্ভরযোগ্যতা একটি প্রকল্প নয় — একটি

অভ্যাস হয়ে ওঠে। নতুন প্রকৌশলী প্রথম সপ্তাহেই শেখেন: এখানে ব্যর্থতা লুকানো হয় না, ব্যর্থতা থেকে শেখা হয়। আর নেতারা শেখেন: নির্ভরযোগ্যতার কাজকে ফিচারের কাজের মতোই গুরুত্ব দিতে হয় — স্প্রিন্টে জায়গা দিতে হয়, স্বীকৃতি দিতে হয়। সংস্কৃতি এভাবেই গড়ে ওঠে — প্রতিদিনের ছোট সিদ্ধান্তে, বড় ঘোষণায় নয়।

পরিস্থিতি ৮ — Production Ownership বাস্তবে। TechNova-এর API দলের একজন ডেভেলপার নতুন Endpoint লিখলেন — কিন্তু তিনি তার Latency ও Error Rate দেখেননি। প্রথম সপ্তাহেই Endpoint-টি ধীর প্রমাণিত হলো, কিন্তু তিনি জানতেন না — কারণ Dashboard তিনি দেখেন না। দল নিয়ম করল: প্রতিটি Endpoint-এর সঙ্গে তার Dashboard লিঙ্ক, তার SLO, আর তার On-call দায়িত্ব — একই পৃষ্ঠায় থাকবে। ডেভেলপার এখন ডেপ্লয়ের পর নিজেই দেখেন তার কোড কেমন চলছে। এই মালিকানার অনুভূতিই Production Ownership — কোড লেখা শেষ নয়, কোড উৎপাদনে কেমন চলছে তা দেখা ও তার দায়িত্ব নেওয়া।

পরিস্থিতি ৪ — নতুন দলে SRE সংস্কৃতি গড়া। TechNova-তে একটি নতুন দল তৈরি হলো, কিন্তু কেউ On-call নিতে চায় না — সবাই মনে করে এটি শাস্তি। দলের নেতা প্রথমে Toil পরিমাপ করলেন: দেখা গেল সপ্তাহে ১৫ ঘণ্টা হাতে করা কাজ হচ্ছে। তিনি সেই কাজগুলোর মধ্যে সবচেয়ে বড় তিনটি স্বয়ংক্রিয় করার পরিকল্পনা করলেন, এবং On-call-এর সঙ্গে একটি নিয়ম যোগ করলেন — On-call সপ্তাহে কোনো নতুন ফিচার নয়, শুধু নির্ভরযোগ্যতার কাজ। তিন মাস পর On-call আর ভয়ের বিষয় রইল না, কারণ Alert কমে গেছে এবং প্রতিটি Alert-এর Runbook আছে। শিক্ষা: সংস্কৃতি নির্দেশ দিয়ে গড়ে ওঠে না — Toil কমিয়ে, Alert ভালো করে, এবং On-call-কে সম্মান দিয়ে গড়ে ওঠে।

পরিস্থিতি ৫ — স্থাপত্য সিদ্ধান্তে SRE-এর ভূমিকা। একটি নতুন ফিচারের নকশা বৈঠকে প্রস্তাব এলো সব ডেটা একটি মাত্র ডেটাবেসে রাখার — সহজ, দ্রুত। SRE প্রকৌশলী জিজ্ঞাসা করলেন: ডেটাবেসটি ভেঙে গেলে কী হবে? ব্যাকআপ কতক্ষণে ফিরবে? ক্যাশ না থাকলে Latency কত বাড়বে? এই প্রশ্নগুলোর উত্তর না থাকায় নকশা বদলানো হলো — Read Replica যোগ করা হলো, ক্যাশ স্তর যোগ করা হলো, এবং ব্যর্থতার মোড নথিভুক্ত করা হলো। এই আলোচনা ডেপ্লয়ের পরে হলে খরচ অনেক বেশি হতো — নকশার সময় একটি প্রশ্নের মূল্য উৎপাদনে একটি বিভ্রাটের চেয়ে অনেক কম।

১৬.৯ নিরাপত্তা ও খরচ বিবেচনা

SRE কাজে নিরাপত্তা ও খরচ দুটোই সরাসরি জড়িত। অটোমেশন স্ক্রিপ্টে কখনো আসল পাসওয়ার্ড বা API Key লিখবেন না — সবসময় Placeholder (যেমন CHANGE-ME-DB-PASSWORD) এবং Secret Manager ব্যবহার করুন। Health Check Endpoint-এ সংবেদনশীল তথ্য ফেরত দেবেন না — শুধু status ও সংস্করণ দিন। On-call অ্যাক্সেসের ক্ষেত্রে Least Privilege নীতি মানুন: যার যা দরকার শুধু ততটুকুই দিন, এবং সব অ্যাক্সেস লগ করুন। খরচের দিকে, অতিরিক্ত Redundancy-ও খরচ বাড়ায় — তিনটি API সার্ভার যেখানে দুটি যথেষ্ট, সেখানে তৃতীয়টি অপচয়। আবার খুব কম Redundancy নির্ভরযোগ্যতা কমায়। সঠিক ভারসাম্য আসে পরিমাপ থেকে: প্রকৃত ট্র্যাফিক, প্রকৃত ব্যর্থতার হার, এবং SLO-এর প্রয়োজন দেখে সিদ্ধান্ত নিন। মনিটরিং নিজেও খরচ করে — অপ্রয়োজনীয় মেট্রিক সংগ্রহ করবেন না, Retention সময় ঠিক করুন।

১৬.১০ সাধারণ ভুল

- SRE-কে শুধু On-call বা সার্ভার রক্ষণাবেক্ষণ মনে করা।
- ১০০% নির্ভরযোগ্যতা লক্ষ্য করা — যা অসম্ভব এবং অপ্রয়োজনীয়ভাবে ব্যয়বহুল।
- Toil-কে স্বাভাবিক মনে করে তা পরিমাপ না করা।
- সবকিছু স্বয়ংক্রিয় করতে যাওয়া — এমনকি বিচার-বিবেচনা দরকার এমন সিদ্ধান্তও।
- ডেভেলপারদের উৎপাদনের দায়িত্ব থেকে মুক্তি দেওয়া।
- স্থাপত্য নকশায় SRE-কে শেষ মুহূর্তে ডাকা।

- ব্যর্থতাকে লজ্জার বিষয় মনে করে তা লুকানো — ফলে একই ভুল বারবার হয়।
- অটোমেশন লিখে তা পরীক্ষা না করা — ভুল অটোমেশন হাতে করা ভুলের চেয়ে দ্রুত ক্ষতি করে।

১৬.১১ সমস্যা সমাধান (Troubleshooting)

> **TROUBLESHOOTING:** সেবা ধীর বা অস্থির মনে হলে প্রথমে তিনটি প্রশ্ন করুন: (১) কী পরিবর্তন হয়েছে — নতুন ডেপ্লয়, কনফিগারেশন, ট্র্যাফিক বৃদ্ধি? (২) কোন স্তরে সমস্যা — নেটওয়ার্ক, API, ডেটাবেস, নাকি বাহ্যিক সেবা? (৩) মেট্রিক কী বলছে — CPU, মেমরি, Latency, Error Rate? পরিবর্তন ছাড়া সমস্যা খুব কমই আসে; তাই Deployment ইতিহাস ও কনফিগারেশন পরিবর্তন আগে দেখুন।

যদি API সাড়া না দেয়, তবে ধাপে ধাপে পরীক্ষা করুন: প্রথমে প্রক্রিয়াটি চলছে কি না (`systemctl status technova-api`), তারপর পোর্ট শুনছে কি না (`ss -tlnp`), তারপর লগে কী আছে (`journalctl`), তারপর ডেটাবেস সংযোগ আছে কি না। যদি MongoDB সংযোগ ব্যর্থ হয়, তবে Connection String, নেটওয়ার্ক, এবং MongoDB প্রক্রিয়ার অবস্থা পরীক্ষা করুন। যদি সমস্যাটি শুধু একটি সার্ভারে হয়, তবে Load Balancer থেকে সেটিকে সরিয়ে নিন এবং পরে তদন্ত করুন — উৎপাদন পুনরুদ্ধার আগে, তদন্ত পরে। যদি সব সার্ভারে একই সমস্যা হয়, তবে সম্ভবত ডেটাবেস বা বাহ্যিক সেবায় সমস্যা, অথবা সাম্প্রতিক ডেপ্লয়ে বাগ আছে — তখন Rollback বিবেচনা করুন।

১৬.১২ সারসংক্ষেপ

SRE হলো অপারেশনকে সফটওয়্যার প্রকৌশল হিসেবে দেখা। এর লক্ষ্য নিখুঁত হওয়া নয় — ব্যবহারকারীর প্রত্যাশা অনুযায়ী নির্ভরযোগ্য হওয়া। DevOps সংস্কৃতি দেয়, SRE পদ্ধতি দেয়। Toil চিনে, পরিমাপ করে ও স্বয়ংক্রিয় করে দল নিজেকে আগুন নেভানো থেকে মুক্ত করে। Production Ownership মানে যে কোড লেখে, সে তার উৎপাদন আচরণেরও মালিক। আর ভালো Service Architecture — Redundancy, Isolation, Graceful Degradation, Observability — নির্ভরযোগ্যতার ভিত্তি। এই ভিত্তির উপর দাঁড়িয়ে পরের অধ্যায়ে আমরা নির্ভরযোগ্যতাকে সংখ্যায় পরিমাপ করতে শিখব: SLI, SLO, SLA ও Error Budget।

১৬.১৩ পর্যালোচনা প্রশ্ন

1. SRE বলতে কী বোঝায়? DevOps থেকে এটি কীভাবে আলাদা?
2. কেন ১০০% নির্ভরযোগ্যতা সঠিক লক্ষ্য নয়? ব্যাখ্যা করুন।
3. Toil কাকে বলে? এর পাঁচটি বৈশিষ্ট্য কী কী?
4. ৫০% Toil নিয়মটি কী? এটি ভাঙলে কী হয়?
5. Production Ownership বলতে কী বোঝায়? TechNova-এর উদাহরণ দিন।
6. Redundancy, Isolation ও Graceful Degradation কীভাবে নির্ভরযোগ্যতা বাড়ায়?
7. কোন কাজ স্বয়ংক্রিয় করা উচিত আর কোনটি নয় — সিদ্ধান্তের মানদণ্ড কী?
8. একটি সেবার স্থাপত্যে SRE-কে শুরু থেকে যুক্ত করা কেন গুরুত্বপূর্ণ?
9. Reliability Culture বলতে কী বোঝায়? এটি কীভাবে গড়ে ওঠে?
10. ভুল অটোমেশন কেন হাতে করা ভুলের চেয়ে বিপজ্জনক হতে পারে?

১৬.১৪ ব্যবহারিক অনুশীলন

1. আপনার নিজের বা TechNova-এর একটি সেবার ক্ষেত্রে গত এক সপ্তাহের সব হাতে করা অপারেশন কাজের তালিকা করুন। প্রতিটির সময় লিখুন, Toil কি না চিহ্নিত করুন, এবং মোট সময়ের শতাংশ বের করুন। সবচেয়ে বড় Toil-টির জন্য একটি অটোমেশন পরিকল্পনা লিখুন।
2. TechNova API-এর জন্য একটি Health Check স্ক্রিপ্ট লিখুন ও পরীক্ষা করুন (lab পরিবেশে)। স্ক্রিপ্টটি API, MongoDB ও ডিস্ক — তিনটি পরীক্ষা করবে এবং ব্যর্থ হলে লগে লিখবে।
3. একটি ছোট সেবার স্থাপত্য চিত্র আঁকুন যাতে Load Balancer, দুটি API সার্ভার, MongoDB রেপ্লিকা ও মনিটরিং স্তর থাকে। প্রতিটি স্তরে একটি করে ব্যর্থতার মোড লিখুন এবং তা কীভাবে সামলানো হবে তা ব্যাখ্যা করুন।

অধ্যায় ১৭ • PART IV — Site Reliability Engineering

Chapter 17 — SLIs, SLOs, SLAs and Error Budgets

১৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি নির্ভরযোগ্যতাকে সংখ্যায় প্রকাশ করতে পারবেন। আপনি Service Level Indicator (SLI), Service Level Objective (SLO) ও Service Level Agreement (SLA)-এর মধ্যে পার্থক্য স্পষ্টভাবে ব্যাখ্যা করতে পারবেন। আপনি Availability, Latency, Error Rate ও Throughput থেকে সঠিক SLI বেছে নিতে পারবেন, এবং যেকোনো Availability শতাংশ থেকে অনুমোদিত Downtime হিসাব করতে পারবেন। আপনি Error Budget ও Burn Rate বুঝবেন এবং তা দিয়ে Release সিদ্ধান্ত নিতে পারবেন — কখন ডেপ্লয় করা যাবে, কখন থামতে হবে। আপনি Availability, Reliability, Durability ও Resilience-এর মধ্যে পার্থক্য ব্যাখ্যা করতে পারবেন, এবং একটি বাস্তব সেবার জন্য একটি যুক্তিসঙ্গত SLO নকশা করতে পারবেন — কোনো কাল্পনিক সর্বজনীন মান ধার না করে।

- SLI, SLO ও SLA সংজ্ঞায়িত করা ও পারস্পরিক সম্পর্ক ব্যাখ্যা করা।
- Availability শতাংশ থেকে অনুমোদিত Downtime নির্ভুলভাবে হিসাব করা।
- Latency, Error Rate ও Throughput-এর SLI নকশা করা।
- Error Budget ও Burn Rate দিয়ে Release সিদ্ধান্ত নেওয়া।
- SLO নকশার অনুশীলন সম্পন্ন করা ও সাধারণ ভুল এড়ানো।

১৭.২ প্রাক-প্রয়োজনীয় পুনরালোচনা

আগের অধ্যায়ে আমরা শিখেছি SRE-এর লক্ষ্য নিখুঁত হওয়া নয়, যথেষ্ট নির্ভরযোগ্য হওয়া। কিন্তু যথেষ্ট মানে কতটুকু? এই প্রশ্নের উত্তর দিতে সংখ্যা দরকার। মনে রাখুন, ব্যবহারকারীর অভিজ্ঞতা হলো আসল মাপকাঠি — সার্ভার চলছে কি না তা নয়, ব্যবহারকারী কাজটি করতে পারছেন কি না তা-ই গুরুত্বপূর্ণ। Monitoring থেকে আমরা Metrics পাই — যেমন অনুরোধের সংখ্যা, সফল অনুরোধের হার, সাড়া দেওয়ার সময়। Percentile ধারণাটিও মনে রাখুন: p50 মানে মধ্যমা, p95 মানে ৯৫% অনুরোধ এর চেয়ে দ্রুত, p99 মানে ৯৯% অনুরোধ এর চেয়ে দ্রুত। গড় (Average) Latency বিভ্রান্তিকর হতে পারে, কারণ কয়েকটি খুব ধীর অনুরোধ গড়কে টেনে নিয়ে যায়, আবার বেশিরভাগ ব্যবহারকারীর অভিজ্ঞতা লুকিয়ে যায়। তাই SLI-তে আমরা প্রায়ই Percentile ব্যবহার করি। এই অধ্যায়ে আমরা আরও ধরে নিচ্ছি আপনি শতাংশ ও সময়ের হিসাব করতে পারেন — একটি মাসে কত মিনিট আছে তা বের করা এই অধ্যায়ের সবচেয়ে গুরুত্বপূর্ণ গাণিতিক দক্ষতা।

১৭.৩ প্রথম নীতি থেকে ব্যাখ্যা: পরিমাপ ছাড়া নির্ভরযোগ্যতা অর্থহীন

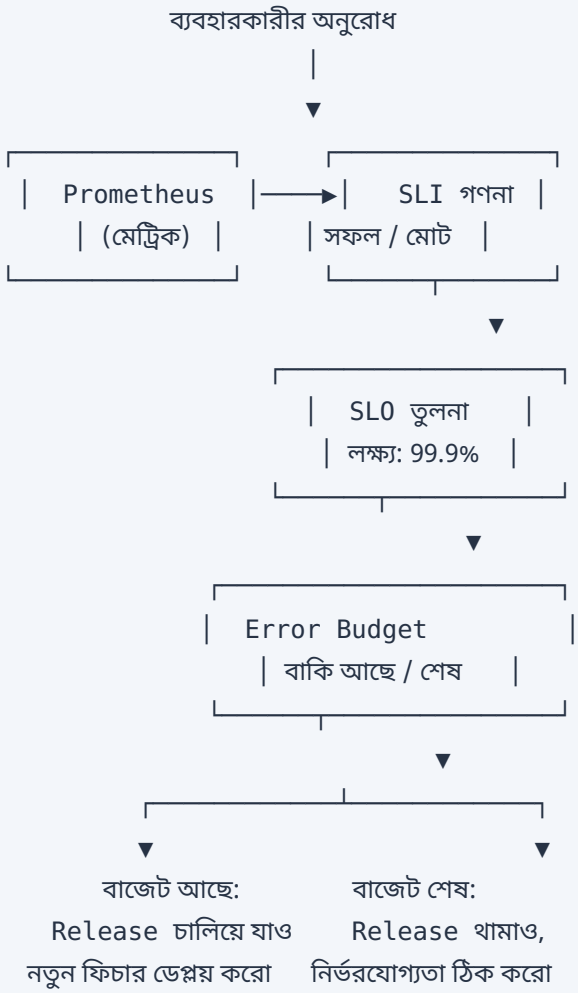
একটি সেবা নির্ভরযোগ্য কি না — এই প্রশ্নের উত্তর অনুভূতি দিয়ে দেওয়া যায় না। ডেভেলপার বলতে পারেন সেবাটি ভালো চলছে, ব্যবস্থাপক বলতে পারেন গ্রাহকরা অভিযোগ করছেন, আর ব্যবহারকারী বলতে পারেন মাঝে মাঝে কাজ করে না। এই তিনটি বক্তব্য একসঙ্গে সত্য হতে পারে, কারণ প্রত্যেকে আলাদা জিনিস দেখছেন। সমাধান হলো একটি ভাগ করা, পরিমাপযোগ্য সংজ্ঞা: আমরা ঠিক কী পরিমাপ করব, কতটুকু ভালো হলে যথেষ্ট বলব, এবং কতটুকু খারাপ হলে ব্যবস্থা নেব। SLI হলো পরিমাপটি — যেমন সফল অনুরোধের অনুপাত। SLO হলো লক্ষ্য — যেমন ৩০ দিনে ৯৯.৯% সফল অনুরোধ। আর SLA হলো চুক্তি — গ্রাহকের সঙ্গে আইনগত প্রতিশ্রুতি, যা ভাঙলে ক্ষতিপূরণ দিতে হয়। এই তিনটি এক জিনিস নয়, এবং তাদের গুলিয়ে ফেলাই সবচেয়ে সাধারণ ভুল।

Availability, Reliability, Durability ও Resilience — এই চারটি শব্দও প্রায়ই গুলিয়ে ফেলা হয়, কিন্তু তাদের অর্থ আলাদা। Availability বলতে বোঝায় সেবাটি যখন দরকার তখন ব্যবহারযোগ্য কি না — সাধারণত শতাংশে প্রকাশ করা হয়।

Reliability বলতে বোঝায় সেবাটি কতটা ধারাবাহিকভাবে সঠিকভাবে কাজ করে — শুধু চালু থাকা নয়, সঠিক ফল দেওয়াও এর অংশ। Durability বলতে বোঝায় ডেটা হারিয়ে যাবে না — সিস্টেম ভেঙে গেলেও সংরক্ষিত ডেটা টিকে থাকবে। আর Resilience বলতে বোঝায় ব্যর্থতা থেকে সেরে ওঠার ক্ষমতা — ভাঙার পর কত দ্রুত ও কতটা সম্পূর্ণভাবে সেবা ফিরে আসে। একটি সেবা Available হতে পারে কিন্তু Reliable নয় — যেমন API সাড়া দিচ্ছে কিন্তু ভুল ফল দিচ্ছে। আবার একটি সেবা Durable হতে পারে কিন্তু Available নয় — ডেটা নিরাপদ, কিন্তু সার্ভার বন্ধ। ভালো SLO নকশা এই পার্থক্যগুলো মাথায় রেখে করা হয়: শুধু Uptime নয়, ব্যবহারকারীর আসল অভিজ্ঞতা পরিমাপ করা হয়।

> **CONCEPT:** SLI হলো পরিমাপ (কী মাপছি), SLO হলো লক্ষ্য (কতটুকু ভালো চাই), SLA হলো চুক্তি (ভাঙলে কী হবে)। SLO একটি প্রকৌশল লক্ষ্য — এটি গ্যারান্টি নয়। SLA একটি ব্যবসায়িক ও আইনগত প্রতিশ্রুতি — এটি SLO-এর চেয়ে সাধারণত কম কঠোর হয়, যাতে চুক্তি ভাঙার ঝুঁকি কম থাকে।

১৭.৪ স্থাপত্য: SLI থেকে সিদ্ধান্ত পর্যন্ত প্রবাহ



এই প্রবাহটি SRE-এর হৃদয়। মেট্রিক থেকে SLI, SLI থেকে SLO তুলনা, SLO থেকে Error Budget, আর Error Budget থেকে Release সিদ্ধান্ত। লক্ষ্য করুন, সিদ্ধান্তটি আর ব্যক্তিগত মতামতের উপর নির্ভর করে না — এটি সংখ্যার উপর নির্ভর করে। ডেভেলপার যখন বলেন নতুন ফিচার ডেপ্লয় করতে চান, আর SRE যখন বলেন সিস্টেম অস্থির, তখন তর্কের মীমাংসা

হয় Error Budget দিয়ে: বাজেট থাকলে ডেপ্লয় হবে, না থাকলে হবে না। এই পদ্ধতিই DevOps-এর গতি ও SRE-এর স্থিতিশীলতার মধ্যে ভারসাম্য আনে।

১৭.৫ অভ্যন্তরীণ কর্মনীতি: Availability হিসাব, Latency ও Error Rate

Availability হিসাবের সূত্রটি সহজ: Availability = (সফল সময় / মোট সময়) × ১০০। কিন্তু Downtime হিসাব করার সময় সময়কালটি স্পষ্ট করা জরুরি — কারণ একই শতাংশ ভিন্ন সময়কালে ভিন্ন Downtime দেয়। একটি ৩০ দিনের মাসে মোট মিনিট হলো ৩০ × ২৪ × ৬০ = ৪৩,২০০ মিনিট। ৯৯.৯৯% Availability মানে ০.০১% Downtime অনুমোদিত, অর্থাৎ ৪৩,২০০ × ০.০০০১ = ৪.৩২ মিনিট, অর্থাৎ **৪ মিনিট ১৯ সেকেন্ড**। এই সংখ্যাটি মুখস্থ রাখুন: **৩০ দিনের মাসে ৯৯.৯৯% মানে ৪ মিনিট ১৯ সেকেন্ড**।

> **WARNING:** ৯৯.৯৯% Availability-এর Downtime হিসেবে ৪ মিনিট ২৩ সেকেন্ড বলা ভুল — এই সংখ্যাটি আসে গড় মাস (~৩০.৪৪ দিন) ধরে হিসাব করলে। গড় মাসের হিসাব বার্ষিক গড়ের জন্য ব্যবহার করা যেতে পারে, কিন্তু একটি নির্দিষ্ট ৩০ দিনের মাসের জন্য সঠিক সংখ্যা ৪ মিনিট ১৯ সেকেন্ড। সবসময় বলুন কোন সময়কালের জন্য হিসাব করছেন — ৩০ দিন, ৩০.৪৪ দিন (গড় মাস), নাকি এক বছর — নইলে সংখ্যাটি অর্থহীন।

একইভাবে হিসাব করুন: ৩০ দিনের মাসে ৯৯.৯% মানে ৪৩.২ মিনিট (প্রায় ৪৩ মিনিট ১২ সেকেন্ড), ৯৯.৫% মানে ২১৬ মিনিট (৩ ঘণ্টা ৩৬ মিনিট), ৯৯% মানে ৪৩২ মিনিট (৭ ঘণ্টা ১২ মিনিট)। লক্ষ্য করুন, প্রতিটি অতিরিক্ত ৯ (Nine) যোগ করা Downtime-কে দশ ভাগের এক ভাগ করে দেয় — কিন্তু খরচ বাড়ায় বহুগুণ। ৯৯% থেকে ৯৯.৯%-এ যাওয়া যতটা কঠিন, ৯৯.৯৯% থেকে ৯৯.৯৯৯%-এ যাওয়া তার চেয়ে অনেক বেশি কঠিন। তাই SLO ঠিক করার সময় ব্যবহারকারীর আসল প্রয়োজন জিজ্ঞাসা করুন, সর্বোচ্চ সংখ্যা নয়।

Availability (৩০ দিনের মাস)	অনুমোদিত Downtime	মন্তব্য
৯৯%	৭ ঘণ্টা ১২ মিনিট	অনেক অভ্যন্তরীণ সেবার জন্য যথেষ্ট
৯৯.৯%	৪৩ মিনিট ১২ সেকেন্ড	সাধারণ উৎপাদন লক্ষ্য
৯৯.৯৫%	২১ মিনিট ৩৬ সেকেন্ড	কঠোর লক্ষ্য
৯৯.৯৯%	৪ মিনিট ১৯ সেকেন্ড	খুব কঠোর; ৩০ দিনের মাসের হিসাব
৯৯.৯৯৯%	২৬ সেকেন্ড	অত্যন্ত ব্যয়বহুল, বিরল ক্ষেত্রে দরকার

Latency SLI-এর ক্ষেত্রে শুধু গড় দেখবেন না। একটি ভালো Latency SLI হতে পারে: ৩০ দিনের উইন্ডোতে ৯৫% অনুরোধ ৩০০ মিলিসেকেন্ডের মধ্যে সাড়া দেবে। Error Rate SLI হতে পারে: সফল অনুরোধের অনুপাত — যেখানে ৫xx ত্রুটি ব্যর্থতা, কিন্তু ৪xx ত্রুটি (যেমন ভুল ইনপুট) সাধারণত ব্যবহারকারীর ভুল, সিস্টেমের ব্যর্থতা নয় — তাই SLI নকশায় কোনটি গুনবেন তা স্পষ্ট করুন। Throughput SLI হতে পারে: প্রতি সেকেন্ডে কতটি অনুরোধ সফলভাবে প্রক্রিয়া করা যায়। প্রতিটি SLI-এর জন্য তিনটি জিনিস ঠিক করুন: কী পরিমাপ করছেন, কোন সময় উইন্ডোতে, এবং কোন উৎস থেকে। উৎসটি ব্যবহারকারীর দৃষ্টিকোণ থেকে হওয়া উচিত — Load Balancer বা API Gateway-এর মেট্রিক, ভিতরের সার্ভারের নিজের দাবি নয়।

Error Budget হলো SLO-এর উল্টো দিক: ১০০% থেকে SLO বিয়োগ করলে যা থাকে, তা-ই বাজেট। ৯৯.৯% SLO মানে ০.১% Error Budget। এই বাজেটই ঝুঁকি নেওয়ার অনুমতি — নতুন ফিচার ডেপ্লয়, পরীক্ষামূলক পরিবর্তন, দ্রুত Release। বাজেট শেষ হয়ে গেলে ঝুঁকি নেওয়া বন্ধ — তখন দল নির্ভরযোগ্যতা ঠিক করতে মনোযোগ দেয়। Burn Rate বলতে বোঝায় বাজেট কত দ্রুত খরচ হচ্ছে। Burn Rate ১ মানে বাজেট ঠিক হারে খরচ হচ্ছে; ২ মানে দ্বিগুণ হারে খরচ হচ্ছে — অর্থাৎ

বাজেট অর্ধেক সময়ে শেষ হবে; ১০ মানে খুব দ্রুত খরচ হচ্ছে — জরুরি ব্যবস্থা দরকার। Burn Rate দিয়ে Alert তৈরি করা যায়: ধীরে পোড়া (Slow Burn) হলে কয়েক দিনের মধ্যে বাজেট শেষ হবে — তখন টিকিট তৈরি হয়; দ্রুত পোড়া (Fast Burn) হলে কয়েক ঘণ্টায় শেষ হবে — তখন On-call-কে ডাকা হয়।

১৭.৬ বাস্তব কমান্ড ও কনফিগারেশন

Prometheus-এ TechNova API-এর Availability SLI গণনার একটি PromQL উদাহরণ:

```
# সফল অনুরোধের অনুপাত (৫xx বাদে), গত ৩০ দিনের উইন্ডোতে
sum(rate(http_requests_total{job="technova-api", status!~"5.."}[30d]))
  /
sum(rate(http_requests_total{job="technova-api"}[30d]))

# p95 latency (সেকেন্ডে)
histogram_quantile(0.95,
  sum(rate(http_request_duration_seconds_bucket{job="technova-api"}[5m])) by (le))

# Error Budget খরচের হার — গত ১ ঘণ্টায়
# (১ - সফল অনুপাত) / (১ - SLO) ; SLO 0.999 হলে ভাজক 0.001
(1 - (
  sum(rate(http_requests_total{job="technova-api", status!~"5.."}[1h]))
  /
  sum(rate(http_requests_total{job="technova-api"}[1h]))
)) / 0.001
```

Downtime হিসাবের জন্য একটি ছোট শেল হিসাব — সবসময় সময়কাল স্পষ্ট করে:

```
# ৩০ দিনের মাসে ৯৯.৯৯% এর অনুমোদিত downtime (মিনিটে)
python3 -c "mins=30*24*60; print('total minutes:', mins); print('allowed downtime (min):',
mins*0.0001); print('= 4m 19s')"
# বার্ষিক হিসাব (৩৬৫ দিন) আলাদা — একে মাসের হিসাবের সঙ্গে গুলিয়ে ফেলবেন না
python3 -c "mins=365*24*60; print('99.99% yearly allowed (min):', mins*0.0001)"
```

একটি SLO নথির ন্যূনতম কাঠামো — TechNova Checkout API-এর উদাহরণ:

```
Service: technova-api — Checkout Endpoint (POST /api/orders)
SLI (Availability): 5xx নয় এমন অনুরোধের অনুপাত, Load Balancer মেট্রিক থেকে
SLO: ৩০ দিনের rolling window-এ ৯৯.৯% সফল অনুরোধ
SLI (Latency): p95 < 300ms, একই উইন্ডোতে
Error Budget: ০.১% (৩০ দিনে ~৪৩ মিনিট)
নীতি: বাজেট ৫০%-এর নিচে নামলে feature freeze; ০% হলে Release বন্ধ
SLA (গ্রাহক চুক্তি): ৯৯.৫% — SLO-এর চেয়ে কম কঠোর, ইচ্ছাকৃতভাবে
```

১৭.৭ উৎপাদন পরিস্থিতি

পরিস্থিতি ১ — বাজেট শেষ, কিন্তু ফিচার জরুরি। TechNova-এর Checkout API-এর Error Budget মাসের ২০ তারিখেই শেষ হয়ে গেল — একটি বাগযুক্ত ডেপ্লয়ের কারণে। ব্যবস্থাপনা চাপ দিচ্ছে নতুন পেমেন্ট ফিচার ডেপ্লয় করতে। সঠিক সিদ্ধান্ত: Release থামান, নির্ভরযোগ্যতা ঠিক করুন, এবং ব্যাখ্যা করুন যে বাজেট শেষ হওয়ার অর্থই হলো ব্যবহারকারীরা ইতিমধ্যে যথেষ্ট ব্যর্থতা সহ্য করেছেন। এই নিয়ম ভাঙলে SLO-এর কোনো অর্থ থাকে না। **পরিস্থিতি ২ — ভুল SLI।** দল Uptime পরিমাপ করছিল সার্ভার চলছে কি না দিয়ে — ৯৯.৯৯% দেখাচ্ছিল। কিন্তু ব্যবহারকারীরা অভিযোগ

করছিলেন Checkout ধীর। আসল সমস্যা ছিল Latency — p95 ছিল ২ সেকেন্ড। শিক্ষা: SLI ব্যবহারকারীর দৃষ্টিকোণ থেকে নিন, সার্ভারের দৃষ্টিকোণ থেকে নয়। **পরিস্থিতি ৩ — অতিরিক্ত কঠোর SLO।** একটি অভ্যন্তরীণ Admin সেবার জন্য ৯৯.৯৯% SLO ঠিক করা হয়েছিল, ফলে দল সারাক্ষণ সতর্কবার্তা ও On-call-এ ব্যস্ত থাকত। আসলে ব্যবহারকারীরা (কর্মীরা) ৯৯% -এই খুশি ছিলেন। SLO কমিয়ে ৯৯.৫% করা হলো, দল মুক্তি পেল, আর ব্যবসার কোনো ক্ষতি হলো না।

পরিস্থিতি ৬ — SLO নকশার অনুশীলন: ভুল উইন্ডো। একটি দল ৯০ দিনের উইন্ডোতে SLO মাপছিল — ফলে গত সপ্তাহের বড় বিভ্রাট ৯০ দিনের গড়ে হারিয়ে গেল, SLO সবুজ দেখাল, যদিও ব্যবহারকারীরা গত সপ্তাহে ভুগেছেন। তারা উইন্ডো বদলে ৩০ দিনের Rolling Window করল — এখন সাম্প্রতিক ব্যর্থতা দ্রুত ধরা পড়ে। শিক্ষা: উইন্ডো খুব লম্বা হলে সাম্প্রতিক ক্ষতি লুকিয়ে যায়, খুব ছোট হলে অস্থায়ী ঝাঁকুনিতে SLO ভেঙে যায়। ৩০ দিন একটি ভালো ভারসাম্য — কিন্তু তা যুক্তি দিয়ে বেছে নিতে হয়, অভ্যাস দিয়ে নয়। আর Rolling Window ব্যবহার করুন — ক্যালেন্ডার মাস নয় — যাতে মাসের শুরুতে বাজেট হঠাৎ রিসেট না হয়।

পরিস্থিতি ৭ — Availability বনাম সঠিকতা। TechNova-এর একটি সেবা ৯৯.৯৯% Available দেখাচ্ছিল — সার্ভার চলছে, অনুরোধ সাড়া দিচ্ছে। কিন্তু একটি বাগের কারণে ৫% অর্ডারে ভুল মূল্য দেখাচ্ছিল। ব্যবহারকারীরা সেবাটিকে নির্ভরযোগ্য মনে করছিলেন না — ঠিকই। শিক্ষা: Availability শুধু চালু থাকা নয় — সঠিক ফল দেওয়াও। তাই SLI-তে শুধু ৫xx ত্রুটি নয়, ব্যবসায়িক সঠিকতাও পরিমাপ করুন — যেমন সফল অর্ডারের অনুপাত, সঠিক মূল্যের অনুপাত। একটি SLI যা ব্যবহারকারীর আসল ক্ষতি ধরে না, তা যত ভালো দেখাক — তা ভুল SLI।

পরিস্থিতি ৮ — SLA ভাঙার পর কী হয়। একটি গ্রাহকের সঙ্গে TechNova-এর SLA ছিল ৯৯.৫% — এক মাসে Availability নেমে গেল ৯৯.২%-এ। চুক্তি অনুযায়ী গ্রাহক ক্ষতিপূরণ পেলেন — বিলের একটি অংশ ফেরত। কিন্তু আসল ক্ষতি ছিল আস্থা — গ্রাহক পরের চুক্তি নবায়নে দ্বিধা করলেন। শিক্ষা: SLA ভাঙার খরচ শুধু অর্থ নয় — আস্থা। তাই SLA-কে SLO-এর চেয়ে কম কঠোর রাখুন — যাতে SLA ভাঙার আগেই SLO সতর্ক করে দেয়, এবং দল ব্যবস্থা নিতে পারে। আর SLA ভাঙলে সত্য বলুন, কারণ লুকিয়ে, আর ক্ষতিপূরণের সঙ্গে সংশোধনের পরিকল্পনাও দিন।

পরিস্থিতি ৪ — SLO নকশার অনুশীলন। TechNova-এর একটি নতুন সেবার জন্য দল বসল SLO ঠিক করতে। প্রথমে তারা ব্যবহারকারীর যাত্রা ম্যাপ করল: ব্যবহারকারী পণ্য খোঁজেন, কার্টে যোগ করেন, Checkout করেন। প্রতিটি ধাপের জন্য তারা জিজ্ঞাসা করল: এই ধাপ ব্যর্থ হলে ব্যবহারকারী কী হারাবেন? Checkout ব্যর্থ হলে অর্ডার হারাবেন — তাই তার SLO সবচেয়ে কঠোর। পণ্য খোঁজা ব্যর্থ হলে ব্যবহারকারী আবার চেষ্টা করতে পারেন — তাই তার SLO কিছুটা শিথিল। তারপর তারা ৩০ দিনের ডেটা দেখল: বর্তমান সাফল্যের হার কত? যদি বর্তমান হার ৯৯.৫% হয়, তবে ৯৯.৯% SLO ঠিক করা অবাস্তব — প্রথমে ৯৯.৫% ঠিক করে, সিস্টেম উন্নত করে, তারপর লক্ষ্য বাড়ানো উচিত। শিক্ষা: SLO বর্তমান বাস্তবতা থেকে শুরু হয়, ইচ্ছা থেকে নয়।

পরিস্থিতি ৫ — Multi-window Burn Rate Alert। দল শুধু এক ঘণ্টার Burn Rate দিয়ে Alert বসিয়েছিল — ফলে একটি ছোট ঝাঁকুনিতেও Paging Alert বাজত। তারা দুটি উইন্ডো যোগ করল: ১ ঘণ্টার দ্রুত পোড়া (Fast Burn, থ্রেশহোল্ড বেশি) এবং ৬ ঘণ্টার ধীর পোড়া (Slow Burn, থ্রেশহোল্ড কম)। দ্রুত পোড়া মানে জরুরি ব্যবস্থা, ধীর পোড়া মানে দিনের বেলায় তদন্ত। এই দুটি উইন্ডো একসঙ্গে ব্যবহার করলে ভুল Alert কমে, আর আসল বিপদ ধরা পড়ে। শিক্ষা: একটি সংখ্যা দিয়ে সব পরিস্থিতি ধরা যায় না — সময়ের মাত্রা যোগ করুন।

১৭.৮ নিরাপত্তা ও খরচ বিবেচনা

SLO-এর নিরাপত্তা দিকটি সূক্ষ্ম: Availability SLI-তে DDoS আক্রমণের সময় ব্যর্থ অনুরোধও গুনতে হবে কি না তা ঠিক করুন — নইলে আক্রমণকারী আপনার Error Budget খেয়ে ফেলবে। সাধারণত আক্রমণ বা বাহ্যিক বিভ্রাটের সময়কালকে SLO থেকে বাদ দেওয়ার নিয়ম (Exclusion) নথিভুক্ত করা হয়, তবে তা অপব্যবহার করা যাবে না। খরচের দিকে, প্রতিটি অতিরিক্ত ৯-এর মূল্য বুঝুন: ৯৯.৯% থেকে ৯৯.৯৯%-এ যেতে মাল্টি-রিজিয়ন, অতিরিক্ত Redundancy ও ২৪ ঘণ্টা On-call

লাগতে পারে — খরচ কয়েকগুণ বাড়তে পারে। SLO ঠিক করার সময় ব্যবসার সঙ্গে বসে জিজ্ঞাসা করুন: এই অতিরিক্ত নির্ভরযোগ্যতা কত আয় রক্ষা করবে, আর তার খরচ কত? যদি খরচ আয়ের চেয়ে বেশি হয়, তবে SLO কমানোই যুক্তিসঙ্গত।

১৭.৯ সাধারণ ভুল

- SLI, SLO ও SLA-কে একই জিনিস মনে করা।
- Downtime হিসাবের সময়কাল না বলা — ৪ মিনিট ১৯ সেকেন্ড বনাম ৪ মিনিট ২৩ সেকেন্ডের বিভ্রান্তি।
- সার্ভারের Uptime-কে ব্যবহারকারীর অভিজ্ঞতা মনে করা।
- গড় Latency দিয়ে SLI তৈরি করা — Percentile ব্যবহার না করা।
- কোনো যুক্তি ছাড়া ৯৯.৯৯৯% SLO ধার করা — সর্বজনীন SLO মান নেই।
- 8xx ত্রুটিকে সিস্টেম ব্যর্থতা হিসেবে গোনা।
- Error Budget শেষ হলেও Release চালিয়ে যাওয়া।
- SLA-কে SLO-এর সমান বা তার চেয়ে কঠোর করা।

১৭.১০ সমস্যা সমাধান (Troubleshooting)

TROUBLESHOOTING: SLI হঠাৎ খারাপ দেখালে প্রথমে পরীক্ষা করুন মেট্রিকের উৎস ঠিক আছে কি না — Prometheus Scrape ব্যর্থ হচ্ছে? নাকি আসলেই ব্যবহারকারীরা ব্যর্থ হচ্ছেন? মেট্রিকের সমস্যা আর সেবার সমস্যা আলাদা করুন। তারপর দেখুন কোন Endpoint, কোন অঞ্চল, কোন সংস্করণে সমস্যা — ভাগ করে দেখলে (Segmentation) মূল কারণ দ্রুত বের হয়।

যদি Error Budget খুব দ্রুত শেষ হয়, তবে Burn Rate Alert-এর থ্রেশহোল্ড পরীক্ষা করুন — হয়তো Alert খুব সংবেদনশীল, অথবা আসলেই একটি বড় বিভ্রাট চলছে। যদি SLO সবসময় পূরণ হয় কিন্তু ব্যবহারকারীরা অসন্তুষ্ট থাকেন, তবে SLI ভুল — ব্যবহারকারীর যাত্রা (User Journey) আবার ম্যাপ করুন: কোন ধাপে তারা ব্যর্থ হচ্ছেন? যদি SLO কখনো পূরণ না হয়, তবে লক্ষ্যটি অবাস্তব কি না পর্যালোচনা করুন — অথবা সিস্টেমে আসলেই মৌলিক সমস্যা আছে কি না।

১৭.১১ সারসংক্ষেপ

SLI হলো পরিমাপ, SLO হলো লক্ষ্য, SLA হলো চুক্তি। Availability, Reliability, Durability ও Resilience আলাদা ধারণা — সবগুলোকে একসঙ্গে দেখতে হয়। ৩০ দিনের মাসে ৯৯.৯৯% মানে ৪ মিনিট ১৯ সেকেন্ড Downtime — সময়কাল সবসময় স্পষ্ট করুন। Error Budget হলো ঝুঁকি নেওয়ার অনুমতি, আর Burn Rate বলে তা কত দ্রুত খরচ হচ্ছে। ভালো SLO ব্যবহারকারীর অভিজ্ঞতা থেকে আসে, কোনো সর্বজনীন তালিকা থেকে নয়। পরের অধ্যায়ে আমরা শিখব এই পরিমাপগুলোকে কীভাবে পর্যবেক্ষণ করা হয়, কখন সতর্কবার্তা দেওয়া হয়, আর On-call জীবন কেমন হয়।

১৭.১২ পর্যালোচনা প্রশ্ন

1. SLI, SLO ও SLA-এর মধ্যে পার্থক্য কী? একটি করে উদাহরণ দিন।
2. ৩০ দিনের মাসে ৯৯.৯% Availability-এর অনুমোদিত Downtime কত? হিসাব দেখান।
3. ৯৯.৯৯% Availability-এর জন্য ৪ মিনিট ১৯ সেকেন্ড কেন সঠিক, আর ৪ মিনিট ২৩ সেকেন্ড কেন ভুল? ব্যাখ্যা করুন।
4. Availability, Reliability, Durability ও Resilience-এর পার্থক্য ব্যাখ্যা করুন।

5. Latency SLI-তে গড়ের বদলে Percentile কেন ব্যবহার করা হয়?
6. Error Budget কী? এটি শেষ হলে কী করা উচিত?
7. Burn Rate ১, ২ ও ১০-এর অর্থ কী?
8. SLA কেন SLO-এর চেয়ে কম কঠোর হওয়া উচিত?
9. ৪xx ও ৫xx ত্রুটিকে SLI-তে একইভাবে গোনা উচিত কি? কেন?
10. একটি Checkout API-এর জন্য দুটি SLI লিখুন — একটি Availability, একটি Latency।

১৭.১৩ ব্যবহারিক অনুশীলন

1. TechNova API-এর জন্য ৯৯%, ৯৯.৯%, ৯৯.৯৯% Availability-এর ৩০ দিনের ও বার্ষিক Downtime হিসাব করুন। একটি টেবিল তৈরি করুন এবং ব্যাখ্যা করুন কোন সময়কালের হিসাব কখন ব্যবহার করবেন।
2. একটি কাল্পনিক সেবার জন্য একটি সম্পূর্ণ SLO নথি লিখুন: দুটি SLI, লক্ষ্য, Error Budget, Burn Rate Alert নীতি, এবং Release নিয়ম। SLO মানটি কেন বেছে নিয়েছেন তার যুক্তি লিখুন — কোনো সর্বজনীন মান ধার করবেন না।
3. PromQL দিয়ে একটি Availability SLI কুয়েরি লিখুন এবং একটি Burn Rate Alert-এর নিয়ম লিখুন (Fast Burn ও Slow Burn আলাদা করে)। Lab পরিবেশে পরীক্ষা করুন।

অধ্যায় ১৮ • PART IV — Site Reliability Engineering

Chapter 18 — Monitoring, Alerting and On-call

১৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Monitoring ও Observability-এর পার্থক্য ব্যাখ্যা করতে পারবেন এবং একটি সেবার জন্য Monitoring কৌশল নকশা করতে পারবেন। আপনি Metrics, Logs ও Traces — তিনটি স্তম্ভের কাজ ও সীমা বুঝবেন। আপনি Golden Signals, RED Method ও USE Method প্রয়োগ করে কী পরিমাপ করবেন তা ঠিক করতে পারবেন। আপনি Prometheus, Grafana ও Alertmanager দিয়ে একটি কার্যকর Alerting ব্যবস্থা বানাতে পারবেন, Alert Fatigue এড়াতে পারবেন, এবং On-call পরিচালনার জন্য Escalation Policy নকশা করতে পারবেন। সবচেয়ে গুরুত্বপূর্ণভাবে, আপনি শিখবেন ভালো Alert কাকে বলে: এমন Alert যা কার্যকর, সময়োপযোগী, এবং যার প্রতিক্রিয়া স্পষ্ট।

- Monitoring বনাম Observability ব্যাখ্যা করা।
- Golden Signals, RED ও USE Method প্রয়োগ করা।
- Prometheus, Grafana ও Alertmanager কনফিগার করা।
- Alert Quality মূল্যায়ন করা ও Alert Fatigue কমানো।
- On-call Rotation ও Escalation Policy নকশা করা।

১৮.২ প্রাক-প্রয়োজনীয় পুনরালোচনা

আগের অধ্যায়ে আমরা SLI ও SLO শিখেছি — কী পরিমাপ করব তা ঠিক করেছি। এই অধ্যায়ে আমরা শিখব কীভাবে পরিমাপ করব এবং কখন ডাকব। মনে রাখুন, Metrics বলতে বোঝায় সংখ্যাগত পরিমাপ — যেমন প্রতি সেকেন্ডে অনুরোধের সংখ্যা, CPU ব্যবহার, Latency-এর বণ্টন। Logs বলতে বোঝায় ঘটনার লিখিত রেকর্ড — যেমন একটি ত্রুটির Stack Trace। Traces বলতে বোঝায় একটি অনুরোধের যাত্রা — ফ্রন্টএন্ড থেকে API, API থেকে ডেটাবেস পর্যন্ত প্রতিটি ধাপে কত সময় লেগেছে। এই তিনটি একসঙ্গে Observability-এর তিন স্তম্ভ। আরেকটি ধারণা মনে রাখুন: Percentile — p95 ও p99 Latency বোঝার জন্য এটি অপরিহার্য। TechNova-এর প্রসঙ্গে ভাবুন: একজন ব্যবহারকারী Checkout-এ ক্লিক করেন — সেই অনুরোধটি React থেকে Express API-তে যায়, API MongoDB-তে লেখে, পেমেন্ট গেটওয়েকে ডাকে, তারপর উত্তর ফেরত দেয়। এই পুরো যাত্রাটি দেখতে পারলেই আমরা বুঝতে পারব কোথায় সময় নষ্ট হচ্ছে।

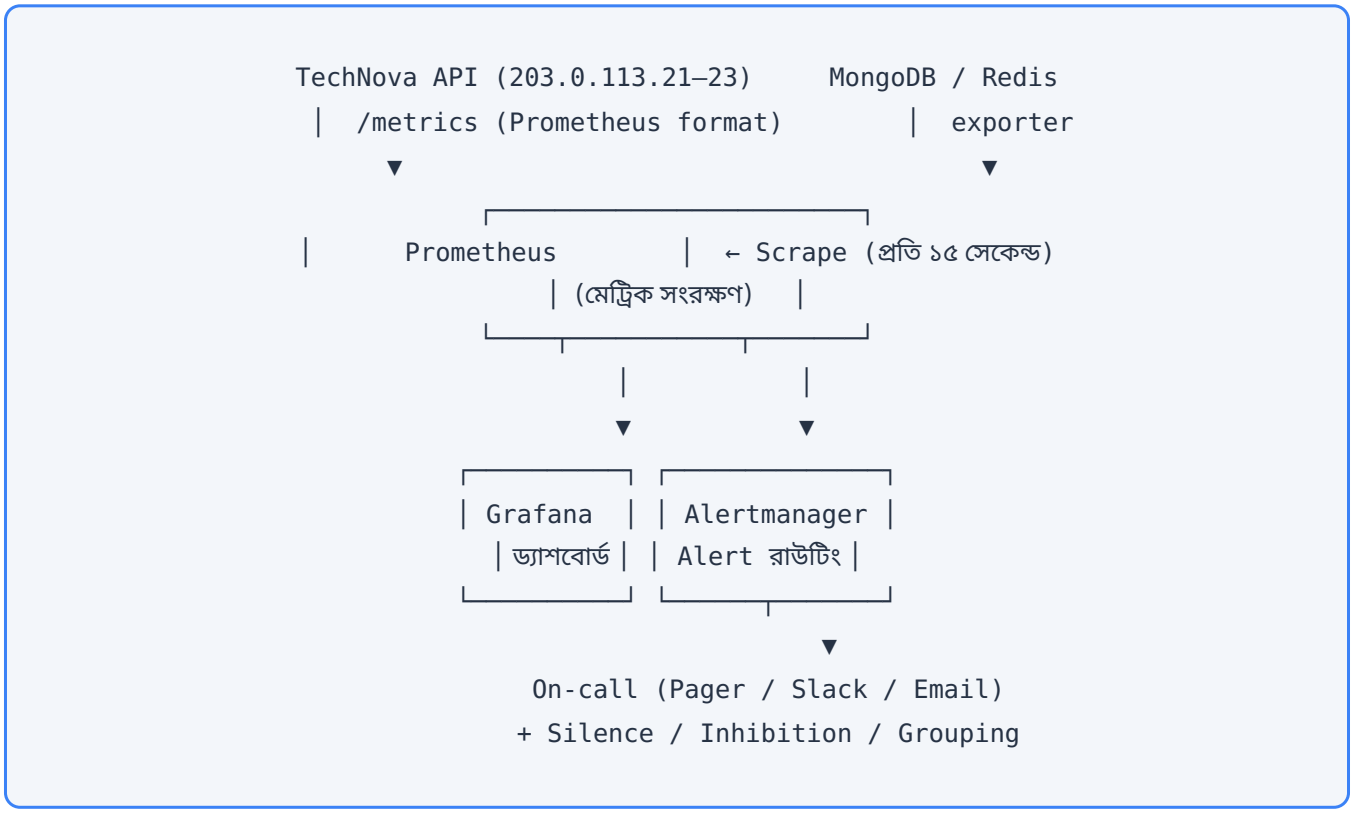
১৮.৩ প্রথম নীতি থেকে ব্যাখ্যা: দেখতে না পারলে ঠিক করতে পারবেন না

একটি উৎপাদন সিস্টেমের মূল সমস্যাটি হলো এটি একটি কালো বাক্সের মতো: ভিতরে কী হচ্ছে তা সরাসরি দেখা যায় না। ব্যবহারকারী বলেন সেবা ধীর, কিন্তু কেন ধীর তা কেউ জানে না। Monitoring এই কালো বাক্সে জানালা খোলে। কিন্তু Monitoring ও Observability এক জিনিস নয় — এই পার্থক্যটি বোঝা গুরুত্বপূর্ণ। Monitoring বলতে বোঝায় আগে থেকে ঠিক করা প্রশ্নের উত্তর জানা: CPU কত? Error Rate কত? আমরা জানি কী দেখতে চাই, তাই ড্যাশবোর্ড বানাই ও Alert বসাই। Observability বলতে বোঝায় আগে থেকে না জানা প্রশ্নেরও উত্তর দিতে পারা: কেন এই একটি ব্যবহারকারীর Checkout ব্যর্থ হচ্ছে? কেন শুধু একটি অঞ্চলে Latency বাড়ছে? Observability মানে সিস্টেমের ভিতরের অবস্থা বাইরে থেকে অনুমান করা যায় — যথেষ্ট তথ্য সংগ্রহ করা আছে বলে যেকোনো প্রশ্ন করা যায়। Monitoring ছাড়া Observability হয় না, কিন্তু শুধু Monitoring থাকলেও Observability হয় না। একটি সিস্টেম Observable হয় তখনই যখন তার Metrics, Logs ও Traces একসঙ্গে যুক্ত থাকে — একটি Trace থেকে সংশ্লিষ্ট Log-এ যাওয়া যায়, একটি Metric থেকে সংশ্লিষ্ট Trace-এ যাওয়া যায়।

কী পরিমাপ করবেন তা ঠিক করার জন্য দুটি বিখ্যাত পদ্ধতি আছে। Golden Signals — Google SRE-এর চারটি সংকেত: Latency (সাড়া দিতে কত সময় লাগে), Traffic (কত অনুরোধ আসছে), Errors (কতটি ব্যর্থ হচ্ছে), Saturation (সিস্টেম কতটা পূর্ণ — CPU, মেমরি, ডিস্ক, কিউ)। এই চারটি দেখলে ব্যবহারকারীর অভিজ্ঞতা ও সিস্টেমের স্বাস্থ্য — দুটোই বোঝা যায়। RED Method — Request Rate, Error Rate, Duration — এটি সেবা-কেন্দ্রিক, প্রতিটি মাইক্রোসার্ভিসের জন্য প্রযোজ্য। USE Method — Utilization, Saturation, Errors — এটি সম্পদ-কেন্দ্রিক, প্রতিটি সার্ভার বা ডেটাবেসের জন্য প্রযোজ্য। TechNova-তে RED দিয়ে API-এর প্রতিটি Endpoint দেখুন, USE দিয়ে MongoDB ও সার্ভার দেখুন, আর Golden Signals দিয়ে পুরো সেবার স্বাস্থ্য দেখুন। এই তিনটি পদ্ধতি একসঙ্গে ব্যবহার করলে কিছুই বাদ পড়ে না।

> **CONCEPT:** Monitoring মানে আগে থেকে ঠিক করা প্রশ্নের উত্তর জানা। Observability মানে যেকোনো প্রশ্নের উত্তর দিতে পারা — এমনকি যা আগে ভাবা হয়নি। Metrics বলে কী হচ্ছে, Logs বলে কেন হচ্ছে, Traces বলে কোথায় হচ্ছে। তিনটি একসঙ্গে থাকলেই একটি সিস্টেম সত্যিকারের Observable হয়।

১৮.৪ স্থাপত্য: Prometheus, Grafana ও Alertmanager প্রবাহ



এই স্থাপত্যের মূল নীতিটি হলো Pull ভিত্তিক সংগ্রহ: Prometheus নিজে গিয়ে প্রতিটি সেবা থেকে মেট্রিক টেনে আনে, সেবা নিজে থেকে পাঠায় না। এর সুবিধা হলো কোনো সেবা বন্ধ হয়ে গেলে Prometheus তা বুঝতে পারে — Scrape ব্যর্থ হয়। Grafana হলো দৃশ্যমান স্তর — ড্যাশবোর্ড, গ্রাফ, SLO প্রদর্শন। Alertmanager হলো সিদ্ধান্ত স্তর — কোন Alert কাকে, কখন, কীভাবে পাঠাবে, একই Alert বারবার পাঠাবে না, এবং রক্ষণাবেক্ষণের সময় Alert চুপ করাবে (Silence)। এই তিনটি আলাদা রাখার কারণ হলো প্রতিটি স্তর স্বাধীনভাবে বদলানো যায় — Grafana বদলালেও Alerting থেমে যায় না।

১৮.৫ অভ্যন্তরীণ কর্মনীতি: Alert Quality ও Alert Fatigue

সবচেয়ে খারাপ Monitoring ব্যবস্থা হলো সেটি যা সবসময় চিৎকার করে। Alert Fatigue বলতে বোঝায় এত বেশি Alert আসে যে প্রকৌশলীরা আর সেগুলোকে গুরুত্ব দেন না — আসল বিপদও উপেক্ষা করেন। Alert Fatigue-এর কারণগুলো

চেনা: অপ্রয়োজনীয় Alert (যেমন CPU ৮০% ছুঁলেই ডাকা, যদিও তাতে ব্যবহারকারীর কোনো ক্ষতি হচ্ছে না), অস্পষ্ট Alert (কী করতে হবে বলা নেই), পুনরাবৃত্ত Alert (একই সমস্যায় বারবার ডাকা), এবং ভুল থ্রেশহোল্ড (সাধারণ ওঠানামায় Alert বাজে)। ভালো Alert-এর চারটি গুণ আছে: এটি কার্যকর (Actionable) — প্রাপক জানে কী করতে হবে; এটি জরুরি (Urgent) — এখনই ব্যবস্থা না নিলে ব্যবহারকারীর ক্ষতি হবে; এটি সঠিক সময়ে আসে — সমস্যা শুরুর শীঘ্রই, কিন্তু অস্থায়ী ঝাঁকুনিতে নয়; এবং এর সঙ্গে Runbook যুক্ত থাকে — ধাপে ধাপে কী করতে হবে তা লেখা।

একটি ব্যবহারিক নিয়ম: Alert দুই ভাগে ভাগ করুন। Paging Alert — যা On-call-কে ঘুম থেকে ডাকে — শুধু সেগুলোই হওয়া উচিত যেখানে ব্যবহারকারীর ক্ষতি হচ্ছে বা হতে যাচ্ছে, এবং এখনই মানুষের হস্তক্ষেপ দরকার। Ticket Alert — যা দিনের বেলায় দেখা যায় — সেগুলো হলো ভবিষ্যৎ ঝুঁকি, যেমন ডিস্ক ৭০% পূর্ণ, বা Burn Rate ধীরে বাড়ছে। যদি একটি Alert-এ কেউ কখনো কোনো ব্যবস্থা না নেয়, তবে সেটি Alert নয় — সেটি একটি ড্যাশবোর্ড প্যানেল হওয়া উচিত। Alert-এর জন্য অপেক্ষা সময় (For Duration) ব্যবহার করুন — যেমন Error Rate ৫ মিনিট ধরে বেশি থাকলেই Alert, এক মিনিটের ঝাঁকুনিতে নয়। আর প্রতিটি Alert-এর পরে পর্যালোচনা করুন: এটি কি কার্যকর ছিল? না হলে থ্রেশহোল্ড বা শর্ত বদলান।

১৮.৬ বাস্তব কমান্ড ও কনফিগারেশন

Prometheus-এ TechNova API-এর জন্য একটি ভালো Alert নিয়ম:

```
groups:
- name: technova-api
  rules:
  - alert: TechnovaHighErrorRate
    expr: |
      sum(rate(http_requests_total{job="technova-api", status=~"5.."}[5m]))
      /
      sum(rate(http_requests_total{job="technova-api"}[5m])) > 0.05
    for: 5m
    labels: { severity: page }
    annotations:
      summary: "TechNova API error rate > 5% for 5 minutes"
      runbook: "https://technova.example/runbooks/api-error-rate"
  - alert: TechnovaDiskFilling
    expr: (1 - node_filesystem_avail_bytes / node_filesystem_size_bytes) > 0.8
    for: 15m
    labels: { severity: ticket }
    annotations:
      summary: "Disk > 80% full on {{ $labels.instance }}"
      runbook: "https://technova.example/runbooks/disk-full"
```

Alertmanager-এ রাউটিং — Severity অনুযায়ী আলাদা গন্তব্য, আর একই Alert-এর পুনরাবৃত্তি দমন:

```
route:
  group_by: [alertname, instance]
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  receiver: default
  routes:
  - match: { severity: page }
    receiver: oncall-pager
  - match: { severity: ticket }
    receiver: team-slack
receivers:
- name: oncall-pager
```

```
  webhook_configs:
  - url: "https://pager.technova.example/webhook/CHANGE-ME-TOKEN"
- name: team-slack
  slack_configs:
  - api_url: "https://hooks.slack.com/services/CHANGE-ME-WEBHOOK"
    channel: "#technova-alerts"
```

On-call Rotation ও Escalation Policy-এর একটি নথি:

```
On-call Rotation (TechNova SRE):
- সাপ্তাহিক rotation, সোমবার ১০:০০ +06 থেকে পরের সোমবার ১০:০০ পর্যন্ত
- Primary on-call: প্রথম ১৫ মিনিটের মধ্যে সাড়া দেবে
- Secondary on-call: Primary সাড়া না দিলে ১৫ মিনিট পর ডাকা হবে
- Manager escalation: Severity 1 ইনসিডেন্টে ৩০ মিনিট পর
- Handover: সোমবার সকালের বৈঠকে খোলা Alert ও চলমান কাজ হস্তান্তর
- ছুটি: rotation-এর আগে অন্তত ১ সপ্তাহ আগে জানাতে হবে; বিকল্প ঠিক করে যেতে হবে
```

দৈনন্দিন পরীক্ষার কমান্ড:

```
curl -s http://203.0.113.21:9090/-/healthy
curl -s http://203.0.113.21:3000/metrics | grep -E 'http_requests_total|
process_resident_memory_bytes' | head
systemctl status prometheus grafana-server alertmanager --no-pager
```

১৮.৭ উৎপাদন পরিস্থিতি

পরিস্থিতি ১ — রাত ৩টায় Paging Alert, কিন্তু কিছু করার নেই। CPU ৯০% ছুঁয়েছে বলে On-call ঘুম থেকে উঠলেন, কিন্তু ব্যবহারকারীর কোনো সমস্যা হচ্ছে না — ট্র্যাফিক বেশি, সার্ভার সামলাচ্ছে। এই Alert-টি ভুল — এটি Ticket হওয়া উচিত ছিল, Page নয়। শিক্ষা: ব্যবহারকারীর ক্ষতি না হলে Page করবেন না। **পরিস্থিতি ২ — Alert এলো, কিন্তু কেউ জানে না কী করতে হবে।** MongoDB Replication Lag Alert বাজল, কিন্তু Runbook নেই। On-call প্রকৌশলী আধ ঘণ্টা ধরে খুঁজলেন কী করতে হবে। শিক্ষা: Runbook ছাড়া Alert অসম্পূর্ণ। **পরিস্থিতি ৩ — নীরব ব্যর্থতা।** পেমেন্ট গেটওয়ের একটি Endpoint ধীরে ধীরে ব্যর্থ হচ্ছে, কিন্তু কোনো Alert নেই — কারণ কেউ সেই Endpoint-এর জন্য SLI ঠিক করেনি। ব্যবহারকারীরা অভিযোগ করার পর ধরা পড়ল। শিক্ষা: প্রতিটি গুরুত্বপূর্ণ User Journey-এর জন্য Alert থাকতে হবে — শুধু সার্ভারের স্বাস্থ্য নয়।

পরিস্থিতি ৬ — Log-এর মাত্রা ও খরচ। TechNova-এর API প্রতিটি অনুরোধে Debug Log লিখছিল — দিনে ৫০ গিগাবাইট Log জমছিল, সংরক্ষণের খরচ বাড়ছিল, আর আসল ত্রুটি খুঁজে পাওয়া কঠিন হচ্ছিল। দল নিয়ম করল: উৎপাদনে Info মাত্রা, Debug শুধু Lab-এ; প্রতিটি Log-এ Request ID, ব্যবহারকারীর ID নয় — গোপনীয়তা রক্ষায়; আর Log-এর Retention ৩০ দিন। ফল: Log-এর আকার এক-দশমাংশ হলো, আর ত্রুটি খোঁজা সহজ হলো। শিক্ষা: বেশি Log মানে ভালো Observability নয় — সঠিক Log মানে ভালো Observability। প্রতিটি Log লাইনের একটি উদ্দেশ্য থাকতে হবে: কে এটি পড়বে, কখন, কেন?

পরিস্থিতি ৭ — On-call Burnout প্রতিরোধ। TechNova-এর একজন SRE তিন সপ্তাহ ধরে On-call ছিলেন — কারণ অন্যরা ছুটিতে ছিলেন। চতুর্থ সপ্তাহে তিনি ভুল সিদ্ধান্ত নিলেন — ক্লান্তিতে। দল নিয়ম করল: কেউ টানা দুই সপ্তাহের বেশি On-call থাকবে না; On-call সপ্তাহের পরের দিন ছুটি; আর On-call-এর সময় নতুন ফিচারের কাজ নয়। এই নিয়মগুলো Burnout প্রতিরোধ করে — আর Burnout প্রতিরোধ করা নির্ভরযোগ্যতার অংশ: ক্লান্ত প্রকৌশলী ভুল করেন, ভুল ইনসিডেন্ট বাড়ায়। On-call টেকসই না হলে SRE টেকসই নয়।

পরিস্থিতি ৮ — Alert-এর মাসিক পর্যালোচনা। প্রতি মাসে দল সব Alert-এর তালিকা দেখে: কোন Alert কতবার বেজেছে, কতবার কার্যকর ছিল, কতবার উপেক্ষা করা হয়েছে। যে Alert তিন মাসে একবারও কার্যকর হয়নি — তা সরিয়ে দেওয়া হয় বা Dashboard-এ নামানো হয়। যে Alert বারবার বেজেছে — তার থ্রেশহোল্ড বা মূল কারণ ঠিক করা হয়। এই পর্যালোচনার ফলে Alert-এর সংখ্যা অর্ধেক হলো, আর প্রতিটি Alert-এর প্রতি আস্থা বাড়ল। শিক্ষা: Alerting একটি জীবন্ত ব্যবস্থা — তা নিয়মিত পরিষ্কার করতে হয়, নইলে তা জঞ্জালে ভরে যায়।

পরিস্থিতি ৪ — Dashboard-এর নকশা। TechNova-এর প্রথম Dashboard-এ ৩০টি গ্রাফ ছিল — কেউ তা দেখত না, কারণ কী গুরুত্বপূর্ণ তা বোঝা যেত না। দল Dashboard-টি আবার বানাল: উপরে মাত্র চারটি Golden Signals, তার নিচে SLO ও Error Budget, তার নিচে বিস্তারিত। নিয়মটি হলো: একটি Dashboard-এ একজন ব্যক্তি ৩০ সেকেন্ডে বুঝতে পারবেন সেবা সুস্থ কি না। যদি না পারেন, তবে Dashboard-টি ব্যর্থ। আর প্রতিটি গ্রাফের সঙ্গে একটি প্রশ্ন যুক্ত থাকতে হবে: এই গ্রাফ খারাপ দেখালে কী করবেন? উত্তর না থাকলে গ্রাফটি সরিয়ে দিন।

পরিস্থিতি ৫ — Trace থেকে মূল কারণ। Checkout ধীর — কিন্তু কোন ধাপে? Metric শুধু বলে মোট সময় বেশি। Trace দেখা গেল: API ৫০ মিলিসেকেন্ড, MongoDB ৪০ মিলিসেকেন্ড, কিন্তু পেমেন্ট গেটওয়ে ১,৮০০ মিলিসেকেন্ড। সমস্যাটি TechNova-তে নয়, বাহ্যিক সেবায়। সমাধান: Timeout কমানো, Circuit Breaker বসানো, এবং ব্যবহারকারীকে স্পষ্ট বার্তা দেওয়া — পেমেন্ট যাচাই করা হচ্ছে, অনুগ্রহ করে অপেক্ষা করুন। Trace ছাড়া এই সিদ্ধান্ত নেওয়া যেত না — দল হয়তো ভুল করে API অপ্টিমাইজ করত।

১৮.৮ নিরাপত্তা ও খরচ বিবেচনা

Monitoring ব্যবস্থা নিজেই একটি নিরাপত্তা ঝুঁকি হতে পারে: Metrics Endpoint-এ সংবেদনশীল তথ্য (যেমন ব্যবহারকারীর ইমেইল, টোকেন) থাকা উচিত নয়। /metrics Endpoint শুধু অভ্যন্তরীণ নেটওয়ার্ক থেকে পৌঁছানো উচিত, ইন্টারনেট থেকে নয়। Alert Webhook URL-গুলো Secret — সেগুলো কনফিগারেশন ফাইলে সরাসরি না লিখে Secret Manager থেকে নিন, এবং এখানে শুধু CHANGE-ME-... Placeholder ব্যবহার করুন। Grafana ড্যাশবোর্ডে সংবেদনশীল ডেটা (যেমন পেমেন্ট তথ্য) দেখাবেন না। খরচের দিকে, Log সংরক্ষণ সবচেয়ে ব্যয়বহুল হতে পারে — সব Log একই মেয়াদে রাখবেন না। Debug Log ৭ দিন, Info Log ৩০ দিন, Audit Log ৯০ দিন — এইভাবে Retention স্তর ঠিক করুন। High-cardinality Metric (যেমন প্রতিটি ব্যবহারকারীর জন্য আলাদা Label) Prometheus-কে ধীর করে দেয় ও খরচ বাড়ায় — Label-এর সংখ্যা সীমিত রাখুন।

১৮.৯ সাধারণ ভুল

- সবকিছুর জন্য Alert বানানো — ফলে Alert Fatigue।
- CPU বা মেমরির মতো সার্ভার মেট্রিকের জন্য Paging Alert — ব্যবহারকারীর অভিজ্ঞতা নয়।
- Runbook ছাড়া Alert দেওয়া।
- For Duration না দেওয়া — অস্থায়ী ঝাঁকুনিতে Alert বাজা।
- Logs-এ সংবেদনশীল তথ্য রাখা।
- High-cardinality Label ব্যবহার করে Prometheus ভারী করা।
- On-call-কে ছুটি বা বিশ্রাম না দেওয়া — Burnout অনিবার্য।
- Alert-এর পর্যালোচনা না করা — একই ভুল Alert বারবার বাজতে থাকা।

১৮.১০ সমস্যা সমাধান (Troubleshooting)

TROUBLESHOOTING: Alert বাজছে না অথচ সমস্যা হচ্ছে — প্রথমে Prometheus-এ কুয়েরিটি হাতে চালিয়ে দেখুন ফল আসে কি না। Scrape ব্যর্থ হচ্ছে কি না দেখুন (Targets পৃষ্ঠায়)। Alertmanager-এ Alertটি Inhibited বা Silenced কি না পরীক্ষা করুন। আর Alert বাজছে অথচ সমস্যা নেই — তাহলে থ্রেশহোল্ড, উইন্ডো, বা For Duration পর্যালোচনা করুন।

Grafana ড্যাশবোর্ড খালি দেখালে প্রথমে Data Source সংযোগ পরীক্ষা করুন, তারপর সময়সীমা (Time Range) — হয়তো ডেটা আছে কিন্তু ভিন্ন সময়ে। Prometheus-এ মেট্রিক না পেলে Exporter চলছে কি না, পোর্ট খোলা কি না, এবং Scrape কনফিগারেশনে Job নাম ঠিক আছে কি না দেখুন। On-call-এ কেউ সাড়া না দিলে Escalation Policy পরীক্ষা করুন — Secondary ও Manager-এর নম্বর হালনাগাদ আছে কি না।

১৮.১১ সারসংক্ষেপ

Monitoring বলে কী হচ্ছে, Observability বলে কেন হচ্ছে। Golden Signals, RED ও USE — তিনটি পদ্ধতি একসঙ্গে ব্যবহার করলে কিছুই বাদ পড়ে না। Prometheus সংগ্রহ করে, Grafana দেখায়, Alertmanager সিদ্ধান্ত নেয় কাকে ডাকবে। ভালো Alert কার্যকর, জরুরি, সময়োপযোগী ও Runbook-যুক্ত। আর On-call টেকসই হয় তখনই যখন Rotation ন্যায্য, Escalation স্পষ্ট, এবং Alert Fatigue নিয়ন্ত্রণে থাকে। পরের অধ্যায়ে আমরা শিখব Alert-এর পর কী হয় — ইনসিডেন্ট কীভাবে সামলানো হয়, এবং কীভাবে তা থেকে শেখা যায়।

১৮.১২ পর্যালোচনা প্রশ্ন

1. Monitoring ও Observability-এর পার্থক্য কী?
2. Metrics, Logs ও Traces-এর প্রত্যেকটি কী প্রশ্নের উত্তর দেয়?
3. Golden Signals-এর চারটি সংকেত কী কী? প্রত্যেকটির একটি উদাহরণ দিন।
4. RED Method ও USE Method কখন ব্যবহার করবেন?
5. ভালো Alert-এর চারটি গুণ কী কী?
6. Paging Alert ও Ticket Alert-এর পার্থক্য কী?
7. Alert Fatigue কাকে বলে? এর তিনটি কারণ ও প্রতিকার লিখুন।
8. Alertmanager-এ Grouping, Inhibition ও Silencing কী কাজে লাগে?
9. Escalation Policy কেন দরকার? একটি উদাহরণ নকশা করুন।
10. High-cardinality Metric কেন ক্ষতিকর?

১৮.১৩ ব্যবহারিক অনুশীলন

1. TechNova API-এর জন্য Golden Signals ড্যাশবোর্ডের পরিকল্পনা লিখুন: প্রতিটি সংকেতের জন্য কোন মেট্রিক, কোন PromQL কুয়েরি, এবং কোন থ্রেশহোল্ড ব্যবহার করবেন।
2. দুটি Alert নিয়ম লিখুন — একটি Paging, একটি Ticket — প্রতিটির সঙ্গে Runbook-এর রূপরেখা। একটি ভুল Alert-কে (যেমন CPU ৮০%) কীভাবে সংশোধন করবেন তা ব্যাখ্যা করুন।

3. একটি সাপ্তাহিক On-call Rotation ও Escalation Policy নথি তৈরি করুন যাতে Handover, ছুটি ও Burnout প্রতিরোধের নিয়ম থাকে।

অধ্যায় ১৯ • PART IV — Site Reliability Engineering

Chapter 19 — Incident Management and RCA

১৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি একটি ইনসিডেন্টের পুরো জীবনচক্র সামলাতে পারবেন — শনাক্তকরণ থেকে পুনরুদ্ধার, তারপর শিক্ষা পর্যন্ত। আপনি Severity শ্রেণিবিন্যাস করতে পারবেন, সঠিক যোগাযোগ কাঠামো তৈরি করতে পারবেন, এবং Mitigation ও Recovery-এর মধ্যে পার্থক্য বুঝবেন। আপনি Root Cause Analysis (RCA) করতে পারবেন — Five Whys পদ্ধতিসহ — এবং Blameless Postmortem লিখতে পারবেন। আপনি Corrective Action ঠিক করতে পারবেন যাতে একই ভুল আবার না হয়, এবং Incident Runbook তৈরি করতে পারবেন। সবচেয়ে গুরুত্বপূর্ণভাবে, আপনি Incident Response ও Problem Management-এর পার্থক্য বুঝবেন: একটি হলো আগুন নেভানো, অন্যটি হলো আগুন লাগার কারণ দূর করা।

- Incident Lifecycle-এর প্রতিটি ধাপ ব্যাখ্যা ও প্রয়োগ করা।
- Severity শ্রেণিবিন্যাস করা ও যোগাযোগ পরিচালনা করা।
- Five Whys দিয়ে RCA করা।
- Blameless Postmortem লেখা ও Corrective Action ঠিক করা।
- Incident Runbook তৈরি করা।

১৯.২ প্রাক-প্রয়োজনীয় পুনরালোচনা

আগের অধ্যায়ে আমরা শিখেছি Alert কীভাবে আসে। এই অধ্যায়ে আমরা শিখব Alert আসার পর কী হয়। মনে রাখুন, SLO ও Error Budget-এর ধারণা — ইনসিডেন্টের গুরুত্ব নির্ধারণে এগুলো কাজে লাগে। Monitoring থেকে আমরা জানি সমস্যাটি কোথায়, কিন্তু ইনসিডেন্ট ব্যবস্থাপনা হলো মানুষের সমন্বয়: কে কী করবে, কাকে জানাতে হবে, কখন কী বলতে হবে। TechNova-এর প্রসঙ্গে ভাবুন: Checkout ব্যর্থ হচ্ছে — এটি শুধু একটি প্রযুক্তিগত সমস্যা নয়, এটি একটি ব্যবসায়িক সংকট: প্রতিটি মিনিটে অর্ডার হারাচ্ছে, গ্রাহকরা অভিযোগ করছেন, সামাজিক মাধ্যমে কথা ছড়াচ্ছে। ইনসিডেন্ট ব্যবস্থাপনা এই চাপের মধ্যে শৃঙ্খলা আনে — যাতে সবাই একই তথ্য জানে, একই লক্ষ্যে কাজ করে, এবং ভুল সিদ্ধান্ত কম হয়।

১৯.৩ প্রথম নীতি থেকে ব্যাখ্যা: ইনসিডেন্টে শৃঙ্খলাই সবকিছু

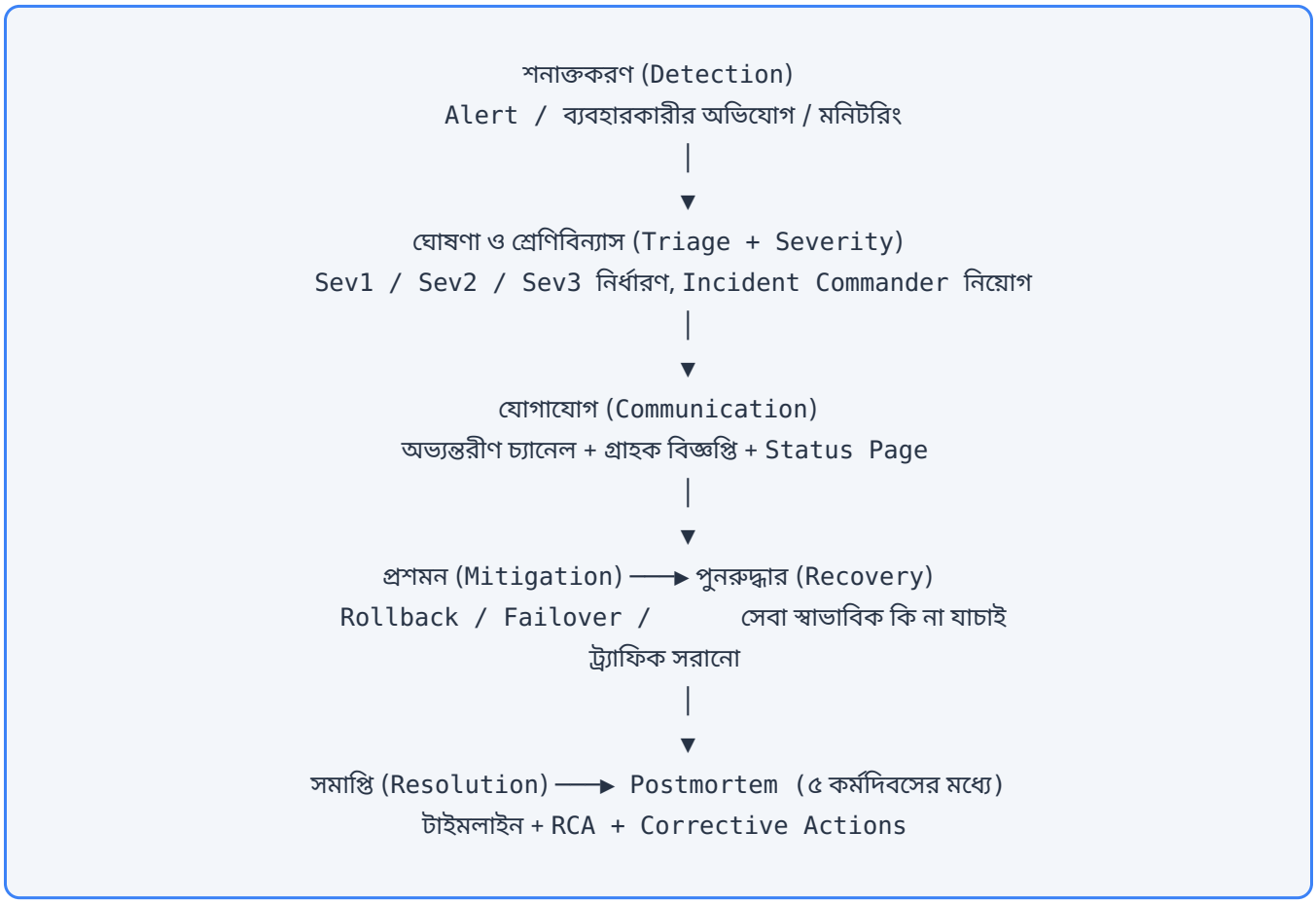
একটি ইনসিডেন্টের সময় সবচেয়ে বড় শত্রু প্রযুক্তি নয় — বিশৃঙ্খলা। পাঁচজন প্রকৌশলী পাঁচটি ভিন্ন জিনিস পরীক্ষা করছেন, কেউ জানে না কে দায়িত্বে আছে, ব্যবস্থাপক বারবার জিজ্ঞাসা করছেন কখন ঠিক হবে, আর গ্রাহকরা কোনো তথ্য পাচ্ছেন না। এই বিশৃঙ্খলা পুনরুদ্ধারকে ধীর করে, ভুল সিদ্ধান্ত বাড়ায়, এবং একই কাজ দুবার হয়। Incident Management-এর উদ্দেশ্য হলো এই বিশৃঙ্খলায় কাঠামো আনা। এর মূল নীতিগুলো সহজ: একজন Incident Commander থাকবে — তিনি সিদ্ধান্ত নেবেন, কিন্তু নিজে হাতে কোড ঠিক করবেন না। যোগাযোগের একটি মাত্র চ্যানেল থাকবে — সব তথ্য সেখানেই আসবে। কাজ ভাগ করা থাকবে — কেউ তদন্ত করবে, কেউ যোগাযোগ করবে, কেউ নথি লিখবে। আর সবচেয়ে গুরুত্বপূর্ণ: প্রথম লক্ষ্য হলো Mitigation — ব্যবহারকারীর ক্ষতি থামানো — মূল কারণ খোঁজা পরে। একটি ভুল ধারণা হলো ইনসিডেন্টের সময়ই মূল কারণ বের করতে হবে। বাস্তবে, প্রথমে সেবা ফিরিয়ে আনুন — Rollback করুন, ট্র্যাফিক সরান, সার্ভার বাড়ান — তারপর শান্তভাবে কারণ খুঁজুন।

Incident Response ও Problem Management-এর পার্থক্য বোঝা জরুরি। Incident Response বলতে বোঝায় সেবা দ্রুত পুনরুদ্ধার করা — এখনই, এই মুহূর্তে। Problem Management বলতে বোঝায় একই সমস্যা আবার যাতে না হয় তার

জন্য মূল কারণ দূর করা — ধীরে, গভীরভাবে, স্থায়ীভাবে। একটি উদাহরণ: ডিস্ক পূর্ণ হয়ে API থেমে গেল — Incident Response হলো লগ মুছে জায়গা করা ও সেবা চালু করা। Problem Management হলো লগ রোটেশন ঠিক করা, সতর্কবার্তা বসানো, এবং ডিস্ক পর্যবেক্ষণ স্বয়ংক্রিয় করা — যাতে আগামী মাসে আবার একই ঘটনা না ঘটে। দুটোই দরকার, কিন্তু তাদের সময় ও লক্ষ্য আলাদা। Postmortem হলো এই দুটোর সেতু — ইনসিডেন্টের পর শেখা, এবং সেই শেখা থেকে স্থায়ী সংশোধন।

> **CONCEPT:** Mitigation মানে ব্যবহারকারীর ক্ষতি থামানো — সেবা ফিরিয়ে আনা, এমনকি মূল কারণ না জেনেও। Recovery মানে সেবা সম্পূর্ণ স্বাভাবিক অবস্থায় ফেরা। Resolution মানে ইনসিডেন্ট বন্ধ করা — কিন্তু Problem Management-এর কাজ তখনো বাকি থাকতে পারে।

১৯.৪ স্থাপত্য: ইনসিডেন্ট জীবনচক্র ও ভূমিকা



Severity	সংজ্ঞা	প্রতিক্রিয়া
Sev 1 — Critical	মূল সেবা বন্ধ; বেশিরভাগ ব্যবহারকারী ক্ষতিগ্রস্ত; রাজস্ব ক্ষতি	তাৎক্ষণিক, সবাই ডাকা হবে, ১৫ মিনিটে Commander
Sev 2 — Major	গুরুত্বপূর্ণ ফিচার ভাঙা; অংশবিশেষ ব্যবহারকারী ক্ষতিগ্রস্ত	১ ঘণ্টার মধ্যে, On-call + দল
Sev 3 — Minor	সীমিত প্রভাব; বিকল্প উপায় আছে	কর্মঘণ্টায়, টিকিট

Sev 4 — Low	প্রসাধনী বা ভবিষ্যৎ ঝুঁকি	নিয়মিত ব্যাকলগ

ভূমিকাগুলো স্পষ্ট রাখুন: Incident Commander — সিদ্ধান্ত ও সমন্বয়; Communications Lead — অভ্যন্তরীণ ও বাহ্যিক যোগাযোগ; Technical Lead — তদন্ত ও Mitigation; Scribe — টাইমলাইন ও সিদ্ধান্ত লিপিবদ্ধ করা। ছোট ইনসিডেন্টে একজন ব্যক্তি একাধিক ভূমিকা নিতে পারে, কিন্তু Commander ও Technical Lead একই ব্যক্তি না হওয়াই ভালো — Commander যদি নিজে কোড ঠিক করতে ব্যস্ত থাকেন, তবে সমন্বয় ভেঙে পড়ে।

১৯.৫ অভ্যন্তরীণ কর্মনীতি: Five Whys ও Blameless Postmortem

Root Cause Analysis (RCA) বলতে বোঝায় সমস্যার গভীরতম কারণ খুঁজে বের করা — শুধু সরাসরি কারণ নয়। Five Whys পদ্ধতিটি সহজ: কেন জিজ্ঞাসা করুন, উত্তরের উপর আবার কেন জিজ্ঞাসা করুন — পাঁচবার। উদাহরণ: Checkout ব্যর্থ হচ্ছে কেন? কারণ API ৫০০ ত্রুটি দিচ্ছে। কেন? কারণ MongoDB সংযোগ ব্যর্থ হচ্ছে। কেন? কারণ Connection Pool শেষ হয়ে গেছে। কেন? কারণ একটি ধীর কুয়েরি সংযোগ আটকে রাখছে। কেন? কারণ নতুন ডেপ্লয়ে একটি Index ছাড়া কুয়েরি যোগ করা হয়েছে, এবং পরীক্ষায় তা ধরা পড়েনি। এই শেষ উত্তরটি আসল মূল কারণ — শুধু প্রথম উত্তরটি জানলে আমরা শুধু সার্ভার রিস্টার্ট করতাম, আর সমস্যা আবার ফিরে আসত। Five Whys ব্যবহারের সময় সতর্ক থাকুন: এটি একটি সরল রেখা নয় — বাস্তবে কারণগুলো শাখা-প্রশাখা বিশিষ্ট হয়। তাই পাঁচবার কেন জিজ্ঞাসা করার পরও যদি একাধিক শাখা থাকে, তবে প্রতিটি শাখা অনুসরণ করুন। আর কখনো একজন ব্যক্তিকে মূল কারণ বানাবেন না — মানুষ ভুল করে, কিন্তু সিস্টেমের উচিত সেই ভুল ধরা।

Blameless Postmortem বলতে বোঝায় এমন পর্যালোচনা যেখানে দোষারোপ করা হয় না। এর যুক্তি ব্যবহারিক, নৈতিক নয়: মানুষ যদি ভয় পায় শাস্তি পাবে, তবে তারা সত্য লুকাবে — আর সত্য না জানলে সিস্টেম কখনো ভালো হবে না। একটি ভালো Postmortem-এ থাকে: টাইমলাইন (কখন কী হয়েছে), প্রভাব (কতজন ব্যবহারকারী, কতক্ষণ, কতটা ক্ষতি), মূল কারণ, কী ভালো হয়েছে, কী খারাপ হয়েছে, এবং Corrective Action — প্রতিটির দায়িত্ব ও সময়সীমাসহ। Corrective Action তিন ধরনের হতে পারে: প্রতিরোধমূলক (আবার যাতে না হয়), শনাক্তকরণমূলক (আবার হলে দ্রুত ধরা যাবে), এবং প্রশমনমূলক (আবার হলে ক্ষতি কম হবে)। প্রতিটি Action-এর মালিক ও তারিখ থাকতে হবে — নইলে Postmortem শুধু একটি নথি হয়ে থাকে, পরিবর্তন আসে না।

> **REAL WORLD:** একটি বড় বিভ্রাটের পর একটি দল দেখল একজন প্রকৌশলী ভুল কনফিগারেশন ডেপ্লয় করেছেন। দোষারোপের বদলে তারা জিজ্ঞাসা করল: কনফিগারেশন যাচাই করার কোনো স্বয়ংক্রিয় পরীক্ষা কেন ছিল না? কেন এক ক্লিকে উৎপাদনে যাওয়া যাচ্ছিল? ফলে তারা একটি Validation ধাপ ও Canary Deployment যোগ করল — আর একই ভুল আর কখনো হয়নি। মানুষটি নয়, সিস্টেমটি বদলানো হয়েছিল।

১৯.৬ বাস্তব কমান্ড ও কনফিগারেশন

ইনসিডেন্টের সময় দ্রুত অবস্থা জানার কমান্ড:

```
# সেবার অবস্থা ও সাম্প্রতিক ডেপ্লয় — কী পরিবর্তন হয়েছে?
kubectl -n technova rollout history deployment/technova-api
kubectl -n technova get pods -o wide | grep -v Running
systemctl status technova-api mongodb mongod --no-pager
tail -n 200 /var/log/technova-api/error.log
# MongoDB সংযোগ ও ধীর কুয়েরি
mongosh --eval 'db.currentOp({"active": true, "secs_running": {"$gt": 5}}).inprog.length' \
```

```
  "mongodb://203.0.113.31:27017/technova" -u CHANGE-ME-USER -p 'CHANGE-ME-PASSWORD' --
authenticationDatabase admin
```

দ্রুত Mitigation — Rollback:

```
kubectl -n technova rollout undo deployment/technova-api
kubectl -n technova rollout status deployment/technova-api --timeout=300s
```

Postmortem টেমপ্লেটের কাঠামো:

```
শিরোনাম: [তারিখ] TechNova Checkout বিভ্রাট — Sev 1
সারসংক্ষেপ: কী হয়েছে, কতক্ষণ, কতজন ক্ষতিগ্রস্ত
টাইমলাইন (সব সময় +06):
 ১৪:০২ Alert: Error rate > 5%
 ১৪:০৫ Sev 1 ঘোষণা, Commander নিয়োগ
 ১৪:১০ Mitigation: Rollback শুরু
 ১৪:২৫ সেবা পুনরুদ্ধার
প্রভাব: ২৩ মিনিট, ~১,২০০ ব্যর্থ Checkout
মূল কারণ: Index ছাড়া কুয়েরি; পরীক্ষায় ধরা পড়েনি
কী ভালো হয়েছে: Alert ২ মিনিটে এসেছে; Rollback দ্রুত হয়েছে
কী খারাপ হয়েছে: Status Page ১৫ মিনিট দেরিতে হালনাগাদ
Corrective Actions:
  [ ] ধীর কুয়েরি সতর্কবার্তা যোগ — মালিক: আয়েশা, তারিখ: ২০২৬-১১-২০
  [ ] ডেপ্লয়-পূর্ব Index পরীক্ষা — মালিক: রাকিব, তারিখ: ২০২৬-১১-২৭
```

১৯.৭ উৎপাদন পরিস্থিতি

পরিস্থিতি ১ — ডেপ্লয়ের পর Checkout ভেঙে গেল। Alert আসার ৩ মিনিটে Sev 1 ঘোষণা, Commander নিয়োগ, Status Page হালনাগাদ। Technical Lead দেখলেন নতুন সংস্করণে সমস্যা — ১০ মিনিটে Rollback, সেবা ফিরল। তারপর Postmortem: কেন পরীক্ষায় ধরা পড়েনি? উত্তর: Staging-এ ডেটার আকার ছোট ছিল, ধীর কুয়েরি ধরা পড়েনি। Corrective Action: Staging-এ বাস্তবসম্মত ডেটা ও ধীর কুয়েরি পরীক্ষা। **পরিস্থিতি ২ — ডেটাবেস ধীর, কিন্তু সেবা চলছে।** এটি Sev 2 — ব্যবহারকারীরা ধীরগতি পাচ্ছেন, কিন্তু কাজ হচ্ছে। Commander সিদ্ধান্ত নিলেন ধীর কুয়েরি চিহ্নিত করে Index যোগ করবেন, কিন্তু ব্যস্ত সময়ে নয় — ট্র্যাফিক কমলে। **পরিস্থিতি ৩ — ভুল Severity।** একটি ছোট সমস্যাকে Sev 1 বলা হলো, সবাই ডাকা হলো, কিন্তু আসলে প্রভাব ছিল সামান্য। ফলে দল ক্লান্ত হলো, আর পরের আসল Sev 1-এ সাড়া ধীর হলো। শিক্ষা: Severity প্রভাব দিয়ে ঠিক করুন, আতঙ্ক দিয়ে নয়।

পরিস্থিতি ৬ — Severity নিয়ে মতবিরোধ। একটি ইনসিডেন্টে ডেভেলপার বললেন Sev 3, ব্যবস্থাপক বললেন Sev 1 — কারণ একজন বড় গ্রাহক অভিযোগ করেছেন। Commander প্রভাব দেখে সিদ্ধান্ত নিলেন: ক্ষতিগ্রস্ত ব্যবহারকারী ২% — Sev 2। তিনি ব্যাখ্যা করলেন: Severity প্রভাব দিয়ে ঠিক হয়, গ্রাহকের গুরুত্ব দিয়ে নয় — বড় গ্রাহকের সমস্যাও যদি সীমিত হয়, তবে তা Sev 2, কিন্তু যোগাযোগে অগ্রাধিকার দেওয়া হবে। এই স্পষ্ট নিয়মের ফলে মতবিরোধ কমল। শিক্ষা: Severity-এর সংজ্ঞা আগে থেকে লিখে রাখুন — ইনসিডেন্টের সময় সংজ্ঞা নিয়ে তর্ক করবেন না।

পরিস্থিতি ৭ — Postmortem-এর সংস্কৃতি। TechNova-তে প্রথম Postmortem বৈঠকে সবাই চুপ ছিলেন — কেউ ভুল স্বীকার করতে চাননি। Facilitator নিয়ম বদলালেন: প্রথমে তিনি নিজের ভুল বললেন, তারপর প্রত্যেকে একটি করে জিনিস বললেন যা ভালো হয়েছে, তারপর একটি করে জিনিস যা বদলাতে হবে — কোনো নাম ছাড়া। ধীরে ধীরে সবাই খুলে বলতে শুরু করলেন। শিক্ষা: Blameless সংস্কৃতি ঘোষণায় আসে না — আচরণে আসে। নেতা যখন নিজের ভুল স্বীকার করেন, তখন অন্যরা নিরাপদ বোধ করেন।

পরিস্থিতি ৮ — Runbook-এর পরীক্ষা। একটি Runbook লেখা ছিল — কিন্তু কেউ তা পরীক্ষা করেনি। ইনসিডেন্টের সময় দেখা গেল Runbook-এর কমান্ডগুলো পুরনো — একটি স্ক্রিপ্টের পথ বদলে গেছে। দল নিয়ম করল: প্রতি তিন মাসে প্রতিটি গুরুত্বপূর্ণ Runbook Lab-এ পরীক্ষা করা হবে — যেন তা কাজ করে। শিক্ষা: Runbook একটি নথি নয় — একটি পরীক্ষিত পদ্ধতি। পরীক্ষা না করা Runbook-এর উপর ভরসা করা বিপজ্জনক।

পরিস্থিতি ৪ — যোগাযোগের টেমপ্লেট। একটি Sev 1 ইনসিডেন্টে Communications Lead প্রতি ৩০ মিনিটে একই কাঠামোতে হালনাগাদ দিলেন: কী হয়েছে, কারা ক্ষতিগ্রস্ত, কী করা হচ্ছে, পরের হালনাগাদ কখন। গ্রাহকরা অভিযোগ কম করলেন — কারণ তারা জানতেন কী হচ্ছে। অভ্যন্তরে, Commander একটি সিদ্ধান্ত লগ রাখলেন: কখন Rollback-এর সিদ্ধান্ত নেওয়া হলো, কেন নেওয়া হলো, কে নিয়েছে। Postmortem-এর সময় এই লগটিই সবচেয়ে মূল্যবান ছিল — স্মৃতির উপর নির্ভর করতে হয়নি। শিক্ষা: যোগাযোগ একটি কাঠামো — তা আগে থেকে ঠিক করে রাখুন, ইনসিডেন্টের সময় আবিষ্কার করবেন না।

পরিস্থিতি ৫ — একই ইনসিডেন্ট দ্বিতীয়বার। তিন মাস আগে ডিস্ক পূর্ণ হয়ে সেবা থেমেছিল। Postmortem-এ Corrective Action লেখা হয়েছিল — কিন্তু কেউ তা করেনি, কারণ তার মালিক চলে গেছেন। তিন মাস পর একই ঘটনা আবার ঘটল। এবার দল নিয়ম বদলাল: প্রতিটি Corrective Action স্প্রিন্টের কাজ হিসেবে নথিভুক্ত হবে, মালিক ছাড়া Action গ্রহণ করা হবে না, এবং প্রতি মাসে খোলা Action পর্যালোচনা হবে। শিক্ষা: Postmortem-এর মূল্য Action-এ — Action না হলে Postmortem শুধু একটি গল্প।

১৯.৮ নিরাপত্তা ও খরচ বিবেচনা

ইনসিডেন্টের সময় নিরাপত্তা শিথিল করবেন না: চাপের মধ্যে কেউ যাতে Production ডেটাবেসে সরাসরি লিখতে না পারে তার জন্য অ্যাক্সেস নিয়ন্ত্রণ বজায় রাখুন — জরুরি অ্যাক্সেসও লগ করুন ও সময়সীমা দিন। Postmortem-এ সংবেদনশীল তথ্য (পাসওয়ার্ড, গ্রাহকের ব্যক্তিগত তথ্য) লিখবেন না। যোগাযোগের সময় গ্রাহককে সত্য বলুন, কিন্তু অতিরিক্ত প্রযুক্তিগত বিবরণ দিয়ে বিভ্রান্ত করবেন না — কী হয়েছে, কী প্রভাব, কখন ঠিক হবে — এই তিনটি যথেষ্ট। খরচের দিকে, প্রতিটি Sev 1 ইনসিডেন্টের ব্যবসায়িক খরচ হিসাব করুন — হারানো রাজস্ব, ক্ষতিপূরণ, দলের সময় — এবং সেই সংখ্যা দিয়ে নির্ভরযোগ্যতায় বিনিয়োগের যুক্তি দিন। Postmortem-এর Corrective Action-এর খরচও হিসাব করুন — সব Action সমান গুরুত্বপূর্ণ নয়।

১৯.৯ সাধারণ ভুল

- Mitigation-এর আগে মূল কারণ খুঁজতে যাওয়া।
- Incident Commander ছাড়া কাজ করা।
- গ্রাহককে দেরিতে বা ভুল তথ্য দেওয়া।
- একজন ব্যক্তিকে দোষ দিয়ে Postmortem শেষ করা।
- Corrective Action-এর মালিক ও তারিখ না দেওয়া।
- Severity আতঙ্ক দিয়ে ঠিক করা।
- Postmortem লিখতে দেরি করা — স্মৃতি ঝাপসা হয়ে যায়।
- একই ইনসিডেন্ট বারবার হওয়ার পরও Problem Management না করা।

১৯.১০ সমস্যা সমাধান (Troubleshooting)

TROUBLESHOOTING: ইনসিডেন্টের সময় সবচেয়ে সাধারণ সমস্যা হলো তথ্যের অভাব — লগ নেই, মেট্রিক নেই, টাইমলাইন নেই। তখন প্রথমে তথ্য সংগ্রহ করুন: কখন শুরু হয়েছে, কী পরিবর্তন হয়েছে, কোন অংশে সমস্যা। যদি সমস্যাটি শুধু একটি অঞ্চলে হয়, তবে সেই অঞ্চলের ট্র্যাফিক সরান। যদি সব জায়গায় হয়, তবে সাম্প্রতিক ডেপ্লয় বা বাহ্যিক সেবা সন্দেহ করুন।

যদি Rollback কাজ না করে, তবে দেখুন ডেটাবেস Migration-ও কি ফিরিয়ে নিতে হবে — কোড Rollback করলেও স্কিমা পরিবর্তন থেকে যেতে পারে। যদি Mitigation-এর পরও Error Rate না কমে, তবে হয়তো সমস্যাটি একাধিক জায়গায় — প্রতিটি স্তর আলাদা করে পরীক্ষা করুন। যদি Postmortem-এর Action বারবার পিছিয়ে যায়, তবে সেগুলোকে স্প্রিন্টের কাজ হিসেবে নথিভুক্ত করুন — স্বেচ্ছাসেবী কাজ হিসেবে নয়।

১৯.১১ সারসংক্ষেপ

ইনসিডেন্ট ব্যবস্থাপনা হলো চাপের মধ্যে শৃঙ্খলা: একজন Commander, একটি চ্যানেল, স্পষ্ট ভূমিকা, এবং Mitigation আগে — মূল কারণ পরে। Severity প্রভাব দিয়ে ঠিক করুন। Five Whys দিয়ে গভীর কারণ খুঁজুন, কিন্তু একজন ব্যক্তিকে কারণ বানাবেন না। Blameless Postmortem-এর উদ্দেশ্য শেখা, শাস্তি নয়। আর Corrective Action-এর মালিক ও তারিখ না থাকলে Postmortem অর্থহীন। Incident Response আগুন নেভায়, Problem Management আগুন লাগার কারণ দূর করে — দুটোই দরকার। পরের অধ্যায়ে আমরা শিখব সিস্টেম যাতে ভেঙে না পড়ে তার জন্য আগে থেকে কীভাবে প্রস্তুতি নিতে হয় — Performance ও Capacity Engineering।

১৯.১২ পর্যালোচনা প্রশ্ন

1. Incident Lifecycle-এর ধাপগুলো কী কী?
2. Mitigation ও Recovery-এর পার্থক্য কী?
3. Severity কীভাবে ঠিক করবেন? Sev 1 ও Sev 2-এর উদাহরণ দিন।
4. Incident Commander-এর কাজ কী? তিনি নিজে কোড ঠিক করবেন না কেন?
5. Five Whys পদ্ধতি ব্যাখ্যা করুন — একটি উদাহরণসহ।
6. Blameless Postmortem কেন গুরুত্বপূর্ণ?
7. Incident Response ও Problem Management-এর পার্থক্য কী?
8. ভালো Corrective Action-এর বৈশিষ্ট্য কী কী?
9. গ্রাহককে ইনসিডেন্টের সময় কী কী জানানো উচিত?
10. Postmortem কখন লিখতে হবে? দেরি করলে কী ক্ষতি?

১৯.১৩ ব্যবহারিক অনুশীলন

1. TechNova Checkout বিভ্রাটের একটি কাল্পনিক দৃশ্য লিখুন (সময়, প্রভাব, টাইমলাইন)। তারপর Five Whys দিয়ে RCA করুন এবং একটি সম্পূর্ণ Postmortem লিখুন — Corrective Action-এর মালিক ও তারিখসহ।
2. একটি Incident Runbook তৈরি করুন: API Error Rate বেড়ে গেলে ধাপে ধাপে কী করবেন — শনাক্তকরণ, Severity, Mitigation (Rollback), যোগাযোগ, এবং যাচাই।

3. তিনটি ভিন্ন দৃশ্যের Severity নির্ধারণ করুন ও যুক্তি দিন: (ক) Checkout সম্পূর্ণ বন্ধ, (খ) Admin প্যানেল ধীর, (গ) একটি বিরল ভাষার অনুবাদ ভুল দেখাচ্ছে।

অধ্যায় ২০ • PART IV — Site Reliability Engineering

Chapter 20 — Performance and Capacity Engineering

২০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি একটি সিস্টেমের Performance পরিমাপ করতে পারবেন, Bottleneck চিহ্নিত করতে পারবেন, এবং ভবিষ্যৎ চাহিদার জন্য Capacity পরিকল্পনা করতে পারবেন। আপনি Load Testing, Stress Testing ও Spike Testing-এর পার্থক্য ব্যাখ্যা করতে পারবেন এবং প্রতিটি কখন ব্যবহার করবেন তা জানবেন। আপনি CPU, মেমরি, ডেটাবেস, Caching ও Queue Backpressure বিশ্লেষণ করতে পারবেন। আপনি Performance Budget ঠিক করতে পারবেন এবং Resilience Pattern — যেমন Circuit Breaker, Bulkhead, Timeout ও Retry — প্রয়োগ করতে পারবেন। সবচেয়ে গুরুত্বপূর্ণভাবে, আপনি শিখবেন Performance একটি একবারের কাজ নয় — এটি একটি চলমান শৃঙ্খলা: পরিমাপ করুন, বাধা খুঁজুন, ঠিক করুন, আবার পরিমাপ করুন।

- Load, Stress ও Spike Testing নকশা ও পরিচালনা করা।
- Capacity Planning করা — ট্র্যাফিক পূর্বাভাস থেকে সম্পদ নির্ধারণ।
- CPU, মেমরি ও ডেটাবেস Bottleneck বিশ্লেষণ করা।
- Caching ও Queue Backpressure নকশা করা।
- Performance Budget ও Resilience Pattern প্রয়োগ করা।

২০.২ প্রাক-প্রয়োজনীয় পুনরালোচনা

আগের অধ্যায়গুলো থেকে মনে রাখুন: Latency (p50, p95, p99), Throughput (প্রতি সেকেন্ডে অনুরোধ), Error Rate — এই তিনটি Performance-এর মূল মাপকাঠি। Monitoring থেকে আমরা জানি সিস্টেম এখন কেমন চলছে; এই অধ্যায়ে আমরা জানব ভবিষ্যতে কেমন চলবে — যখন ট্র্যাফিক দ্বিগুণ হবে, যখন একটি বড় প্রচার চলবে, যখন ডেটাবেসের আকার দশগুণ হবে। TechNova-এর প্রসঙ্গে ভাবুন: ঈদের প্রচারে অর্ডার দশগুণ বেড়ে যায়। যদি সিস্টেম সেই চাপ সামলাতে না পারে, তবে সবচেয়ে লাভজনক সময়েই সেবা ভেঙে পড়বে। Capacity Engineering-এর উদ্দেশ্য হলো সেই মুহূর্তের আগেই প্রস্তুত থাকা — অনুমানের উপর নয়, পরীক্ষার উপর ভিত্তি করে। আরেকটি ধারণা মনে রাখুন: Little's Law — একটি সিস্টেমে গড়ে যতগুলো অনুরোধ থাকবে, তা Throughput ও গড় সময়ের গুণফলের সমান। অর্থাৎ Latency বাড়লে, একই Throughput ধরে রাখতে আরও বেশি সমান্তরাল অনুরোধ সামলাতে হয় — আর তাতেই সম্পদ শেষ হয়ে যায়।

২০.৩ প্রথম নীতি থেকে ব্যাখ্যা: বাধা সবসময় এক জায়গায় থাকে

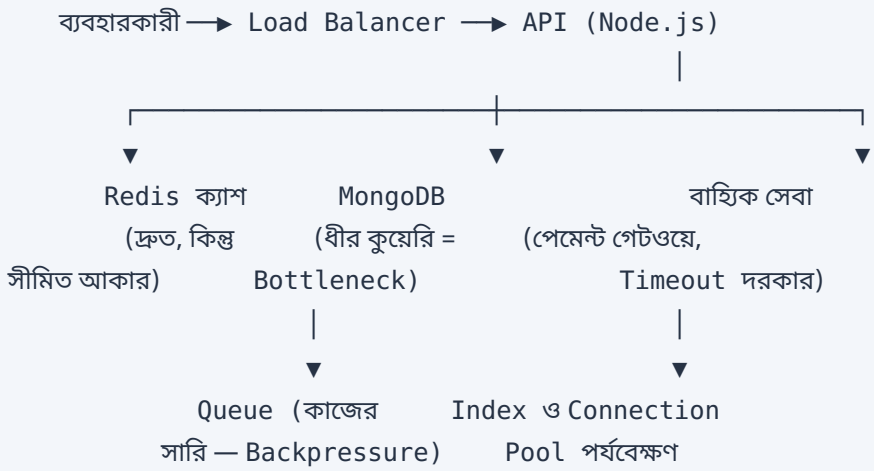
একটি সিস্টেমের Performance-এর মূল সত্যটি হলো: যেকোনো মুহূর্তে একটি মাত্র Bottleneck থাকে — সবচেয়ে ধীর অংশটি পুরো সিস্টেমের গতি ঠিক করে দেয়। CPU যথেষ্ট থাকলেও ডেটাবেস ধীর হলে সিস্টেম ধীর হবে; ডেটাবেস দ্রুত হলেও নেটওয়ার্ক ধীর হলে সিস্টেম ধীর হবে। তাই Performance উন্নয়নের প্রথম ধাপ হলো Bottleneck খুঁজে বের করা — অনুমান করে নয়, পরিমাপ করে। একটি সাধারণ ভুল হলো সবচেয়ে সহজ জায়গায় অপ্টিমাইজ করা — যেমন কোডের একটি ছোট লুপ দ্রুত করা — অথচ আসল বাধা ডেটাবেসে। আরেকটি মূল নীতি: Performance ও Capacity এক জিনিস নয়। Performance বলতে বোঝায় এখনকার গতি — Latency কত, Throughput কত। Capacity বলতে বোঝায় কতটা চাপ সামলানো যাবে — সর্বোচ্চ Throughput কত, কখন সিস্টেম ভেঙে পড়বে। একটি সিস্টেম দ্রুত হতে পারে কিন্তু

Capacity কম হতে পারে — যেমন একটি সার্ভার ১০০ অনুরোধ দ্রুত সামলায়, কিন্তু ১০১তম অনুরোধে ভেঙে পড়ে। ভালো Engineering দুটোই দেখে।

Load Testing বলতে বোঝায় প্রত্যাশিত চাপে সিস্টেম পরীক্ষা করা — যেমন স্বাভাবিক দিনের সর্বোচ্চ ট্র্যাফিক। Stress Testing বলতে বোঝায় প্রত্যাশার চেয়ে বেশি চাপ দেওয়া — যতক্ষণ না সিস্টেম ভেঙে পড়ে — যাতে ভাঙার সীমা জানা যায়। Spike Testing বলতে বোঝায় হঠাৎ চাপ বাড়ানো — যেমন প্রচারের প্রথম মিনিটে ট্র্যাফিক দশগুণ বেড়ে গেলে কী হয়। এই তিনটি পরীক্ষা আলাদা প্রশ্নের উত্তর দেয়: Load Test বলে স্বাভাবিক অবস্থায় কেমন চলবে, Stress Test বলে সীমা কোথায়, Spike Test বলে আকস্মিক পরিবর্তন সামলাতে পারে কি না। তিনটি পরীক্ষাই উৎপাদনের মতো পরিবেশে (বা তার কাছাকাছি) করতে হয় — ছোট Staging-এ পরীক্ষা করে উৎপাদনের সিদ্ধান্ত নেওয়া বিপজ্জনক।

> **CONCEPT:** Bottleneck হলো সিস্টেমের সবচেয়ে ধীর অংশ — সেটিই পুরো সিস্টেমের গতি ঠিক করে। Bottleneck ঠিক না করে অন্য জায়গায় অপ্টিমাইজ করা সময়ের অপচয়। পরিমাপ করে Bottleneck খুঁজুন, ঠিক করুন, আবার পরিমাপ করুন — এই চক্রই Performance Engineering।

২০.৪ স্থাপত্য: অনুরোধের যাত্রা ও বাধার জায়গা



এই চিত্রে প্রতিটি স্তরে Bottleneck লুকিয়ে থাকতে পারে। API স্তরে Bottleneck হতে পারে CPU বা Event Loop ব্লক হওয়া — Node.js-এ একটি ভারী সিঙ্ক্রোনাস কাজ পুরো প্রক্রিয়াকে থামিয়ে দিতে পারে। Redis স্তরে Bottleneck হতে পারে মেমরি পূর্ণ হওয়া বা Eviction। MongoDB স্তরে Bottleneck হতে পারে Index-এর অভাব, ধীর কুয়েরি, বা Connection Pool শেষ হওয়া। বাহ্যিক সেবায় Bottleneck হতে পারে Timeout না থাকা — পেমেন্ট গেটওয়ে ধীর হলে API-এর সব সংযোগ আটকে যায়। Queue-এর কাজ হলো চাপ শোষণ করা: হঠাৎ অনেক কাজ এলে Queue-তে জমে থাকে, আর Worker ধীরে ধীরে প্রক্রিয়া করে। কিন্তু Queue-ও অসীম নয় — Queue পূর্ণ হলে Backpressure দরকার: নতুন অনুরোধকে স্পষ্টভাবে প্রত্যাখ্যান করা (যেমন ৫০৩ ও Retry-After), যাতে সিস্টেম সম্পূর্ণ ভেঙে না পড়ে।

২০.৫ অভ্যন্তরীণ কর্মনীতি: CPU, মেমরি, ডেটাবেস, Caching ও Resilience Pattern

CPU Bottleneck চিনবেন কীভাবে? Load Average ও CPU Utilization দেখুন — কিন্তু শুধু গড় নয়, Steal Time ও I/O Wait-ও দেখুন। Node.js-এ Event Loop Lag পরিমাপ করুন — Event Loop যদি ১০০ মিলিসেকেন্ডের বেশি আটকে থাকে, তবে কোনো ভারী কাজ মূল থ্রেডে চলছে। মেমরি Bottleneck চিনবেন Heap ব্যবহার ও Garbage Collection সময়

দেখে — GC যদি ঘন ঘন ও দীর্ঘ হয়, তবে মেমরি লিক বা অতিরিক্ত বরাদ্দ সন্দেহ করুন। ডেটাবেস Bottleneck চিনবেন ধীর কুয়েরি লগ (Slow Query Log), Index ব্যবহার, ও Connection Pool ব্যবহার দেখে। একটি কুয়েরি যদি Collection Scan করে — অর্থাৎ Index ছাড়া পুরো সংগ্রহ পড়ে — তবে ডেটা বাড়ার সঙ্গে সঙ্গে তা আরও ধীর হবে। সমাধান: সঠিক Index, কুয়েরি পুনর্লিখন, Pagination, এবং প্রয়োজনে Read Replica।

Caching-এর নীতিটি সহজ: একই উত্তর বারবার হিসাব করবেন না। কিন্তু Caching-এর কঠিন অংশ হলো Invalidations — কখন ক্যাশ মুছে ফেলবেন? ভুল ক্যাশ মানে ব্যবহারকারী পুরনো তথ্য দেখবেন। TechNova-তে পণ্যের তালিকা ক্যাশ করা যায় (কম বদলায়), কিন্তু অর্ডারের অবস্থা ক্যাশ করা ঝুঁকিপূর্ণ (ঘন ঘন বদলায়)। Cache-Aside, Read-Through, Write-Through — প্রতিটি Pattern-এর সুবিধা ও অসুবিধা আছে; সঠিক Pattern বেছে নিন ডেটার পরিবর্তনের হার দেখে। Resilience Pattern-গুলো সিস্টেমকে আংশিক ব্যর্থতা থেকে রক্ষা করে: Circuit Breaker — বাহ্যিক সেবা বারবার ব্যর্থ হলে সাময়িকভাবে ডাকা বন্ধ করে, যাতে সিস্টেম নিজে না ডোবে; Bulkhead — সম্পদ ভাগ করে দেয়, যাতে একটি ফিচারের ব্যর্থতা অন্যকে ডোবাতে না পারে; Timeout — প্রতিটি বাহ্যিক ডাকে সময়সীমা, যাতে একটি ধীর সেবা সব সংযোগ আটকে রাখতে না পারে; Retry — সাময়িক ব্যর্থতায় সীমিত সংখ্যকবার আবার চেষ্টা, কিন্তু Exponential Backoff-সহ — নইলে Retry-ই সিস্টেমকে ডুবিয়ে দেয়।

Performance Budget বলতে বোঝায় একটি সীমা যা পার হলে ব্যবস্থা নিতে হবে — যেমন p95 Latency ৩০০ মিলিসেকেন্ড, পৃষ্ঠার আকার ৫০০ কিলোবাইট, API প্রতিক্রিয়া ২০০ মিলিসেকেন্ড। Budget CI-তে পরীক্ষা করা যায় — বাজেট পার হলে Build ব্যর্থ হবে। Capacity Planning-এর ধাপগুলো: বর্তমান ব্যবহার পরিমাপ করুন, বৃদ্ধির হার অনুমান করুন (ট্র্যাফিক, ডেটা, ব্যবহারকারী), ভবিষ্যৎ চাহিদা হিসাব করুন, এবং তার চেয়ে কিছুটা বেশি সম্পদ রাখুন (Headroom) — সাধারণত ৩০–৫০%। কিন্তু Headroom-এর খরচ আছে — অতিরিক্ত সার্ভার অপচয়। তাই পরিকল্পনা পরিমাপ দিয়ে করুন, অনুমানে নয়।

২০.৬ বাস্তব কমান্ড ও কনফিগারেশন

Load Testing — k6 দিয়ে TechNova API পরীক্ষা (lab পরিবেশে, উৎপাদনে নয়):

```
import http from 'k6/http';
import { check, sleep } from 'k6';
export const options = {
  stages: [
    { duration: '2m', target: 50 },   // ধীরে ধীরে বাড়ান
    { duration: '5m', target: 200 },  // প্রত্যাশিত সর্বোচ্চ চাপ
    { duration: '2m', target: 0 },    // ধীরে ধীরে কমান
  ],
  thresholds: {
    http_req_duration: ['p(95)<300'],
    http_req_failed: ['rate<0.01'],
  },
};
export default function () {
  const res = http.get('https://technova.example/api/products');
  check(res, { 'status 200': (r) => r.status === 200 });
  sleep(1);
}
```

সিস্টেম সম্পদ পরীক্ষা:

```
uptime; vmstat 1 5; iostat -x 1 3
ss -s; ss -tan | grep -c ESTAB
# Node.js Event Loop Lag — /metrics থেকে
```

```
curl -s http://203.0.113.21:3000/metrics | grep -E 'eventloop_lag|process_cpu|nodejs_heap'
# MongoDB ধীর কুয়েরি (lab)
mongosh --eval 'db.setProfilingLevel(1, { slowms: 100 })' "mongodb://203.0.113.31:27017/
technova" \
  -u CHANGE-ME-USER -p 'CHANGE-ME-PASSWORD' --authenticationDatabase admin
```

Express-এ Timeout ও Circuit Breaker-এর নমুনা ধারণা:

```
// বাহ্যিক ডাকে সবসময় timeout দিন — কখনো অসীম অপেক্ষা নয়
const controller = new AbortController();
const timer = setTimeout(() => controller.abort(), 3000);
try {
  const res = await fetch('https://payments.technova.example/charge', {
    method: 'POST', signal: controller.signal,
    headers: { 'Authorization': 'Bearer CHANGE-ME-PAYMENT-KEY' },
  });
  // সাড়া প্রক্রিয়া করুন
} catch (err) {
  // timeout বা ব্যর্থতা — Circuit Breaker গণনা বাড়ান, fallback দিন
} finally { clearTimeout(timer); }
```

২০.৭ উৎপাদন পরিস্থিতি

পরিস্থিতি ১ — প্রচারে সাইট ধীর। ঈদের প্রচারের প্রথম ঘণ্টায় ট্র্যাফিক পাঁচগুণ বেড়ে গেল, p95 Latency ৩০০ মিলিসেকেন্ড থেকে ২ সেকেন্ডে উঠল। তদন্তে দেখা গেল MongoDB-তে একটি কুয়েরি Index ছাড়া চলছে। সমাধান: Index যোগ করা হলো, Latency আবার ২৫০ মিলিসেকেন্ডে নামল। শিক্ষা: প্রচারের আগে Load Test করুন — প্রচারের সময় নয়। **পরিস্থিতি ২ — Queue পূর্ণ হয়ে গেল।** ইমেইল পাঠানোর Queue-তে কাজ জমে গেল, কারণ Worker-এর সংখ্যা কম। Backpressure না থাকায় API নতুন অনুরোধ নিতেই থাকল, আর মেমরি শেষ হয়ে গেল। সমাধান: Queue-এর আকার সীমিত করা, Worker বাড়ানো, এবং Queue পূর্ণ হলে ৫০৩ দেওয়া। **পরিস্থিতি ৩ — Retry ঝড়।** পেমেন্ট গেটওয়ে ধীর হলে API বারবার Retry করছিল — প্রতিটি ব্যর্থ অনুরোধ তিনবার চেষ্টা হচ্ছিল, ফলে চাপ তিনগুণ বেড়ে গেল। সমাধান: Exponential Backoff, Retry সীমা, ও Circuit Breaker।

পরিস্থিতি ৬ — ডেটাবেস Index-এর পরিকল্পনা। TechNova-এর Orders সংগ্রহে একটি কুয়েরি প্রতিদিন ধীর হচ্ছিল — ডেটা বাড়ার সঙ্গে সঙ্গে। Explain দেখা গেল Collection Scan হচ্ছে — Index নেই। Index যোগ করার পর কুয়েরি ২ সেকেন্ড থেকে ২০ মিলিসেকেন্ডে নামল। কিন্তু Index-এরও খরচ আছে — লেখা ধীর হয়, Storage বাড়ে। তাই দল নিয়ম করল: প্রতিটি নতুন কুয়েরির সঙ্গে তার Index পরিকল্পনা আসবে, আর প্রতি মাসে অব্যবহৃত Index পর্যালোচনা করা হবে। শিক্ষা: Index একটি বিনিয়োগ — সঠিক Index সোনা, ভুল Index বোঝা।

পরিস্থিতি ৭ — Autoscaling-এর সীমা। TechNova-তে HPA বসানো হলো — CPU ৭০% ছুঁলেই Pod বাড়বে। প্রচারের সময় Pod বাড়ল — কিন্তু MongoDB Connection Pool শেষ হয়ে গেল, কারণ প্রতিটি নতুন Pod নতুন সংযোগ খুলছিল। শিক্ষা: একটি স্তরে স্কেল করলে অন্য স্তরেও চাপ পড়ে — স্কেলিং-এর পরিকল্পনা পুরো সিস্টেমের জন্য করতে হয়। Connection Pool-এর সীমা, ডেটাবেসের ক্ষমতা, বাহ্যিক সেবার Rate Limit — সবকিছু একসঙ্গে দেখুন। নইলে Autoscaling সমস্যার সমাধান না হয়ে সমস্যার স্থানান্তর হয়।

পরিস্থিতি ৮ — Performance Budget CI-তে। দল Performance Budget-কে CI-তে যোগ করল: Bundle আকার ৫০০ কিলোবাইট পার হলে Build ব্যর্থ হবে, API-এর p95 Latency পরীক্ষায় ৩০০ মিলিসেকেন্ড পার হলে Build ব্যর্থ হবে। প্রথমে ডেভেলপাররা আপত্তি করলেন — কিন্তু কয়েক সপ্তাহ পর তারা নিজেরাই Budget-এর মধ্যে থাকতে শিখলেন। শিক্ষা:

Performance-কে পরে ঠিক করার কাজ নয় — প্রতিটি পরিবর্তনে পরীক্ষা করার কাজ। Budget CI-তে থাকলে Performance আর কারো দয়া নয় — নিয়ম।

পরিস্থিতি ৪ — Connection Pool শেষ। ট্র্যাফিক বাড়ার সঙ্গে সঙ্গে API-এর Error বেড়ে গেল — কিন্তু CPU ও মেমরি স্বাভাবিক। তদন্তে দেখা গেল MongoDB Connection Pool শেষ হয়ে গেছে: প্রতিটি অনুরোধ সংযোগের জন্য অপেক্ষা করছে। কারণ: Pool-এর আকার ছোট ছিল, আর কিছু কুয়েরি সংযোগ ধরে রাখছিল। সমাধান: Pool-এর আকার ট্র্যাফিক অনুযায়ী বাড়ানো, ধীর কুয়েরি ঠিক করা, এবং Pool ব্যবহারের মেট্রিক বসানো — যাতে Pool ৮০% ছুঁলেই সতর্কবার্তা আসে। শিক্ষা: Bottleneck সবসময় CPU নয় — কখনো তা একটি সংখ্যা: Pool-এর আকার, Queue-এর দৈর্ঘ্য, ফাইল Descriptor-এর সীমা।

পরিস্থিতি ৫ — Frontend Performance। API দ্রুত, কিন্তু পৃষ্ঠা ধীর — কারণ React Bundle-এর আকার ৩ মেগাবাইট, ছবিগুলো সংকুচিত নয়, এবং প্রতিটি পৃষ্ঠায় অপ্রয়োজনীয় ডেটা আসছে। Performance Budget-এ পৃষ্ঠার আকার ৫০০ কিলোবাইট ঠিক করা হলো, ছবি CDN থেকে দেওয়া হলো, Code Splitting করা হলো, আর API থেকে শুধু দরকারি ক্ষেত্র ফেরত দেওয়া হলো। ফল: পৃষ্ঠা লোডের সময় অর্ধেক হলো। শিক্ষা: Performance শুধু সার্ভারের কাজ নয় — Frontend, নেটওয়ার্ক ও ডেটা — সব মিলিয়ে ব্যবহারকারীর অভিজ্ঞতা।

২০.৮ নিরাপত্তা ও খরচ বিবেচনা

Load Testing-এর সময় নিরাপত্তা: কখনো উৎপাদনে বিনা পরিকল্পনায় Load Test করবেন না — এতে আসল ব্যবহারকারীরা ক্ষতিগ্রস্ত হবেন, এবং তা DDoS আক্রমণের মতো দেখাতে পারে। সবসময় Lab বা Staging পরিবেশে পরীক্ষা করুন, অথবা উৎপাদনে করলে ট্র্যাফিক সীমিত রাখুন ও সংশ্লিষ্ট দলকে জানান। Performance অপ্টিমাইজেশনের খরচের দিক: বড় সার্ভার কেনা সবসময় সঠিক উত্তর নয় — অনেক সময় একটি Index বা একটি ক্যাশ বড় সার্ভারের চেয়ে সস্তা ও কার্যকর। আবার Caching-এরও খরচ আছে — Redis-এর মেমরি, Invalidation-এর জটিলতা। প্রতিটি অপ্টিমাইজেশনের আগে জিজ্ঞাসা করুন: এই পরিবর্তনে Latency কত কমবে, খরচ কত বাড়বে, আর জটিলতা কত বাড়বে? সবচেয়ে সস্তা Performance উন্নয়ন প্রায়ই সবচেয়ে সহজটি: অপ্রয়োজনীয় কাজ না করা — যেমন প্রতিটি অনুরোধে পুরো তালিকা না পাঠিয়ে Pagination করা।

২০.৯ সাধারণ ভুল

- Bottleneck না খুঁজে অনুমান করে অপ্টিমাইজ করা।
- Staging-এ পরীক্ষা করে উৎপাদনের সিদ্ধান্ত নেওয়া।
- গড় Latency দিয়ে Performance বিচার করা।
- Timeout ছাড়া বাহ্যিক সেবা ডাকা।
- Retry-তে Backoff না দেওয়া — Retry ঝড় তৈরি করা।
- Queue-কে অসীম মনে করা — Backpressure না রাখা।
- Index ছাড়া কুয়েরি উৎপাদনে পাঠানো।
- Capacity পরিকল্পনা না করে প্রচার চালানো।

২০.১০ সমস্যা সমাধান (Troubleshooting)

TROUBLESHOOTING: সিস্টেম ধীর মনে হলে প্রথমে ভাগ করুন: Latency কোথায় বাড়ছে — নেটওয়ার্ক, API, ডেটাবেস, নাকি বাহ্যিক সেবা? Trace দেখুন — প্রতিটি ধাপের সময় আলাদা করুন। যদি একটি ধাপে সময় বেশি লাগে, তবে সেই ধাপে গভীরে যান। যদি সব ধাপে সমানভাবে বেশি লাগে, তবে সম্ভবত সম্পদ শেষ — CPU বা মেমরি দেখুন।

CPU বেশি দেখালে দেখুন কোন প্রক্রিয়া ব্যবহার করছে — হয়তো একটি অনিয়ন্ত্রিত লুপ বা GC। মেমরি বেশি দেখালে Heap Dump বা প্রোফাইলিং করুন — লিক খুঁজুন। ডেটাবেস ধীর হলে Slow Query Log দেখুন ও Explain দিয়ে কুয়েরি পরিকল্পনা পরীক্ষা করুন। Queue বাড়তে থাকলে Worker-এর সংখ্যা ও প্রতিটি কাজের সময় দেখুন — হয়তো একটি কাজ আটকে আছে।

২০.১১ সারসংক্ষেপ

Performance Engineering-এর মূলমন্ত্র: পরিমাপ করুন, Bottleneck খুঁজুন, ঠিক করুন, আবার পরিমাপ করুন। Load, Stress ও Spike Testing আলাদা প্রশ্নের উত্তর দেয় — তিনটিই দরকার। CPU, মেমরি, ডেটাবেস, ক্যাশ ও কিউ — প্রতিটি স্তরে বাধা থাকতে পারে। Timeout, Circuit Breaker, Bulkhead ও সীমিত Retry সিস্টেমকে আংশিক ব্যর্থতা থেকে রক্ষা করে। Performance Budget সীমা ঠিক করে, Capacity Planning ভবিষ্যৎ প্রস্তুত করে। পরের অধ্যায়ে আমরা শিখব সবচেয়ে খারাপ পরিস্থিতির জন্য প্রস্তুতি — যখন সবকিছু ভেঙে পড়ে, তখন কীভাবে ফিরে আসবেন: Disaster Recovery ও Business Continuity।

২০.১২ পর্যালোচনা প্রশ্ন

1. Bottleneck কাকে বলে? এটি না খুঁজে অপ্টিমাইজ করলে কী ক্ষতি?
2. Load, Stress ও Spike Testing-এর পার্থক্য কী?
3. Little's Law ব্যাখ্যা করুন — Latency বাড়লে কী হয়?
4. MongoDB-তে ধীর কুয়েরি কীভাবে চিনবেন ও ঠিক করবেন?
5. Caching-এর সবচেয়ে কঠিন অংশ কী? কেন?
6. Circuit Breaker কীভাবে কাজ করে? এটি না থাকলে কী হয়?
7. Queue Backpressure বলতে কী বোঝায়? এটি না থাকলে কী হয়?
8. Performance Budget কী? CI-তে এটি কীভাবে পরীক্ষা করা যায়?
9. Capacity Planning-এর ধাপগুলো কী কী?
10. Retry-তে Exponential Backoff কেন দরকার?

২০.১৩ ব্যবহারিক অনুশীলন

1. k6 দিয়ে TechNova API-এর একটি Load Test লিখুন (lab পরিবেশে): স্বাভাবিক চাপ, সর্বোচ্চ চাপ, এবং Spike — তিনটি ধাপ। ফল বিশ্লেষণ করে Bottleneck অনুমান করুন।
2. একটি ধীর MongoDB কুয়েরির উদাহরণ নিন, Explain দিয়ে পরিকল্পনা দেখুন, Index যোগ করে আবার পরীক্ষা করুন, এবং উন্নতির পরিমাণ লিখুন।

3. TechNova-এর আসন্ন প্রচারের জন্য একটি Capacity পরিকল্পনা লিখুন: বর্তমান Throughput, প্রত্যাশিত বৃদ্ধি, প্রয়োজনীয় সার্ভার ও ডেটাবেস সম্পদ, এবং Headroom — সংখ্যাসহ।

PART V • DEVOPS ENGINEERING MASTERY

Enterprise Reliability & Operations

Disaster Recovery ↺ Business Continuity, Chaos Engineering, FinOps ↺ Operational Governance, Enterprise DevSecOps ↺ SRE Architecture |

অধ্যায় ২১ • PART V — Enterprise Reliability & Operations

Chapter 21 — Disaster Recovery and Business Continuity

২১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Backup ও Disaster Recovery (DR)-এর পার্থক্য ব্যাখ্যা করতে পারবেন এবং একটি সম্পূর্ণ DR পরিকল্পনা তৈরি করতে পারবেন। আপনি Recovery Point Objective (RPO) ও Recovery Time Objective (RTO) সংজ্ঞায়িত করতে পারবেন এবং ব্যবসার প্রয়োজন থেকে সেগুলো নির্ধারণ করতে পারবেন। আপনি Backup কৌশল নকশা করতে পারবেন, Restore পরীক্ষা করতে পারবেন, এবং Multi-region ও Cross-cloud Recovery-এর সুবিধা ও জটিলতা বিশ্লেষণ করতে পারবেন। আপনি Failover ও Failback পরিচালনা করতে পারবেন, Database Consistency-এর চ্যালেঞ্জ বুঝবেন, এবং Business Continuity পরিকল্পনা নথিভুক্ত করতে পারবেন। সবচেয়ে গুরুত্বপূর্ণ শিক্ষাটি হলো: পরীক্ষা না করা ব্যাকআপ কোনো ব্যাকআপ নয় — Restore পরীক্ষা ছাড়া DR পরিকল্পনা শুধু একটি কাগজ।

- Backup বনাম DR ব্যাখ্যা করা।
- RPO ও RTO নির্ধারণ করা।
- Backup কৌশল ও Restore পরীক্ষা নকশা করা।
- Multi-region ও Cross-cloud Recovery বিশ্লেষণ করা।
- Failover, Failback ও Business Continuity পরিকল্পনা করা।

২১.২ প্রাক-প্রয়োজনীয় পুনরালোচনা

আগের অধ্যায়গুলো থেকে মনে রাখুন: Redundancy, Replication, Monitoring ও Incident Management। MongoDB-তে Replica Set বলতে বোঝায় একাধিক কপি — একটি Primary, বাকিগুলো Secondary — যাতে একটি নষ্ট হলে অন্যটি দায়িত্ব নেয়। Load Balancer বলতে বোঝায় ট্র্যাফিক একাধিক সার্ভারে বণ্টন করা। এই অধ্যায়ে আমরা আরও বড় স্কেলে ভাবব: পুরো ডেটা সেন্টার বা পুরো অঞ্চল (Region) নষ্ট হয়ে গেলে কী হবে? একটি সার্ভার নষ্ট হওয়া আর একটি অঞ্চল নষ্ট হওয়া এক জিনিস নয় — দ্বিতীয়টির জন্য আলাদা পরিকল্পনা দরকার। TechNova-এর প্রসঙ্গে ভাবুন: ঢাকার ডেটা সেন্টারে আগুন লাগল, বা ক্লাউড অঞ্চলে বড় বিভ্রাট হলো — তখন অর্ডার, পেমেন্ট, গ্রাহকের ডেটা — সবকিছু কীভাবে ফিরে পাবেন? আর কতক্ষণের মধ্যে? এই দুটি প্রশ্নের উত্তরই RPO ও RTO।

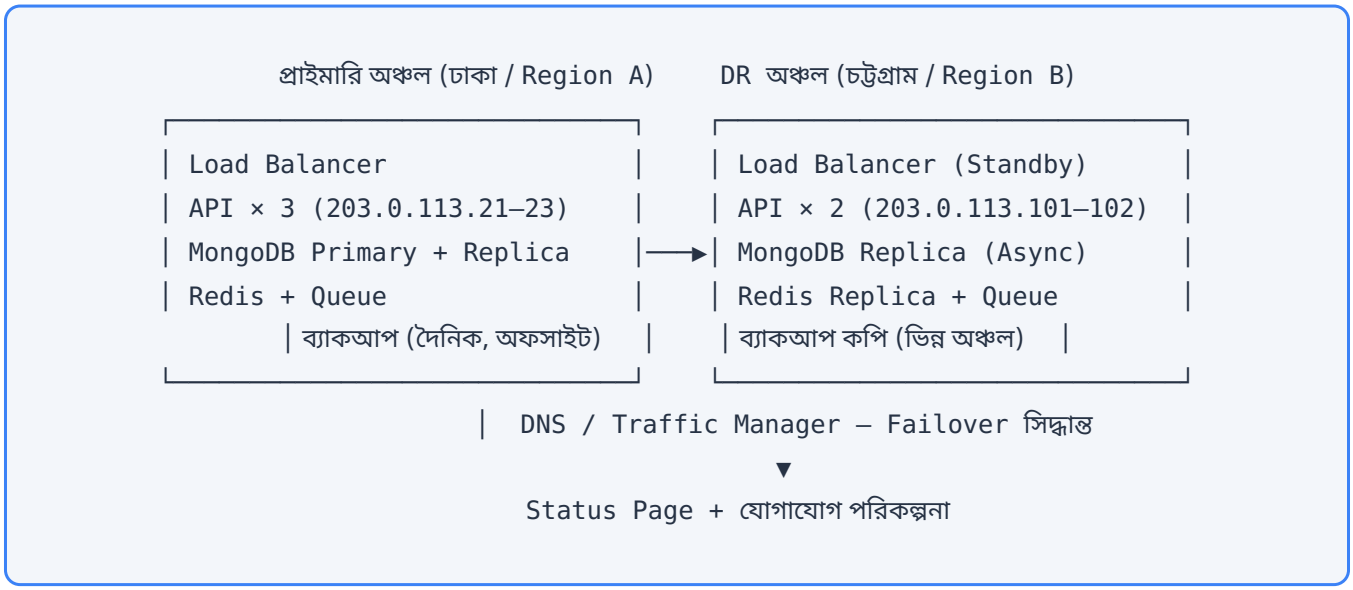
২১.৩ প্রথম নীতি থেকে ব্যাখ্যা: ব্যাকআপ মানে পুনরুদ্ধার, সংরক্ষণ নয়

অনেক সংস্থা মনে করে ব্যাকআপ নেওয়া মানেই নিরাপদ। কিন্তু ব্যাকআপের উদ্দেশ্য সংরক্ষণ নয় — পুনরুদ্ধার। একটি ব্যাকআপ তখনই মূল্যবান যখন তা থেকে ডেটা ফিরিয়ে আনা যায়, সঠিক সময়ে, সঠিক অবস্থায়। বাস্তবে বহু সংস্থা আবিষ্কার করেছে তাদের ব্যাকআপ নষ্ট, অসম্পূর্ণ, বা এত পুরনো যে তা কাজে লাগে না — কিন্তু তখন অনেক দেরি হয়ে গেছে। তাই প্রথম নীতিটি হলো: ব্যাকআপ নেওয়া নয়, Restore পরীক্ষা করাই আসল কাজ। দ্বিতীয় নীতিটি হলো: Backup ও Disaster Recovery এক জিনিস নয়। Backup বলতে বোঝায় ডেটার কপি রাখা — যাতে ভুল করে মুছে ফেলা, দুর্নীতি, বা Ransomware-এর পর ডেটা ফিরে পাওয়া যায়। Disaster Recovery বলতে বোঝায় পুরো সেবা ফিরিয়ে আনা — সার্ভার, নেটওয়ার্ক, ডেটাবেস, কনফিগারেশন, সবকিছু — একটি বড় বিভ্রাটের পর। Backup DR-এর একটি অংশ, কিন্তু DR-এর জন্য আরও অনেক কিছু দরকার: Failover পরিকল্পনা, DNS পরিবর্তন, Runbook, যোগাযোগ পরিকল্পনা, এবং নিয়মিত মহড়া (Drill)।

RPO ও RTO এই পরিকল্পনার দুটি সংখ্যা। RPO (Recovery Point Objective) বলতে বোঝায় কতটুকু ডেটা হারানো গ্রহণযোগ্য — অর্থাৎ সর্বশেষ ব্যাকআপ থেকে বিভ্রাট পর্যন্ত কত সময়ের ডেটা হারাতে পারবেন। RPO ১ ঘণ্টা মানে সর্বোচ্চ ১ ঘণ্টার ডেটা হারানো যাবে — তাই অন্তত প্রতি ঘণ্টায় ব্যাকআপ নিতে হবে। RPO ০ মানে কোনো ডেটা হারানো যাবে না — তাই Synchronous Replication দরকার। RTO (Recovery Time Objective) বলতে বোঝায় কতক্ষণের মধ্যে সেবা ফিরিয়ে আনতে হবে — বিভ্রাট শুরু থেকে সেবা স্বাভাবিক হওয়া পর্যন্ত সর্বোচ্চ সময়। RTO ৪ ঘণ্টা মানে ৪ ঘণ্টার মধ্যে সেবা ফিরতে হবে। RPO ও RTO যত ছোট, খরচ তত বেশি — Synchronous Replication, Hot Standby, Multi-region — সবই ব্যয়বহুল। তাই ব্যবসার সঙ্গে বসে ঠিক করুন: এই সেবা ৪ ঘণ্টা বন্ধ থাকলে কত ক্ষতি হবে? ১ ঘণ্টার ডেটা হারালে কত ক্ষতি হবে? সেই সংখ্যা থেকেই RPO ও RTO আসবে — প্রযুক্তিগত ইচ্ছা থেকে নয়।

> **CONCEPT:** RPO = কতটুকু ডেটা হারানো যাবে (সময়ে মাপা)। RTO = কতক্ষণে সেবা ফিরবে (সময়ে মাপা)। Backup = ডেটার কপি। Disaster Recovery = পুরো সেবা ফিরিয়ে আনা। Backup ছাড়া DR হয় না, কিন্তু শুধু Backup থাকলেই DR হয় না।

২১.৪ স্থাপত্য: Multi-region Recovery নকশা



DR কৌশল তিন স্তরে ভাগ করা যায়। Backup & Restore — সবচেয়ে সস্তা, সবচেয়ে ধীর: বিভ্রাটের পর নতুন পরিবেশ তৈরি করে ব্যাকআপ থেকে Restore করা হয়। RTO কয়েক ঘণ্টা থেকে দিন। Pilot Light — DR অঞ্চলে সেবার ছোট সংস্করণ সবসময় চলছে: ডেটাবেস Replica, সামান্য সার্ভার। বিভ্রাটের পর স্কেল বাড়ানো হয়। RTO কয়েক ঘণ্টা। Warm Standby — DR অঞ্চলে পূর্ণ সেবা চলছে, কিন্তু ট্র্যাফিক কম। Failover-এ শুধু DNS বদলাতে হয়। RTO কয়েক মিনিট থেকে ঘণ্টা। Hot Standby / Active-Active — দুটি অঞ্চলেই পূর্ণ সেবা, ট্র্যাফিক ভাগ করা। RTO প্রায় শূন্য, কিন্তু খরচ সবচেয়ে বেশি, এবং Database Consistency সবচেয়ে কঠিন। TechNova-এর জন্য সঠিক স্তর ব্যবসার RPO ও RTO থেকে আসবে — সব সেবার জন্য Hot Standby দরকার নেই।

২১.৫ অভ্যন্তরীণ কর্মনীতি: Backup, Consistency, Failover ও Failback

Backup কৌশলের মূল উপাদানগুলো: কী ব্যাকআপ নেবেন (ডেটাবেস, ফাইল, কনফিগারেশন, Secret-এর এনক্রিপ্টেড কপি), কত ঘন ঘন (RPO থেকে নির্ধারিত), কোথায় রাখবেন (একই অঞ্চলে নয় — ভিন্ন অঞ্চলে, অফসাইটে, Immutable Storage-এ যাতে Ransomware মুছে ফেলতে না পারে), এবং কতদিন রাখবেন (Retention — দৈনিক ৭ দিন, সাপ্তাহিক ৪

সপ্তাহ, মাসিক ১২ মাস — একটি সাধারণ নীতি)। ৩-২-১ নিয়মটি মনে রাখুন: ডেটার ৩টি কপি, ২টি ভিন্ন মাধ্যমে, ১টি ভিন্ন স্থানে। MongoDB-তে Backup-এর জন্য mongodump, Filesystem Snapshot, বা Point-in-Time Recovery — প্রতিটির সুবিধা ও সীমা আছে। mongodump সহজ, কিন্তু বড় ডেটাবেসে ধীর। Snapshot দ্রুত, কিন্তু Restore-এর সময় Consistency যাচাই দরকার।

Database Consistency হলো DR-এর সবচেয়ে কঠিন অংশ। Asynchronous Replication-এ DR অঞ্চলের ডেটা সবসময় একটু পিছিয়ে থাকে — বিভ্রাটের মুহূর্তে কিছু লেখা হারিয়ে যেতে পারে। এটি RPO-এর সঙ্গে সরাসরি সম্পর্কিত। Synchronous Replication-এ কোনো ডেটা হারায় না, কিন্তু প্রতিটি লেখা ধীর হয় — কারণ দুটি অঞ্চলে লেখা নিশ্চিত করতে হয়। Failover-এর সময় আরও প্রশ্ন আসে: DNS কত দ্রুত বদলাবে? পুরনো Primary আবার ফিরে এলে কী হবে — Split Brain (দুটি Primary) যাতে না হয়? Failback বলতে বোঝায় মূল অঞ্চলে ফিরে যাওয়া — এটি Failover-এর চেয়ে প্রায়ই কঠিন, কারণ DR অঞ্চলে নতুন ডেটা তৈরি হয়েছে, তা আবার মূল অঞ্চলে ফিরিয়ে আনতে হয়। তাই Failback-এর পরিকল্পনাও আগে থেকে লিখে রাখুন, পরীক্ষা করুন।

২১.৬ বাস্তব কমান্ড ও কনফিগারেশন

MongoDB ব্যাকআপ ও Restore পরীক্ষা (lab পরিবেশে):

```
# দৈনিক ব্যাকআপ — mongodump (সংকুচিত, তারিখসহ)
mongodump --uri="mongodb://203.0.113.31:27017/technova" \
  -u CHANGE-ME-USER -p 'CHANGE-ME-PASSWORD' --authenticationDatabase admin \
  --gzip --archive=/backup/technova-$(date +%F).archive.gz

# ব্যাকআপের তালিকা ও আকার যাচাই
ls -lh /backup/ | tail -n 10
# Restore পরীক্ষা — কখনো উৎপাদনে নয়, আলাদা lab ডেটাবেসে
mongorestore --uri="mongodb://203.0.113.41:27017/technova-restore-test" \
  -u CHANGE-ME-USER -p 'CHANGE-ME-PASSWORD' --authenticationDatabase admin \
  --gzip --archive=/backup/technova-2026-11-08.archive.gz
# Restore-এর পর নথির সংখ্যা মিলিয়ে দেখুন
mongosh --eval 'db.orders.countDocuments(); db.users.countDocuments()' \
  "mongodb://203.0.113.41:27017/technova-restore-test" \
  -u CHANGE-ME-USER -p 'CHANGE-ME-PASSWORD' --authenticationDatabase admin
```

DR Runbook-এর ন্যূনতম কাঠামো:

```
DR Runbook — TechNova Checkout (সংস্করণ 1.2, পর্যালোচনা: ২০২৬-১১-০১)
RPO: ১ ঘণ্টা | RTO: ৪ ঘণ্টা | কৌশল: Pilot Light (চট্টগ্রাম অঞ্চল)
ঘোষণা: Incident Commander Sev 1 ঘোষণা করবেন; Status Page হালনাগাদ
ধাপ ১: প্রাইমারি অঞ্চলের অবস্থা নিশ্চিত করুন (মনিটরিং + ক্লাউড Status)
ধাপ ২: DR অঞ্চলে API স্কেল বাড়ান (replicas: 2 → 3)
ধাপ ৩: MongoDB Replica-কে Primary হিসেবে প্রমোট করুন
ধাপ ৪: DNS / Traffic Manager DR অঞ্চলে সরান; TTL আগে থেকে কমিয়ে রাখুন
ধাপ ৫: Health Check ও Smoke Test চালান; গ্রাহককে জানান
ধাপ ৬: Failback পরিকল্পনা — মূল অঞ্চল ফিরলে ডেটা সমন্বয় করে ফিরুন
যোগাযোগ: Commander, DBA, Network, ব্যবস্থাপনা, গ্রাহক সহায়তা — নম্বর সংযুক্ত
```

২১.৭ উৎপাদন পরিস্থিতি

পরিস্থিতি ১ — ভুল করে ডেটা মুছে ফেলা। একজন প্রকৌশলী ভুল কুয়েরিতে ১০,০০০ অর্ডার মুছে ফেললেন। DR-এর দরকার নেই — Backup থেকে Restore দরকার। কিন্তু Restore করলে মুছে ফেলার পরের সঠিক অর্ডারগুলোও হারিয়ে

যাবে। সমাধান: Point-in-Time Recovery — মুছে ফেলার ঠিক আগের মুহূর্তে ফিরে যাওয়া, তারপর পরের সঠিক পরিবর্তনগুলো আবার প্রয়োগ করা। শিক্ষা: শুধু দৈনিক ব্যাকআপ যথেষ্ট নয় — Point-in-Time ক্ষমতা দরকার। **পরিস্থিতি ২ — অঞ্চল বিভ্রাট।** প্রাইমারি অঞ্চলে বড় বিভ্রাট, ২ ঘণ্টা ধরে সেবা বন্ধ। DR অঞ্চলে Failover করা হলো, কিন্তু DNS TTL বেশি থাকায় কিছু ব্যবহারকারী আরও ১ ঘণ্টা পুরনো অঞ্চলে যাচ্ছিলেন। শিক্ষা: DNS TTL আগে থেকে কমিয়ে রাখুন (যেমন ৬০ সেকেন্ড) — বিভ্রাটের সময় TTL কমানো যায় না, কারণ পুরনো মান ক্যাশে থাকে। **পরিস্থিতি ৩ — Restore পরীক্ষায় ব্যাকআপ নষ্ট ধরা পড়ল।** নিয়মিত Drill-এ দেখা গেল গত মাসের ব্যাকআপ ফাইলটি দূষিত। ভাগ্য ভালো — এটি Drill-এ ধরা পড়েছে, আসল বিভ্রাটে নয়। শিক্ষা: এই ঘটনাই প্রমাণ করে Restore পরীক্ষা কেন অপরিহার্য।

পরিস্থিতি ৬ — Cross-cloud Recovery-এর জটিলতা। TechNova একটি Cross-cloud DR পরিকল্পনা বিবেচনা করল: প্রাইমারি ক্লাউড A, DR ক্লাউড B। পরীক্ষায় দেখা গেল দুটি ক্লাউডের নেটওয়ার্ক, Identity, ও Secret ব্যবস্থা আলাদা — Failover-এর সময় প্রতিটি জিনিস হাতে বদলাতে হয়। খরচও বেশি — দুটি ক্লাউডের দক্ষতা দরকার। দল সিদ্ধান্ত নিল: প্রথমে Multi-region (একই ক্লাউডে) DR শক্ত করা হবে, Cross-cloud পরে — যখন দলের দক্ষতা বাড়বে। শিক্ষা: সবচেয়ে শক্তিশালী সমাধান সবসময় সঠিক নয় — দলের দক্ষতা ও পরিচালনার জটিলতাও হিসাব করুন। একটি সহজ DR পরিকল্পনা যা পরীক্ষা করা যায়, একটি জটিল পরিকল্পনার চেয়ে ভালো যা কেউ বোঝে না।

পরিস্থিতি ৭ — Business Continuity-তে মানুষের দিক। একটি বড় বিভ্রাটের সময় TechNova-এর অফিসে বিদ্যুৎ ছিল না — প্রকৌশলীরা বাড়ি থেকে কাজ করছিলেন, কিন্তু যোগাযোগের মাধ্যম (Slack) নিজেই বিভ্রাটে ছিল। দল বিকল্প যোগাযোগের মাধ্যম ঠিক করল: ফোন, SMS, আর একটি আলাদা Status Page — যা মূল সিস্টেম থেকে স্বাধীন। আর প্রতিটি গুরুত্বপূর্ণ ব্যক্তির বিকল্প (Backup) ঠিক করা হলো — একজন না থাকলে অন্যজন দায়িত্ব নেবে। শিক্ষা: Business Continuity শুধু সার্ভারের নয় — মানুষ, যোগাযোগ, ও সিদ্ধান্তেরও। প্রযুক্তি ফিরে এলেও মানুষ না থাকলে সেবা ফেরে না।

পরিস্থিতি ৮ — RPO ও RTO-এর পুনঃপর্যালোচনা। TechNova-এর Checkout-এর RPO ছিল ১ ঘণ্টা, RTO ৪ ঘণ্টা — দুই বছর আগে ঠিক করা। কিন্তু এখন অর্ডারের পরিমাণ তিনগুণ — ১ ঘণ্টার ডেটা হারানো এখন অনেক বড় ক্ষতি। দল ব্যবসার সঙ্গে বসে RPO ১৫ মিনিটে নামাল — Replication-এর ফ্রিকোয়েন্সি বাড়িয়ে। শিক্ষা: RPO ও RTO স্থির নয় — ব্যবসা বদলালে তা বদলাতে হয়। প্রতি বছর, বা ব্যবসায় বড় পরিবর্তন হলে, RPO ও RTO আবার পর্যালোচনা করুন।

পরিস্থিতি ৪ — Ransomware ও Immutable Backup। একটি আক্রমণে TechNova-এর ফাইল সার্ভার এনক্রিপ্ট হয়ে গেল — আক্রমণকারী ব্যাকআপও মুছে ফেলার চেষ্টা করল। কিন্তু ব্যাকআপ Immutable Storage-এ ছিল — মুছে ফেলা বা বদলানো যায় না। ফলে ডেটা ফিরিয়ে আনা গেল। শিক্ষা: ব্যাকআপের নিরাপত্তা ডেটার নিরাপত্তার মতোই গুরুত্বপূর্ণ — ব্যাকআপে অ্যাক্সেস সীমিত করুন, এনক্রিপ্ট করুন, এবং নিয়মিত Restore পরীক্ষা করুন। আরেকটি শিক্ষা: ব্যাকআপের অ্যাকাউন্টের Credential উৎপাদনের Credential থেকে আলাদা রাখুন — একটি আপস হলে অন্যটি নিরাপদ থাকবে।

পরিস্থিতি ৫ — DR Drill-এর মূল্য। TechNova প্রতি ছয় মাসে একটি DR Drill করে: শনিবার সকালে, Lab-এর মতো করে, DR অঞ্চলে Failover-এর মহড়া। প্রথম Drill-এ দেখা গেল Runbook-এর তিনটি ধাপ পুরনো — DNS-এর নাম বদলে গেছে, একজন দায়িত্বশীল ব্যক্তি চলে গেছেন, একটি স্ক্রিপ্ট আর চলে না। Drill-এর সময় এগুলো ঠিক করা হলো — আসল বিভ্রাটে নয়। দ্বিতীয় Drill-এ Failover সময় ৪ ঘণ্টা থেকে ৯০ মিনিটে নামল। শিক্ষা: DR পরিকল্পনা একটি জীবন্ত নথি — Drill ছাড়া তা পুরনো হয়ে যায়, আর পুরনো পরিকল্পনা বিপদের সময় কাজ করে না।

২১.৮ নিরাপত্তা ও খরচ বিবেচনা

Backup নিজেই একটি নিরাপত্তা ঝুঁকি: ব্যাকআপ ফাইলে পুরো ডেটাবেস থাকে — গ্রাহকের তথ্যসহ। ব্যাকআপ এনক্রিপ্ট করুন (At Rest ও In Transit), অ্যাক্সেস সীমিত করুন, এবং ব্যাকআপের অ্যাক্সেস লগ করুন। Immutable Backup ব্যবহার করুন — যাতে Ransomware ব্যাকআপ মুছে বা এনক্রিপ্ট করতে না পারে। Cross-cloud Recovery-এর ক্ষেত্রে ডেটা এক ক্লাউড থেকে আরেক ক্লাউডে যাওয়ার সময় এনক্রিপশন ও ডেটা সার্বভৌমত্ব (Data Residency) আইন মেনে চলুন। খরচের দিকে, Hot Standby মানে দ্বিগুণ খরচ — সব সেবার জন্য তা যুক্তিসঙ্গত নয়। RPO ও RTO-এর সঙ্গে খরচের

ভারসাম্য করুন: গুরুত্বপূর্ণ সেবায় বেশি বিনিয়োগ, কম গুরুত্বপূর্ণ সেবায় Backup & Restore যথেষ্ট। DR Drill-এরও খরচ আছে — সময়, সম্পদ — কিন্তু Drill না করার খরচ আরও বেশি: আসল বিভ্রাটে ব্যর্থতা।

২১.৯ সাধারণ ভুল

- Backup নেওয়া কিন্তু Restore পরীক্ষা না করা।
- Backup ও DR-কে একই জিনিস মনে করা।
- Backup একই অঞ্চলে রাখা — অঞ্চল বিভ্রাটে ব্যাকআপও হারানো।
- RPO ও RTO ব্যবসার সঙ্গে আলোচনা না করে ঠিক করা।
- DNS TTL বেশি রাখা — Failover ধীর হওয়া।
- Failback-এর পরিকল্পনা না রাখা।
- DR Runbook লিখে তা কখনো মহড়া না করা।
- Backup-এ Secret বা কনফিগারেশন না রাখা — Restore-এর পর সেবা চালু না হওয়া।

২১.১০ সমস্যা সমাধান (Troubleshooting)

TROUBLESHOOTING: Restore ব্যর্থ হলে প্রথমে ব্যাকআপ ফাইলের অখণ্ডতা যাচাই করুন — Checksum মিলছে কি না। সংস্করণের মিল দেখুন — ব্যাকআপ যে MongoDB সংস্করণে নেওয়া, Restore সেই সংস্করণেই করুন। ডিস্ক জায়গা পরীক্ষা করুন — Restore-এর জন্য ব্যাকআপের চেয়ে বেশি জায়গা দরকার হয়। আর Failover-এর পর সেবা না চললে DNS, সংযোগ স্ট্রিং ও Secret — তিনটি পরীক্ষা করুন।

যদি Replication Lag বাড়তে থাকে, তবে নেটওয়ার্ক ব্যান্ডউইথ ও Primary-এর লেখার হার দেখুন — লেখার হার Replication-এর ক্ষমতার চেয়ে বেশি হলে Lag বাড়বেই। যদি Failover-এর পর ডেটা না মেলে, তবে Replication-এর অবস্থা পরীক্ষা করুন — কোন লেখাগুলো পৌঁছায়নি তা শনাক্ত করুন এবং ব্যবসাকে জানান।

২১.১১ সারসংক্ষেপ

Backup মানে ডেটার কপি, DR মানে পুরো সেবা ফিরিয়ে আনা। RPO বলে কতটুকু ডেটা হারানো যাবে, RTO বলে কতক্ষণে ফিরবে। ৩-২-১ নিয়ম, Immutable Backup, ও নিয়মিত Restore পরীক্ষা — এই তিনটি ব্যাকআপ কৌশলের ভিত্তি। Multi-region কৌশল Backup & Restore থেকে Active-Active পর্যন্ত বিস্তৃত — সঠিক স্তর ব্যবসার RPO ও RTO থেকে আসে। Failover-এর মতো Failback-এরও পরিকল্পনা দরকার, আর DR Runbook-এর মূল্য মহড়ায় প্রমাণিত হয় — কাগজে নয়। পরের অধ্যায়ে আমরা শিখব ব্যর্থতাকে ভয় না পেয়ে তাকে পরীক্ষাগারে নিয়ে আসা — Chaos Engineering।

২১.১২ পর্যালোচনা প্রশ্ন

1. Backup ও Disaster Recovery-এর পার্থক্য কী?
2. RPO ও RTO সংজ্ঞায়িত করুন — একটি উদাহরণসহ।
3. ৩-২-১ নিয়মটি কী? কেন এটি গুরুত্বপূর্ণ?
4. Backup & Restore, Pilot Light, Warm Standby ও Hot Standby-এর পার্থক্য কী?
5. পরীক্ষা না করা ব্যাকআপ কেন কোনো ব্যাকআপ নয়?

6. Asynchronous বনাম Synchronous Replication — কখন কোনটি?
7. Failover ও Failback-এর পার্থক্য কী? Failback কেন কঠিন?
8. DNS TTL Failover-কে কীভাবে প্রভাবিত করে?
9. Point-in-Time Recovery কখন দরকার?
10. Business Continuity পরিকল্পনায় প্রযুক্তির বাইরে কী কী থাকে?

২১.১৩ ব্যবহারিক অনুশীলন

1. TechNova Checkout-এর জন্য RPO ও RTO নির্ধারণ করুন — ব্যবসায়িক যুক্তিসহ। তারপর সেই RPO ও RTO-এর জন্য উপযুক্ত DR কৌশল (Backup & Restore / Pilot Light / Warm Standby / Hot Standby) বেছে নিন ও যুক্তি দিন।
2. Lab পরিবেশে MongoDB-এর একটি ব্যাকআপ নিন, আলাদা ডেটাবেসে Restore করুন, এবং নথির সংখ্যা মিলিয়ে যাচাই করুন। Restore-এর সময় মাপুন — তা আপনার RTO-এর মধ্যে পড়ে কি না দেখুন।
3. একটি এক পৃষ্ঠার DR Runbook লিখুন: ঘোষণা, Failover ধাপ, যোগাযোগ তালিকা, এবং Failback পরিকল্পনা — এবং একটি Drill-এর সময়সূচি ঠিক করুন।

অধ্যায় ২২ • PART V — Enterprise Reliability & Operations

Chapter 22 — Chaos Engineering

২২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Chaos Engineering-এর নীতি ব্যাখ্যা করতে পারবেন এবং তা অনিয়ন্ত্রিত ব্যর্থতা থেকে আলাদা করতে পারবেন। আপনি Hypothesis-চালিত পরীক্ষা নকশা করতে পারবেন — কী আশা করছেন তা আগে লিখে, তারপর ব্যর্থতা ঘটিয়ে, তারপর ফল যাচাই করে। আপনি Kubernetes-এ Pod Termination, Network Failure ও Resource Exhaustion পরীক্ষা করতে পারবেন — শুধুমাত্র বিচ্ছিন্ন Lab পরিবেশে, কঠোর Safety Control ও Rollback পরিকল্পনাসহ। আপনি Blast Radius সীমিত করতে পারবেন, পরীক্ষার ফল বিশ্লেষণ করতে পারবেন, এবং Recovery যাচাই করতে পারবেন। সবচেয়ে গুরুত্বপূর্ণভাবে, আপনি শিখবেন Chaos Engineering-এর উদ্দেশ্য সিস্টেম ভাঙা নয় — সিস্টেমের দুর্বলতা নিরাপদে আবিষ্কার করা।

- Chaos Engineering নীতি ব্যাখ্যা করা ও অনিয়ন্ত্রিত ব্যর্থতা থেকে আলাদা করা।
- Hypothesis-চালিত পরীক্ষা নকশা করা।
- Kubernetes-এ নিরাপদ Chaos পরীক্ষা পরিচালনা করা (Lab-এ)।
- Blast Radius সীমিত করা ও Safety Control প্রয়োগ করা।
- পরীক্ষা বিশ্লেষণ ও Recovery যাচাই করা।

২২.২ প্রাক-প্রয়োজনীয় পুনরালোচনা

আগের অধ্যায়গুলো থেকে মনে রাখুন: Redundancy, Failover, Circuit Breaker, Monitoring, Incident Management ও Disaster Recovery। Chaos Engineering এই সব ধারণার উপর দাঁড়িয়ে আছে — এটি জিজ্ঞাসা করে: আমরা যে নিরাপত্তা ব্যবস্থাগুলো বানিয়েছি, সেগুলো কি আসলেই কাজ করে? Load Balancer কি সত্যিই একটি সার্ভার ভেঙে গেলে ট্র্যাফিক সরায়? MongoDB Replica কি সত্যিই Primary ভেঙে গেলে দায়িত্ব নেয়? Circuit Breaker কি সত্যিই খোলে? এই প্রশ্নগুলোর উত্তর কাগজে নয় — পরীক্ষায় পাওয়া যায়। TechNova-এর প্রসঙ্গে ভাবুন: আপনার স্থাপত্যে তিনটি API সার্ভার আছে — কিন্তু যদি একটি সার্ভার হঠাৎ বন্ধ হয়ে যায়, তবে কি ব্যবহারকারীরা কিছু টের পাবেন? উত্তরটি অনুমান করবেন না — Lab-এ পরীক্ষা করে জানুন। এই অধ্যায়ের সব পরীক্ষা শুধুমাত্র বিচ্ছিন্ন Lab পরিবেশে করতে হবে — কখনো উৎপাদনে নয়, কখনো গ্রাহকের ডেটা নিয়ে নয়।

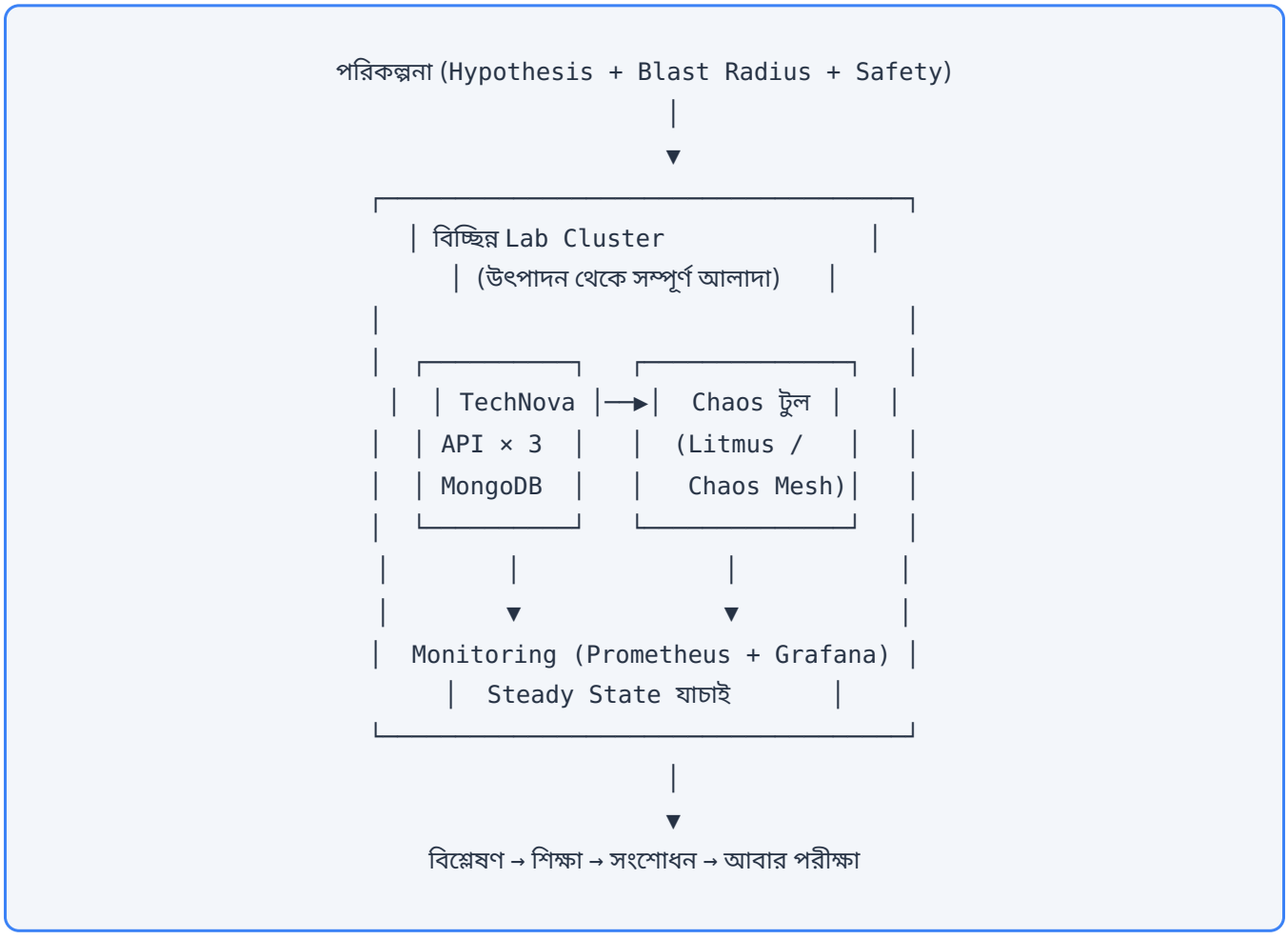
২২.৩ প্রথম নীতি থেকে ব্যাখ্যা: ব্যর্থতা অনিবার্য — তাই তা অনুশীলন করুন

একটি বিতরণকৃত সিস্টেমে ব্যর্থতা স্বাভাবিক — সার্ভার নষ্ট হয়, নেটওয়ার্ক ছিঁড়ে যায়, ডিস্ক পূর্ণ হয়। প্রশ্ন হলো ব্যর্থতা কখন হবে তা নয় — ব্যর্থতা হলে সিস্টেম কী করবে তা। Chaos Engineering-এর মূল ধারণাটি হলো: ব্যর্থতার জন্য অপেক্ষা না করে, নিয়ন্ত্রিতভাবে ব্যর্থতা ঘটিয়ে দেখুন সিস্টেম কী করে। এটি অনিয়ন্ত্রিত ব্যর্থতা থেকে সম্পূর্ণ আলাদা। অনিয়ন্ত্রিত ব্যর্থতা বলতে বোঝায় হঠাৎ, অপ্রত্যাশিত, অনিয়ন্ত্রিত ভাঙন — যার Blast Radius জানা নেই, যার Rollback পরিকল্পনা নেই, যার ফল কেউ পর্যবেক্ষণ করছে না। Chaos Engineering বলতে বোঝায় পরিকল্পিত, সীমিত, পর্যবেক্ষণকৃত পরীক্ষা — যার Hypothesis আছে, Blast Radius সীমিত, Safety Control আছে, এবং যেকোনো মুহূর্তে থামানো যায়। পার্থক্যটি উদ্দেশ্যে নয় — পদ্ধতিতে। একজন চিকিৎসক যেমন টিকা দিয়ে শরীরকে প্রস্তুত করেন, Chaos Engineering তেমন সিস্টেমকে ব্যর্থতার জন্য প্রস্তুত করে — ছোট, নিয়ন্ত্রিত মাত্রায়।

একটি ভালো Chaos পরীক্ষার চারটি উপাদান আছে। প্রথম, Steady State Hypothesis — স্বাভাবিক অবস্থায় সিস্টেম কেমন আচরণ করে তা সংজ্ঞায়িত করুন: যেমন Error Rate ১%-এর নিচে, p95 Latency ৩০০ মিলিসেকেন্ডের নিচে। দ্বিতীয়, Hypothesis — ব্যর্থতা ঘটলে কী হবে বলে আশা করছেন: যেমন একটি Pod বন্ধ হলে Error Rate সাময়িকভাবে বাড়বে, কিন্তু ৩০ সেকেন্ডের মধ্যে স্বাভাবিক হবে। তৃতীয়, পরীক্ষা — সীমিত Blast Radius-এ ব্যর্থতা ঘটান: একটি Pod, একটি Namespace, একটি Lab Cluster — কখনো পুরো সিস্টেম নয়। চতুর্থ, বিশ্লেষণ — Hypothesis সত্য হলো কি না যাচাই করুন, এবং যা শিখলেন তা নথিভুক্ত করুন। যদি Hypothesis ভুল প্রমাণিত হয় — যেমন Error Rate ২ মিনিট ধরে বেশি থাকল — তবে সেটিই পরীক্ষার সাফল্য: একটি দুর্বলতা ধরা পড়েছে, তা ঠিক করার আগেই।

> **WARNING:** Chaos পরীক্ষা শুধুমাত্র বিচ্ছিন্ন Lab পরিবেশে, কঠোর Safety Control ও Rollback পরিকল্পনাসহ করতে হবে। কখনো উৎপাদনে, কখনো গ্রাহকের ডেটা নিয়ে, কখনো অনুমোদন ছাড়া Chaos পরীক্ষা করবেন না। একটি পরীক্ষার আগে সবসময় লিখিত অনুমোদন নিন, Blast Radius স্পষ্ট করুন, এবং Abort করার পদ্ধতি ঠিক রাখুন।

২২.৪ স্থাপত্য: নিরাপদ Chaos পরীক্ষার প্রবাহ



এই স্থাপত্যের সবচেয়ে গুরুত্বপূর্ণ অংশটি হলো বিচ্ছিন্নতা: Lab Cluster উৎপাদন থেকে সম্পূর্ণ আলাদা — আলাদা নেটওয়ার্ক, আলাদা ডেটা, আলাদা অ্যাকাউন্ট। Chaos টুল (যেমন LitmusChaos বা Chaos Mesh) শুধু Lab Cluster-এ ইনস্টল করা থাকে। Monitoring আলাদাভাবে চলে — যাতে পরীক্ষার সময় সিস্টেমের আচরণ পর্যবেক্ষণ করা যায়। আর Safety Control হিসেবে থাকে: সময়সীমা (পরীক্ষা নির্দিষ্ট সময় পর নিজে থেকে থামবে), স্বয়ংক্রিয় Abort (Error Rate একটি সীমা পার হলে পরীক্ষা থামবে), এবং Rollback পরিকল্পনা (সবকিছু আগের অবস্থায় ফেরানো যাবে)।

২২.৫ অভ্যন্তরীণ কর্মনীতি: পরীক্ষার ধরন, Blast Radius ও Safety Control

Pod Termination পরীক্ষা: একটি API Pod হঠাৎ বন্ধ করে দিন। প্রত্যাশা: Kubernetes নতুন Pod তৈরি করবে, Load Balancer ট্র্যাফিক সরিয়ে নেবে, ব্যবহারকারীরা কিছু টের পাবেন না। যদি Error Rate বাড়ে বা Recovery ৩০ সেকেন্ডের বেশি লাগে, তবে Readiness Probe, Replica সংখ্যা, বা Pod Startup সময় পরীক্ষা করুন। Network Failure পরীক্ষা: API ও MongoDB-এর মধ্যে নেটওয়ার্কে বিলম্ব (Latency) বা বিচ্ছিন্নতা (Partition) তৈরি করুন। প্রত্যাশা: Timeout কাজ করবে, Circuit Breaker খুলবে, ব্যবহারকারী স্পষ্ট ত্রুটি বার্তা পাবেন — কিন্তু সিস্টেম সম্পূর্ণ থেমে যাবে না। Resource Exhaustion পরীক্ষা: CPU বা মেমরি সীমিত করে দিন। প্রত্যাশা: Resource Limit কাজ করবে, একটি Pod-এর সমস্যা অন্যকে প্রভাবিত করবে না, HPA (Horizontal Pod Autoscaler) প্রয়োজনে Pod বাড়াবে। প্রতিটি পরীক্ষায় Blast Radius সীমিত করুন: প্রথমে একটি Pod, তারপর একটি Deployment, কখনো পুরো Namespace একসঙ্গে নয়। এবং প্রতিটি পরীক্ষার আগে Steady State পরিমাপ করুন — পরীক্ষার আগের স্বাভাবিক অবস্থা না জানলে পরীক্ষার ফল বোঝা যাবে না।

২২.৬ বাস্তব কমান্ড ও কনফিগারেশন

Lab-এ একটি নিরাপদ Pod Termination পরীক্ষার ধারণা — kubectl দিয়ে হাতে (সবচেয়ে সহজ নিয়ন্ত্রিত পরীক্ষা):

```
# LAB ONLY — কখনো উৎপাদনে নয়
# ১. Steady State পরিমাপ করুন (পরীক্ষার আগে)
kubectl -n technova-lab get pods
curl -s http://203.0.113.51:3000/health
# ২. একটি Pod বন্ধ করুন (Deployment নিজে নতুন Pod তৈরি করবে)
kubectl -n technova-lab delete pod -l app=technova-api --grace-period=0 --force=false
# ৩. পর্যবেক্ষণ করুন — নতুন Pod কতক্ষণে Ready হয়?
kubectl -n technova-lab get pods -w
# ৪. Recovery যাচাই — Health Check ও Error Rate দেখুন
curl -s http://203.0.113.51:3000/health; echo
kubectl -n technova-lab rollout status deployment/technova-api --timeout=120s
```

Chaos Mesh-এ Network Delay পরীক্ষার একটি নমুনা (Lab-এ):

```
apiVersion: chaos-mesh.org/v1alpha1
kind: NetworkChaos
metadata:
  name: technova-lab-network-delay
  namespace: technova-lab
spec:
  action: delay
  mode: one
  selector:
    namespaces: [technova-lab]
    labelSelectors: { app: technova-api }
  delay:
    latency: "200ms"
  duration: "2m"
  # Safety: পরীক্ষা ২ মিনিট পর নিজে থেকে শেষ হবে
```

পরীক্ষা নথির কাঠামো:

```
পরীক্ষা: Pod Termination — TechNova API (Lab)
তারিখ: ২০২৬-১১-০৯ | পরিবেশ: বিচ্ছিন্ন Lab Cluster | অনুমোদন: [নাম]
```

```
Hypothesis: একটি Pod বন্ধ হলে Error Rate ৩০ সেকেন্ডের মধ্যে স্বাভাবিক হবে
Blast Radius: ১টি Pod, technova-lab Namespace, কোনো গ্রাহক ডেটা নয়
Safety: ২ মিনিট সময়সীমা; Error Rate > ৫% হলে Abort
ফল: Recovery সময় ১৮ সেকেন্ড; Hypothesis সত্য
শিক্ষা: Readiness Probe সঠিকভাবে কাজ করছে
পরবর্তী: Network Partition পরীক্ষা (পরের সপ্তাহে)
```

২২.৭ উৎপাদন পরিস্থিতি

পরিস্থিতি ১ — Pod Termination-এ প্রত্যাশার চেয়ে ধীর Recovery। Lab পরীক্ষায় দেখা গেল নতুন Pod Ready হতে ৯০ সেকেন্ড লাগছে — কারণ Image Pull ধীর ও Startup-এ ডেটাবেস সংযোগে সময় লাগছে। শিক্ষা: Image আগে থেকে Pull করা, Readiness Probe ঠিক করা, এবং Startup সময় কমানো। এই দুর্বলতা উৎপাদনে ধরা পড়লে ব্যবহারকারীরা ৯০ সেকেন্ড ধরে ব্যর্থতা দেখতেন। **পরিস্থিতি ২ — Network Partition-এ Split Brain।** Lab-এ API ও MongoDB-এর মধ্যে নেটওয়ার্ক বিচ্ছিন্ন করা হলো — দেখা গেল API ভুলভাবে নিজেকে সুস্থ মনে করছে, আর ত্রুটি বার্তা অস্পষ্ট। শিক্ষা: Health Check-এ ডেটাবেস সংযোগ যাচাই করা, এবং Timeout স্পষ্ট করা। **পরিস্থিতি ৩ — Resource Exhaustion-এ একটি Pod অন্যকে ডোবাল।** একটি Pod-এর মেমরি লিক পুরো Node-কে প্রভাবিত করছিল — কারণ Resource Limit বসানো হয়নি। শিক্ষা: প্রতিটি Pod-এ Request ও Limit বসানো, এবং Node-এর সম্পদ পর্যবেক্ষণ করা।

পরিস্থিতি ৬ — Hypothesis লেখার শিল্প। একটি দলের প্রথম Hypothesis ছিল: সিস্টেম ভাঙবে না। এটি পরীক্ষা করার মতো নয় — কারণ তা ভুল প্রমাণ করা যায় না, পরিমাপ করা যায় না। তারা Hypothesis আবার লিখল: একটি Pod বন্ধ হলে Error Rate ১%-এর নিচে থাকবে, p95 Latency ৩০০ মিলিসেকেন্ডের নিচে থাকবে, আর Recovery ৩০ সেকেন্ডের মধ্যে হবে। এখন পরীক্ষার ফল স্পষ্ট — সংখ্যা বলে Hypothesis সত্য না মিথ্যা। শিক্ষা: ভালো Hypothesis সুনির্দিষ্ট, পরিমাপযোগ্য, ও সময়সীমাযুক্ত। অস্পষ্ট Hypothesis থেকে অস্পষ্ট শিক্ষা আসে।

পরিস্থিতি ৭ — Chaos ও নিরাপত্তা। একটি Chaos পরীক্ষায় Network Partition তৈরি করা হলো — কিন্তু পরীক্ষাটি ভুল করে Lab-এর বাইরে একটি সেবাকেও প্রভাবিত করল — কারণ Network Policy ঠিক ছিল না। ভাগ্য ভালো, প্রভাব সামান্য ছিল। দল নিয়ম কঠোর করল: Lab Cluster-এর নেটওয়ার্ক সম্পূর্ণ বিচ্ছিন্ন, Chaos টুলের অ্যাক্সেস সীমিত, আর প্রতিটি পরীক্ষার আগে Blast Radius দুজন ব্যক্তি যাচাই করবেন। শিক্ষা: Chaos পরীক্ষার নিরাপত্তা পরীক্ষার চেয়ে গুরুত্বপূর্ণ — একটি অনিরাপদ পরীক্ষা একটি বিভ্রাটের চেয়ে খারাপ, কারণ তা নিজের হাতে তৈরি।

পরিস্থিতি ৮ — Chaos থেকে স্থাপত্য পরিবর্তন। Lab পরীক্ষায় বারবার দেখা গেল একটি সেবা একক ব্যর্থতায় ভেঙে পড়ে — কারণ তার কোনো Redundancy নেই। দল স্থাপত্য বদলাল: সেবাটির Replica বাড়ানো হলো, Circuit Breaker যোগ করা হলো, আর Health Check-এ ডেটাবেস সংযোগ যাচাই যোগ করা হলো। পরের পরীক্ষায় সেবাটি টিকে গেল। শিক্ষা: Chaos পরীক্ষার আসল ফল একটি রিপোর্ট নয় — একটি স্থাপত্য পরিবর্তন। পরীক্ষা করে যদি কিছু না বদলায়, তবে পরীক্ষাটি সময়ের অপচয়।

পরিস্থিতি ৪ — Game Day। TechNova প্রতি তিন মাসে একটি Game Day করে: একটি নির্ধারিত দিনে, Lab-এ, দল একসঙ্গে বসে একটি বড় পরীক্ষা চালায় — যেমন MongoDB Primary বন্ধ করে দেওয়া। সবাই দেখে কী হয়, কে কী করে, যোগাযোগ কেমন হয়। প্রথম Game Day-এ দেখা গেল কেউ জানে না Failover-এর সিদ্ধান্ত কে নেবে — Commander-এর ভূমিকা স্পষ্ট ছিল না। পরের Game Day-এর আগে ভূমিকা লিখে রাখা হলো। শিক্ষা: Chaos পরীক্ষা শুধু প্রযুক্তি পরীক্ষা করে না — মানুষ ও প্রক্রিয়াও পরীক্ষা করে। Game Day সেই পরীক্ষার সেরা উপায়।

পরিস্থিতি ৫ — পরীক্ষার অগ্রাধিকার। দলের কাছে পরীক্ষা করার মতো দশটি ধারণা ছিল, কিন্তু সময় ছিল একটির। তারা প্রতিটি ধারণাকে দুটি মাপে মাপল: এই ব্যর্থতা কতটা সম্ভাব্য, আর হলে কতটা ক্ষতি হবে। MongoDB Primary ব্যর্থতা সম্ভাব্য ও ক্ষতিকর — তাই তা প্রথমে। একটি বিরল নেটওয়ার্ক বিভ্রাট কম সম্ভাব্য — তাই তা পরে। শিক্ষা: Chaos পরীক্ষা ঝুঁকি দিয়ে

অগ্রাধিকার দিন — সব ব্যর্থতা সমান নয়। আর প্রতিটি পরীক্ষার পর শিক্ষা নথিভুক্ত করুন — নইলে পরের পরীক্ষায় একই প্রশ্ন আবার উঠবে।

২২.৮ নিরাপত্তা ও খরচ বিবেচনা

Chaos Engineering-এর নিরাপত্তা নিয়মগুলো আপসহীন: শুধু Lab-এ, শুধু অনুমোদনসহ, শুধু সীমিত Blast Radius-এ। Chaos টুলের অ্যাক্সেস সীমিত রাখুন — যে কেউ যাতে Pod মুছে ফেলতে না পারে। পরীক্ষার সময় কোনো আসল Secret বা গ্রাহক ডেটা ব্যবহার করবেন না — Lab ডেটা কাল্পনিক হতে হবে। খরচের দিকে, Lab Cluster-এর খরচ আছে — কিন্তু তা উৎপাদন বিভ্রাটের খরচের তুলনায় সামান্য। পরীক্ষার ফ্রিকোয়েন্সি ঠিক করুন: প্রতি সপ্তাহে একটি ছোট পরীক্ষা, প্রতি মাসে একটি বড় পরীক্ষা — নিয়মিত, কিন্তু অতিরিক্ত নয়। আর প্রতিটি পরীক্ষার শিক্ষা থেকে সংশোধন করুন — পরীক্ষা করে ফল ফেলে রাখলে তা সময়ের অপচয়।

২২.৯ সাধারণ ভুল

- উৎপাদনে বা গ্রাহক ডেটা নিয়ে Chaos পরীক্ষা করা।
- Hypothesis ছাড়া পরীক্ষা করা — ফলে ফল বোঝা যায় না।
- Blast Radius সীমিত না করা — পুরো সিস্টেম একসঙ্গে ভাঙা।
- Steady State না মেপে পরীক্ষা শুরু করা।
- Safety Control ও Abort পদ্ধতি ছাড়া পরীক্ষা করা।
- পরীক্ষার ফল থেকে সংশোধন না করা।
- Chaos-কে সিস্টেম ভাঙার খেলা মনে করা।
- অনুমোদন ছাড়া পরীক্ষা করা।

২২.১০ সমস্যা সমাধান (Troubleshooting)

TROUBLESHOOTING: পরীক্ষার পর সিস্টেম স্বাভাবিক না হলে প্রথমে পরীক্ষাটি থামান (Abort) — Chaos টুলের Stop কমান্ড চালান, বা Pod-গুলো হাতে পুনরায় তৈরি করুন। তারপর Steady State-এর সঙ্গে তুলনা করুন: কোন মেট্রিক এখনো স্বাভাবিক নয়? লগ দেখুন — পরীক্ষাটি কোন অপ্রত্যাশিত প্রভাব ফেলেছে? যদি Lab Cluster-ই অস্থির হয়ে যায়, তবে Cluster পুনরায় তৈরি করুন — Lab-এর উদ্দেশ্যই হলো নিরাপদে ভাঙা।

যদি পরীক্ষা কোনো প্রভাব না ফেলে, তবে সন্দেহ করুন: Chaos টুল সঠিক Namespace-এ চলছে কি না, Selector ঠিক আছে কি না, নাকি সিস্টেম সত্যিই এত শক্তিশালী। যদি পরীক্ষা প্রত্যাশার চেয়ে বড় প্রভাব ফেলে, তবে Blast Radius পর্যালোচনা করুন — হয়তো Selector ভুল ছিল।

২২.১১ সারসংক্ষেপ

Chaos Engineering হলো নিয়ন্ত্রিত, Hypothesis-চালিত ব্যর্থতা পরীক্ষা — অনিয়ন্ত্রিত ভাঙন নয়। এর উদ্দেশ্য সিস্টেম ভাঙা নয়, দুর্বলতা নিরাপদে আবিষ্কার করা। প্রতিটি পরীক্ষায় Hypothesis, সীমিত Blast Radius, Safety Control ও Rollback পরিকল্পনা থাকতে হবে — এবং তা শুধুমাত্র বিচ্ছিন্ন Lab পরিবেশে করতে হবে। Pod Termination, Network Failure ও Resource Exhaustion পরীক্ষা করে আমরা জানতে পারি আমাদের Redundancy, Timeout ও Circuit

Breaker আসলেই কাজ করে কি না। পরীক্ষার ফল থেকে সংশোধন করাই Chaos Engineering-এর আসল মূল্য। পরের অধ্যায়ে আমরা শিখব নির্ভরযোগ্যতার আরেকটি দিক — খরচ: FinOps ও Operational Governance।

২২.১২ পর্যালোচনা প্রশ্ন

1. Chaos Engineering কাকে বলে? এটি অনিয়ন্ত্রিত ব্যর্থতা থেকে কীভাবে আলাদা?
2. Steady State Hypothesis কী? কেন এটি দরকার?
3. Blast Radius বলতে কী বোঝায়? এটি কীভাবে সীমিত করবেন?
4. Pod Termination পরীক্ষা থেকে কী শেখা যায়?
5. Network Partition পরীক্ষায় কী প্রত্যাশা করবেন?
6. Safety Control-এর তিনটি উদাহরণ দিন।
7. Chaos পরীক্ষা কেন শুধু Lab-এ করতে হবে?
8. Hypothesis ভুল প্রমাণিত হলে তা কি পরীক্ষার ব্যর্থতা? কেন?
9. Recovery Verification বলতে কী বোঝায়?
10. Chaos পরীক্ষার ফল থেকে সংশোধন না করলে কী ক্ষতি?

২২.১৩ ব্যবহারিক অনুশীলন

1. একটি Chaos পরীক্ষার সম্পূর্ণ নথি লিখুন: Hypothesis, Blast Radius, Safety Control, Rollback পরিকল্পনা, অনুমোদন — Pod Termination পরীক্ষার জন্য (Lab-এর জন্য)।
2. Lab Kubernetes Cluster-এ একটি Pod Termination পরীক্ষা চালান (অথবা তার ধাপগুলো লিখে অনুশীলন করুন): Steady State মাপুন, Pod বন্ধ করুন, Recovery সময় মাপুন, ফল বিশ্লেষণ করুন।
3. তিনটি ভিন্ন পরীক্ষার Hypothesis লিখুন — Network Delay, Resource Exhaustion, ও MongoDB Replica ব্যর্থতা — প্রতিটির প্রত্যাশিত ফল ও সাফল্যের মানদণ্ডসহ।

অধ্যায় ২৩ • PART V — Enterprise Reliability & Operations

Chapter 23 — FinOps and Operational Governance

২৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি FinOps-এর মূলনীতি ব্যাখ্যা করতে পারবেন এবং Cost Reduction ও Cost Optimization-এর পার্থক্য বুঝতে পারবেন। আপনি Cloud Billing বিশ্লেষণ করতে পারবেন, Cost Allocation ও Tagging নকশা করতে পারবেন, এবং Budget ও Alert বসাতে পারবেন। আপনি Rightsizing, Idle Resource Detection, Storage Optimization ও Data Egress খরচ বিশ্লেষণ করতে পারবেন। আপনি Kubernetes Cost Allocation করতে পারবেন, Showback ও Chargeback-এর পার্থক্য বুঝবেন, এবং Cost Governance নীতি তৈরি করতে পারবেন। সবচেয়ে গুরুত্বপূর্ণভাবে, আপনি শিখবেন খরচ কমানো নয় — মূল্যের জন্য খরচ করা — FinOps-এর লক্ষ্য। কোনো নির্দিষ্ট মূল্য এখানে উদ্ভাবন করা হয়নি; সব সংখ্যা আপনার নিজের বিল থেকে নিতে হবে।

- FinOps নীতি ও Cost Optimization ব্যাখ্যা করা।
- Cloud Billing বিশ্লেষণ ও Tagging নকশা করা।
- Rightsizing ও Idle Resource শনাক্ত করা।
- Kubernetes Cost Allocation করা।
- Showback, Chargeback ও Cost Governance প্রয়োগ করা।

২৩.২ প্রাক-প্রয়োজনীয় পুনরালোচনা

আগের অধ্যায়গুলো থেকে মনে রাখুন: Redundancy, Multi-region, Monitoring, Capacity Planning — এই সবকিছুরই খরচ আছে। একটি অতিরিক্ত সার্ভার নির্ভরযোগ্যতা বাড়ায়, কিন্তু বিলও বাড়ায়। একটি Hot Standby DR অঞ্চল RTO কমায়, কিন্তু খরচ দ্বিগুণ করে। FinOps হলো এই সিদ্ধান্তগুলোকে অর্থের ভাষায় দেখা: আমরা কীসের জন্য খরচ করছি, তা থেকে কী মূল্য পাচ্ছি, আর কোথায় অপচয় হচ্ছে। TechNova-এর প্রসঙ্গে ভাবুন: API সার্ভার তিনটি চলছে, MongoDB Replica চলছে, Redis চলছে, Lab Cluster চলছে, ব্যাকআপ সংরক্ষণ চলছে — প্রতিটির বিল আসছে। কিন্তু কোন দল, কোন ফিচার, কোন পরিবেশ কত খরচ করছে — তা কি জানা যায়? Tagging ছাড়া জানা যায় না। আর না জানলে অপচয় চোখে পড়ে না। এই অধ্যায়ে আমরা কোনো ক্লাউডের নির্দিষ্ট দাম বলব না — দাম বদলায়, অঞ্চলভেদে বদলায়। বদলে আমরা শিখব দাম যাচাই করার পদ্ধতি: নিজের বিল থেকে, নিজের ব্যবহার থেকে।

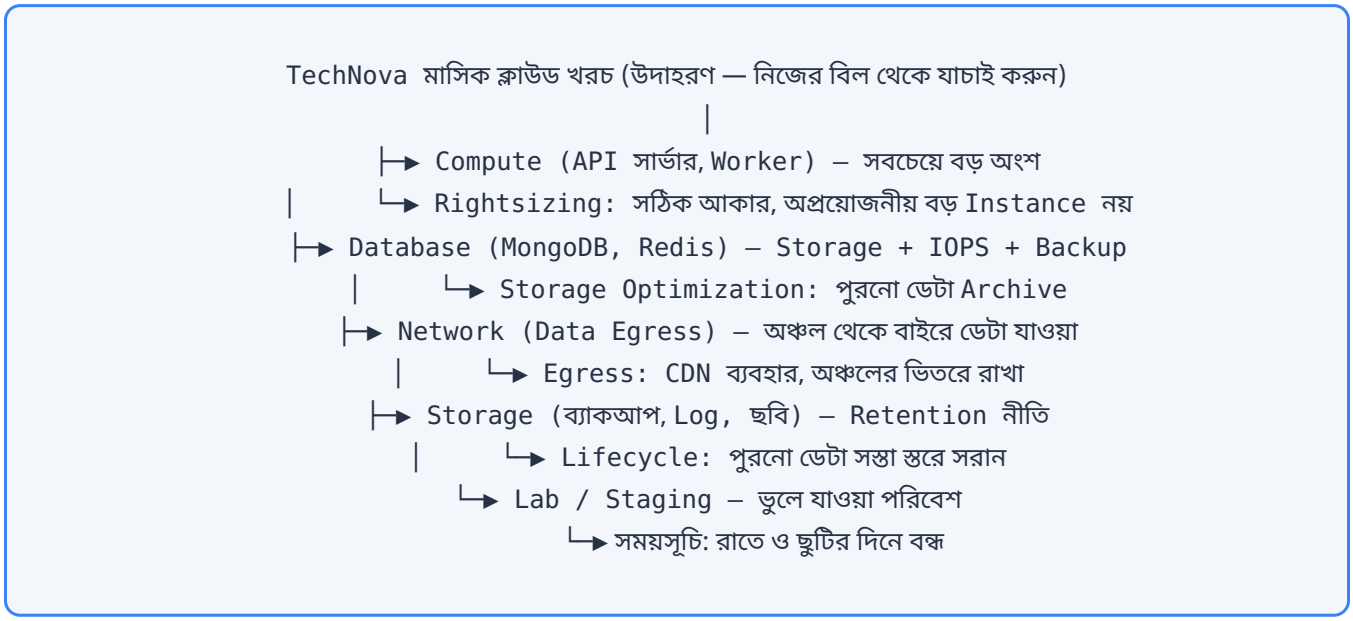
২৩.৩ প্রথম নীতি থেকে ব্যাখ্যা: খরচ কমানো নয়, মূল্য বাড়ানো

FinOps-এর সবচেয়ে সাধারণ ভুল বোঝাবুঝি হলো একে খরচ কমানোর কাজ মনে করা। Cost Reduction বলতে বোঝায় শুধু খরচ কমানো — সার্ভার বন্ধ করা, সেবা ছোট করা — ফল যাই হোক। Cost Optimization বলতে বোঝায় মূল্যের জন্য খরচ করা — যে খরচ মূল্য দেয় তা রাখা, যে খরচ অপচয় তা বন্ধ করা। পার্থক্যটি গুরুত্বপূর্ণ: একটি সার্ভার বন্ধ করে খরচ কমানো যায়, কিন্তু তাতে যদি Checkout ধীর হয়ে গ্রাহক চলে যায়, তবে সেই সাশ্রয় ক্ষতির চেয়ে কম। আবার একটি Idle সার্ভার — যা কেউ ব্যবহার করে না — বন্ধ করা খাঁটি অপচয় দূর করা। FinOps-এর মূল নীতিটি হলো: প্রতিটি খরচের পিছনে মূল্য থাকতে হবে, এবং সেই মূল্য পরিমাপযোগ্য হতে হবে। একটি নতুন ফিচারের জন্য সার্ভার বাড়ানো যুক্তিসঙ্গত — যদি ফিচারটি আয় বাড়ায়। একটি Lab Cluster-এর খরচ যুক্তিসঙ্গত — যদি তা বিভ্রাট রোধ করে। কিন্তু একটি ভুলে যাওয়া Test সার্ভার, একটি অপ্রয়োজনীয় বড় ডিস্ক, একটি কখনো পড়া হয় না এমন Log সংরক্ষণ — এগুলো অপচয়।

FinOps তিনটি ধাপে কাজ করে: Inform (জানুন) — খরচ কোথায় যাচ্ছে তা দৃশ্যমান করুন; Optimize (অপ্টিমাইজ করুন) — অপচয় দূর করুন, সঠিক আকার দিন; Operate (পরিচালনা করুন) — নিয়মিত পর্যালোচনা করুন, Budget ও Alert বসান। এই চক্রটি চলমান — একবার করলেই শেষ নয়। ক্লাউড খরচের একটি বৈশিষ্ট্য হলো এটি প্রতিদিন বদলায়: নতুন সেবা যোগ হয়, ট্র্যাফিক বাড়ে, দাম বদলায়। তাই FinOps একটি সংস্কৃতি — অর্থ, প্রকৌশল ও ব্যবসা একসঙ্গে বসে খরচের সিদ্ধান্ত নেয়। প্রকৌশলী জানেন কী চলছে, অর্থ দল জানেন কত খরচ হচ্ছে, ব্যবসা জানেন কী মূল্য পাওয়া যাচ্ছে — তিনটি মিললেই ভালো সিদ্ধান্ত হয়।

> **CONCEPT:** Cost Reduction = শুধু খরচ কমানো, ফল যাই হোক। Cost Optimization = মূল্যের জন্য খরচ করা — অপচয় দূর করা, মূল্যবান খরচ রাখা। FinOps-এর লক্ষ্য দ্বিতীয়টি। একটি খরচের প্রশ্ন সবসময়: এই খরচ থেকে কী মূল্য পাচ্ছি?

২৩.৪ স্থাপত্য: খরচ কোথা থেকে আসে



এই চিত্রে খরচের প্রধান উৎসগুলো দেখানো হয়েছে। Compute সাধারণত সবচেয়ে বড় অংশ — তাই Rightsizing সবচেয়ে বেশি সাশ্রয় দেয়। Database-এর খরচ শুধু সার্ভার নয় — Storage, IOPS, Backup, Replication — সব মিলিয়ে। Network-এর মধ্যে Data Egress সবচেয়ে চতুর খরচ: ডেটা ক্লাউডে ঢোকা সাধারণত বিনামূল্যে, কিন্তু বের হওয়ার সময় খরচ হয় — আর তা দ্রুত বাড়তে পারে। Storage-এর খরচ ধীরে ধীরে বাড়ে — Log ও ব্যাকআপ জমতে থাকে, কেউ মুছে না। আর Lab ও Staging পরিবেশ প্রায়ই ভুলে যাওয়া খরচ — কেউ ব্যবহার করে না, কিন্তু বিল আসতেই থাকে।

২৩.৫ অভ্যন্তরীণ কর্মনীতি: Tagging, Rightsizing, Kubernetes খরচ ও Governance

Cost Allocation-এর ভিত্তি হলো Tagging — প্রতিটি সম্পদে Label লাগানো: কোন দল, কোন সেবা, কোন পরিবেশ (উৎপাদন / Staging / Lab), কোন প্রকল্প। Tagging ছাড়া বিল একটি বড় সংখ্যা — কার খরচ তা জানা যায় না। একটি Tagging নীতি ঠিক করুন: বাধ্যতামূলক Tag (যেমন team, service, environment, cost-center), নামকরণের নিয়ম, এবং নিয়মিত Audit — Tag ছাড়া সম্পদ তৈরি হলে তা চিহ্নিত করা। Showback বলতে বোঝায় প্রতিটি দলকে তাদের খরচ দেখানো — কিন্তু বিল না করা। এর উদ্দেশ্য সচেতনতা: দল যখন দেখে তাদের ফিচার কত খরচ করছে, তখন তারা নিজেরাই

অপ্টিমাইজ করে। Chargeback বলতে বোঝায় আসলেই বিল করা — দলের বাজেট থেকে খরচ কাটা। Chargeback শক্তিশালী, কিন্তু জটিল — ভাগ করার নিয়ম নিয়ে বিতর্ক হতে পারে। বেশিরভাগ সংস্থা Showback দিয়ে শুরু করে, তারপর ধীরে ধীরে Chargeback-এ যায়।

Rightsizing বলতে বোঝায় সম্পদের আকার প্রকৃত ব্যবহারের সঙ্গে মেলানো। একটি সার্ভার যদি গড়ে ১০% CPU ব্যবহার করে, তবে তা প্রয়োজনের চেয়ে অনেক বড় — ছোট করুন। Idle Resource Detection বলতে বোঝায় এমন সম্পদ খোঁজা যা চলছে কিন্তু ব্যবহার হচ্ছে না — যেমন একটি ডেটাবেস যা গত ৩০ দিনে কেউ ব্যবহার করেনি, একটি Load Balancer যার পিছনে কোনো সার্ভার নেই, একটি ডিস্ক যা কোনো সার্ভারে লাগানো নেই। Storage Optimization-এর মধ্যে আছে: পুরনো Log ও ব্যাকআপ Archive করা, অপ্রয়োজনীয় Snapshot মুছে ফেলা, এবং Storage Class বদলানো — ঘন ঘন পড়া ডেটা দ্রুত স্তরে, বিরল ডেটা সস্তা স্তরে। Data Egress কমানোর উপায়: CDN ব্যবহার করা (ব্যবহারকারীর কাছাকাছি থেকে ডেটা দেওয়া), অঞ্চলের ভিতরে ডেটা রাখা, এবং অপ্রয়োজনীয় বাহ্যিক স্থানান্তর এড়ানো। Kubernetes Cost Allocation-এর জন্য প্রতিটি Namespace, Deployment ও Pod-এর CPU ও মেমরি ব্যবহার পরিমাপ করুন — Kubecost বা OpenCost-এর মতো টুল ব্যবহার করে — এবং সেই অনুযায়ী খরচ ভাগ করুন। একটি Pod যদি Cluster-এর ১০% CPU ব্যবহার করে, তবে তার খরচ Cluster খরচের প্রায় ১০%।

২৩.৬ বাস্তব কমান্ড ও কনফিগারেশন

খরচ পর্যালোচনার জন্য একটি সহজ চেকলিস্ট (সংখ্যা নিজের বিল থেকে নিন):

```
# ১. চলমান সম্পদের তালিকা — কী চলছে, কী ব্যবহার হচ্ছে?
#    (ক্লাউড কনসোল বা CLI থেকে — এখানে কোনো নির্দিষ্ট দাম নেই)
# ২. Kubernetes-এ Pod-এর সম্পদ ব্যবহার দেখুন
kubectl -n technova top pods
kubectl -n technova top nodes
# ৩. ডিস্ক ব্যবহার — কোন ডিস্ক কতটা পূর্ণ?
df -h
du -sh /var/log/* 2>/dev/null | sort -hr | head -n 10
# ৪. MongoDB সংগ্রহের আকার — কোন সংগ্রহ কত বড়?
mongosh --eval 'db.getCollectionNames().forEach(c => print(c, db[c].stats().size))' \
  "mongodb://203.0.113.31:27017/technova" -u CHANGE-ME-USER -p 'CHANGE-ME-PASSWORD' \
  --authenticationDatabase admin
```

Budget Alert নীতির উদাহরণ:

```
FinOps Budget নীতি — TechNova (মাসিক)
- মোট বাজেট: ব্যবসার সঙ্গে ঠিক করা সংখ্যা (নিজের বিল থেকে)
- Alert ৫০%: অর্থ দলকে জানান — পর্যালোচনা শুরু
- Alert ৮০%: প্রকৌশল দলকে জানান — নতুন সম্পদ তৈরিতে অনুমোদন দরকার
- Alert ১০০%: ব্যয় বন্ধ — জরুরি পর্যালোচনা, অপ্রয়োজনীয় সম্পদ বন্ধ
- Tagging নীতি: team, service, environment, cost-center — চারটি Tag বাধ্যতামূলক
- পর্যালোচনা: প্রতি মাসের প্রথম সপ্তাহে FinOps বৈঠক (অর্থ + প্রকৌশল + ব্যবসা)
```

Kubernetes-এ সম্পদের সীমা — Rightsizing-এর ভিত্তি:

```
resources:
  requests: { cpu: "250m", memory: "512Mi" }  # প্রয়োজনের ভিত্তিতে
  limits: { cpu: "1000m", memory: "1Gi" }    # সীমা — একটি Pod অন্যকে ডোবাবে না
# Request খুব বেশি হলে সম্পদ অপচয়; খুব কম হলে Pod Throttle হবে
# প্রকৃত ব্যবহার (kubectl top) দেখে Request ঠিক করুন
```

২৩.৭ উৎপাদন পরিস্থিতি

পরিস্থিতি ১ — ভুলে যাওয়া Lab Cluster। একটি পরীক্ষার জন্য তৈরি Lab Cluster ছয় মাস ধরে চলছে, কেউ ব্যবহার করছে না — কিন্তু বিল আসছে। FinOps পর্যালোচনায় ধরা পড়ল। সমাধান: Lab-এর জন্য সময়সূচি — রাতে ও ছুটির দিনে স্বয়ংক্রিয়ভাবে বন্ধ, প্রয়োজনে চালু। **পরিস্থিতি ২ — অতিরিক্ত বড় ডেটাবেস।** MongoDB Instance-এর CPU গড়ে ১৫% — প্রয়োজনের চেয়ে তিনগুণ বড়। Rightsizing করে ছোট Instance-এ নেওয়া হলো — Performance একই, খরচ কমল। **পরিস্থিতি ৩ — Egress খরচ হঠাৎ বাড়ল।** একটি নতুন ফিচার প্রতিটি অনুরোধে বড় ছবি অন্য অঞ্চল থেকে টানছিল — Egress খরচ দ্বিগুণ হয়ে গেল। সমাধান: CDN ব্যবহার করা, ছবি ব্যবহারকারীর কাছাকাছি অঞ্চলে রাখা।

পরিস্থিতি ৬ — Tagging-এর ব্যর্থতা ও পুনরুদ্ধার। TechNova-তে Tagging নীতি ছিল — কিন্তু কেউ তা মানছিল না, কারণ Tag দেওয়া ঐচ্ছিক ছিল। বিলের ৪০% খরচের মালিক জানা যাচ্ছিল না। দল Policy বসাল: Tag ছাড়া সম্পদ তৈরি করা যাবে না — IaC-তে Tag বাধ্যতামূলক, নইলে Deploy ব্যর্থ হবে। তিন মাস পর অজানা খরচ ৪০% থেকে ৫%-এ নামল। শিক্ষা: নীতি ঐচ্ছিক হলে তা নীতি নয় — পরামর্শ। Tagging-এর মতো ভিত্তিমূলক কাজে বাধ্যবাধকতা দরকার, আর তা স্বয়ংক্রিয়ভাবে প্রয়োগ করতে হয়।

পরিস্থিতি ৭ — Reserved Capacity বনাম নমনীয়তা। TechNova-এর বেসলাইন ট্র্যাফিক স্থিতিশীল — তাই কিছু Compute-এর জন্য দীর্ঘমেয়াদী প্রতিশ্রুতি (Reserved) নেওয়া হলো — বিল কমল। কিন্তু প্রচারের সময়ের অতিরিক্ত ট্র্যাফিকের জন্য On-demand রাখা হলো — নমনীয়তা। শিক্ষা: সব খরচ একইভাবে কিনবেন না — স্থিতিশীল অংশে প্রতিশ্রুতি, পরিবর্তনশীল অংশে নমনীয়তা। আর প্রতিশ্রুতি দেওয়ার আগে ব্যবহারের ইতিহাস দেখুন — অনুমানে প্রতিশ্রুতি দিলে অপচয় হয়। নির্দিষ্ট দাম এখানে দেওয়া হয়নি — আপনার নিজের বিল ও ব্যবহার থেকে সিদ্ধান্ত নিন।

পরিস্থিতি ৮ — FinOps বৈঠকের কাঠামো। TechNova-এর মাসিক FinOps বৈঠকে তিনটি প্রশ্ন করা হয়: গত মাসে খরচ কোথায় বেড়েছে, কেন বেড়েছে, আর কী করব? প্রতিটি বৃদ্ধির একটি মালিক থাকে, আর পরের মাসে তার ফল পর্যালোচনা করা হয়। এই বৈঠকের ফলে খরচ আর কারো অজান্তে বাড়ে না — প্রতিটি বৃদ্ধি ব্যাখ্যা করা হয়। শিক্ষা: FinOps-এর সবচেয়ে শক্তিশালী টুল একটি ড্যাশবোর্ড নয় — একটি নিয়মিত কথোপকথন, যেখানে অর্থ, প্রকৌশল ও ব্যবসা একসঙ্গে বসে।

পরিস্থিতি ৪ — Unit Economics। TechNova-এর অর্থ দল জিজ্ঞাসা করল: প্রতিটি অর্ডারে আমাদের খরচ কত? দল হিসাব করল: মোট ক্লাউড খরচ ভাগ অর্ডারের সংখ্যা। সংখ্যাটি দেখে সবাই চমকে গেল — একটি ছোট ফিচার, যা কেউ ব্যবহার করে না, মোট খরচের ২০% নিচ্ছে। ফিচারটি বন্ধ করা হলো — গ্রাহকরা টেরই পেলেন না। শিক্ষা: খরচকে ব্যবসায়িক এককের সঙ্গে যুক্ত করুন — প্রতি অর্ডার, প্রতি ব্যবহারকারী, প্রতি অনুরোধ। তখন অপচয় চোখে পড়ে, আর সাশ্রয়ের সিদ্ধান্ত সহজ হয়।

পরিস্থিতি ৫ — সাশ্রয়ের ভুল সিদ্ধান্ত। একটি দল খরচ কমাতে Redis Cluster-এর Replica কমিয়ে দিল — বিল কমল, কিন্তু একটি Node ভেঙে গেলে ক্যাশ হারিয়ে গেল, ডেটাবেসের উপর চাপ বেড়ে গেল, আর Checkout ধীর হয়ে গেল। সাশ্রয়ের চেয়ে ক্ষতি বেশি হলো। শিক্ষা: প্রতিটি সাশ্রয় সিদ্ধান্তের সঙ্গে নির্ভরযোগ্যতার প্রভাব মূল্যায়ন করুন — FinOps-এর সিদ্ধান্ত SRE-এর সঙ্গে বসে নিন, একা নয়।

২৩.৮ নিরাপত্তা ও খরচ বিবেচনা

FinOps-এর নিরাপত্তা দিকটি হলো অ্যাক্সেস নিয়ন্ত্রণ: কে সম্পদ তৈরি করতে পারে তা সীমিত করুন — নইলে কেউ ভুলে বা ইচ্ছাকৃতভাবে বড় সম্পদ তৈরি করে বিল বাড়িয়ে দিতে পারে। Budget Alert-এর পাশাপাশি Policy বসান — যেমন Tag ছাড়া সম্পদ তৈরি করা যাবে না, একটি নির্দিষ্ট আকারের চেয়ে বড় Instance তৈরি করতে অনুমোদন দরকার। Billing ডেটাতে সংবেদনশীল তথ্য থাকতে পারে — যেমন কোন গ্রাহক কত ব্যবহার করছে — তাই Billing ড্যাশবোর্ডের অ্যাক্সেস সীমিত রাখুন। খরচের দিকে, FinOps-এর সবচেয়ে বড় ঝুঁকি হলো অতিরিক্ত সাশ্রয় — নির্ভরযোগ্যতা কমিয়ে খরচ কমানো। একটি

Redundant সার্ভার বন্ধ করে খরচ কমানো যায়, কিন্তু তাতে SLO ভাঙার ঝুঁকি বাড়ে। তাই প্রতিটি সাশ্রয় সিদ্ধান্তের সঙ্গে নির্ভরযোগ্যতার প্রভাবও মূল্যায়ন করুন।

২৩.৯ সাধারণ ভুল

- Cost Reduction ও Cost Optimization-কে এক মনে করা।
- Tagging ছাড়া খরচ বিশ্লেষণ করা।
- নির্দিষ্ট দাম মুখস্থ করা — দাম বদলায়, বিল থেকে যাচাই করুন।
- Rightsizing না করে শুধু বড় সার্ভার কেনা।
- Idle সম্পদ মাসের পর মাস চালিয়ে রাখা।
- Egress খরচ উপেক্ষা করা।
- Log ও ব্যাকআপের Retention নীতি না রাখা।
- নির্ভরযোগ্যতা কমিয়ে খরচ কমানো।

২৩.১০ সমস্যা সমাধান (Troubleshooting)

> **TROUBLESHOOTING:** বিল হঠাৎ বেড়ে গেলে প্রথমে বিলের ভাঙন দেখুন — কোন সেবায় বেড়েছে? তারপর সেই সেবায় কী পরিবর্তন হয়েছে — নতুন ডেপ্লয়, নতুন ফিচার, ট্র্যাফিক বৃদ্ধি, নাকি ভুল কনফিগারেশন? Tag দিয়ে ভাগ করুন — কোন দল, কোন পরিবেশে বেড়েছে? যদি একটি সেবায় বেড়ে থাকে, তবে তার সম্পদ ব্যবহার দেখুন — CPU, Storage, Egress — কোনটি বেড়েছে?

যদি Rightsizing-এর পর Performance খারাপ হয়, তবে Request খুব কম দেওয়া হয়েছে কি না পরীক্ষা করুন — প্রকৃত ব্যবহারের চেয়ে কম Request দিলে Pod Throttle হবে। যদি Tagging-এর পরও খরচ ভাগ করা না যায়, তবে Tagging নীতি ও Audit পরীক্ষা করুন — হয়তো কিছু সম্পদে Tag নেই।

২৩.১১ সারসংক্ষেপ

FinOps হলো মূল্যের জন্য খরচ করা — শুধু খরচ কমানো নয়। Inform, Optimize, Operate — এই চক্রে খরচ দৃশ্যমান করুন, অপচয় দূর করুন, নিয়মিত পর্যালোচনা করুন। Tagging ছাড়া Cost Allocation হয় না; Rightsizing ও Idle Detection সবচেয়ে বেশি সাশ্রয় দেয়; Egress ও Storage খরচ লুকিয়ে বাড়ে। Showback দিয়ে শুরু করুন, Chargeback-এ ধীরে যান। আর কোনো নির্দিষ্ট দাম মুখস্থ করবেন না — নিজের বিল থেকে যাচাই করুন। পরের অধ্যায়ে আমরা সবকিছু একসঙ্গে জুড়ব — Enterprise DevSecOps ও SRE Architecture, এবং Capstone-এর মাধ্যমে পুরো Volume-এর সমাপ্তি।

২৩.১২ পর্যালোচনা প্রশ্ন

1. Cost Reduction ও Cost Optimization-এর পার্থক্য কী?
2. FinOps-এর তিনটি ধাপ কী কী?
3. Tagging কেন Cost Allocation-এর ভিত্তি?
4. Showback ও Chargeback-এর পার্থক্য কী? কোনটি দিয়ে শুরু করবেন?

5. Rightsizing বলতে কী বোঝায়? কীভাবে করবেন?
6. Idle Resource-এর তিনটি উদাহরণ দিন।
7. Data Egress খরচ কী? এটি কীভাবে কমাবেন?
8. Kubernetes-এ খরচ কীভাবে ভাগ করবেন?
9. Budget Alert-এর তিনটি স্তর কী কী হতে পারে?
10. নির্ভরযোগ্যতা কমিয়ে খরচ কমানো কেন ভুল?

২৩.১৩ ব্যবহারিক অনুশীলন

1. TechNova-এর জন্য একটি Tagging নীতি লিখুন: বাধ্যতামূলক Tag, নামকরণের নিয়ম, Audit পদ্ধতি — এবং একটি সম্পদের উদাহরণে Tag প্রয়োগ করুন।
2. একটি কাল্পনিক মাসিক বিলের ভাঙন তৈরি করুন (Compute, Database, Network, Storage, Lab — শতাংশে) এবং প্রতিটি খাতে একটি করে অপ্টিমাইজেশন সুযোগ চিহ্নিত করুন। নির্দিষ্ট দাম উদ্ভাবন করবেন না — শতাংশ ও পদ্ধতিতে লিখুন।
3. Kubernetes-এ একটি Deployment-এর প্রকৃত CPU ও মেমরি ব্যবহার পরিমাপ করে Rightsizing প্রস্তাব লিখুন: বর্তমান Request/Limit, প্রকৃত ব্যবহার, প্রস্তাবিত নতুন মান, এবং ঝুঁকি।

অধ্যায় ২৪ • PART V — Enterprise Reliability & Operations

Chapter 24 — Enterprise DevSecOps and SRE Architecture

২৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পুরো Volume-এর সব ধারণা একসঙ্গে জুড়ে একটি Enterprise Architecture নকশা করতে পারবেন। আপনি Enterprise Security Architecture, Secure CI/CD, SRE Operating Model, Reliability Governance, Incident Management, Disaster Recovery, FinOps ও Platform Engineering-এর মধ্যে সম্পর্ক ব্যাখ্যা করতে পারবেন। আপনি একটি Capstone Architecture তৈরি করতে পারবেন — TechNova-এর MERN অ্যাপ্লিকেশনের জন্য একটি সম্পূর্ণ, উৎপাদন-প্রস্তুত নকশা — এবং Production Readiness Review (PRR) পরিচালনা করতে পারবেন। এই অধ্যায়টি এই Volume-এর সমাপ্তি: এখানে আপনি শুধু নতুন ধারণা শিখবেন না, আগের সব অধ্যায়ের ধারণা একসঙ্গে প্রয়োগ করবেন।

- Enterprise Security Architecture নকশা করা।
- Secure CI/CD ও SRE Operating Model ব্যাখ্যা করা।
- Reliability Governance ও Platform Engineering একীভূত করা।
- Capstone Architecture তৈরি করা।
- Production Readiness Review পরিচালনা করা।

২৪.২ প্রাক-প্রয়োজনীয় পুনরালোচনা

এই অধ্যায়ে প্রবেশের আগে পুরো Volume-এর মূল ধারণাগুলো মনে রাখুন। DevSecOps: নিরাপত্তা শুরু থেকে — Shift Left, Secret ব্যবস্থাপনা, Dependency স্ক্যান, Secure CI/CD। SRE: SLI, SLO, Error Budget, Toil, Automation, Production Ownership। Monitoring: Golden Signals, Prometheus, Grafana, Alertmanager, On-call। Incident: Lifecycle, Severity, Five Whys, Blameless Postmortem। Performance: Load Testing, Bottleneck, Caching, Resilience Pattern। DR: RPO, RTO, Backup, Failover। Chaos: Hypothesis-চালিত পরীক্ষা, Lab-এ। FinOps: Cost Optimization, Tagging, Rightsizing। এই অধ্যায়ে আমরা এই সবগুলোকে একটি স্থাপত্যে জুড়ব — যাতে প্রতিটি অংশ অন্যটিকে শক্তিশালী করে, দুর্বল না করে। TechNova-এর MERN অ্যাপ্লিকেশন আমাদের Capstone উদাহরণ — React ফ্রন্টএন্ড, Express API, MongoDB, technova.example, 203.0.113.x, Ubuntu 24.04 LTS।

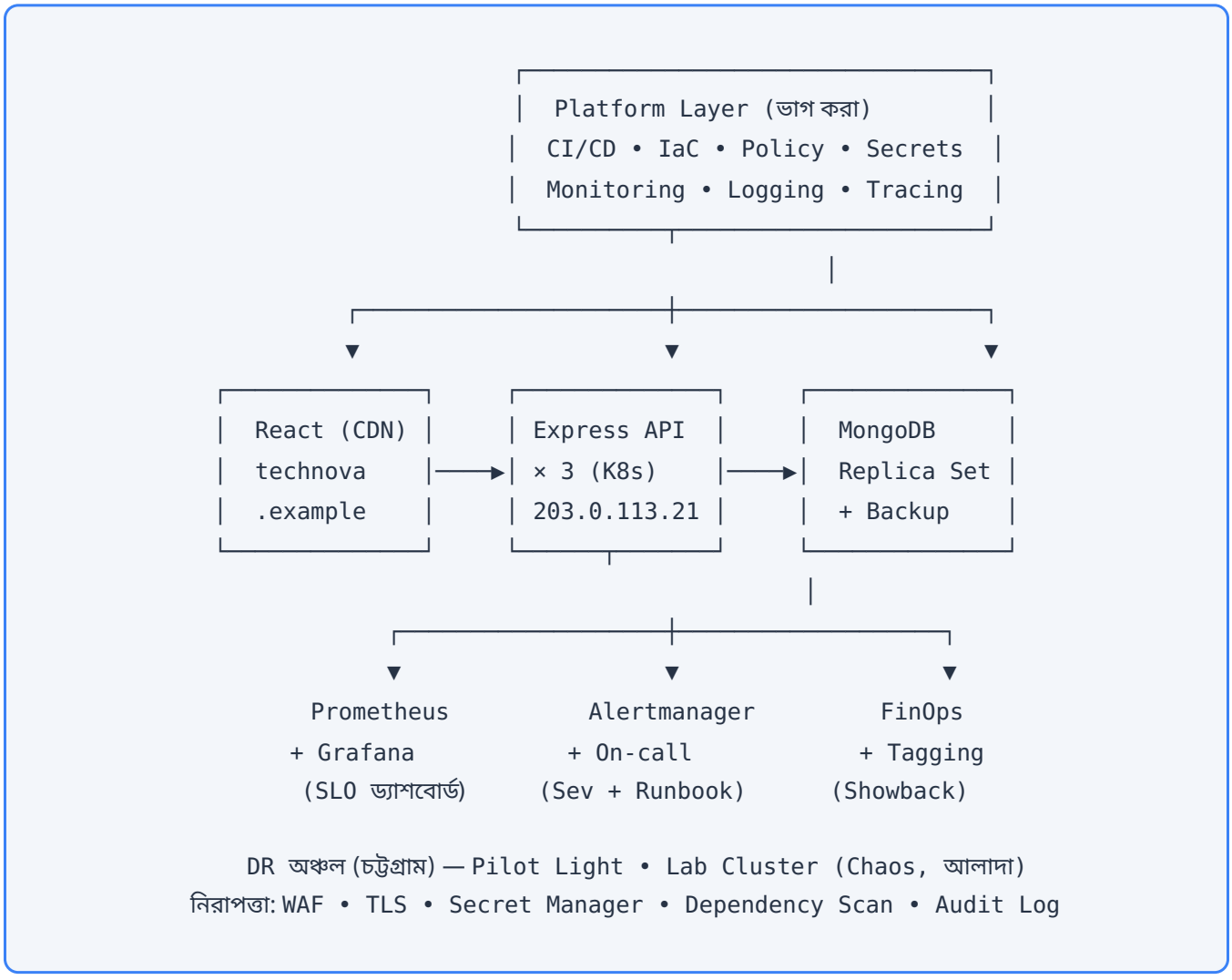
২৪.৩ প্রথম নীতি থেকে ব্যাখ্যা: এন্টারপ্রাইজ মানে একসঙ্গে কাজ করা

একটি ছোট দলে একজন প্রকৌশলী সবকিছু করতে পারেন — কোড লেখা, ডেপ্লয় করা, মনিটরিং দেখা, ইনসিডেন্ট সামলানো। কিন্তু একটি এন্টারপ্রাইজে অনেক দল, অনেক সেবা, অনেক নিয়ম — তখন শৃঙ্খলা ছাড়া কিছুই চলে না। Enterprise Architecture-এর মূল সমস্যাটি হলো: অনেক দল কীভাবে একসঙ্গে কাজ করবে যাতে প্রতিটি দল দ্রুত চলতে পারে, কিন্তু পুরো সিস্টেম নিরাপদ ও নির্ভরযোগ্য থাকে? উত্তরটি হলো Platform Engineering: একটি ভাগ করা প্ল্যাটফর্ম তৈরি করা — CI/CD পাইপলাইন, মনিটরিং, নিরাপত্তা স্ক্যান, ডেপ্লয়মেন্ট টেমপ্লেট — যা প্রতিটি দল ব্যবহার করে, কিন্তু প্রতিটি দলকে আলাদা করে বানাতে হয় না। প্ল্যাটফর্ম দল প্ল্যাটফর্ম বানায়, পণ্য দল পণ্য বানায়। এই ভাগ করে নেওয়ার ফলে প্রতিটি দল দ্রুত চলে, আর নিরাপত্তা ও নির্ভরযোগ্যতার নিয়ম সব জায়গায় একইভাবে প্রয়োগ হয়।

SRE Operating Model বলতে বোঝায় SRE দল কীভাবে সংগঠিত হবে: Centralized SRE দল (একটি দল সব সেবা দেখে), Embedded SRE (প্রতিটি পণ্য দলে একজন SRE), নাকি Hybrid (কেন্দ্রীয় দল প্ল্যাটফর্ম দেয়, Embedded SRE পণ্য দলে কাজ করে)? কোনো একক সঠিক উত্তর নেই — সংস্থার আকার, সেবার সংখ্যা, ও দলের পরিপক্কতা দেখে ঠিক করতে হয়। ছোট সংস্থায় Hybrid ভালো কাজ করে: একটি ছোট কেন্দ্রীয় দল প্ল্যাটফর্ম ও SLO কাঠামো দেয়, আর প্রতিটি পণ্য দল নিজের নির্ভরযোগ্যতার মালিকানা নেয়। Reliability Governance বলতে বোঝায় নির্ভরযোগ্যতার নিয়ম ও পর্যালোচনা: SLO পর্যালোচনা, Error Budget নীতি, PRR, Incident পর্যালোচনা — এই সবকিছু নিয়মিত, স্বচ্ছ, ও নথিভুক্ত হতে হবে। Governance মানে আমলাতন্ত্র নয় — Governance মানে সবাই একই নিয়মে খেলছে তা নিশ্চিত করা।

> **CONCEPT:** Platform Engineering বলতে বোঝায় পণ্য দলগুলোর জন্য একটি ভাগ করা, স্ব-সেবা প্ল্যাটফর্ম তৈরি করা — CI/CD, মনিটরিং, নিরাপত্তা, ডেপ্লয়মেন্ট — যাতে প্রতিটি দল দ্রুত চলতে পারে, কিন্তু নিরাপত্তা ও নির্ভরযোগ্যতার নিয়ম সব জায়গায় একই থাকে। প্ল্যাটফর্ম দল প্ল্যাটফর্ম বানায়; পণ্য দল পণ্য বানায়।

২৪.৪ স্থাপত্য: TechNova Enterprise Capstone Architecture



এই Capstone স্থাপত্যে প্রতিটি স্তর আগের অধ্যায়গুলোর একটি ধারণা বাস্তবায়ন করে। Platform Layer হলো DevSecOps ও Platform Engineering: CI/CD পাইপলাইনে নিরাপত্তা স্ক্যান, IaC দিয়ে পরিবেশ, Policy দিয়ে নিয়ম, Secret Manager দিয়ে গোপন তথ্য। Application Layer হলো MERN সেবা — Redundancy, Health Check, Resource Limit-সহ। Data Layer হলো MongoDB Replica Set — Backup, Point-in-Time Recovery, ও DR

Replica-সহ। Observability Layer হলো Prometheus, Grafana, Alertmanager — SLO ড্যাশবোর্ড, Paging ও Ticket Alert, Runbook-সহ। Governance Layer হলো SLO পর্যালোচনা, PRR, Incident Postmortem, FinOps পর্যালোচনা — নিয়মিত বৈঠক ও নথিভুক্ত সিদ্ধান্ত। আর DR ও Lab হলো নিরাপত্তা জাল: DR অঞ্চলে Pilot Light, Lab Cluster-এ Chaos পরীক্ষা — উৎপাদন থেকে সম্পূর্ণ আলাদা।

২৪.৫ অভ্যন্তরীণ কর্মনীতি: Secure CI/CD, Governance ও Production Readiness Review

Secure CI/CD পাইপলাইনের ধাপগুলো: কোড Commit হলে প্রথমে Static Analysis ও Secret Scan — কোডে গোপন তথ্য আছে কি না, পরিচিত দুর্বলতা আছে কি না। তারপর Dependency Scan — ব্যবহৃত লাইব্রেরিতে পরিচিত CVE আছে কি না। তারপর Unit ও Integration Test — কার্যকারিতা ও Performance Budget পরীক্ষা। তারপর Container Image Scan — Image-এ দুর্বলতা আছে কি না, এবং Image Sign করা — যাতে উৎপাদনে শুধু স্বাক্ষরিত Image চলে। তারপর Staging-এ Deploy ও Smoke Test — তারপর Canary Deployment — ধীরে ধীরে ট্র্যাফিক বাড়ানো, Error Budget ও SLO পর্যবেক্ষণ করা। কোনো ধাপে ব্যর্থ হলে পাইপলাইন থেমে যাবে — উৎপাদনে যাবে না। এই পাইপলাইনটি Platform দল বানায়, পণ্য দল ব্যবহার করে — ফলে নিরাপত্তা নিয়ম সব দলে একইভাবে প্রয়োগ হয়।

Production Readiness Review (PRR) বলতে বোঝায় একটি সেবা উৎপাদনে যাওয়ার আগে একটি চেকলিস্ট পর্যালোচনা — নতুন সেবার জন্য, বা বড় পরিবর্তনের জন্য। PRR-এর প্রশ্নগুলো আগের অধ্যায়গুলো থেকে আসে: SLI ও SLO ঠিক করা হয়েছে কি? Monitoring ও Alert বসানো হয়েছে কি? Runbook লেখা হয়েছে কি? On-call ঠিক করা হয়েছে কি? Backup ও Restore পরীক্ষা করা হয়েছে কি? DR পরিকল্পনা আছে কি? Load Test করা হয়েছে কি? নিরাপত্তা স্ক্যান পাস করেছে কি? খরচের অনুমান করা হয়েছে কি? Rollback পরিকল্পনা আছে কি? প্রতিটি প্রশ্নের উত্তর হ্যাঁ না হলে সেবা উৎপাদনে যাবে না — অথবা ঝুঁকি স্বীকার করে, নথিভুক্ত করে, সময়সীমাসহ যাবে। PRR-এর উদ্দেশ্য বাধা দেওয়া নয় — ঝুঁকি দৃশ্যমান করা।

> **REAL WORLD:** একটি দল PRR ছাড়া একটি নতুন পেমেন্ট ফিচার উৎপাদনে পাঠাল — Monitoring ছিল না, Runbook ছিল না, Rollback পরিকল্পনা ছিল না। ফিচারটি ভেঙে গেলে কেউ বুঝতে পারেনি কী হয়েছে, কেউ জানত না কী করতে হবে, আর Rollback করতে দুই ঘণ্টা লেগেছে। পরের বার থেকে দলটি PRR বাধ্যতামূলক করল — একটি এক পৃষ্ঠার চেকলিস্ট, ৩০ মিনিটের পর্যালোচনা — আর একই ভুল আর হয়নি।

২৪.৬ বাস্তব কমান্ড ও কনফিগারেশন

Secure CI/CD পাইপলাইনের রূপরেখা (GitHub Actions ধারণা):

```
jobs:
  secure-build:
    runs-on: ubuntu-24.04
    steps:
    - uses: actions/checkout@v4
    - name: Secret scan
      run: gitleaks detect --no-git -v
    - name: Dependency scan
      run: npm audit --audit-level=high
    - name: Unit tests + coverage
      run: npm test -- --coverage
    - name: Build image
      run: docker build -t registry.technova.example/api:${{ github.sha }} .
```

```
    - name: Image scan
      run: trivy image registry.technova.example/api:${{ github.sha }}
    - name: Deploy to staging
      run: kubectl -n technova-staging set image deployment/technova-api
api=registry.technova.example/api:${{ github.sha }}
    - name: Smoke test
      run: curl -fsS https://staging.technova.example/health
```

PRR চেকলিস্ট — TechNova (সংক্ষিপ্ত):

```
Production Readiness Review — সেবা: _____ তারিখ: _____
[ ] SLI ও SLO নথিভুক্ত (Availability + Latency)
[ ] Dashboard ও Alert বসানো (Paging + Ticket, Runbook-সহ)
[ ] On-call Rotation ও Escalation ঠিক
[ ] Backup ও Restore পরীক্ষা করা হয়েছে
[ ] DR পরিকল্পনা ও RPO/RTO নথিভুক্ত
[ ] Load Test করা হয়েছে (ফল সংযুক্ত)
[ ] নিরাপত্তা স্ক্যান পাস (Secret, Dependency, Image)
[ ] খরচের অনুমান ও Tagging করা হয়েছে
[ ] Rollback পরিকল্পনা লেখা হয়েছে
[ ] Postmortem ও Incident প্রক্রিয়া জানা আছে
সিদ্ধান্ত: [ ] অনুমোদিত  [ ] শর্তসাপেক্ষে  [ ] স্থগিত — ঝুঁকি: _____
```

দৈনন্দিন Governance পরীক্ষা:

```
# SLO অবস্থা — Error Budget কত বাকি?
# Prometheus কুয়েরি দিয়ে যাচাই করুন (অধ্যায় ১৭ দেখুন)
kubectl -n technova get pods,svc,ingress
kubectl -n technova top pods
# Secret-এ Placeholder আছে কি না যাচাই — আসল Secret কখনো Git-এ নয়
grep -r "CHANGE-ME" --include="*.yaml" --include="*.env" . | head
```

২৪.৭ উৎপাদন পরিস্থিতি

পরিস্থিতি ১ — নতুন দল, নতুন সেবা, কিন্তু প্ল্যাটফর্ম প্রস্তুত। TechNova-তে একটি নতুন দল একটি নতুন সেবা বানাতে চায়। প্ল্যাটফর্ম দল তাদের CI/CD টেমপ্লেট, মনিটরিং ড্যাশবোর্ড টেমপ্লেট, ও PRR চেকলিস্ট দেয় — দলটি প্রথম দিন থেকেই নিরাপত্তা ও নির্ভরযোগ্যতার নিয়ম মেনে চলে, কিন্তু নিজের গতিতে কাজ করে। **পরিস্থিতি ২ — Error Budget শেষ, কিন্তু ব্যবসা চাপ দিচ্ছে।** Governance বৈঠকে সিদ্ধান্ত হয়: বাজেট শেষ হওয়ায় দুই সপ্তাহ Feature Freeze — নির্ভরযোগ্যতা ঠিক করা হবে। ব্যবসা প্রথমে আপত্তি করে, কিন্তু SLO ডেটা দেখে মেনে নেয় — কারণ সংখ্যা নিরপেক্ষ। **পরিস্থিতি ৩ — PRR-এ ধরা পড়ল Backup পরীক্ষা হয়নি।** একটি সেবা উৎপাদনে যাওয়ার আগে PRR-এ দেখা গেল Restore পরীক্ষা করা হয়নি। সেবাটি এক সপ্তাহ পিছিয়ে গেল — কিন্তু তার বদলে একটি বড় ঝুঁকি এড়ানো গেল।

পরিস্থিতি ৬ — Platform-এর বিবর্তন। TechNova-এর প্ল্যাটফর্ম প্রথমে শুধু CI/CD ছিল — তারপর মনিটরিং যোগ হলো, তারপর নিরাপত্তা স্ক্যান, তারপর Cost Dashboard। প্রতিটি যোগের আগে প্ল্যাটফর্ম দল পণ্য দলকে জিজ্ঞাসা করল: আপনাদের সবচেয়ে বড় কষ্ট কী? উত্তর থেকে পরের বৈশিষ্ট্য এলো। শিক্ষা: প্ল্যাটফর্ম ব্যবহারকারীর কষ্ট থেকে গড়ে ওঠে — প্রযুক্তির ফ্যাশন থেকে নয়। আর প্ল্যাটফর্মের সাফল্য মাপা হয় ব্যবহার দিয়ে — কত দল ব্যবহার করে, কত সময় বাঁচে — বৈশিষ্ট্যের সংখ্যা দিয়ে নয়।

পরিস্থিতি ৭ — SRE Operating Model-এর পরিবর্তন। TechNova প্রথমে Centralized SRE দল দিয়ে শুরু করল — একটি দল সব সেবা দেখে। সেবা বাড়ার সঙ্গে সঙ্গে দলটি বোঝা হয়ে গেল — তারা সব সেবার গভীরতা জানত না। তখন

Hybrid Model-এ যাওয়া হলো: কেন্দ্রীয় দল প্ল্যাটফর্ম ও SLO কাঠামো দেয়, প্রতিটি পণ্য দলে একজন Embedded SRE থাকেন। এই পরিবর্তনে সময় লেগেছে, কিন্তু এখন প্রতিটি সেবার গভীর জ্ঞান ও কেন্দ্রীয় শৃঙ্খলা — দুটোই আছে। শিক্ষা: Operating Model স্থির নয় — সংস্থা বাড়লে তা বদলাতে হয়। যা ৫টি সেবায় কাজ করে, তা ৫০টি সেবায় কাজ নাও করতে পারে।

পরিস্থিতি ৮ — Capstone থেকে বাস্তবায়ন। TechNova-এর Capstone স্থাপত্য কাগজে সুন্দর ছিল — কিন্তু বাস্তবায়নের সময় দেখা গেল কিছু অংশে দক্ষতা নেই, কিছু অংশে বাজেট নেই। দল স্থাপত্যকে পর্যায়ে ভাগ করল: প্রথম পর্যায়ে SLO, Monitoring ও PRR — কারণ এগুলো সবচেয়ে বেশি মূল্য দেয়, খরচ কম। দ্বিতীয় পর্যায়ে DR ও Chaos Lab। তৃতীয় পর্যায়ে Cross-cloud। প্রতিটি পর্যায়ের শেষে পর্যালোচনা — কী শিখলাম, পরের পর্যায়ে কী বদলাব? শিক্ষা: একটি বড় স্থাপত্য একদিনে বাস্তবায়ন হয় না — পর্যায়ে, মূল্যের ক্রমে, শেখার সঙ্গে। কাগজের স্থাপত্যের মূল্য তখনই, যখন তা বাস্তবায়নের পথ দেখায়।

পরিস্থিতি ৪ — Zero Trust বাস্তবায়ন। TechNova-এর পুরনো নকশায় ভিতরের নেটওয়ার্ককে বিশ্বাস করা হতো — একবার ভিতরে ঢুকলে সব সেবা খোলা। একটি নিরাপত্তা পর্যালোচনায় দেখা গেল একটি আপস হওয়া সেবা থেকে পুরো ডেটাবেসে যাওয়া যায়। নতুন নকশায় প্রতিটি সেবা প্রতিটি অনুরোধ যাচাই করে: mTLS, Service Account, ন্যূনতম অনুমতি। বাস্তবায়ন ধীরে ধীরে করা হলো — প্রথমে সবচেয়ে সংবেদনশীল সেবা, তারপর বাকিগুলো। শিক্ষা: Zero Trust একটি যাত্রা — একদিনে হয় না, কিন্তু প্রতিটি ধাপ নিরাপত্তা বাড়ায়। আর প্রতিটি ধাপে পরীক্ষা করুন — নিরাপত্তা বাড়াতে গিয়ে যাতে সেবা না ভাঙে।

পরিস্থিতি ৫ — Governance-এর মাসিক ছন্দ। TechNova-তে প্রতি মাসের প্রথম সপ্তাহে চারটি বৈঠক হয়: SLO পর্যালোচনা (সোমবার), FinOps পর্যালোচনা (মঙ্গলবার), Incident পর্যালোচনা (বুধবার), এবং PRR (বৃহস্পতিবার)। প্রতিটি বৈঠক ৩০ মিনিট, প্রতিটির সিদ্ধান্ত লিখিত। এই ছন্দের ফলে নির্ভরযোগ্যতা, খরচ ও নিরাপত্তা — তিনটি নিয়মিত চোখে পড়ে, কোনোটিই ভুলে যাওয়া হয় না। শিক্ষা: Governance মানে বড় নথি নয় — নিয়মিত, ছোট, লিখিত পর্যালোচনা। এই ছন্দই একটি এন্টারপ্রাইজকে একটি ছোট দল থেকে আলাদা করে।

২৪.৮ নিরাপত্তা ও খরচ বিবেচনা

Enterprise স্তরে নিরাপত্তা মানে স্তরে স্তরে প্রতিরক্ষা (Defense in Depth): WAF, TLS, Authentication, Authorization, Secret Manager, Image Scan, Audit Log, Network Policy — কোনো একটিই যথেষ্ট নয়, সবগুলো একসঙ্গে কাজ করে। Zero Trust নীতি মানুন: ভিতরের নেটওয়ার্ককেও বিশ্বাস করবেন না — প্রতিটি সেবা প্রতিটি অনুরোধ যাচাই করবে। সব অ্যাক্সেস লগ করুন, নিয়মিত Audit করুন, এবং Least Privilege মানুন — প্রতিটি দল ও প্রতিটি সেবার শুধু যতটুকু দরকার ততটুকু অ্যাক্সেস। খরচের দিকে, Platform Engineering-এর বিনিয়োগ প্রথমে বেশি মনে হতে পারে — প্ল্যাটফর্ম দল, টুল, প্রশিক্ষণ — কিন্তু প্রতিটি দল আলাদা করে একই জিনিস বানানোর চেয়ে তা সস্তা। FinOps Governance-এর মাধ্যমে প্রতিটি দলের খরচ দৃশ্যমান করুন, এবং প্রতি মাসে পর্যালোচনা করুন। নিরাপত্তা ও নির্ভরযোগ্যতায় বিনিয়োগের যুক্তি দিন ব্যবসার ভাষায়: একটি বিভ্রাটের খরচ, একটি নিরাপত্তা লঙ্ঘনের খরচ — সেই সংখ্যার সঙ্গে বিনিয়োগের তুলনা করুন।

২৪.৯ সাধারণ ভুল

- প্রতিটি দল আলাদা করে CI/CD ও মনিটরিং বানানো — Platform-এর সুবিধা হারানো।
- PRR-কে আমলাতন্ত্র মনে করে এড়িয়ে যাওয়া।
- নিরাপত্তাকে শেষ ধাপ মনে করা — Shift Left না করা।
- Error Budget নীতি লিখে তা প্রয়োগ না করা।
- Governance-কে শুধু নথি মনে করা — পর্যালোচনা না করা।

- Platform দলকে পণ্য দল থেকে বিচ্ছিন্ন করা — প্ল্যাটফর্ম ব্যবহারকারীর কথা না শোনা।
- Capstone স্থাপত্য আঁকা কিন্তু তা বাস্তবায়ন না করা।
- একটি বিভ্রাটের পর Governance শিথিল করা।

২৪.১০ সমস্যা সমাধান (Troubleshooting)

> **TROUBLESHOOTING:** Enterprise স্থাপত্যে সমস্যা হলে প্রথমে জিজ্ঞাসা করুন: সমস্যাটি কোন স্তরে — Platform, Application, Data, নাকি Governance? Platform সমস্যা (যেমন CI/CD ভাঙা) সব দলকে প্রভাবিত করে — তাই তা আগে ঠিক করুন। Application সমস্যা একটি সেবায় সীমিত — সেই দলের Runbook অনুসরণ করুন। Governance সমস্যা (যেমন PRR ছাড়া ডেপ্লয়) প্রক্রিয়ার সমস্যা — তা Postmortem-এ তুলুন।

যদি প্ল্যাটফর্ম দল ও পণ্য দলের মধ্যে দ্বন্দ্ব হয়, তবে প্ল্যাটফর্মকে পণ্য হিসেবে দেখুন — পণ্য দল তার গ্রাহক। নিয়মিত ফিডব্যাক নিন, প্ল্যাটফর্মের ব্যবহার পরিমাপ করুন, এবং অপ্রয়োজনীয় বাধা দূর করুন। যদি PRR বারবার বাধা হয়ে দাঁড়ায়, তবে চেকলিস্টটি পর্যালোচনা করুন — হয়তো কিছু প্রশ্ন অপ্রাসঙ্গিক, অথবা স্বয়ংক্রিয় করা যায়।

২৪.১১ সারসংক্ষেপ

Enterprise DevSecOps ও SRE Architecture হলো সবকিছু একসঙ্গে জোড়া: নিরাপত্তা শুরু থেকে, SRE পরিমাপ দিয়ে, Platform ভাগ করা ভিত্তি হিসেবে, Governance নিয়ম হিসেবে, আর PRR দরজা হিসেবে। TechNova-এর Capstone স্থাপত্যে প্রতিটি স্তর আগের অধ্যায়ের একটি শিক্ষা বাস্তবায়ন করে — Redundancy, SLO, Monitoring, Incident, DR, Chaos, FinOps — সব একসঙ্গে। এই Volume-এর মূল শিক্ষাটি মনে রাখুন: নির্ভরযোগ্যতা কোনো একটি টুল বা দলের কাজ নয় — এটি একটি সংস্কৃতি, একটি শৃঙ্খলা, এবং একটি স্থাপত্য — যা প্রতিদিন, প্রতিটি সিদ্ধান্তে, প্রতিটি ডেপ্লয়ে প্রয়োগ করতে হয়। আপনি এখন সেই শৃঙ্খলার ভিত্তি জানেন — এবার তা অনুশীলন করে নিজের করে নিন।

২৪.১২ পর্যালোচনা প্রশ্ন

1. Platform Engineering কী? এটি পণ্য দলকে কীভাবে সাহায্য করে?
2. SRE Operating Model-এর তিনটি ধরন কী কী? কখন কোনটি?
3. Reliability Governance বলতে কী বোঝায়?
4. Secure CI/CD পাইপলাইনের ধাপগুলো কী কী?
5. Production Readiness Review (PRR) কী? এর পাঁচটি প্রশ্ন লিখুন।
6. Defense in Depth বলতে কী বোঝায়? তিনটি স্তরের উদাহরণ দিন।
7. Capstone স্থাপত্যে প্রতিটি স্তর কোন অধ্যায়ের শিক্ষা বাস্তবায়ন করে?
8. Error Budget নীতি Governance-এর সঙ্গে কীভাবে যুক্ত?
9. Platform দলকে পণ্য হিসেবে দেখা কেন গুরুত্বপূর্ণ?
10. এই Volume-এর সবচেয়ে গুরুত্বপূর্ণ তিনটি শিক্ষা কী? আপনার নিজের ভাষায় লিখুন।

২৪.১৩ ব্যবহারিক অনুশীলন

1. TechNova-এর জন্য একটি সম্পূর্ণ Capstone Architecture চিত্র আঁকুন: Platform, Application, Data, Observability, Governance, DR ও Lab — প্রতিটি স্তরে ব্যবহৃত টুল ও নীতি লিখুন।
2. একটি নতুন সেবার জন্য একটি সম্পূর্ণ PRR পরিচালনা করুন (কাল্পনিক সেবা): চেকলিস্টের প্রতিটি প্রশ্নের উত্তর দিন, ঝুঁকি চিহ্নিত করুন, এবং সিদ্ধান্ত লিখুন।
3. একটি Secure CI/CD পাইপলাইনের নকশা লিখুন: প্রতিটি ধাপ, প্রতিটি ধাপে কী পরীক্ষা হবে, ব্যর্থ হলে কী হবে — এবং একটি Canary Deployment পরিকল্পনা যোগ করুন।